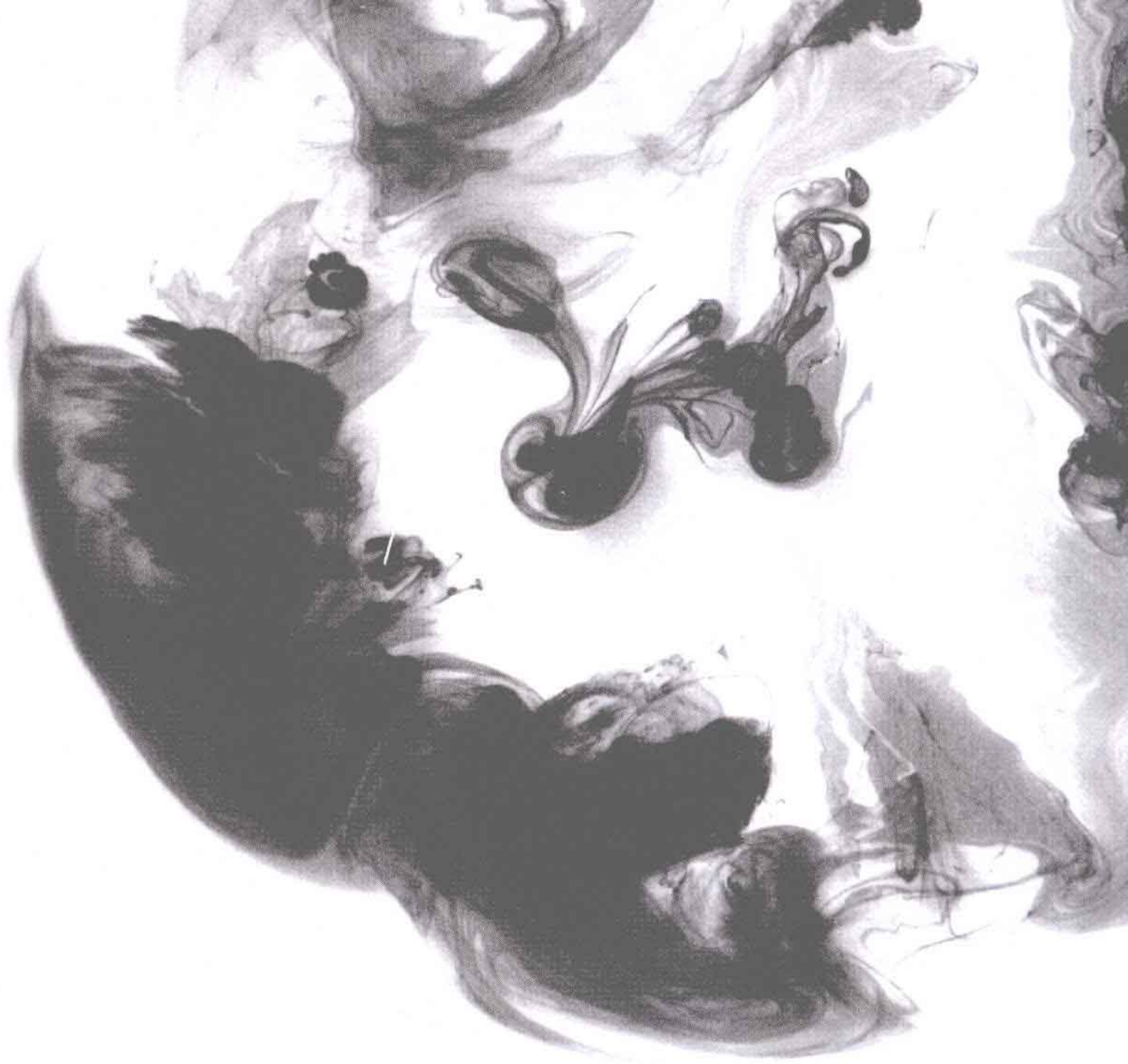
Broadview®
www.broadview.com.cn

◎理论结合实践　深入浅出　通俗易懂　实用性强

◎大量经典案例　源于实际的工程经验和方法

◎常见问题分析　提供各种常见故障解决方案



Linux运维之道

• 丁明一　编著 •

電子工業出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
http://www.phei.com.cn

目　录

第 1 篇　基础知识

第 1 章　部署操作系统 2

1.1　光盘安装 Linux 操作系统 2

1.1.1　操作系统版本的选择 2

1.1.2　光盘安装 Linux 系统案例 2

1.2　无人职守自动安装 Linux 操作系统 15

1.2.1　大规模部署案例 15

1.2.2　PXE 简介 16

1.2.3　Kickstart 技术 17

1.2.4　配置安装服务器 18

1.2.5　自动化安装案例 19

1.3　常见问题分析 27

第 2 章　命令工具 30

2.1　基本命令 31

2.1.1　目录及文件的基本操作 31

2.1.2　查看文件内容 35

2.1.3　链接文件 37

2.1.4　压缩及解压 38

2.1.5　命令使用技巧 39

2.1.6　帮助 40

2.2　Vim 文档编辑 42

2.2.1　Vim 工作模式 42

2.2.2　Vim 光标操作 43

2.2.3　Vim 编辑文档 43

2.2.4　Vim 查找与替换 44

2.2.5　Vim 保存与退出 45

2.2.6　Vim 小技巧 45

2.3　账户与安全 46

2.3.1　账户及组的概念 46

2.3.2　创建账户及组 46

2.3.3　修改账户及组 48

2.3.4　删除账户及组 49

2.3.5　账户与组文件解析 49

2.3.6　文件及目录权限 51

2.3.7　账户管理案例 53

2.3.8　ACL 访问控制权限 54

2.4　存储管理 55

2.4.1　磁盘分区 56

2.4.2　格式化与挂载文件系统 60

2.4.3　LVM 逻辑卷概述 62

2.4.4　创建 LVM 分区实例 64

2.4.5　修改 LVM 分区容量 68

2.4.6　删除 LVM 分区 69

2.4.7　RAID 磁盘阵列概述 69

2.4.8　RAID 级别 70

2.4.9　创建与管理软 RAID 实例 74

2.4.10　RAID 性能测试 77

2.4.11 RAID 故障模拟 78
2.5 软件管理 79
2.5.1 Linux 常用软件包类型 79
2.5.2 RPM 软件包管理 79
2.5.3 使用 YUM 安装软件包 81
2.5.4 YUM 使用技巧 83
2.5.5 源码编译安装软件 84
2.5.6 常见问题分析 85
2.5.7 服务管理 86
2.6 计划任务 88
2.6.1 at 一次性计划任务 88
2.6.2 cron 周期性计划任务 89
2.6.3 计划任务权限 90
2.7 性能监控 90
2.7.1 监控 CPU 使用情况——uptime 命令 90
2.7.2 监控内存及交换分区使用情况——free 命令 91
2.7.3 监控磁盘使用情况——df 命令 91
2.7.4 监控网络使用情况——ifconfig 和 netstat 命令 92
2.7.5 监控进程使用情况——ps 和 top 命令 94
2.8 网络配置 95
2.8.1 命令行设置网络参数 95
2.8.2 文件修改网络参数 97
2.8.3 网络故障排错 99
2.9 内核模块 101
2.9.1 内核模块存放位置 101
2.9.2 查看已加载内核模块 102
2.9.3 加载与卸载内核模块 102
2.9.4 修改内核参数 103

第 3 章 自动化运维 104
3.1 Shell 简介 104
3.2 Bash 功能介绍 105
3.2.1 命令历史 105
3.2.2 命令别名 105
3.2.3 管道与重定向 106
3.2.4 快捷键 107
3.3 Bash 使用技巧 107
3.3.1 重定向技巧 107
3.3.2 命令序列使用技巧 108
3.3.3 作业控制技巧 108
3.3.4 花括号{}的使用技巧 109
3.4 变量 109
3.4.1 自定义变量 109
3.4.2 变量的使用范围 110
3.4.3 环境变量 111
3.4.4 位置变量 112
3.4.5 变量的展开替换 112
3.4.6 数组 113
3.4.7 算术运算与测试 114
3.5 Shell 引号 116
3.5.1 反斜线 116
3.5.2 单引号 117
3.5.3 双引号 117
3.5.4 反引号 117
3.6 正则表达式 118
3.6.1 基本正则表达式（Regular Expression） 119
3.6.2 扩展正则表达式（Extended Regular Expression） 122
3.6.3 POSIX 规范 122

3.6.4 GNU 规范 123
3.7 Sed 124
3.7.1 Sed 简介 124
3.7.2 Sed 基本语法格式 124
3.7.3 Sed 入门范例 125
3.7.4 Sed 指令与脚本 127
3.7.5 Sed 高级应用 132
3.8 Awk 136
3.8.1 Awk 简介 136
3.8.2 Awk 工作流程 136
3.8.3 Awk 基本语法格式 137
3.8.4 Awk 操作指令 138
3.8.5 Awk 高级应用 142
3.9 Shell 脚本 145
3.9.1 脚本格式范例 146
3.9.2 运行脚本的方式 146
3.9.3 Shell 脚本简单案例 147
3.9.4 判断语句应用 150
3.9.5 循环语句应用 154
3.9.6 控制语句应用 158
3.9.7 Shell 函数应用 159
3.9.8 综合案例 162
3.9.9 图形脚本 166

第 2 篇　网络服务

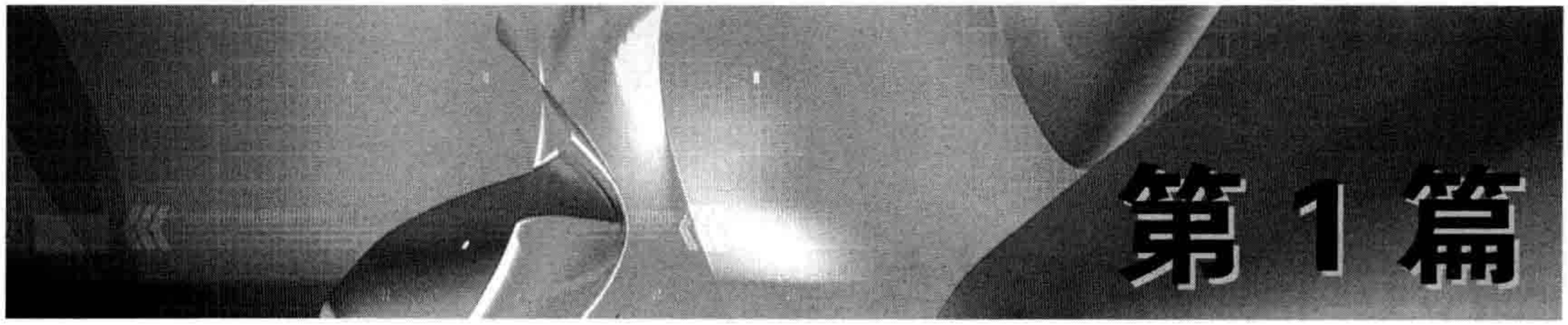
第 4 章　搭建网络服务 170
4.1 NFS 文件共享 170
4.1.1 NFS 服务器配置 171
4.1.2 客户端访问 NFS 共享 173
4.1.3 NFS 高级设置 174
4.1.4 常见问题分析 177
4.2 Samba 文件共享 178
4.2.1 快速配置 Samba 服务器 178
4.2.2 访问 Samba 共享 180
4.2.3 配置文件详解 182
4.2.4 Samba 应用案例 183
4.2.5 常见问题分析 186
4.3 vsftpd 文件共享 187
4.3.1 FTP 工作模式 188
4.3.2 安装与管理 vsftpd 189
4.3.3 配置文件解析 189
4.3.4 账户权限 190
4.3.5 vsftpd 应用案例 191
4.3.6 常见问题分析 195
4.4 ProFTPD 文件共享 196
4.4.1 安装 ProFTPD 软件 196
4.4.2 配置文件解析 197
4.4.3 ProFTPD 权限设置 198
4.4.4 虚拟用户应用案例 198
4.4.5 常见问题分析 202
4.5 Subversion 版本控制 202
4.5.1 Subversion 简介 202
4.5.2 Subversion 服务器对比 204
4.5.3 安装 Subversion 软件 205
4.5.4 svnserve 服务器搭建 205
4.5.5 svnserve+SSH 服务器搭建 210
4.5.6 Apache+SVN 服务器搭建 211
4.5.7 常见问题 213
4.6 网络存储服务器 214
4.6.1 iSCSI 网络存储 215
4.6.2 Rsync 文件同步 219
4.6.3 Rsync+Inotify 实现文件自动同步 225
4.7 DHCP 服务器 231

4.7.1 安装软件 231
4.7.2 配置文件解析 232
4.7.3 DHCP 应用案例 233
4.7.4 常见问题分析 234
4.8 DNS 域名服务器 235
4.8.1 DNS 简介 235
4.8.2 安装 DNS 软件 237
4.8.3 配置文件解析 238
4.8.4 部署主域名服务器 241
4.8.5 部署从域名服务器 245
4.8.6 DNS 视图应用案例 247
4.8.7 常见问题分析 249
4.9 Apache 网站服务器 250
4.9.1 Apache 简介 250
4.9.2 安装 Apache 软件 250
4.9.3 配置文件解析 252
4.9.4 虚拟主机应用案例 257
4.9.5 网站安全应用案例 259
4.9.6 常见问题分析 260
4.10 Nginx 网站服务器 261
4.10.1 Nginx 简介 261
4.10.2 安装 Nginx 软件 261
4.10.3 配置文件解析 264
4.10.4 虚拟主机应用案例 267
4.10.5 SSL 网站应用案例 269
4.10.6 HTTP 响应状态码 270
4.11 数据库基础 271
4.11.1 MySQL 数据库简介 271
4.11.2 安装 MySQL 软件 272
4.11.3 MySQL 管理工具 273
4.11.4 数据库定义语言 278
4.11.5 数据库操作语言 283
4.11.6 数据库查询语言 285
4.11.7 MySQL 与安全 286
4.11.8 MySQL 数据库备份与还原 289
4.12 动态网站架构案例 290
4.12.1 论坛系统应用案例 291
4.12.2 博客系统应用案例 297
第 5 章 系统监控 301
5.1 Cacti 监控系统 301
5.1.1 简介 301
5.1.2 Cacti 监控应用案例 302
5.2 Nagios 监控系统 310
5.2.1 简介 310
5.2.2 Nagios 监控应用案例 311
第 6 章 网络安全 323
6.1 iptables 防火墙 323
6.1.1 iptables 防火墙语法格式 324
6.1.2 iptables 防火墙应用案例 327
6.1.3 防火墙备份与还原 330
6.2 SELinux 简介 331
6.2.1 SELinux 配置文件 331
6.2.2 SELinux 软件包 332
6.2.3 SELinux 安全上下文 333
6.2.4 SELinux 排错 334
6.2.5 修改安全上下文 336
6.2.6 查看与修改布尔值 338
6.2.7 SELinux 应用案例 339
6.2.8 httpd 相关的 SELinux 安全策略 339
6.2.9 FTP 相关的 SELinux 安全策略 340
6.2.10 MySQL 相关的 SELinux 安全策略 341

6.2.11 NFS 相关的 SELinux 安全策略 342
6.2.12 Samba 相关的 SELinux 安全策略 343
6.3 OpenVPN 344
6.3.1 OpenVPN 简介 344
6.3.2 安装 OpenVPN 服务 345
6.3.3 OpenVPN 客户端 348

第 3 篇 高级应用

第 7 章 虚拟化技术 352
7.1 虚拟化产品对比 352
7.1.1 VMware 虚拟化技术 353
7.1.2 Xen 虚拟化技术 353
7.1.3 KVM 虚拟化技术 354
7.2 KVM 虚拟化应用案例 354
7.2.1 安装 KVM 组件 354
7.2.2 创建虚拟机操作系统 355
7.2.3 监控虚拟机操作系统 360
7.2.4 命令工具使用技巧 362
7.2.5 虚拟存储与虚拟网络 367
第 8 章 集群及高可用 377
8.1 集群 377
8.1.1 LVS 负载均衡简介 377
8.1.2 基于 NAT 的 LVS 负载均衡 378
8.1.3 基于 TUN 的 LVS 负载均衡 379
8.1.4 基于 DR 的 LVS 负载均衡 380
8.1.5 LVS 负载均衡调度算法 381
8.1.6 部署 LVS 服务 382
8.1.7 LVS 负载均衡应用案例 385
8.1.8 常见问题分析 392
8.2 Keepalived 双机热备 393
8.2.1 Keepalived 简介 393
8.2.2 VRRP 协议简介 394
8.2.3 安装 Keepalived 服务 394
8.2.4 配置文件解析 395
8.2.5 Keepalived+LVS 应用案例 396
8.2.6 常见问题分析 404
8.3 Squid 代理服务器 405
8.3.1 Squid 简介 405
8.3.2 安装 Squid 服务 405
8.3.3 常见代理服务器类型 406
8.3.4 配置文件解析 407
8.3.5 Squid 应用案例 408
8.4 HAProxy 负载均衡 412
8.4.1 HAProxy 简介 412
8.4.2 配置文件解析 413
8.4.3 HAProxy 应用案例 416
8.5 Nginx 高级应用 420
8.5.1 Nginx 负载均衡 420
8.5.2 Nginx 负载均衡案例 422
8.5.3 Nginx rewrite 规则 425
8.6 MySQL 高可用 429
8.6.1 MySQL 复制 429
8.6.2 一步一步操作 MySQL 复制 430



基础知识

第1章

部署操作系统

1.1 光盘安装 Linux 操作系统

1.1.1 操作系统版本的选择

首先我们需要弄清楚两个基本的概念：Linux 与 Linux 系统。Linux 仅代表系统的内核，同时 Linux 商标的所有者是 Linus Torvalds。而 Linux 系统指的是基于 Linux 内核的操作系统。一份完整的 Linux 系统一般由内核与程序结合组成，这样的系统正式对外发行即成了现在市面上常见的 Linux 发行版本，这种发行版本又分为商业版本与社区版本。

当前比较流行的发行版本有 RedHat Enterprise Linux、Fedora、CentOS、SuSE、Debian、Ubuntu 等。这些版本有些由商业公司维护，有些则由社区维护，大家可以根据自己的实际需求选择适合自己的发行版本。本书案例均以 CentOS 6.3 系统为操作平台。

1.1.2 光盘安装 Linux 系统案例

安装 Linux 操作系统最简单的方式是通过光盘安装，我们可以在 CentOS 官方网站上下载

ISO 镜像[1]，然后刻录成光盘。下面以 CentOS 6.3 为例讲解安装步骤。

在 BIOS 中设置光盘启动，计算机启动后进入如图 1-1 所示的欢迎界面，安装菜单的功能如表 1-1 所示，这里我们选择最简单的安装或升级操作系统菜单，并按“Enter”键确定。

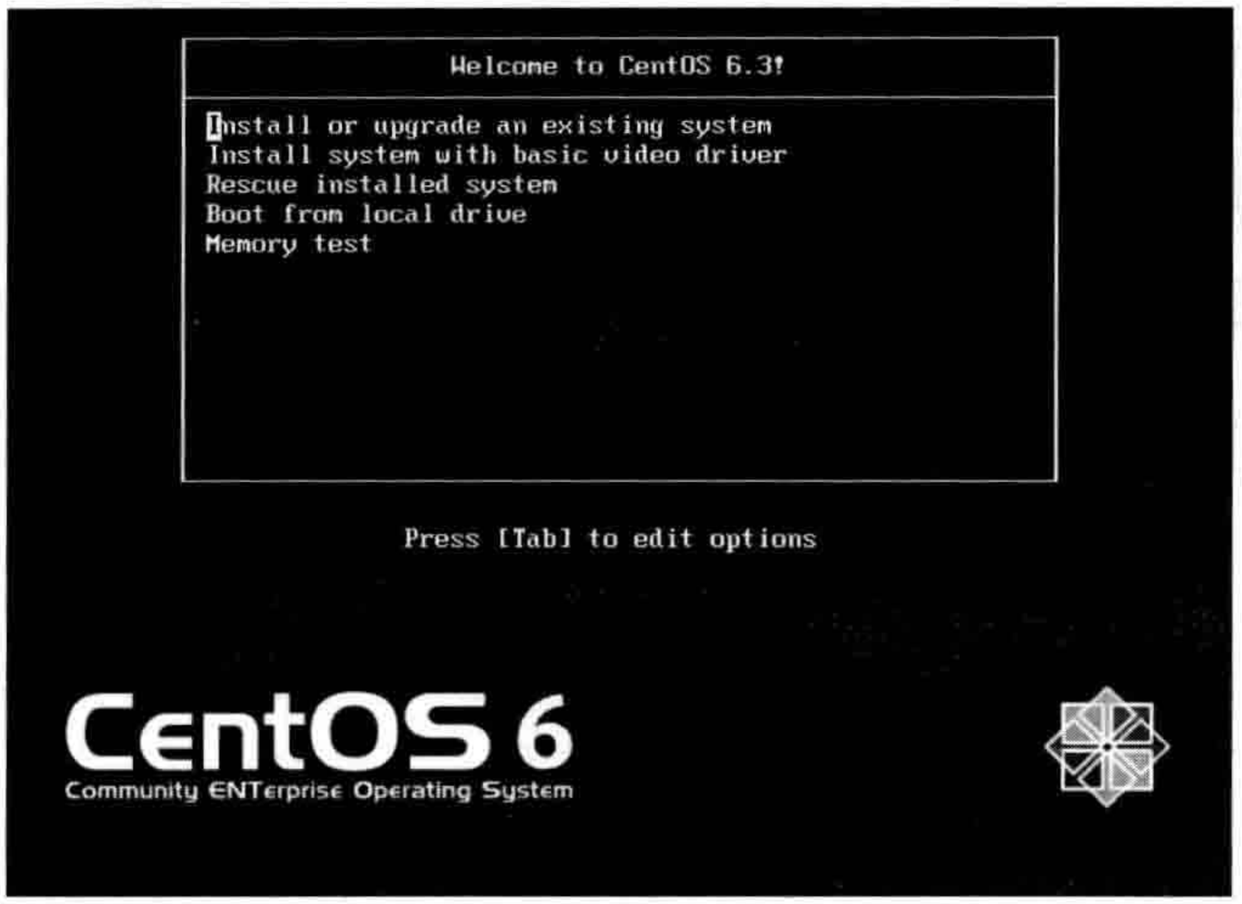


图 1-1

表 1-1

安装菜单	功能描述
Install or upgrade an existing system	安装或升级操作系统
Install system with basic video driver	使用简单显卡驱动安装系统
Rescue installed system	救援模式
Boot from local drive	从硬盘启动
Memory test	内存测试

选择安装方式后，提示是否检查光盘介质的完整性，如图 1-2 所示。在安装前检查完整性可以避免安装过程中出现无法读取光盘内容的错误，单击“OK”按钮进行检查，单击“Skip”按钮则跳过检查。

完成光盘介质的检查后，进入语言选择界面，如图 1-3 所示。这里的语言设置仅在安装过程中有效，如选择简体中文，即可设置后续的安装步骤为中文界面。

[1] 官方网站下载地址 http://mirror.neu.edu.cn/centos/6.3/isos/x86_64/。

图 1-2

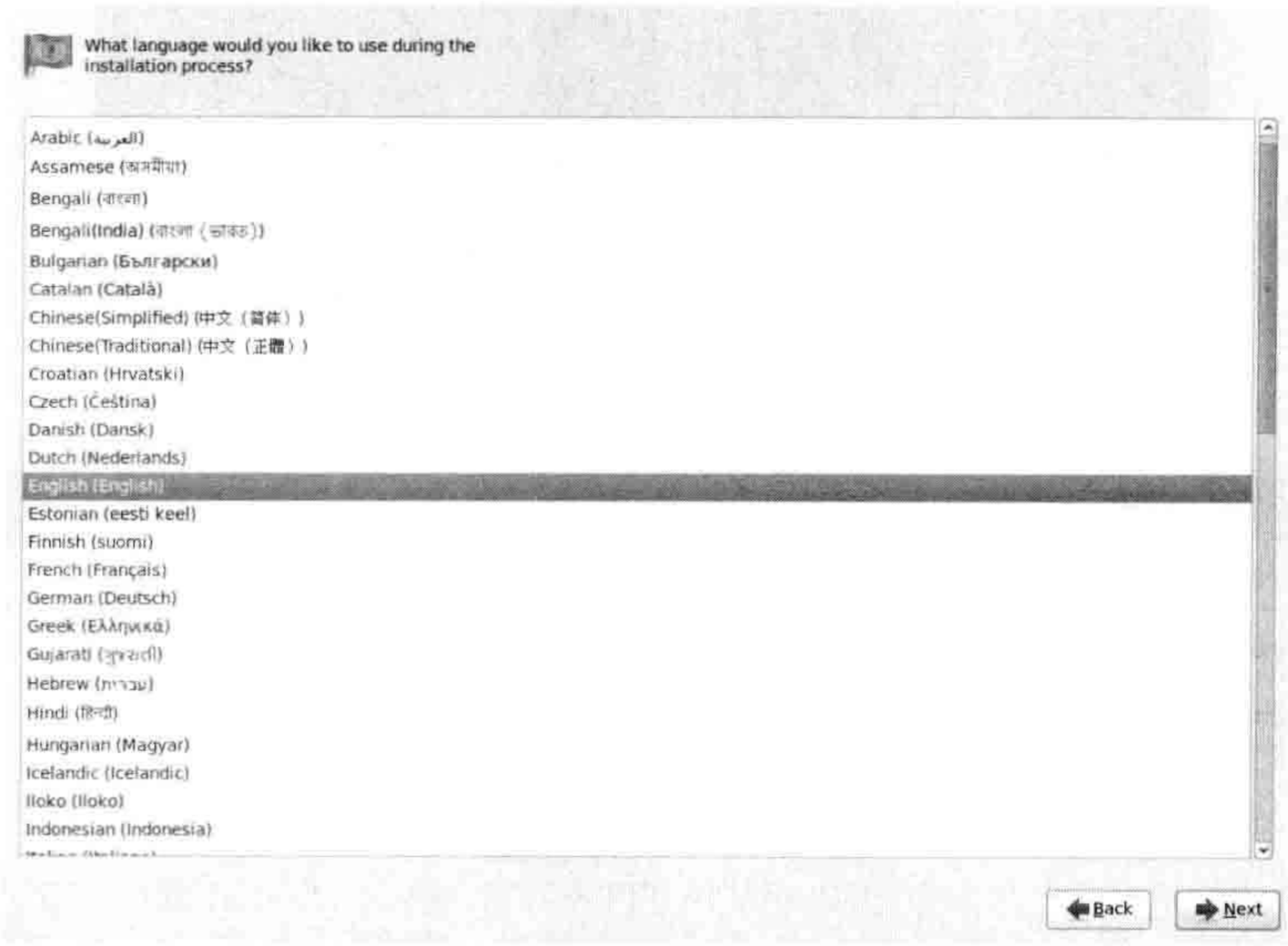


图 1-3

设置语言后，单击“Next”按钮进入键盘类型的选择界面，如图 1-4 所示。默认的键盘类型为 US（美式）键盘，我们国家的键盘一般都是这种类型的键盘，单击“Next”按钮进入设备选择界面。

选择安装设备，如图 1-5 所示，该步骤选择将 Linux 系统安装在什么设备上，默认为 Basic Storage Devices（基本存储设备），即使用本机磁盘设备安装操作系统。如果希望使用的存储设备为远程 SAN 设备，可以选择 Specialized Storage Devices（特定的存储设备）。

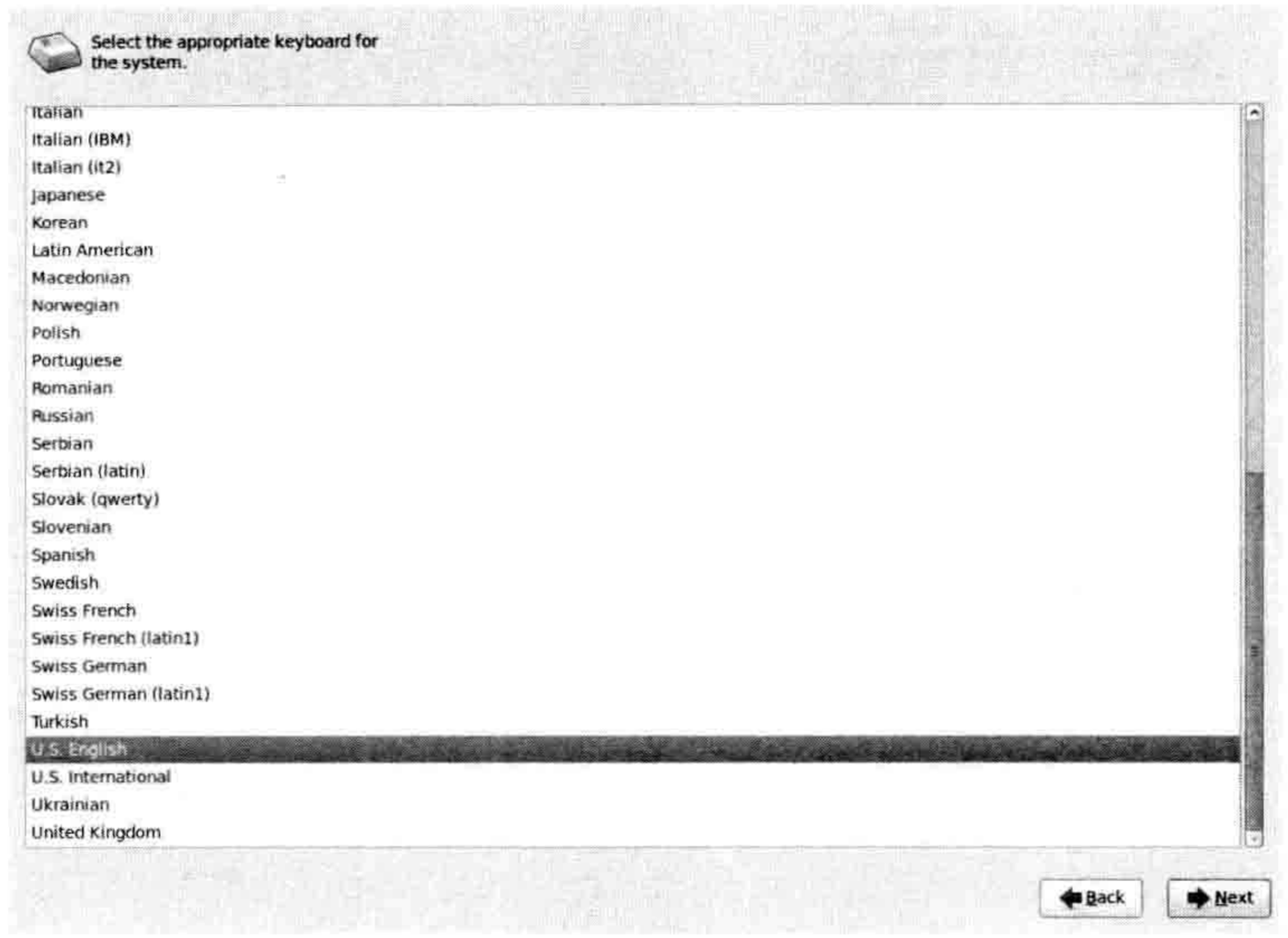


图 1-4

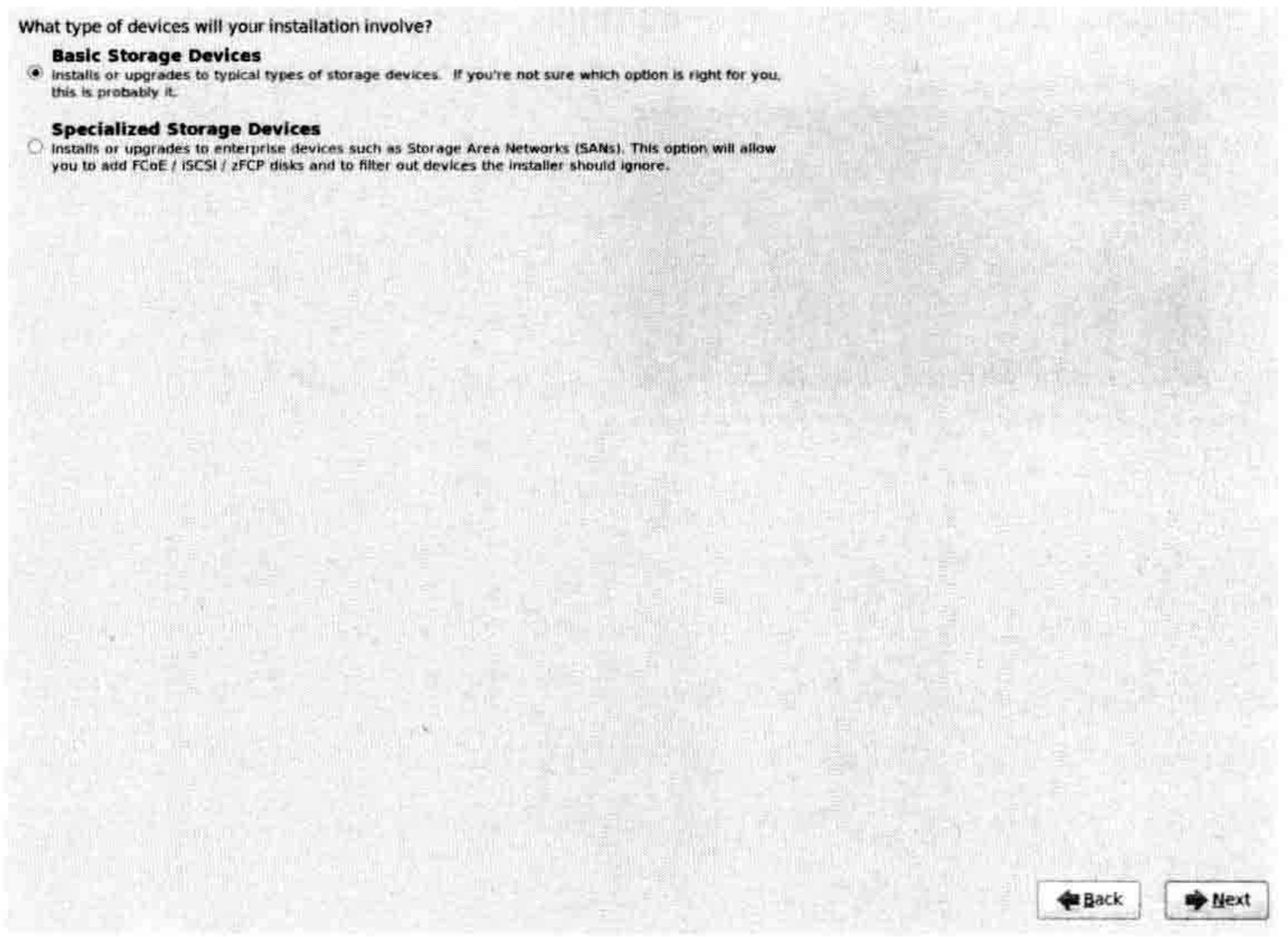


图 1-5

完成存储设备的选择后，将进入一系列简单的主机名称、时区、管理员密码与分区方式的设置。如图 1-6 所示，完成主机名称的输入。

图 1-6

单击“Next”按钮后，进入如图 1-7 所示的界面，选择合适的时区。

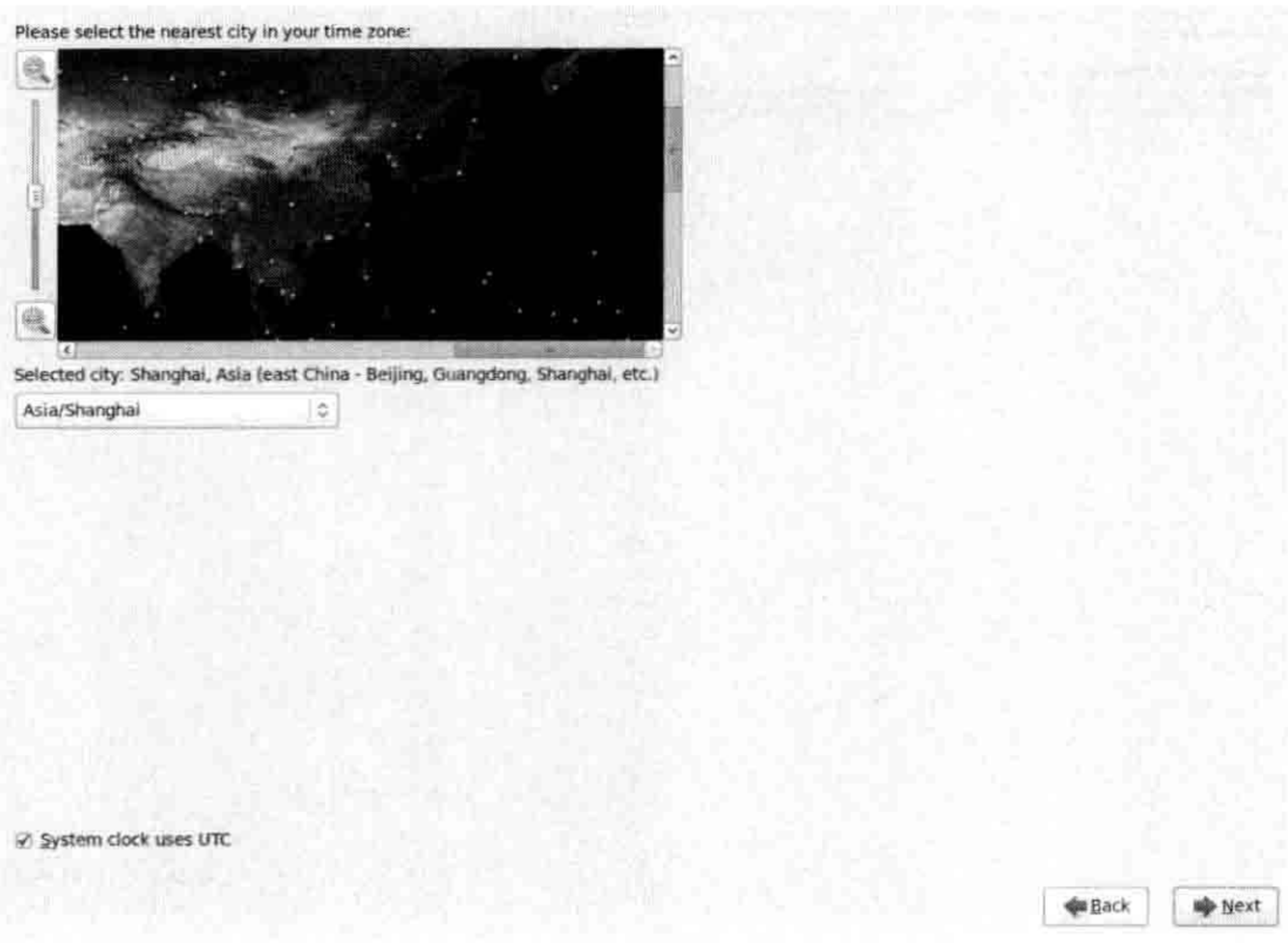


图 1-7

单击“Next”按钮后进入如图 1-8 所示的界面，输入 root（管理员）密码。

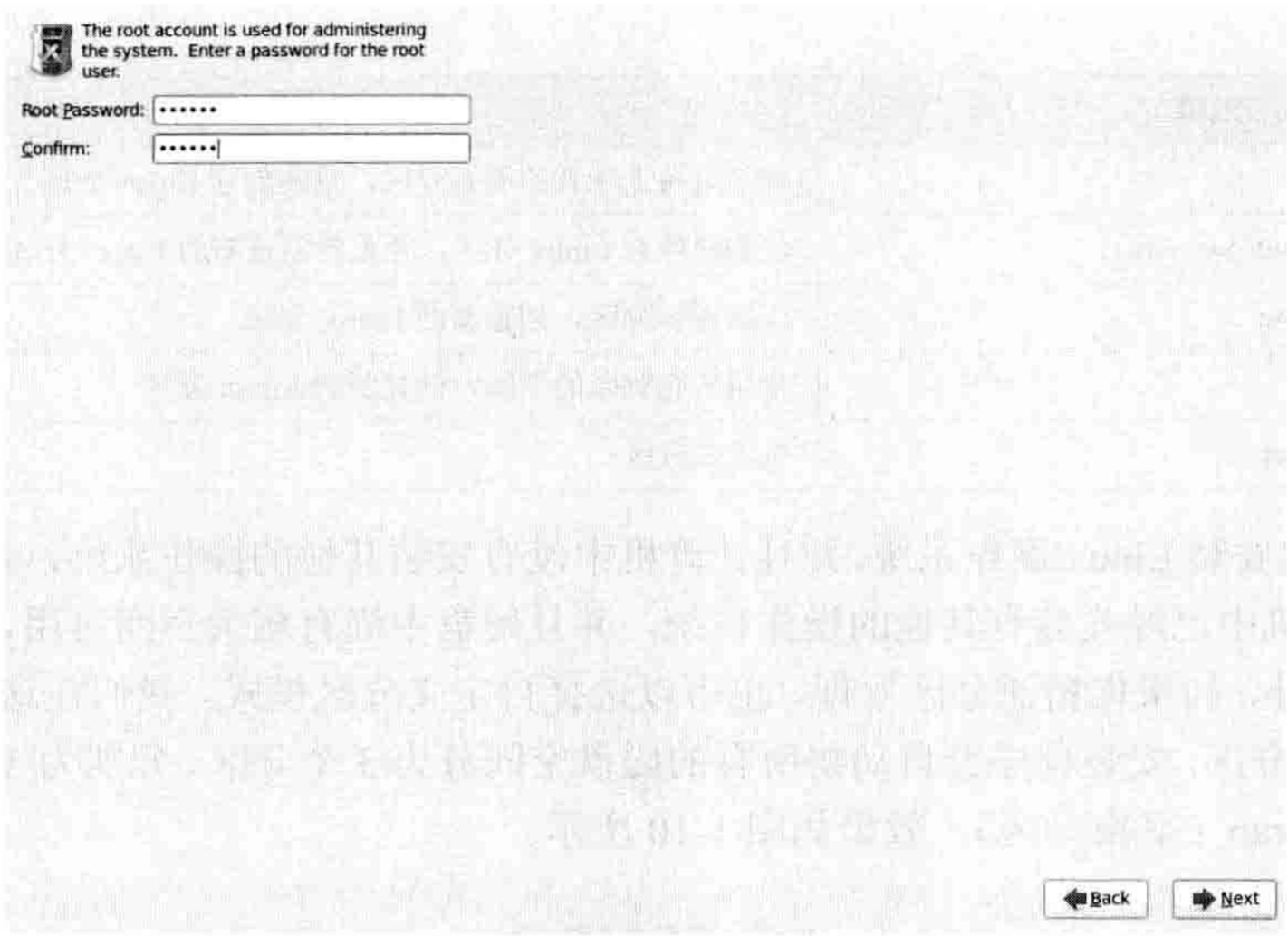


图 1-8

完成后单击“Next”按钮进入如图 1-9 所示的界面，选择分区方式，选项功能见表 1-2。

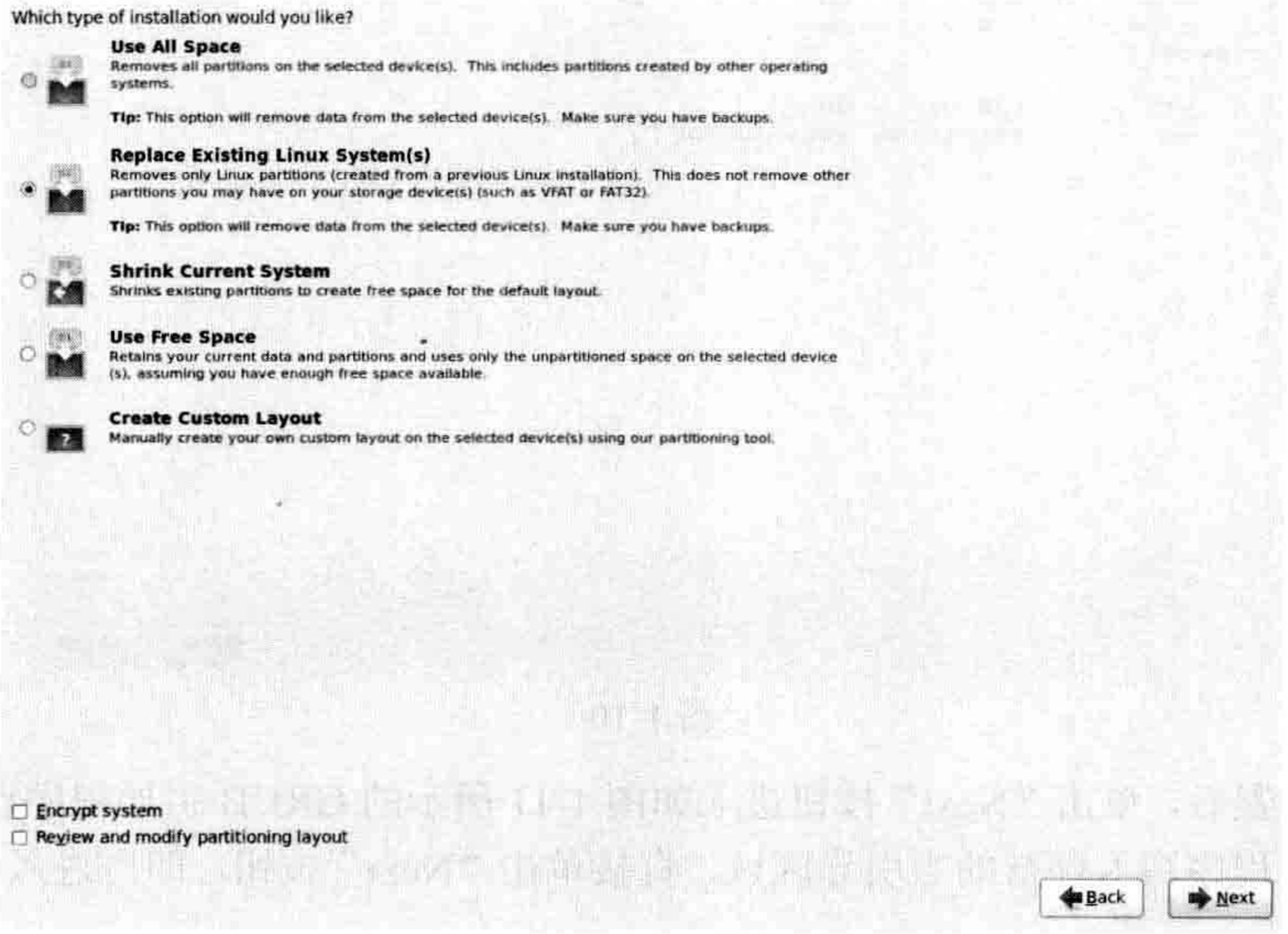


图 1-9

表 1-2

选项	功能描述
Use All Space	删除设备上现有的所有分区，重新创建 Linux 分区
Replace Existing Linux System(s)	仅删除现有 Linux 分区，并重新创建新的 Linux 分区
Shrink Current System	压缩现有分区，创建新的 Linux 分区
Use Free Space	使用所有剩余的空间，创建新的 Linux 分区
Create Custom Layout	自定义分区

若是第一次安装 Linux 操作系统，并且计算机中没有安装其他的操作系统，可以选择 Use All Space。若计算机中已经安装有其他的操作系统，并且硬盘中尚有剩余空间可用，可以选择 Use Free Space。另外，如果你精通分区原理，也可以选择自定义分区模式。我们在这里选择 Use All Space 方式自动分区，安装程序会自动将所有的磁盘空间分为 3 个分区，分别为/（根分区）、/boot（启动分区）、swap（交换分区），效果如图 1-10 所示。

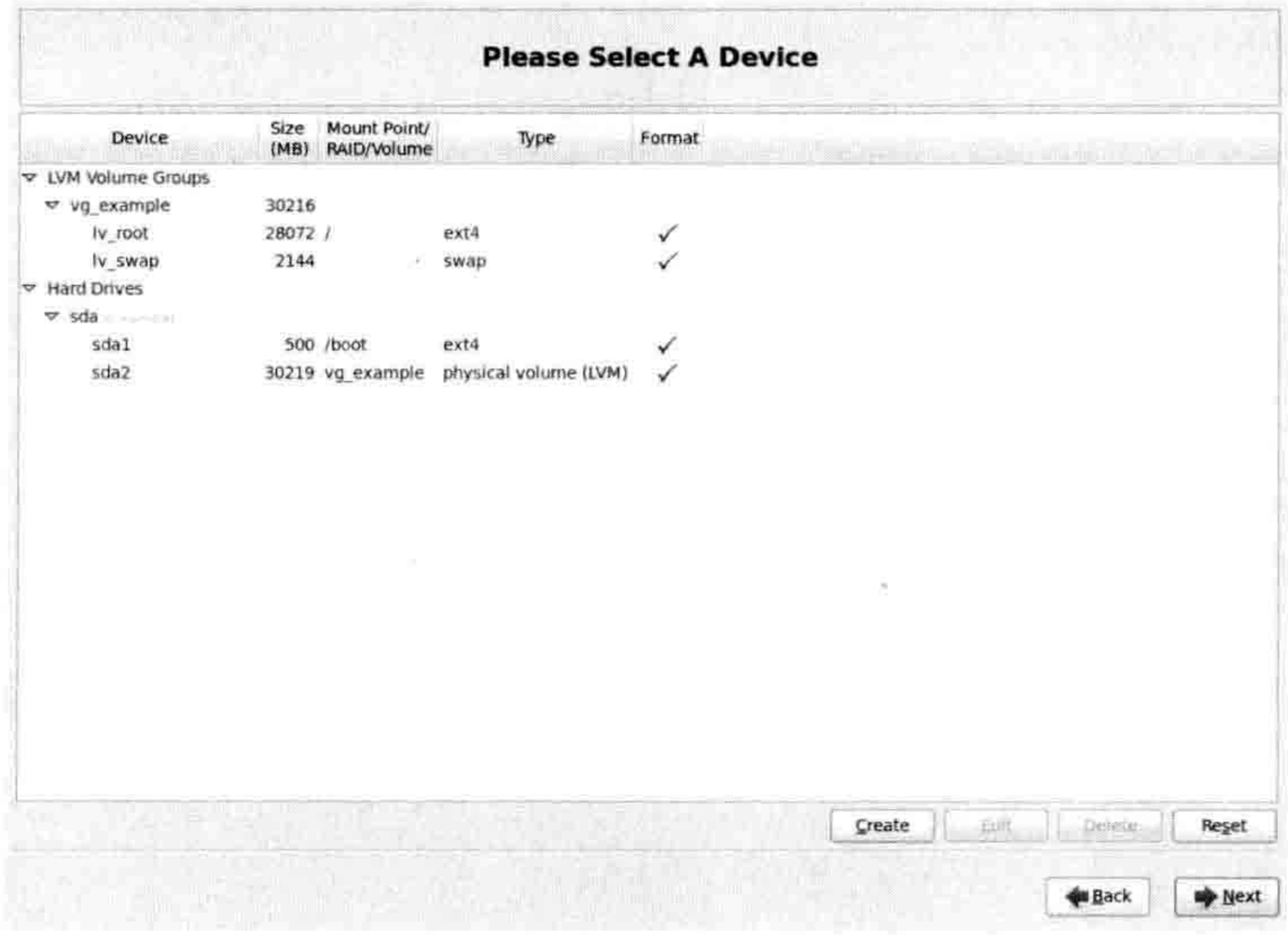


图 1-10

完成分区设置后，单击“Next”按钮进入如图 1-11 所示的 GRUB 引导程序设置，默认会将 GRUB 系统引导程序写入硬盘的主引导区域，直接单击“Next”按钮，即可进入软件包的选择。

如图 1-12 所示，根据需求安装不同的 Linux 应用，默认为最小化安装。CentOS 将应用大致分为桌面、最小化桌面、最小化安装、基础服务器、数据库服务器、网站服务器、虚拟主机、软件开发 8 项，选择不同的应用环境后，最终系统安装的软件包将有所不同。如果需要自定义

软件包的安装，可以选择图 1-12 下方的“Customize now”选项，并单击“Next”按钮，进入如图 1-13 所示的自定义选择软件包的界面。

图 1-11

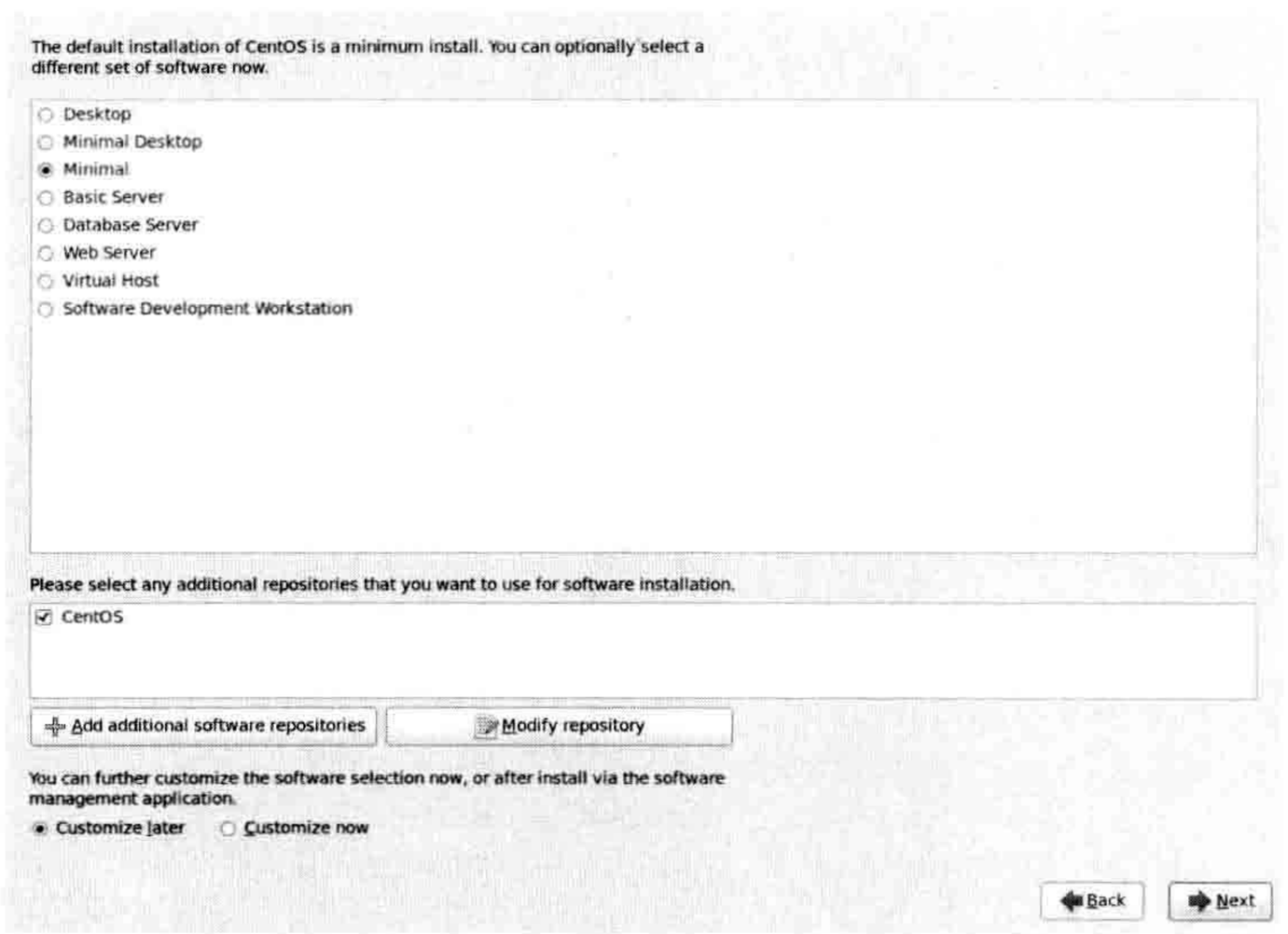


图 1-12

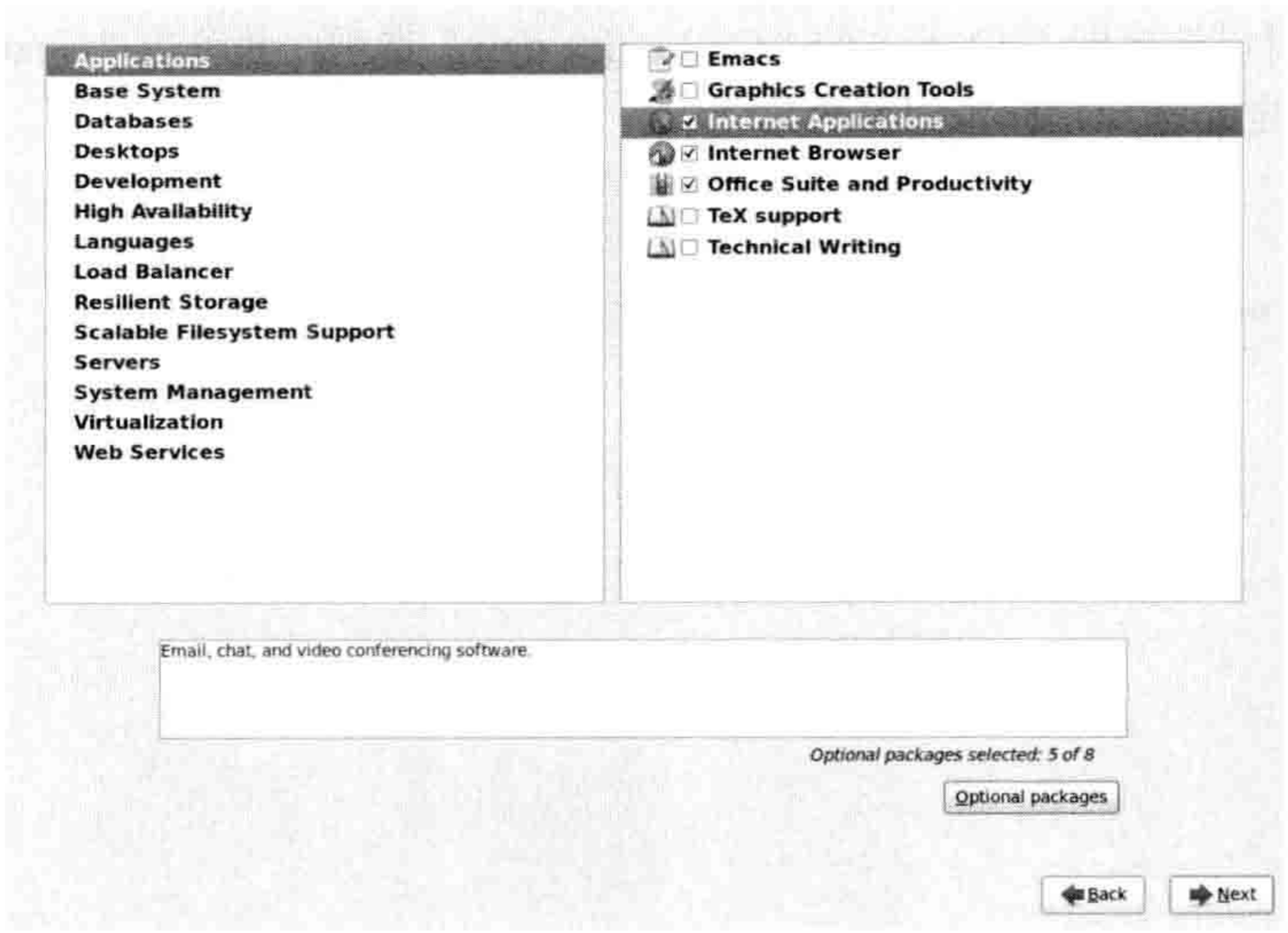


图 1-13

完成所有软件包的选择后，系统开始安装软件包，如图 1-14 所示。这一过程会因为所选软件包的多少而相应的等待时间也不同。当所有的软件包安装完成后，系统会提示重启，如图 1-15 所示。

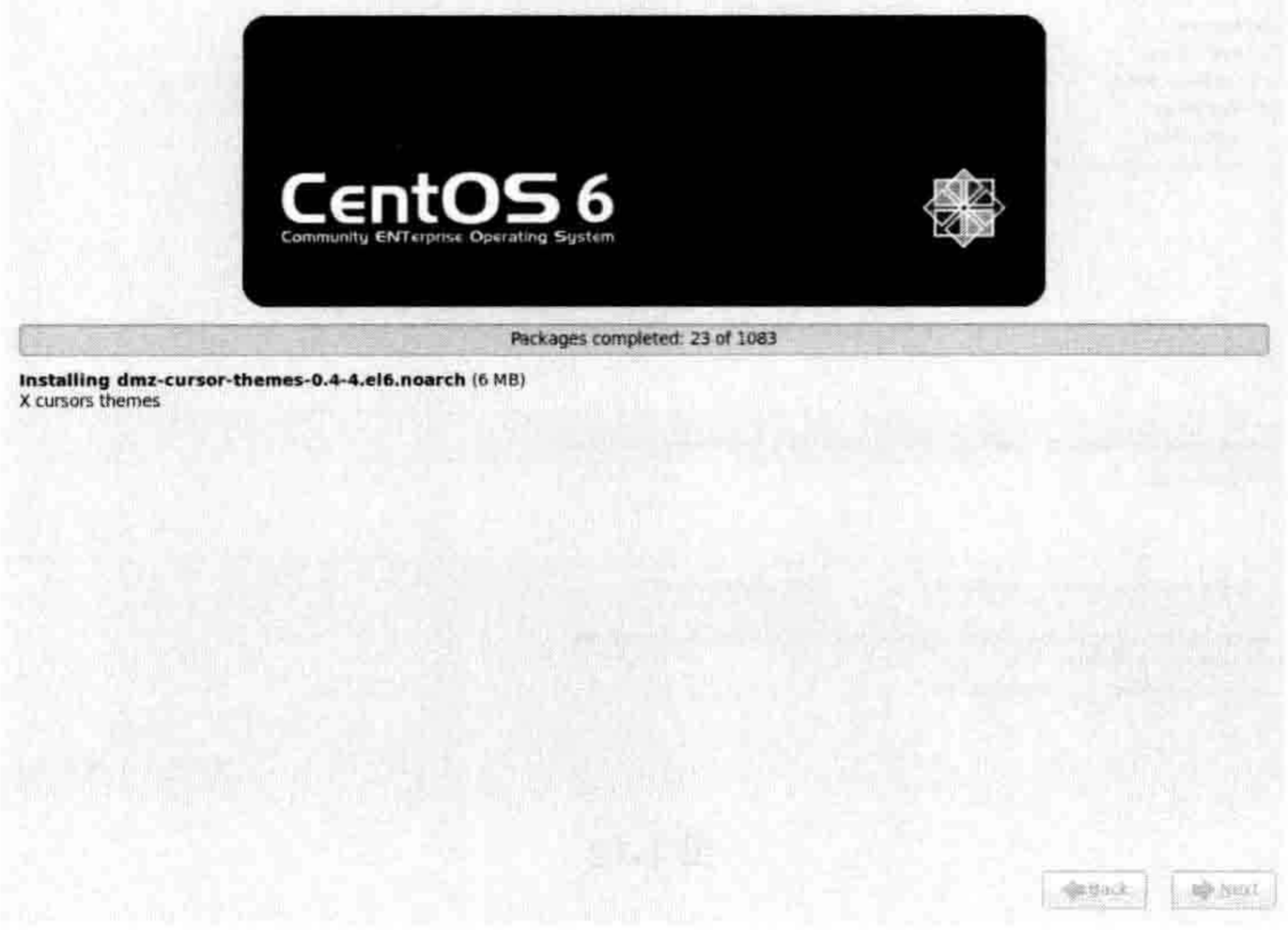


图 1-14

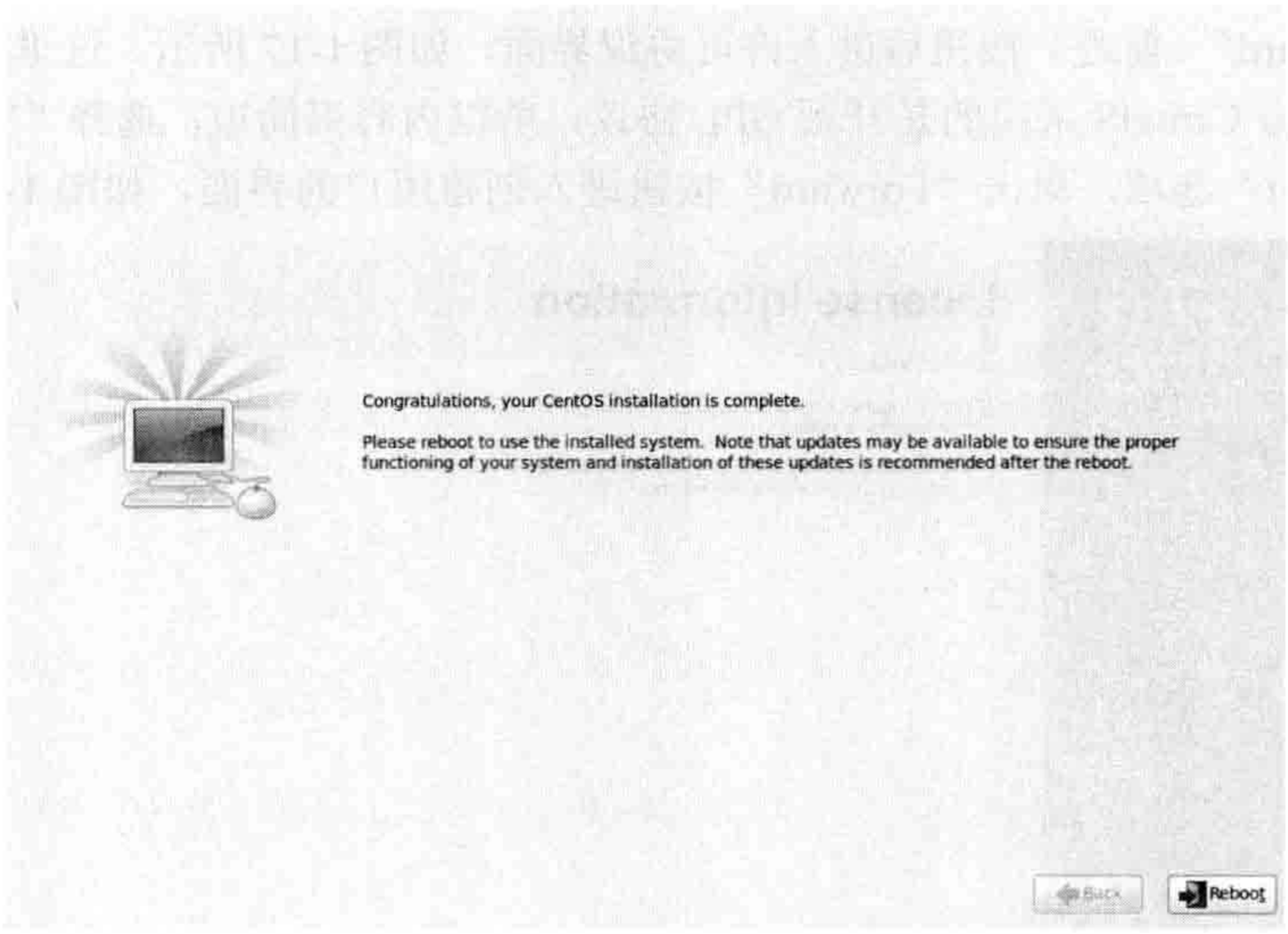


图 1-15

计算机完成重启后，还需要完成系统初始化操作，初始化欢迎界面如图 1-16 所示。

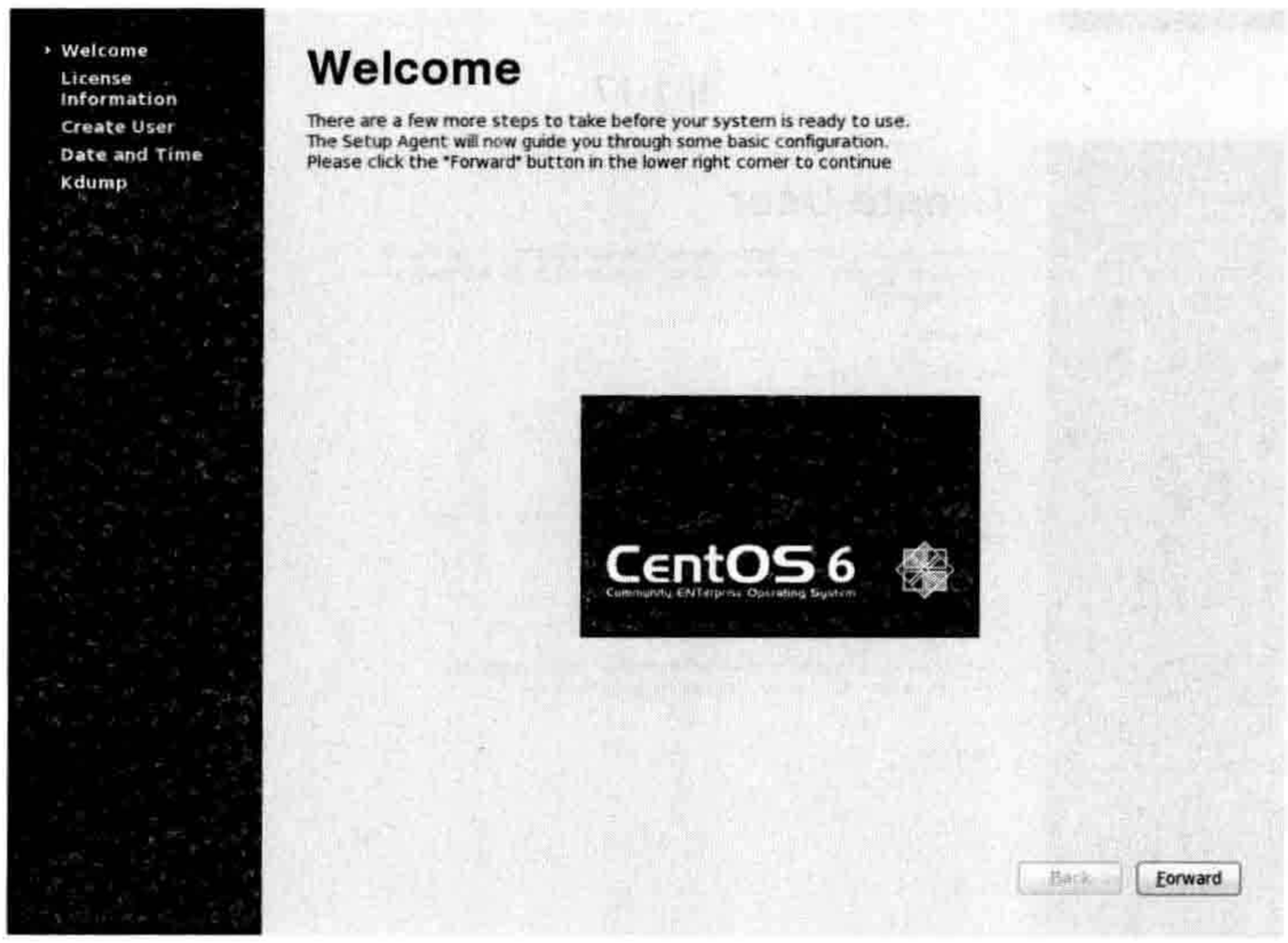


图 1-16

提示

计算机重启后，应该将计算机重新设置为从硬盘启动，否则会因为从光盘启动的原因造成反复安装操作系统的问题。

单击“Forward”（前进）按钮后进入许可协议界面，如图 1-17 所示。这里是对用户权利与责任的声明，因为 CentOS 采用的是开源 GPL 协议，所以内容很简短，选择“Yes, I agree to the License Agreement”选项，单击“Forward”按钮进入创建用户的界面，如图 1-18 所示。

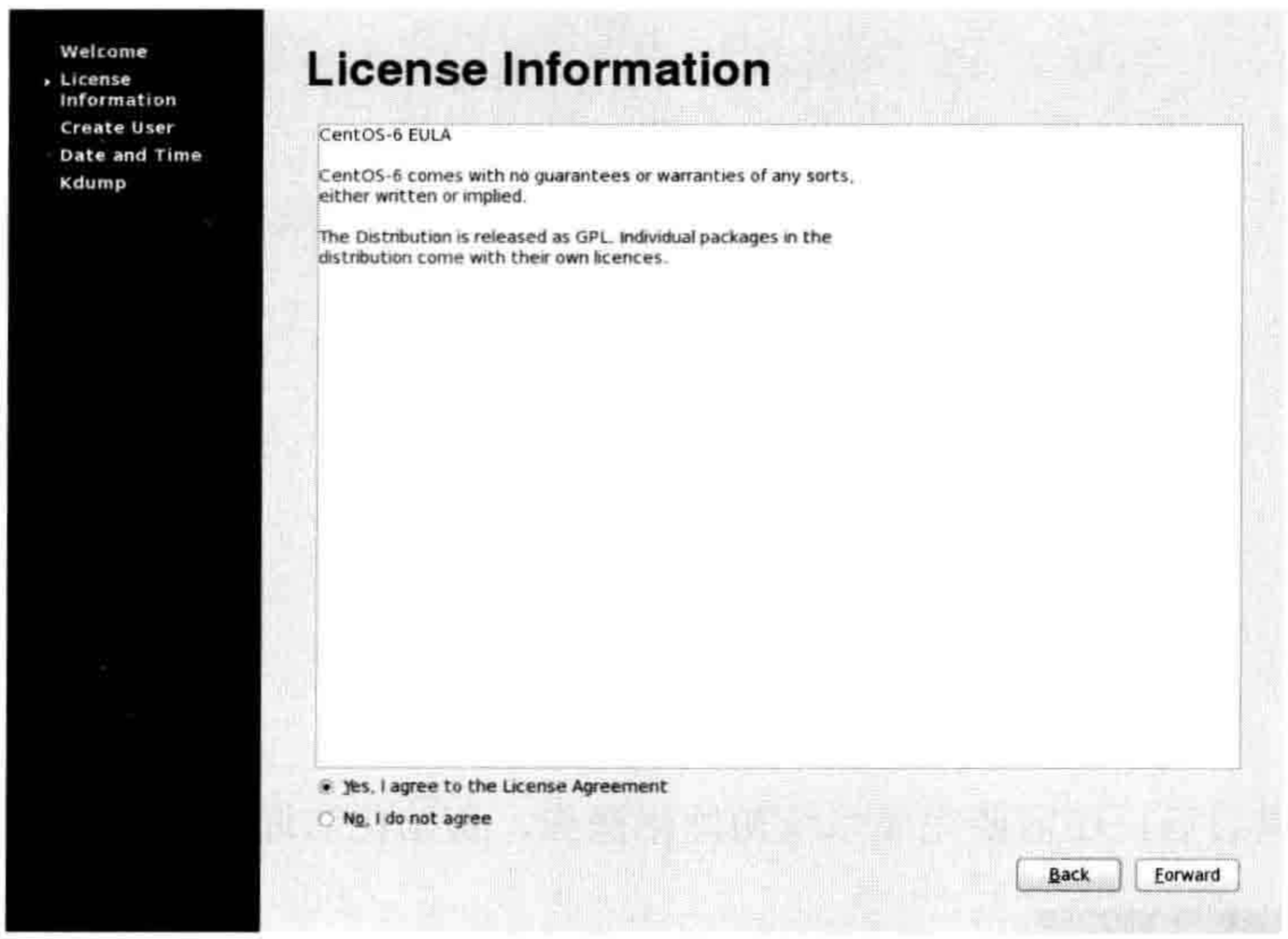


图 1-17

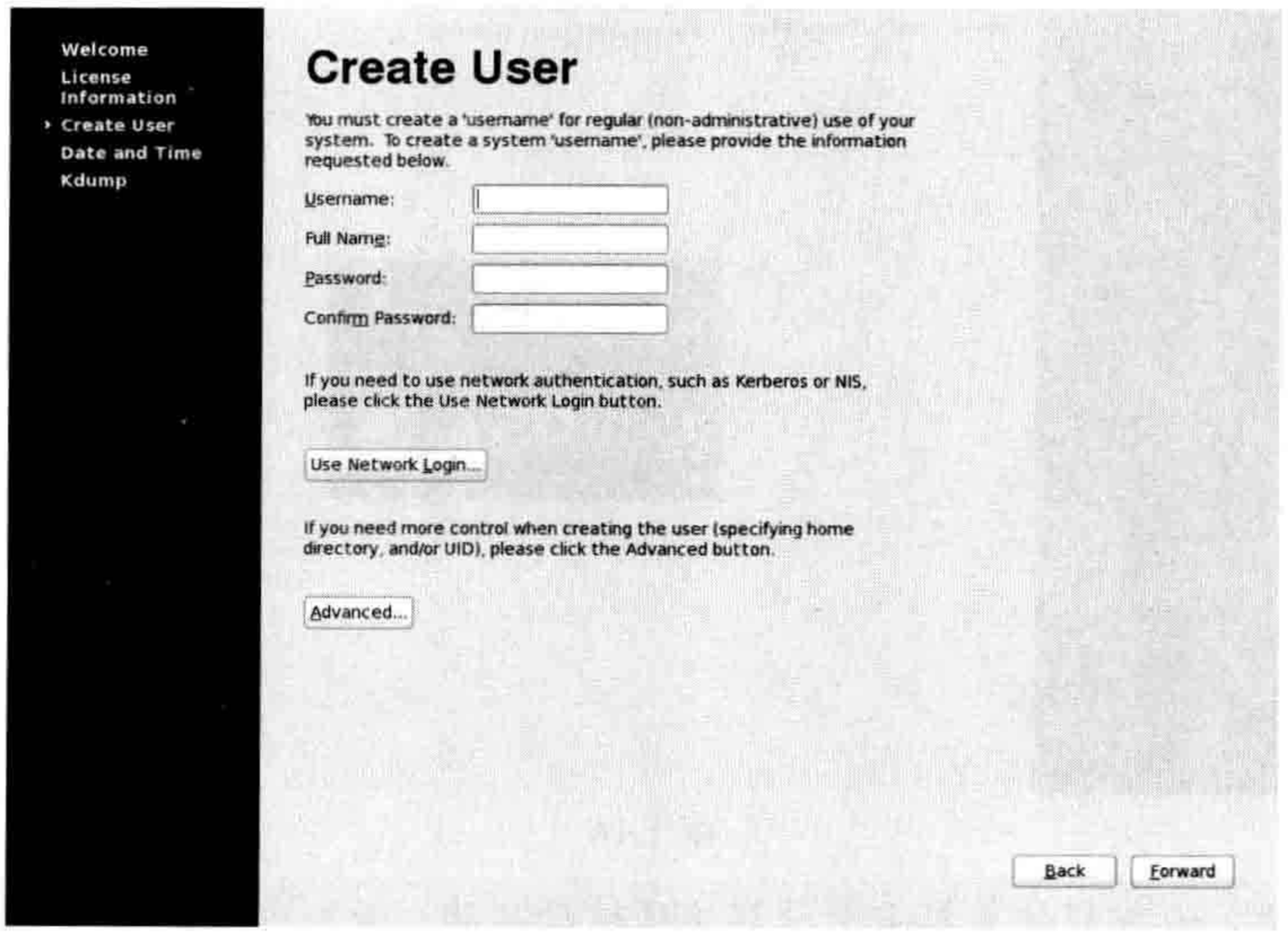


图 1-18

系统默认有一个超级管理员账户 root，并且我们在图 1-18 中已经为该账户设置了密码，但由于超级管理员的权限很大，所以系统初始化时会提示用户创建一个或多个普通用户，如图 1-18 所示，依次输入用户名、用户名全称、密码和确认密码即可。

提示 如果不希望创建普通用户，可以直接单击“Forward”按钮选择强制跳过，如图 1-19 所示。

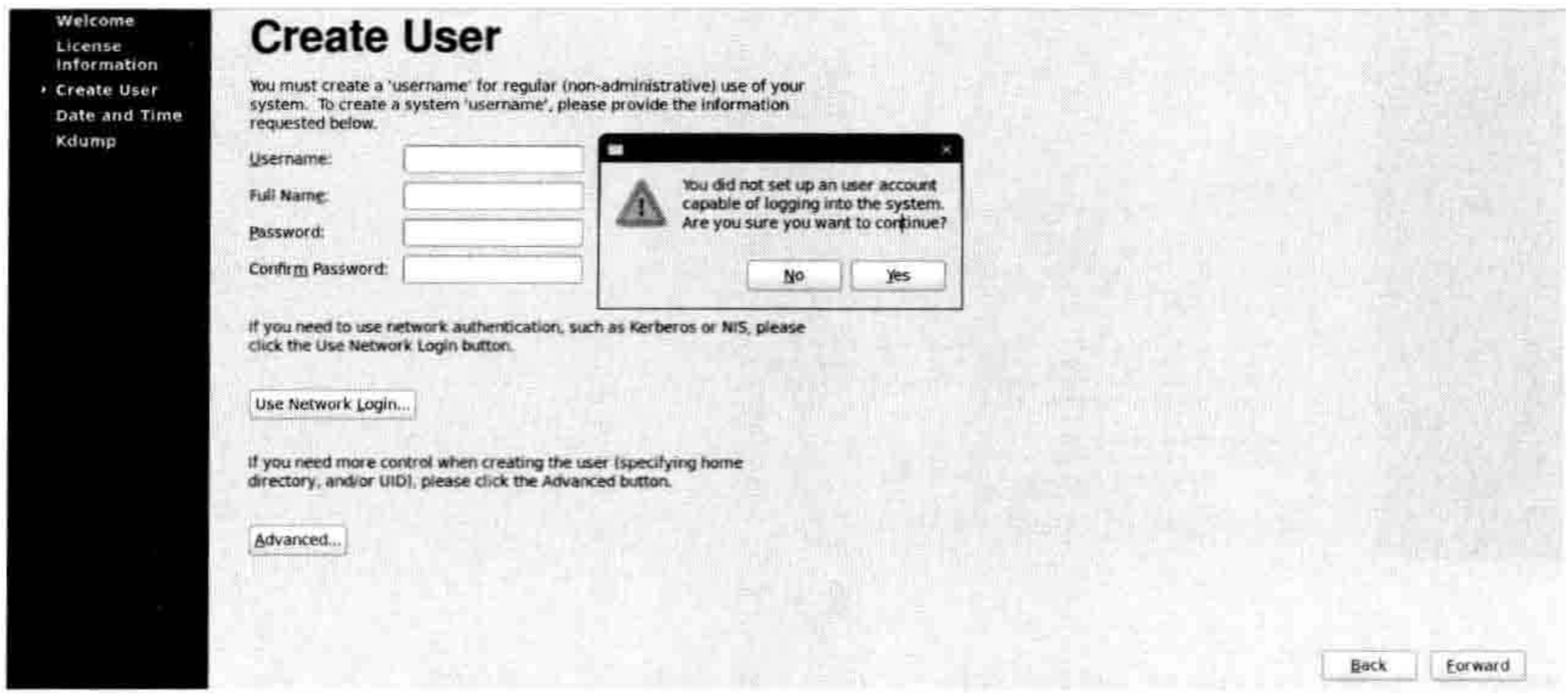


图 1-19

将系统时间调整为本地正确的时间，如图 1-20 所示。

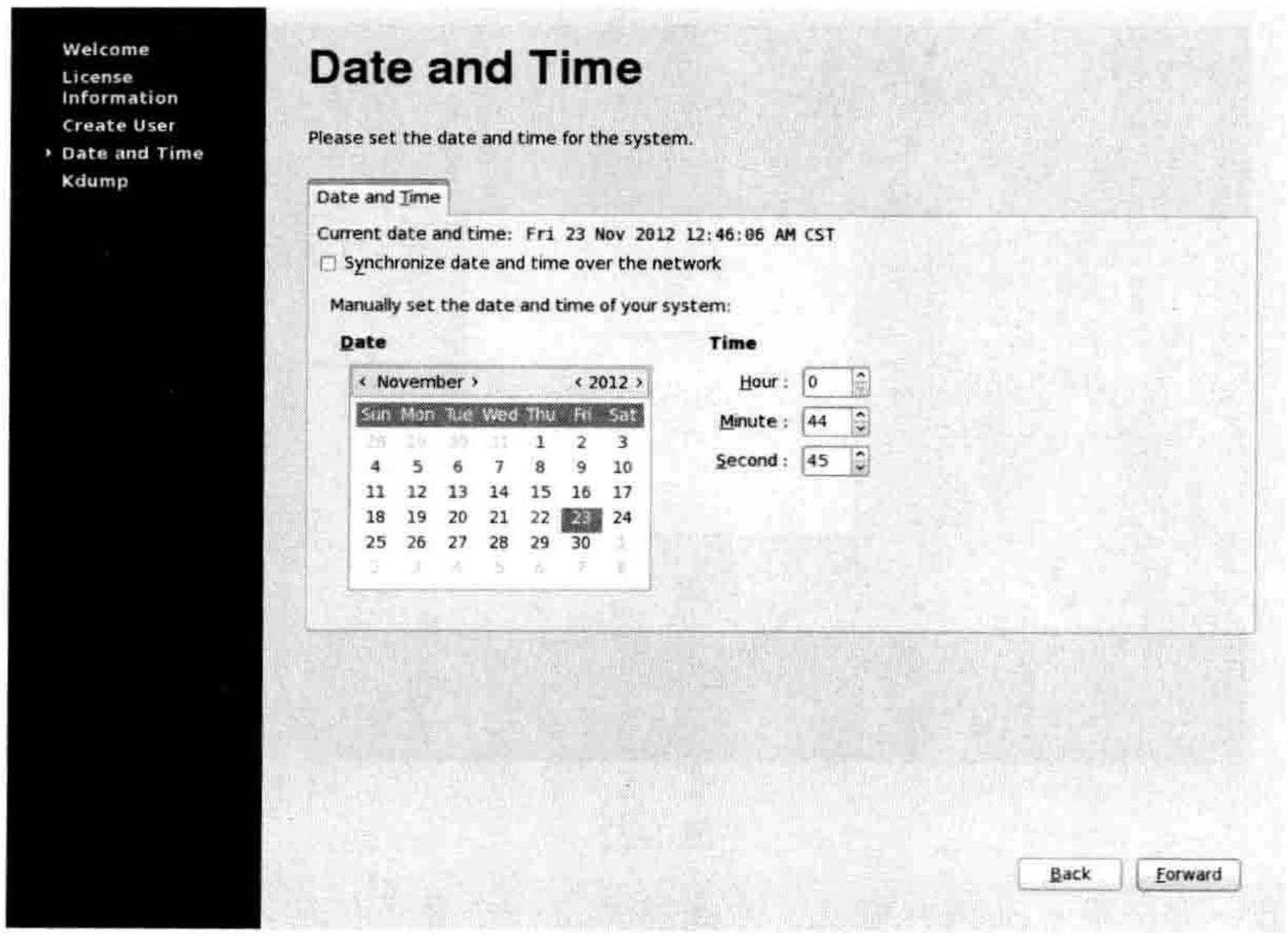


图 1-20

提示 图 1-20 中的 Kdump 是一种内核崩溃转储机制，当系统出现致命的问题时，可以利用它快速启动另一个内核。此时计算机内存需要足够大，否则 Kdump 将出现无法激活的状态，如图 1-21 所示。

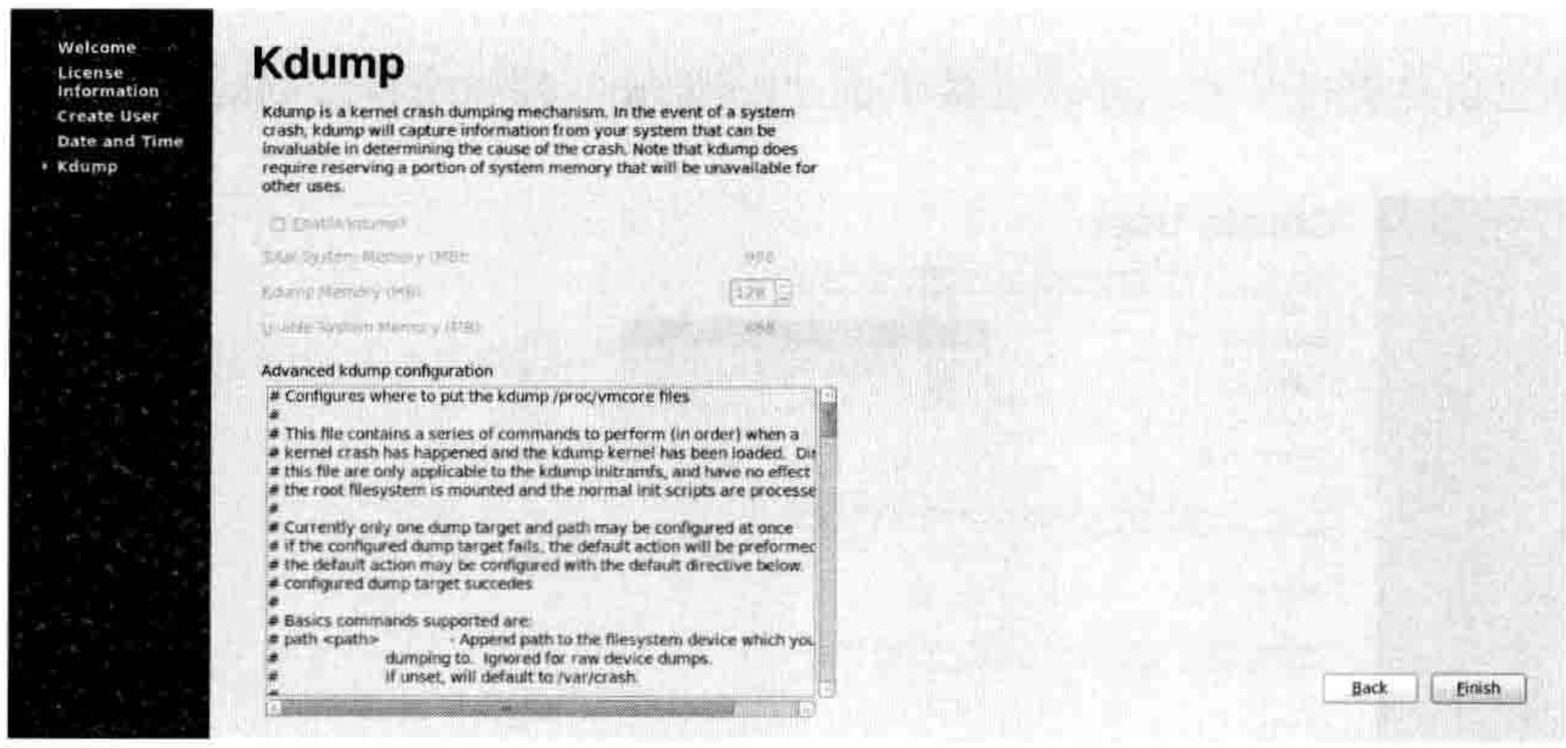


图 1-21

如果你已经完成了以上所有的初始化工作，恭喜你，你的第一份 Linux 操作系统已经安装完成，单击“Finish”按钮即可进入系统的登录画面，如图 1-22 所示。输入用户名与密码，开始感受一下全新的操作体验吧！

图 1-22

提示 如果你是第一次使用 root 账户登录，系统将弹出一个警告信息，提示你正在使用超级管理员账户，勾选“Do not show me this again”选项后关闭该提示，以后将不再提示。

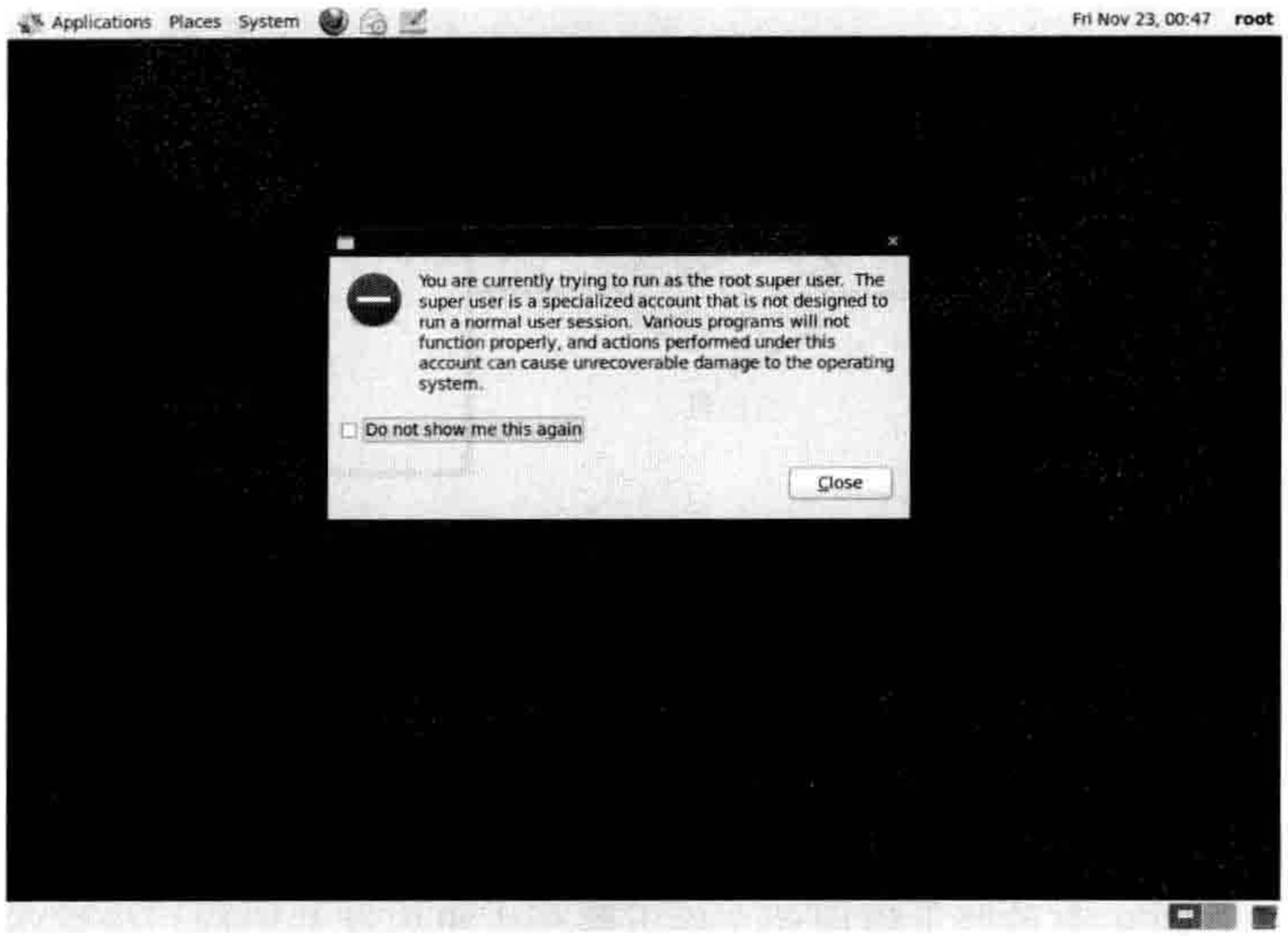


图 1-23

1.2　无人职守自动安装 Linux 操作系统

1.2.1　大规模部署案例

前面我们详细介绍了如何通过光盘手动安装部署 CentOS 6.3 操作系统，但这种安装方式并不适用于所有的应用环境，以作者的实验室机房为例，每个机房平均有 30 台主机，总计 10 个机房，而这些主机都需要统一安装部署 CentOS 6.3 操作系统。再如，像新浪、网易、淘宝这样的大型网络公司，会有上千台服务器需要部署 Linux 系统，如果此时依然使用光盘逐一为每台主机安装操作系统，则效率极其低下，这时就需要一种更加高效快捷的方式来统一部署这些系统。

目前行业中普遍采用的解决方案是通过网络方式安装并结合自动应答文件，实现无人职守自动安装部署操作系统。这种安装方式需要我们配置至少一台安装服务器，所有需要安装系统的客户端通过网络的方式连接服务器端启动安装程序，再根据服务器中存放的自动应答文件实现大规模自动安装部署系统。整体环境的拓扑结构如图 1-24 所示。

由于这种无人职守的解决方案需要提前部署一台包含 DHCP、TFTP、NFS 等服务的安装服务器，因此，如果你对这些服务还比较陌生，可以选择跳过本节内容，当你阅读完后面关于服务的相关章节后再看这部分内容会容易很多。

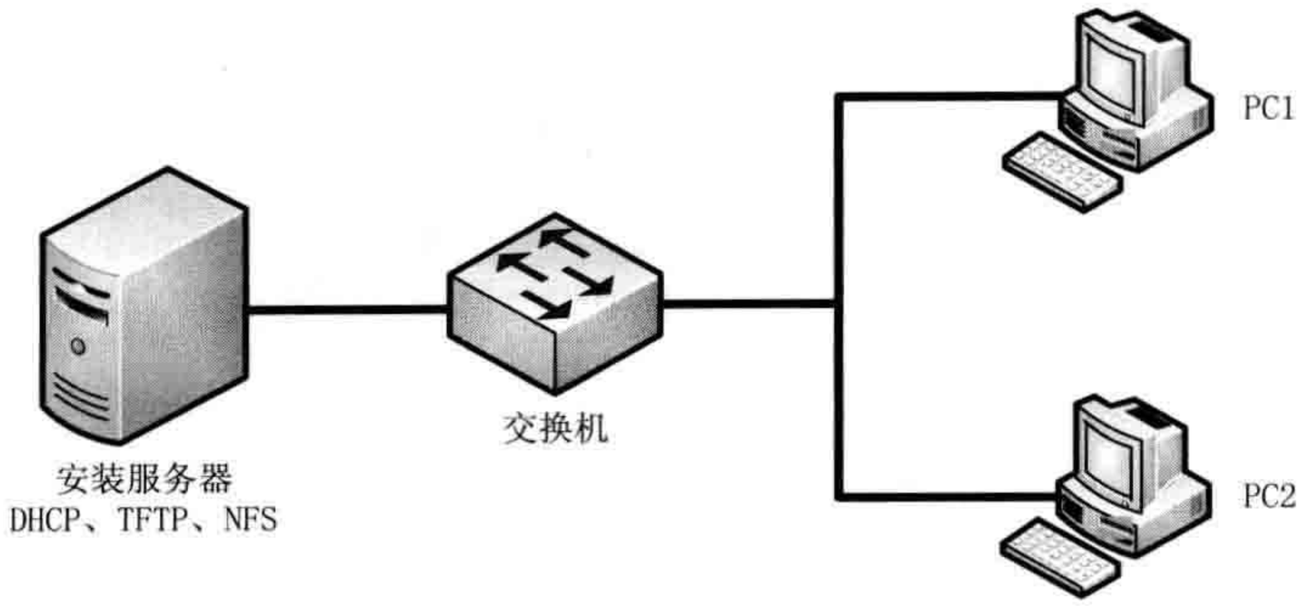


图 1-24

安装部署流程为：客户端首先需要在 BIOS 中设置通过网络启动，当客户端启动后，就会通过发送广播包的方式寻找 DHCP 服务器，如果找到 DHCP 服务器，即可向该服务器申请获得包括 IP 地址在内的网络参数等，并通过 DHCP 获得 TFTP[2]的位置，当客户端获得 TFTP 服务器的地址后，即可从 TFTP 服务器上将启动文件下载至本机内存并运行，最终实现无盘启动的功能。我们可以在启动文件中设置 Kickstart 文件共享的位置，这样客户端启动后，即可自动寻找 Kickstart 文件实现无人职守安装系统。注意，Kickstart 文件需要事先通过网络共享[3]。在 Kickstart 文件中描述了如何安装设置操作系统、运行部署脚本等。

1.2.2 PXE 简介

如今，计算机启动的方式有很多，前面我们介绍了计算机从光盘启动，读取光盘中的引导文件实现系统的安装，本节需要实现的是无光盘网络启动的无人职守安装，这里我们就需要一种名为 PXE 的技术。PXE 是由 Intel 公司开发的基于客户端/服务器模式的一种技术，其核心功能是让客户端通过网络从远端服务器下载启动镜像，从而实现网络启动。整个过程中，客户端要求服务器分配 IP 地址，再用 TFTP 协议下载位于服务器上的启动镜像到本机内存中并执行，由这个启动文件完成客户端基本软件的设置。

本书介绍的案例也需要客户端能从网络启动，读取位于服务器上的启动文件，实现安装系统的技术，但这样的技术只能实现从网络启动，当读取安装程序进入安装界面后，剩余的步骤如语言设置、系统管理员密码、网络参数等还需要我们手动配置，至此，我们仅可实现无光盘网络启动，若要实现无人职守自动安装，还需要一种所谓 Kickstart 的技术。

---

[2] TFTP 即简单文件共享服务，本环境中的 TFTP 存放了可供客户端启动的启动文件。

[3] 本节案例使用 NFS 共享 Kickstart 文件。

1.2.3　Kickstart 技术

Kickstart 安装是目前主要的一种无人职守自动部署安装操作系统的方式，使用这种技术，我们可以很轻松地实现自动安装及配置操作系统。这种技术的核心是自动应答文件（Kickstart 文件），即将本来在安装过程中需要我们手动设置的语言、密码、网络参数等通过读取自动应答文件实现自动设置。也就是说，我们需要事先将对操作系统的设置写入自动应答文件，开始安装操作系统时，指定安装程序读取自动应答文件实现自动安装及部署操作系统。

Kickstart 文件可以通过如下三种方式生成。

- 手动书写（仅需要一个文本编辑工具即可）。
- 通过 system-config-kickstart 图形工具。
- 通过红帽的安装程序 Anaconda 自动生成。

这里主要以 system-config-kickstart 图形工具为主，介绍如何生成一份自动应答文件，具体步骤如下。

首先需要安装 system-config-kickstart 图形工具，在终端命令行输入如下命令：

```
[root@CentOS6 ~]# yum -y install system-config-kickstart
```

安装完成后，可以通过输入 system-config-kickstart 命令运行该图形工具，效果如图 1-25 所示。

```
[root@CentOS6 ~]# system-config-kickstart
```

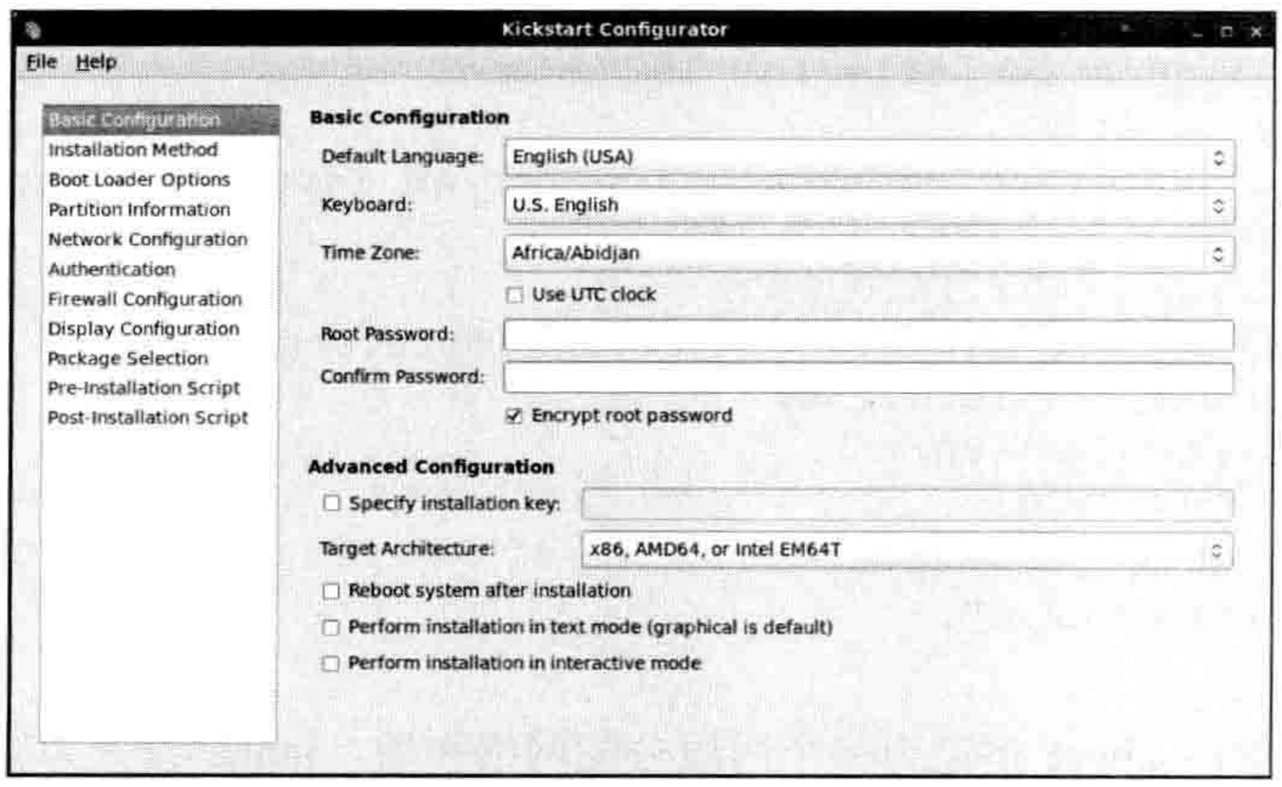


图 1-25

在图 1-25 中，我们可以通过该图形工具进行系统基本配置、安装方式配置、启动程序配置、分区配置、网络配置、认证配置、防火墙配置、显示配置、安装包配置。除此之外，还可以编

写系统安装前及安装后需要运行的脚本。在 1.2.5 节中，将给出使用该图形工具生成 Kickstart 文件自动部署 Linux 操作系统的完整案例。

1.2.4 配置安装服务器

从图 1-24 中可以看出，如果最终要实现无人职守自动安装部署操作系统，需要提前定制安装服务器，其中包括 DHCP、TFTP、NFS 服务。下面将分别介绍这三种服务的配置方法。

1. DHCP 服务

DHCP 服务器主要实现在企业内部网络为客户端分配 IP 地址等网络参数。在我们的无人职守环境中，当客户端选择从网络启动后，就会通过发送广播数据包的形式寻找 DHCP 服务器，从 DHCP 获得 IP 地址等参数后才可以通过 TFTP 共享读取启动文件。以下是在 CentOS 6.3 平台上安装部署 DHCP 的简单步骤。

首先，使用如下命令进行安装 DHCP 服务操作。

```
[root@CentOS6 ~]# yum -y install dhcp
```

安装完成后，DHCP 服务的主配置文件为/etc/dhcp/dhcpd.conf。我们可以修改该配置文件以实现为客户端分配网络参数，以下为简单的配置文件样本。

```
[root@CentOS6 ~]# cat /etc/dhcp/dhcpd.conf
# dhcpd.conf
# Sample configuration file for ISC dhcpd
log-facility local7;
# A slightly different configuration for an internal subnet.
subnet 172.16.0.0 netmask 255.255.0.0 {
  range 172.16.0.1 172.16.0.250;
  option domain-name-servers centos6.example.com;
  option routers 172.16.0.254;
  default-lease-time 600;
  max-lease-time 7200;
  next-server 172.16.0.254;
  filename "pxelinux.0";
}
```

上述样本文件中，subnet 指定为哪个网段分配网络参数，range 设置为客户端分配的 IP 地址池（一个地址区间），这里可以为客户端分配从 172.16.0.1 到 172.16.0.250 之间的 IP 地址。学习完本书后面章节后，我们还可以根据客户端 MAC 地址分配固定 IP。domain-name-servers 设置为客户端分配的 DNS 服务器地址，routers 设置为客户端分配的网关地址。对网络启动至关重

要的参数是 next-server 与 filename，从安装部署流程可以看出，客户端启动计算机通过 DHCP 获得 IP 地址后，还需要从 TFTP 下载启动文件，而 next-server 设置的即 TFTP 服务器的地址，filename 设置的是在该 TFTP 文件服务器上共享的启动文件名称，客户端通过这两个参数连接 TFTP 服务器，并从中下载启动文件。

2. TFTP 服务

TFTP 服务器为客户端提供一种简单的文件共享，它不具备 FTP 那样丰富的功能。由于其简单的设计，TFTP 非常适用于传输小且简单的 PXE 启动文件。使用如下命令安装该软件：

```
[root@CentOS6 ~]# yum -y install tftp-server
```

安装完成 TFTP 服务后，还需要继续修改配置文件，默认的 TFTP 配置文件为 /etc/xinetd.d/tftp。在该文件中，disable 的默认值为 yes，也就是说，该服务默认为禁用状态，需要手动修改其值为 no 来启动 TFTP 服务。另外，该配置文件中 server_args 的值定义了共享的目录，也就是我们的共享文件要放置的目录路径。

3. NFS 服务

NFS 是 Network File System 的简写，即网络文件系统。NFS 服务的特点是服务器配置简单，客户端通过网络访问 NFS 文件系统就像访问自己的本地磁盘一样方便。这里使用它作为 CentOS 系统文件的共享服务平台，当客户端从网络启动正式进入安装界面后，还需要读取 CentOS 光盘中的系统文件，以完成最后的安装。这些文件就通过 NFS 共享给网络用户。一台 Linux 系统默认就安装有 NFS 服务，所以仅通过修改配置文件提供共享文件即可。配置文件的样本文件如下：

```
[root@CentOS6 ~]# cat /etc/exports
/media      172.16.0.0/255.255.0.0(ro)
```

默认该配置文件的内容为空，样本文件中编写了共享一个/media 目录，后面的 IP 地址表示仅 172.16.0.0 网络可以访问该共享目录，ro 表示客户端访问的权限为只读（read only）。

1.2.5　自动化安装案例

本节介绍一个自动化安装部署操作系统的完整案例，其拓扑结构图如图 1-26 所示。从图中可以看出，为了减轻安装服务器的负载，我们将 NFS 单独放置在一台独立的服务器中以提高性能。两台安装服务器的 IP 地址分别为 172.16.0.253 和 172.16.0.254，对应的主机名分别为 boot.example.com 和 nfs.example.com。

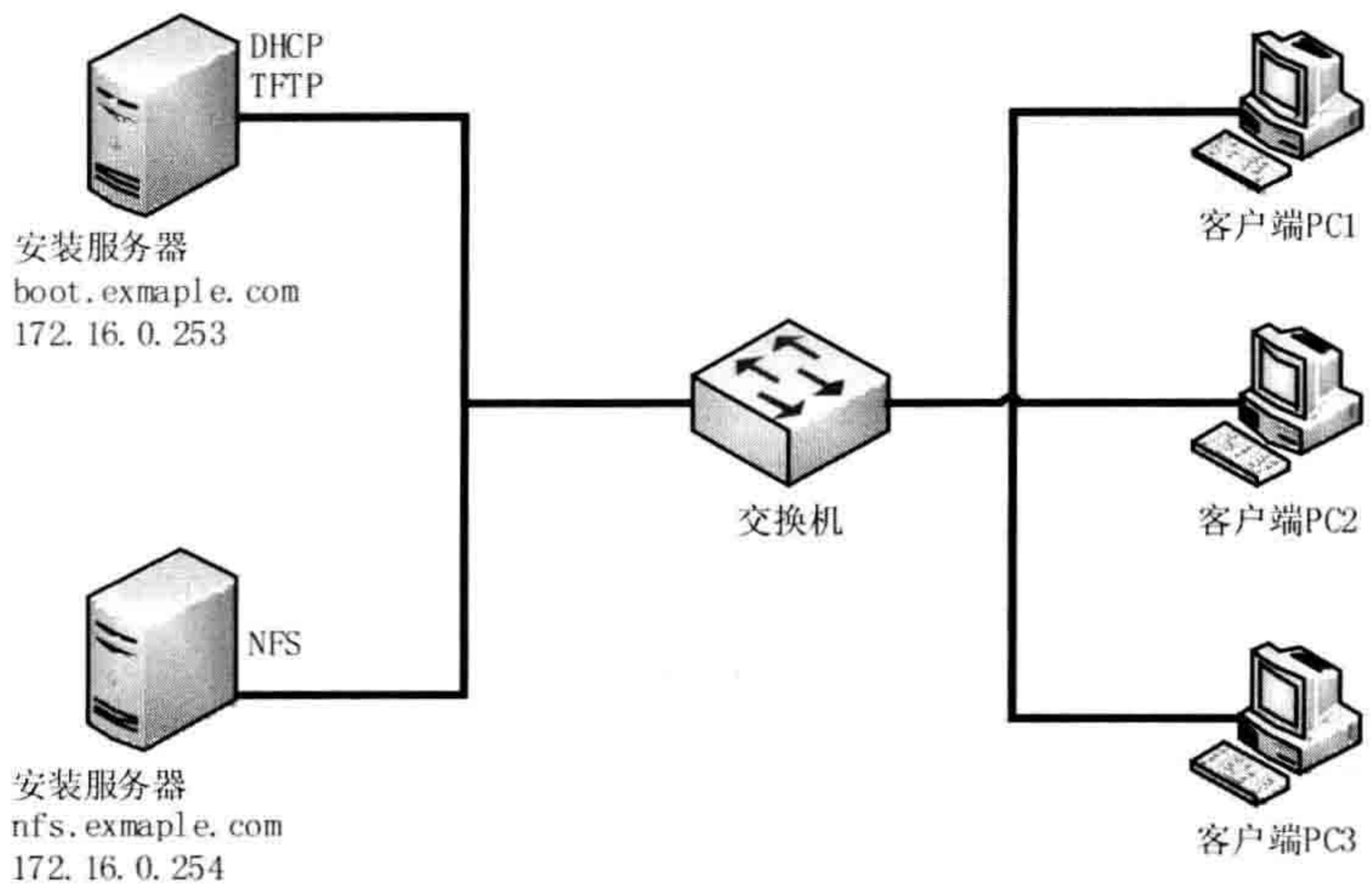


图 1-26

具体实现步骤如下。

（1）安装部署 DHCP 服务器。

```
[root@boot ~]# yum -y install dhcp
```

修改配置文件的内容如下：

```
[root@boot ~]# vim /etc/dhcp/dhcpd.conf
# dhcpd.conf
# Sample configuration file for ISC dhcpd
log-facility local7;
# A slightly different configuration for an internal subnet.
subnet 172.16.0.0 netmask 255.255.0.0 {
  range 172.16.0.100 172.16.0.200;
  option domain-name-servers 202.106.0.20;
  option routers 172.16.0.1;
  default-lease-time 600;
  max-lease-time 7200;
  next-server 172.16.0.253;
  filename "pxelinux.0";
}
```

配置说明：这里指定为 172.16.0.0/16 网络分配动态 IP 地址，动态地址池从 172.16.0.100 至 172.16.0.200，客户端获取的网关地址为 172.16.0.1，TFTP 地址为 172.16.0.253，启动文件名称为 pxelinux.0。

启动 DHCP 服务并设置为开机启动：

```
[root@boot ~]# service dhcpd start
[root@boot ~]# chkconfig dhcpd on
```

（2）安装部署 TFTP 服务器。

```
[root@boot ~]# yum -y install tftp-server
[root@boot ~]# vim /etc/xinetd.d/tftp                    #修改配置文件
service tftp
{
        socket_type             = dgram
        protocol                = udp
        wait                    = yes
        user                    = root
        server                  = /usr/sbin/in.tftpd
        server_args             = -s /var/lib/tftpboot
        disable                 = no
        per_source              = 11
        cps                     = 100 2
        flags                   = IPv4
}
```

配置说明：共享路径为/var/lib/tftpboot。

（3）将客户端所需启动文件复制到 TFTP 服务器。

```
[root@boot ~]# yum -y install syslinux
[root@boot ~]# cp /usr/share/syslinux/pxelinux.0 /var/lib/tftpboot/
```

将 CentOS 6.3 光盘放入光驱并运行如下命令，从光盘中复制启动镜像文件和启动配置文件至 TFTP 共享目录。

```
[root@boot ~]# umount /dev/cdrom
[root@boot ~]# mount /dev/cdrom /media
[root@boot ~]# cp /media/isolinux/{vmlinuz,initrd.img} /var/lib/tftpboot/
[root@boot ~]# mkdir /var/lib/tftpboot/pxelinux.cfg
[root@boot ~]# cp /media/isolinux/isolinux.cfg\
>/var/lib/tftpboot/pxelinux.cfg/default
[root@boot ~]# chmod 644 /var/lib/tftpboot/pxelinux.cfg/default
```

修改启动配置文件如下：

```
[root@boot ~]# vim /var/lib/tftpboot/pxelinux.cfg/default
default linux
timeout 600
display boot.msg
menu title Welcome to CentOS 6.3!
label linux
  menu label ^Install or upgrade an existing system
```

```
  menu default
  kernel vmlinuz
  append initrd=initrd.img ks=nfs:172.16.0.254:/ks/ks.cfg
label rescue
  menu label ^Rescue installed system
  kernel vmlinuz
  append initrd=initrd.img rescue
```

配置说明：default 指定默认引导方式，从配置文件可以看出，有一种以 label 开始的名称为 Linux 的正常引导方式，另一种是名称为 rescue 的救援模式引导方式，而 default 决定了默认采用哪种引导方式引导安装程序。此外，kernel 指定的是系统内核文件，在上面的步骤中，我们已经从光盘将其复制到 TFTP 共享目录下。如果没有后面的 ks 参数，至此，我们可以实现无光盘启动，后面的安装步骤需要我们手动进行，设置 ks 参数可以指定自动应答文件的位置，从而实现无人职守自动安装部署操作系统。本例设置为通过访问 172.16.0.254 的 NFS 共享方式读取 Kickstart 文件。

重启 TFTP 服务并设置为开机启动：

```
[root@boot ~]# service xinetd restart
[root@boot ~]# chkconfig xinetd on
```

关闭防火墙防止客户端无法访问服务器：

```
[root@boot ~]# service iptables stop
```

（4）创建 Kickstart 自动应答文件。

在 172.16.0.254 主机上安装图形工具软件 system-config-kickstart。

```
[root@nfs ~]# yum -y install system-config-kickstart
[root@nfs ~]# mkdir /ks/
```

运行 system-config-kickstart，设置效果如图 1-27～图 1-38 所示。

```
[root@nfs ~]# system-config-kickstart
```

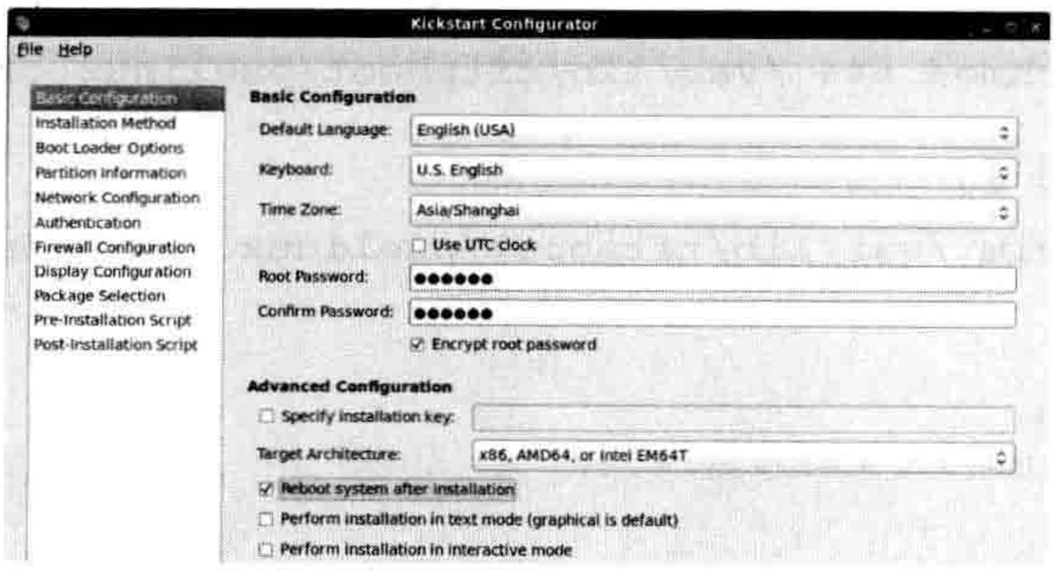


图 1-27

图 1-27 的配置说明：配置语言为英文，键盘设置为美式键盘，时区为亚洲/上海，root 管理员密码根据自己的实际情况设置。

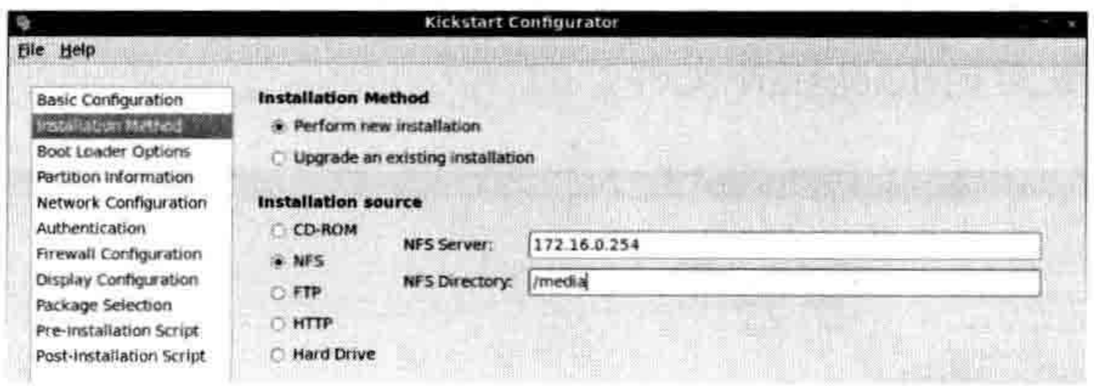


图 1-28

图 1-28 的配置说明：配置安装方式可以选择为全新安装或升级安装，本例选择全新安装，设置安装过程中所需安装文件（CentOS 6.3 系统光盘里的内容）通过 NFS 共享读取，这里的 NFS 服务器为 172.16.0.254，共享目录为/media。

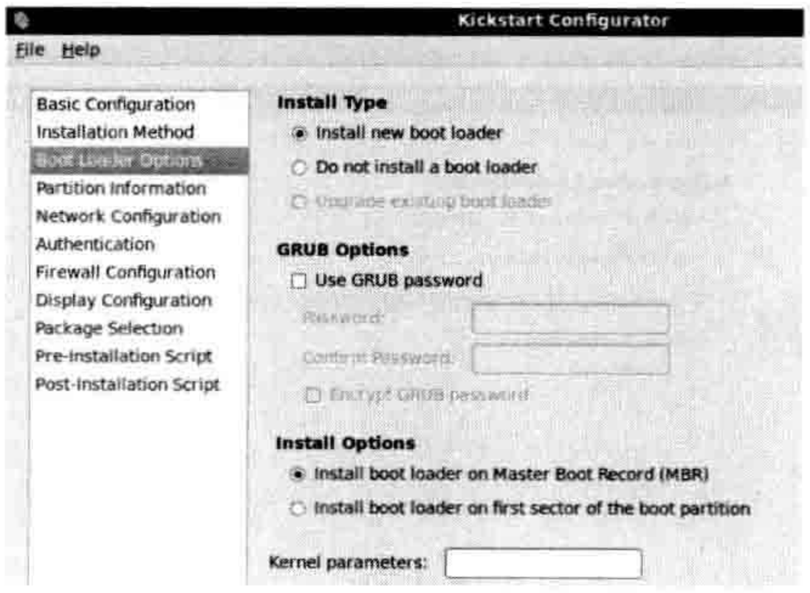


图 1-29

图 1-29 的配置说明：此处配置系统引导程序，安装方式选择全新安装 GRUB 引导程序，GRUB 引导密码未设置，将系统引导程序安装于 MBR 区域，内核参数未设置任何特殊参数。

图 1-30

图 1-30 的配置说明：分区选择移除所有的现有分区后重新分区，并初始化磁盘标签。

注意 这里通过图形生成的 Kickstart 文件无法使用 LVM 逻辑卷分区，如果需要使用 LVM 逻辑卷分区，则要手动修改生成后的 Kickstart 文件。

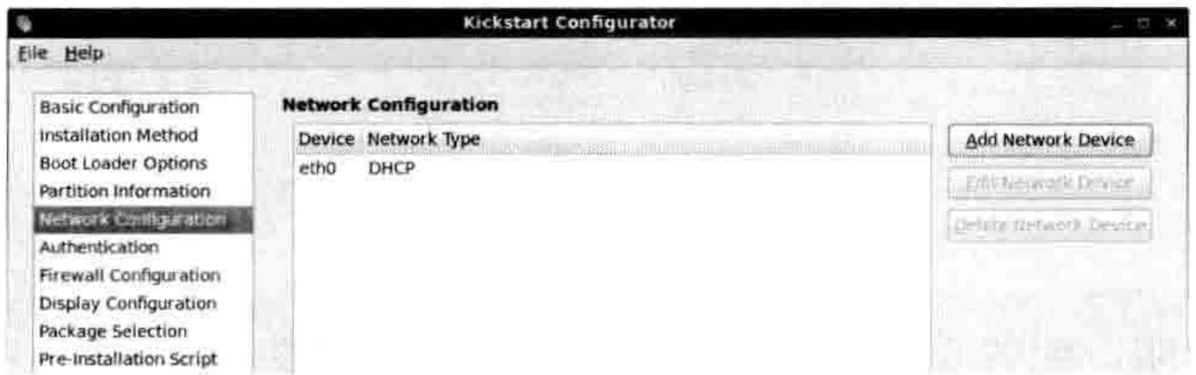


图 1-31

图 1-31 的配置说明：单击 Add Network Device 按钮添加网络设备 eth0，并设置为通过 DHCP 获得网络参数。

图 1-32

图 1-32 的配置说明：这里是认证设置，所有的用户密码都使用 SHA512 算法加密。

提示 如果需要将本机加入活动目录的话，可以修改 LDAP 参数加入活动目录。

图 1-33

图 1-33 的配置说明：这里的 SELinux 与防火墙均被设置为激活状态，并设置防火墙开放 HTTP 与 SSH 服务。

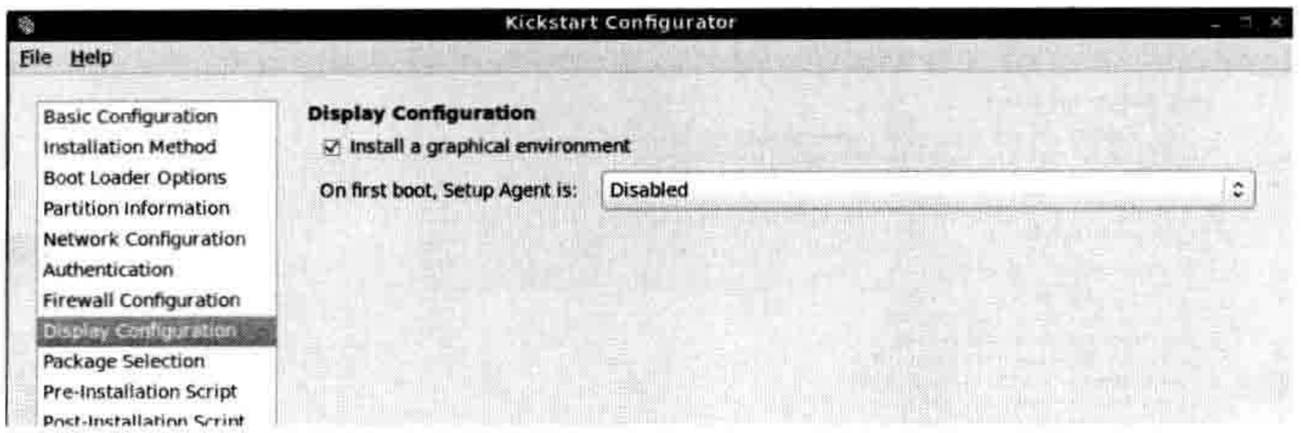


图 1-34

图 1-34 的配置说明：显示配置项目，这里勾选安装图形环境。

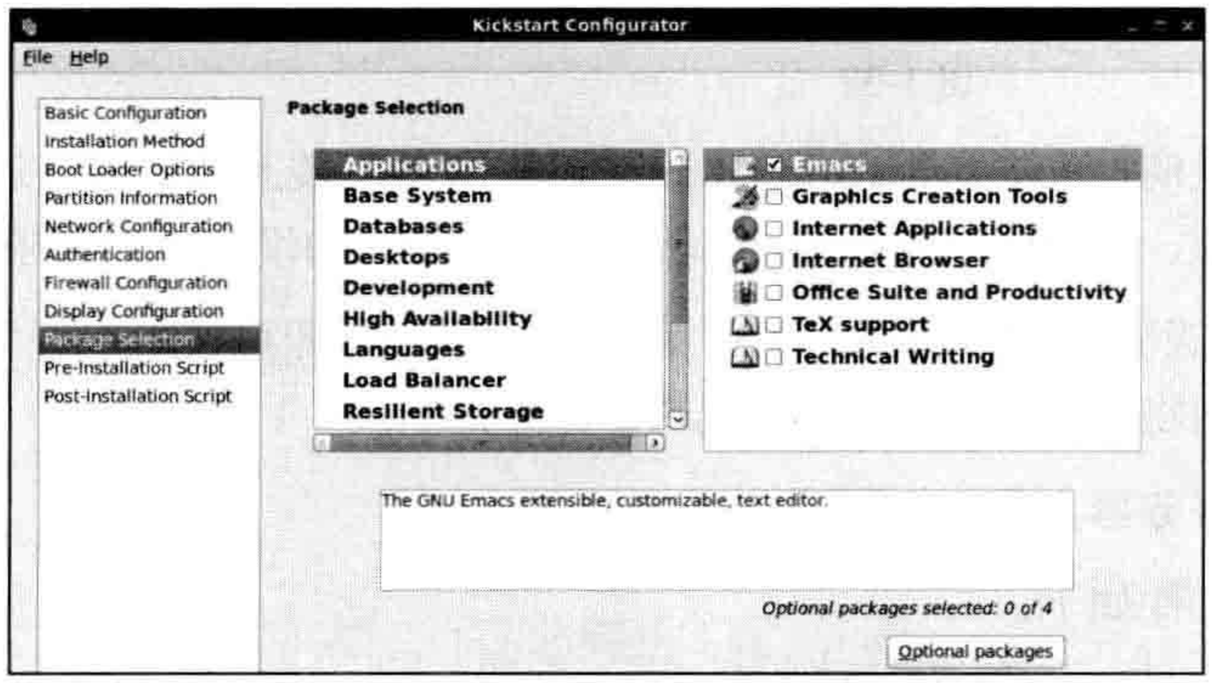


图 1-35

图 1-35 的配置说明：安装软件时需要根据自己的实际需要有选择地安装，每个软件包组里都有很多丰富的软件，可以通过 Optional packages 查看详细信息。若不知道该如何选择，也可以全选。

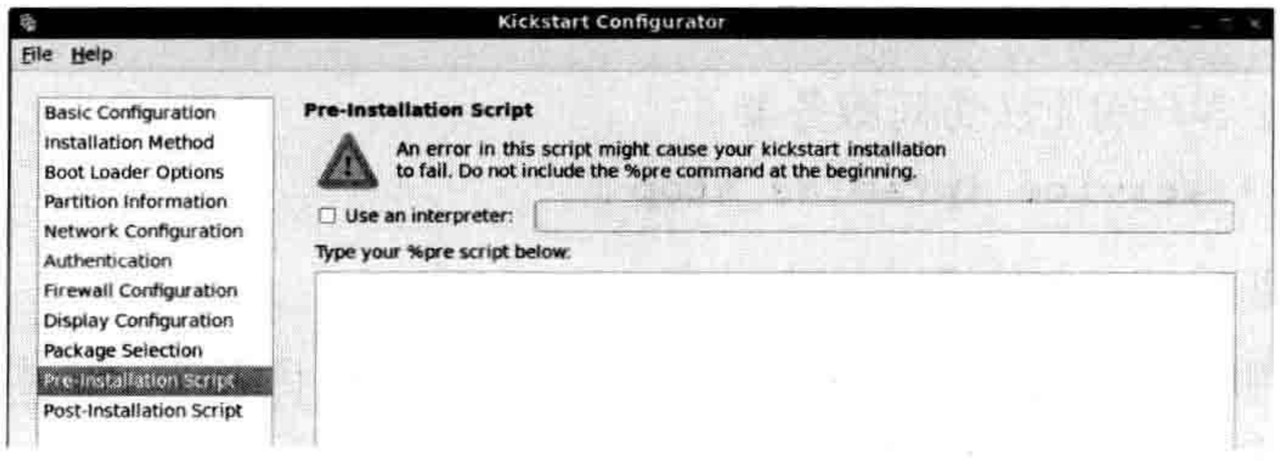


图 1-36

图 1-36 的配置说明：Pre-Installation Script 可以写入在安装系统前需要运行的脚本，若没有特殊需要，该脚本可以为空。

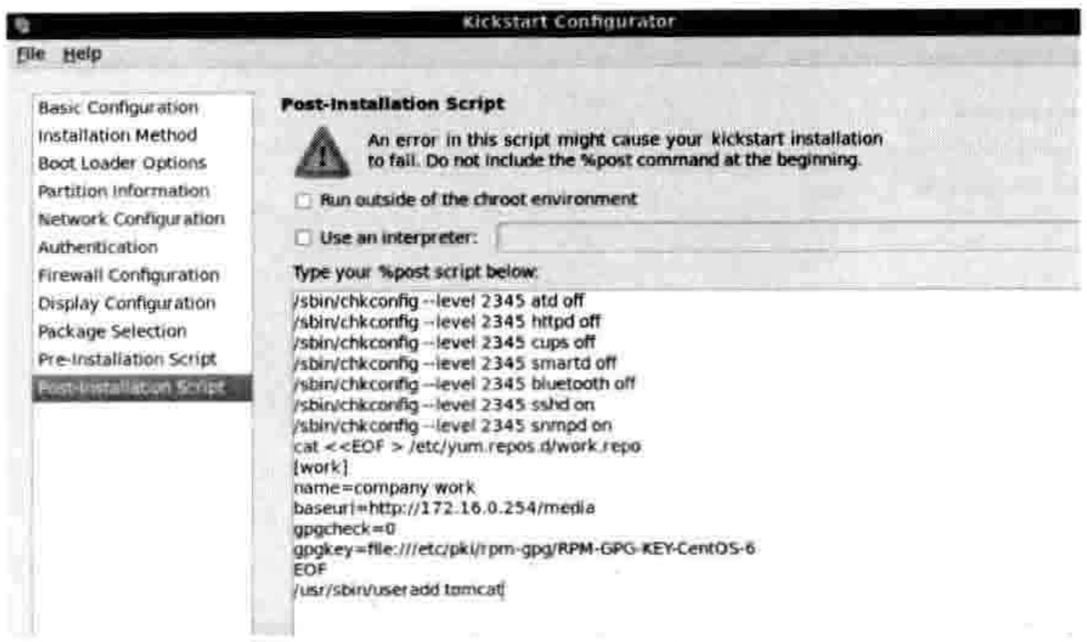


图 1-37

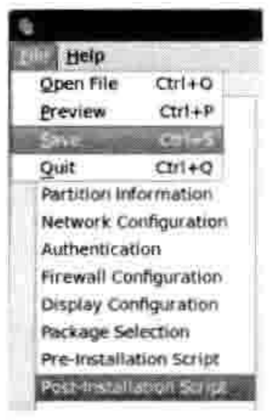


图 1-38

图 1-37 的配置说明：Post-Installation Script 可以写入安装系统后需要运行的部署脚本，常用脚本为创建 Yum 源、初始化用户账号、部署软件服务等，如没有特殊需求，该脚本可以为空。

所有的配置选项设置完成后，通过 File 菜单中的 Save，即可实现保存生成 Kickstart 文件，如图 1-38 所示。本例将其保存至/ks 目录下，取名为 ks.cfg。

（5）部署 NFS 服务器。

修改 NFS 配置文件如下：

```
[root@nfs ~]# vim /etc/exports
/ks     172.16.0.0/255.255.0.0(ro)
/media  172.16.0.0/255.255.0.0(ro)
```

启动 NFS 服务并设置为开机启动：

```
[root@nfs ~]# service nfs start
[root@nfs ~]# chkconfig nfs on
```

关闭防火墙防止客户端无法访问服务器：

```
[root@nfs ~]# service iptables stop
```

（6）将系统光盘或 ISO 文件通过 NFS 共享。

部署完成 NFS 服务后，需要将光盘中的文件复制到/media 目录中，如果有光盘与光驱，可以通过 mount 的方式将光盘直接挂载至/media 目录下，方法如下。

首先，将光盘从默认的挂载点卸载：

```
[root@nfs ~]# umount /dev/cdrom
```

其次，将光盘重新挂载至/media 目录下：

```
[root@nfs ~]# mount /dev/cdrom /media
```

如果没有光盘，仅有 ISO 镜像文件，也可以将 ISO 镜像文件挂载至/media 目录，方法如下：

```
[root@nfs ~]# mount -o loop -t iso9660 镜像文件 /media
```

（7）启动客户端，安装部署系统。

在所有客户端主机的 BIOS 中，将第一启动方式设置为 PXE 网络启动，或通过类似于开机按 F12 键这样的快捷方式设置启动方式。注意，由于不同型号的主机设置网络启动的方式不同，用户可根据计算机的说明书设置。设置完成后，重启所有的客户端计算机即可完成大规模集中安装部署操作系统。

1.3　常见问题分析

1. 无法从 RAID 卡启动

如果执行了安装程序但无法正常启动系统，则可能需要重新创建分区。

因为有些 BIOS 启动模式并不支持从 RAID 卡启动，在安装结束后，会有一个字符界面的引导提示符（如 GRUB:），并伴随着闪烁的光标，这时就需要重新为系统分区。

无论是手动安装还是自动安装，/boot 分区都不要使用 RAID 阵列创建，可以使用单独的一个磁盘或分区。

2. 系统提示 Signal 11 错误

这一般被认为是段错误，表示程序访问了未分配给它的内存空间，这个错误可能是软件 Bug 或硬件错误。

3. 图形安装错误

有些显示卡无法从启动图形安装程序，如果安装程序无法运行默认设置，它将自动以低分辨率模式进行安装，如果这样依然失败，安装程序将试图以字符界面安装系统。

可以在启动菜单中选择 Install system with basic video drive，或使用 xdriver=vesa 引导选项手动解决该问题。

另外，也可以通过 resolution=选项强制修改分辨率。

4. 安装过程中提示找不磁盘（No devices found to install Red Hat Enterprise Linux）

如果出现该提示信息，可能是由于 SCSI 控制器未被识别引起的，请检查硬件是否在 CentOS 所支持的硬件列表中。

5. 分区表错误

如果在磁盘分区设置完成后提示：

```
The partition table on device hda was unreadable. To create new partitions
it must be initialized, causing the loss of ALL DATA on this drive.
```

这说明磁盘没有分区表或分区表无法识别，出现这种情况时，首先要备份数据，然后尝试修复分区表。

6. 其他分区问题

如果手动进行分区，并且无法进行下一步操作，可能是由于你没有创建系统所需要的所有分区。

通常情况下，至少要有两个分区：/(根分区)和 swap(交换分区)。

7. 图形环境问题

如果安装了 X Window 系统，但无法进入图形环境，可以尝试在命令行输入 startx 命令进入图形。

运行 startx 后，图形界面也开启了。注意，这仅是临时的修复方式，如果需要永久生效，需要编辑/etc/inittab 文件，内容如下。

```
[root@localhost ~]#vim  /etc/inittab
# Default runlevel. The runlevels used are:
# 0 - halt (Do NOT set initdefault to this)
# 1 - Single user mode
# 2 - Multiuser, without NFS (The same as 3, if you do not have networking)
# 3 - Full multiuser mode
# 4 - unused
# 5 - X11
# 6 - reboot (Do NOT set initdefault to this)
#
id:3:initdefault:
```

要改变从图形登录，需要修改 id:3:initdefault:行，将其中的 3 改为 5。

8. 无法开启图形

在上一操作过程中，若 startx 未能将图形开启，则可能是未安装 X Window。

可以通过 CentOS 6.3 安装光盘执行升级操作安装该软件，需要选择 X Window 软件包，并选择 GNOME 与 KDE 软件两者之一或全部。

9. 图形登录问题

如果已经开启了图形，但所有的用户都无法登录进系统，则可能是磁盘已经满。

在字符界面执行 df　-h 命令可以查看磁盘使用情况。注意，/home 和/tmp 可能会被用户很快用完。

10. 忘记密码

如果忘记了 root 用户的密码，就需要进入单用户模式。

在出现 GRUB 引导程序时，按 e 键可以编辑 GRUB 引导参数，选择以 kernel 开始的行按 e 键进行编辑，在 kernel 行末尾加入 single 后按回车键，按 b 键启动系统并进入单用户模式。

在单用户模式中，你可以通过输入如下命令修改 root 密码：

```
passwd  root                    #输入两次密码后完成
shutdown -r now                 #重启系统
```

第2章

命令工具

Linux 操作系统拥有字符与图像两种工作界面，在企业生产环境中，Linux 主要担当服务器的角色，而图形界面会占用大量的系统资源。因此，从运行效率以及资源占用率的角度考虑，通常会使用命令行完成日常工作。下面会逐步介绍 Linux 中常用命令的用法与应用情况。

使用命令行的方法如下。

- 开机直接进入字符界面，如图 2-1 所示。

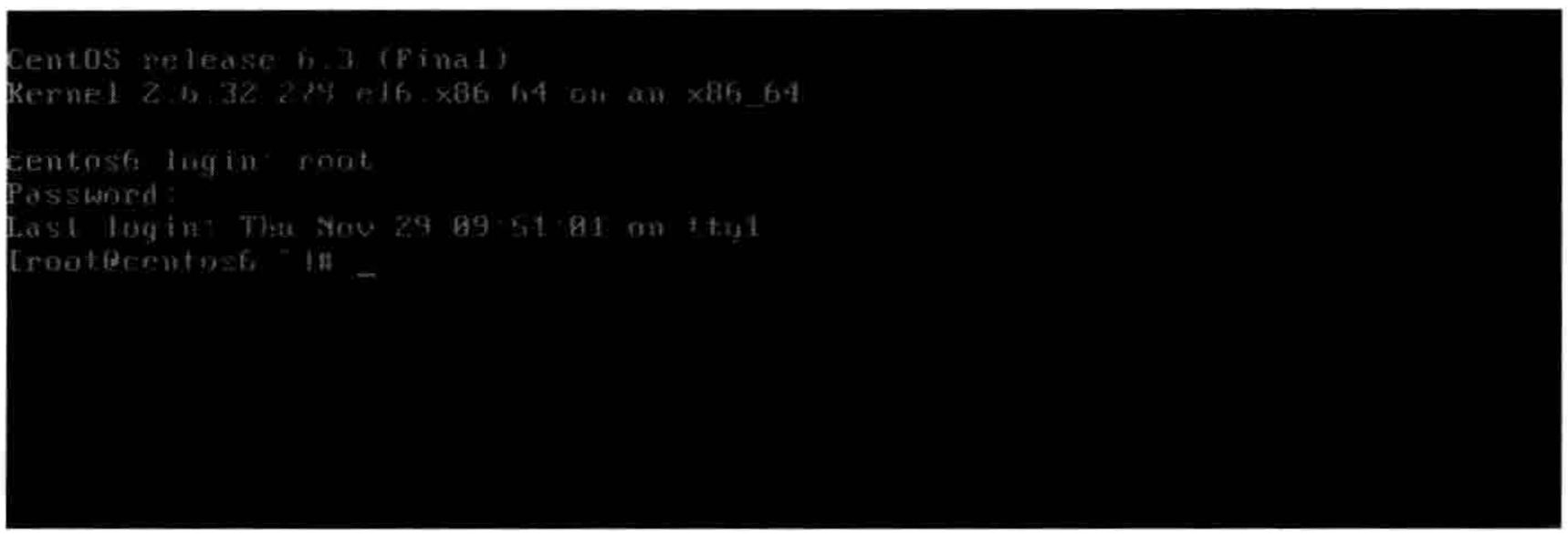


图 2-1

- 图形界面中开启超级终端，选择 Applications→System Tools→Terminal，如图 2-2 所示。

图 2-2

2.1　基本命令

2.1.1　目录及文件的基本操作

1. pwd

描述：pwd 命令的作用是显示当前工作目录的名称。

用法：pwd [选项]...

选项：-P 显示链接的真实路径。

```
[root@centos6 Desktop]# pwd          #返回当前工作目录/root/Desktop
/root/Desktop
[root@centos6 test]# pwd             #返回当前工作目录/tmp/test
/tmp/test
[root@centos6 test]# pwd -P          #返回链接的真实路径/tmp/pass[1]
/tmp/pass
```

2. cd

描述：cd 命令的作用是切换当前工作目录。

[1] /tmp/test 是/tmp/pass 的链接文件。

```
[root@centos6 ~]# cd /usr/src/          #切换工作目录至/usr/src/
[root@centos6 src]# cd ..               #切换工作目录至当前目录的上一级目录
[root@centos6 usr]# cd -                #返回至/usr/src 目录
[root@centos6 ~]# cd                    #切换工作目录至当前用户的家目录
```

3. ls

描述：命令的作用是显示目录与文件信息。

用法：ls [选项]... [文件/目录]...

选项：-a 显示所有的信息，包括隐藏文件与目录。

-d 显示目录本身的信息，而非目录下的资料信息。

-h 人性化显示容量信息。

-l 长格式显示详细信息。

-c 显示文件或目录属性最后修改的时间。

-u 显示文件或目录最后被访问的时间。

-t 以修改时间排序，默认按文件名称排序。

```
[root@centos6 test]# ls                 #显示当前目录下的子文件与目录名称
[root@centos6 test]# ls /etc            #显示/etc 目录下的子文件与目录名称
[root@centos6 test]# ls -a              #查看以.开始的隐藏文件与目录
[root@centos6 test]# ls -l              #查看文件与目录的详细信息[2]
[root@centos6 test]# ls -ld /root       #查看当前 root 目录自身的详细信息
[root@centos6 test]# ls -lh             #人性化显示容量信息
[root@centos6 test]# ls -lc /etc/passwd #查看/etc/passwd 文件的属性变更时间
[root@centos6 test]# ls -lu /etc/passwd #查看/etc/passwd 最后访问时间
[root@centos6 test]# ls -lt             #查看档案信息并以修改时间排序
```

4. touch

描述：创建或修改文件时间。

```
[root@centos6 test]# touch hello.txt
```

如果 hello.txt 不存在，则创建，如果已存在，则更新文件所有的时间为当前系统时间。

5. mkdir

描述：创建目录。

---

[2] 默认显示的时间为文件被修改的时间，容量单位为字节。

用法：mkdir [选项]... [目录]...
选项：-p 创建多级目录。

```
[root@centos6 test]# mkdir leo
[root@centos6 test]# mkdir -p /tmp/test/jerry/book/computer
```

6. cp

描述：复制文件与目录。
用法：cp [选项] 源 目标
选项：-r 递归，复制子文件与子目录，一般复制目录时使用。

```
[root@centos6 test]# cp /etc/hosts /tmp/          #复制文件/etc/hosts至/tmp目录下
[root@centos6 test]# cp /etc/hosts /tmp/host      #复制文件/etc/hosts至/目录下并
                                                    改名为host
[root@centos6 test]# cp -r /var/log/ /tmp/        #复制目录/var/log至/tmp/目录下
```

7. rm

描述：删除文件或目录。
用法：mv [选项]... 文件...
选项：-f 不提示，强制删除。
　　-i 删除前，提示是否删除。
　　-r 删除目录以及目录下的所有内容。

```
[root@centos6 test]# rm readme.txt          #删除文件readme.txt
[root@centos6 test]# rm -rf Jacob           #删除目录且不提示
```

8. mv

描述：移动（重命名）文件或目录。

```
[root@centos6 test]# mv hello.txt hello.doc   #将hello.txt改名为hello.doc
[root@centos6 test]# mv hello.doc /root/      #将hello.doc移至/root目录下
```

9. find

描述：搜索文件或目录。
用法：find [命令选项] [路径] [表达式选项]
选项：-empty 查找空白文件或目录。
　　-group 按组查找。

-name 按名称查找。
-iname 按名称查找，不区分大小写。
-mtime 按修改时间查找。
-size 按容量大小查找。
-type 按档案类型查找，文件（f）、目录（d）、设备（b，c）、链接（l）等。
-user 按用户查找。
-exec 对找到的档案执行命令。
-a 并且。
-o 或者。

```
[root@centos6 test]# find -name hello.doc                    #查找当前目录下名为 hello.doc 的档案[3]
[root@centos6 test]# find /root-name "*.log"                 #查找/root 目录下所有名称以.log 结尾的档案
[root@centos6 test]# find -iname "Jacob"                     #不区分大小写查找档案 jacob
[root@centos6 test]# find / -empty                           #查找计算机中所有的空文档
[root@centos6 test]# find / -group tom                       #查找计算机中所属组为 tom 的档案
[root@centos6 test]# find / -mtime -3                        #查找计算机中所有 3 天内被修改过的档案
[root@centos6 test]# find / -mtime +4                        #查找计算机中所有 4 天前被修改过的档案
[root@centos6 test]# find / -mtime 2                         #查找计算机中 2 天前的当天被修改过的档案
[root@centos6 test]# find ./ -size+10M                       #查找当前目录下大于 10MB 的档案
[root@centos6 test]# find ./ -type f                         #查找当前目录下的所有普通文件
[root@centos6 test]# find / -user tom                        #查找计算中 tom 所拥有的所有档案
[root@centos6 test]# find./ -size +1M -exec ls -l {} \;      #查找大于 1MB 的文件后列出文件的详细信息
[root@centos6 test]# find / -size +1M -a -type f             #查找计算机中所有大于 1MB 的文件
```

10. du

描述：计算文件或目录的容量。
用法：du [选项]... [文件或目录]...
选项：-h 人性化显示容量信息。
-s 仅显示总容量。

```
[root@centos6 test]# du /root                                #查看/root 目录及子目录的容量信息
```

[3] 这里的档案指的是文件或目录。

```
[root@centos6 test]# du -sh/root                    #查看/root 所占磁盘空间总和
```

2.1.2　查看文件内容

1. cat

描述：查看文件内容。

用法：cat [选项]… [文件]…

选项：-b 显示行号，空白行不显示行号。

　　-n 显示行号，包括空白行。

```
[root@centos6 test]# cat /root/install.log
[root@centos6 test]# cat -n  /etc/passwd
[root@centos6 test]# cat -b  /etc/passwd
```

2. more

描述：分页查看文件内容，通过空格键查看下一页，q 键则退出查看。

```
[root@centos6 test]# more /root/install.log
```

3. less

描述：分页查看文件内容，空格（下一页）、方向键（上下回翻）、q 键（退出查看）。

```
[root@centos6 test]# less /root/install.log
```

4. head

描述：查看文件头部内容，默认显示前 10 行。

用法：head [选项]… [文件]…

选项：-c nK　显示文件前 nKB 的内容。

　　-n　　显示文件前 n 行的内容。

```
[root@centos6 test]# head -c 2K /root/install.log  #查看文件的前 2KB 的内容
[root@centos6 test]# head -20   /root/install.log  #查看文件的前 20 行的内容
```

5. tail

描述：查看文件的尾部内容，默认显示末尾 10 行。

用法：tail [选项]… [文件]…

选项：-c nK　显示文件末尾 nKB 的内容。
　　-n　　显示文件末尾 n 行的内容。
　　-f　　动态显示文件内容，按 Ctrl+C 组合键退出。

```
[root@centos6 test]# tail -c 2K /root/install.log  #查看文件末尾 2KB 的内容
[root@centos6 test]# tail -20 /root/install        #查看文件末尾 20 行的内容
[root@centos6 test]# tail -f /var/log/messages     #实时动态查看文件内容
```

6. wc

描述：显示文件的行、单词与字节统计信息。
用法：wc [选项]... [文件]...
选项：-c　显示文件字节统计信息。
　　-l　显示文件行数统计信息。
　　-w　显示文件单词统计信息。

```
[root@centos6 test]# wc /root/install.log          #依次显示文件的行数、单词数、字节数
[root@centos6 test]# wc -c /root/install.log       #显示文件的字节信息
[root@centos6 test]# wc -l /root/install.log       #显示文件行数
[root@centos6 test]# wc -w /root/install.log       #显示文件的单词个数
```

7. grep

描述：查找关键词并打印匹配的行。
用法：grep [选项] 匹配模式 [文件]...
选项：-i　忽略大小写。
　　-v　取反匹配。
　　-w　匹配单词。
　　--color　显示颜色。

```
[root@centos6 test]# grep th test.txt              #在 test.txt 文件中过滤出包含 th 的行
[root@centos6 test]# grep --color th test.txt      #对匹配的关键词显示颜色
[root@centos6 test]# grep -i the test.txt          #过滤包含 the 的行（不区分大小写）
[root@centos6 test]# grep -w num test.txt          #过滤单词 num（不过滤 number）
[root@centos6 test]# grep -v the test.txt          #过滤不包含 the 关键词的行
```

8. 基本命令：echo

描述：显示一行指定的文本。

用法：echo [选项]... [字符串]...

选项：-n 不输出换行，默认 echo 输出内容后会换行。

　　-e 支持反斜线开始的转义字符，屏蔽反斜线后面字符的原本含义。

如果使用-e 选项，则可以识别如下字符序列的特殊含义。

\\　反斜线。

\a　报警器。

\b　退格键。

\c　不生成格外输出，默认 echo 会自动添加换行。

\f　输入表单格式，换行后保留光标位置。

\n　换行。

\t　生成水平 Tab。

\v　生成垂直 Tab。

实例：

```
[root@centos6 ~]# echo "Hello The Wrod"          #直接输出指定的字串
Hello The Wrod
[root@centos6 ~]# echo -e "\\"                   #默认 echo 无法输出\符号
\
[root@centos6 ~]# echo -e "\a"                   #计算机蜂鸣器会响一声

[root@centos6 ~]# echo -e "11\b22"               #回删一个字符
122
[root@centos6 ~]# echo -e "hello\c"              #不换行，等同于-n 选项
hello[root@centos6 ~]#
[root@centos6 ~]# echo -e "hello\fthe wrold"     #表单格式
hello
the wrold
[root@centos6 ~]# echo -e "hello\tthe\tworld"    #水平 Tab 键
hello    the    world
[root@centos6 ~]# echo -e "hello\vthe\vworld"    #垂直 Tab 键
hello
  the
      world
```

2.1.3　链接文件

Linux 中的链接文件不同于 Windows 的快捷方式，Linux 的链接文件分为软链接与硬链接，软链接可以跨分区，但源文件不可删除，硬链接不可以跨分区，但可以将源文件删除。

1. 软链接

```
[root@centos6 test]# ln -s /test/hello.txt /tmp/hi.txt   #创建文件软链接
[root@centos6 test]# ln -s /test/  /var/test             #创建目录软链接
```

2. 硬链接

```
[root@centos6 test]# ln /test/hello.txt /test/hi.txt
[root@centos6 test]# rm /test/hello.txt      #删除源文件后，链接文件仍可以正常使用
```

2.1.4 压缩及解压

1. gzip

描述：压缩与解压缩。

用法：gzip [选项]... [文件名称]...

选项：-d 解压。

```
[root@centos6 test]# gzip hello.txt          #文件压缩后名为 hello.txt.gz
[root@centos6 test]# gzip -d hello.txt.gz    #解压 gz 文件
```

2. bzip2

描述：压缩与解压缩。

```
[root@centos6 test]# bzip2 hello.txt          #文件压缩后名为 hello.txt.bz2
[root@centos6 test]# bzip2 -d hello.txt.bz2   #解压 gz 文件
```

提示

gzip 与 bzip2 工具不可以对目录做打包压缩操作。

3. tar

描述：打包与解包文件。

用法：tar 模式 [选项] [路径]...

模式：-c　　创建打包文件。

　　--delete　从打包文件中删除文件。

　　-r　　追击文件至打包文档。

　　-t　　列出打包文档的内容。

　　-x　　释放打包文件。

选项：-C　　指定解压路径。
　　-f　　指定打包后的文件名称。
　　-j　　打包后通过 bzip2 格式压缩。
　　--remove-files　打包后删除源文件。
　　-z　　打包后通过 gzip 格式压缩。

```
[root@centos6 test]# tar -cf etc.tar /etc/
                                        #将/etc/目录打包保存为 etc.tar
[root@centos6 test]# tar -czf boot.tar.gz /boot/
                                        #将/boot 目录打包并压缩为 etc.tar.gz
[root@centos6 test]# tar -cjf etc.tar.bz2 /tmp/
                                        #将/tmp/目录打包并压缩为 etc.tar.bz2
[root@centos6 test]# tar --delete etc/hosts -f etc.tar
                                        #从打包文档中删除文件 hosts
[root@centos6 test]# tar -f etc.tar -r /root/install.log
                                        #追加文件至打包文档 etc.tar 中
[root@centos6 test]# tar -tf boot.tar.gz
                                        #查看打包文档中档案的信息
[root@centos6 test]# tar-tvf etc.tar    #查看打包文档中档案的详细信息
[root@centos6 test]# tar -xzf boot.tar.gz
                                        #解压 gz 格式的打包文档至当前目录
[root@centos6 test]# tar -xjf etc.tar.bz2
                                        #解压 bz2 格式的打包文档至当前目录
[root@centos6 test]# tar -xzf boot.tar.gz -C /tmp
                                        #指定解压路径为/tmp
[root@centos6 test]# tar -czf mess.tar.gz /var/log/messages --remove-files
                                        #打包压缩后删除源文件
```

2.1.5　命令使用技巧

1. 善于利用 Tab 键

在 Linux 中，利用 Tab 键可以自动补齐命令或路径，从而提高工作效率。通过键盘输入 bzi 后按下 Tab 键，即可补齐以 bzi 开头的命令。当命令不唯一时，如通过键盘输入 c 后按下 Tab 键，则屏幕不会进行命令补齐，因为以 c 开头的命令不止一个，此时连续按下两次 Tab 键，即可显示所有以 c 开头的命令。

2. 使用命令历史

在 Linux 中输入的命令会被记录，对于已经输入过的命令，没有必要重复输入，这时可以直接调用历史命令记录。使用命令历史最简单的方法是通过上、下方向键翻阅历史命令，CentOS

默认会记录 1000 条命令历史。输入 history 命令可以显示所有的命令记录，每条记录都有相应的编号，如果你想执行编号为 500 的命令历史，可以通过!500 来调用该命令。

3. 适时清屏

当命令输入特别多或屏幕显示特别乱时，可以通过快捷键 Ctrl+l 或输入命令 clear 清屏。

4. 查找常用命令存储位置

通过 which 命令可以找到常用命令的存储位置，如输入 which find，系统将返回 find 命令的实际存储位置/bin/find。

2.1.6 帮助

1. man

通过 man(manual)手册文档可以帮助我们快速掌握命令的用法，man 手册一般保存在/usr/share/man 目录下，查看手册文档可以直接通过 man 命令读取。

例如，如果想获取 ls 命令的 man 手册，输入 man ls 即可，显示结果大致如下：

```
LS(1)                          User Commands                          LS(1)
NAME
       ls - list directory contents
SYNOPSIS
       ls [OPTION]... [FILE]...
DESCRIPTION
       List information about the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort.
       Mandatory arguments to long options are mandatory for short  options
       too.
       -a, --all
              do not ignore entries starting with .
       -A, --almost-all
              do not list implied . and ..
……此处省略……
```

其中，NAME 为命令的名称与简单描述，SYNOPSIS 为命令的语法格式，DESCRIPTION 为命令的详细描述，后面一般为命令的具体选项以及功能描述。

通过<man 命令名>这种方法可以找到绝大多数命令的用法与描述，空格键表示向下翻页，q 键表示退出 man 手册。另外，在查看命令手册的过程中，随时可以通过</关键词>来搜索你需要的内容（不包含尖括号），如/file 查看包含 file 的行，按键盘上的 n 键查看下一行。

提示

如果查找显示 Pattern not found (press RETURN)，则表示未找到匹配的行。

2. info

info 信息与 man 手册的内容类似，但 info 信息是模块化的，它通过链接显示不同的信息块，查看起来有点类似于网页。

Info ls 查看 ls 命令的 info 信息，内容大致如下：

```
    File: coreutils.info,  Node: ls invocation,  Next: dir invocation,  Up:
Directory 1\
    isting

    10.1 `ls': List directory contents
    ===================================

    The `ls' program lists information about files (of any type, including
    directories).  Options and file arguments can be intermixed
    arbitrarily, as usual.
```

其中，File 说明当前的 info 文件名称为 coreutils.info，当前查看的信息块为 ls invocation，N 键进入下一信息块（dir invocation），P 键进入上一信息块，U 键返回上一层（一般用来查看 info 信息块目录），空格键翻页，q 键退出。

3. help

man 手册与 info 信息的内容往往比较多，如果你没有精力细看这些资料，而仅仅需要的是简短的帮助信息，可以通过--help 来获得简要信息。

例如，ls --help 显示信息如下：

```
    Usage: ls [OPTION]... [FILE]...
    List information about the FILEs (the current directory by default).
    Sort entries alphabetically if none of -cftuvSUX nor --sort.

    Mandatory arguments to long options are mandatory for short options too.
      -a, --all                  do not ignore entries starting with .
      -A, --almost-all           do not list implied . and ..
          --author               with -l, print the author of each file
      -b, --escape               print octal escapes for nongraphic characters
```

其中，Usage 为命令的语法格式，紧接着是说明命令的功能，最后是对每个命令选项的简短说明。

2.2 Vim 文档编辑

Vim 是由 vi 发展演变过来的文本编辑器，因其具有语法高亮显示、多视窗编辑、代码折叠、支持插件等功能，现已成为众多 Linux 发行版本的标配。对于初学者来说，Vim 往往是生涩、难以学习的文本编辑器，但当你完全掌握了这种编辑器后，你会发现自己的工作效率会比没有使用 Vim 之前提升了很多倍。

2.2.1 Vim 工作模式

Vim 具有多种工作模式，常用的工作模式有：普通模式、插入模式、命令模式。其中，普通模式可以实现基本的光标移动与大量的快捷键操作，插入模式可实现文本的基本编辑功能，命令模式通过输入特定的指令实现特定的功能，如保存与退出等。

直接输入 vim 命令，即可开启该文本编辑器，默认将创建一个新的文档（保存时需要指定文件名）。另外，如果 vim 命令后跟了文件名参数，则需要看该文件是否存在，如果存在，Vim 将打开该文件，如果不存在，Vim 将创建该文件。

Vim 编辑器默认会进入普通模式，插入模式可以通过以下按键进入（参见表 2-1）。

表 2-1

按键	功能描述
a	进入插入模式，后续输入的内容将插入至当前光标的后面
A	进入插入模式，后续输入的内容将插入至当前段落的段尾
i	进入插入模式，后续输入的内容将插入至当前光标的前面
I	进入插入模式，后续输入的内容将插入至当前段落的段首
o	进入插入模式并在当前行的后面创建新的空白行
O	进入插入模式并在当前行的前面创建新的空白行

当需要退回到普通模式或不知道自己当前处于什么模式时，可以通过 Esc 键返回到普通模式。

最后，在普通模式下输入“:”，即可进入命令模式；在“:”后输入指令，即可实现特定的功能。

2.2.2　Vim 光标操作

Vim 中最简单的移动光标的方式是使用方向键（上、下、左、右）操作，但这种方式的效率低下，更高效的方式是使用快捷键，常用的快捷键如表 2-2 所示，所有的快捷键均在普通模式下直接使用。

表 2-2

快捷键	功能描述
h	光标向左移动一位
j	光标向下移动一行（以回车为换行符）
k	光标向上移动一行
l	光标向右移动一位
gg	移动光标至文件首行
G	移动光标至文件末尾
nG	移动光标至第 n 行（n 为数字，如 n 为 10 时表示第 10 行）
^	光标移至当前行的首字符
$	光标移至当前行的尾字符
fx	光标移至当前行的下一个 x 字符处
Fx	光标移至当前行的上一个 x 字符处
w	光标向右移动一个单词
nw	光标向右移动 n 个单词（n 为数字）
b	光标向左移动一个单词
nb	光标向左移动 n 个单词（n 为数字）

2.2.3　Vim 编辑文档

在 Vim 编辑器中编辑文档内容主要有两种常用的方式：进入编辑模式操作和快捷键操作。进入编辑模式的方法已经在表 2-1 中有详细的描述，进入编辑模式后，即可以通过移动光标进行增加、删除、修改的基本操作，这种方法也是最简单的方式。另外，快捷键操作方式是在普通模式下输入相应的快捷键实现对应的功能，快捷键功能描述参见表 2-3。

表 2-3

操作快捷键	功能描述
x	删除光标当前字符

续表

操作快捷键	功能描述
dd	删除一行
ndd	删除 n 行（n 为数字）
d$	删除光标至行尾的内容
J	删除换行符，可以将两行合并为一行
u	撤销上一步操作，可以多次使用 uu 表示撤销两步操作
rx	将光标当前字符替换为 x（x 为任何键盘单个输入）
yy	复制当前行
p	粘贴至当前行之后
P	粘贴至当前行之前

2.2.4 Vim 查找与替换

当文档很长时，我们可以通过查找快速定位要找的内容，在 Vim 中通过“/”关键词实现自上往下的查找功能，如，/host 在当前文档的光标处向下查找 host 并显示，如果一个文档中有多个 host，可以通过快捷键 n 跳转至下一个匹配的关键词处，快捷键 N 将跳转至上一个匹配的关键词处。另外，“?”关键词实现了自下往上的查找功能，如，?host 从当前文档的光标处向上查找 host 并显示，此时快捷键 n 表示查看上一匹配，N 表示查看下一匹配。

Vim 提供了非常好用的替换功能，可以快速完成大量的替换工作。

```
[root@centos6 etc]# cp /etc/passwd /root/
[root@centos6 etc]# vim /root/passwd
```

通过上面两条命令复制一份临时测试文档并编辑，我们可以对该文件实现多种替换功能，如表 2-4 所示（在普通模式下，输入冒号“:”进入命令模式完成替换命令）。

表 2-4

指令	功能描述
:s/root/admin/	将光标当前行中第一个出现的 root 替换为 admin，没有则不替换
:s/root/admin/g	将光标当前行中所有的 root 替换为 admin
:3,5 s/sbin/bin/g	将第三行至第五行之间的所有 sbin 替换为 bin
:% s/nologin/fault/g	将所有行的 nologin 都替换为 fault

2.2.5　Vim 保存与退出

一般情况下，我们会通过命令模式输入特定的指令实现保存与退出功能，常用指令详见表 2-5。

表 2-5

指令	功能描述
:q!	不保存并退出
:wq	保存并退出
:x	保存并退出
:w	保存
:w b.txt	另存为 b.txt

提示 Vim 提示错误信息 E32: No file name 时，说明你没有为文件设置文件名，需要在 w 后设置文件名称。

另一种保存方式为快捷键操作，在普通模式下输入 ZZ，即可实现保存并退出文档。

2.2.6　Vim 小技巧

1. 显示行号

显示当前行是第几行的方式有很多，可以通过为文档添加行号来实现，添加行号的方法是在命令模式下输入:set number 或简写:set nu 即可。

2. 忽略大小写

在 Vim 中查找时，你可能不清楚所要找的关键词的大小写，而 Vim 默认是区分大小写的，这时可以通过在命令模式下输入:set ignorecase，实现忽略大小写。

3. 多窗口编辑

当需要同时编辑多个文档时，分割窗口就尤为重要了，最简单的方式是在命令模式下输入:split。如此可以实现编辑同一个文档的不同行，至于窗口的切换，可以使用如下快捷方式：

Ctrl+w+h	Ctrl+w+l
Ctrl+w+j	Ctrl+w+k

Ctrl+w+h 快捷键表示跳转至左边一个窗口，Ctrl+w+l 快捷键表示跳转至右边一个窗口，Ctrl+w+j 快捷键表示跳转至上面一个窗口，Ctrl+w+k 快捷键表示跳转至下面一个窗口。

在命令模式下输入:close，可以关闭当前窗口。

在命令模式下输入:split second.txt，此命令会分割窗口并打开新的文件，如此实现多窗口多文件的编辑工作。默认的:split 为水平分割窗口，垂直分割可以使用:vsplit 指令。

4. 执行 Shell 命令

使用 Vim 编辑文档的过程中，如需要执行一条 Shell 命令而不想退出 Vim 编辑器，可以通过:!{命令}的方式实现。例如，需要查看当前目录下档案的名称，则在命令模式下输入指令:!ls 即可，执行完成后按回车键返回 Vim 编辑器。

5. 自动补齐

如果需要输入的内容在前面的行中已经出现过，那么 Vim 可以根据上文内容自动补齐输入。例如，在文件第三行定义了一个变量 FIRST_TIME=09，以后需要再次输入 FIRST_TIME 时，可以仅输入 FI 后使用快捷键 Ctrl+N，实现自动补齐功能。

2.3 账户与安全

2.3.1 账户及组的概念

Linux 系统对账号与组的管理是通过 ID 号来实现的，我们在登录系统时，输入用户名与对应的密码，后台系统会将用户名转化为 ID 号后再判断该账号是否存在，并对比密码是否匹配。Linux 中，用户 ID 号被称为 UID，组 ID 号被称为 GID。其中，UID 为 0，代表超级管理员，也就是通常所说的 root 账号，1~499 之间的 ID 号系统会预留下来。这样我们创建的普通用户 ID 号会从 500 算起。

Linux 的组有基本组与附加组之分，一个用户只可以加入一个基本组中，但可以同时加入多个附加组。创建用户时，系统默认会自动创建同名的组，并设置用户加入该基本组中。

2.3.2 创建账户及组

使用系统命令 useradd 可以创建我们需要的账户，groupadd 命令用来创建组账户。需要注

意的是，创建账户及组时需要有管理员权限。

1. useradd

描述：创建新账号。

用法：useradd [选项] 用户名称

选项：-c　　设置账号描述信息，一般为账号全称。

-d　　设置账号家目录，默认为/home/用户名。

-e　　设置账户的失效日期，格式为 YYYY-MM-DD。

-g　　设置账户的基本组。

-G　　设置账户的附加组，多个附加组中间用逗号隔开。

-M　　不创建账户家目录，一般与-s 结合使用。

-s　　设置账户的登录 Shell，默认为 bash。

-u　　指定账户 UID。

```
[root@centos6 ~]# useradd Frank          #创建普通账户 Frank
[root@centos6 ~]# useradd -c administrator -d /home/admin -e 2013-12-24 \
-g root -G bin,adm,mail admin
```

创建系统账户名称为 admin，全名为 administrator，账户家目录为/home/admin，账户失效日期为 2013 年 12 月 24 日，账户基本组为 root，附加组为 bin、adm、mail。

提示

\的作用是换行输入命令，命令比较长时经常会用到\符号。

```
[root@centos6 ~]# useradd -s /sbin/nologin -M user2
                                  #创建无法登录系统且没有家目录的用户 user2
```

2. groupadd

描述：创建组账户。

用法：groupadd [选项] 组名称

选项：-g　　设置组 ID 号。

```
[root@centos6 ~]# groupadd tom              #创建 tom 组
[root@centos6 ~]# groupadd -g 1000 jerry    #创建 GID 为 1000 的组 jerry
```

3. id

描述：显示账户及组信息。

```
[root@centos6 ~]# id root                          #查看 root 账户及相关组信息
```

2.3.3 修改账户及组

1. passwd

描述：更新账号认证信息。

用法：passwd [选项] [账户名称]

选项：-l　　锁定账户，仅 root 可使用此选项。

--stdin　从文件或管道读取密码。

-u　　解锁账户。

-d　　快速清空账户密码，仅 root 可使用此选项。

```
[root@centos6 ~]# passwd                           #为当前用户设置新密码
```

修改密码时若提示(current) UNIX password 信息，则先输入原始密码以修改新密码；提示 New password 信息时，即可输入新的密码；提示 Retype new password 信息时，再次确认输入的密码，注意密码的复杂度问题。提示 passwd: all authentication tokens updated successfully 信息说明修改成功。

```
[root@centos6 ~]# passwd tom                       #指定修改 tom 的密码
[root@centos6 ~]# echo "qwer0987" |passwd --stdin tom
                                                   #设置 tom 的密码为 qwer0987
 [root@centos6 ~]# passwd -l tom                   #锁定账户 tom
[root@centos6 ~]# passwd -u tom                    #解锁账户 tom
[root@centos6 ~]# passwd -d tom                    #清空账户 tom 的密码（无密码可登录系统）
```

2. usermod

描述：修改账户信息。

用法：usermod [选项] 账户名称

选项：-d 修改账户家目录。

-e 修改账户失效日期。

-g 修改账户所属基本组。

-G修改账户所属附加组。

-s 修改账户登录 Shell。

-u 修改账户 UID。

```
[root@centos6 ~]# usermod -d /home/tomcat tom
                                    #修改 tom 的家目录，/home/tomcat 目录必须存在
[root@centos6 ~]# usermod -e 2013-10-01 tom
[root@centos6 ~]# usermod -g mail tom
                                    #修改账户 tom 的基本组为 mail
[root@centos6 ~]# usermod -s /bin/bash user2
                                    #修改 user2 登录 Shell 为 bash
[root@centos6 ~]# usermod -u 1001 tom
                                    #修改 tom 的 UID 为 1001
```

2.3.4　删除账户及组

1. userdel

描述：删除账户及相关文件。

用法：userdel [选项] 账户名称

选项：-r　　删除账户及相关文件。

```
[root@centos6 ~]# userdel tom          #删除账户 tom，但该账户的文件不删除
[root@centos6 ~]# userdel -r tom       #删除账户 tom，并删除相应的家目录
```

2. groupdel

描述：删除组账户。

```
[root@centos6 ~]# groupdel jerry       #删除组 jerry
```

2.3.5　账户与组文件解析

1. 账户信息文件

账户信息被保存在/etc/passwd 文件中，通过命令 cat /etc/passwd 查看文件内容如下：

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
……中间部分省略……
oracle:x:501:502::/home/oracle:/bin/bash
```

```
admin:x:502:0:administrator:/home/admin:/bin/bash
```

文件以冒号为分隔符，第一列为账户名称，第二列为密码占位符（x 表示该账户需要密码才可以登录，为空时，账户无须密码即可登录），第三列为账户 UID，第四列为 GID，第五列为账户附加基本信息，一般存储账户名全称、联系方式等信息，第六列为账户家目录位置，第七列为账户登录 Shell，/bin/bash 为可登录系统 Shell，/sbin/nologin 表示账户无法登录系统。

2. 账户密码文件

账户密码信息被保存在/etc/shadow 文件中，通过命令 cat /etc/shadow 查看文件内容如下：

```
root:$6$8nVyGqXERxMrIm5/$kZTa8mZdGlPs0zPs0TqgD9faUxVwbyswCJVtITJTzs4
hm5BdMQfbqnBIsw6cyi0I/CikByS2wfAZ1vwfAP2kk.:15678:0:99999:7:::
bin:*:15513:0:99999:7:::
……中间部分省略……
tom:!!:15690:0:99999:7:::
admin:$6$R73em9Ix$8kXDlkzFVRyMBkXwjmWCMv28V0bT58tIg1vFTlT9Kh//gPH4t
77OLkVK9U3HdsFzFxWSW9XQhLVjN25QGsDj0.:15690:0:99999:7::16063:
```

文件以冒号为分隔符，第一列为账户名称，第二列为密码（账户未设置密码时为!!，设置密码后加密显示，CentOS 6 默认采用 SHA-512 算法），第三列为上次修改密码的时间距离 1970 年 01 月 01 日有多少天（依此推算最后一次修改密码的日期），第四列为密码最短有效天数（密码至少使用多少天，0 代表无限制），第五列为密码最长有效天数（默认 99999 可以理解为永不过期），第六列为过期前的警告天数（默认过期前提前 7 天警告，但进入警告日期后仍可以使用旧密码登录系统），第七列为密码过期后的宽限天数（密码过期后，预留几天给账户修改密码，此时已无法使用旧密码登录系统），第八列为账户失效日期（从 1970 年 01 月 01 日起多少天后账户失效），第九列暂时保留未使用。

3. 组账户信息文件

组账户信息被保存在/etc/group 文件中，通过命令 cat /etc/group 查看文件内容如下：

```
……此处省略……
wheel:x:10:
mail:x:12:mail,postfix,admin
uucp:x:14:
……此处省略……
```

文件以冒号为分隔符，第一列为组账户名称，第二列为密码占位符，第三列为 GID，第四列为组成员信息（注意，这里仅显示基本成员，附加成员不显示）。

4. 组账户密码文件

组账户密码信息被保存在/etc/gshadow 文件中，通过命令 cat /etc/gshadow 查看内容如下：

```
root:::
bin:::bin,daemon,admin
daemon:::bin,daemon
……中间部分省略……
dba:!::oracle
tom:!::
```

文件以冒号为分隔符，第一列为组账号名称，第二列为组密码（一般为组管理员密码），第三列为组管理员，第四列为组成员（与/etc/group 第四列相同）。

通过<gpasswd 组名>的方式可以为组设置密码，通过<gpasswd -A 账户名称 组账户名称>可以为组添加管理员。

```
[root@centos6 ~]# gpasswd admin                 #设置组密码
[root@centos6 ~]# gpasswd -A mail admin         #将mail账户设置为组admin的管理员
```

2.3.6　文件及目录权限

1. 概念

Linux 权限主要分为读、写、执行三种控制，使用 ls -l 命令查看文件或目录信息时，系统会显示为 r（读取权限）、w（写入权限）、x（执行权限），以下为 ls -l 显示的信息：

```
[root@centos6 ~]# ls -l
total 56
-rw-------.  1  root root 1094 Dec  4  09:30 anaconda-ks.cfg
drwxr-xr-x. 2  root root 4096 Dec  7  10:14 Desktop
drwxr-xr-x. 3  root root 4096 Dec  7  10:14 Documents
drwxr-xr-x. 2  root root 4096 Dec  4  11:40 Downloads
-rw-r--r--.  1  root root 9064 Dec  4  09:30 install.log
-rw-r--r--.  1  root root 3091 Dec  4  09:16 install.log.syslog
```

第一列的第一个字符为文件类型：-代表普通文件，d 代表目录，l 代表链接文件，b 或 c 代表设备。第二至第九个字符代表权限，三位一组分别为所有者权限、所属组权限、其他账户权限，rwxrwxrwx 表示文档所有者（user）、组（group）、其他账户（other）权限均为可读、可写、可执行，rwxr--r--表示文档所有者权限为读写执行，所属组权限为只读，其他账户权限为只读。第二列为链接数量或子目录个数，第三列为文档的所有者，第四列为文件的所属组，第五列为容量，第六列为最近文档被修改的月份，第七列为文档被修改的日期，第八列为文档被修改的

时间，第九列为文件或目录名称。

对于权限的表示，除了可以使用比较直观的 rwx 表示外，还可以使用数字表示，表 2-6 给出了它们之间的对应关系，以及这些权限对文件与目录的含义。

表 2-6

数字	字符	文件	目录
4	R	查看文件内容	查看目录下的文件与目录名称
2	W	修改文件内容	在目录下增、删、改文件与目录名称
1	X	可执行，一般为程序或脚本	可以用 cd 命令进入该目录

2. 修改文档属性

（1）chmod

描述：改变文件或目录权限。

用法：chmod [选项] 权限 文件或目录

选项：--reference=RFILE　　#根据参考文档设置权限

　　-R　　#递归将权限应用于所有的子目录与子文件

chmod 命令参数中，u 代表所有者，g 代表所属组，o 代表其他用户，a 代表所有人。

先查看 install.log 文件的权限为文件所有者可读可写，所属组只读，其他人只读。通过 chmod 修改文件所有者权限、所属组权限、其他人权限均设置为可读、可写、可执行，完成修改后，使用 ls -l 命令查看修改结果。

```
[root@centos6 ~]# ls -l install.log
-rw-r--r--. 1 root root 9064 Dec  4 09:30 install.log
[root@centos6 ~]# chmod u=rwx,g=rwx,o=rwx install.log
[root@centos6 ~]# ls -l install.log
-rwxrwxrwx. 1 root root 9064 Dec  4 09:30 install.log
[root@centos6 ~]# chmod a=rw  install.log
[root@centos6 ~]# ls -l install.log
-rw-rw-rw-. 1 root root 9064 Dec  4 09:30 install.log
```

使用字符修改权限的另一种形式是在原有权限的基础上修改权限，方法是使用+/-权限的方式。下面的例子通过 chmod 命令将 install.log 文件所属组权限在原有权限基础上减去执行的权限，其他人的权限也在原有权限的基础上减去修改与执行的权限。

```
[root@centos6 ~]# chmod g-x,o-wx install.log
[root@centos6 ~]# ls -l install.log
-rwxrw-r--. 1 root root 9064 Dec  4 09:30 install.log
```

chmod 可以使用+符号来增加对应的权限，也可以使用-符号来减去相应的权限。

除了使用字符的方式修改权限外，chmod 还支持使用数字的方式修改权限，数字与权限的对应关系参见表 2-6，下面通过实例说明使用方法。

```
[root@centos6 ~]# chmod 700 install.log          #修改文件权限为 rwx------
[root@centos6 ~]# chmod 644 install.log          #修改文件权限为 rw-r--r--
[root@centos6 ~]# chmod 755 /home                #修改目录权限为 rwxr-xr-x
```

最后以其他文档作为参考修改权限。

```
[root@centos6 ~]# chmod --reference=install.log.syslog  install.log
                                    #以 install.log.syslog 为标准修改 install.log 的权限
```

（2）chown

描述：修改文件或目录的所有者与所属组。

用法：chown [选项] [所有者] [:[所属组]] 文件或目录

选项：-R #递归将权限应用于所有的子目录与子文件。

```
[root@centos6 ~]# chow user2:mail install
                                    #修改文件的所有者为 user2,所属组为 mail
[root@centos6 ~]# chow :root install    #仅修改文件所属组为 root
[root@centos6 ~]# chow root install     #仅修改文件所属者为 root
```

2.3.7　账户管理案例

下面将介绍一个模拟公司的组织结构，EXAMPLE 公司是一个从事教育培训的组织，公司结构主要分为教研室（teach）、校长办公室（office）、财务部（finance）、行政部（admin）、市场部（market）。为了后期部署文件共享服务器，我们需要为每个部门创建各自独立的共享目录，根据要求为所有的普通员工创建各自独立的账户，这些账户仅需要读权限即可，并为每个部门的负责人创建一个管理账户（管理员名称为 Op_部门名称），这个账户要求可以进行读写操作。为了后期管理的方便，需要为每个部门创建与部门名称同名的组账户。

```
#创建共享目录
[root@centos6 ~]# mkdir -p /var/{teach,office,finance,admin,market}
#创建组账户
[root@centos6 ~]# groupadd teach
[root@centos6 ~]# groupadd finance
[root@centos6 ~]# groupadd admin
[root@centos6 ~]# groupadd market
#创建组管理员账户
[root@centos6 ~]# useradd -g teach Op_teach
[root@centos6 ~]# useradd -g office Op_office
```

```
[root@centos6 ~]# useradd -g finance Op_finance
[root@centos6 ~]# useradd -g admin Op_admin
[root@centos6 ~]# useradd -g market Op_market
#创建普通员工账户
[root@centos6 ~]# useradd -g teach endy
[root@centos6 ~]# useradd -g office lucy
[root@centos6 ~]# useradd -g finance jacob
[root@centos6 ~]# useradd -g admin jerry
[root@centos6 ~]# useradd -g market marry
[root@centos6 ~]# id jacob
#添加组管理员
[root@centos6 ~]# gpasswd -A Op_teach teach
[root@centos6 ~]# gpasswd -A Op_office office
[root@centos6 ~]# gpasswd -A Op_finance finance
[root@centos6 ~]# gpasswd -A Op_admin admin
[root@centos6 ~]# gpasswd -A Op_market market
[root@centos6 ~]# chown Op_teach.teach /var/teach
[root@centos6 ~]# chown Op_office.office /var/office
[root@centos6 ~]# chown Op_teach.finance /var/finance
[root@centos6 ~]# chown Op_teach.admin /var/admin
[root@centos6 ~]# chown Op_teach.market /var/market
[root@centos6 ~]# chmod 755 /var/{teach,office,finance,admin,market}
```

2.3.8 ACL 访问控制权限

由于系统的基本权限是针对文档所有者、所属组或其他账户进行控制的，无法针对某个单独的账户进行控制，所以就有了 ACL（Account Control List）访问控制列表的概念，使用 ACL，我们可以针对单一账户设置文档的访问权限。

Linux 系统使用 getfacl 查看文档的 ACL 权限，使用 setfacl 来设置文档的 ACL 权限。

getfacl 输出内容的格式大致如下：

```
1:  # file: somedir/
2:  # owner: lisa
3:  # group: staff
4:  # flags: -s-
5:  user::rwx
6:  user:joe:rwx              #effective:r-x
7:  group::rwx                #effective:r-x
8:  group:cool:r-x
9:  mask::r-x
10:  other::r-x
```

以上输出信息中，第一行为文件或目录名称，第二行为文档所有者，第三行为文档所属组，第四行为 suid、sgid、sticky 权限的标记位，第五行为文档所有者权限，第七行为文档所属组权限，第十行为其他账户权限，第五、七、十行为系统基本权限，第六行为通过 ACL 指令添加的对账户的访问控制权限，第八行为通过 ACL 指令添加的对组账户的访问控制权限，第九行为权限掩码行。

```
[root@centos6 ~]# getfacl install.log
# file: install.log
# owner: root
# group: root
user::rw-
group::rw-
other::rw-
```

从以上输出信息可以看出，该文件未设置附加的 ACL 访问控制条目，仅有基本的文件所有者、所属组、其他账户的访问控制。

setfacl 的作用和用法如下。

描述：设置文档访问控制列表。
用法：setfacl [选项] [{-m|-x} acl 条目] 文件或目录
选项：-b　　#删除所有附加的 ACL 条目
　　-k　　#删除默认的 ACL
　　-m　　#添加 ACL 条目
　　-x　　#删除指定的 ACL 条目
　　-R　　#递归处理所有的子文件与子目录

```
[root@centos6 tmp]# setfacl -m u:user1:rw test.txt
                                #添加 ACL 条目，使用户 user1 对 test.txt 文件可读可写
[root@centos6 tmp]# setfacl -m g:user1:r test.txt
                                #添加 ACL 条目，使 user1 组对 test.txt 文件可读
[root@centos6 tmp]# setfacl -x g:user1 test.txt
                                #删除 user1 组的 ACL 条目
[root@centos6 tmp]# setfacl -x u:user1 test.txt
                                #删除账户 user1 的 ACL 条目
[root@centos6 tmp]# setfacl -b test.txt
                                #删除所有附加的 ACL 条目
```

2.4　存储管理

硬盘设备购买回来后需要对其进行分区才可以使用，分区可以使公司业务数据得到更有效

的管理。正如生活中一个大的衣柜往往会被分割为多个大小不一的小格子一样，这样就可以根据自己的不同需要将衣物分类存放，而计算机中的数据要比生活中的衣物更多、更复杂，如果大量的数据都堆积在一起管理会很困难，并且随着时间的推移，管理员会越来越痛苦！这时我们就需要提前规划好磁盘的存储空间，至于一块硬盘到底要分多少个区，每个分区需要划分多大的空间，这需要根据自己的文档数量、大小、类型等因素进行不同的设计。

有关硬盘的识别，Linux 根据设备类型对存储设备进行识别，如果是 IDE 存储设备，在计算机中将被识别为 hd，第一个 IDE 设备会被识别为 hda，第二个 IDE 设备会被识别为 hdb，依此类推。如果是 SATA、USB 或 SCSI 设备，会被识别为 sd，同样第一个此类设备为 sda，第二个此类设备为 sdb，依此类推。

对于分区，Linux 使用数字来表示。如第一块 SATA 硬盘的第一个分区为 sda1，第二块 SATA 硬盘的第二个分区为 sdb2。

2.4.1 磁盘分区

传统的 MBR 分区方式是一块硬盘最多可以分四个主分区，如图 2-3 所示，我们为硬盘分配了四个主分区，那么即使硬盘还有剩余空间，也无法再继续分区。

图 2-3

传统的 MBR 分区方式中，如果需要更多的分区，则需要使用在扩展分区中创建逻辑分区的方法来实现，解决方法如图 2-4 所示。此时可以在扩展分区中划分出多个逻辑分区，所有逻辑分区的总和为扩展分区的大小。逻辑分区一定是以编号为 5 开始的，也就是说，SATA 磁盘的第一个逻辑分区一定为 sda5。

图 2-4

以下是具体的操作步骤。

```
[root@centos6 Desktop]# fdisk -cul                    #查看磁盘分区表

 Disk /dev/sda: 23.6 GB, 23596105728 bytes      #第一块磁盘的总容量为 23.6GB
…中间部分省略…
   Device Boot      Start         End      Blocks   Id  System
```

```
/dev/sda1   *        2048     1026047      512000   83  Linux
/dev/sda2         1026048    46086143    22530048   8e  Linux LVM
#第一块磁盘有两个分区：/dev/sda1 与/dev/sda2

Disk /dev/sdb: 8589 MB, 8589934592 bytes        #第二块磁盘容量约 8GB
…中间部分省略…
   Device Boot      Start         End      Blocks   Id  System

#第二块磁盘的分区表为空，暂时没有分区
```

```
[root@centos6 Desktop]# fdisk -cu /dev/sdb      #这里为第二块磁盘进行分区

Command (m for help): m                         #在此输入指令 m 获得帮助
Command action
   a   toggle a bootable flag                   #切换分区启动标记
   b   edit bsd disklabel                       #编辑 bsd 磁盘标签
   c   toggle the dos compatibility flag        #切换 dos 兼容模式
   d   delete a partition                       #删除分区
   l   list known partition types               #显示分区类型
   m   print this menu                          #显示帮助菜单
   n   add a new partition                      #新建分区
   o   create a new empty DOS partition table   #创建新的空分区表
   p   print the partition table                #显示分区表信息
   q   quit without saving changes              #不保存退出
   s   create a new empty Sun disklabel         #创建新的 Sun 磁盘标签
   t   change a partition's system id           #修改分区 id，可以通过 l 查看 id
   u   change display/entry units               #修改容量单位，磁柱或扇区
   v   verify the partition table               #检验分区表
   w   write table to disk and exit             #保存并退出
   x   extra functionality (experts only)       #扩展功能

Command (m for help): n
Command action
   e   extended                                 #输入 e 创建扩展分区
   p   primary partition (1-4)                  #输入 p 创建主分区
p
Partition number (1-4): 1                       #输入分区编号为 1
First sector (2048-16777215, default 2048):     #回车从默认的 2048 扇区开始分区
Using default value 2048
Last sector, +sectors or +size{K,M,G} (2048-16777215, default 16777215): +2G
                                                #创建 2GB 的分区

Command (m for help): p                         #显示是否创建成功
…中间部分省略…
```

```
   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     4196351     2097152   83  Linux
#可以看到/dev/sdb1 分区已经被创建，容量为 2GB

Command (m for help): n                                    #创建新分区
Command action
   e   extended
   p   primary partition (1-4)
e                                                          #选择创建扩展分区
Partition number (1-4): 2
First sector (4196352-16777215, default 4196352):
                                                           #回车从默认的 4196352 磁柱开始分区
Using default value 4196352
Last sector, +sectors or +size{K,M,G}(4196352-16777215, default 16777215):
                                                           #回车将剩余的所有空间划分为扩展分区
Using default value 16777215

Command (m for help): n                                    #创建新分区
Command action
   l   logical (5 or over)
   p   primary partition (1-4)
l                                                          #选择创建逻辑分区
First sector (4198400-16777215, default 4198400):
                                                           #回车从默认的 4198400 扇区开始
Using default value 4198400
Last sector, +sectors or +size{K,M,G} (4198400-16777215, default 16777215): +2G
                                                           #创建 2GB 的分区
Command (m for help): p                                    #查看分区情况
…中间部分省略…
   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     4196351     2097152   83  Linux
/dev/sdb2         4196352    16777215     6290432    5  Extended
/dev/sdb5         4198400     8392703     2097152   83  Linux

Command (m for help): d                                    #删除分区
Partition number (1-5): 5                                  #输入需要删除的分区编号

Command (m for help): p                                    #查看分区情况
…中间部分省略…
   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048     4196351     2097152   83  Linux
/dev/sdb2         4196352    16777215     6290432    5  Extended

Command (m for help):w                                     #保存并退出
```

我们可以使用 partprobe 命令让内核立即读取新的分区表，这样无须重启系统，即可识别新创建的分区。

```
[root@centos6 Desktop]# partprobe /dev/sdb
```

对于上面所讲述的传统 MBR 分区方式而言，有多种限制。如，最多 4 个主分区，无法创建大于 2TB 的分区，而现在有一种新的 GPT 分区方式则不受这样的限制，此外，GPT 分区提供了分区表的冗余以实现分区表的备份与安全。但 fdisk 命令不支持 GPT 分区方式，这里我们需要使用 Linux 的另一个分区工具 parted，命令格式为：

```
parted [选项] [磁盘 [命令]]
```

注意，改变分区方式后，原有磁盘中的数据将会全部丢失，因此要保存好自己的数据备份。这里我们选择第三块磁盘来进行 GPT 分区演示，具体操作方法如下。

1. 修改分区表类型

```
[root@centos6 ~]# parted /dev/sdc mklabel gpt      #修改分区表格式
Warning: The existing disk label on /dev/sdc will be destroyed and all data
on this disk will be
lost. Do you want to continue?                     #提示磁盘数据将丢失
Yes/No?yes                                         #输入 yes 完成修改
```

修改完成后，我们可以通过 print 指令查看修改效果：

```
[root@centos6 ~]# parted /dev/sdc print            #查看系统分区表信息
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdc: 8590MB
Sector size (logical/physical): 512B/512B
Partition Table: gpt                               #以看出分区表类型为 gpt
```

2. 创建与删除分区

创建新的分区需要使用 parted 命令的 mkpart 指令，语法格式如下：

```
parted [磁盘] mkpart 分区类型 文件系统类型 开始 结束
```

其中，mkpart 指令为创建新的分区，分区类型有：primary、logical、extended 三种，文件系统类型有：fat16、fat32、ext2、ext3、linux-swap 等，开始与结束标记区分开始与结束的位置（默认单位为 MB）。

```
[root@centos6 ~]# parted /dev/sdc mkpart primary ext3 1 2G
```

上面的命令将创建一个格式为 ext3 的主分区，从磁盘的第 1MB 开始分区，到 2GB 的位置

（大小为 2GB 的主分区）。

```
[root@centos6 ~]# parted /dev/sdc mkpart primary ext3 2G 4G
```

上面命令将创建一个容量为 2GB 的分区（从磁盘的第 2 个 GB 位置开始分区，到第 4 个 GB 的位置结束）。

```
[root@centos6 ~]# parted /dev/sdc print      #查看分区表信息
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdc: 8590MB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Number  Start    End     Size     File system  Name     Flags
 1      1049kB   2000MB  1999MB                primary
 2      2000MB   4000MB  2001MB               primary
[root@centos6 ~]# parted /dev/sdc rm 2        #使用 rm 指令可以删除分区
```

除了基本的分区创建与删除外，利用 parted 命令还可以进行分区检查、调整分区大小、还原误删除分区等操作，关于 parted 命令的更多使用方法，可以查阅 man 手册。

2.4.2 格式化与挂载文件系统

对磁盘进行分区后，接下来需要格式化与挂载文件系统，在某些操作系统中，格式化完成后，系统会自动挂载至一个盘符上，然后就可以直接访问并使用它，但 Linux 需要我们手动完成挂载操作。

我们使用 mkfs 命令来完成格式化操作，其中 Linux 系统默认文件系统格式为 ext4 格式。

```
[root@centos6 Desktop]# mkfs.ext4 /dev/sdc1    将/dev/sdc1 格式化为 ext4 格式
mke2fs 1.41.12 (17-May-2010)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
122160 inodes, 487936 blocks
24396 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=503316480
15 block groups
32768 blocks per group, 32768 fragments per group
8144 inodes per group
```

```
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
This filesystem will be automatically checked every 32 mounts or
180 days, whichever comes first.  Use tune2fs -c or -i to override.
```

另外，对于交换分区，我们需要使用单独的 mkswap 命令实现格式化的工作。

```
[root@centos6 Desktop]# mkswap /dev/sdc2          将/dev/sdc1 格式化为 swap 分区
Setting up swapspace version 1, size = 1953788 KiB
no label, UUID=fa924034-29cf-4a16-b0c7-6feb159810ed
```

格式化完成后，需要我们手动挂载文件系统，挂载可以使用命令或者修改系统文件两种方式，使用命令挂载文件系统可以立即生效并使用，但计算机重启后无效。另一种需要修改系统文件，这种方式挂载的文件系统在计算机重启后仍然有效。下面分别介绍这两种方式。

第一种方式使用 mount 命令实现文件系统的挂载。

名称：mount

描述：挂载文件系统。

用法：mount [选项] [-o [选项]] 设备|目录

选项：-a　　挂载/etc/fstab 文件中所有未挂载的文件系统。

-t　　指定文件系统类型（包括：adfs、affs、autofs、cifs、coda、coherent、cramfs、debugfs、devpts、efs、ext、ext2、ext3、ext4、hfs、hfsplus、hpfs、iso9660、jfs、minix、msdos、ncpfs、nfs、nfs4、ntfs、proc、qnx4、ramfs、reiserfs、romfs、squashfs、smbfs、sysv、tmpfs、ubifs、udf、ufs、umsdos、usbfs、vfat、xenix、xfs、xiafs），默认类型为 ext4。

-o　　指定挂载属性，具体属性可参考 man 帮助手册。

```
[root@centos6 ~]# mkdir /data1                          #创建挂载目录
[root@centos6 ~]# mount /dev/sdc1 /data1/               #将 sdc1 挂载至/data1 目录
[root@centos6 ~]# mount                                 #查看挂载状态
… …此处省略… …
/dev/sdc1 on /data1 type ext4 (rw)
[root@centos6 ~]# touch /data1/test.txt                 #创建测试文件
[root@centos6 ~]# mount /dev/cdrom /media/              #挂载光盘至/media 目录
mount: block device /dev/sr0 is write-protected, mounting read-only
[root@centos6 ~]# umount /dev/cdrom                     #卸载光盘设备
[root@centos6 ~]# umount /dev/sdc1                      #卸载 sdc1 文件系统
```

第二种方式是修改系统文件/etc/fstab，该文件共计六列，第一列为设备名称或设备标签，第二列为挂载点名称（挂载点为用户自己创建的目录），第三列为文件系统类型，第四列为挂载属性，第五列指定该文件系统是否使用 dump 进行备份（0 表示不备份，1 表示每天备份），第六列指定该文件系统在开机后使用 fsck 程序进行磁盘检测的顺序（根文件系统需要指定为 1，其他文件系统一般为 2，如果为 0，则表示不需要进行磁盘检测）。

```
[root@centos6 ~]# vim /etc/fstab          #修改文件插入如下一行内容
/dev/sdc1/data1ext4defaults0  0
[root@centos6 ~]# mount -a                #挂载 fstab 文件中尚未挂载的所有文件系统
[root@centos6 ~]# mount                   #检查文件系统的挂载情况
```

2.4.3 LVM 逻辑卷概述

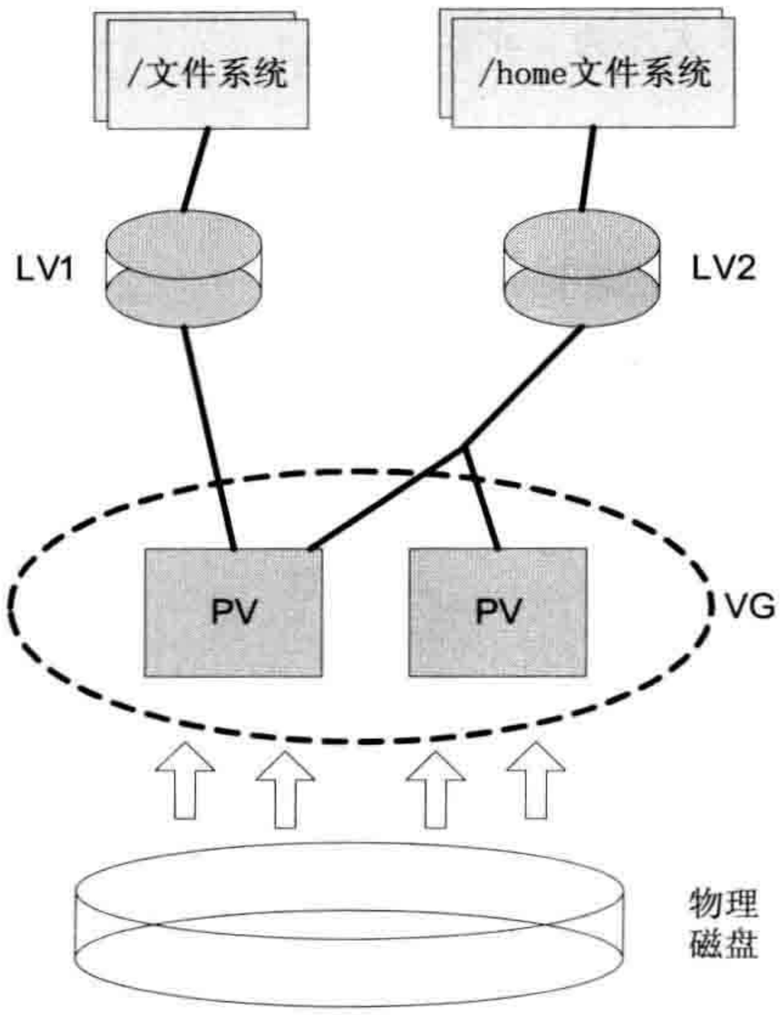
试想有一天随着业务的增加，你的文件系统负载会越来越大，终于到了空间不足的情况，而如果我们还在使用传统的分区方式进行磁盘管理，就不得不将现有的所有分区全部删除，并重新规划新的存储解决方案。其实从一开始我们就需要这样一种管理机制来帮助我们动态地管理存储，而 LVM 就提供了这种功能。

LVM（Logical Volume Manager）是基于内核的一种逻辑卷管理器，LVM 适合于管理大存储设备，并允许用户动态调整文件系统的大小。此外，LVM 的快照功能可以帮助我们快速备份数据。LVM 为我们提供了逻辑概念上的磁盘，使得文件系统不再关心底层物理磁盘的概念。

使用 LVM 创建逻辑卷需要我们提前了解以下几个概念。

- 物理卷（Physical Volume，PV）：物理卷是 LVM 的最底层概念，是 LVM 的逻辑存储块，物理卷与磁盘分区是逻辑的对应关系。生活中的圆形积木在没有附加工具时是不能固定组合的，但方形却有组合的功能，我们可以假设普通的分区是一个圆形，而物理卷可以假设为方形。LVM 提供了命令工具可以将分区转换为物理卷，通过组合物理卷可以生成卷组。
- 卷组（Volume Group，VG）：卷组是 LVM 逻辑概念上的磁盘设备，通过将单个或多个物理卷组合后生成卷组。卷组的大小取决于物理卷的容量以及个数。在 CentOS 6.3 系统中，对容量与个数没有限制。
- 物理长度（Physical Extent，PE）：物理长度是将物理卷组合为卷组后，所划分的最小存储单位，即逻辑意义上磁盘的最小存储单元。LVM 默认 PE 大小为 4MB。
- 逻辑卷（Logical Volume，LV）：逻辑卷就是 LVM 逻辑意义上的分区，我们可以指定从卷组中提取多少容量来创建逻辑卷，最后对逻辑卷格式化并挂载使用。

图 2-5 给出了是 LVM 整体概念的示意图。



LVM概念示意图

图 2-5

通过对以上概念的了解，接下来要说明具体使用到的命令。将分区转换为物理卷的命令是 pvcreate，将物理卷组合为卷组的命令是 vgcreate，从卷组中提取容量创建逻辑卷的命令是 lvcreate。创建完成后可以分别使用 pvdisplay、vgdisplay、lvdisplay 查看效果。

1. pvcreate

描述：使用 LVM 对磁盘或分区进行初始化。

用法：pvcreate [选项] 物理卷 [物理卷...]

例子：

```
[root@centos6 ~]# pvcreate /dev/sdc4 /dev/sde
[root@centos6 ~]# pvcreate /dev/sdb{1,2,3}
```

2. vgcreate

描述：创建卷组。

用法：vgcreate [选项] 卷组名称 物理设备路径 [物理设备路径...]

例子：

```
[root@centos6 ~]# vgcreate test_vg1 /dev/sdb5 /dev/sdb6
[root@centos6 ~]# vgcreate test_vg2 -s 16M /dev/sdc5 /dev/sdc6
                  #指定 PE 大小为 16MB
```

3. lvcreate

描述：在已存在的卷组中创建逻辑卷。

用法：lvcreate [选项] 卷组名称或路径 [物理卷路径]

选项：-l 指定使用多少个卷组中的 PE 创建逻辑卷，PE 的容量在创建卷组时已经指定。

-L 直接指定逻辑卷的容量大小，单位可以是 b（B）、s（S）、k（K）、m（M）、g（G）、t（T）、p（P）或 e（E）。

-n 指定逻辑卷名称。

例子：

```
[root@centos6 ~]# lvcreate -L 2G -n test_lv1 test_vg1
                                  #创建容量为 2GB 的逻辑卷
[root@centos6 ~]# lvcreate -l 200 -n test_lv2 test_vg2
                                  #使用 200 个 PE 创建逻辑卷,前面定义 PE 大小为 16MB
[root@centos6 ~]# lvcreate -L 2G -n test_lv1 test_vg1 /dev/sdb6
                                  #指定使用/dev/sdb6 这个物理卷的存储空间
```

2.4.4 创建 LVM 分区实例

这里我们使用 sdb 这块 500GB 的硬盘创建四个 100GB 的分区，并使用这四个分区创建一个名称为 test_vg 的卷组，最后从该卷组中创建出两个大小为 120GB 的逻辑卷，名称分别为 test_web、test_data。

1. 使用 fdisk 创建分区

```
[root@centos6 ~]# fdisk -cu /dev/sdb
```

```
Command (m for help): n
Command action
   e   extended
   p   primary partition (1-4)
p
Partition number (1-4): 1
First sector (2048-1048575999, default 2048):
Using default value 2048
Last sector, +sectors or +size{K,M,G} (2048-1048575999, default 1048575999): +100G
```

```
    Command (m for help): n
    Command action
       e   extended
       p   primary partition (1-4)
    p
    Partition number (1-4): 2
    First sector (209717248-1048575999, default 209717248):
    Using default value 209717248
    Last sector, +sectors or +size{K,M,G} (209717248-1048575999, default 1048575999):
+100G

    Command (m for help): n
    Command action
       e   extended
       p   primary partition (1-4)
    p
    Partition number (1-4): 3
    First sector (419432448-1048575999, default 419432448):
    Using default value 419432448
    Last sector, +sectors or +size{K,M,G} (419432448-1048575999, default
1048575999): +100G

    Command (m for help): n
    Command action
       e   extended
       p   primary partition (1-4)
    e
    Selected partition 4
    First sector (629147648-1048575999, default 629147648):
    Using default value 629147648
    Last sector, +sectors or +size{K,M,G} (629147648-1048575999, default
1048575999):
    Using default value 1048575999

    Command (m for help): n
    First sector (629149696-1048575999, default 629149696):
    Using default value 629149696
    Last sector, +sectors or +size{K,M,G} (629149696-1048575999, default
1048575999): +100G
    Command (m for help): t
    Partition number (1-5): 1                          #指定分区编号
    Hex code (type L to list codes): 8e                #修改分区标签为 LVM
     (其他四个分区做相同的操作…)
    Command (m for help): p
    …中间部分省略…
```

```
Device Boot      Start         End      Blocks   Id  System
/dev/sdb1            2048   209717247   104857600   8e  Linux LVM
/dev/sdb2       209717248   419432447   104857600   8e  Linux LVM
/dev/sdb3       419432448   629147647   104857600   8e  Linux LVM
/dev/sdb4       629147648  1048575999   209714176    5  Extended
/dev/sdb5       629149696   838864895   104857600   8e  Linux LVM

Command (m for help): w
The partition table has been altered!
Calling ioctl() to re-read partition table.
Syncing disks.
```

```
[root@centos6 ~]# partprobe /dev/sdb
```

2. 使用 pvcreate 创建物理卷并使用 pvdisplay 查看结果

```
[root@centos6 ~]# pvcreate /dev/sdb{1,2,3,5}
[root@centos6 ~]# pvdisplay
  "/dev/sdb1" is a new physical volume of "100.00 GiB"
  --- NEW Physical volume ---
  PV Name               /dev/sdb1                     #物理卷名称
  VG Name
  PV Size               100.00 GiB                    #物理卷大小
  Allocatable           NO
  PE Size               0
  Total PE              0
  Free PE               0
  Allocated PE          0
  PV UUID               MEO4oH-He1C-QGeX-JR3W-PnO0-iaKo-inQmwG
  …其他物理卷信息省略…
```

3. 使用 vgcreate 创建卷组并使用 vgdisplay 查看结果

```
[root@centos6 ~]# vgcreate test_vg /dev/sdb{1,2,3,5}
--- Volume group ---
  VG Name               test_vg                       #卷组名称
  System ID
  Format                lvm2
  Metadata Areas        4
  Metadata Sequence No  1
  VG Access             read/write
  VG Status             resizable
  MAX LV                0
  Cur LV                0
  Open LV               0
```

```
Max PV                0
Cur PV                4
Act PV                4
VG Size               399.98 GiB                    #卷组大小
PE Size               4.00 MiB                      #PE 大小
Total PE              102396
Alloc PE / Size       0 / 0
Free  PE / Size       102396 / 399.98 GiB           #剩余容量
VG UUID               rKFfkh-pcLn-l8BK-7NFK-c0Yd-5rzT-t9ftpU
```

4. 使用 lvcreate 创建逻辑卷并使用 lvdisplay 查看结果

```
[root@centos6 ~]# lvcreate -n test_web -L 120G test_vg
[root@centos6 ~]# lvcreate -n test_data -l 30720 test_vg
[root@centos6 ~]# lvdisplay
  --- Logical volume ---
  LV Path                /dev/test_vg/test_web     #逻辑卷路径
  LV Name                test_web                  #逻辑卷名称
  VG Name                test_vg                   #卷组名称
  LV UUID                DGR3d9-K9WS-iDRg-Z90X-CsWV-Acx7-lzQ6Ek
  LV Write Access        read/write
  LV Creation host, time centos6.example.com, 2012-12-31 13:27:54 +0800
  LV Status              available
  # open                 0
  LV Size                120.00 GiB                #逻辑卷大小
  Current LE             30720
  Segments               2
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           253:2
```

5. 格式化并挂载

```
[root@centos6 ~]# mkfs.ext4 /dev/test_vg/test_web
[root@centos6 ~]# mkfs.ext4 /dev/test_vg/test_data
[root@centos6 ~]# mkdir -p /test/{web,data}
[root@centos6 ~]# cat >> /etc/fstab <<EOF
> /dev/test_vg/test_data  /test/data  ext4  defaults  0  0
>/dev/test_vg/test_web  /test/web  ext4  defaults  0  0
> EOF
[root@centos6 ~]# mount -a
[root@centos6 ~]# df -h                                  #查看文件系统挂载状态
```

2.4.5 修改 LVM 分区容量

随着时间的推移，逻辑卷 test_data 现有的存储空间已经不能满足企业大业务量的需要，我们需要将存储容量增大至 240GB 时，LVM 随时可以帮助我们动态调整文件系统的大小。本节继续 2.4.4 节的案例，由于 test_vg 这个卷组还有足够的剩余空间可以划分给逻辑卷，所以我们可以直接使用 lvextend 命令调整逻辑卷大小。但 lvextend 仅调整逻辑卷大小，使用该命令调整完逻辑卷大小后，还需要使用 resize2fs 命令更新文件系统的大小，具体方法如下：

```
[root@centos6 ~]# lvextend -L +120G /dev/test_vg/test_data
[root@centos6 ~]# resize2fs /dev/test_vg/test_data
[root@centos6 ~]# df -h
```

此外，当我们的卷组没有足够的空间划分给逻辑卷时，LVM 的卷组也是可以动态调整容量大小的，这时需要使用 fdisk 创建一个新的分区，并使用 pvcreate 命令转换为物理卷格式，接着通过 vgextend 将新的物理卷追加至现有的卷组空间中，最后使用 lvextend 调整逻辑卷的容量。下面我们再次扩展逻辑卷 test_data 的存储空间至 360GB。

```
[root@centos6 ~]# fdisk -cu /dev/sdb
```

```
Command (m for help): n
First sector (838866944-1048575999, default 838866944):
Using default value 838866944
Last sector, +sectors or +size{K,M,G} (838866944-1048575999, default
1048575999):
Using default value 1048575999

Command (m for help): t
Partition number (1-6): 6
Hex code (type L to list codes): 8e
Changed system type of partition 6 to 8e (Linux LVM)
Command (m for help): w
```

```
[root@centos6 Desktop]# pvcreate /dev/sdb6
[root@centos6 Desktop]# vgextend test_vg /dev/sdb6
[root@centos6 Desktop]# vgdisplay test_vg
  --- Volume group ---
   VG Name               test_vg
   VG Size               499.98 GiB                #总容量 499GB
   Alloc PE / Size       92160 / 360.00 GiB        #已经使用 360GB
  Free  PE / Size       35835 / 139.98 GiB        #剩余 139GB
(以上输出信息部分内容省略)
[root@centos6 ~]# lvextend -L 360G /dev/test_vg/test_data
                                                  #将逻辑卷扩展至 360GB
```

```
[root@centos6 ~# resize2fs /dev/test_vg/test_data
[root@centos6 ~# df -h
```

提示

在企业生产环境中，为文件系统调整大小前一定要做好数据的备份工作。

2.4.6　删除 LVM 分区

当我们不再需要逻辑卷分区时，可以使用 LVM 命令轻松地删除之前创建的物理卷、卷组、逻辑卷。删除的顺序应该与创建时的顺序相反，也就是卸载文件系统、删除逻辑卷、删除卷组、删除物理卷这样的顺序。卸载文件系统时需要注意，如果有必要应该更新/etc/fstab 文件。

```
[root@centos6 ~]# umount /dev/test_vg/test_data
[root@centos6 ~]# umount /dev/test_vg/test_web
[root@centos6 ~# lvremove /dev/test_vg/test_data
Do you really want to remove active logical volume test_data? [y/n]: y
Logical volume "test_data" successfully removed
[root@centos6 ~]# lvremove /dev/test_vg/test_web
Logical volume test_vg/test_web contains a filesystem in use.
[root@centos6 Desktop]# lvremove /dev/test_vg/test_web
Do you really want to remove active logical volume test_web? [y/n]: y
Logical volume "test_web" successfully removed
[root@centos6 Desktop]# vgremove test_vg
Volume group "test_vg" successfully removed
[root@centos6 Desktop]# pvremove /dev/sdb{1,2,3,5,6}
Labels on physical volume "/dev/sdb1" successfully wiped
  Labels on physical volume "/dev/sdb2" successfully wiped
  Labels on physical volume "/dev/sdb3" successfully wiped
  Labels on physical volume "/dev/sdb5" successfully wiped
  Labels on physical volume "/dev/sdb6" successfully wiped
```

2.4.7　RAID 磁盘阵列概述

早在 1978 年，美国加州大学伯克利分校就提出了 RAID（Redundant Array of Independent Disk）虚拟存储系统。RAID 即独立冗余磁盘阵列，其思想是将多块独立的磁盘按不同的方式组合为一个逻辑磁盘，从而提高存储容量，提升存储性能或提供数据备份功能。RAID 存储系统的组合方式根据 RAID 级别定义。

RAID 分为软件 RAID 和硬件 RAID 两类。在现有的操作系统中，如 Windows、Linux、UNIX 等已经集成了软 RAID 的功能。软件 RAID 可以实现与硬件 RAID 相同的功能，但由于其没有

独立的硬件控制设备，所以性能不如硬件 RAID，但软件 RAID 实现简单，不需要额外的硬件设备。硬件 RAID 通常需要有 RAID 卡，RAID 卡本身会有独立的控制部件和内存，所以不会占有系统资源，效率高、性能强。当然目前市面上有很多主板已经集成了 RAID 卡，具体的使用方式可以参考硬件说明书。

现在 RAID 存储系统被广泛应用于生产环境作为存储解决方案。

2.4.8 RAID 级别

RAID 根据组合的方式不同，有多种设计解决方案，以下介绍几种常见的 RAID 方案（RAID 级别）。

1. RAID 0（不含校验与冗余的条带存储）

多块磁盘组合为 RAID 0 后，数据将被分割并分别存储在每块硬盘中，所以能最大地提升存储性能与存储空间，但无法容错，RAID 0 至少需要两块磁盘。

如图 2-6 所示，如果有一个文件要写入 RAID 0，则该文件会被分割为多个部分（图 2-6 中分割为四个数据块），DATA1 与 DATA2 被分别同步存入磁盘 1 与磁盘 2，其他部分依此类推。读取该文件时，将同时从磁盘 1 与磁盘 2 中读取数据。如果两块磁盘的存储空间均为 120GB，则 RAID 0 的总存储空间为 240GB。

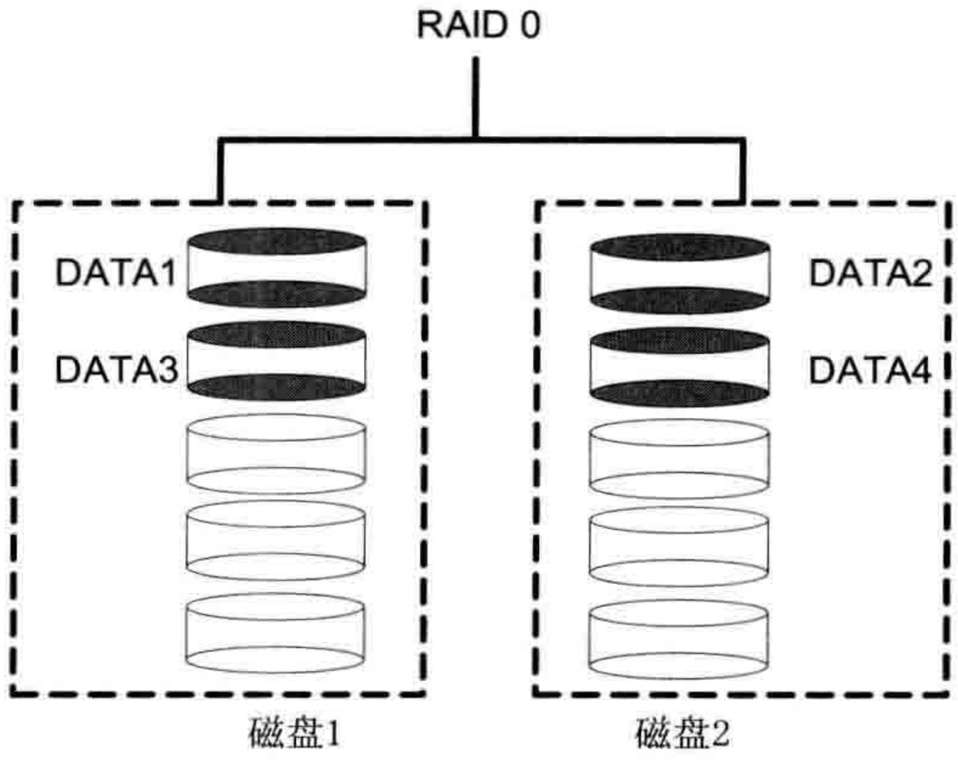


图 2-6

缺点：因为没有校验与备份，两块硬盘中如果有一块磁盘损坏，即磁盘中的任何一个数据块损坏将导致整个文件无法读取。

2. RAID 1（不含校验的镜像存储）

多块磁盘组合为 RAID 1 后，数据将被同时复制到每块硬盘中，制作这种磁盘阵列至少需要两块硬盘，该级别的 RAID 只要有一块磁盘可用，即可正常工作，该级别的阵列安全性是最好的，磁盘利用率是最低的。

如图 2-7 所示，如果有一个文件要写入 RAID 1，则该文件会写入磁盘 1，同时以复制备份的形式写入磁盘 2，DATA1 与 DATA2 被同时存入磁盘 1 与磁盘 2，其他部分依此类推。RAID 1 写入数据的效率会降低，因为相同数据需要同时写入两块硬盘，但 RAID 1 读取数据的效率会提升，因为可以同时从两块硬盘读取数据。

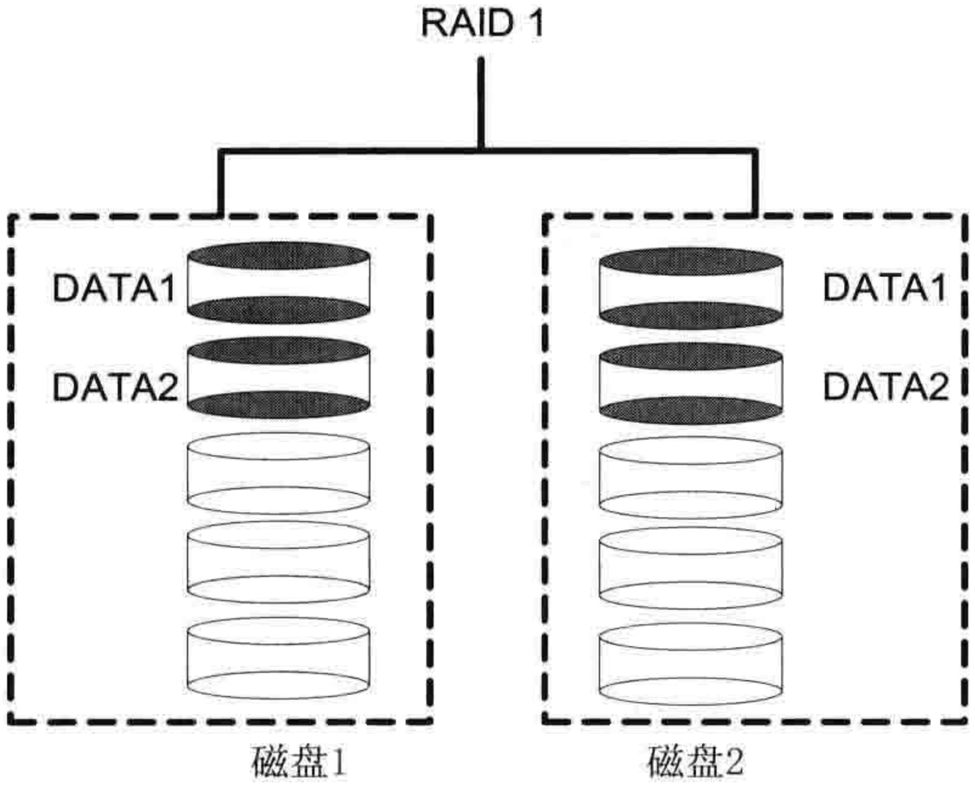


图 2-7

缺点：RAID 1 的写入效率低下。此外，磁盘的利用率极低，如果两块磁盘的存储空间均为 120GB，则 RAID 1 的总存储空间依然为 120GB。

3. RAID 2（位级别的校验式条带存储）

多块磁盘组合为 RAID 2 后，数据将以位（bit）为单位同步式分别存储在不同的硬盘上，并采用海明码（海明码是一种具有纠错功能的校验码）对数据进行校验与恢复。

如图 2-8 所示，如果有一个文件要写入 RAID 2，则该文件会被分成数据位分别同步写入不同的磁盘中，DATA1 与 DATA2 进行海明码运算后再写入校验盘中，其他部分依此类推。拥有校验位的 RAID 2 允许三块磁盘中的任何一块磁盘损坏，并能够对未损坏的两块磁盘进行运算后还原已损坏磁盘上的数据，从而实现数据恢复的功能。RAID 2 对大数据量的读写具有很高的性能，但对少量数据的读写时性能反而不好，该磁盘阵列至少需要三块硬盘。

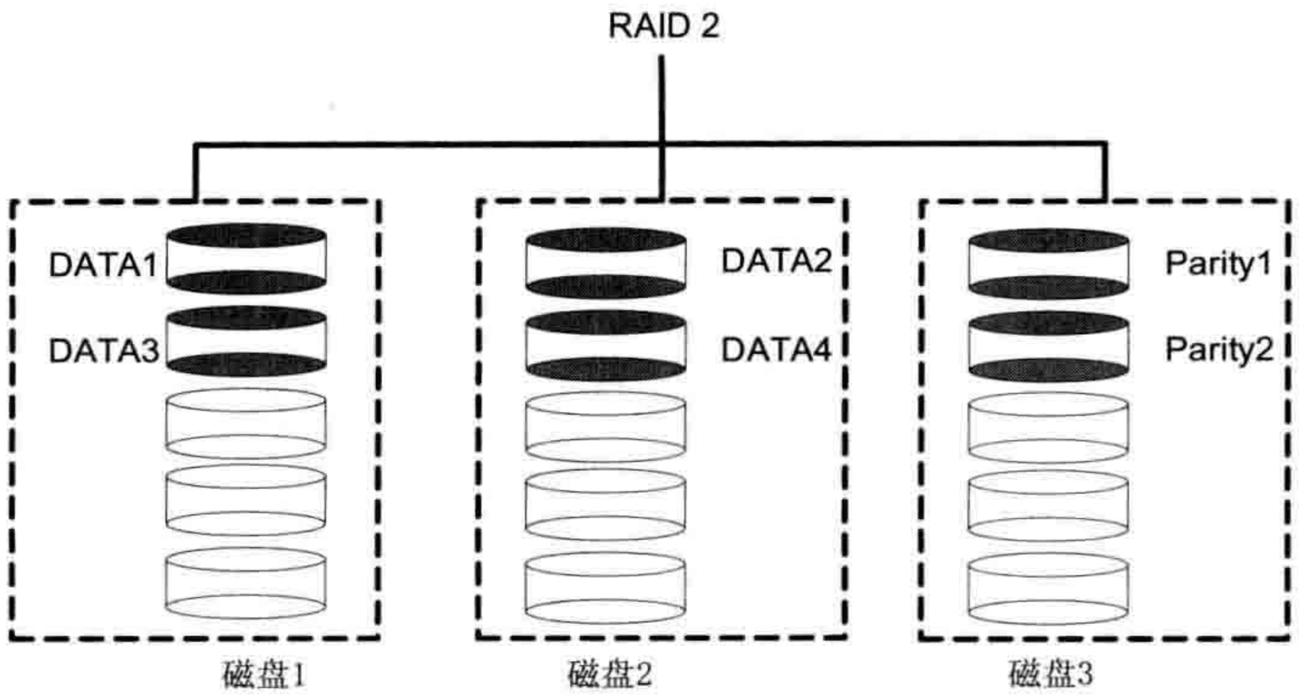


图 2-8

4. RAID 3（字节级别的校验式条带存储）

该级别的磁盘阵列可以参考 RAID 2 的读写原理，仅数据分割的单位为字节。

5. RAID 4（数据块级别的校验式条带存储）

该级别的磁盘阵列与 RAID 2 类似，仅数据分割单位不同，图 2-9 为 RAID 4 存储原理图。

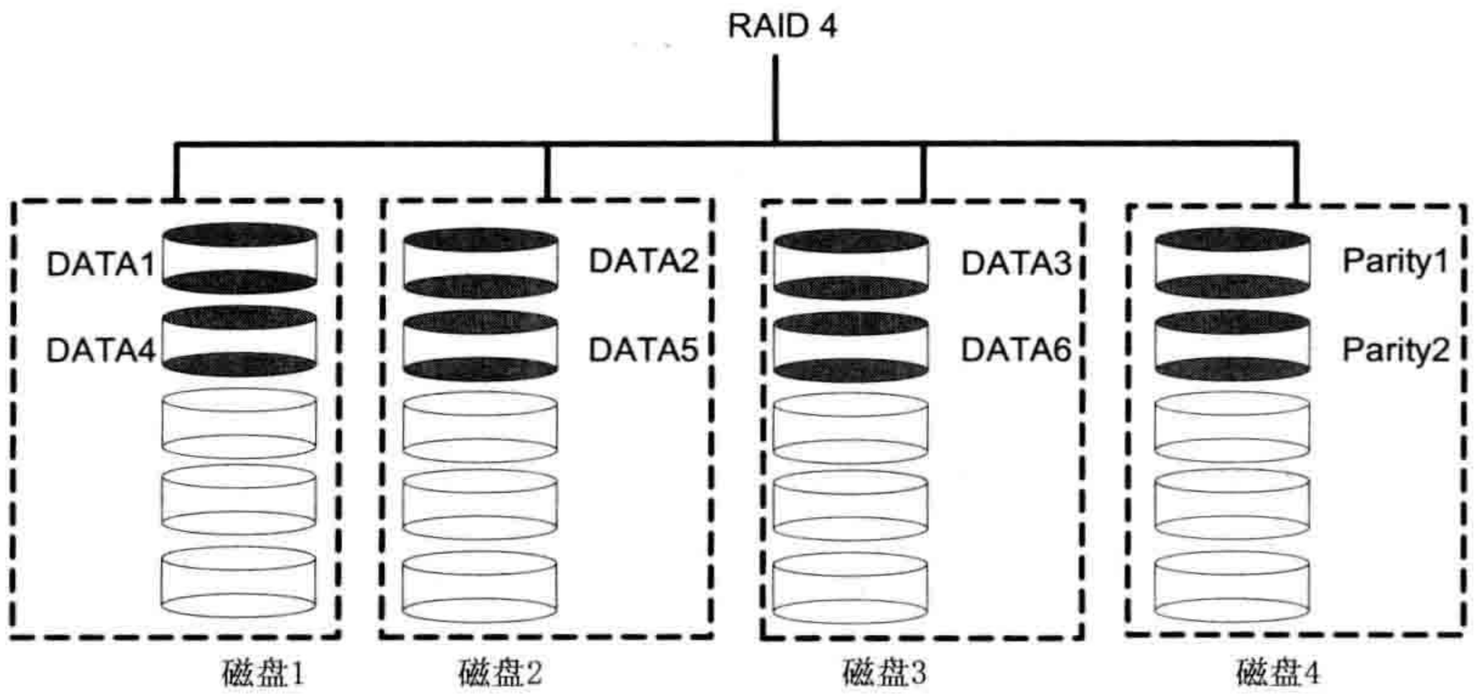


图 2-9

6. RAID 5（数据块级别的分布式校验条带存储）

多块磁盘组合为 RAID 5 后，数据将以块（block）为单位同步式分别存储在不同的硬盘上，并对数据进行海明码运算，与其他级别不同的是，该级别的海明码会被写入不同的磁盘。图 2-10

为 RAID 5 存储原理图。

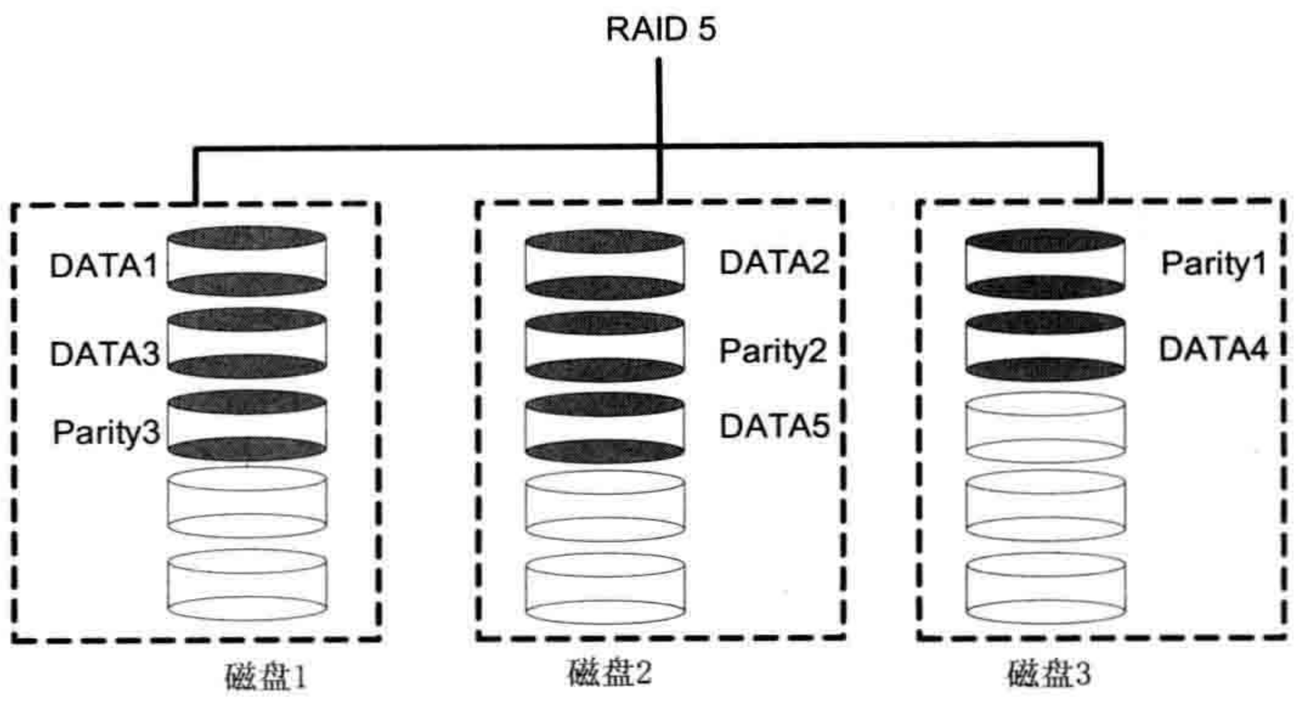


图 2-10

7. RAID 10（镜像与条带存储）

RAID 10 不是独创的一种 RAID 级别，它由 RAID 0 与 RAID 1 结合而成，RAID 10 继承了 RAID 0 的快速与高效，同时也继承了 RAID 1 的数据安全，RAID 10 至少需要四块硬盘。RAID 10 通常有两种结构，一种是 RAID 0+1 结构，一种是 RAID 1+0 结构。如果是 RAID 1+0，则先使用四块硬盘组合成两个独立的 RAID 1，然后将两个 RAID 1 再组合为一个 RAID 0。

如图 2-11 所示，当数据写入时，首先会以条带的方式同步写入由两个 RAID 1 组合而成的 RAID 0 中，随后，当数据被写入由具体磁盘组成的 RAID 1 时，又可以实现双磁盘镜像功能。

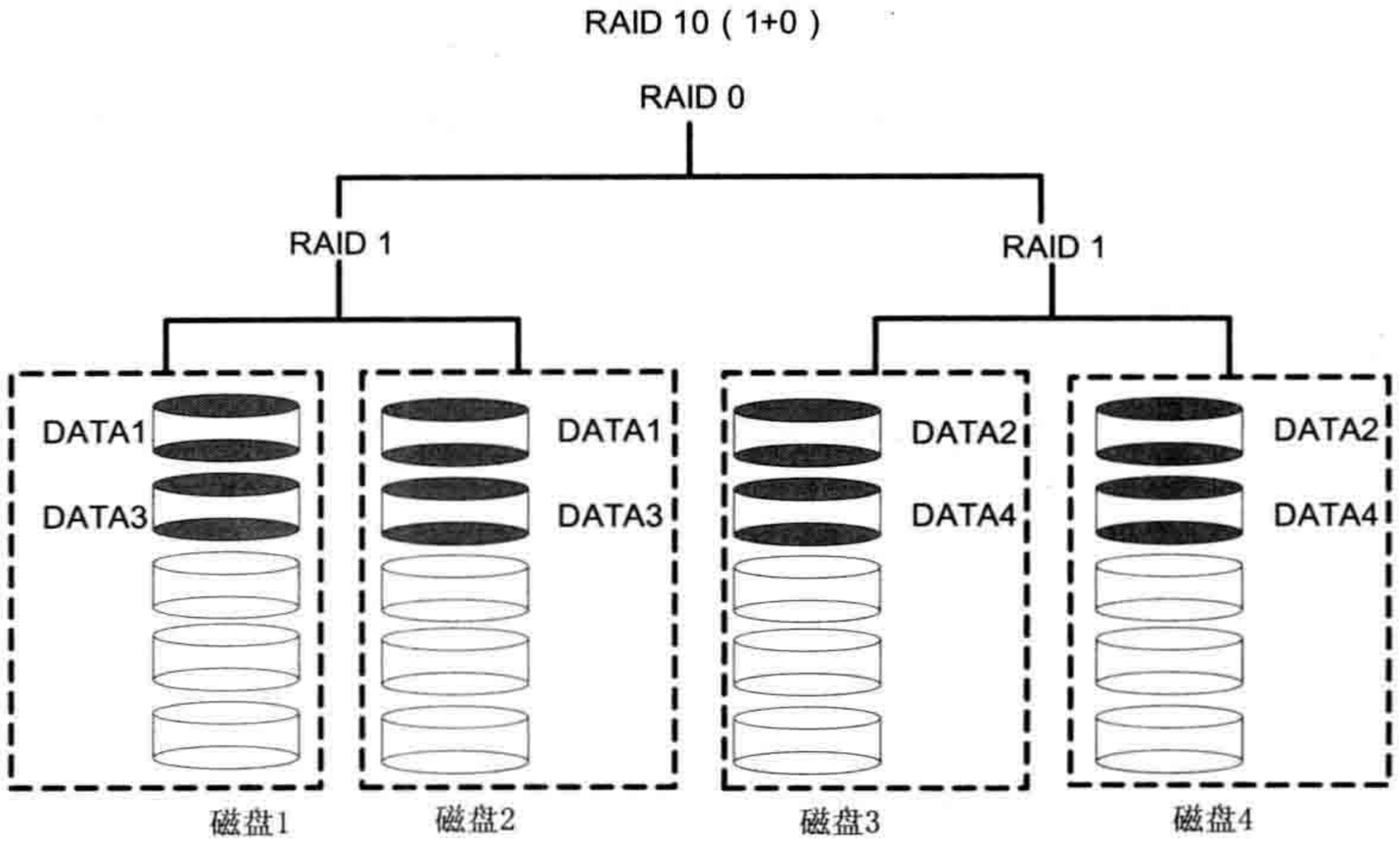


图 2-11

缺点：RAID 10 继承了 RAID 0 和 RAID 1 的优点，但同时也继承了一些缺点，RAID 10 的磁盘利用率很低，只有 50%。

2.4.9 创建与管理软 RAID 实例

创建与管理软 RAID 实例的步骤如下。

1. 查看磁盘信息

```
[root@localhost ~]# fdisk -cul
…部分内容省略…
Disk /dev/sdb: 2147 MB, 2147483648 bytes
255 heads, 63 sectors/track, 261 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk /dev/sdb doesn't contain a valid partition table

Disk /dev/sdc: 2147 MB, 2147483648 bytes
255 heads, 63 sectors/track, 261 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk /dev/sdc doesn't contain a valid partition table

Disk /dev/sdd: 2147 MB, 2147483648 bytes
255 heads, 63 sectors/track, 261 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Disk /dev/sdd doesn't contain a valid partition table
```

从以上输出信息可以看出，本机有 sdb、sdc、sdd 三块硬盘未分区，如果需要在 Linux 中实现软 RAID，可以将每块硬盘分一个区组建 RAID，也可以对硬盘分多个区实现软 RAID，这里我们将每块硬盘分两个区。

2. 创建硬盘分区

```
[root@localhost ~]# fdisk /dev/sdb
Command (m for help): n
Command action
   e   extended
   p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-261, default 1):
Using default value 1
```

```
Last cylinder or +size or +sizeM or +sizeK (1-261, default 261): +1G

Command (m for help): n
Command action
   e   extended
   p   primary partition (1-4)
p
Partition number (1-4): 2
First cylinder (124-261, default 124): (回车)
Using default value 124
Last cylinder or +size or +sizeM or +sizeK (124-261, default 261): (回车)
Using default value 261

Command (m for help): t
Partition number (1-4): 1
Hex code (type L to list codes): fd              #格式 fd 为 RAID 格式
Command (m for help): t
Partition number (1-4): 2
Hex code (type L to list codes): fd
Changed system type of partition 2 to fd (Linux raid autodetect)

Command (m for help): p
 Disk /dev/sdb: 2147 MB, 2147483648 bytes
255 heads, 63 sectors/track, 261 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
   Device Boot      Start         End      Blocks   Id  System
/dev/sdb1               1         123      987966   fd  Linux raid autodetect
/dev/sdb2             124         261     1108485   fd  Linux raid autodetect

Command (m for help): w
```

```
**********************************************
****      其他两块硬盘执行相同的操作!!!      ****
**********************************************
[root@localhost ~]# partprobe -a
```

3. 创建 RAID（本例将分别创建 RAID0 与 RAID5）

Linux 中创建磁盘阵列可以使用 mdadm 命令，在 CentOS 6.3 中，目前支持的 RAID 级别有 RAID0、RAID1、RAID4、RAID5、RAID6、RAID10。下面我们看看该命令的使用方法。

名称：mdadm

描述：管理 Linux 软件 RAID 设备。

用法：mdadm [模式] 选项 <raid 设备>

选项：-C　　创建软件 RAID（create）。
　　-l　　指定 RAID 级别（level）。
　　-n　　指定磁盘个数。
　　-x　　指定备用设备个数。

```
[root@localhost ~]# mdadm -C /dev/md0 -l 0 -n 3 /dev/sdb1 /dev/sdc1 /dev/sdd1
mdadm: array /dev/md0 started.
[root@localhost ~]# mdadm  -C  /dev/md1 -l 5 -n 3 -x 1 \
> /dev/sdb2  /dev/sdc2  /dev/sdd2  /dev/sde2
mdadm: array /dev/md1 started.
```

以上两条命令分别创建了名称为 md0 的 RAID 0 设备以及名称为 md1 的 RAID5 设备，这里的名称可以根据自己的习惯自行指定。

刚刚创建的 RAID 0 及 RAID 5 的详细信息如下。

```
[root@localhost ~]# mdadm --detail  /dev/md0    #查看 md0 信息
/dev/md0:
        Version : 0.90
  Creation Time : Thu May 17 03:46:55 2012      #创建时间
     Raid Level : raid0                         #RAID 级别
     Array Size : 2963520 (2.83 GiB 3.03 GB)    #RAID 磁盘空间
   Raid Devices : 3                             #磁盘个数
  Total Devices : 3
Preferred Minor : 0
    Persistence : Superblock is persistent
    Update Time : Thu May 17 03:46:55 2012
          State : clean
 Active Devices : 3                             #活动磁盘个数
Working Devices : 3                             #工作磁盘个数
 Failed Devices : 0                             #错误磁盘个数
  Spare Devices : 0                             #备用磁盘个数
     Chunk Size : 64K
           UUID : 62e9bd3e:f4dcf02d:319e2a62:8099eb72   设备 UUID
         Events : 0.1
    Number   Major   Minor   RaidDevice State
       0       8       17        0      active sync   /dev/sdb1
       1       8       33        1      active sync   /dev/sdc1
       2       8       49        2      active sync   /dev/sdd1
[root@localhost ~]# mdadm --detail  /dev/md1   #查看 md1 信息
/dev/md1:
(…中间部分省略…)
    Number   Major   Minor   RaidDevice State
       0       8       18        0      active sync   /dev/sdb2
```

```
       1       8       34       1      active sync   /dev/sdc2
       2       8       50       2      active sync   /dev/sdd2

       3       8       66       -      spare   /dev/sde2     #该磁盘为备用磁盘
```

4. 格式化与挂载

```
[root@localhost ~]# mkfs.ext4 /dev/md0
[root@localhost ~]# mkdir /raid0
[root@localhost ~]# mount  /dev/md0 /raid0
[root@localhost ~]# mkfs.ext4 /dev/md1
[root@localhost ~]# mkdir /raid5
[root@localhost ~]# mount /dev/md1 /raid5
```

将磁盘阵列设备设置为开机自动挂载项，CentOS 系统开机会读取磁盘阵列的配置文件 /etc/mdadm.conf 以启动 RAID 设备。下面两条命令可以帮助我们快速创建这样的配置文件，DEVICE 行代表填写所有磁盘阵列的设备成员名称，ARRAY 行描述磁盘阵列具体的基本信息，包括名称、级别、UUID 等信息。

```
[root@localhost ~]#echo "DEVICE /dev/sdb1 /dev/sdb2 \
/dev/sdc1 /dev/sdc2 /dev/sdd1 /dev/sdd2 /dev/sde2" >/etc/mdadm.conf
[root@localhost ~]# mdadm -Evs >>/dev/mdadm.conf
ARRAY /dev/md/0 level=raid0 metadata=1.2 num-devices=3 UUID=4282a2c2:
2591cc4d:a588f799:88123b79 name=centos6.example.com:0
   devices=/dev/sdd1,/dev/sdc1,/dev/sdb1
ARRAY /dev/md/1 level=raid5 metadata=1.2 num-devices=3 UUID=d6cc7bdc:
6bc3a612:969239d8:7a78a301 name=centos6.example.com:1
   spares=1   devices=/dev/sde2,/dev/sdd2,/dev/sdc2,/dev/sdb2
[root@localhost ~]# echo "/dev/md0 /raid0 ext4 defaults 0  0" >> /etc/fstab
[root@localhost ~]# echo "/dev/md1 /raid5 ext4 defaults 0  0" >> /etc/fstab
```

2.4.10　RAID 性能测试

```
**************************
**  普通磁盘：写入模拟  **
**************************
[root@localhost ~]# time dd if=/dev/zero of=txt bs=1M count=1000
1000+0 records in
1000+0 records out
1048576000 bytes (1.0 GB) copied, 21.7965 seconds, 48.1 MB/s
real    0m23.262s
user    0m0.001s
sys     0m2.209s
```

可以看出，对普通磁盘写入 1GB 的数据所需总时间为 23.262s。

```
**************************
**  RAID 0：写入模拟   **
**************************
[root@localhost raid0]# time dd if=/dev/zero of=txt bs=1M count=1000
1000+0 records in
1000+0 records out
1048576000 bytes (1.0 GB) copied, 3.87193 seconds, 271 MB/s
real    0m4.308s
user    0m0.001s
sys     0m1.604s
```

可以看出，对 RAID 0 写入 1GB 数据所需总时间为 4.308s。

```
**************************
**   RAID 5：写入模拟   **
**************************
[root@localhost raid5]# time dd if=/dev/zero of=txt bs=1M count=1000
1000+0 records in
1000+0 records out
1048576000 bytes (1.0 GB) copied, 12.5047 seconds, 83.9 MB/s
real    0m12.614s
user    0m0.004s
sys     0m3.705s
```

可以看出，由于 RAID 5 需要生成校验位数据，所以写入数据的速度比 RAID 0 慢，但比普通磁盘快，写入 1GB 数据所需总时间为 12.614s。

注意 以上测试仅以软 RAID 为例。

2.4.11 RAID 故障模拟

```
[root@localhost raid5]# mdadm /dev/md1 -f /dev/sdb2
mdadm: set /dev/sdb2 faulty in /dev/md1
```

注释：使用-f 选项（failed）模拟磁盘 sdb2 的损坏。

由于上面做性能测试时已经往 RAID 5 中写入了 1GB 的数据，所以使用命令模拟磁盘损坏后，快速查看 RAID 详细信息可以看出正在使用/dev/sde2 重建数据（还原数据），而原来的/dev/sdb2 成了损坏的空闲设备。

```
[root@localhost ~]# mdadm --detail /dev/md1
... ... ... ... ... ... ...
... ... ... ... ... ... ...
```

```
... ... ... ... ... ... ...
   Number   Major   Minor   RaidDevice State
      3       8       66        0      spare rebuilding   /dev/sde2
      1       8       34        1      active sync   /dev/sdc2
      2       8       50        2      active sync   /dev/sdd2
      4       8       18        -      faulty spare   /dev/sdb2
```

2.5　软件管理

2.5.1　Linux 常用软件包类型

Linux 中的软件包类型种类就像 Linux 发行版本一样丰富多样，但这种多样性也给用户带来了不少烦恼，终端用户需要考虑哪个软件包的格式适用于相应的 Linux 发行版本，因为很多特定的软件包格式仅适用于特定的发行版本。目前比较流行的软件包格式有：可直接执行的 RPM 与 DEB、源代码形式的 gzip 与 bzip2 压缩包。

2.5.2　RPM 软件包管理

RPM 是 RedHat Package Manager 的简写，也即是红帽软件包工具。RPM 格式的软件包最早在 1997 年被用在红帽的操作系统上，RPM 设计思路是提供一种可升级、具有强大查询功能、支持安全验证的通用型 Linux 软件包管理工具。现在 RPM 软件包已经被应用到很多 GNU/Linux 发行版本中，包括 Red Hat Enterprise Linux、Fedora、Novell SUSE Linux Enterprise、openSUSE、CentOS、Mandriva Linux 等。CentOS 6.3 光盘中的所有软件包均采用 RPM 格式。

1. 安装与卸载 RPM 包

对于 RPM 软件包的安装，我们可以使用<rpm -i 软件包名称>这样的方式进行安装，但该命令默认选项为静默式安装，如果需要了解安装过程的信息，则需要使用-v 来显示详细的信息，-h 显示安装进度（安装进度以#符号标识）。

这里我们安装 CentOS 6.3 光盘中的 ftp-0.17-51.1.el6.x86_64.rpm 软件：

```
[root@centos6 Packages]# rpm -vih ftp-0.17-51.1.el6.x86_64.rpm
warning: ftp-0.17-51.1.el6.x86_64.rpm: Header V3 RSA/SHA256 Signature, key
ID c105b9de: NOKEY
Preparing...        ########################################### [100%]
   1:ftp            ########################################### [100%]
```

如果安装时提示：package ftp-0.17-51.1.el6.x86_64 is already installed，则表示该软件包已经被安装。

如果安装时提示：libenchant.so.1()(64bit) is needed by evolution-2.28.3-24.el6.x86_64，则表示安装 evolution 软件需要依赖 libenchant.so.1 文件，此时我们需要先安装相应的依赖软件包后才可以成功安装 evolution 软件。

对于 RPM 格式软件包的卸载，可以使用<rpm -e 软件包名称>这样的格式完成。

```
[root@centos6 Packages]# rpm -e ftp      #卸载 ftp 软件，具体版本号可以不写
```

RPM 还提供了升级功能，使用<rpm -U 软件名称>这种方式，rpm 会删除旧版本软件（仅保留配置文件），再安装新的软件版本。

2. 软件包信息查询

RPM 命令工具提供了强大的软件查询功能，使用<rpm -q >这样的格式可以进行大量的查询工作。查询时可以使用很多选项，具体描述如下。

-q 查询指定软件包是否已经安装，如果已安装则显示详细名称，否则显示 package xxx[4] is not installed。

-qa 查询系统中已经安装的所有软件。

-qi 查询指定安装软件包的详细信息。

-ql 查询指定软件的安装路径与文件列表。

-qc 查询指定软件的配置文件。

-qf 查询指定文件为哪个软件所安装。

```
[root@centos6 ~]# rpm -q ftp
[root@centos6 ~]# rpm -qa
[root@centos6 ~]# rpm -qi bash
[root@centos6 ~]# rpm -ql bash
[root@centos6 ~]# rpm -qc bash
[root@centos6 ~]# rpm -qf /etc/inittab
```

使用 rpm -q 格式进行的查询仅针对已经安装的软件，如果你希望知道某个尚未安装的软件包的详细信息，则可以使用 rpm -qp 进行查询，如 rpm -qpi ftp-0.17-51.1.el6.x86_64。

[4] xxx 为指定查询的软件名称，根据查询内容而变。

3. 安全验证

RPM 提供的验证功能可以随时追踪软件的状态，当软件被人篡改时，使用 rpm -V 命令就会提示该软件的哪些状态被修改，未被篡改的软件使用该命令将无任何提示。

```
[root@centos6 ~]# rpm -V bash
```

下面列出所有可能出现的提示字符及其含义。

- 5 —— MD5 值已经改变。
- S —— 文件大小已经改变。
- L —— 链接文件的源已经改变。
- T —— 文件最后修改时间已经改变。
- D —— 设备改变。
- U —— 用户发生改变。
- G —— 组发生改变。
- M —— 包括权限与类型在内的模式已经改变。
- ? —— 不可读的文件。

2.5.3　使用 YUM 安装软件包

YUM 是 Yellow dog Update Modified 的简写形式。YUM 是改进版的 RPM 软件管理器，YUM 很好地解决了 RPM 所面临的软件包依赖问题，大量的软件依赖会让 RPM 成为恶魔。YUM 可以从很多源中搜索软件以及它们的依赖包，并自动安装相应的依赖软件。使用 YUM 安装软件时至少需要一个 YUM 源，YUM 源就是存放有很多 RPM 软件的文件夹，用户可以使用 HTTP、FTP 或本地文件夹的方式访问 YUM 源。

1. 设置 YUM 源

CentOS 6.3 默认的 YUM 源定义文件存放在/etc/yum.repos.d/目录下，用户可以自行定义任意可以使用的 YUM 源，但文件名的扩展名必须是 repo，其文件格式如表 2-7 所示。

表 2-7

选项	功能描述
[]	[]中填写 YUM 源唯一的 ID，可以为任意字符串
name	指定 YUM 源名称，可以为任意字符串

续表

选项	功能描述
baseurl	指定 YUM 源的 URL 地址（可以是 HTTP、FTP 或本地路径）
mirrorlist	指定镜像站点目录
enabled	是否激活该 YUM 源（0 代表禁用，1 代表激活，默认为激活）
gpgcheck	安装软件时是否检查签名（0 代表禁用，1 代表激活）
gpgkey	检查签名的密钥文件

查看 CentOS 6.3 默认的 YUM 源/etc/yum.repos.d/CentOS-Base.repo 的内容如下：

```
[base]
name=CentOS-$releasever - Base
mirrorlist=http://mirrorlist.centos.org/?release=$releasever&arch=$basea
rch&repo=os
#baseurl=http://mirror.centos.org/centos/$releasever/os/$basearch/
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-CentOS-6
(…部分内容省略…)
```

下面我们通过创建 media.repo 文件，使用 CentOS 6.3 系统光盘作为 YUM 源，文件内容如下：

```
[media]
name=CentOS-6.3
baseurl=file:///media/CentOS_6.3_Final
enabled=1
gpgcheck=0
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-CentOS-6
```

2. YUM 命令

描述：交互式，基于 RPM 的包管理工具。

用法：yum [选项] [指令] [软件包…]

选项：-y 执行非交互式安装，假定安装过程中出现的所有提示回答为 YES。

指令：		
	install package1 [package2] [...]	使用 yum 源安装软件包。
	update [package1] [package2] [...]	使用 yum 源升级软件包。
	check-update	检查 yum 源中所有可用的升级。
	remove | erase package1 [package2] [...]	卸载软件包。
	list [...]	列出系统中已经安装的以及 yum 源中所有可用的软件包。

info [...]	查看软件包信息。
clean all	清空所有的缓存信息。
groupinstall group1 [group2] [...]	使用 yum 源安装组包。
grouplist	列出系统中已经安装的以及 yum 源中所有可用的组包。
groupremove group1 [group2] [...]	卸载组包。
search string1 [string2] [...]	根据关键词查找软件。
localinstall rpmfile1 [rpmfile2] [...]	通过本地 rpm 文件安装软件。
history	查看历史记录。

实例：

```
[root@centos6 ~]# yum clean all          #清空 YUM 缓存
[root@centos6 ~]# yum install dialog
[root@centos6 ~]# yum -y expect          #非交互式安装
[root@centos6 ~]# yum update             #检查所有的软件并更新
[root@centos6 ~]# yum remove dialog
[root@centos6 ~]# yum list               #已经安装的软件包使用@标记
[root@centos6 ~]# yum clean all
[root@centos6 ~]# yum groulist
[root@centos6 ~]# yum groupinstall "Chinese Support"   #安装中文组包
[root@centos6 ~]# yum search web         #查找 Web 相关软件
[root@centos6 ~]# yum history
```

2.5.4　YUM 使用技巧

1. 创建 YUM 源

当我们收集的软件越来越多时，有必要将这个软件汇总并创建属于自己的 yum 源，最终实现软件的高效、集中管理。实现步骤如下。

首先，需要安装 createrepo。

其次，将所有的软件包保存在某个目录下，如/mysoft/。

最后，运行 createrepo /mysoft 即可。

2. YUM 变量

有时你会在一个 yum 文件中看到类似$basearch 这样的变量，以下给出了可以使用或参考的

yum 命令和 Yum 配置文件内置的变量说明。

$releasever	代表系统发行版本号
$arch	代表 CPU 架构
$basearch	代表系统架构
$YUM0-9	使用 Shell 对应的 0~9 个同名变量替换

2.5.5 源码编译安装软件

软件的源代码是软件的原始数据，任何人都可以通过源代码查看该软件的设计架构与实现方法，但软件源代码不可以在计算机中直接运行安装。我们需要通过配置软件功能，然后将软件源代码通过编译转换为计算机可以识别的机器语言，最后才可以执行安装操作。不同于 RPM 或其他的二进制软件包的安装方式，通过源码编译安装的软件，使得安装者可以根据自己的需要定制自己的软件，这是实际工作中所需要的关键因素。

通过源码安装软件的具体步骤如下。

（1）我们所获得的软件源码一般都会被打包并压缩，常见的格式有.tar.gz 或.tar.bz2 格式。此时，我们需要使用前面所学的 tar 命令将压缩包解压，具体参数及功能可以参考 2.1.4 节“压缩及解压”的内容。

（2）运行 configure 脚本，通过特定的选项修改软件设置与功能，该脚本一般可以用来指定安装路径、开启关闭软件的特定功能等。脚本选项可以通过阅读安装说明或./configure --help 查看软件支持的功能选项。而有些软件没有提供 configure 脚本，则可以直接执行 make 编译步骤。

（3）运行 make 命令将软件源代码编译为计算机可以直接识别的机器语言。由于需要相应的编译软件才可以将源代码编译为机器语言，所以在执行 configure 脚本时就会检查对应的开发工具是否已安装，常用的开发工具有：gcc、python、perl、make、automake 等，如果没有安装，则提示 error 错误。

（4）通过 make install 根据配置阶段指定的路径和功能将软件以特定的方式安装到指定位置。

接下来从网络上下载 Apache 软件作为演示案例。Apache 是当今最流行的 Web 服务器软件，由于 Apache 需要依赖 APR 库等软件，所以我们连同相关依赖包一起安装。

```
[root@centos6 ~]# yum -y install gcc pcre pcre-devel libtool
[root@centos6 ~]# wget http://mirror.bit.edu.cn/apache/apr/apr-1.4.6. tar.gz
[root@centos6 ~]# wget http://mirror.bit.edu.cn/apache/apr/apr-util-1.5.1. tar.gz
[root@centos6 ~]# wget http://labs.mop.com/apache-mirror/httpd/httpd-2.4.3.tar.bz2
```

```
[root@centos6 ~]# tar -xjf httpd-2.2.23.tar.bz2 -C /usr/src/
[root@centos6 ~]# tar -xzf apr-1.4.6.tar.gz -C /usr/src/
[root@centos6 ~]# tar -xzf apr-util-1.5.1.tar.gz -C /usr/src/
[root@centos6 ~]# cd /usr/src/apr-1.4.6/
[root@centos6 ~]# ./configure && make && make install
[root@centos6 ~]# cd /usr/src/apr-util-1.5.1/
[root@centos6 ~]# ./configure --with-apr=/usr/local/apr/
[root@centos6 ~]# make && make install
[root@centos6 ~]# cd /usr/src/httpd-2.2.23/
[root@centos6 ~]# ./configure
[root@centos6 ~]# make
[root@centos6 ~]# make install
[root@centos6 ~]# /usr/local/apache2/bin/apachectl start   #开启 Web 服务
```

提示 configure 的具体选项可以通过./configure --help 了解详情，每个软件支持的选项功能不尽相同，apache 的具体配置在后面会详细讲解。

现在你可以通过浏览器输入如下本地 IP 地址浏览默认测试页面，如果一切正常，你会看到浏览器显示“It works!”。

```
[root@centos6 ~]# firefox http://127.0.0.1
```

2.5.6　常见问题分析

1. 软件包依赖问题

使用 RPM 安装时，如果系统提示软件之间有相互依赖关系，但当没有相互依赖包的情况下，则可以使用--nodeps 选项来忽略依赖关系，在此并不提倡用该选项来安装软件。此外，使用 RPM 卸载软件时，有时也会提示其他软件依赖于正要卸载的软件，一般提示如：libpcre.so.0()(64bit) is needed by (installed) grep-2.6.3-3.el6.x86_64 这样的信息，同样可以使用--nodeps 选项忽略依赖关系。

2. RPM 数据库损坏

RPM 软件包的相关数据库存放在/var/lib/rpm/目录下，如果数据库出现损坏，可以用 rpm --rebuilddb 修复数据库资料。

3. 软件安装的时间问题

在安装软件时，系统有时会提示“warning:clock skew detected.”错误，这说明系统时间发生了严重的错误，可以通过 date –s "2012-11-12 14:00"这种命令格式修改系统时间，并通过命令 hwclock -w 更新写入 CMOS 时间。

4. YUM 繁忙问题

当我们使用 yum 命令进行安装、查询时，系统有时会提示如下信息：

```
Loaded plugins: fastestmirror, refresh-packagekit, security
Existing lock /var/run/yum.pid: another copy is running as pid 16613.
Another app is currently holding the yum lock; waiting for it to exit...
  The other application is: yum
    Memory :  23 M RSS (904 MB VSZ)
    Started: Tue Jan 22 16:30:27 2013 - 00:09 ago
    State  : Sleeping, pid: 16613
```

该提示说明有另外一个程序在使用 YUM 而导致了 YUM 被锁。有时候系统在后台进行自动升级时就会提示该信息，如果你确实想终止该 YUM 程序，可以使用 kill 命令杀死提示信息中的 pid 号，上面的提示信息说明 pid 为 16613 的进程正在使用 YUM，执行 kill 16613 即可终止该进程。

5. GCC 编译器问题

使用源码安装软件的时候最常出现的就是相关依赖包没有安装的错误，一般会在运行 configure 脚本阶段检查并提示错误，经常由于没有安装 gcc 而报错，而很多软件需要 gcc 编译才可以安装，解决方法是使用 yum 安装 gcc 后，再次执行 configure 脚本即可。

2.5.7 服务管理

服务一般是放置在后台运行的进程，有些是系统服务，有些则是独立的网络服务。对于这些服务，我们可以通过运行主程序来启动或关闭服务，也可以通过系统提供的 service 以及 chkconfig 命令来管理大量的服务，下面介绍这些不同的服务管理方法。

1. 通过主程序管理服务

我们可以通过手动执行主程序来启动服务，例如，手动启动 Apache 仅需要知道 Apache 主

程序的存储路径即可，我们可以通过执行/usr/local/apache2/bin/apachectl start 开启服务，并通过执行/usr/local/apache2/bin/apachectl stop 来关闭服务。只要我们知道主程序的路径，就可启动相应的进程服务。绝大多数通过源码安装的软件都需要我们手动指定路径启动服务。

2. 通过 service 管理服务

如果是通过 RPM 或者 YUM 安装，则应用程序一般会在/etc/init.d/目录下创建对应的启动脚本，我们可以通过系统提供的 service 命令来管理这些服务。下面我们通过 service 来管理 ssh 远程连接服务。

```
[root@centos6 ~]# service sshd start        #开启 sshd 服务
[root@centos6 ~]# service sshd stop         #关闭 sshd 服务
[root@centos6 ~]# service sshd status       #查看服务当前状态
```

有时修改了服务的配置后，在不想关闭服务的情况下使新的配置生效，还可以通过 reload 参数来重新加载配置文件，甚至当你修改完配置文件后不知道新的设置是否正确，或者重启服务后是否会导致服务无法启动，可以使用 condrestart 选项，此时系统会测试新的配置文件是否有问题，没问题的情况下才会重启，否则不会使用新的配置来重启服务。

```
[root@centos6 ~]# service sshd reload
[root@centos6 ~]# service sshd condrestart
```

3. 开机启动服务

由于 Linux 是服务器版的操作系统，所以我们需要将很多服务设置为开机启动，比如，我们希望计算机开机便可以提供 Web 网站服务、FTP 下载服务、NFS 共享服务等。需要事先声明的是，Linux 操作系统正常情况下可以在六种模式（4 暂时未使用）下启动并分别用数字表示：0 代表关机模式、1 代表单用户模式、2 代表无 NFS 网络的字符界面模式、3 代表全功能的字符界面模式、4 暂时未定义使用、5 代表图形模式、6 代表重启模式。在系统中，你可以通过 runlevel 命令查看当前所处的运行级别，也可以通过 init n[5]来改变当前的运行级别。

在 CentOS 6.3 系统中可以通过 chkconfig 命令来管理开机启动项，但设置开机启动时需要指定运行级别，chkconfig 命令说明如下。

描述：更新与查询系统服务的运行级别信息。
用法：chkconfig 指令 [服务名称] [on|off]
指令：--list　　查询系统服务运行级别信息。

[5] 这里的 n 代表数字，可以是 0 至 6 的任意数字。

--level　　指定操作的运行级别，不指定级别时默认为 2345 级别。

示例：

```
[root@centos6 ~]# chkconfig --list sshd
                                        #查看 sshd 服务所有的运行级别信息[6]
[root@centos6 ~]# chkconfig --list
                                        #查看所有系统服务的运行级别信息
[root@centos6 ~]# chkconfig --level 15 sshd off
                                        #设置 sshd 服务在 1、5 级别开机不启动
[root@centos6 ~]# chkconfig sshd on
                                        #设置 sshd 服务在 2、3、4、5 级别开机启动
```

2.6　计划任务

作为一名运维人员，你经常需要将某些命令或脚本放入计划任务中去执行。例如，服务器在白天的访问量一般较大，而服务器在承受着巨大访问压力的同时对其进行全备份是不合适的，这时就可以考虑将备份工作放入系统计划任务，这样系统可以在夜间访问量小的时候自动执行备份任务。又或者，你需要定期执行服务器之间的数据同步操作等，往往都需要使用到计划任务的功能，而 Linux 系统为我们准备了两种计划任务，一种是只会执行一次 at 计划任务，一种是可以周期性地执行的 cron 计划任务。

2.6.1　at 一次性计划任务

使用 at 指定一次性计划任务前需要确保 atd 服务是开启的，否则计划任务不会被执行，使用 service atd start 开启服务，并使用 chkconfig atd on 确保该服务开机启动。

at 命令的作用和用法如下。

描述：在指定的时间执行特定命令。

用法：at 时间

选项：-m　　当计划任务执行结束后发送邮件给用户。

-l　　查看用户计划任务。

-d　　删除用户计划任务。

-c　　查看 at 计划任务具体内容。

---

[6] on 表示开机启动，off 表示开机不启动。

示例：

```
[root@centos6 ~]# at 23:11                    #指定将在当天 23 点 11 分执行的计划任务
at> tar -cjf log.tar.bz2 /var/log             #计划任务内容
at> shutdown -h now                           #计划任务内容
at> <EOT>                                     #可以输入多条命令，输入完毕后按 Ctrl+D 快捷键结束
job 1 at 2013-01-24 23:11                     #系统提示有编号为 1 的计划任务
[root@centos6 ~]# at -l                       #查看计划任务
[root@centos6 ~]# at -c 1                     #查看编号为 1 的计划任务的具体内容
[root@centos6 ~]# at -d 1                     #删除编号为 1 的计划任务
```

at 命令可以使用的时间格式有很多，at 小时:分钟（默认代表当天的时间），at 4pm + 3 days（代表 3 天以后的下午 4 点执行计划任务），at 12:00 2014-12-12（指定年月日及日期的计划任务），具体可以参考 at 帮助手册。

2.6.2　cron 周期性计划任务

使用 cron 指定计划任务前需要确保 crond 服务是开启的，否则计划任务不会被执行，使用 service crond start 开启服务，并使用 chkconfig crond on 确保该服务开机启动。

crontab 命令的作用和用法如下。

描述：为每个用户维护周期性的计划任务文件。

用法：crontab [-u 用户] [-l|-r|-e]

选项：-u　　指定计划任务的用户，默认为当前用户。

　　-l　　查看计划任务。

　　-r　　删除计划任务。

　　-e　　编辑计划任务。

　　-i　　使用-r 删除计划任务时，要求用户确认删除。

用户的 cron 计划任务文件格式含义如表 2-8 所示。使用 24 小时制时，分钟范围为 00~59，小时范围为 00~23，日期范围为 1~31，月范围为 1~12，周范围为 0~7（其中 0 或 7 都可以表示周日），如果需要指定的是时间段，可以使用横杠（-）表示一段连续的时间，使用逗号（,）表示若个不连续的时间，使用星号（*）表示所有的时间，使用除号（/）表示间隔时间。

表 2-8

第一列	第二列	第三列	第四列	第五列	第六列
分	时	日	月	周	命令

示例：

```
[root@centos6 ~]# crontab -e                          #为当前用户设置计划任务
23 23 * * 5   tar -czf log.tar.bz2 /var/log   #每周五晚 23 点 23 分执行日志备份
00 */3 * * *  who                                     #每 3 小时的整点检查用户登录情况
00 10 * * 3,5  free |mail -s "Mem" admin@xx.com
```

#每周三、周五 10 点将系统内存信息发送给自己的邮箱

#完成后退出方式同 vim

其实，CentOS 系统自定义了很多计划任务脚本，分别存放在/etc/下的 cron.hourly、cron.daily、cron.weekly 等目录中，如果你有脚本需要每天执行，可以直接将脚本存放在 cron.daily 目录中，系统会自动帮你每天运行该脚本。

2.6.3 计划任务权限

为了控制用户随意定义自己的计划任务，管理员可以进行 ACL 访问控制，at 计划任务的控制文件分别为/etc/at.allow 和/etc/at.deny，默认 at.allow 不存在。cron 计划任务的控制文件分别为/etc/cron.allow 和/etc/cron.deny，默认 cron.allow 不存在。

在这些控制文件中仅需要写入用户名即可，格式为一行一个用户名，当 allow 文件存在时，仅在 allow 文件中出现的用户可以使用对应的计划任务，如果 allow 文件与 deny 文件同时存在内容一样的账户，则仅出现在 allow 中的用户可以使用计划任务。如果没有 allow 文件而仅有 deny 文件，则所有出现在 deny 中的用户无法使用计划任务，其他所有的用户均可使用计划任务。当非法用户使用 crontab -e 编辑自己的计划任务时，系统将提示类似如下的内容：

```
You (tom) are not allowed to use this program (crontab)。
```

2.7 性能监控

2.7.1 监控 CPU 使用情况——uptime 命令

该命令的描述为：打印当前时间，系统已经运行了多久，当前登录用户数以及系统平均负载。

```
[root@centos6 ~]# uptime
15:31:07 up 240 days, 32 min,  1 user,  load average: 0.04, 0.03, 0.08
```

从上面的信息可以看出，当前系统时间为 15 点 31 分，系统已经运行了 240 天 32 分钟，当前有一个用户在登录，CPU 负载分别显示的是最近一分钟、五分钟、十五分钟的负载情况。需要说明的是，这里的负载表示单位时间段内 CPU 等待队列中平均有多少进程在等待，等待的进程个数越多，说明 CPU 越忙。

2.7.2　监控内存及交换分区使用情况——free 命令

该命令的描述为：显示系统内存及交换分区信息。

用法：free [-b|-k|-m]

选项：-b|-k|-m　　指定输出容量的单位，分别为 Byte、KB、MB。

```
[root@centos6 ~]# free
             total       used       free     shared    buffers     cached
Mem:       1025640    1006440      19200          0     112120     729044
-/+ buffers/cache:                             165276     860364
Swap:      2706424          0    2706424
```

提示

Linux 在开机后会预先提取一部分内存，并划分为 buffer 与 cache 以后随时提供给进程使用。

以上输出信息中，Mem 一行的 total 代表内存总量为 1025640 Byte；used 代表系统将内存中的 1006440 Byte 划分成了 buffer 与 cache，也就是 buffer 与 cache 的总容量；free 代表内存总容量减去 buffer 与 cache 的总和之后的剩余容量为 19200 Byte；buffers 代表当前 buffer 的剩余容量为 112120 Byte；cached 代表当前 cache 的剩余容量为 729044 Byte。

第二行 used 代表 buffer 与 cache 当前总共使用了 165276 Byte；free 代表 buffer 与 cache 总剩余容量加内存未被划分的剩余容量之和。也就是说，860364 Byte = 112120 Byte + 729044 Byte + 19200 Byte，这个值是系统中内存未被使用的实际容量。

第三行为交换分区的使用情况，total 代表交换分区总容量为 2706424 Byte；used 代表已经使用了 0 Byte；free 代表剩余交换分区为 2706424 Byte。

2.7.3　监控磁盘使用情况——df 命令

该命令的描述为：生成系统磁盘空间的使用量信息。

用法：df [选项]...

选项：-h　　人性化方式显示容量信息。

-i　　显示磁盘 inode 使用量信息。

-T　　显示文件系统类型。

```
[root@centos6 ~]# df -hT
Filesystem     Type    Size  Used  Avail  Use%  Mounted on
/dev/mapper/VolGroup-lv_root
               ext4    21G   4.6G  15G    24%   /
tmpfs          tmpfs   428M  272K  428M   1%    /dev/shm
/dev/sda1      ext4    485M  32M   429M   7%    /boot
```

以上信息中根分区的类型为 ext4，总容量为 21GB，已经使用 4.6GB，剩余 15GB 可用，使用率为 24%，挂载点为/。

```
[root@centos6 ~]# df -i
Filesystem            Inodes     IUsed     IFree     IUse%   Mounted on
/dev/mapper/VolGroup-lv_root
                      1346400    75754     1270646   6%      /
tmpfs                 109411     6         109405    1%      /dev/shm
/dev/sda1             128016     38        127978    1%      /boot
```

以上信息中，根分区 inode 总个数为 1346400 个，已经使用了 75754 个，剩余 1270646 个，使用率为 6%。这里的 inode 个数决定了该分区可以创建的文件个数，有多少个 inode 节点，就可以在该分区创建多少个文件。在上面的显示案例中，如果在根分区下再创建 1270645 个空文件，则即使系统显示磁盘剩余空间为 15GB，也无法再创建文件，因为 inode 节点已经耗尽。

2.7.4　监控网络使用情况——ifconfig 和 netstat 命令

1. ifconfig 命令

ifconfig 命令可以查看网卡接口信息，在 Linux 中以太网卡一般被标识为 ethx，第一块以太网卡为 eth0，第二块以太网卡为 eth1，依此类推。此外，我们一般使用 netstat 命令查看服务器开启的端口信息以及网络连接状态。

```
[root@centos6 ~]# ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:41:09:BC
          inet addr:192.168.0.201  Bcast:192.168.255.255  Mask:255.255.0.0
          inet6 addr: fe80::a00:27ff:fe41:9bc/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:384884 errors:0 dropped:0 overruns:0 frame:0
          TX packets:480696 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:162878956 (162.8 MB)  TX bytes:501121958 (501.1 MB)
```

从以上信息中可以看出，该网卡的 IP 地址为 192.168.0.201，广播地址为 192.168.255.255，子网掩码为 255.255.0.0；系统开机至现在总共接收了 384884 个数据包；系统开机至现在总共发送了 480696 个数据包；计算机总共接收了 162.8MB 数据，总共发送了 501.1MB 数据。

2. netstat 命令

描述：打印网络连接、路由表、网络接口统计等信息。

用法：netstat [选项]

选项：-s　　显示各种协议数据统计信息。

-n　　使用数字形式的 IP、端口号、用户 ID 替代主机、协议、用户等名称信息。

-p　　显示进程名称及对应进程 ID 号。

-l　　仅显示正在监听的 shocket 接口信息。

-u　　查看 udp 连接信息。

-t　　查看 tcp 连接信息。

```
[root@centos6 ~]# netstat -nutlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address   Foreign Address   State  PID/Program name
tcp       0   0 0.0.0.0:6000        0.0.0.0:*         LISTEN   1594/X
tcp       0   0 0.0.0.0:22          0.0.0.0:*         LISTEN   1234/sshd
tcp       0   0 127.0.0.1:25        0.0.0.0:*         LISTEN   1329/master
tcp       0   0 :::6000             :::*              LISTEN   1594/X
tcp       0   0 :::22              :::*              LISTEN   1234/sshd
tcp       0   0 ::1:25             :::*              LISTEN   1329/master
udp       0  0 0.0.0.0:69          0.0.0.0:*         1242/xinetd
udp       0  0 0.0.0.0:54479       0.0.0.0:*         1053/avahi-daemon
udp       0  0 0.0.0.0:5353        0.0.0.0:*         1053/avahi-daemon
udp       0  0 0.0.0.0:67          0.0.0.0:*         1253/dhcpd
[root@centos6 ~]# netstat -s
Ip:
    130 total packets received
    0 forwarded
    0 incoming packets discarded
    75 incoming packets delivered
    94 requests sent out
Icmp:
    2 ICMP messages received
    0 input ICMP message failed.
    ICMP input histogram:
```

```
        destination unreachable: 2
    2 ICMP messages sent
    0 ICMP messages failed
    ICMP output histogram:
        destination unreachable: 2
... ...(部分输出省略)... ...
```

2.7.5 监控进程使用情况——ps 和 top 命令

1. ps 命令

描述：查看当前进程信息。

用法：ps 命令版本众多，有多种语法种类，如 UNIX、BSD 以及 GNU Linux。

标准语法格式：

```
[root@centos6 ~]# ps -e                    #查看所有的进程信息
[root@centos6 ~]# ps -ef                   #全格式显示进程信息
```

BSD 语法格式：

```
[root@centos6 ~]# ps -ax
[root@centos6 ~]# ps -axu
```

命令的输出信息中，UID 或 USER 代表进程的执行用户，PID 为进程的唯一编号，PPID 代表父进程 ID 编号，%CPU 代表进程的 CPU 占用率，%MEM 代表进程的内存占用率，VSZ 代表进程所使用的虚拟内存大小（单位为 KB），RSS 代表进程所使用的真实内存大小（单位为 KB），TTY 为终端，STIME 或 START 代表进程启动时间，STAT 代表进程状态（D：不可中断的进程，R：正在运行的进程，S：正在睡眠的进程，T：停止或被追踪的进程，X：死掉的进程，Z：僵死进程），TIME 代表进程占有 CPU 的总时间，CMD 或 COMMAND 代表进程命令。

2. top 命令

描述：动态查看进程信息。

选项：-d　　#top 刷新间隔，默认为 3 秒。

　　-p　　#查看指定 PID 的进程信息。

```
[root@centos6 ~]# top
top - 23:27:24 up  16:56,  4 users,  load average: 0.77, 0.33, 0.26
Tasks: 136 total,   2 running, 134 sleeping,   0 stopped,   0 zombie
Cpu(s): 12.4%us, 14.2%sy,  0.0%ni, 56.0%id,  1.3%wa,  7.1%hi,  8.9%si,  0.0%st
Mem:    875288k total,   478784k used,   396504k free,    23464k buffers
```

```
Swap:  1015800k total,        0k used,  1015800k free,   197572k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
 1938 root      20   0  283m  14m 9324 S 15.8  1.6   2:19.65 gnome-terminal
 1645 root      20   0  197m 1684 1072 S 10.8  0.2   5:52.82 VBoxClient
```

(…部分输出省略…)

通过 top 可以动态查看 10 个进程的信息，默认按 CPU 使用率排序，输入 M 可以按照内存使用率排序，输入 N 可以按照进程号排序，输入 z 可以高亮显示颜色。

```
[root@centos6 ~]# top -d 1 -p 1,2
(…部分输出省略…)
PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
1 root      20   0 19348 1552 1244 S  0.0  0.1   0:04.67 init
2 root      20   0     0    0    0 S  0.0  0.0   0:00.07 kthreadd
```

2.8　网络配置

CentOS 6.3 定位于提供服务的网络型操作系统，所以为其设置完整的网络参数至关重要。通常，Linux 都可以通过两种方式为系统设置网络参数。一种是通过命令设置的方式，这种方式的优点是方便、快捷，并且设置后可以立刻生效。但缺点是计算机重启后，所有的设置均失效。另一种是通过修改系统配置文件来实现，这种方式的优点是设置后可以永久保存，计算机重启后仍有效。缺点是这样的设置不会立刻生效，需要重启计算机或相关服务才可以使其生效。下面分别介绍这两种设置网络参数的具体方法。

2.8.1　命令行设置网络参数

1. 网络接口参数——ifconfig 命令

描述：显示或设置网络接口信息。

用法：ifconfig interface 选项 | 地址

设置 eth0 网卡接口 IP 地址为 192.168.0.31，子网掩码为 255.255.255.0。

```
[root@centos6 ~]# ifconfig eth0 192.168.0.31 netmask 255.255.255.0
```

查看 eth0 网卡接口信息：

```
[root@centos6 ~]# ifconfig eth0
```

关闭 eth0 网卡接口：

```
[root@centos6 ~]# ifconfig eth0 down
```

开启 eth0 网卡接口：

```
[root@centos6 ~]# ifconfig eth0 up
```

2. 主机名参数——hostname 命令

描述：显示或设置系统主机名称。

用法：hostname [选项]

```
[root@centos6 ~]# hostname                          #查看主机名称
[root@centos6 ~]# hostname centos.example.com       #设置主机名称
[root@centos6 ~]# hostname -i                       #查看本机 IP 信息
```

提示 hostname -i 需要读取/etc/hosts 来判断本机 IP 地址，如果该文件无记录，则提示 hostname: Unknown host。

3. 路由参数——route 命令

描述：显示或设置静态 IP 路由表。

用法：route [选项]　　#查看路由信息

　　route add 目标网络 gw 网关地址　　#添加路由表记录

　　route del 目标网络　　#删除路由表记录

查看当前路由表：

```
[root@centos6 ~]# route
```

使用数字地址替代主机名称：

```
[root@centos6 ~]# route -n
```

添加默认网关为 192.168.0.254：

```
[root@centos6 ~]# route add default gw 192.168.0.254
```

添加指定网段的网关：

```
[root@centos6 ~]# route add -net 172.16.0.0/16 gw 192.168.0.254
```

添加路由记录，指定通过 eth0 网卡传输到 192.56.76.0 网段的数据：

```
[root@centos6 ~]# route add -net 192.56.76.0 netmask 255.255.255.0 dev eth0
```

删除默认网关：

```
[root@centos6 ~]# route del default gw 192.168.0.254
```

删除指定网段的网关记录：

```
[root@centos6 ~]# route del -net 172.16.0.0/16
```

2.8.2　文件修改网络参数

1. 网络接口参数

CentOS 6.3 系统中网卡配置文件为/etc/sysconfig/network-scripts/ifcfg-<iface>，其中，iface 为网卡接口名称，如果只有一块以太网卡，则一般为 ifcfg-eth0，该文件的语法格式见表 2-9。

表 2-9

选项	功能描述	值
DEVICE	设备名称	eth0、eth1...
TYPE	设备类型	ethernet
BOOTPROTO	启动协议	none|dhcp|static
HWADDR	硬件地址	52:54:00:B9:F2:8E
NM_CONTROLLED	网卡是否可以被 NetworkManager 控制	yes|no
ONBOOT	开机是否启动该网卡，对虚拟接口无效	yes|no
ONPARENT	真实接口启动后虚拟接口是否启动	yes|no
IPADDR	IP 地址	
PREFIX	网络位掩码个数，对虚拟接口无效	
NETMASK	子网掩码	
GATEWAY	默认网关	
DNS{1,2}	DNS 服务器，多个服务器使用数字标记，如 DNS1，DNS2 等	

实例 1：设置 eth0 网卡通过 dhcp 获取网络参数。

```
[root@centos6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE="eth0"
BOOTPROTO="dhcp"
HWADDR="52:54:00:B9:F2:8E"
NM_CONTROLLED="yes"
ONBOOT="yes"
TYPE="Ethernet"
[root@centos6 ~]# service network restart        #重启网络服务使新设置生效
```

实例 2：设置 eth0 网卡参数，IP 地址为 192.168.0.31、子网掩码为 255.255.255.0、网关为 192.168.0.254、主 DNS 为 202.106.0.20、辅助 DNS 为 202.106.46.151。

```
[root@centos6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE="eth0"
BOOTPROTO="static"
HWADDR="52:54:00:B9:F2:8E"
NM_CONTROLLED="yes"
ONBOOT="yes"
TYPE="Ethernet"
IPADDR="192.168.0.31"
PREFIX="24"
GATEWAY="192.168.0.254"
DNS1="202.106.0.20"
DNS2="202.106.46.151"
[root@centos6 ~]# service network restart          #重启网络服务使新设置生效
```

Linux 中除了可以为真实的网卡接口配置网络参数外，我们还可以通过虚拟接口的方式为一个物理网卡配置多个网络参数，虚拟接口的名称一般为 iface:n，如 eth0:0、eth0:1 等。注意，默认虚拟接口的配置文件并不存在，需要手动创建新文件并设置参数。

提示 在 CentOS 6.3 操作平台上，NetworkManager 服务与网卡别名不兼容。所以实验前需要将 NetworkManager 服务关闭，并使用 chkconfig 确保开机不启动该服务。

实例 3：设置 eth0 网卡虚拟接口 eth0:0，IP 地址为 192.168.0.32、子网掩码为 255.255.255.0、网关为 192.168.0.254、主 DNS 为 202.106.0.20、辅助 DNS 为 202.106.46.151。

```
[root@centos6 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0:0
DEVICE="eth0:0"
BOOTPROTO="static"
HWADDR="52:54:00:B9:F2:8E"
NM_CONTROLLED="yes"
ONPARENT="yes"
TYPE="Ethernet"
IPADDR="192.168.0.32"
NETMASK="24"
GATEWAY="192.168.0.254"
DNS1="202.106.0.20"
DNS2="202.106.46.151"
[root@centos6 ~]# service network restart          #重启网络服务使新设置生效
```

2. 主机名参数

主机名配置文件为/etc/sysconfig/network，通过修改其中的 HOSTNAME 值实现主机名称的设置，HOSTNAME=centos6.example.com[7]。

[7] 重启计算机后生效。

3. 路由参数

通过上面对网卡接口设置网络参数后，可以指定网卡的默认网关，但无法设定通往特定网段的静态路由信息。此时，需要创建指定网卡的路由配置文件（默认文件不存在），配置文件名称为 route-<iface-name>，该文件每行对应一条静态路由记录。

实例：为 eth0 网卡指定去往 172.16.0.0/16 网段的网关为 192.168.0.200。

```
[root@centos6 ~]# vim /etc/sysconfig/network-scripts/route-eth0
172.16.0.0/16 via 192.168.0.200
[root@centos6 ~]# service network restart        #重启网络服务使新设置生效
```

2.8.3　网络故障排错

随着公司 IT 技术的不断扩展与延伸，当网络越来越大时，故障就时有发生，此时我们需要有良好的排错思路与优秀的排错工具。排错的思路为：首先，从最近一次操作定位问题所在，或根据公司服务器工作手册[8]查找最近对服务器所做的修改定位关键问题；其次，检查这些操作与配置有无错误，同时获取与问题相关的信息（如，硬件型号、软件版本、网络拓扑等信息）；然后，根据所搜集的信息修复问题，修复可以是修改配置文件、替换相应的设备或进行版本升级等。问题不是一成不变的，但解决问题的思路确是万变不离其宗的。下面介绍几个帮助解决网络配置常见问题的工具。

1. ping

当网络不通时，可以使用简单的 ping 命令来定位问题节点的位置，一般，我们会按顺序进行 ping 命令，依次为本地回环、本地 IP、网关 IP、外网 IP，ping 的对象一般都会给予回应，如果没有回应，则表示网络不通（某些环境下，对象主机会禁止被 ping），据此来判断网络断点的位置。

```
[root@centos6 ~]# ping 127.0.0.1          #ping 本地回环，测试本地网络协议是否正常
[root@centos6 ~]# ping 192.168.0.31       #ping 本地 IP，测试本地网络接口是否正常
[root@centos6 ~]# ping 192.168.0.254      #ping 网关，测试网关是否工作正常
[root@centos6 ~]# ping 202.106.0.20       #ping 外部网络，测试服务商网络是否工作正常
```

2. traceroute

一个数据包从本地发送后一般会经过多个路由转发数据，如果有一个数据包进入互联网后

[8] 很多公司要求将关键服务器的所有操作记录成工作手册。

因为中间的某个路由转发有问题，而导致最终的数据发送失败。这种情况下，我们并不知道问题路由的位置及 IP 信息，利用 traceroute 可以帮助我们解决这些问题，它可以跟踪数据包的路由过程，以此判断问题所在。Linux 下的 traceroute 默认使用 UDP 封装跟踪包，如果希望使用 ICMP 封装，可以使用-I 选项。

```
[root@centos6 ~]# traceroute -I www.google.com
traceroute to www.google.com (74.125.128.147), 30 hops max, 60 byte packets
 1  172.16.0.1 (172.16.0.1)  0.528 ms  0.713 ms  4.457 ms
 3  218.249.205.41 (218.249.205.41)  4.524 ms  19.531 ms  20.307 ms
 4  10.0.18.1 (10.0.18.1)  23.974 ms  24.958 ms  27.032 ms
(... 部分输出省略 ...)
```

3. nslookup

目前互联网的高速发展，使网络资源出现爆发式的增长。网络本身是使用 IP 地址来唯一对应网络资源的，但用户是无法记忆这么多的数字 IP 信息的。所以，我们一般会通过域名来访问网络资源，此时 DNS 服务就至关重要了，使用 nslookup 可以帮助检查本地设置的 DNS 服务器工作是否正常。输入下面的命令检查本机设置的 DNS 服务器是否可以解析 www.google.com 域名。

```
[root@centos6 ~]# nslookup www.google.com
Server:     219.141.140.10
Address:    219.141.140.10#53
Non-authoritative answer:
Name:   www.google.com
Address: 74.125.128.99
Name:   www.google.com
Address: 74.125.128.103
(...部分输出省略...)
```

输出信息说明：本次解析是由 219.141.140.10 这台 DNS 服务器提供的，解析的结果是 www.google.com 域名对应了多个 IP 地址。

4. dig

nslookup 仅可以查询域名与 IP 地址之间的对应关系，如果你需要更多关于 DNS 记录的信息，可以使用 dig 命令，查看包括 MX（邮件记录）、NS（域名服务器记录）等信息。

```
[root@centos6 ~]# dig www.google.com
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 30154
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 4, ADDITIONAL: 4
;www.google.com.                IN  A
```

```
www.google.com.          135    IN   A    74.125.128.147
google.com.            98733    IN   NS   ns4.google.com.
ns1.google.com.        93029    IN   A    216.239.32.10
;; Query time: 1121 msec
```

以上输出信息为删减后的主要内容，读者在实际查询时会获得更多的记录信息。

```
[root@centos6 ~]# dig google.com MX          #查看 google.com 域的邮件记录
```

5. netstat

如果你怀疑有人非法闯入了你的系统，而你又不知道此人在哪里时，可以通过 netstat 强大的网络监控能力找出连接系统的所有人。或是当你启动了一个网络服务后，远程客户却无法访问该服务，我们也可以使用 netstat 命令查看服务对应的端口是否已经正常开启，来判断服务启动是否正常。

```
[root@centos6 ~]# netstat -an                #查看网络连接状态
tcp    0  0  192.168.0.102:22      192.168.0.101:50132      ESTABLISHED
```

从该条输出信息中可以看出，有一个远程 IP（192.168.0.101）使用 50132 端口，连接了本地的 22 端口。

```
[root@centos6 ~]# netstat -nutlp             #查看当前系统开启端口信息
```

2.9　内核模块

Linux 内核采用的是模块化技术，这样的设计使得系统内核可以保持最小化，同时确保了内核的可扩展性与可维护性，模块化设计允许我们在需要时才将模块加载至内核，实现动态内核的调整。

2.9.1　内核模块存放位置

Linux 内核模块文件的命名方式通常为<模块名称.ko>，CentOS 6.3 系统的内核模块被集中存放在/lib/modules/`uname -r`/[9]目录下。下面通过几个实例说明对内核模块的基本操作。

[9] `是反引符号，一般位于键盘的 Tab 键上方。

2.9.2 查看已加载内核模块

lsmod 命令用来显示当前 Linux 内核模块状态，不使用任何参数会显示当前已经加载的所有内核模块。输出的三列信息分别为模块名称、占用内存大小、是否在被使用，如果第三列为 0，则该模块可以随时卸载，非 0 则无法执行 modprobe 删除模块。

```
[root@centos6 ~]# lsmod
Module                  Size    Used by
bridge                  79950   0
stp                     2173    1  bridge
llc                     5642    2  bridge,stp
fuse                    66891   2
autofs4                 27212   3
sunrpc                  263516  1
ipt_REJECT              2351    2
nf_conntrack_ipv4       9506    2
nf_defrag_ipv4          1483    1  nf_conntrack_ipv4
iptable_filter          2793    1
ip_tables               17831   1  iptable_filter
```

2.9.3 加载与卸载内核模块

modprobe 命令可以动态加载与卸载内核模块：

```
[root@centos6 ~]# modprobe ip_vs              #动态加载 ip_vs 模块
[root@centos7 ~]# lsmod |grep ip_vs           #查看模块是否加载成功
[root@centos7 ~]# modprobe -r ip_vs           #动态卸载 ip_vs 模块
```

modinfo 命令可以查看内核模块信息：

```
[root@centos7 ~]# modinfo ip_vs
filename:       /lib/modules/2.6.32-279.el6.x86_64/kernel/net/netfilteripvs/
                ip_vs.ko
license:        GPL
srcversion:     01386EABC060B63920E181B
depends:        ipv6,libcrc32c
vermagic:       2.6.32-279.el6.x86_64 SMP mod_unload modversions
```

通过上述 modprobe 方式加载的内核模块仅在当前有效，计算机重启后并不会再次加载该模块，如果希望系统开机自动挂载内核模块，则需要将 modprobe 命令写入/etc/rc.sysinit 文件中，或者通过修改/etc/modprobe.d/目录下的相关配置文件实现：

```
[root@centos7 ~]# echo "modprobe ip_vs" >> /etc/rc.sysinit
```

当内核模块不再需要时，可以通过将/etc/rc.sysinit 文件中对应的 modprobe 命令删除，但需要重启计算机才生效。此时，可以通过 modprobe -r 命令来立刻删除内核模块：

```
[root@centos6 ~]# modprobe -r ip_vs
```

2.9.4　修改内核参数

1. 临时调整内核参数

Linux 内核参数随着系统的启动会被写入内存中，我们可以直接修改/proc 目录下的大量文件来调整内核参数，并且这种调整是立刻生效的，下面我们来看几个实例。

开启内核路由转发功能（通过 0 或 1 设置开关）：

```
[root@centos6 ~]# echo "1" > /proc/sys/net/ipv4/ip_forward
```

禁止所有的 icmp 回包（禁止其他主机 ping 本机）功能：

```
[root@centos6 ~]# echo "1" > /proc/sys/net/ipv4/icmp_echo_ignore_all
```

调整所有的进程可以打开的文件总数量（当大量的用户访问网站资源时，可能会因该数字过小而导致错误）：

```
[root@centos6 ~]# echo "108248" >/proc/sys/fs/file-max
```

2. 永久调整内核参数

通过 man proc 可以获得大量关于内核参数的描述信息。但以上通过直接修改/proc 相关文件的方式在系统重启后将不再有效，如果希望设置参数并永久生效，可以修改/etc/sysctl.conf 文件，文件格式为“选项=值”，我们通过 vim 修改该文件将前面三个实例参数设置为永久有效：

```
[root@centos6 ~]# vim /etc/sysctl.conf
net.ipv4.ip_forward = 1
net.ipv4.icmp_echo_ignore_all = 1
fs.file-max = 108248
```

注意，通过 sysctl.conf 文件修改的内核参数不会立刻生效，修改完成后，使用 sysctl -p 命令可以使这些设置立刻生效。

第3章

自动化运维

3.1 Shell 简介

一个完整的计算机体系结构包括：硬件与软件，而软件又分为系统软件与应用软件，负责对硬件进行管理与操作的是系统软件的内核部分，用户是无法与硬件或内核打交道的，用户通过应用程序或部分系统软件发出指令（可能是通过浏览器发送一封邮件），这些指令会被翻译并传给内核，内核在得知用户的需求后调度硬件资源来完成操作（比如，使用网卡发送数据包）。在 Linux 环境下，我们可以通过 Shell 与内核进行交流，并最终实现我们想要使用计算机资源的目的。由于 Linux 的开放性特点，使得在 Linux 下对 Shell 的选择也有很多，CentOS 6.3 系统中可以使用的 Shell 有/bin/sh、/bin/bash、/bin/tcsh、/bin/csh 这几种，/etc/shells 文件说明了当前系统有哪些可用的 Shell。不同的 Shell 有不同的特点以及操作方式，这里以 CentOS 默认使用的 Shell 为例讲解，也即是 Bash。

3.2　Bash 功能介绍

3.2.1　命令历史

Bash 拥有自动记录命令历史的功能，用户所执行的命令会在注销时自动记录到自己家目录下的.bash_history[1]隐藏文件中。查看这些命令历史记录的方法有很多，可以通过直接打开文件来查看，也可以通过键盘的上下键来上翻或下翻历史记录，还可以通过 history 命令查看历史记录，所有的命令历史都有记录编号。命令历史能够记录的信息数量由 HISTSIZE 变量所决定，CentOS 6.3 默认通过/etc/profile 文件定义了 HISTSIZE=1000，也就是最多可以记录最近所使用的 1000 条命令，当有第 1001 条命令执行时，第一条命令会被覆盖，执行 history -c 命令可以清空所有的历史记录。

记录命令历史除了可以查看历史记录外，还可以在需要时直接调用历史记录再次执行该命令。

- 上下键翻阅历史命令，找到合适的命令后直接回车键即可执行。
- 输入!string 调用命令历史（string 为关键字），如!vim 将调用最后一次执行的以 vim 开头的命令。或者通过!n 来准确定位历史记录，如!242 将直接调用命令历史的第 242 条记录并执行。
- 通过 Ctrl+r 快捷键打开搜索功能，接着输入关键字，即可在命令历史中搜索相关命令，按回车键完成执行操作。如果没有搜索到适合的命令，按 Esc 键退出搜索。

3.2.2　命令别名

在 CentOS 6.3 系统中，我们可以直接使用 ll 命令来显示文件的详细信息，其实系统中并没有 ll 命令，它只是被提前定义好的一个别名而已，别名的好处是可以把本来很长的指令简化缩写，为常用且复杂的命令及选项创建别名可以大大提高日常工作效率。

```
[root@centos6 ~]# alias                                    #查看系统当前所有的别名
alias cp='cp -i'
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
```

[1] Linux 中以“.”开头的文件或目录为隐藏文件或目录。

```
alias mv='mv -i'
alias rm='rm -i'
[root@centos6 ~]# alias h5='head -5'                    #定义新的别名
[root@centos6 ~]# unalias h5                            #取消别名定义
```

3.2.3 管道与重定向

Bash 的标准输入设备是键盘、鼠标、手写板等设备，标准输出设备为显示器，一般我们通过键盘输入命令并执行，系统将返回信息显示在屏幕上，默认不管是正确的还是错误的信息，都将输出至显示器。标准输入的文件描述符为 0，标准输出的文件描述符为 1，错误输出的文件描述符为 2。但有时我们需要改变这样的标准输入与输出方式，Linux 中可以使用重定向符（<、>、<<、>>、|）重新定义输入与输出。

管道使得我们可以将多条命令连接在一起使用，它的作用是将一个命令的标准输出重定向给下一个命令，并作为该命令的标准输入。例如，使用 ifconfig eth0 | grep 'inet addr'命令过滤包含 IP 地址的行，ifconfig 本身会输出大量的网络接口信息，由于这里使用了管道符号（|），所以 ifconfig 命令的所有输出都将作为 grep 命令的输入内容，最终实现过滤包含 IP 地址的行。

对于标准的输出信息是输出至显示器，而有时你可能并不需要这些输出信息，又或者是暂时不需要在屏幕上看到这些信息。另外，对于输入，一般是通过键盘输入，而有时你或许希望可以从文件中读取输入信息。输出重定向可以使用>或>>符号，使用>可以将输出导入至文件，如果文件不存在，则创建该文件，如果文件已经存在，则会覆盖该文件的内容；而使用>>可以将输出追加至文件；对应错误信息的重定向需要使用 2>或 2>>符号实现。输入重定向可以使用<符号，它可以帮助我们从文件中提取输入信息。

下面通过几个简单的实例演示重定向的使用方法。

```
[root@centos6 ~]# rpm -qa |grep gcc          #查询计算机中是否安装了 gcc 软件
[root@centos6 ~]# echo "pass" | passwd --stdin tom #设置 tom 的密码为 pass
[root@centos6 ~]# ls                         #查看当前文件列表
anaconda-ks.cfg install.log    install.log.syslog
[root@centos6 ~]# ls > list.txt              #将输出保存至 list.txt，屏幕无输出
[root@centos6 ~]# hostname >> list.txt       #将主机名追加至 list.txt 文件末尾
[root@centos6 ~]# mail -s test xx@gmail.com < list.txt #发送邮件，邮件内容来自文件
[root@centos6 ~]# ls -l abc install.log      #查看文件详细信息，abc 并不存在
ls: cannot access abc: No such file or directory
-rw-r--r--. 1 root root 9064 Dec 4 09:30 install.log
[root@centos6 ~]# ls -l abc install.log 2> error.txt #仅将错误重定向，不影响正确输出
-rw-r--r--. 1 root root 9064 Dec 4 09:30 install.log
[root@centos6 ~]# ls -l abc install.log > all 2>&1 #标准输出与错误输出均导入至 all
```

```
[root@centos6 ~]# ls -l abc install.log >> all 2>&1 #标准输出与错误输出均追加至 all
[root@centos6 ~]# ls -l abc install.log &> all  #标准输出与错误输出均导入至 all
```

3.2.4　快捷键

Bash 提供了大量的快捷键为用户所使用，熟练掌握这些快捷键会提高工作效率。表 3-1 列出了常用的快捷键及其功能描述。

表 3-1

快捷键	功能描述	快捷键	功能描述
Ctral+a	光标移动至行首	Ctrl+k	删除光标至行尾的字符
Ctrl+e	光标移动至行尾	Ctrl+c	终止进程
Ctrl+f	光标右移一个字符	Ctrl+z	挂起进程（jobs 命令查看挂起的进程）
Ctrl+b	光标左移一个字符	Ctrl+w	删除光标前一个单词（以空格为分隔符）
Ctrl+l	清屏，等同于 clear 命令	Alt+d	删除光标后一个单词
Ctrl+u	删除光标至行首的字符	Tab	自动补齐

3.3　Bash 使用技巧

3.3.1　重定向技巧

如果你编写了一个脚本实现自动修改系统账号的密码，那么默认当密码修改成功后系统会出现 successfully 提示信息，如果需要修改 20 个账户密码，则显示 20 条这样的信息。像这样的案例，大量的成功提示信息并不是我们关注的重点，管理员更多的是关心错误提示。所以这时可以考虑将标准正确输出屏蔽，Linux 中提供了一个特殊设备/dev/null，它很像太阳黑洞，任何东西往里写入都将永远消失。对大量无意义的输出信息，可以通过管道导入/dev/null 设备，命令如下。

```
[root@centos6 ~]# echo "pass" | passwd --stdin root >/dev/null
```

标准输出与错误输出分离重定向，很多时候自动运行的脚本编写完成后，是在深夜服务器负载最低时以无人职守方式运行的，那么当管理员去检查脚本运行情况时，会希望看到哪些命令执行成功了，哪些命令出现了问题，并且为了方便管理，需要将正确输出与错误输出分别保存在两个不同的记录文件中，那么分离重定向就显得尤为重要了。下面检查系统中是否存在 tom

用户，如果存在，则将 tom 相关信息记录至 user 文件中，否则记录至 error 文件中。

```
[root@centos6 ~]# id tom >> user 2>> error
```

3.3.2 命令序列使用技巧

在 Linux 中，我们可以使用控制字符（;、&&、||、&）来控制命令的执行方式。其中，[&]控制符使得命令开启一个子 Shell，并在后台执行；使用[;]控制字符可以把多个命令组合，但多个命令之间没有任何逻辑上的关系，仅按循序执行；使用[&&]控制字符也可以将多个命令组合，但仅当前一个命令执行成功后才会执行[&&]控制符后面的命令；[||]控制字符的作用与[&&]刚好相反，仅当前一个命令执行失败才会执行[||]控制符后面的命令。下面通过实例演示。

实例 1： 火狐浏览器通过前端启动，使当前 Shell 将暂时无法使用。

```
[root@centos6 ~]# firefox
```

实例 2： 后台运行浏览器，不影响当前 Shell 的使用。

```
[root@centos6 ~]# firefox &
```

实例 3： 所有的命令按顺序执行（不管前面的命令是否成功，后面的命令一定正常执行）。

```
[root@centos6 ~]# ls /tmp ; ls /root ; ls /home
```

实例 4： 如果某个文件存在，则显示该文件内容，否则报错（仅当 ls 执行成功才会执行 cat）。

```
[root@centos6 ~]# ls test.txt && cat test.txt
```

实例 5： 如果有 gedit 编辑器，则打开该程序，否则打开 vim 编辑器。

```
[root@centos6 ~]# gedit || vim
```

实例 6： 如果 id tom 执行成功，则说明用户存在，屏幕将显示 Hi,tom，否则显示 No such user。

```
[root@centos6 ~]# id tom &>/dev/null && echo "Hi,tom" || echo "No such user"
```

实例 6 首先通过&>重定向符将 id 命令的所有输出屏蔽，然后使用&&与||来判断 id 命令是否执行成功，成功就显示问候语，否则显示没有该用户。

3.3.3 作业控制技巧

Bash 环境中通过命令开启进程的时候使用&符可以使该进程进入后台执行，或者一个命令执行后使用 Ctrl+Z 快捷键可以将该进程放入后台并暂停执行，以后随时可以使用 jobs 命令查看这些后台进程，并且会为每个这样的进程分配一个编号，通过 fg <编号>的形式可以将这些后台

进程再次调回前台执行。

```
[root@centos6 ~]# firefox &
[root@centos6 ~]# jobs
[1]+  Running                 firefox &
[root@centos6 ~]# fg 1
```

3.3.4　花括号{}的使用技巧

通过括号扩展可以生成命令行或脚本所需要的字串，括号中可以包含连续的序列或使用逗号分割的多个项目，连续的序列包括一个起点与一个终点，并使用“..”分隔。接下来看看具体的语法格式案例。

```
[root@centos6 ~]# echo {a,b,c}
a b c
[root@centos6 ~]# echo user{1,5,8}
user1 user5 user8
[root@centos6 ~]# echo {0..10}
0 1 2 3 4 5 6 7 8 9 10
[root@centos6 ~]# echo {0..10..2}
0 2 4 6 8 10
[root@centos6 ~]# echo a{2..-1}
a2 a1 a0 a-1
[root@centos6 ~]# mkdir /tmp/{dir1,dir2,dir3}
[root@centos6 ~]# ls -ld /tmp/dir{1,2,3}
[root@centos6 ~]# chmod 777 /tmp/dir{1,2}
```

3.4　变量

3.4.1　自定义变量

变量是用来存储非固定值的载体，它具有一个值，以及零个或多个属性。在 Shell 中定义变量的语法格式为：

```
name=[value]
```

如果 value 没有指定，变量将被赋值为空字符串，变量定义后使用<$变量名称>来调用变量的值。变量名称为字母、数字以及下画线的组合，但首字母不可以为数字，变量名称没有硬性要求大小写，在此建议统一使用大写或首字母大写。默认 NAME 的值是可以被修改的，但我们

可以通过 typeset 为这个变量添加只读属性来防止误操作，如果需要限定变量的值仅可以设置为整数，也可以使用该命令添加相应的属性。

```
[root@centos6 ~]# NAME=tomcat
[root@centos6 ~]# echo $NAME
tomcat
[root@centos6 ~]# typeset -r NAME                    #添加 readonly 只读属性
[root@centos6 ~]# NAME=jerry
bash: NAME: readonly variable
```

如果希望预先定义一个变量，但暂时不赋值给它，可以使用 declare 命令。

```
[root@centos6 ~]# declare INT NUMBER
[root@centos6 ~]# typeset -i INT_NUMBER      #使用-i 选项设置整数变量
[root@centos6 ~]# INT_NUMBER=test            #强制为该变量赋值字符型数据
[root@centos6 ~]# echo $INT_NUMBER           #查看变量结果为 0
0
[root@centos6 ~]# INT_NUMBER=200
[root@centos6 ~]# echo $INT_NUMBER
200
```

此外，还可以通过 read 命令来设置变量，read 从标准输入中读取变量值，使用-p 选项可以添加相应的提示信息。

```
[root@centos6 ~]# read P_NUMBER              #回车后输入要赋值给 P_NUMBER 的值
22
[root@centos6 ~]# echo $P_NUMBER
22
[root@centos6 ~]# read -p "Please input a number:" P_NUMBER
Please input a number:100                    #在提示符后输入要赋值给变量的值
[root@centos6 ~]# echo $P_NUMBER
100
[root@centos6 ~]# set                        #查看当前系统中设置的所有变量及值
[root@centos6 ~]# unset P_NUMBER             #删除变量
```

3.4.2 变量的使用范围

使用 name=[value]的形式定义的变量默认仅在当前 Shell 中有效，子进程不会继承这样的变量，使用 export 命令会将变量放入环境中，这样新的进程会从父进程那里继承环境，export 可以直接定义环境变量并赋值，也可以先定义一个普通的用户变量，再通过 export 转换为环境变量。

```
[root@centos6 ~]# TEST=pass
[root@centos6 ~]# echo $TEST
[root@centos6 ~]# bash                       #在当前 Shell 下开启新进程 bash
```

```
[root@centos6 ~]# echo $TEST                    #查看变量值为空
[root@centos6 ~]# exit
[root@centos6 ~]# export TEST                   #将已有用户变量添加至环境
[root@centos6 ~]# export NAME=tom               #直接定义环境变量
```

3.4.3　环境变量

Bash 为预设了很多环境变量，在实际工作中，我们可以直接调用这些变量，表 3-2 列出了一些比较常用的变量名称及其含义。关于变量的详细定义，可以参考 Bash 手册。

表 3-2

变量名称	含义	变量名称	含义
BASHPID	当前 bash 进程的进程号	UID	当前用户的 ID 号码
GROUPS	当前用户所属组的组 ID 号	HISTSIZE	命令历史的记录条数
HOSTNAME	当前主机的主机名称	HOME	当前用户的家目录
PWD	当前工作目录	PATH	命令搜索路径
OLDPWD	前一个工作目录	PS1	主命令提示符
RANDOM	0~32767 之间的随机数	PS2	次命令提示符

以上这些变量中，有些是可以设置的，有些则是只读而不可设置的。这里我们重点看看 PATH 变量，在 Linux 系统中，我们输入一个 ls 命令就可以查看当前目录的文件列表，那么系统怎么知道 ls 命令存放在哪里？其实，Linux 系统是通过 PATH 变量来搜索命令的，PATH 变量的值就是一个目录集合，系统会按照顺序查找这些命令，如果找不到，则提示命令未找到[2]。

```
[root@centos6 ~]# echo $PATH
/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:/root/bin
[root@centos6 ~]# ls
```

根据以上说明，我们知道用户在命令行中输入 ls 命令后，系统就开始在/usr/local/sbin/目录下查找这个命令，若找到，就执行该命令，没有则继续查找下一个目录，如果搜索到最后都没有找到 ls 程序，则提示命令没有找到。

对于 PATH 路径的修改需要注意，不可以直接对其赋新值，否则将覆盖 PATH 现有的值。下面将 PATH 直接赋值/root 目录，则以后再执行一个简单的 ls 命令都将提示命令没找到，因为系统只会在/root 目录下查找有没有 ls 命令。

```
[root@centos6 ~]# PATH=/root
[root@centos6 ~]# ls
```

[2] 这里我们暂时不考虑别名与函数的问题。

```
bash: ls: command not found
```

上面的修改因为并没有通过写入文件的方式把修改永久保存至环境变量，所以只要退出命令行重新登录终端 PATH 变量将复原即可。那么对 PATH 变量修改的正确方法是在原来的基础上修改，所以在赋值时需要引用旧值。

```
[root@centos6 ~]# PATH=$PATH:/root                    #在原有路径列表后追加目录
```

3.4.4 位置变量

位置变量的使用一般体现在脚本中，读者可以结合后面的脚本章节来看本节内容。位置变量使得在脚本中的命令可以调用运行脚本时不同位置的参数，参数一般使用空格分隔，$0 代表当前 Shell 程序的文件名称，$1 代表运行 Shell 程序时的第一个参数，$2 为第二个参数，依此类推（范围为$1~$9）。使用 Shell 位置变量有助于我们查看很多相关信息，$#代表 Shell 程序所有参数的个数；$*与$@均代表所有参数的内容，区别是$*将所有的参数作为一个整体，而$@将所有的参数分别作为个体看待；$$代表当前进程的 ID 号码，$?表示程序的退出代码（一般 0 代表命令执行成功，非 0 代表命令执行失败）。下面通过编写一个脚本并运行它来查看效果。

```
[root@centos6 ~]# vim /tmp/test.sh          #编辑脚本内容如下
#!/bin/bash
#This is test script for parameter!
echo "This is the file name: $0"
echo "This is first parameter: $1"
echo "This is second parameter: $2"
echo "This is the number of all parameter: $#"
echo "This is the all parameter: $*"
echo "This is the all parameter: $@"
echo "This is PID: $$"
[root@centos6 ~]# bash /tmp/test.sh a b c  #允许该脚本使用 3 个参数
This is the file name: /tmp/test.sh
This is first parameter: a                          #结果显示第一个参数为 a
This is second parameter: b                         #结果显示第二个参数为 b
This is the number of all parameter: 3              #结果显示所有参数的个数为 3
This is the all parameter: a b c                    #所有参数的内容为'a b c'
This is the all parameter: a b c                    #所有参数的内容为'a' 'b' 'c'
This is PID: 2666                                   #当前进程 PID 为 2666
```

3.4.5 变量的展开替换

Linux 中一般可以使用${变量名}的形式展开变量的值，如果有一个变量 NAME=Jacob，

则 echo ${NAME}用于显示该变量的值。除此之外，系统还提供了更加丰富的变量展开功能，下面四组展开方式主要应用于需要确定变量是否被正确设置的环境。

```
${varname:-word}      如 varname 存在且非 null，则返回其值，否则返回 word
${varname:=word}      如 varname 存在且非 null，则返回其值，否则设置为 word
${varname:?message}   如 varname 存在且非 null，则返回其值，否则显示 varname:message
${varname:+word}      如 varname 存在且非 null，则返回 word，否则返回 null
[root@centos6 ~]# NAME=Jacob
[root@centos6 ~]# echo ${NAME:-no user}; echo {NAME}
[root@centos6 ~]# echo ${EMAN:-no user}; echo {EMAN}
[root@centos6 ~]# echo ${NAME:=no user}; echo {NAME}
[root@centos6 ~]# echo ${EMAN:=no user}; echo {EMAN}
[root@centos6 ~]# echo ${NAME:?no defined}; echo {NAME}
[root@centos6 ~]# echo ${EMAN:?no defined}; echo {EMAN}
[root@centos6 ~]# echo ${NAME:+OK}; echo {NAME}
[root@centos6 ~]# echo ${EMAN:+ERROR}; echo {EMAN}
```

下面六组展开方式主要应用于需要对变量的值做修改后输出的场景。

```
${variable#key}       从头开始删除关键词，执行最短匹配
${variable##key}      从头开始删除关键词，执行最长匹配
${variable%key}       从尾开始删除关键词，执行最短匹配
${variable%%key}      从尾开始删除关键词，执行最长匹配
${variable/old/new}   将 old 替换为 new，仅替换第一个出现的 old
${variable//old/new}      将 old 替换为 new，替换所有 old
[root@centos6 ~]# USR=$(head -1 /etc/passwd)#将$()中命令的执行结果赋值给 USR
[root@centos6 ~]# echo $USR
root:x:0:0:root:/root:/bin/bash
[root@centos6 ~]# echo ${USR#*:}          # root:x:0:0:root:/root:/bin/bash
x:0:0:root:/root:/bin/bash
[root@centos6 ~]# echo ${USR##*:}         # root:x:0:0:root:/root:/bin/bash
/bin/bash
[root@centos6 ~]# echo ${USR%:*}          # root:x:0:0:root:/root:/bin/bash
root:x:0:0:root:/root
[root@centos6 ~]# echo ${USR%:*}          # root:x:0:0:root:/root:/bin/bash
root
[root@centos6 ~]# echo ${USR/root/admin}
admin:x:0:0:root:/root:/bin/bash
[root@centos6 ~]# echo ${USR//root/admin}
admin:x:0:0:admin:/admin:/bin/bash
```

3.4.6　数组

Bash 提供了一维数组变量的功能，数组中的所有变量都会被编录成索引，数组的索引是以

0 开始的整数，且没有大小限制。一般可以使用两种方式创建数组变量，第一种使用 name[subscript]=value 的语法格式定义的变量自动创建索引数组，subscript 必须是大于或等于 0 的整数或表达式；第二种使用 name=(value1 value2 ... valuen)的语法格式创建。使用 declare -a <name>可以预定义一个空数组变量。

数组定义完成后，我们使用${name[subscript]}索引格式来调用数组变量的值，如果 subscript 是@或者*符号，则将调用所有的数组成员。使用${#name[subscript]可以返回${name[subscript]}的长度，如果 subscript 是*或@，则返回数组中元素的个数。

下面的实例可以很好的帮助理解这些内容：

```
[root@centos6 ~]# A[1]=11
[root@centos6 ~]# A[2]=22
[root@centos6 ~]# A[3]=33
[root@centos6 ~]# echo ${A[1]},${A[2]},${A[3]}
11,22,33
[root@centos6 ~]# echo ${A[1]}:${A[2]}:${A[3]}
11:22:33
[root@centos6 ~]# echo ${A[*]}
11 22 33
[root@centos6 ~]# echo ${A[@]}
11 22 33
[root@centos6 ~]# B=(aa bbb cccc)
[root@centos6 ~]# echo ${B[0]}:${B[1]}:${B[2]}
aa:bbb:cccc
[root@centos6 ~]# echo "length of B_0 is ${#B[0]}"
legth of B_0 is 2
[root@centos6 ~]# echo "length of B_1 is ${#B[1]}"
legth of B_1 is 3
[root@centos6 ~]# echo "length of B_2 is ${#B[2]}"
legth of B_2 is 4
[root@centos6 ~]# echo ${#B[@]}
3
```

3.4.7 算术运算与测试

在 Shell 中进行算术运算时，使用$((expression))语法格式可以实现整数级的算术运算功能。其中，expression 为算术表达式，表 3-3 给出了常用的算术表达式及对应的含义。

表 3-3

表达式	含义
$((x+y))	加法运算，$((1+2))=3
$((x-y))	减法运算，$((10-8))=2

续表

表达式	含义
$((x/y))	除法运算，$((8/3))=2，结果仅保留整数位
$((x*y))	乘法运算，$((3*3))=9
$((x%y))	取余运算，$((8%3))=2，8/3 余数为 2
$((x++))	自加运算，x 自加 1
$((x--))	自减运算，x 自减 1
$((x**y))	幂运算，x 的 y 次方

此外，命令工具 expr 也可以实现如下类似的算术运算。

```
expr arg1 + arg2        加法
expr arg1 - arg2        减法
expr arg1 \* arg2       乘法
expr arg1 / rg2         除法
expr arg1 % arg2        取余
```

Shell 除了提供上述所说的算术运算外，还提供了一些判断功能，使用 Bash 内置命令 test 即可实现测试工作或者使用[测试表达式]，两种方式的最终效果相同。注意，使用[表达式]的时候记得在表达式两边都要留有空格，[表达式]这种格式是错误的格式(因为没有空格)。Linux 可以对很多内容做测试工作，表 3-4 列出了常用的测试条件及其对应的含义。

表 3-4

测试选项	含义	测试选项	含义
-d FILE	FILE 是否存在且为目录	-e FILE	文件是否存在
-f FILE	文件是否存在，且为普通文件	-r FILE	文件是否存在且可读
-w FILE	文件是否存在且可写	-x FILE	文件是否存在且可执行
-s FILE	文件是否存在且非空	-h FILE	文件是否为链接文件
-n STRING	字符串长度非 0	-z STRING	字符串长度为 0
STRING1 = STRING2	字符串相等	STRING1 != STRING2	字符串不相等
INTER1 -eq INTER2	整数 1 与整数 2 相等	INTER1 -ge INTER2	整数 1 大于或等于整数 2
INTER1 -gt INTER2	整数 1 大于整数 2	INTER1 -le INTER2	整数 1 小于整数 2
INTER1 -lt INTER2	整数 1 小于或等于整数 2	INTER1 -ne INTER2	整数 1 不等于整数 2

```
[root@centos6 ~]# test -d /etc/ && echo "Y" || echo "N"
                                        #etc 是目录就显示 Y，否则显示 N
Y
```

```
[root@centos6 ~]# test -d /etc/passwd && echo "Y" || echo "N"
N
[root@centos6 ~]# [ -d /etc/ ] && echo "Y" || echo "N"
Y
[root@centos6 ~]# [ -e /etc/passwd ] && echo "Y" || echo "N"
Y
[root@centos6 ~]# [ -f /etc/password ] && echo "Y" || echo "N"
N
[root@centos6 ~]# [ -h /etc/grub.conf ] && echo "Y" || echo "N"
Y
[root@centos6 ~]# [ -n $PATH ] && echo "Y" || echo "N"
                                              #PATH 变量有定义，所以为非空
Y
[root@centos6 ~]# [ -z $tt ] && echo "Y" || echo "N"
                                                #tt 变量没有定义，所以为空字串
Y
[root@centos6 ~]# [ -z $PATH ] && echo "Y" || echo "N"
N
[root@centos6 ~]# [ 22 -eq 22 ] && echo "Y" || echo "N"
Y
[root@centos6 ~]# [ 32 -ge 22 ] && echo "Y" || echo "N"
Y
[root@centos6 ~]# [ 10 -ge 22 ] && echo "Y" || echo "N"
N
[root@centos6 ~]# [ 10 -le 22 ] && echo "Y" || echo "N"
Y
```

3.5 Shell 引号

在 Shell 中可以通过使用单引号、双引号、反引号（键盘中 Tab 键上方的按键）、反斜线来转换某些 Shell 元字符的含义。如我们有时希望 echo 命令输出的字符中就包含$符号本身，但一般情况下，Shell 会将$视为取变量值，像这样的情况我们需要使用某些功能来屏蔽$符号本身的特殊含义，使其还原字面意义。

3.5.1 反斜线

反斜线可以将紧随其后的单个字符视为字面意义上的字符，如*在 Shell 中代表任意字符，在查找时经常会使用*来查找多个匹配的文件，然而有可能你需要找的就是*字符本身，此时，*将会把*作为字母意义上的普通字符。

另外，如果在命令的末尾使用\回车后，\可以将回车的命令提交功能屏蔽，从而将回车认为是换行继续输入命令，实现命令的多行输入功能。

```
[root@centos6 ~]# echo *                                    #显示当前目录的所有文件列表
anaconda-ks.cfg install.log install.log.syslog
[root@centos6 ~]# echo \*                                   #显示*字符
*
[root@centos6 ~]# echo \>                                   #显示>符号
>
[root@centos6 ~]# find / \                                  #换行输入多行命令
> -name "test.txt" \
> -type f \
> -size +5M
```

3.5.2　单引号

单引号可以将它中间的所有任意字符还原为字面意义，实现屏蔽 Shell 元字符的功能。注意，不可以在两个单引号中间单独插入一个单引号，单引号必须成对出现。

```
[root@centos6 ~]# echo '$HOME'                              #单引号将屏蔽$的特殊功能
$HOME
[root@centos6 ~]# echo 'test\'                              #默认\为转义换行，这里也被屏蔽
test\
```

3.5.3　双引号

双引号类似于单引号，但其不会屏蔽 `、\和$ 这三个 Shell 元字符的含义，如果需要屏蔽这些字符含义，必须前置一个\符号，其他字符的功能将被屏蔽（包括单引号）。也就是说，两个双引号之间的单引号不必成对出现。

```
[root@centos6 ~]# echo "This's book."
This's book.
[root@centos6 ~]# echo "$HOME"
/root
[root@centos6 ~]# echo "\$HOME"
$HOME
```

3.5.4　反引号

Shell 中使用反引号进行命令替换，命令替换使 Shell 可以将命令字符替换为命令执行结果

的输出内容。同样的功能也可以使用$()来实现。

```
[root@centos6 ~]# echo "Today is `date +%D`"
Today is 02/09/13
[root@centos6 ~]# echo "Today is $(date +%D)"
Today is 02/09/13
```

3.6 正则表达式

实际工作中，公司需要对外招聘人才，但大千世界人才众多，并不一定每个人都适合该岗位，这时我们可以有很多方法找到公司需要的人。常用的方式有两种，第一，通过朋友介绍直接精准地定位到人；第二，写招聘简章（对需要的人才进行描述：学历、经验、技能、语言等），写完后，通过招聘会、网络招聘等方式广纳人才，通常，描述写得越细，越能快速精准地定位所需人才。

而正则表达式就是一种计算机描述语言，你可以直接告诉计算机你需要的是字母 A 来精确匹配定位，也可以告诉计算机你需要的是 26 个字母中的任意一个匹配，等等。现在很多程序、文本编辑工具、编程语言都支持正则表达式，但任何语言都需要遵循一定的语法规则，正则表达式也不例外，正则表达式的发展经历了基本正则表达式与扩展正则表达式，扩展正则表达式是在基本正则表达式的基础上添加一些更加丰富的匹配规则而成。在 Linux 世界中有句古老的说法“Everything is a file（一切皆文件）”，而且很多配置文件是纯文本文件，工作中，我们时常需要对大量的服务器进行配置的修改，如果以手动方式在海量数据中进行查找匹配并最终完成修改，其效率极低。此时，使用正则表达式是非常明智的选择。接下来，我们分别看看每种表达式的具体规则。由于不同的工具对正则的支持有所不同，表 3-5 列出了系统常用编辑工具与正则表达式的对应关系。

注意，正则表达式中有些匹配字符与 Shell 中的通配符符号一样，但含义却不同。

表 3-5

编辑工具	基本正则表达式	扩展正则表达式
grep	√	
egrep	√	√
vi	√	
sed	√	
awk	√	√

3.6.1　基本正则表达式（Regular Expression）

1. 基本正则表达式及其含主

表 3-6 列出了基本正则表达式及其对应的含义。

表 3-6

字符	含义
c	匹配字母 c
.	匹配任意单个字符
*	匹配前一个字符出现零次或多次
.*	匹配任意多个任意字符
[]	匹配集合中的任意单个字符，括号中为一个集合
[x-y]	匹配连续的字串范围
^	匹配字串的开头
$	匹配字串的结尾
[^]	匹配否定，对括号中的集合取反
\	匹配转义后的字串
\{n,m\}	匹配前一个字符重复 n 到 m 次
\{n,\}	匹配前一个字符重复至少 n 次
\{n\}	匹配前一个字符重复 n 次
\(\)	将\(与\)之间的内容存储在“保留空间”，最大存储 9 个
\n	通过\1 至\9 调用保留空间中的内容

2. 基本正则表达式案例

```
[root@centos6 ~]# cp /etc/passwd /tmp/                    #复制模板文件
```

提示

由于模板文件的内容在每个系统中略有差异，以下案例的输出结果可能有所不同。

查找包含 root 的行：

```
[root@centos6 ~]# grep root /tmp/passwd
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
```

查找:与 0:之间包含任意两个字符的字串，并显示该行（--color 代表以颜色加亮显示匹配的内容）：

```
[root@centos6 ~]# grep --color :..0: /tmp/passwd
root:x:0:0:root:/root:/bin/bash
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
games:x:12:100:games:/usr/games:/sbin/nologin
avahi-autoipd:x:170:170:Avahi IPv4LL Stack:/var/lib/avahi-autoipd:/sbin/nologin
```

查找包含至少一个 0 的行（第一个 0 必须出现，第二个 0 可以出现 0 次或多次）：

```
[root@centos6 ~]# grep --color 00* /tmp/passwd
root:x:0:0:root:/root:/bin/bash                          #该行有两处匹配
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin      #匹配 0 出现 2 次
gopher:x:13:30:gopher:/var/gopher:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
avahi-autoipd:x:170:170:Avahi IPv4LL Stack:/var/lib/avahi-autoipd:/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin
```

查找包含 oot 或 ost 的行：

```
[root@centos6 ~]# grep --color  o[os]t /tmp/passwd
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
postfix:x:89:89::/var/spool/postfix:/sbin/nologin
```

查找包含数字 0~9 的行（因输出内容较多，这里仅列出部分输出）：

```
[root@centos6 ~]# grep -color [0-9] /tmp/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
```

查找包含字母 f~q 的行（因输出内容较多，这里仅列出部分输出）：

```
[root@centos6 ~]# grep --color  [f-q] /tmp/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
```

查找以 root 开头的行：

```
[root@centos6 ~]# grep --color ^root /tmp/passwd
root:x:0:0:root:/root:/bin/bash
```

查找以 bash 结尾的行：

```
[root@centos6 ~]# grep --color bash$ /tmp/passwd
root:x:0:0:root:/root:/bin/bash
```

查找 sbin/后面不跟 n 的行：

```
[root@centos6 ~]# grep --color sbin/[^n] /tmp/passwd
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
```

查找数字 0 出现 1 次、2 次的行：

```
[root@centos6 ~]# grep --color '0\{1,2\}' /tmp/passwd
root:x:0:0:root:/root:/bin/bash
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
gopher:x:13:30:gopher:/var/gopher:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
avahi-autoipd:x:170:170:Avahi IPv4LL Stack:/var/lib/avahi-autoipd:/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin
```

查找包含两个 root 的行（注意，grep 在使用\(\)过滤时，匹配条件必须使用引号）：

```
[root@centos6 test]# grep --color "\(root\).*\1" /tmp/passwd
root:x:0:0:root:/root:/bin/bash
```

查找包含以 root:开头并以:root 结尾的字串行：

```
[root@centos6 test]# grep --color "\(root\)\(:\).*\2\1" /tmp/passwd
root:x:0:0:root:/root:/bin/bash
```

过滤文件的空白行：

```
[root@centos6 test]# grep ^$ /tmp/passwd
```

过滤文件的非空白行：

```
[root@centos6 test]# grep -v ^$ /tmp/passwd
```

3.6.2 扩展正则表达式（Extended Regular Expression）

1. 扩展正则表达式及其含义

表 3-7 列出了扩展正则表达式及其对应的含义。

表 3-7

字符	含义
{n,m}	等同于基本正则表达式的\{n,m\}
+	匹配前一个字符出现一次或多次
?	匹配前一个字符出现零次或一次
|	匹配逻辑或者，即匹配|前或后的字串
()	匹配正则集合

2. 扩展正则表达式案例

由于输出信息与基本正则表达式类似，这里仅写出命令，不再打印输出信息。

查找数字 0 出现 1 次和 2 次的行：

```
[root@centos6 ~]# egrep --color '0{1,2}' /tmp/passwd
```

查找包含至少一个 0 的行：

```
[root@centos6 ~]# egrep --color '0+' /tmp/passwd
```

查找包含 root 或者 admin 的行：

```
[root@centos6 ~]# egrep --color '(root|admin)' /tmp/passwd
```

3.6.3 POSIX 规范

由于基本正则表达式会有语系的问题，所以这里我们需要了解 POSIX 规范的正则表达式规则。例如，在基本正则表达式中可以使用 a~z 来匹配所有的字母，但如果我们需要匹配的对象是中文或其他语言怎么办？所以，a~z 仅针对的是英语语系中的所有字母，POSIX 其实是由一系列规范组成的，有些类似于 ISO 国家标准规范，这里仅介绍 POSIX 正则表达式规范，POSIX 的正则规范帮助我们解决了语系问题，另外，POSIX 的表达方式也比较接近于自然语言。表 3-8 列出了 POSIX 字符集规范。

表 3-8

字符集	含义	字符集	含义
[:alpha:]	字母字符	[:graph:]	非空格字符
[:alnum:]	字母与数字字符	[:print:]	任何可以显示的字符
[:cntrl:]	控制字符	[:space:]	任何产生空白的字符
[:digit:]	数字字符	[:blank:]	空格与 Tab 键字符
[:xdigit:]	十六进制数字字符	[:lower:]	小写字符
[:punct:]	标点符号	[:upper:]	大写字符

Linux 允许通过方括号使用 POSIX 标准规则，如[[:alnu:]]将匹配任意单个字母和数字字符。下面通过几个简单的例子来说明用法（由于过滤输出的内容较多，以下仅列出部分输出）。

```
[root@centos6 ~]# grep --color [[:digit:]] /tmp/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
[root@centos6 ~]# grep --color [[:alpha:]] /tmp/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
[root@centos6 ~]# grep --color [[:punct:]] /tmp/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
[root@centos6 ~]# grep [[:space:]] /tmp/passwd
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
vcsa:x:69:69:virtual console memory owner:/dev:/sbin/nologin
```

3.6.4　GNU 规范

在 Linux 中的 GNU 软件一般还会支持转义元字符，这些转义元字符包括：\b（边界字符，匹配单词的开始或结尾，例如，then，hello the world.字串每个单词的前后都有一个\b，可以使用\bthe\b 来匹配单词，但不会匹配 then）、\B（与\b 为反义词，\Bthe\B 不会匹配单词 the，仅会匹配 the 在中间的单词，如 atheist）。其中，\b 等同于[:alnum:]，\B 等同于[^[:alnum:]]。接下来看几个简单的例子。

```
[root@centos6 ~]# grep --color "i\b" /tmp/passwd          #匹配 i 结尾的单词
avahi-autoipd:x:170:170:Avahi IPv4LL Stack:/var/lib/avahi-autoipd:/sbin/nologin
avahi:x:70:70:Avahi mDNS/DNS-SD Stack:/var/run/avahi-daemon:/sbin/nologin
[root@centos6 ~]# grep --color "\W" /tmp/passwd
```

```
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
```

3.7 Sed

3.7.1 Sed 简介

Sed 是一款流编辑工具，用来对文本进行过滤与替换操作，特别是当你想要对几十个配置文件做统一修改时，你会感受到 Sed 的魅力！Sed 通过一次仅读取一行内容来对某些指令进行处理后输出，所以 Sed 更适合于处理大数据文件。首先，Sed 通过文件或管道读取文件内容，但 Sed 默认并不直接修改源文件，而是将读入的内容复制到缓冲区中，我们称之为模式空间（pattern space），所有的指令操作都是在模式空间中进行的，然后 Sed 根据相应的指令对模式空间中的内容进行处理并输出结果，默认输出至标准输出（即屏幕上）。Sed 工作流程如图 3-1 所示。

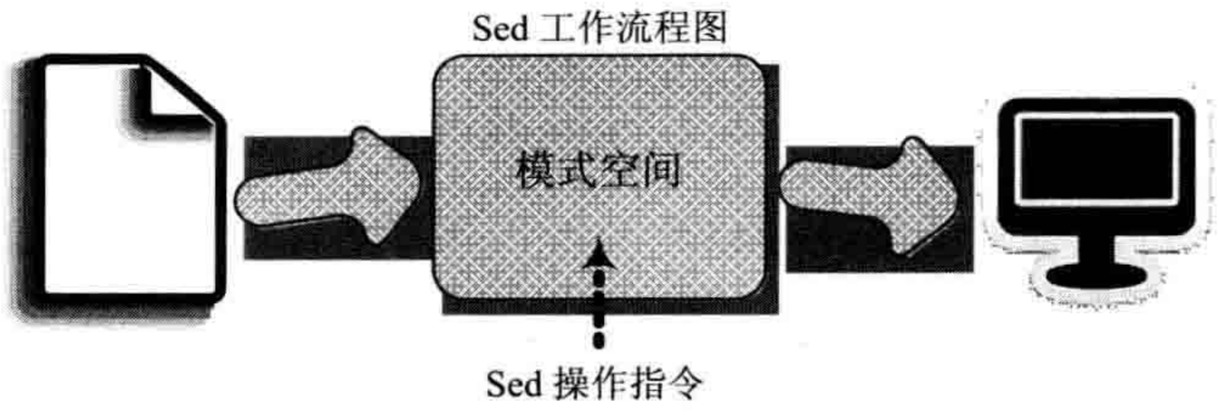


图 3-1

3.7.2 Sed 基本语法格式

Sed 从文件中读取数据，如果没有输入文件，则默认对标准输入进程数据进行处理，脚本指令是第一个非“-”开头的参数，具体语法格式如下。

用法：sed [选项]... {脚本指令} [输入文件]...

选项：--version　　显示 sed 版本。

--help　　显示帮助文档。

-n,--quiet,--silent　　静默输出，默认情况下，sed 程序在所有的脚本指令执行完毕后，将自动打印模式空间中的内容，该选项可以屏蔽自动打印。

-e script	允许多个脚本指令被执行。
-f script-file	从文件中读取脚本指令，对编写自动脚本程序很实用。
-i,--in-place	慎用，该选项将直接修改源文件。
-l N	该选项指定 l 指令可以输出的行长度，l 指令为输出非打印字符。
--posix	禁用 GNU sed 扩展功能。
-r	在脚本指令中使用扩展正则表达式。
-s, --separate	默认情况下，sed 将把输入的多个文件名作为一个长的连续的输入流。而 GNU sed 则允许把它们当作单独的文件。
-u, --unbuffered	最低限度的缓存输入与输出。

3.7.3　Sed 入门范例

1. 基本格式范例

Sed 通过特定的脚本指令对文件进行处理，这里就简单介绍几个脚本指令操作作为 Sed 程序的范例。a,append 表示追加指令；i,insert 表示插入指令；d,delete 表示删除指令；s,substitution 表示替换指令。sed 脚本指令的基本格式是：[地址]命令（有些命令仅可以对一行操作，有些可以对多行操作），命令也可以用花括号进行组合，使命令序列可以作用于同一个地址。

```
address{
command1
command2
command3
}
```

警告 第一个命令可以和左花括号在同一行，但右花括号必须单独处于一行。此外，命令后添加空格会产生错误。

下面的 test.txt 为操作样本源文件（注意有若干空白行），介绍 Sed 的用法。

```
[root@centos6 ~]# cat test.txt
DEVICE=eth0
BOOTPROTO=static
IPADDR=192.168.0.1
NETMASK=255.255.255.0
```

```
GATEWAY=192.168.0.254

ONBOOT=yes
[root@centos6 ~]# sed '2a TYPE=Ethernet' test.txt #第二行后追加 TYPE=Ethernet
[root@centos6 ~]# sed '3i TYPE=Ethernet' test.txt #第三行前追加 TYPE=Ethernet
[root@centos6 ~]# sed 's/yes/no/g' test.txt       #将样本文件中的所有 yes 替换为 no
[root@centos6 ~]# sed '3,4d' test.txt             #删除第 3、4 行的内容
```

以上大多数操作指令都依据行号定位操作对象（地址），如：2a 即第二行后追加。

实际工作中，可能大多数情况你并不确定你要操作对象（地址）的行号，这时更多的会使用正则表达式确定操作对象（地址）。下面是使用正则表达式定位操作行的示例：

匹配到包含 ONBOOT 的行，并在其后添加 TYPE=Ethernet：

```
[root@centos6 ~]# sed '/ONBOOT/a TYPE=Ethernet' test.txt
```

匹配以 GATEWAY 开始的行，并删除该行：

```
[root@centos6 ~]# sed '/^GATEWAY/d' test.txt
```

另外，我们的操作指令可以写入到脚本文件中，并通过 sed 的-f 选项读取，脚本文件中的注释行是以#开始的行，如果#后面的字符为 n，则屏蔽 Sed 程序的自动输出功能，等同于命令选项-n。

创建一个 sed 脚本，内容如下：

```
[root@centos6 ~]# cat sed.sh                  #脚本内容为匹配到空白行后，删除该行
#This is a test sed command
/^$/d
[root@centos6 ~]# sed -f sed.sh test.txt      #对 test.txt 文件执行 sed.sh 脚本指令
```

而当你需要执行多个指令时，可以使用以下三种方法：

```
[root@centos6 ~]# sed 's/yes/no/;s/static/dhcp/' test.txt  #使用分号隔开指令
[root@centos6 ~]# sed -e 's/yes/no/' -e 's/static/dhcp/' test.txt   #使用-e 选项
[root@centos6 ~]# sed '                                    #利用分行
>s/yes/no/
>s/static/dhcp/' test.txt
```

然而在命令行上输入过长的指令是愚蠢的，这时需要使用-f 选项指定 sed 脚本文件，在脚本文件中可以包含多行指令，而且也便于修改。

2. 操作地址匹配范例

通过以上范例不难发现，我们编写的脚本指令需要指定一个地址来决定操作范围，如果不

指定，则默认对文件的所有行进行操作。如：sed 'd' test.txt 将删除 test.txt 的所有行，而'2d'则仅删除第二行。Sed 为我们提供了以下这些方式来确定需要操作地址的范围。

```
number          指定输入文件的唯一行号
first~step      指定以 first 开始，并指定操作步长为 step，如 1、2 指定第一行、第三行、
                第五行... ...为操作地址。2~5 指定第二行开始，每 5 行匹配一次操作地址
[root@centos6 ~]# sed -n '1~2p' test.txt    #打印文件的奇数行
$               匹配文件的最后一行
/regexp/        //中间包含的是正则表达式，通过正则表达式匹配操作地址。如果//中正则表
                达式为空，匹配最近一次正则表达式的匹配地址，后面会有范例
\cregexpc       \c 与 c 之间匹配扩展正则表达式，c 字符可以使用任意字符替代
addr1,addr2     匹配从操作地址 1 到操作地址 2 的所有行
[root@centos6 ~]# sed '2,8d' test.txt       #删除 2~8 之间的所有行
addr1,+N        匹配地址 1 以及后面的 N 行内容
```

3.7.4　Sed 指令与脚本

1. Sed 常用指令汇总

表 3-9 给出了常用的 sed 脚本指令的说明，下面分别看看每个指令的详细用法。

表 3-9

指令	功能	指令	功能
s	替换	d	删除
a	追加	i	插入
c	更改	l	打印（显示非打印字符）
y	按字符转换	L	打印（不显示非打印字符）
p	打印	r	读入文件内容
w	保存至文件	q	退出

2. 部分指令详解

（1）替换指令（s,Substitution）

指令格式：　[address]s/pattern/replacement/flags

address 为操作地址，s 为替换指令，/pattern/匹配需要替换的内容，/replacement/为替换的新内容。Flags 标记可以是：

n　　1~512 之间的数字，表示对模式空间中指定模式的第 n 次出现进行替换。如一行中有 3 个 A，而只想替换第二个 A。

g	对模式空间的所有匹配进行全局更改。没有 g 则只有第一次匹配被替换，如一行中有 3 个 A，则仅替换第一个 A。
p	打印模式空间的内容。
w file	将模式空间的内容写到文件 file 中。

replacement 为字符串，用来替换与正则表达式匹配的内容。在 replacement 部分，只有下列字符有特殊含义。

&	用正则表达式匹配的内容进行替换。
\n	匹配第 n 个子串，该子串之前在 pattern 中用\(\)指定。
\	转义（转义替换部分包含：&、\等）。

（2）删除指令（d ,delete）

删除指令用于删除匹配的行，而且删除命令还会改变 sed 脚本中命令的执行顺序。因为匹配的行一旦被删除，模式空间将变为“空”，自然不会再执行 sed 脚本后续的命令。删除命令将导致读取新的输入行（下一行），而 sed 脚本中的命令则从头开始重新执行。需要注意的是，删除时是删除整行，而不只是删除匹配的内容（如要删除匹配的内容，可以使用替换）。

（3）转换指令（y）

按字符转换（Transform）的语法格式为：[address]y/source-chars/dest-chars/，其中，[address]用来定位需要修改的行，source-chars 为需要被修改的字符，dest-chars 为准备替换的字符。

3. Sed 脚本指令范例

（1）范例 1

范例 1 所使用的样本文件为：

```
[root@centos6 ~]# cat test.txt
<html>
<title>First Web</title>
<body>Hello the World! <body>
</html>
```

范例 1：将样本文件中的第二个<body>替换为</body>。

编写 sed 脚本，替换与行匹配相同的内容，即将 body 替换为/body，但仅替换第二个 body 为/body）。

```
[root@centos6 ~]# cat sed.sh
/body/{
s//\/body/2
}
```

执行 sed 程序的结果如下：

```
[root@centos6 ~]# sed -f sed.sh test.txt
<html>
<title>First Web</title>
<body>Hello the World!</body>
</html>
```

（2）范例 2

范例 2 所使用的样本文件为：

```
[root@centos6 ~]# cat test.txt
<html>
<title>First Web</title>
<body>
h1Helloh1
h2Helloh2
h3Helloh3
</body>
</html>
```

范例 2：给所有第一个的 h1,h2 等添加<>；第二个 h1,h2 添加</>。

编写 sed 脚本为：

```
[root@centos6 ~]# cat sed.sh
/h[0-9]/{
s//\<&\>/1
s//\<\/&\>/2
}
```

说明 匹配 h 紧跟一个数字的行，替换与上一行中匹配内容相同的内容，即将 h[0-9]替换为<&>，其中&为前面要替换的内容。上一条指令仅替换第一个 h1,h2...，本行指令用来替换第二个 h1,h2...。

执行 sed 程序的结果如下：

```
[root@centos6 ~]# sed -f sed.sh test.txt
<h1>Hello</h1>
<h2>Hello</h2>
<h3>Hello</h3>
```

技巧 关于's///'命令的另一个妙处是：'/'分隔符有许多替换方案，如果规则表达式或替换字符串中有许多斜杠，则可以通过在 's' 之后指定一个不同的字符来更改分隔符。

示例： sed -e 's:/usr/local:/usr:g' mylist.txt

此时是替换分隔符，sed 会将/usr/local 替换为/usr。

（3）范例 3

范例 3 所使用的样本文件为（注意有空白行）：

```
[root@centos6 ~]# cat test.txt
DEVICE=eth0
ONBOOT=yes
BOOTPROTO=static

IPADDR=192.168.0.1
NETMASK=255.255.255.0

GATEWAY=192.168.0.254
```

范例 3：删除文件中的空白行。

编写 sed 脚本为：

```
[root@centos6 ~]# cat sed.sh
/.*/{
/^$/d
}
```

执行 sed 程序的结果如下：

```
[root@centos6 ~]# sed -f sed.sh test.txt
DEVICE=eth0
ONBOOT=yes
BOOTPROTO=static
IPADDR=192.168.0.1
NETMASK=255.255.255.0
GATEWAY=192.168.0.254
```

（4）范例 4~范例 7

范例 4~范例 7 所使用的样本文件为：

```
[root@centos6 ~]# cat test.txt
DEVICE=eth0
ONBOOT=yes
BOOTPROTO=static
```

```
NETMASK=255.255.255.0
GATEWAY=192.168.0.254
```

范例 4：在 static 行后添加一行，内容为 IPADDR=192.168.0.1。

```
[root@centos6 ~]# sed '/static/a IPADDR=192.168.0.1' test.txt
```

范例 5：在匹配 NETMASK 的行前插入内容 IPADDR=192.168.0.1。

```
[root@centos6 ~]# sed '/NETMASK/i IPADDR=192.168.0.1' test.txt
```

范例 6：将包含 ONBOOT 行的内容更改为 ONBOOT=yes。

```
[root@centos6 ~]# sed '/ONBOOT/c ONBOOT=yes' test.txt
```

范例 7：列印（l）显示模式空间中的内容，显示非打印字符，一般与-n 一起使用，否则会输出两次。

```
[root@centos6 ~]# sed -n '1,21' test.txt #在 sed 脚本文件中，需要#n 屏蔽自动输出
```

结果如下：

```
DEVICE=eth0$
ONBOOT=yes$
```

（5）范例 8 和范例 9

范例 8、范例 9 所使用的样本文件为：

```
[root@centos6 ~]# cat test.txt
DEVICE=eth0
ONBOOT=yes
BOOTPROTO=static
netmask=255.255.255.0
GATEWAY=192.168.0.254
```

范例 8：将小写转换为大写。

编写 sed 脚本为：

```
[root@centos6 ~]# cat sed.sh
/.*/{
/netmask/y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/
}
```

执行 sed 程序的结果如下：

```
[root@centos6 ~]# sed -f sed.sh test.txt
DEVICE=eth0
ONBOOT=yes
BOOTPROTO=static
NETMASK=255.255.255.0
```

```
GATEWAY=192.168.0.254
```

范例 9：显示第一、二行的内容。

打印（p）：作用类似于 l（列印），但不显示非显示字符，一般与-n 配合使用。

```
[root@centos6 ~]# sed -n '1,2p' test.txt
DEVICE=eth0
ONBOOT=yes
```

（6）范例 10 和范例 11

范例 10、范例 11 所使用的样本文件为：

```
[root@centos6 ~]# cat name.txt
Jacob
Tom
Jerry
[root@centos6 ~]# cat mail.txt
jacob@gmail.com
tom@gmail.com
jerry@gmail.com
```

范例 10：先读取 name.txt 文件内容，再读取 mail.txt 文件内容。

编写 sed 脚本为：

```
[root@centos6 ~]# cat sed.sh
/.*/{
$r mail.txt
}
```

执行 sed 程序的结果如下：

```
[root@centos6 ~]# sed -f sed.sh name.txt
Jacob
Tom
Jerry
jacob@gmail.com
tom@gmail.com
jerry@gmail.com
```

范例 11：显示 name.txt 内容的前两行内容后退出 sed 指令。

```
[root@centos6 ~]# sed '2q' test.txt
```

3.7.5 Sed 高级应用

正常的 Sed 数据处理流程是读取文档的一行至模式空间，然后对该行应用相应的 Sed 指令，

当指令完成后输出该行并清空模式空间，依此循环读入文档的下一行数据，直至文档数据结尾。然而在真实环境中的数据可能不会那么有规律，有时我们会把数据分多行写入文档，如：

```
姓名：张三，
邮箱：zhangsan@gmail.com
姓名：李四，
邮箱：lisi@gmail.com
```

从上面的模板文件中可以看出，实际上每两行为一条完整的记录，而此时如果需要使用 Sed 对该文档进行处理，就需要对 Sed 工作流程进行人工干预。

1. 多行操作 Next

Next（N）指令通过读取新的输入行，并将它追加至模式空间的现有内容之后，来创建多行模式空间。模式空间的最初内容与新的输入行之间用换行符分隔。在模式空间中插入的换行符可以使用\n 匹配。

Next 范例 1 所使用的样本文件为：

```
[root@centos6 ~]# cat test.txt
Name:Tom,
Mail:Tom@gmail.com
Name:Jerry
Mail:Jerry@gmail.com
```

编写 Sed 指令脚本如下（读取样本文件内容至模式空间，当读取的内容与 Name 匹配时，立刻读取下一行内容，再输出模式空间中的内容，#n 用来屏蔽自动输出）。

```
[root@centos6 ~]# cat sed.sh
]#n
/Name/{
N
L
}
[root@centos6 ~]# sed -f sed.sh test.txt          #运行脚本结果如下
Name:Tom, mail:Tom@gmail.com
name:Jerry, mail:Jerry@gmail.com
```

Next 范例 2 所使用的样本文件为：

```
[root@centos6 ~]# cat test.txt
111
222
222
222
333
```

编写 Sed 指令脚本（读取样本文件内容至模式空间，当读取的内容与 222 匹配时，立刻读取下一行内容，再输出模式空间中的内容，小写的 l 会打印非打印字符）。

```
[root@centos6 ~]# cat sed.sh
#n
/222/{
N
l
}
[root@centos6 ~]# sed -f sed.sh test.txt                    #运行脚本结果如下
222\n222$
222\n333$
```

2. 多行操作 Print

Print（P）即多行打印 P，它与打印 p 稍有不同，前者仅输出多行模式空间中的第一部分，直到第一个插入的\n 换行符为止。

综合范例所使用的样本文件为：

```
[root@centos6 ~]# cat test.txt
aaa
bbb
ccc
ddd
eee
fff
```

下面通过多条 sed 命令对比不同打印方式的差别，输出结果见表 3-10，各个 sed 命令输出信息分析如下。

表 3-10

sed '/.*/N' test.txt	sed '/.*/N;L' test.txt	sed '/.*/N;P' test.txt	sed '/.*/N;p' test.txt
aaa	aaa bbb	aaa	aaa
bbb	aaa	aaa	bbb
ccc	bbb	bbb	aaa
ddd	ccc ddd	ccc	bbb
eee	ccc	ccc	ccc
fff	ddd	ddd	ddd
	eee fff	eee	ccc
	eee	eee	ddd
	fff	fff	eee
			fff
			eee
			fff

第一个 sed 命令使用了 N 读取下一行，新读取的内容与原有内容直接使用\n 分隔。但 sed 读取下一行后没有任何后续指令，所以 sed 自动输出，即输出源文件的所有内容。

第二个 sed 命令使用了 N 读取下一行，L 表示显示模式空间的内容，即 aaa bbb，同时 sed 命令的自动输出功能会把源文件内容显示出来，即 aaa，bbb。依此类推，sed 继续读取第三行 ccc，并使用 N 将 ddd 追加至行尾，使用 L 显示模式空间的内容（不显示非打印字符），sed 自动输出后再把源文件内容显示出来。

第三个 sed 命令使用 N 将下一行追加至行尾，现在模式空间中的内容为 aaa\nbbb，而 P 指令的作用是打印模式空间中的第一部分内容直到\n 结尾，即仅打印 aaa，这时 sed 的自动输出功能输出 aaa，bbb（sed 自动输出会将\n 输出为换行）。依此类推，读取第三行 ccc，N 将 ddd 追加至行尾，P 打印\n 前的内容，同时 sed 命令也将自动输出。

第四个 sed 命令的原理类似于第三个 sed 命令，但 p 打印时，\n 看作是回车换行，所以打印出来的是 aaa 回车 bbb。

3. 多行删除操作 Delete（D）

d 指令为删除命令，其作用是删除模式空间中的内容并读入新的输入行，而如果 sed 在 d 指令后还有多条指令，则余下的指令将不再执行，而是返回第一条指令对新读入行进行处理。而多行删除指令 D 将删除模式空间中直到第一个插入的换行符（\n）前的这部分内容，它不会读入新的输入行，并返回 sed 脚本的顶端，使得剩余指令继续应用于模式空间中剩余的内容。

4. Hold（h,H），Get（g,G）

我们知道，模式空间是存放当前输入行的缓冲区。除此之外，Sed 还有一个称为保持空间（hold space）的缓冲区。模式空间的内容可以复制到保持空间，保持空间的内容同样可以复制到模式空间，有一组 Sed 命令用于在两者之间移动数据。

Hold(h|H)	将模式空间的内容复制或追加到保持空间
Get(g|G)	将保持空间的内容复制或追加到模式空间
Exchange(x)	交换保持空间与模式空间的内容

保留空间范例所使用的样本文件如下：

```
[root@centos6 ~]# cat test.txt
aaa
bbb
ccc
ddd
```

```
[root@centos6 ~]# cat  sed.sh
/aaa/{
h
d
}
/ccc/{
G
}
[root@centos6 ~]# sed -f sed.sh test.txt
bbb
ccc
aaa
ddd
```

3.8 Awk

3.8.1 Awk 简介

Awk 是一种编程语言，用于在 Linux/UNIX 下对文本和数据进行扫描与处理，数据可以来自标准输入、文件、管道。Awk 分别代表其作者姓氏的第一个字母，因为它的作者是三个人，分别是 Alfred Aho、Peter Weinberger、Brian Kernighan。实际上，awk 有很多种版本，如：awk、nawk、mawk、gawk、MKS awk、tawk 等，这其中有开源产品，也有商业产品。目前在 Linux 中常用的 awk 编译器版本有 mawk、gawk，其中以 RedHat 为代表使用的是 gawk，以 Ubuntu 为代表使用的是 mawk。gawk 是 GNU Project 的 awk 解释器的开源代码实现。本书将以 CentOS 6.3 系统自带的 gawk 作为讲解工具。

3.8.2 Awk 工作流程

Awk 的工作流程是：逐行扫描文件，从第一行到最后一行，寻找匹配特定模式的行，并在这些行上进行用户想要的操作。Awk 基本结构由模式匹配和处理过程（即处理动作）组成。

```
pattern  {action}
```

Awk 读取文件内容的每一行时，将对比该行是否与给定的模式相匹配，如果匹配，则执行处理过程，否则对该行不做任何处理。如果没有指定处理脚本，则把匹配的行显示到标准输出，即默认处理动作是 print 打印行；如果没有指定模式匹配，则默认匹配所有的数据。

Awk 有两个特殊的模式：BEGIN 和 END，它们被放置在没有读取任何数据之前以及在所有的数据读取完成以后执行。

图 3-2 展示了 Awk 工作的整体流程，从图中可以看出，在读取文件内容前，BEGIN 后面的指令将被执行，然后读取文件内容并判断是否与特定的模式匹配，如果匹配，则执行正常模式后面的动作指令，最后执行 END 模式命令，并输出文档处理后的结果。

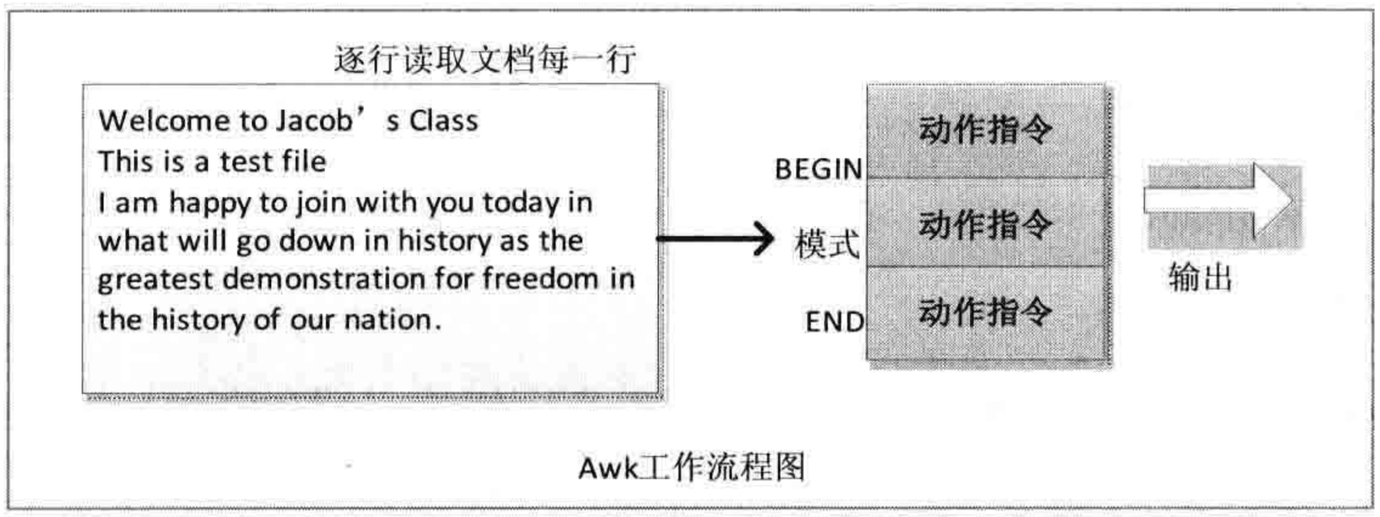


图 3-2

3.8.3　Awk 基本语法格式

Awk 的基本语法格式如下：

gawk [选项] -f program-file [--] file ...

选项如下：

◎　-F fs，--field-separator fs

指定以 fs 作为输入行的分隔符（默认分隔符为空格或制表符）。

◎　-v var=val，--assign var=val

在执行处理过程以前，设置一个变量 var 值为 val。

◎　-f program-file，--file program-file

从脚本文件中读取 Awk 指令，以取代在命令参数中输入处理脚本。

◎　-W compat，-W traditional，--compat，--traditional

使用兼容模式运行 Awk，GNU 扩展选项将被忽略。

◎　-W copyleft，-W copyright，--copyleft，--copyright

输出简短的 GNU 版权信息。

◎ -W dump-variables[=file]，--dump-variables[=file]

打印全局变量（变量名、类型、值）到文件中，如果没有提供文件名，则自动输出至名为dump-variables 的文件中。

示例如下：

```
[root@centos6 ~]# awk -W dump-variables=out.txt 'x=1 {print x}' test.txt
-W exec file, --exec file
```

类似于-f 选项，但脚本文件需要以#!开头。另外，命令行的变量将不再生效。

◎ -W help，-W usage，--help，--usage

显示各个选项的简短描述。

awk 程序的语法结构是：一个 awk 程序包含一系列的模式 {动作指令} 或是函数定义，模式可以是 BEGIN、END、表达式，用来限定操作对象的多个表达式使用逗号分隔；动作指令需要以{}引起来。

简单示例：

（1）通过正则表达式/^$/匹配空白行，动作为打印 Blank line，即文件如果有 N 个空白行，awk 将在屏幕打印 N 个 Blank line。

```
[root@centos6 ~]# awk '/^$/ {print "Blank line}' test.txt
```

（2）打印包含主机名的行，没有指定动作指令，默认动作为打印。

```
[root@centos6 ~]# awk '/HOSTNAME/' /etc/sysconfig/network
```

（3）提前编辑一个 awk 脚本，再通过-f 选项调用该脚本。

```
[root@centos6 ~]# cat awk.sh
/^$/ {print "Blank line}
[root@centos6 ~]# awk -f awk.sh test.txt
```

3.8.4 Awk 操作指令

1. 记录与字段

Awk 一次从文件中读取一条记录，并将记录存储在字段变量$0 中。记录被分割为字段并存储在$1,$2,…, $NF 中（默认使用空格或制表符为分隔符）。内建变量 NF 为记录的字段个数。

示例：

读取输入行并输出第一个字段、第二个字段、第三个字段。

```
[root@centos6 ~]# echo hello the world | awk  '{print  $1,$2,$3}'
```

读取输入行并输出该行。

```
[root@centos6 ~]# echo hello the world | awk  '{print  $0}'
```

读取输入行并输出该行的字段个数。

```
[root@centos6 ~]# echo hello the world | awk  '{print  NF}'
```

读取输入行并输出该行最后一个字段。

```
[root@centos6 ~]# echo hello the world | awk  '{print  $NF}'
```

2. 字段分隔符

默认 Awk 读取数据以空格或制表符作为分隔符，但可以通过-F 或 FS（field separator）变量来改变分隔符。

```
[root@centos6 ~]# awk -F: '{print $1}' /etc/passwd
[root@centos6 ~]# awk 'BEGIN {FS = ":"} {print $1}' /etc/passwd
```

注意，以上两个示例均将字段的分隔符改为冒号（:），即以冒号为分隔符打印 passwd 文件的第一个字段（账号名称）。如果使用 FS 改变分隔符，需要在 BEGIN 处定义 FS，因为在读取第一行前，就需要改变字段分隔符。

进阶：指定多个字段分隔符(文档内容为：hello the:word,!)

```
[root@centos6 ~]# echo 'hello the:word,!' | awk 'BEGIN {FS="[:, ]"} {print $1,$2,$3,$4}'
```

3. 内置变量

表 3-11 为 Awk 内置变量列表。

表 3-11

变量名称	描述
ARGC	命令行参数个数
FILENAME	当前输入文档的名称
FNR	当前输入文档的当前记录编号，尤其是当有多个输入文档时有用
NR	输入流的当前记录编号
NF	当前记录的字段个数
FS	字段分隔符
OFS	输出字段分隔符，默认为空格
ORS	输出记录分隔符，默认为换行符\n
RS	输入记录分隔符，默认为换行符\n

示例：

```
[root@centos6 ~]# cat test1.txt
This is a test file.
Welcome to Jacob's Class.
[root@centos6 ~]# cat test2.txt
Hello the wrold.
Wow! I'm overwhelmed.
Ask for more.
```

输出当前文档的当前行编号，第一个文件两行，第二个文件三行：

```
[root@centos6 ~]# awk '{print FNR}' test1.txt test2.txt
1
2
1
2
3
```

Awk 将两个文档作为一个整体的输入流，通过 NR 输入当前行编号：

```
[root@centos6 ~]# awk '{print NR}' test1.txt test2.txt
1
2
3
4
5
[root@centos6 ~]# awk '{print NF}' test1.txt
#文档的第一行有 5 个字段，第二行有 4 个字段
5
4
[root@centos6 ~]# awk  'BEGIN {FS = ":"} {print $1}'  /etc/passwd
[root@centos6 ~]# awk  '{print $1,$2,$3}'  test1.txt
```

说明

默认 print 输出时，各参数将输出分隔符默认为空格，所以输出内容如下：

```
This is a
Welcome to Jacob's
```

下面通过 OFS 将输出分隔符设置为"-"，这个 print 在输出第一、二、三个字段时，中间的分隔符为"-"，结果如下：

```
[root@centos6 ~]# awk 'BEGIN {OFS="-"} {print $1,$2,$3}' test1.txt
This-is-a
Welcome-to-Jacob's
[root@centos6 ~]# cat test3.txt
mail from: tomcat@gmail.com
```

```
subject:hello
data:2012-07-12 17:00
content:Hello, The world.

mail from: jerry@gmail.com
subject:congregation
data:2012-07-12 08:31
content:Congregation to you.

mail from: jacob@gmail.com
subject:Test
data:2012-07-12 10:20
content:This is a test mail.
```

读取输入数据，以空白行为记录分隔符，即第一个空白行前的内容为第一个记录，第一个记录中字段分隔符为换行符：

```
[root@centos6 ~]# awk 'BEGIN {FS="\n"; RS=""} {print $3}' test3.txt
```

以上 Awk 实现打印所有的邮件时间，即每个记录的第三个字段。

4. 表达式与操作符

表达式由变量、常量、函数、正则表达式、操作符组成，Awk 中变量有字符变量和数字变量。如果在 Awk 中定义的变量没有初始化，则初始值为空字串或 0。注意，字符操作时一定记住需要加引号。

变量定义示例：

```
a="welcome to beijing"
b=12
```

操作符（awk 操作符与 C 语言类似）如下。

+	加
-	减
*	乘
/	除
%	取余
^	幂运算
++	自加 1
--	自减 1
+=	相加后赋值给变量(x+=9 等同于 x=x+9)

-=	相减后赋值给变量(x-=9 等同于 x=x-9)
*=	相乘后赋值给变量(x*=9 等同于 x=x*9)
/=	相除后赋值给变量(x/=9 等同于 x=x/9)
>	大于
<	小于
>=	大于或等于
<=	小于或等于
==	等于
!=	不等于
~	匹配
!~	不匹配
&&	与
||	或

操作符简单示例：

```
[root@centos6 ~]# echo "test" | awk 'x=2 {print x+3}'
[root@centos6 ~]# echo "test" | awk 'x=2,y=3 {print x*2, y*3}'
[root@centos6 ~]# awk '/^$/ {print x+=1}' test.txt     #统计所有的空白行
[root@centos6 ~]# awk '/^$/ {x+=1} END {print x}' test.txt   #打印总空白行个数
[root@centos6 ~]# awk -F: '$1~/root/ {print $3}' /etc/passwd#打印 root 的 ID 号
[root@centos6 ~]# awk -F: '$3>500 {print  $1}' /etc/passwd
                                                    #列出计算机中 ID 号大于 500 的用户名
```

3.8.5 Awk 高级应用

1. IF 条件判断

If 语法格式 1：

```
if（表达式）
动作 1
else
动作 2
```

If 语法格式 2：

```
if（表达式） 动作 1; else 动作 2
```

如果表达式的判断结果为真，则执行动作 1，否则执行动作 2。

示例：判断 boot 分区可用容量小于 20MB 时报警，否则显示 OK。

```
[root@centos6 ~]# df |grep "boot" |awk '{if($4<20000) print "Alart"; else print "OK"}'
```

2. while 循环

while 语法格式 1：

```
while（条件）
动作
```

语法格式示例如下：

```
x=1
while (i < 10) {
        print $i
    }
```

示例：

```
[root@centos6 ~]# awk 'i=1 {} BEGIN { while (i<=10) {++i; print i}}' test.txt
```

While 语法格式 2：

```
do
动作
while (条件)
```

示例：

```
[root@centos6 ~]# awk 'BEGIN { do {++x;print x} while (x<=10)}' test.txt
```

3. for 循环

```
for (变量；条件；计数器)
     动作
```

示例：

```
[root@centos6 ~]# awk 'BEGIN {for(i=1;i<=10;i++) print i}' test.txt
[root@centos6 ~]# awk 'BEGIN {for(i=10;i>=1;i--)  print i}' test.txt
```

因为以上循环语句使用的 awk 均使用 BEGIN 模式，也就是说，在未读取文档内容前就会将 BEGIN 代码执行完毕，所以输入文档可以为任意文档。

4. Break 与 Continue

break 跳出循环
continue　　终止当前循环

示例（打印 1-4，6-10）：

```
for  (i=1; i<=10;i++) {
    if (i=5)
        continue
    print i
}
```

示例（仅打印 1-4）：

```
for  (i=1; i<=10;i++) {
    if (i=5)
        break
    print i
}
```

5. 函数

（1）rand ()函数

作用：产生 0~1 之间的浮点类型的随机数，rand 产生随机数时需要通过 srand()设置一个参数，否则单独的 rand()每次产生的随机数都是一样的。

示例：

```
[root@centos6 ~]# awk 'BEGIN {print  rand(); srand(); print srand()}'  test.txt
```

（2）gsub(x,y,z)函数

作用：在字串 z 中使用字串 y 替换与正则表达式 x 相匹配的所有字串，z 默认为$0。

（3）sub(x,y,z)函数

作用：在字串 z 中使用字串 y 替换与正则表达式 x 相匹配的第一个字串，z 默认为$0。

将 passwd 每行中所有的 root 修改为 jacob 显示至屏幕：

```
[root@centos6 ~]# awk -F: 'gsub(/root/,"jacob",$0) {print $0}' /etc/passwd
```

将 passwd 每行中第一个 root 修改为 jacob 显示至屏幕：

```
[root@centos6 ~]# awk -F: 'sub(/root/,"jacob",$0) {print $0}' /etc/passwd
```

sub 相当于 sed 中的 s///，gsub 相当于 sed 中的 s///g。

（4）length(z)函数

作用：计算并返回字串 z 的长度。

显示 test.txt 文档每行的字符长度：

```
[root@centos6 ~]# awk '{print length()}' test.txt
```

（5）getline 函数

作用：从输入中读取下一行内容。从下面 df -h 命令的输出结果可以看出，分区的剩余容量显示在第 4 列，但唯独/根分区的记录显示在了两行。我们需要判断当记录的字段个数为 1 时，读取下一行，并将该行的第 3 列显示至屏幕。

```
[root@centos6 ~]# df  -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/mapper/VolGroup00-LogVol00
                       19G  3.6G   15G  21% /
/dev/sda1              99M   14M   81M  15% /boot
tmpfs                 141M     0  141M   0% /dev/shm
none                  140M  104K  140M   1% /var/lib/xenstored
[root@centos6 ~]# df -h |awk '{if(NF==1) {getline; print $3}; if(NF==6) print $4}'
[root@centos6 ~]# df -h |awk 'BEGIN {print "Disk Free:"}  \
> {if(NF==1) {getline; print $3}; if(NF==6) print $4}'
```

3.9　Shell 脚本

经常有人说脚本就是命令的堆积，这种观点是错误的。命令是脚本，尤其是 Shell 脚本的重要组成部分，但不是全部，脚本中还包含变量设置、控制与循环、逻辑运算等。与使用 C 语言等其他编程语言所编写的程序不同，脚本运行时需要调用相应的解释器来翻译脚本中的内容，从而实现最终被成功执行。根据我们编写的脚本格式不同，需要的解释器也不同，如果你编写的是一段 Shell 脚本代码，那么就需要调用 bash 程序来告诉系统如何执行这个 Shell 脚本，如果你编写的是一段 Python 脚本，则执行该脚本时就需要调用 Python 程序来解释翻译你所编写的脚本代码，最终完成脚本的正常运行工作。目前脚本的种类众多，常见的脚本有：Shell 脚本、Java 脚本、PHP 脚本、Python 脚本、Perl 脚本等，其中最简单的是 Shell 脚本。

因为我们所编写的脚本如果想被执行，每次都需要调用相应的解释器翻译一遍脚本代码的含义，所以，脚本程序的执行效率会比 C++之类的编程语言慢很多。但脚本的编写效率往往比一些高级的程序设计语言要高很多，有些简单的操作使用 Shell 脚本仅需一分钟就可以编写完成，但想使用高级编程语言来完成同样的工作则可能需要花费一天的时间来编写相应的代码。

另外，脚本的优势在于简单（易学易用），很多人都可以很快完成对脚本的学习，并迅速上手编写属于自己的脚本，而且脚本一般都以文本文档格式存在，我们可以使用任何文本编辑工具对其进行修改，使得工作中查看、编辑、审核脚本内容变得非常容易，只需要一款文本编辑器即可。本节我们就来看看如何编写一些基本的 Shell 脚本程序，最终实现运维工作自动化、智能化，从而提高工作效率和生产效率。

3.9.1 脚本格式范例

每一个完善的脚本都要遵循一些既定的规则，Shell 脚本也一样，有自己独特的格式，下面我们来看一个完整的 Shell 脚本，分析其格式风格。第一个例子是 Hello the world，创建脚本文件/root/print.sh，脚本内容如下：

```
#!/bin/bash
# Copyright © 2012, Jacob Publish Press. All rights reserved.
#
# This program is print the "Hello the world" to screen.
# Date:2012-12-12
# Version:0.1

echo "Hello the world"
exit 0
```

在 Hello the world 的例子中，我们给出了一个相对完善的脚本所应该具有的基本框架。第一行#!的作用是指定该脚本程序的命令解释器，例子中是/bin/bash。也就是说，脚本执行后，系统内核读取#!后面的路径查找解释器，最终使用该解释器翻译脚本代码并运行。脚本文件中#后面的部分为注释，在脚本程序执行时，该部分会被忽略，这些注释为代码的编写与阅读提供辅助信息。接下来就是脚本的代码部分，这些代码默认会按顺序依次被执行，但通过控制语句控制执行顺序时例外。

3.9.2 运行脚本的方式

脚本编写完成后，我们需要运行并实现脚本程序的功能，那么如何执行脚本呢？其实，脚本的执行可以有很多种方式，而且各种方式运行的效果也不尽相同，下面我们来分别看看运行脚本程序的三种不同方式。

1. 赋予权限，直接运行脚本

```
[root@centos6 ~]# chmod a+x print.sh
```

```
[root@centos6 ~]# /root/print.sh                    #通过绝对路径执行脚本
[root@centos6 ~]# ./print.sh                        #通过相对路径执行脚本
```

2. 没有权限，通过 bash 或 sh 运行脚本

```
[root@centos6 ~]# bash print.sh                     #调用 bash 程序解释脚本内容并执行
[root@centos6 ~]# sh print.sh                       #调用 sh 程序解释脚本内容并执行
```

3. 没有权限，通过.或 source 运行脚本

```
[root@centos6 ~]# source print.sh
[root@centos6 ~]# . print.sh                        #.与脚本之间有空格
```

不同执行方式的区别在于，赋予脚本权限后直接运行脚本将在用户当前 Shell 下开启一个新的子进程，并在子进程的环境中运行脚本程序；通过 bash 或 sh 命令加载脚本文件并执行，此时系统将不再关心#!后面的解释器，而是直接使用 bash 或 sh 作为解释器解释脚本内容并执行；通过.或 source 方式执行脚本，则脚本将在用户当前 Shell 环境中运行。

3.9.3　Shell 脚本简单案例

案例 1：使用脚本计算《红楼梦》一书的作者

我国的红学研究界一直以来都在为谁是《红楼梦》一书的作者争论不已，每个人的论点不一，各有各的论点与论据。其中比较靠谱的有两种说法，一种说法是《红楼梦》全书为曹雪芹所著，另一种说法是《红楼梦》前八十回为曹雪芹著，后四十回为程伟元与高鹗著。但追究根源，目前没有一种说法是十全十美的。仅目前，红学界公认的普遍观点是《红楼梦》的原作者是曹雪芹，高鄂只是续写者。《红楼梦》第一回正文中提到，此书经“曹雪芹于悼红轩中披阅十载，增删五次，纂成目录，分出章回”。而在这些证明《红楼梦》作者的方法中，有一种是比较新颖的方式，就是统计小说的用词频率，即一般人写作总有自己的风格、习惯用语。所以有人通过对比《红楼梦》前八十回与后四十回的用词频率，看是否相同，词频类似于文章的指纹。

无独有偶，1983 年到 1987 年，贝尔实验室的 Jon Bentley 发表了有趣的系列专栏 *Programming Pearls*，Jon 在专栏中提出了一个问题：写一个程序，找出 *n* 个出现最频繁的单词，并在输出结果中显示这些单词出现的次数，最后按次数排序。很多人花费了大量的时间解决该问题，但花费的时间与程序效率尚需考虑，同时贝尔实验室的 Doug McIlroy 提供了一个六步解决方案，几分钟就可完成编写，效率相当高，该程序使用 Shell 脚本编写。但该脚本并不适合于中文文档的处理，因为英文单词与单词之间总存在空格，这样脚本第一步才可以将整篇文档处理为一行一

个单词继，而通过排序求词频，这是前提。但中文的汉字与汉字之间是不存在空格的，所以基于上面的案例，在此改写了一份脚本用以处理中文，脚本内容如下。

```
#!/bin/bash
for i in {1..80}
do
  cut -c$i $1 >>one.txt
  cut -c$i $2 >>two.txt
done
grep "[[:alnum:]]" one.txt > onebak.tmp
grep "[[:alnum:]]" two.txt > twobak.tmp
sort onebak.tmp |uniq -c |sort -k1,1n -k2,2n |tail  -30 > oneresult
sort twobak.tmp |uniq -c |sort -k1,1n -k2,2n |tail  -30 > tworesult
rm -rf one.txt two.txt onebak.tmp twobak.tmp
vimdiff oneresult tworesult
```

脚本说明：这里假设屏幕每行可以输出 80 个汉字，sort 命令对文档排序；uniq -c 去除重复的字符且显示字符出现的个数；sort -k1,1n -k2 按第一列排序（按词频排序），n 表示以数字格式排序，如果第一列的词频相同，则按后面的字符继续排序。此外，关于 for 循环语句，将在 3.9.5 节有详细的解释与案例。

下面的对比数据是使用该脚本计算《红楼梦》前八十回与后四十回文章词频的结果，脚本计算后将显示词频最高的 30 个汉字，下面仅给出前十五个汉字的对比数据。

8541 一	3942 来
7376 来	3758 道
7197 道	3741 是
6715 人	3649 人
6377 我	3474 一
6299 是	3392 说
6231 说	2741 我
5417 他	2382 这
5359 这	2378 着
5104 你	2245 他
4148 去	2203 里
4081 个	2138 贾
4024 也	2066 儿
4008 儿	2065 玉
3945 玉	2046 有

由于前后章节的基数不同，所以对比的数据词频的量有一定的差异，但通过观察不难发现，前后章节所使用的频率最高的汉字中有大量的重叠。也就是说，前八十回与后四十回写作风格与用词习惯应该是相同的。

案例 2：制作程序菜单

下面编写一个脚本实现程序主菜单的显示，脚本主题使用 echo 命令实现，脚本内容如下。

```
#!/bin/bash
#This is a menu script
clear
echo "************************************"
echo -e "*\033[1;31m\t\tMenu\t\t\033[0m*"
echo "************************************"
echo "1.Display system CPU info and system load"
echo "2.Display system Mem and swap info"
echo "3.Display filesystem mount info"
echo "4.Display system network interface info"
echo "5.Exit"
```

脚本说明：整个脚本仅实现菜单输出，菜单输出前通过 clear 将屏幕清空，使用的是 echo 命令实现菜单内容的输出。另外，echo 使用-e 选项开启转义功能，通过\033[1;31m 可以输出颜色，1 为样式与前景色，31 为字体颜色，这里使用红色输出 Menu 后再次使用\033[0m 来关闭颜色设置，也就是后续的输出不再使用红色输出。

案例 3：统计系统基本信息

为了说明脚本的基本格式，这里使用最简单的命令堆积的方式实现计算机系统基本信息的搜集，该脚本没有对输出信息做任何设计与排版处理，仅仅作为脚本文件基本格式讲解示例使用，如果需要在生产环境中使用类似功能的脚本，还需要稍作处理与优化，在后续章节中会有更加完善的脚本以供参考，脚本内容如下。

```
#!/bin/bash
#This script can be used to collect system basic information.
echo "------------------------------------"
echo "Display CPU info:"
echo $(cat /proc/cpuinfo)
echo "------------------------------------"
echo "Display system load:"
echo $(uptime)
echo "------------------------------------"
```

```
    echo "Display Mem and swap info:"
    echo $(free)
    echo "-----------------------------------"
    echo "Display filesystem mount info:"
    echo "df -h"
    echo "-----------------------------------"
    echo "Display network interface info:"
    echo $(ip addr show)
    echo "-----------------------------------"
```

Shell 脚本如果仅仅是命令的堆积的话，那么也就失去了 Shell 编程的意义，Shell 脚本的强大之处就在于它可以像其他高级编程语言一样进行判断、循环、控制以及函数处理。接下来我们将重点关注这些内容，只有完全掌握了这些知识，再结合 Linux 命令才可以写出满足实际需要的优秀脚本。当然，初学者可以先从模仿入手，多看多练，通过日积月累的知识沉淀，你会发现编写脚本其实很简单。

3.9.4 判断语句应用

判断语句的使用使得脚本具有了智能判断的功能，通过判断，我们可以分析计算机操作环境、工作状态，实现脚本的智能化，让脚本根据不同的环境执行不同的操作。Shell 脚本的判断可以使用 if 语句或 case 语句，下面分别看看这两种语句的语法格式，并给出应用案例说明使用方法。

1. if 语句

表 3-12 为常用 if 语句的语法格式，其中命令序列代表命令集合，可以是一条命令，也可以是多条命令。也就是说，当判断条件满足时，可以仅执行一条命令，也可以执行一系列命令，格式中对条件的判断可以使用 test 命令，也可以使用[]测试。

表 3-12

语法格式 1	语法格式 2	语法格式 3
if 条件 then 命令序列 fi	if 条件 then 命令序列 else 命令序列 fi	if 条件 then 命令序列 elif 条件 then 命令序列 elif 条件

续表

语法格式 1	语法格式 2	语法格式 3
		then 命令序列 else 命令序列 fi

if 语句示例如下。

示例 1：判断当前用户是否为 root 管理员，如果是，则执行 tar 对/etc/目录执行备份。

```
#!/bin/bash
#If current user is root then bakcup the /etc.
if [ "$(id -u)" -eq "0" ]; then
tar -czf /root/etc.tar.gz /etc&>/dev/null
fi
```

示例 2：要求用户输入密码，判断密码是否正确。

```
#!/bin/bash
#Read password and test.
read -p "Enter a password:" password
if [ "$password" == "pass" ]; then
echo "OK"
fi
```

示例 3：要求用户输入密码，判断密码是否正确。若正确，显示 OK；若不正确，则显示 ERROR。

```
#!/bin/bash
#Read password and test.
read -p "Enter a password:" password
if [ "$password" == "pass" ]
then
echo "OK"
else
echo "ERROR"
fi
```

示例 4：读取参数判断成绩，成绩小于 60 分将显示 Fail；成绩大于或等于 60 分但小于 70 分，则显示 pass；成绩大于或等于 70 分但小于 80 分，则显示 fine；成绩在 80 分以上则显示 excellence。

```
#!/bin/bash
```

```
#Test score, and print the level.
if [ $1 -ge 80 ];then
echo "excellence"
elif [ $1 -ge 70 ];then
echo "fine"
elif [ $1 -ge 60 ];then
echo "pass"
else
echo "Fail"
fi
```

2. case 语句

在 Shell 脚本中，除了使用 if 语句进行判断外，还可以使用 case 语句进行判断，表 3-13 给出了 case 语句的语法结构，case 语句是多重 if 判断语句的替换解决方案，它易读、易写。case 语句通过检查模式与变量值是否匹配，如果匹配，则执行相应模式下的命令序列，命令序列可以是单条命令，也可以是多条命令的集合，*)下的命令为 case 默认动作，当变量与所有的模式都没有匹配时，case 将执行*)下的命令序列。模式可以使用通配符，模式下的命令序列最后必须要以;;结尾的默认命令序列除外，代表该模式下的命令到此结束。最后，case 语句使用 esac 结束整个语句。

表 3-13

语法格式 1	语法格式 2
case $变量名称 in 模式 1) 命令序列 ;; 模式 2) 命令序列 ;; 模式 N) 命令序列 ;; *) esac	case $变量名称 in 模式 1|模式 2) 命令序列 ;; 模式 3|模式 4) 命令序列 ;; 模式 5|模式 6) 命令序列 ;; *) esac

case 语句示例如下。

示例 1：根据时间备份/var/log 日志目录，仅备份周三、周五的数据。

```
#!/bin/bash
DATE=$(date +%a)
TIME=$(date +%Y%m%d)
case $DATE in
   Wed|Fri)
tar -czf /usr/src/${TIME}_log_tar.gz /var/log/&>/dev/null
;;
   *)
echo "Today neither Wed nor Fri."
esac
```

示例2：根据用户输入的参数不同返回不同的提示字符，如果用户输入字符，则提示 You have type a character；如果用户输入数字，则提示 You have type a number，否则提示 ERROR。

```
#!/bin/bash
case $1 in
   [a-z]|[A-Z])                          #判断$1 是否是字母
echo "You have type a character."
;;
   [[:digit:]])                           #判断$1 是否是数字
echo "You have type a number."
;;
   *)
echo "Error."
esac
```

示例3：编写 firefox 火狐浏览器启动脚本，支持 start、stop、restart 功能。

```
#!/bin/bash
case $1 in
   start)
firefox &
;;
   stop)
pkill firefox
;;
   restart)
pkill firefox
firefox &
;;
```

```
    *)
echo "Usage:$0 (start|stop|restart)"
esac
```

3.9.5 循环语句应用

1. for 语句

日常系统管理工作中有大量需要重复运行的指令，Shell 编程提供了 for、while、until、select 循环语句以实现特定指令的反复执行功能，在所有的循环语句中，变量必须要有初始值，每次运行命令序列前都需要对条件进行过滤，满足条件才会运行命令，否则不执行相关操作。表 3-14 为 for 语句的语法格式，for 语句通过对变量的定义取值决定循环的执行次数，根据变量赋值的次数，for 循环内的指令将被执行相同的次数，如定义一个变量 i 取值为 1、3、5，则最终 for 循环将循环执行命令 3 次。表 3-14 中，for 语句的语法格式 1 对变量的赋值通过在 in 后面直接赋值，多个赋值通过空格隔开；语法格式 2 则是首先通过初始化变量的值，也就是定义一个变量的初始值，然后通过运算修改变量的值，当结束循环的条件满足时，for 循环结束。

表 3-14

语法格式 1	语法格式 2
for 变量 in 值 1 值 2 … 值 N do 命令序列 done	for ((初始化变量值;结束循环条件;运算)) do 命令序列 done

for 语句示例如下。

示例 1：给多个用户群发邮件，邮件内容为/var/log/messges 文件中的内容，标题为 Log。

```
#!/bin/bash
DOMAIN=gmail.com
for MAIL_U in tom jerry smith                    #MAIL_U 分别赋值为 tom、jerry、smith
do
mail -s "Log" $MAIL_U@$DOMAIN < /var/log/messges
done
```

示例 2：多次给 NUM 变量赋值，并显示变量 NUM 的值。

```
#!/bin/bash
#Print the variable values.
for NUM in {1..20}                               #NUM 取值为 1 到 20 之间的整数
do
```

```
echo $NUM
done
```

示例 3：通过 for 语句打印 9×9 的乘法表。

```
#!/bin/bash

for i in {1..9}
do
        for ((j=1;j<=i;j++))
        do
                printf "%-8s" $j*$i=$((j*i))
        done
        echo
done
```

2. while 语句

表 3-15 为 while 语句的语法格式，while 语句在循环前对运行条件进行判断，运行条件满足时，循环将一直被执行，直到测试条件失败时循环结束。while 语句可以通过 read 命令每次读取一行文件内容，文件内容有多少行，while 循环就执行多少次，读取文件结束时，循环结束。

表 3-15

语法格式 1	语法格式 2
while [条件] do 命令序列 done	while read -r line do 命令序列 done < file

while 语句示例如下。

示例 1：批量添加 20 个用户，用户名称为 userN，N 为 1 至 20 的数字。

```
#!/bin/bash
#Add twenty users through wile loop.
U_NUM=1
while [ $U_NUM -le 20 ]                    #只要 U_NUM 的值小于或等于 20，循环就一直执行
do
useradd user${U_NUM}
U_NUM=$((U_NUM+1))
done
```

示例 2：打印网卡配置文件的每一行。

```
#!/bin/bash
#Read /etc/sysconfig/network-scripts/ifcfg-eth0 and print out.

FILE=/etc/sysconfig/network-scripts/ifcfg-eth0
while read -r line
do
echo $line
done < $FILE
```

示例 3：无限循环菜单，根据用户的选择实现不同菜单的功能，最后通过相应的菜单项退出脚本。

```
#!/bin/bash
while true                                  #无限循环 while
do
clear
echo "-----------------------------------"
echo "1. Display CPU info:"
echo "2. Display system load:"
echo "3. Display Mem and swap info:"
echo "4. Display filesystem mount info:"
echo "5. Exit Program:"
echo "-----------------------------------"
read -p "Please select an iterm(1-5):" U_SELECT
case $U_SELECT in
   1)
echo $(cat /proc/cpuinfo)
read -p "Press Enter to continue:"
;;
   2)
echo $(uptime)
read -p "Press Enter to continue:"
;;
   3)
echo $(free)
read -p "Press Enter to continue:"
```

```
;;
    4)
echo "$(df -h)"
read -p "Press Enter to continue:"
;;
5)
exit
;;

    *)
read -p "Please Select 1-5, Press Enter to continue:"
esac
done
```

3. until 语句

表 3-16 为 until 语句的语法格式，该语句根据条件判断循环是否继续执行，until 代表的是直到条件满足时循环结束。

表 3-16

语法格式
until [条件] do 命令序列 done

until 语句示例如下。

批量删除用户，用户名称为 userN，N 是 1~20 之间的数字。

```
#!/bin/bash
#Delete user.
U_NUM=20
until [ $U_NUM -eq 0 ]
do
userdel user${U_NUM}
U_NUM=$((U_NUM-1))
done
```

4. select 语句

select 语句可以用来生成菜单项目，select 循环与 for 循环格式相同。下面通过 select 生成询问籍贯的提问菜单，并通过 echo 回显用户的选择项。

```
#!/bin/bash
echo "Where are you from? "
select  var  in  "Beijing"  "Shanghai"  "New York"  "Chongqing"
    do
break
    done
echo "You are from $var"
```

3.9.6 控制语句应用

Shell 支持的控制语句有 shift、continue、break、exit，Shell 脚本默认会按顺序依次执行脚本中的命令，通过控制语句可以人为控制脚本的执行顺序与流程。

shift 的作用是将位置参数左移一位，也就是执行一次 shift 命令后，$2 将变为$1，依此类推，这样可以通过$1 调用所有的命令参数。下面的示例可以很好地说明具体的使用细节。

```
[root@centos6 ~]# cat shift.sh
#!/bin/bash
for i in $@
do
echo $1
shift
done
[root@centos6 ~]# chmod a+x shift.sh
[root@centos6 ~]# ./shift.sh hello the world
hello
the
world
```

运行脚本时指定了三个参数，分别是 hello、the 和 world，参数之间用空格隔开，此时$1=hello,$2=the,$3=world，运行 for 指令先输出$1，然后执行 shift 命令，该命令执行后，位置参数将左移一位。也就是说，the 现在变成了$1 的值，world 变成了$2 的值，所以 shift 命令执行结束后继续运行 for 循环，第二次输出的$1 是 the，输出 the 后继续执行 shift 命令，使位置变

量再次左移一位，此时 world 变成了$1 的值。

continue 用来在 for、while、until 循环中使当前循环中断执行，从而进入下一次循环体，而 break 则可以用来结束整个 for、while、until 循环语句的执行，最后 exit 可以用来结束脚本的运行。下面的示例脚本说明了这些控制语句的功能与区别。

```
#!/bin/bash
for IP_SUFFIX in {1..254}
do
case $IP_SUFFIX in
   10)
      continue
      ;;
   15)
      break
esac
   echo ${IP_SUFFIX}
done
sleep 5                                          #让脚本暂停 5 秒钟
exit
echo "The would't be printed content"
```

IP_SUFFIX 为 10 时，由于执行了 continue 命令，使当前 for 循环后续的指令不再被执行，从而直接进入下一循环，也就是 IP_SUFFIX 将赋值为 11。当 IP_SUFFIX 为 15 时，整个 for 循环将结束。也就是说，第 15~254 次循环根本就不会运行，因为到 15 时，for 循环执行了 break 命令，break 将使 for 循环强制结束，不管 for 循环内部后续是否还有需要执行的指令，但脚本不会结束，如果 for 循环外面还有其他命令，将按顺序继续执行。最后，exit 用来退出当前 Shell 脚本，所以示例脚本中的最后一条 echo 命令永远都不会被执行。

3.9.7　Shell 函数应用

在编写脚本时，有些语句会被重复使用多次，把这些可能重复使用的代码写成函数，这样我们通过函数名称可以更高效地重复利用它们。如果想让自己写的脚本代码可以为别人所使用，同样需要函数的功能。另外，当脚本比较复杂时，我们需要将脚本划分为多个模块以简化脚本，这也是函数的应用案例。表 3-17 为 Shell 函数的语法格式，该函数需要先定义，后使用，使用时直接通过函数名称调用即可。

表 3-17

语法格式 1	语法格式 2
name() {	function name {
命令序列	命令序列
}	}

函数应用示例：根据用户对菜单的选择调用不同的函数功能。

```
#!/bin/bash
#Simple function demo.
HINT(){                                                    #打印提示符
read -p "Press Enter tocontinue:"
}
CPU_INFO(){                                                #查看 CPU 信息
echo
echo -e "\033[4;31mPrint the CPU info:\033[0m"
cat /proc/cpuinfo |awk 'BEGIN {FS=":"} /model name/{print "CPU Model:" $2}'
cat /proc/cpuinfo |awk 'BEGIN {FS=":"} /cpu MHz/{print "CPU Speed:" $2"MHz"}'
grep -Eq 'svm|vmx' /proc/cpuinfo && echo "Virtualization: Support" || \
echo "Virtualization: No support"
echo
}
LOAD_INFO(){                                               #查看系统负载
echo
echo -e "\033[4;31mPrint the system load:\033[0m"
uptime |awk 'BEGIN{FS=":"}{print $5}'|awk 'BEGIN{FS=","}\      #\为转义换行
{print "Last 1 minutes system load:"$1"\n""Last 5 minutes system load:"$2"\n"\
"Last 15 minutes system load:"$3}'
echo
}
MEM_INFO(){                                                #查看内存与交换分区信息
echo
echo -e "\033[4;31mPrint the Memory and Swap info:\033[0m"
free |grep buffers/cache|awk '{print "Mem free:"$4" Bytes"}'
free |grep Swap|awk '{print "Swap free:"$4" Bytes"}'
echo
}
DISK_INFO(){                                               #查看磁盘挂载信息
```

```
echo
echo -e "\033[4;31mPrint system disk space usage:\033[0m"
df -h
echo
}
while true
do
clear
echo "----------------------------------"
echo "1. Display CPU info:"
echo "2. Display system load:"
echo "3. Display Mem and swap info:"
echo "4. Display filesystem mount info:"
echo "5. Exit Script:"
echo "----------------------------------"
read -p "Please select an iterm(1-4):" U_SELECT
case $U_SELECT in
   1)
CPU_INFO                                    #通过函数名称调用函数
HINT
;;
   2)
LOAD_INFO
HINT
;;
   3)
MEM_INFO
HINT
;;
   4)
DISK_INFO
HINT
;;
   5)
exit
;;
   *)
read -p "Please Select 1-4, Press Enter to continue:"
```

```
    esac
    done
```

3.9.8 综合案例

案例 1：快速自动化安装配置 DHCP 服务器脚本

```
#!/bin/bash
#Auto deploy DHCP Server for 192.168.0.0/24 network
#This script need you have yum repository.
#Author: Jacob
#变量定义，主要包括网络、子网、地址池等信息
NET=192.168.0.0
MASK=255.255.255.0
RANGE="192.168.0.10 192.168.0.50"
DNS=202.106.0.20
DOMAIN_NAME="example.com"
ROUTER=192.168.0.254
#函数定义，测试 yum 源
test_yum(){
yum list dhcp >/dev/null 2&>1
if [ $? -ne 0 ] ; then
    echo
    echo "There was an error to connect to Yum repository."
    echo "Please verify your yum repository settings and try again."
    echo
    exit
fi
}
#保存原有的配置文件
test_conf(){
if [ -f /etc/dhcp/dhcpd.conf ];then
    mv /etc/dhcp/dhcpd.conf /etc/dhcp/dhcp.conf.save
fi
}
#创建新的配置文件
create_conf(){
cat > /etc/dhcp/dhcpd.conf <<EOF
```

```
# dhcpd.conf
# Sample configuration file for ISC dhcpd
#
# option definitions common to all supported networks...
default-lease-time 600;
max-lease-time 7200;
# This is a very basic subnet declaration.
# A slightly different configuration for an internal subnet.
subnet $NET netmask $MASK {
  range $RANGE;
  option domain-name-servers $DNS;
  option domain-name "$DOMAIN_NAME";
  option routers $ROUTER;
}
# Fixed IP addresses can also be specified for hosts.
host passacaglia {
  hardware ethernet 0:0:c0:5d:bd:95;
  fixed-address 192.168.0.1;
}
EOF
}
#通过函数名称调用函数功能，实现安装与配置 DHCP 服务
rpm -q dhcp >/dev/null 2&>1
if [ $? -ne 0 ] ; then
test_yum
yum -y install dhcp >/dev/null 2&>1
fi
test_conf
create_conf
service dhcpd start                                    #启动服务
chkconfig dhcpd on                                     #开机启动
```

案例 2：检查密码，如果用户三次输入密码均错误，则退出脚本

```
#!/bin/bash
#Check user password, set the number can be retrying.
NUM=3                                  #密码最多尝试次数
PASSWD=Jacob                           #初始密码
SUM=0                                  #计数器
```

```
while true
do
read -p "Please input your password:" pass        #读取用户输入
SUM=$((SUM+1))                                    #计数器加 1
if [ $pass == $PASSWD ] ; then                    #判断密码是否正确
    echo "Your Are Right, OK"
    exit
elif [ $SUM -lt 3 ] ; then                        #判断密码尝试次数
    continue
else
    exit
fi
done
```

案例 3：Proftpd 启动脚本

```
#!/bin/bash

#Startup script for proftpd
#chkconfig: - 89 77
#Description:Proftpd is a secure and configurable FTP Server.
#Program:/usr/local/proftpd/sbin/proftpd
#Configure file:/etc/proftpd.conf

#Synopsis:
#   proftpd --help {start|stop|restart|status}

#Define variable
Proftpd=/usr/local/proftpd/sbin/proftpd
Pidfile=/usr/local/proftpd/var/proftpd.pid
Program=$(basename $0)
Hint_color="echo -en \\033[1;32m"
Orig_color="echo -e \\033[0m"
Tab="\t\t\t\t\t\t\t"
#Functions

usage(){
echo "Usage:$Program [--help] {start|stop|restart|status}"
}
```

```
start(){
if [ -e $Pidfile ];then
echo "Proftpd alread running..."
else
echo -en "Starting Proftpd:${Tab}[ "
/usr/local/proftpd/sbin/proftpd
${Hint_color}OK;${Orig_color}" ]"
fi
}
stop(){
if [ -e $Pidfile ];then
echo -en "Stopping Proftpd:${Tab}[ "
kill -TERM $(cat ${Pidfile})
${Hint_color}OK;${Orig_color}" ]"
else
echo "Proftpd already stopped."
fi
}
status(){
if [ -e $Pidfile ];then
echo "Proftpd ($(cat $Pidfile)) is running..."
else
echo "Proftpd is stopped."
fi
}
if [ $# -gt 0 ];then
case $1 in
    start)
        start
        ;;
    stop)
        stop
        ;;
    restart)
        stop
        sleep 0.2
        start
```

```
        ;;
    status)
        status
        ;;
    --help|*)
        usage
esac
else
usage
fi
```

案例 4：备份 MySQL 数据库

```
#!/bin/bash
DATE=$(date +%Y-%d-%m)
DES=/usr/src/mysql_bak
MYSQL_U="root"                          #mysql 用户名称
MYSQL_P="xxx"                           #mysql 密码，根据需要修改密码
MYSQL_H="127.0.0.1"                     #mysql 服务器 IP
if [ ! -d "$DES" ] ; then
      mkdir -p "$DES"
fi
#获取数据库名称列表
DB=$(mysql -u $MYSQL_U -h $MYSQL_H -p$MYSQL_P -Bse 'show databases')
#通过循环执行备份所有的 MySQL 数据库
for database in $DB
do
      if [ ! $database == "information_schema" ] ;then
          mysqldump -u $MYSQL_U -h $MYSQL_H -p$MYSQL_P $database \
          |bzip2> "$DES/${DATE}_mysql.gz"
      fi
done
```

3.9.9 图形脚本

在现代计算机的发展中，图形越来越被人接受与喜爱，因为图形直观，更容易接受，使用 Shell 编写脚本多数情况下是为了使运维工作自动化、智能化，然而有时我们也需要编写一些交互式的、更易操作的图形程序，Linux 平台下有一款 dialog 软件可以帮助我们解决这类问题，

dialog 工具可以用来创建终端图形对话框，在脚本中嵌入这样的图形对话框可以让你的脚本更直观、易操作，但缺点是必须要安装 dialog 软件才可以使用。

dialog 命令的描述和用法如下。

描述：在 Shell 环境中显示对话框。

用法：dialog 通用选项 对话框选项

通用选项：--backtitle　　#背景标题

--begin y x　　#指定对话框位置 y 为水平坐标，x 为垂直坐标

--clear　　#清屏

--insecure　　#使密码更人性化，但不安全

--shadow　　#对话框阴影

--height　　#高度

--width　　#宽度

--title　　#对话框标题

dialog 支持很多种对话框类型，例如，打开文件对话框、确认对话框、输入对话框、消息对话框、密码对话框等，通用选项就是对所有的对话框都是有效可用的属性，对话框属性则是针对某种具体对话框的特殊选项。下面通过 dialog 软件自带的几个实例来说明这些对话框的使用方法。

1. 日历对话框

语法格式：--calendar text height width day month year

calendar 指定对话框为日历对话框，title 设置对话框标题，text 为对话框中的提示符可以输入任意字符，day、month、year 分别为日历对话框默认显示的时间，如果命令中没有指定日期，则 dialog 将默认显示系统当前日期。

```
[root@centos6 ~]# dialog --title "CALENDAR" --calendar "Please choose a date..." 0 0 7 7 1981
```

2. 选择对话框

语法格式：--checklist text height width list-height [tag item status] ...

checklist 指定对话框为检查对话框，text 为提示符，list-height 设置列表高度，tag 为每个选择项目的标签，iterm 为具体选项内容，status 为默认状态（on 或 off）。

```
[root@centos6 ~]# dialog --checklist "Select " 10 40 8 1 "Man" "on" 2 "Woman" "off"
```

3. 图形进度条

语法格式：--gauge text height width [percent]

percent 代表进度值，如 10 即代表 10%，注意，该图形模式默认不会自动退出，需要手动按 Ctrl+C 快捷键终止。

```
[root@centos6 ~]# dialog --title "Guage" --guage "percent XXX"  6 60 20
```

4. 图形密码框

语法格式：--passwordbox text height width [init]

需要注意的是，对该图形密码框不要轻易添加 init 参数，该参数为默认密码且不可见（默认系统不会显示用户输入的密码）。下面的案例演示初始密码为 123，最终用户输入的密码将被导入至/tmp/pass 文件。

```
[root@centos6 ~]# dialog --title "INPUT BOX" --clear \
>--passwordbox "INPUT YOUR PASSWORD 123 2> /tmp/pass
```

5. 消息框

语法格式：--msgbox text height width

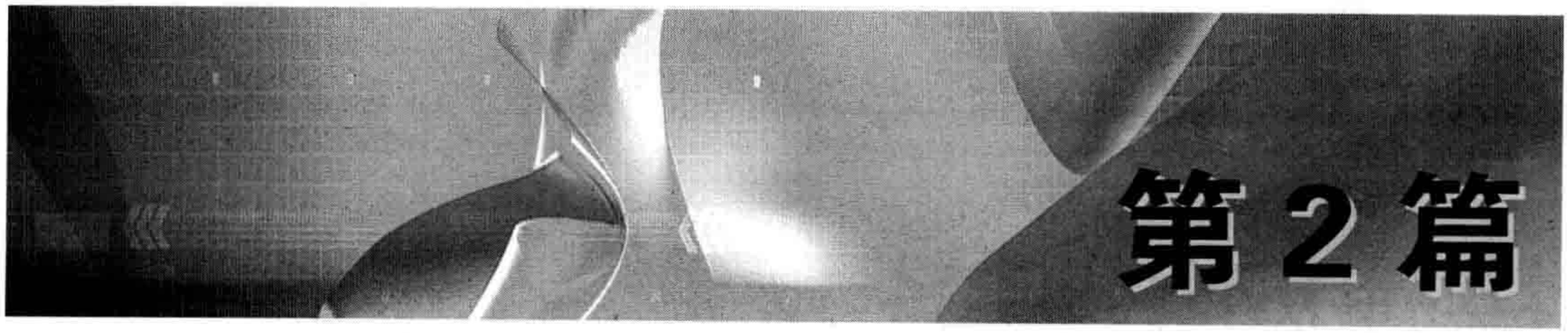
```
[root@centos6 ~]# dialog --title "Message" --clear --msgbox 'Hello The World!' 6 25
```

6. 确认框

语法格式：--yesno text height width

```
[root@centos6 ~]# dialog --title "YES or NO" --clear --yesno "Are You Sure
Your Choice?" 6 30
```

CentOS 6.3 操作平台通过 yum 方式安装完成 dialog 软件后，该软件为我们提供了足够多的完整案例，读者可以在/usr/share/doc/dialog-1.1/samples 目录中找到这些案例。



网络服务

第4章

搭建网络服务

4.1 NFS 文件共享

NFS（Network File System）即网络文件系统，是由 Sun 公司开发的一种通过网络方式共享文件系统的通用共享解决方案。目前 NFS 有三个版本，分别为 NFSv2、NFSv3、NFSv4。NFSv2 是一个古老的版本，但却被众多的操作系统所支持，这样它的兼容性会更好；NFSv3 拥有更多的特点，包括更快的速度、更大的单个文件大小、更多便于排错的错误及成功信息、对 TCP 协议的支持等；NFSv4 提供了有状态的连接，更容易追踪连接状态，增强了安全特性。CentOS 6.3 版本默认使用版本 4 提供 NFS 网络文件系统共享服务，NFS 监听在 TCP 2049 端口。

当两台计算机需要通过网络建立连接时，双方主机就一定需要提供一些基本信息，如：IP 地址、服务端口号等，当有 100 台客户端需要访问某台服务器时，服务器就需要记住这些客户端的 IP 地址以及相应的端口号等信息，而这些信息是需要程序来管理的。在 Linux 中，这样的信息可以由某个特定服务自己来管理，也可以委托给 RPC（Remote Procedure Call）来帮助自己管理。RPC 是远程过程调用协议，RPC 协议为远程通信程序管理通信双方所需的基本信息，这样 NFS 服务就可以专注于如何共享数据，至于通信的连接以及连接的基本信息，则全权委托给

RPC 管理，CentOS 6.3 系统由 rpcbind[1]服务提供 RPC 协议的支持，目前 NFSv4 虽然不再需要与 rpcbind 直接交互，但 rpc.mountd 依然是 NFSv4 所必需的服务。所以，如果在 CentOS 6.3 平台上实现 NFS 共享，需要同时启动 NFS 与 rpcbind 服务。

所需软件包括：nfs-utils、rpcbind。

4.1.1　NFS 服务器配置

NFS 服务器通过读取/etc/exports 配置文件设定哪些客户端可以访问哪些 NFS 共享文件系统，该文件的书写原则如下。

- 空白行将被忽略。
- 以#符号开头的内容为注释。
- 配置文件中可以通过\符号转义换行。
- 每个共享的文件系统需要独立一行条目。
- 客户端主机列表需要使用空格隔开。
- 配置文件中支持通配符。

一条完整的共享条目语法结构如下，其中，客户端主机可以是一个网段、单台主机或主机名。

```
共享路径  客户端主机（选项）
```

我们也可以为多个客户主机设置不同的访问选项，语法结构如下：

```
共享路径      客户端主机 1（选项）  客户端主机 2 （选项）  …
```

最简单的 NFS 配置可以仅给定一个共享路径与一个客户端主机，而不指定选项，因为没有选项时，NFS 将使用默认设置，而默认属性为 ro、sync、wdelay、root_squash。具体的 NFS 属性及其对应的含义见表 4-1，查看 exports 的帮助文档可以找到很多服务器配置模板。

表 4-1

NFS 选项	功能描述	NFS 选项	功能描述
ro	只读共享	rw	可读可写共享
sync	同步写操作	async	异步写操作
wdelay	延迟写操作	root_squash	屏蔽远程 root 权限
no_root_squash	不屏蔽远程 root 权限	all_squash	屏蔽所有的远程用户权限

[1] rpcbind 用来替换 CentOS 老版本中的 portmap 提供 RPC 协议服务。

以上选项中，ro 与 rw 比较容易理解，用来定义客户端访问共享时可以获得的权限是只读访问还是可读写访问。计算机对数据进行修改时会先将修改的内容写入快速的内存，随后才会慢慢写入慢速的硬盘设备中，async 选项允许 NFS 服务器在没有完全把数据写入硬盘前就返回成功消息给客户端，而此时数据实际还存放在内存中，但客户端则显示数据已经写入成功。注意，该选项仅影响操作消息的返回时间，并不决定如何进行写操作，sync 选项将确保在数据真正写入存储设备后才会返回成功信息。wdelay 为延迟写入选项。也就是说，它决定了先将数据写入内存，再写入硬盘，然后将多个写入请求合并后写入硬盘，这样可以减少对硬盘 I/O 的次数，从而优化性能，该选项可以优化 NFS 性能，但有可能导致非正常关闭 NFS 时数据丢失情况的发生。与此相反的选项是 no_wdelay，但该选项与 async 选项一起使用时将不会生效，因为 async 是基于 wdelay 实现对客户端的一种响应功能。默认情况下，NFS 会自动屏蔽 root 用户的权限，root_squash 使得客户端使用 root 账号访问 NFS 时，服务器系统默认会自动将 root 映射为服务器本地的匿名账号，通过 anonuid 可以指定匿名账号 ID，默认 anonuid 为 65534，也就是 nfsnobody 账号，使用 no_root_squash 可以防止这种映射而保留 root 权限，all_squash 选项则可以屏蔽所有的账户权限，将所有的用户对 NFS 的访问自动映射为服务器本地的匿名账户。默认情况下，普通账号的权限是保留的，也就是没有进行 squash 操作。

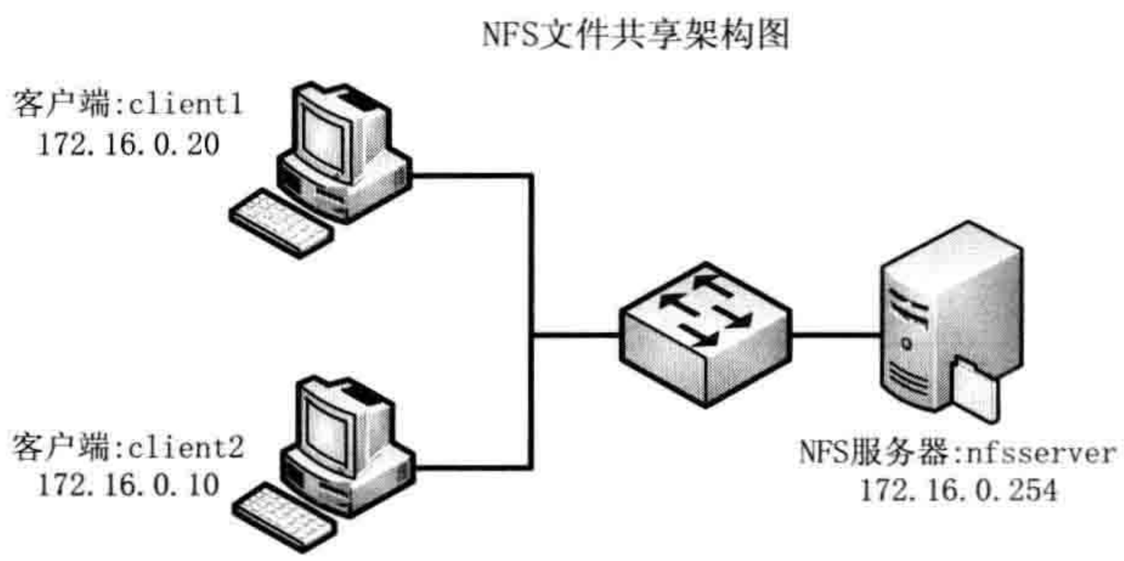


图 4-1

下面通过案例演示 NFS 服务器的搭建过程，服务框架见图 4-1，共享/var/web/与/var/cloud 目录，在该案例中，172.16.0.0/16 网段内的所有主机均可以异步可读可写访问 web 目录，且不屏蔽 root 用户对 web 目录的访问权限，任何主机都可以同步只读访问/var/cloud 目录。操作步骤主要包括：安装软件、添加共享账户、创建共享目录、修改权限、设置 NFS 共享配置文件以及启动共享服务。

```
[root@nfsserver ~]# yum -y install nfs-utils rpcbind
[root@nfsserver ~]# rpm -qa |grep nfs-utils
nfs-utils-lib-devel-1.1.5-4.el6.x86_64
nfs-utils-1.2.3-26.el6.x86_64
nfs-utils-lib-1.1.5-4.el6.x86_64
[root@nfsserver ~]# rpm -qa |grep rpcbind
rpcbind-0.2.0-9.el6.x86_64
```

```
[root@nfsserver ~]# useradd -u 1003 jerry
[root@nfsserver ~]# mkdir /var/{web,cloud}
[root@nfsserver ~]# chmod a+w /var/web
[root@nfsserver ~]# cat /etc/exports
/var/web/  172.16.0.20(rw,async,no_root_squash)
/var/cloud/  *(ro,sync)
[root@nfsserver ~]# /etc/init.d/rpcbind restart
[root@nfsserver ~]# /etc/init.d/nfs restart
[root@nfsserver ~]# chkconfig rpcbind on
[root@nfsserver ~]# chkconfig nfs on
```

4.1.2　客户端访问 NFS 共享

客户端可以通过 showmount 命令查看服务器共享信息，通过 mount 挂载 NFS 共享，mount 挂载属性有很多，这些属性为我们提供了丰富的挂载特性与功能。

client1 主机挂载 nfsserver 共享目录/var/web 至本机/var/web 目录，该共享目录可以读写，且 root 账号不会被映射为匿名账号（root 的权限会被保留）：

```
[root@client1 /]# showmount -e 172.16.0.254
Export list for 192.168.0.254:
/var/cloud *
/home      *
/var/web   172.16.0.20
[root@client1 ~]# mkdir /var/web
[root@client1 ~]# useradd -u 1003 jerry
[root@client1 ~]# mount 172.16.0.254:/var/web /var/web      #手动挂载
[root@client1 ~]# echo\                                     #设置开机自动挂载
>"172.16.0.254:/var/web /var/web  nfs  defaults  0 0" >> /etc/fstab
[root@client1 ~]# chmod a+w /var/web                        #给所有的用户可写权限
[root@client1 web]# cd /var/web ; touch root.txt
[root@centos6 /]# ll /var/web/
total 0
-rw-r--r--. 1 root     root     0 Mar 10 08:20 root.txt
[root@client1 web]# su - jerry
[root@client1 ~]# cd /var/web/
[root@client1 web]# touch jerry.txt
[root@client1 web]# ls -l
total 0
-rw-r--r--.  1 root  root  0 Mar  10 08:20 root.txt
-rw-rw-r--. 1 jerry  jerry  0 Mar  9 23:21 jerry.txt    #jerry 为正常用户权限
```

client2 挂载使用 NFS 服务器的共享目录/var/cloud 至本机/var/cloud 目录，该目录为只读共享，默认 root 权限会自动映射为 nfsnobody 账号，普通账号权限将保留。

```
[root@client2 ~]# mkdir /var/cloud
[root@client2 ~]# useradd -u 1003 jerry
[root@client2 ~]# mount 172.16.0.254:/var/cloud /var/cloud
[root@client2 ~]# echo\                                    #设置开机自动挂载
>"172.16.0.254:/var/cloud /var/cloud  nfs  defaults  0 0" >> /etc/fstab
[root@client2 ~]# cd /cloud/
[root@client2 cloud]# touch root.txt                       #提示该文件系统只读
touch: cannot touch `root.txt': Read-only file system
```

详细剖析权限问题，由于共享数据实际是存储在 NFS 服务器上的，所以所有的操作实际是以服务器本机的账户进行的，只是服务器会根据不同的情况将远程客户端的访问账户转换为不同的服务器本机账户。

- 客户端使用普通用户连接服务器时，默认情况下，如果客户端使用的账户 UID 在服务器上也有相同的账户 UID，则服务器将使用服务器本机上该 UID 账号进行读写操作，如果客户端访问服务器所使用的账户 UID 不在服务器上，则服务器自动将账号转换为 nobody 账号。此外，如果服务器端对共享属性配置了 all_squash 选项，则服务器会根据 anonuid 选项的值，将所有的账户自动转换为匿名账号。
- 客户端使用 root 连接服务器时，默认会将 root 转换为服务器上的 nfsnobody 账号，如果服务器端对共享属性配置了 no_root_squash 选项，则服务器会将远程 root 账户转换为本机 root 账号进行读写操作。

4.1.3 NFS 高级设置

1. NFS 所需服务

为了在 CentOS 6.3 系统平台上提供NFS服务，该操作系统平台提供了很多有用的服务进程，以下进程在实现 NFS 共享的过程中需要相互协同工作。

- nfs：NFS 服务器主程序。
- nfslock：为 NFS 文件系统提供锁机制。
- rpcbind：提供地址与端口注册服务。
- rpc.mountd：该进程被 NFS 服务用来处理 NFSv2 与 NFSv3 的 mount 请求。
- rpc.nfsd：动态处理客户端请求。
- lockd：lockd 是内核线程，在服务器端与客户端运行，用来实现 NLM 网络协议，允许 NFSv2 与 NFSv3 客户端对文件加锁。
- rpc.statd：该进程实现网络状态监控（NSM）协议。

- rpc.rquotad：该进程提供用户配额信息。
- rpc.idmapd：提供 NFSv4 名称映射，/etc/idmapd.conf 必须被配置。

2. NFS 客户端配置

若要想客户端主机可以访问并使用服务器所提供的共享目录，可通过 mount 命令挂载 NFS 共享，格式如下：

```
# mount -t nfs -o 选项 服务主机:/服务器共享目录  /本地挂载目录
```

具体挂载选项如下。

- Intr：当服务器宕机时允许中断 NFS 请求。
- nfsvers=version：指定使用哪个版本的 NFS 协议，version 可以是 2、3 或 4。
- noacl：关闭 ACL，仅与老版本操作系统兼容时使用。
- nolock：关闭文件锁机制，仅用来连接老版本 NFS 服务器。
- noexec：在挂载的文件系统中屏蔽可执行的二进制程序。
- port=num：指定 NFS 服务器端口号，默认 num 为 0，此时如果远程 NFS 进程没有在 rpcbind 注册端口信息，则使用标准 NFS 端口号（TCP 2049 端口）。
- rsize=num：设置最大数据块大小调整 NFS 读取数据的速度，num 单位为字节。
- wsize=num：设置最大数据块大小调整 NFS 写入数据的速度，num 单位为字节。
- tcp：使用 TCP 协议挂载。
- udp：使用 UDP 协议挂载。

3. 使用 NFS 命令工具

NFS 软件还为我们提供了很多便利的命令工具，这些工具可以帮助我们在不重启服务的情况下，应用新的共享设置，查看 NFS 连接状态，查询实时的端口注册信息。

（1）exportfs 命令

描述：当 nfs 服务启动时，/usr/sbin/exportfs 命令会自动启动并读取/etc/exports 文件，通过控制 rpc.mountd（如果使用 NFSv2 或 NFSv3）处理挂载请求，然后 rpc.nfsd 使文件系统对远程主机可见，手动运行 exportfs 命令允许 root 在不重启 NFS 服务的情况下选择共享或取消部分共享目录。

选项：-r　　重新读取/etc/exports 文件。

-a　　全部共享或全部取消共享。

-u　　取消共享，与-a 一起使用可以取消全部共享文件系统。

-v　　显示详细信息。

（2）nfsstat 命令

描述：查看 NFS 共享状态。

选项：-s,--server　不使用该参数时，默认显示服务器端与客户端状态，使用该参数后将仅显示服务器端状态。

-c,--client　仅显示客户端状态。

-n,--nfs　仅显示 NFS 状态，默认显示 NFS 与 RPC 信息。

-n　n 为数字 2、3 或 4，仅显示 NFS 版本为 n 的状态信息。

-m　显示挂载信息。

-l　以列表形式显示信息。

（3）rpcinfo 命令

描述：生成 RPC 信息报表。

选项：-m　显示指定主机 rpcbind 操作信息表。

-p　显示指定主机 RPC 注册信息。

-s　显示指定主机所有注册 RPC 的信息程序，不指定主机时默认显示本机信息。

示例：#rpcinfo

#rpcinfo -p 127.0.0.1

#rpcinfo -m 127.0.0.1

#rpcinfo -s

4. 在防火墙后端运行 NFS

NFS 需要向 rpcbind 动态注册端口信息，这将导致运行在防火墙后端的 NFS 服务器无法进行防火墙配置，因为端口信息是动态随机生成的，防火墙可以开放某些固定的端口允许客户端进行连接，但无法设置随机端口规则。允许客户端访问位于防火墙后面的 NFS 共享需要我们编辑/etc/sysconfig/nfs 配置文件，配置固定的端口号。注意，端口号不可以指定为已经被其他程序使用的，固定的端口配置选项如下。

- MOUNTD_PORT=端口号：设置 mountd 程序端口号。
- LOCKD_TCPPORT=端口号：设置 tcp 的 lockd 程序端口号。
- LOCKD_UDPPORT=端口号：设置 udp 的 lockd 程序端口号。
- STATD_PORT=端口号：设置 rpc.statd 程序端口号。

配置完固定的端口号之后，还需要对防火墙进行设置，需要为防火墙编写策略允许所有端口的数据通信。此外，还需要为 NFS 开启 TCP 与 UDP 的 2049 端口，以及 TCP 与 UDP 的 111 端口，111 端口被 rpcbind 所使用。

4.1.4　常见问题分析

1. 权限问题

很多时候，当你在/etc/exports 配置文件中设置共享目录为可读写时，却忘记了修改相应系统层面的文件及目录权限，从而导致客户端实际挂载使用时无写权限，系统提示信息一般为“Permission denied”，我们在对配置文件设置写权限后一定要记住修改相关目录、文件的权限。

另外，默认客户端使用 root 访问 NFS 共享进行读写操作时，服务器会自动把 root 转换为服务器本机的 nfsnobody 账号，这会导致 root 无法进行相应的操作，如果要保留 root 权限，则需要在配置文件中添加 no_root_squash 选项。

2. rpcbind 问题

在没有启动 rpcbind 的情况下，启动 nfs 服务时系统会报错：NFS mountd、rpc.rquotad、rpc.nfsd 无法启动，因为这些服务都依赖于 rpcbind 服务，这样就需要先确保 rpcbind 启动后再开启 nfs 以及相关服务进程。通过 rpcinfo -p 可以查看基于 RPC 协议的服务是否成功与 rpcbind 通信，并注册信息。

3. 兼容性问题

在工作环境中，当客户端需要使用NFSv3版本挂载以满足兼容性要求时，则需要使用nfsvers选项设置特定的版本信息，并且在/etc/fstab 开机自动挂载文件中也需要进行相应的修改。

```
[root@client1 ~]# mount -o nfsvers=3 172.16.0.254:/var/web /var/web
```

/etc/fstab 文件书写格式为：

```
172.16.0.254:/var/web  /var/web  nfs   defaults,nfsvers=3  0  0
```

4. 挂载错误

系统提示 No such file or directory，说明服务器上没有相应的挂载点目录，应检查确定目录名称是否正确。

5. 防火墙错误

系统提示 mount: mount to NFS server '172.16.0.254em Error: No route to host，这说明 nfs 服务的默认端口 2049 被防火墙屏蔽，需要修改 iptables 规则开放 2049 端口。

4.2 Samba 文件共享

Samba 是 Linux、UNIX 与 Windows 之间进行交互操作的软件组件，Samba 是基于 GPL 协议的自由开源软件。自 1992 年以来，Samba 开始通过 SMB/CIFS 协议为 Windows、OS/2、Linux，以及众多其他支持该协议的操作系统提供安全、稳定、快速的文件与打印服务，在使用 winbind 将 Linux/UNIX 无缝整合到活动目录环境中时，Samba 也是一个重要组件。

所需软件包括：Samba（服务器端软件包）、Samba-client（客户端软件包）和 Samba- common（Samba 公共文件软件包）。其中，Samba 由 smbd 和 nmbd 两个守护进程组成，两个进程的启动脚本是独立的。下面分别介绍每个守护进程的作用。

smbd 服务进程为客户端提供文件共享与打印机服务。另外，该进程还负责用户权限验证以及锁功能，smbd 默认监听的端口是 TCP 协议的 139 与 445，Samba 通过 smb 服务启动 smbd 进程，使用 netstat -nutlp 查看进程端口信息。

nmbd 进程提供 NetBIOS 名称服务，以满足基于 Common Internet File System（CIFS）协议的共享访问环境。Samba 通过 nmb 服务启动 nmbd 进程，该进程默认使用的是 UDP 137 端口。

由于 iptables 与 SELinux 默认策略会阻止远程用户对 Samba 的访问，目前我们暂时先关闭这些服务，以确保共享服务的正常使用。关于如何合理设置防火墙与 SELinux 策略，将在第 6 章中详细介绍。

4.2.1 快速配置 Samba 服务器

下面的第 1 至第 3 步的操作是准备工作，主要是关闭防火墙及 SELinux 安全组件、安装相应软件包，以及创建共享目录和用于测试的文件。

1. 关闭防火墙与 SELinux

```
[root@centos6 ~]# setenforce 0                                  #临时关闭 SELinux
[root@centos6 ~]# sed -i "/SELINUX=/c SELINUX=disable" \
> /etc/sysconfig/selinux                                        #永久关闭 SELinux
```

```
[root@centos6 ~]# service iptables stop                    #临时关闭防火墙
[root@centos6 ~]# chkconfig iptables off                   #永久关闭防火墙
```

2. 安装 Samba 软件

```
[root@centos6 ~]# yum -y install samba
```

3. 创建共享目录

```
[root@centos6 ~]# mkdir /common
[root@centos6 ~]# chmod 777 /common
[root@centos6 ~]# echo "hello the world" > /common/smb.txt   #创建测试文件
```

4. 修改配置文件

在做完以上准备工作后，如果希望 Samba 服务可以共享指定的目录给客户端，就需要在其配置文件中写入目录共享的代码段，Samba 默认配置文件是/etc/samba/smb.conf。默认情况下，Samba 已经配置为允许用户通过远程共享访问账户自己的家目录，为了将我们在准备阶段创建的/common 目录共享给客户端，需要在该配置文件末尾追加对 common 目录的共享设置，实现快速自定义共享。

```
[root@centos6 ~]# vim /etc/samba/smb.conf
[common]                                    #共享名称为 common
comment = Common share                      #共享注释
path = /common                              #重要，指定共享路径
browseable = yes                            #所有人可见
guest ok = no                               #拒绝匿名访问
writable = yes                              #支持写入数据
```

5. 创建访问账号

客户端访问 Samba 共享时所使用的账户名称就是服务器端操作系统中真实存在的系统账户名称，但不同于微软的共享设置，访问 Samba 共享的密码必须是独立的 Samba 密码，而不可以使用系统密码，这样即使有人获得了 Samba 账户和密码，也不能使用这些信息登录服务器本机的操作系统。所以，成功访问 Samba 服务器还需要使用 smbpasswd 将系统账户添加到 Samba，并设置相应的密码，CentOS 6.3 系统自带的 Samba 软件包安装后，默认会将账户与密码文件存放在/var/lib/samba/private 目录下。

```
[root@centos6 ~]# useradd -s /sbin/nologin smbuser
[root@centos6 ~]# smbpasswd -a smbuser
New SMB password:                                  #提示输入账户密码
```

```
Retype new SMB password:                    #确认一次密码
Added user smbuser.                         #成功添加账号与密码
```

smbpasswd 命令的描述和用法如下。

描述：修改账户 samba 密码。

用法：smbpasswd [选项] 账户名称

选项：-a　　添加账户并设置密码。

-x　　删除 SMB 账户。

-d　　禁用 SMB 账户。

-e　　启用 SMB 账户。

6. 启动服务

```
[root@centos6 ~]# service smb start
[root@centos6 ~]# chkconfig smb on
```

4.2.2　访问 Samba 共享

1. Windows 客户端访问

在 Windows 环境中，通过“开始”→“运行”菜单或直接按 Win+R 快捷键开启运行对话框，在该对话框中输入\\IP（其中的 IP 为 Samba 服务器的 IP 地址），单击“确定”按钮访问 Samba 共享，如果是基于用户名和密码的访问，系统会提示输入账户与密码，如图 4-2 所示。登录成功后可以看到 Samba 共享了两个目录，一个是 common 目录，另一个是 smbuser 账户的家目录。

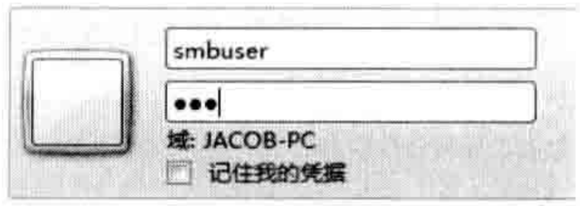


图 4-2

2. Linux 客户端访问

Linux 客户端使用 smbclient 命令可以访问服务器上的共享资源，如果安装有图形环境，也可以通过 Linux 图形界面连接服务器，下面是具体的方法。

```
[root@centos6 ~]# smbclient -L //192.168.0.101 #查看192.168.0.101 主机共享信息
Enter root's password:                          #仅查看不需要密码，直接按回车键
Anonymous login successful
Domain=[MYGROUP] OS=[Unix] Server=[Samba 3.5.10-125.el6]   #服务器基本信息

Sharename       Type      Comment
```

```
---------          ----      -------
common          Disk      Common share          #共享与注释
IPC$             IPC       IPC Service (Samba Server Version 3.5.10-125.el6)
[root@centos6 ~]# smbclient -U smbuser //192.168.0.101/common
                                                 #访问 samba 共享目录
Enter smbuser's password:                        #输入账户密码
Domain=[MYGROUP] OS=[Unix] Server=[Samba 3.5.10-125.el6]
smb: \> ls                                       #通过 ls 查看共享目录
  .                                   D        0  Mon Mar 11 00:27:57 2013
  ..                                 DR        0  Mon Mar 11 20:29:20 2013
smb.txt                                         16  Mon Mar 11 23:09:29 2013
    41355 blocks of size 524288. 27640 blocks available
smb: \>help                                      #查看 smblient 可用指令
smb: \>quit                                      #退出
```

有时候，挂载 Samba 共享目录实现如本地文件系统一样的使用体验是很有意义的，使用 root 管理员身份运行 mount 命令可以挂载这样的文件系统，将挂载信息写入 fstab 文件可以实现永久挂载的目的。

```
[root@centos6 ~]# mkdir /com
[root@centos6 ~]# mount -t cifs //192.168.0.101/common /com \
> -o username=smbuser,password=<password>
[root@centos6 ~]# echo "//192.168.0.101/common /com /smb  \ #实现永久挂载
> cifs  defaults,username=smbuser,password=<password>  0 0  /etc/fstab
```

若希望使用图形访问 Samba 共享时，首先通过 nautilus-connect-server 命令开启图形配置界面，如图 4-3 所示，输入 Samba 服务器 IP 地址、共享目录、账户名称等信息，输入正确的密码并成功登录后，将出现如图 4-4 所示的画面，由图 4-4 可以看到，服务器共享的测试文件 smb.txt。另外，通过图形访问 Samba 后会在桌面生成挂载图标，以后可以随时通过双击该图标打开共享目录。

```
[root@centos6 ~]# nautilus-connect-server        #图形访问共享
```

图 4-3

图 4-4

4.2.3 配置文件详解

Samba 配置文件非常简洁明了，所有的设置都在/etc/samba/smb.conf 配置文件中进行，通过对该配置文件的修改，可以将 Samba 配置为一台匿名文件服务器、基于账户的文件服务器或打印服务器，默认情况下，Samba 会已开启本地账号家目录共享与打印机共享，配置文件中以#或;符号开头的行为注释行，配置文件分为若干段，除了 global（全局配置段）之外的其余所有段用来描述共享资源，全局段中的配置代表全局有效，是全局的默认设置。但如果全局配置段中的设置项与共享段中的设置项有冲突，则共享设置段中的设置为实际有效值。下面具体说明配置文件中各个配置选项的含义。

```
[global]                                    #定义全局策略
workgroup = MYGROUP                         #定义工作组
server string = Samba Server Version %v
#服务器提示字串，默认显示 samba 版本，建议修改默认值以防止针对版本的网络攻击
;interfaces = lo eth0 192.168.12.2/24 192.168.13.2/24
#如果你的服务器有多个网络接口，可以通过 interfaces 选项指定 Samba 监听哪些网络接口
;hosts allow = 127. 192.168.12. 192.168.13. EXCEPT 192.168.13.13
#hosts allow 指定仅允许哪些主机有权访问 Samba 服务器资源，该参数可以放置在全局
#段也可以放置在共享段。与此相反的选项是 hosts deny，用来设置黑名单列表，这里可
#以指定允许访问的主机名、IP 地址或网段，当指定网段时可以使用 192.168.12.或
#192.168.12.0/255.255.255.0 两种格式，使用 EXCEPT 可以指定例外的 IP 地址
log file = /var/log/samba/log.%m
#定义日志文件，因为使用了 Samba 变量%m，所以每个访问共享的主机会产生独立的
#日志文件，%m 会被替换为客户端的主机名
max log size = 50
#定义日志单个文件最大容量为 50KB
security = user
#设置 security 选项将影响客户端访问 samba 的方式，是非常重要的设置选项之一。security
#可以被设置为 uesr、share、server 或 domain，user 代表通过用户名、密码验证访问者的身份，
#账户需要是服务器本机系统账户；share 代表匿名访问；server 代表基于验证身份的访问，
#但账户信息保存在另一台 SMB 服务器上；domain 同样是基于验证的访问，账户信息保存在活动目
#录中
passdb backend = tdbsam
#账户与密码的存储方式，smbpasswd 代表使用老的明文格式存储账户及密码；tdbsam 代
#表基于 TDB 的密文格式存储；ldapsam 代表使用 LDAP 存储账户资料
deadtime = 10
#客户端在 10 分钟内没有打开任何 Samba 资源，服务器将自动关闭会话，在大量的并发访问环境中，
#这样的设置可以提高服务器性能
display charset = UTF8                      #设置显示使用的字符集为 UTF8
max connections = 0
```

```
#设置最大连接数，0 代表无限制，若设置该规则超过此限制的连接请求服务器将拒绝连接
guest account = nobody                    #设置匿名账号为 nobody
load printers = yes                       #是否共享打印机
cups options = raw                        #打印属性
   [homes]                                #共享名称
comment = Home Directories                #注释，共享的描述信息
browseable = no                           #共享目录是否可以被浏览
writable = yes                            #共享目录是否可以进行写操作
 [printers]                               #打印机共享
comment = All Printers
path = /var/spool/samba                   #打印机共享池
browseable = no
guest ok = no
writable = no
printable = yes
 [common]                                 #共享名称为 common
comment = Common share                    #注释，共享描述信息
path = /common                            #重要，指定共享路径
   valid users =tom jerry                 #有效账户列表
create mask = 0750                        #客户端上传文件的默认权限，默认为 0744
directory mask = 0775                     #客户端创建目录的默认权限，默认为 0755
browseable = yes                          #共享目录是否对所有的人可见（yes 或 no）
writable = no
write list = tom                          #写权限账户列表，这里设置 tom 可写
admin users =tom                          #该共享的管理员，具有完全权限
invalid users = root bin                  #禁止 root 与 bin 访问 common 共享
guest ok = no  #是否允许匿名访问，仅当全局设置 security=share 时有效（yes 或 no）
```

4.2.4　Samba 应用案例

ABC 是一家网站设计公司，公司下有商务部、页面设计部、开发部、运维部四个部门，商务部门负责与客户沟通并调研客户需求，为客户制定网站建设方案。页面设计部门根据商务部与客户的沟通记录与调研报告，确定主页设计风格与方案。网站设计具体方案经客户审核确认后提交开发部完成网站代码的编写，向客户提交完成后的网站，待客户确认后上传公司服务器由运维部门负责网站的运行维护工作。根据这些信息可以看出该公司部门之间的衔接是非常密切的，部门之间有大量的数据需要共享，为了加强部门之间互联互通，优化工作流程与效率，ABC 公司决定部署一台 Samba 服务器，满足部门间可快速共享数据的需求，从而实现流水线办公流程。

1. 创建目录结构

共享数据首先需要我们创建用于共享的目录。根据 ABC 公司的情况，可以为该例创建五个一级目录和四个部门共享目录，共享目录仅对部门内部员工共享数据，每个部门员工可以在相应的部门共享目录下创建自己的个人目录与文件；另一个公共共享目录，用于所有的部门之间相互访问彼此的数据使用，在公共共享目录下为每个部门创建每个部门的共享子目录。/ABC/sales 目录为商务部共享目录，/ABC/design 目录为设计部共享目录，/ABC/develop 目录为开发部共享目录，/ABC/ops 目录为运维部共享目录。另外，/ABC/share 目录为整个公司的公共共享目录，用于部门间的数据共享，我们会在该目录下为每个部门创建对应的子目录。

```
[root@centos6 ~]# mkdir -p /ABC/{sales,design,develop,ops,share/{sales,
design,develop,ops}}
[root@centos6 ~]# tree /ABC/
/ABC/
├── design
├── develop
├── ops
├── sales
└── share
    ├── design
    ├── develop
    ├── ops
    └── sales
```

2. 添加账户

为了提升安全性，公司要求所有的员工访问共享时都必须使用账户名和密码。初始状态下，我们为每个部门创建两个账户，并创建所有共享目录的管理员，以便后期管理员可以根据公司发展情况添加或删除账户信息。下面通过脚本创建部门初始账户，语法规则可以参考本书第 3 章的内容。

```
[root@centos6 ~]# cat user.sh
#!/bin/bash
#Add users for share
DEPART=(sales design develop ops)
for g in ${DEPART[@]}
  do
       groupadd $g
       for u in $(seq 1 2)
          do
              useradd -M -s /sbin/nologin -g ${g} ${g}$u
          done
```

```
  done
[root@centos6 ~]# chmod a+x user.sh
[root@centos6 ~]# ./user.sh
[root@centos6 ~]# smbpasswd -a sales1
[root@centos6 ~]# smbpasswd -a sales2
```

其他账户按照此模板逐个添加即可。

3. 配置文件

创建完共享目录及系统账户后，我们还需要修改 Samba 主配置文件，在配置文件中加入我们需要共享的目录信息以及相应权限的设置。下面是/etc/samba/smb.conf 文件修改后的具体配置清单。

```
#====================== Global Settings ==================================
[global]
workgroup = STAFF
server string = ABC.corp share
log file = /var/log/samba/log.%m
max log size = 50
security = user
passdb backend = tdbsam
load printers = yes
cups options = raw
#============================ Share Definitions ============================
[homes]
comment = Home Directories
browseable = no
writable = yes
[sales]
comment = sales share
path = /ABC/sales
browseable = yes
guest ok = no
writable = no
write list = @sales
[design]
comment = design share
path = /ABC/design
browseable = yes
guest ok = no
writable = no
write list = @design
[develop]
comment = develop share
```

```
    path = /ABC/develop
    browseable = yes
    guest ok = no
    writable = no
    write list = @develop
    [ops]
    comment = ops share
    path = /ABC/ops
    browseable = yes
    guest ok = no
    writable = no
    write list = @ops
    [share]
    comment = common share
    path = /ABC/share
    browseable = yes
    guest ok = no
    writable = yes
```

4. 修改权限

为了使员工访问服务器共享资料后，可以在属于自己的对应目录下创建文件与目录，除了要在 Samba 主配置文件中定义权限设置外，还需要为系统目录修改正确的权限。

```
[root@centos6 ~]# chmod 1770 /ABC/{design,develop,ops,sales} #添加了 sticky 权限
[root@centos6 ~]# chmod 1777 /ABC/share
[root@centos6 ~]# chown :design /ABC/design
[root@centos6 ~]# chown :develop /ABC/develop
[root@centos6 ~]# chown :ops /ABC/ops
[root@centos6 ~]# chown :sales /ABC/sales
```

5. 重启 Samba 服务

```
[root@centos6 ~]# service smb restart
```

4.2.5 常见问题分析

1. NT_STATUS_BAD_NETWORK_NAME

若提示该错误信息，则说明输入了错误的共享名称，一般为输入性错误，需要检查客户端请求的共享资源在服务器中是否存在。

2. NT_STATUS_LOGON_FAILURE

若提示该错误信息，则说明登录失败，一般是由于账户名称或密码不对，则需要检查账户与密码后重试。

3. NT_STATUS_ACCESS_DENIED

若提示该错误信息，则说明访问被拒绝，权限不足。这里可能是 Samba 服务设置的访问权限，也有可能是服务器文件系统的访问权限不允许客户端访问。

4. Error NT_STATUS_HOST_UNREACHABLE

若提示该错误信息，则说明客户端无法连接 Samba 服务器，一般是由于网络故障或防火墙问题引起的，需要检查客户端与服务器的网络连接是否正常。此外，还要检查防火墙规则是否允许客户端请求，Samba 端口有 137、138、139、445。

5. Not enough '\' characters in service

若提示该错误信息，则说明客户端访问时共享路径输入有误，特别是//IP 与//IP/是不同的，使用//IP 格式访问服务器时会报错。

4.3　vsftpd 文件共享

vsftpd 是 Very Secure FTP 的简写形式，是一款非常安全的 FTP 软件。该软件是基于 GPL 开发的，被设计为 Linux 平台下稳定、快速、安全的 FTP 软件，它支持 IPv6 以及 SSL 加密。vsftpd 的安全性主要体现在三个方面：进程分离，处理不同任务的进程彼此是独立运行的；进程运行时均以最小权限运行；多数进程都使用 chroot 进行了禁锢，防止客户访问非法共享目录，这里的 chroot 是改变根的一种技术，如果我们通过 vsftpd 共享了/var/ftp/目录，则该目录对客户端而言就是共享的根目录。

Vsftpd 虽然是一款 FTP 软件，但 FTP 不同于其他的互联网协议，它使用多端口通信。当客户端连接服务器请求资源时，服务器会使用 21 端口与客户端进行通信，该端口专门处理客户端发送给服务器的请求命令，也就是俗称的命令端口。而最终服务器与客户端进行数据传输时，还需要使用另一个数据端口，数据端口号取决于服务器运行的模式是主动模式还是被动模式。

4.3.1 FTP 工作模式

1. 主动 FTP 工作模式

主动 FTP 模式（active mode）的工作步骤如下。

第一步，客户端随机开启大于 1024 的 X 端口与服务器的 21 端口建立连接通道，通道建立后，客户端随时可以通过该通道发送上传或下载的命令。

第二步，当客户端需要与服务器进行数据传输时，客户端会再开启一个大于 1024 的随机端口 Y，并将 Y 端口号通过之前的命令通道传送给服务器的 21 端口。

第三步，服务器获取到客户端的第二个端口后会主动连接客户端的该端口，通过三次握手后，完成服务器与客户端数据通道的建立，所有的数据均通过该数据通道进行传输。

2. 被动 FTP 工作模式

被动 FTP 模式（passive mode）的工作过程如下。

第一步，客户端随意开启大于 1024 的 X 端口与服务器的 21 端口建立连接通道。

第二步，当客户端需要与服务器进行数据传输时，客户端从命令通道发送数据请求要求上传或下载数据。

第三步，服务器收到数据请求后会随机开启一个端口 Y，并通过命令通道将该端口信息传送给客户端。

第四步，客户端在收到服务器发送过来的数据端口 Y 的信息后，将在客户端本地开启一个随机端口 Z，此时客户端再主动通过本机的 Z 端口与服务器的 Y 端口进行连接，通过三次握手连接完成后，即可进行数据传输。

综上所述，不像其他大多数互联网协议，FTP 协议需要使用多个网络端口才可以正常工作，其中一个端口专门用于命令的传输（命令端口），另一个端口专门用于数据的传输（数据端口）。主动模式在传输数据时，服务器会主动连接客户端；被动模式在传输数据时，由客户端主动连接服务器。FTP 协议最初使用主动模式工作，但现在客户端主机多数都位于防火墙后面，而且防火墙策略一般不允许入站数据。也就是说，客户端主机可以连接外网，但外网不可以直接接入客户端主机。这样采用主动模式的 FTP 服务器最终将无法正常工作，所以就有了后来的被动模式的 FTP 服务。

4.3.2　安装与管理 vsftpd

要想使用 vsftpd 实现 FTP 数据共享，首先需要安装 vsftpd 软件，该软件在 Cent OS 安装光盘中已经包含，所以也可以在安装部署操作系统时选择安装，如果在部署操作系统时没有安装，也可以随时通过 YUM 方式安装该软件。

```
[root@centos6 ~]# yum -y install vsftpd
[root@centos6 ~]# service vsftpd start
[root@centos6 ~]# chkconfig vsftpd on
```

RPM 格式的 vsftpd 软件安装完成后，该软件的主程序是/usr/sbin/vsftpd，下面是 vsftpd 相关的核心文件与目录列表的说明。

```
/etc/logrotate.d/vsftpd（日志轮转备份配置文件）
/etc/pam.d/vsftpd（基于 PAM 的 vsftpd 验证配置文件）
/etc/rc.d/init.d/vsftpd（vsftpd 启动脚本，可以使用 service 调用）
/etc/vsftpd（vsftpd 软件主目录）
/etc/vsftpd/ftpusers（默认的 vsftpd 黑名单）
/etc/vsftpd/user_list（可以通过主配置文件设置该文件为黑名单或白名单）
/etc/vsftpd/vsftpd.conf（vsftpd 主配置文件）
/usr/sbin/vsftpd（vsftpd 主程序）
/usr/share/doc/vsftpd-2.2.2（vsftpd 文档资料路径）
/var/ftp（默认 vsftpd 共享目录）
```

4.3.3　配置文件解析

vsftpd 配置文件默认位于/etc/vsftpd 目录下，vsftpd 会自动寻找以.conf 结尾的配置文件，并使用此配置文件启动 FTP 服务。配置文件的格式为：选项=值（中间不可以有任何空格符），以#开头的行会被识别为注释行。表 4-2 给出了 vsftpd 的主要配置选项及其对应的含义。

表 4-2

账户类别	设置项	功能描述
全局设置	listen=YES	是否监听端口，独立运行守护进程
	listen_port=21	监听入站 FTP 请求的端口号
	write_enable=YES	是否允许写操作命令，全局开关
	download_enable=YES	如果设置为 NO，则拒绝所有的下载请求
	dirmessage_enable=YES	用户进入目录是否显示消息
	xferlog_enable=YES	是否开启 xferlog 日志功能

续表

账户类别	设置项	功能描述
	xferlog_std_format=YES	Xferlog 日志文件格式
	connect_from_port_20=YES	使用主动模式连接，启用 20 端口
	pasv_enable=YES	是否启用被动模式连接，默认为被动模式
	pasv_max_port=24600	被动模式连接的最大端口号
	pasv_min_port=24500	被动模式连接的最小端口号
	userlist_enable=YES	是否启用 userlist 用户列表文件
	userlist_deny=YES	是否禁止 userlist 文件中的账户访问 FTP
	max_clients=2000	最大允许同时 2000 客户端连接，0 代表无限制
	max_per_ip=0	每个客户端的最大连接限制，0 代表无限制
	tcp_wrappers=YES	是否启用 tcp_wrappers
	guest_enable=YES	如果为 YES，则所有的非匿名登录都映射为 guest_username 指定的账户
	guest_username=ftp	设定来宾账户
	user_config_dir=/etc/vsftpd/conf	指定目录，在该目录下可以为用户设置独立的配置文件与选项
	dual_log_enable=NO	是否启用双日志功能，生成两个日志文件
	anonymous_enable=YES	是否开启匿名访问功能，默认为开启
匿名账户	anon_root=/var/ftp	匿名访问 FTP 的根路径，默认为/var/ftp
	anon_upload_enable=YES	是否允许匿名账户上传，默认禁止
	anon_mkdir_write_enable=YES	是否允许匿名账户创建目录，默认禁止
	anon_other_write_enable=YES	是否允许匿名账户进行其他所有的写操作
	anon_max_rate=0	匿名数据传输率（B/s）
	anon_umask=077	匿名上传权限掩码
本地账户	local_enable=YES	是否启用本机账户 FTP 功能
	local_max_rate=0	本地账户数据传输率（B/s）
	local_umask=077	本地账户权限掩码
	chroot_local_user=YES	是否禁锢本地账户根目录，默认为 NO
	local_root=/ftp/common	本地账户访问 FTP 根路径

4.3.4 账户权限

vsftpd 支持的常用登录方式有：匿名登录、本地账户登录、虚拟账户登录三种方式。

匿名账户登录一般应用于下载服务器。相信很多读者都使用 FTP 下载过资料，这种下载服

务器往往是对外开放的，无须输入用户名与密码即可使用，vsftpd 默认开启的是匿名共享，默认共享路径为/var/ftp。

本地账户登录则需要使用系统账户以及对应的系统密码才可以登录使用 FTP，安装完成系统自带的 RPM 包 vsftpd 软件后，默认的配置文件中，anonymous_enable 与 local_enablede 均被设置为 YES，此时 FTP 为匿名访问模式，如果需要开启本地账户 FTP 功能，需要将 anonymous_enable 设置为 NO，默认共享路径为账户个人家目录。需要注意的是，开启本地账户登录后，用户可以离开家目录，从而进入系统中的其他目录，这样是非常危险的，如果在配置文件中使用 chroot_local_user，用户将被禁锢在自己的家目录下，防止用户进入系统中的其他目录。由于 SELinux 默认不允许 FTP 共享家目录，因此，在没有完全掌握 SELinux 设置方法的情况下，建议关闭 SELinux 以完成下面的案例。

当有大量的用户需要使用 FTP 时，vsftpd 支持虚拟账户模式登录 FTP，从而避免了创建大量的系统账户，通过 guest_enable 可以开启 vsftpd 的虚拟账户功能，guest_username 用来指定本地账户的虚拟映射名称。

vsftpd 有两个文件(黑名单文件和白名单文件)可以对用户进行 ACL 控制，/etc/vsftpd/ftpusers 默认是一个黑名单文件，存储在该文件中的所有用户都将无法访问 FTP，格式为每行一个账户名称。/etc/vsftpd/user_list 文件会根据主配置文件中配置项设定的不同，而成为黑名单文件或白名单文件。此外，也可以禁用该文件。主配置文件中的 userlist_enable 决定了是否启用 user_list 文件，如果启用，还需要根据 userlist_deny 来决定该文件是黑名单还是白名单文件，如果 userlist_deny=YES，则该文件为黑名单，如果 userlist_deny=NO，则该文件为白名单。需要注意的是，黑名单表示仅拒绝名单中的账户访问 FTP，也就是其他所有的账户默认允许访问 FTP。而白名单表示仅允许白名单中的账户访问 FTP，没有在白名单中的其他所有账户则默认将拒绝访问 FTP。

4.3.5　vsftpd 应用案例

案例 1：本地账户 FTP

为了满足企业员工移动办公的需要，ABC 公司决定搭建一台 FTP 服务器，为每个员工创建账户，由于 FTP 是一种互联网协议，因此无论员工在什么地方，只要有计算机能够接入网络，就可以访问公司的 FTP 服务器，然后进行下载与上传数据。如图 4-5 所示，员工不管是在公司网络访问 FTP，还是在家或出差通过互联网访问 FTP，最终都可以连接服务器进行数据操作。

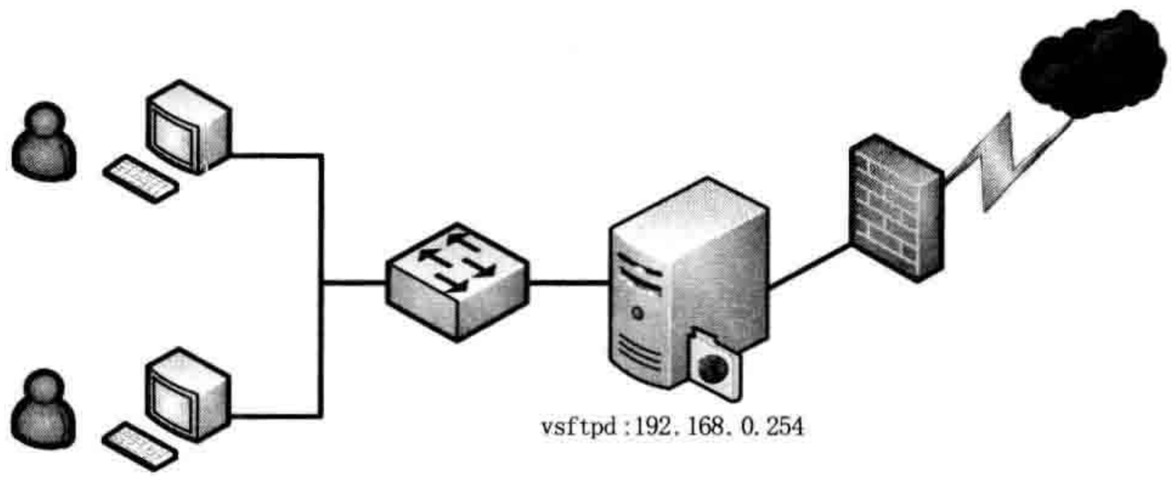


图 4-5

（1）修改配置文件

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022
dirmessage_enable=YES
xferlog_enable=YES
xferlog_std_format=YES
chroot_local_user=YES
listen=YES
pam_service_name=vsftpd
userlist_enable=YES
tcp_wrappers=YES
```

（2）创建系统账户与测试文件

```
[root@centos6 ~]# useradd -s /sbin/nologin tom
[root@centos6 ~]# useradd -s /sbin/nologin jerry
[root@centos6 ~]# useradd -s /sbin/nologin smith
```

如果用户还需要创建其他的公司账户，按照此模板创建即可。下面再通过 touch 命令创建几个测试文件。

```
[root@centos6 ~]# touch /home/{tom,jerry,smith}/test.txt
```

（3）服务管理

完成以上配置后，启动 vsftpd 服务进程即可实现基本的 FTP 共享服务，在不熟悉 Linux 安装组件 SELinux 以及防火墙的情况下，我们可以选择暂时关闭这两个安全组件。

```
[root@centos6 ~]# setenforce 0                    #暂停 SELinux
[root@centos6 ~]# service iptables stop           #暂停防火墙
[root@centos6 ~]# service vsftpd start
[root@centos6 ~]# chkconfig vsftpd on
```

（4）客户端访问

客户端访问 FTP 非常简单，使用任何一款浏览器即可访问 FTP 服务器，如：在火狐浏览器中输入 ftp://192.168.0.254，就可以打开 192.168.0.254 服务器的 FTP 共享，然而多数浏览器默认仅提供浏览与下载的功能而无法实现上传数据。这时我们还可以使用更加专业的客户端工具，包括开源的与商业的客户端软件。这里推荐一款开源的 FTP 客户端软件 FileZilla，读者可以去官网下载使用。

案例 2：虚拟账户 FTP，满足大量账户的访问需求

如果需要访问 FTP 的用户不多，则可以直接创建系统账户以满足对 FTP 访问的需求，但当用户量变得越来越庞大时，继续创建更多的系统用户是不明智的，这时就需要为 vsftpd 创建虚拟账户。但 vsftpd 虚拟账户的数据需要保存在 Berkeley DB 格式的数据文件中，所以需要安装 db4-utils 工具来创建这样的数据文件，具体步骤如下。

（1）创建虚拟用户数据库

首先需要创建明文密码文件，明文文件奇数行为用户名，偶数行为密码。然后使用 db_load 工具将其转换为数据库文件，db_load 工具需要通过安装 db4-utils 软件获得，最后可以通过修改文件权限以增强数据资料的安全性。

```
[root@centos6 ~]# yum -y install db4-utils
[root@centos6 ~]# vim /etc/vsftpd/vlogin
tomcat
123456
jerry
654321
[root@centos6 ~]# db_load -T -t hash -f /etc/vsftpd/vlogin /etc/vsftpd/vlogin.db
[root@centos6 ~]# chmod 600 /etc/vsftpd/{vlogin,vlogin.db}
```

（2）创建 PAM 文件，设置基于虚拟账户验证

Linux 一般通过 PAM 文件设置账户的验证机制，然后通过创建新的 PAM 文件，使用新的数据文件进行登录验证，PAM 文件中的 db 选项于指定并验证账户和密码的数据库文件，数据库文件无须.db 的名称后缀。注意，因为本环境使用的是 CentOS 6.3 64 位操作系统，所以验证模块调用的是 lib64 目录下的文件。

```
[root@centos6 ~]# vim /etc/pam.d/vsftpd.pam
auth required /lib64/security/pam_userdb.so db=/etc/vsftpd/vlogin
account required /lib64/security/pam_userdb.so db=/etc/vsftpd/vlogin
```

（3）设置虚拟账户共享目录

因为所有的虚拟账户最终都需要映射到一个真实的系统账户，所以这里需要添加一个系统账户并设置家目录，为了进行测试，这里复制一份测试文件。

```
[root@centos6 ~]# useradd -s /sbin/nologin -d /home/ftp virtual
[root@centos6 ~]# cp /etc/redhat-release /home/ftp
```

（4）修改主配置文件

与配置本地账户 FTP 一样，我们需要修改 vsftpd 的主配置文件，使用 guest_enable 选项开启虚拟账户功能，所有的虚拟账户都将被映射为 guest_username 指定的一个系统真实账户。如果需要对虚拟账户进行权限设置，使用与匿名账户一样的设置项即可，如 anon_mkdir_write_enable=NO 就可以控制虚拟账户无法创建目录。下面是 vsftpd.conf 主配置文件的具体修改内容。

```
anonymous_enable=NO
local_enable=YES
write_enable=YES
anon_upload_enable=YES
anon_mkdir_write_enable=NO
anon_other_write_enable=NO
chroot_local_user=YES
guest_enable=YES
guest_username=virtual
listen=YES
listen_port=21
pasv_enable=YES
pasv_min_port=30000
pasv_max_port=30999
pam_service_name=vsftpd.pam
user_config_dir=/etc/vsftpd_user_conf
user_sub_token=$USER
```

（5）为每个用户设置独立的共享路径

通过在主配置文件中使用 user_config_dir 选项，设置一个基于账户的配置目录，在该目录下可以创建若干个与账户名称同名的文件，并在文件中为此账户设置独立的配置选项，包括权限与共享路径等设置，这样就可以为每个账户做单独的权限设置等操作。本例仅以 tomcat 为例，其他账户参考模板文件修改即可。当然，如果你还需要对权限、限速、并发量等选项进行设置，可以参考匿名账号的设置选项添加至账户独立的配置文件中。

```
[root@centos6 ~]# mkdir /etc/vsftpd_user_conf
[root@centos6 ~]# mkdir -p /home/ftp/tomcat
[root@centos6 ~]# vim /etc/vsftpd_user_conf/tomcat
local_root=/home/ftp/$USER
```

（6）重启服务

```
[root@centos6 ~]# service vsftpd restart
```

4.3.6　常见问题分析

排错时建议使用专业的 FTP 客户端软件，这样可以获得更多的报错信息，因为一般使用浏览器访问 FTP 服务器时，浏览器会自动屏蔽很多错误信息。

1. 提示错误代码：530 Login incorrect

如果登录提示 530 错误，说明登录过程中账户验证失败。则可能是因为你使用的是 64 位操作系统，而 pam 文件中库文件的调用却使用的是/lib/security/pam_userdb.so。或者是用户名或密码输入有错误。此外，也可能是 vsftpd 主配置文件中 pam_service_name 设置的 pam 文件名称与/etc/pam.d 中创建的 pam 文件名称不一致，导致无法验证成功。

2. 提示错误代码：500 OOPS: cannot change directory:/home/ftp/$USER

该提示代表目录不存在或无权限导致的无法切换至目录，也有可能是由于 SELinux 导致的无法共享账户家目录，默认 SELinux 不允许共享家目录。

3. 使用 Windows 系统访问主动模式的 vsftpd 服务器时无法访问成功

默认 Windows 会使用被动模式连接 FTP 服务器，如果需要以主动模式连接服务器，需要修改 IE 浏览器的属性，方法是查找 Internet 选项的“高级”选项卡，找到使用被动 FTP，取消该功能即可。

4. 账户登录后无法上传数据

根据不同的登录类型，检查主配置文件的设置，匿名账号与虚拟账户检查以 anon_开头的权限设置，本地账户检查以 local_开头的权限设置，并且要确保全局 write_enable 设置为 YES。此外，文件系统目录的权限也需要修改，确保客户端账户有权限访问该目录。

5. 启动服务时报错：500 OOPS：bad bool value in config file

vsftpd 配置文件设置错误，检查配置文件。配置文件要求每个设置项占用独立的一行，并且不可以有多余的空格。

4.4 ProFTPD 文件共享

ProFTPD 是一个安全、配置简单的 FTP 服务器软件，该软件采用与 Apache 相似的配置格式，比较容易配置与管理。在该项目开发以前，最流行的 FTP 软件是 wu-ftpd，wu-ftpd 具有很高的性能，但缺乏 Win32 FTP 服务器的一些特色功能，而且 wu-ftpd 的安全问题一直存在。ProFTPD 借鉴了 wu-ftpd 的优点，并且增加了很多 wu-ftpd 所不具备的功能。

4.4.1 安装 ProFTPD 软件

```
[root@centos6 ~]# wget ftp://ftp.proftpd.org/distrib/source/proftpd-1.3.4c.tar.gz
[root@centos6 ~]# cd /usr/src/proftpd-1.3.4c/
```

安装前先建议阅读 INSTALL 与 README 文档，在这些文档中有比较详细的安装和使用说明。接下来的操作是解压具体的软件包，configure 配置软件安装参数，make 编译软件，make install 安装软件。

```
[root@centos6 ~]# tar -xzf proftpd-1.3.4c.tar.gz -C /usr/src/
[root@centos6 ~]# cd /usr/src/proftpd-1.3.4c/
[root@centos6 proftpd-1.3.4c]# ./configure --prefix=/usr/local/proftpd\
>--sysconfdir= /etc/--enable-nls --enable-openssl --enable-shadow
```

configure 部分选项说明：	
--prefix=PREFIX	指定安装路径（--prefix=/usr/local/）
--sysconfdir=DIR	指定 FTP 服务配置文件路径（--sysconfdir=/etc）
--localstatedir=DIR	指定运行状态的文件存放位置（默认/var/proftpd）
--with-modules=mod_ldap	指定需要加载的功能模块
--enable-memcache	支持缓存功能
--enable-nls	支持多语言环境（如中文），安装完成后在主配置文件中需要指定字符编码（UseEncoding UTF-8 CP936）
--enable-openssl	支持 TLS 加密 FTP 服务
--enable-shadow	支持使用/etc/shadow 验证用户密码

```
[root@centos6 proftpd-1.3.4c]# make && make install
```

安装完成后，ProFTPD 主程序目录位于/usr/local/proftpd 目录下，服务器主配置文件为/etc/proftpd.conf。下面通过修改 PATH 变量增加 proftpd 命令的搜索路径，并为 proftpd 创建启动账户。

```
[root@centos6 ~]# PATH= $PATH:/usr/local/proftpd/bin
[root@centos6 ~]# useradd -M -s /sbin/nologin proftp
```

4.4.2　配置文件解析

ProFTPD 主配置文件共分三个部分：全局设置、目录设置和匿名访问设置。全局设置为全局生效的参数，参数与值直接使用空格分隔。目录设置可以指定共享路径以及相关权限，目录设置使用<Directory "路径">开头，以</Directory>结尾，使用 Limit 设置路径权限。最后如果需要开启匿名账户访问，则使用<anonymouse "路径"></anonymouse>设置匿名访问权限及访问策略。

```
ServerName              "ProFTPD Default Installation" #客户端连接后显示的提示字符
ServerType              standalone                     #服务器启动模式，独立后台进程
DefaultServer           on                             #作为默认服务器
Port                    21                             #默认监听 21 端口
UseIPv6                 off                            #禁用 IPv6
Umask                   022                            #权限掩码
MaxInstances            30                             #最大并发连接为 30
User                    nobody                         #启动服务器的账户
Group                   nogroup                        #启动服务器的组账户
AllowRetrieveRestart    on                             #允许断点继传（上传）
AllowStoreRestart       on                             #允许断点继传（下载）
UseEncoding             UTF-8 CP936                    #支持的编码格式(中文)
RootLogin               off                            #禁止 root 登录 ftp
SystemLog               /var/log/proftp.log            #产生独立的日志文件
TransferLog             /var/log/proftp.log            #记录用户下载的日志信息

#DefaultRoot ~                                         #默认共享路径的根路径

AllowOverwrite          on                             #是否允许使用文件覆盖权限
<Limit SITE_CHMOD>                                     #权限设置
  DenyAll
</Limit>
<Anonymous ~ftp>                                       #匿名访问设置，默认为匿名访问
  User                  ftp
  Group                 ftp
  UserAlias             anonymous ftp
  MaxClients            10
  DisplayLogin          welcome.msg
  DisplayChdir          .message
  <Limit WRITE>
    DenyAll
  </Limit>
</Anonymous>
```

4.4.3 ProFTPD 权限设置

ProFTPD 可以通过在目录属性中添加<Limit>的方式设置访问权限，访问权限设置可以根据自己的实际需求有选择地添加，表 4-3 列出了权限指令以及对应的功能说明。

表 4-3

权限指令	全称	含义
CWD	Change Working Directory	进入该目录
MKD	Make Directory	创建目录
RNFR	Rename from	更名
DELE	Delete	删除文件
RMD	Remove Directory	删除目录
READ	Read	可读
WRITE	Write	可写
STOR	Transfer a file from the client to the server	可上传
RETR	Transfer a file from the server to the client	可下载
DIRS	List directory	允许列出目录
LOGIN	Login to the server	允许登录
ALL	All	全部
AllowUser		设置允许的账户，多个账户使用逗号隔开
AllowGroup		设置允许的组账户，多个账户使用逗号隔开
AllowAll		允许所有
DenyAll		拒绝所有
DenyUser		设置拒绝的账户，多个账户使用逗号隔开
DenyGroup		设置拒绝的组账户，多个账户使用逗号隔开

4.4.4 虚拟用户应用案例

ABC 网站设计公司有商务部、页面设计部、开发部、运维部四个部门，各部门用户访问 FTP 后可以看到所有的目录，但只可以访问本部门的目录，且需要开启 FTP 日志功能，FTP 认证采用基于文件认证的方式，共享目录为/var/ftp。

1. 创建启动账户及共享目录

```
[root@centos6 ~]# useradd -M -s /sbin/nologin proftp
```

```
[root@centos6 ~]# mkdir -p /var/proftp/{develop,ops,sales,design}
[root@centos6 ~]# chmod 777 /var/proftp/{develop,ops,sales,design}
```

2. 修改配置文件

部署服务最主要的步骤就是修改配置文件，下面的输出是 proftpd 主配置文件 /etc/proftpd.conf 被修改后的内容，部分参数在初始配置文件中不存在，需要手动输入。

```
ServerName                  "ProFTPD Default Installation"
ServerType                  standalone
DefaultServer               on
UseEncoding                 UTF-8 CP936
Port                        21
AllowRetrieveRestart        on
AllowStoreRestart           on
UseIPv6                     off
Umask                       022
RootLogin                   off
MaxInstances                30
SystemLog                   /var/log/proftp.log
TransferLog                 /var/log/proftp.log
User                        proftp
Group                       proftp
DefaultRoot                 /var/ftp
AllowOverwrite              on
#下面的匿名共享部分全部使用#注释，取消匿名访问功能
#<Anonymous ~ftp>
#  User                     ftp
#  Group                    ftp
#  UserAlias                anonymous ftp
#  MaxClients               10
#  DisplayLogin             welcome.msg
#  DisplayChdir             .message
#  <Limit WRITE>
#    DenyAll
#  </Limit>
#</Anonymous>
#用户登录是否需要 shell（对虚拟用户很重要）
RequireValidShell off
#通过文件认证用户登录，需要使用 ftpasswd 命令创建该文件
AuthUserFile /usr/local/proftpd/ftpd.passwd
<Directory "/var/ftp/*">
```

```
<Limit CWD READ>          #允许所有的人可以查看根目录
AllowAll
</Limit>
</Directory>
<Directory "/var/ftp/ops">
<Limit CWD MKD RNFR READ WRITE STOR RETR>
#拒绝所有的人往该目录下执行 Limit 后的操作指令
    DenyAll
</Limit>
<Limit DELE>
#禁止任何人在该目录下删除文件
    DenyAll
</Limit>
<Limit CWD MKD RNFR READ WRITE STOR RETR>
#仅允许 tomcat 用户可以执行 Limit 后的所有指令
    AllowUser tomcat
</Limit>
</Directory>
<Directory "/var/ftp/develop">
<Limit CWD MKD RNFR READ WRITE STOR RETR>
DenyAll
</Limit>
<Limit DELE>
    DenyAll
</Limit>
<Limit CWD MKD RNFR READ WRITE STOR RETR>
    AllowUser jacob,jack
</Limit>
</Directory>
<Directory "/var/ftp/sales">
<Limit CWD MKD RNFR READ WRITE STOR RETR>
    DenyAll
</Limit>
<Limit DELE>
    DenyAll
</Limit>
<Limit CWD MKD RNFR READ WRITE STOR RETR>
    AllowUser sales1
</Limit>
</Directory>
<Directory "/var/ftp/design ">
<Limit CWD MKD RNFR READ WRITE STOR RETR>
```

```
    DenyAll
</Limit>
<Limit DELE>
    DenyAll
</Limit>
<Limit CWD MKD RNFR READ WRITE STOR RETR>
    AllowUser design1
</Limit>
</Directory>
```

3. 创建虚拟账户

修改完配置文件后，接下来就需要创建访问 FTP 所需要的账户和密码，proftpd 提供了一个 ftppasswd 命令，使用该命令就可以创建我们需要的账户信息。

该命令的描述和用法如下。

描述：创建用户文件、组文件，默认创建的用户文件为 ftpd.passwd。

选项：--passwd　　创建密码文件，即 AuthUserFile 指定的文件。

--group　　创建组文件。

--name　　指定创建的用户名。

--uid　　指定用户虚拟 UID。

--gid　　指定虚拟 GID。

--home　　指定用户家目录。

--shell　　指定用户 Shell。

--file　　指定创建的文件名，默认为 ftpd.passwd。

```
[root@centos6 ~]# cd /usr/local/proftpd/
[root@centos6 proftpd]# ftpasswd --passwd --name=tomcat --uid=1001 --gid=1001 \
>--home=/home/nohome --shell=/bin/false
[root@centos6 proftpd]# ftpasswd --passwd --name=jacob --uid=1002 --gid=1002 \
>--home=/home/nohome --shell=/bin/false
[root@centos6 proftpd]# ftpasswd --passwd --name=jack --uid=1003 --gid=1003 \
>--home=/home/nohome --shell=/bin/false
[root@centos6 proftpd]# ftpasswd --passwd --name=sales1 --uid=1004 --gid=1004 \
>--home=/home/nohome --shell=/bin/false
[root@centos6 proftpd]# ftpasswd --passwd --name=design1 --uid=1005 --gid=1005 \
>--home=/home/nohome --shell=/bin/false
```

4. 启动 proftpd 服务

```
[root@centos6 ~]# /usr/local/proftpd/sbin/proftpd
```

4.4.5 常见问题分析

1. 如何对匿名账户隐藏一个目录

使用 HideGroup 或 HideUser 指令可以隐藏目录。下面我们对匿名服务器中的 test 目录进行隐藏操作，语法格式片段如下：

```
<Anonymous ~ftp>
…
<Directory test>
HideUser user1
</Directory>
```

2. 如何限制带宽

在新的 Proftpd 1.28 版本后，软件加入了 TransferRate 指令，该指令支持对每个连接限制带宽，限制对上传与下载都有效。

3. 如何限制上传和下载文件的最大容量

Proftpd 1.2.7rc1 版本后提供了 MaxRetrieveFileSize 和 MaxStoreFileSize，用来控制上传和下载文件的最大容量。

4. 如何隐藏 Proftpd 软件的版本信息，防止针对版本漏洞的攻击

在主配置文件中添加 ServerIdent off 指令，即可对用户隐藏软件的版本信息。

5. 如何确保 Proftpd 可以开机启动

源码安装的 Proftpd 软件并不提供开机启动脚本，需要我们自己编写。在第 3 章的综合案例中，我们给出了该脚本，可以供读者借鉴使用。将 Proftpd 启动脚本存放在/etc/init.d 目录下，使用 chkconfig --add proftpd 将服务加入开机启动项，最后通过命令 chkconfig proftpd on 将该服务设置为 2345 级别开机启动。

4.5 Subversion 版本控制

4.5.1 Subversion 简介

Subversion（简称 SVN）是一款自由开放的版本控制软件，Subversion 可以管理文件、文件

夹以及记录它们的修改状况。Subversion 常用来帮助我们管理软件开发的源代码或是公司手册文档。Subversion 通过将文档导入至版本库中进行管理，版本库类似于文件服务器，但比文件服务器更强大的是，它可以记录用户每一次对文件或目录的修改状况，并提供还原数据至老版本的功能。

版本控制对于软件开发而言是至关重要的，现在的软件开发一般作为项目进行，一个项目中会有很多人参与，那么每个人手中都有自己的一份修改备份，最后谁的版本是最新的？有时一款软件开发出来之后会有很多个版本，比如 1.0 版本、2.0 版本，而且每个大版本下还有小版本号，如 1.1.1、2.1.1 等版本，如此多的代码如何管理？SVN 可以很好地帮助我们管理这些代码。

这里我们设定一个模拟环境，如图 4-6 所示，版本库是我们存放资料的地方，本节后面的案例中，我们都以 172.16.0.118 作为版本库服务器的 IP 地址。图中有两个用户需要经常对版本库中的资料进行读写操作。从图中的 I 开始，两个用户都从版本库中将数据复制到了本地，此时用户本地电脑上的数据是版本库的副本（也叫工作副本）。在图 II 中，两个用户在各自的电脑中对副本文件分别修改为 A1 与 A2，并且图III中的一个用户先与版本库进行了数据同步，将本地的最新版本数据更新到服务器上，服务器版本被更新至 A1 版本，随后图Ⅳ中的另一个用户也想将自己修改的 A2 版本与服务器数据版本同步，则服务器会提示该用户 A2 版本已过期，如果两个人的修改并不冲突（如两个修改的是版本库中的不同文件或修改的是同一个文件的不同地方），可以选择将两个版本合并后再上传至服务器，如果两个人的修改有冲突，则需要人工判断谁的修改有效，最后再上传至版本库。

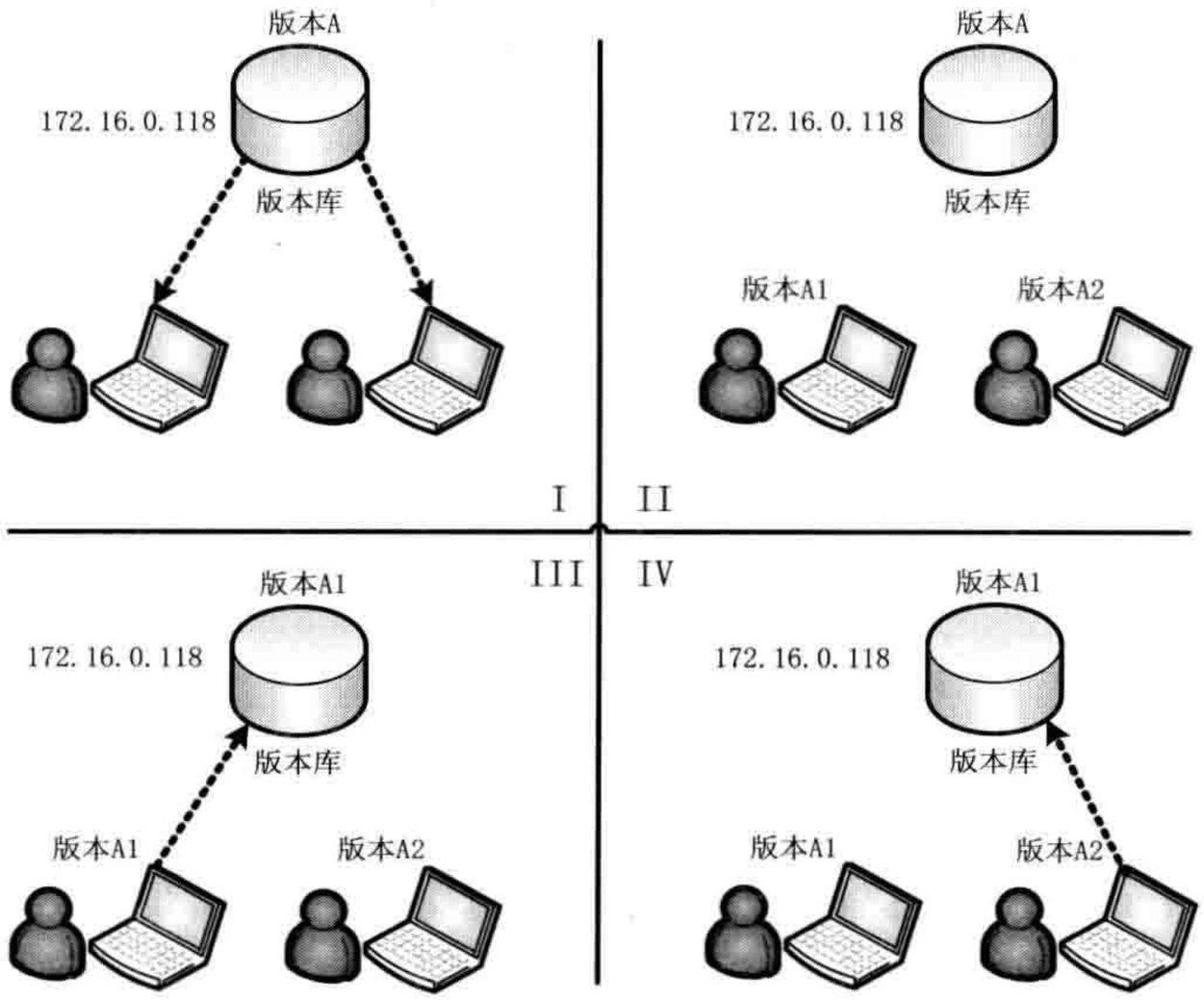


图 4-6

图 4-7 是 Subversion 的架构图，从图中可以看出，服务器端保存着 Subversion 的版本库，客户端通过命令或图形工具连接到服务器，并建立本地“工作副本”，也就是从服务器端将版本库中的资料复制至客户端本地。当然，客户端主机可以将自己本地的版本更新至服务器版本库。而客户端可以通过三种方式连接服务器取得版本库中的数据。4.5.2 节将介绍这些连接方式的区别。

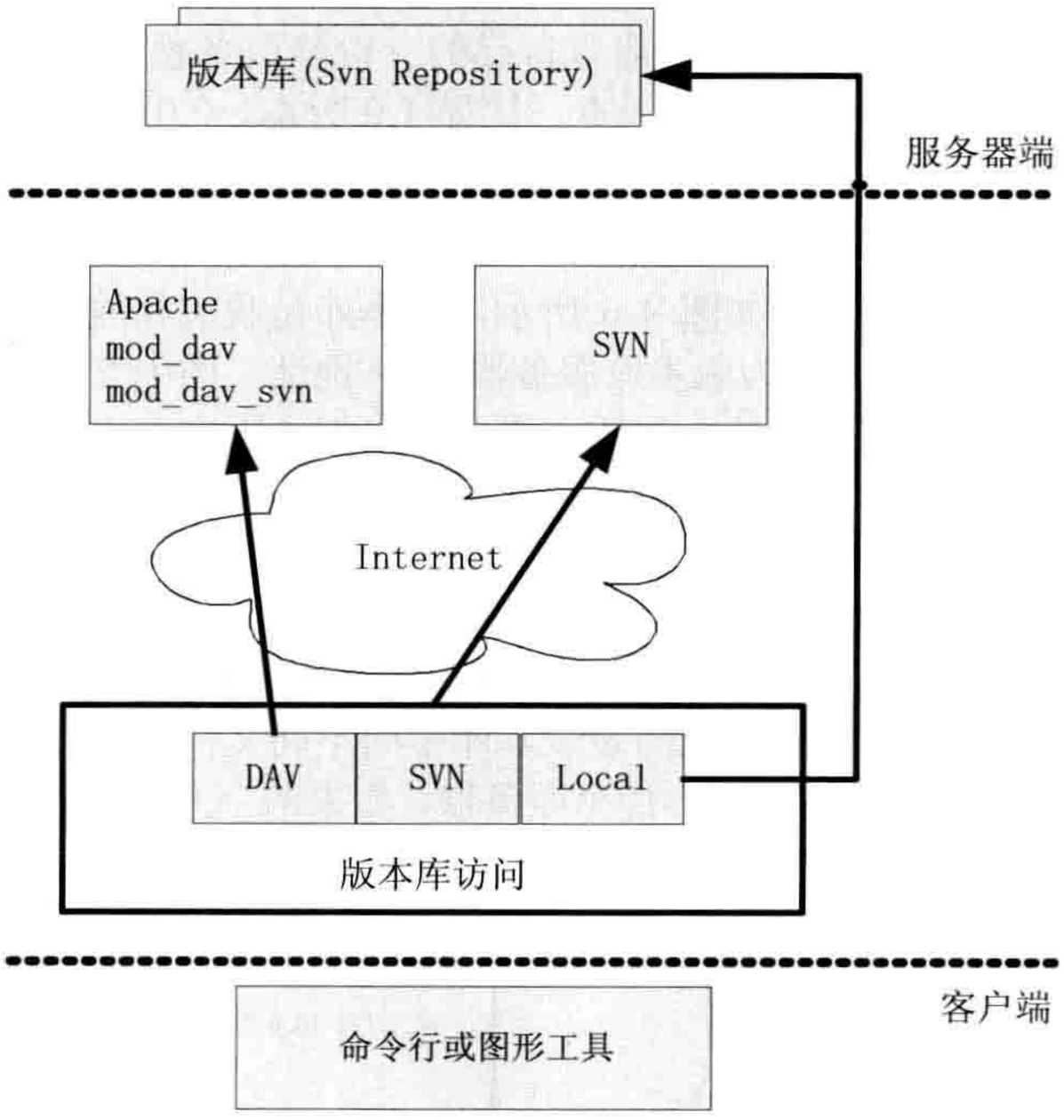


图 4-7

4.5.2 Subversion 服务器对比

客户端访问服务器时可以通过三种方式访问：svnserve 访问、svnserve+SSH 访问、Apache 间接访问，表 4-4 对比了三种不同服务器的功能与特色。

- svnserve 是一个小巧、轻便的服务器程序，客户端通过使用 Subversion 专用的协议进行访问，其特点是简单。
- 由于 svnserve 本身并不提供数据加密通信的功能，所以，如果你需要更加安全地连接，可以使用 svnserve+ssh 的方式与服务器建立 SSH 隧道连接，再通过 SSH 调用 svnserve 程序，实现数据的安全加密传输。

◎ Apache 通过 mod_dav_svn 模块可以访问版本库，这样客户端可以通过访问 Apache 取得版本库资料，从而完成对数据的版本控制。

表 4-4

功能	Apache+DAV	svnserve	Svnserve+SSH
认证	HTTP 基本认证、LDAP、证书	支持 MD5 认证	SSH 认证
权限	可对版本库整体设置权限，也可对指定目录设置权限	可对版本库整体设置权限，也可对指定目录设置权限	仅对版本库整体设置权限
加密	支持 SSL 加密	无	支持 SSH 隧道加密
日志	完善的 Apache 日志	无	无
速度	慢	快	快
设置	相对复杂	简单	相对简单

4.5.3　安装 Subversion 软件

Subversion 版本控制软件在 CentOS 6.3 的系统光盘中有 RPM 格式的版本，但 RPM 格式的软件无法进行自定义设置。所以，如果需要更高的灵活性或高度的可定制性，可以到 Subversion 官方网站下载源码软件，源码包可以到网址 http://apache.etoak.com/subversion/subversion-1.7.8.tar.bz2 处下载。本书案例采用的是系统盘中自带的 RPM 软件包，下面通过 YUM 方式安装该软件包。

```
[root@centos6 ~]# yum -y install subversion
```

4.5.4　svnserve 服务器搭建

版本库软件安装完成以后，我们首先要做的是创建一个版本库，svnadmin 工具是 Subversion 软件所提供的版本库管理工具，该工具可以用来创建库、备份库、修订版本等操作。其次，我们还需要为版本库建立一套有效的认证机制以增强版本数据库的安全性。最后，当服务器端的所有设置均完成后，启动服务器端相应的服务进程即可。

1. 创建服务器版本库

利用 svnadmin 命令可创建服务器版本库。该命令的描述和用法如下。

描述：subversion 版本库管理工具，通过 svnadmin help 可以查看命令帮助。
用法：svnadmin 命令 /版本库路径 [选项]

命令：create　　创建一个新的版本库
　　hotcopy　　版本库热备
　　lslocks　　打印所有的锁描述

svnadmin 实例：

```
[root@centos6 ~]# svnadmin create /var/svn/project1
[root@centos6 ~]# svnadmin hotcopy /var/svn/project1 /var/svn/project1_copy
[root@centos6 ~]# svnadmin lslocks /var/svn/project1
```

ABC 公司近期刚刚接到一个网站项目，由于项目开发是多人进行的，所以公司希望使用版本控制系统高效地管理项目代码，项目代码保存在/var/web code 目录下，下面我们将为该项目创建名为 web_project 的版本库。

```
[root@centos6 ~]# mkdir /var/svn
[root@centos6 ~]# svnadmin create /var/svn/web_project
```

通过 svnadmin 命令已经创建了一个名为 web_project 的空版本库，下面我们将使用 svn 命令将项目代码导入到版本库中。import 代表执行导入操作，将本地 web_code 目录下的资料导入至刚刚创建的空版本库 web_project 中，-m 后面跟的是说明性的字符串，可以为任意字符。

```
[root@centos6 ~]# svn import /var/web_code/ file:///var/svn/web_project/ -m "Initial DaTA"
[root@centos6 ~]# svn list file:///var/svn/web_project/ #列出版本库中的资料列表
```

2. 认证与授权

使用 SVN 内置的认证机制可以有效地增强客户端访问版本库的安全性，当客户端访问版本库服务器时，服务器会根据版本库目录下的 conf/svnserve.conf 文件中定义的认证与授权策略实现权限的控制。下面是该文件的核心配置说明。

```
[root@centos6 ~]# cat /var/svn/web_project/conf/svnserve.conf
…
[general]
anon-access = none          #设置拒绝匿名账户访问，此处可以设置为 none、read、write
auth-access = write         #经过认证的账户权限为可写
password-db = passwd        #指定账户名称与密码的存放文件名，该文件在 conf 目录下
authz-db = authz            #指定基于路径的访问控制文件名（可以对文件或目录设置权限）
real = My First Repository
                    #设置版本库域，如果两个版本库的认证域相同，它们将使用相同的密码数据
```

在 svnserve.conf 中已经配置好了账户密码文件，下面分别看看 passwd 与 authz 文件的内容，默认该文件存放在版本库的 conf 目录下。在 passwd 文件中需要设置账户信息，在 authz 文件中需要设置访问控制权限。

```
[root@centos6 ~]# cat /var/svn/web_project/conf/passwd
...
[users]
harry = harryssecret          #用户名为 harry，密码为 harryssecret
sally = sallyssecret          #用户名为 sally，密码为 sallyssecret
```

```
[root@centos6 ~]# cat /var/svn/web_project/conf/authz
...
[groups]
admins = harry,sally          #定义组，组成员有 harry 与 sally
[/]                           #对版本库根路径设置权限，可以设置为需要控制的路径
@admins = rw                  #admins 组中的用户可读、可写权限
* = r                  #其他所有的人只读，权限可以设置为只读('r')、读写('rw')、无权限('')
```

3. 启动 svnserve 服务

svnserve 命令的描述和用法如下。

描述：SVN 服务器程序

用法：svnserve [选项]

选项：-d　　以守护进程方式运行 svnserve。

--listen-port=port　指定监听的端口，默认监听端口号为 3690。

-r root　　为版本库指定一个虚拟路径，默认客户端要指定绝对路径访问库。

```
[root@centos6 ~]# svnserve -d
```

直接运行 svnserve 命令即可启动 SVN 服务进程，但如果需要该服务作为后台程序持续监听客户端访问，可以使用-d 选项使该程序以守护进程的方式启动 svnserve 服务，SVN 服务默认监听端口 3690，如果防火墙处于开启状态，需要注意对防火墙的正确设置。svnserve 运行后，会将所有的版本库发布至网络（假设有多个版本库）。此时，客户端需要指定绝对路径访问版本库，如 svn://centos.example.com/var/svn/web_project。同时，服务器端如果需要在 authz 文件中为目录设置权限，路径应该为[web_projet:/]或[project2:/test]，即这里的[web_project:/]表示 web_project 版本库的根，[project2:/test]表示 project2 下的 test 目录。

默认 SVN 会将服务器计算机中所有的版本库共享给网络用户，但有时我们仅希望发布其中一个版本库时，就需要限制仅发布 web_project 一个版本至网络，这样客户端也可以使用相对路径访问版本库，如，svn://centos.example.com/web_project（客户端会访问服务/var/svn/目录下的 web_project 项目）。同时，服务器端如果需要在 authz 文件中为目录设置权限，路径应该为[/]或[/test]，即这里的根（/）仅表示 web_project 版本库，/test 表示 web_project 下的 test 目录。如果需要仅发布个别版本库给网络用户，可以使用 svnserve 命令的-r 选项，该选项后面接版本库

的路径。

```
[root@centos6 ~]# svnserve -d -r /var/svn/
```

4. 客户端访问

版本库服务器创建完成后，我们可以通过多种方式访问 SVN 服务器的版本库，可以使用命令行或图形工具，也可以通过本地磁盘或网络协议访问。但不管你使用哪种方式，都需要提供一个 URL 地址来定位版本库的位置，表 4-5 中具体说明了每种 URL 的使用格式及含义。

表 4-5

URL 格式	含义
file:///	直接访问本地磁盘上的版本库（客户端与服务器端在一台机器上）
http://	配置 Apache 的 WebDAV 协议，通过网页访问版本库
https://	与 http://相似，但使用了 SSL 进行数据加密
svn://	通过 svnserve 定义的协议访问版本库
svn+ssh://	与 svn://相似，但使用了 SSH 封装加密数据

在客户端访问服务器版本库的众多方法中，命令行的方式是高效、功能完善、无须安装第三方软件的一种简单方式，SVN 软件为我们提供了一个名为 svn 的命令行程序。

（1）svn 命令

描述：subversion 客户端命令行工具。

用法：svn 命令 [选项]

选项：--password　　密码

--username　　用户名

--revision(-r)　　指定要检查的特定版本

命令：add　　添加文件、目录或符号链接

cat　　输出特定文件的内容

checkout　　URL[@REV] [PATH]

从服务器版本库中复制一份副本至本地，URL 定位版本库，通过 REV 可以下载特定版本的数据，PATH 为本地工作副本路径。

commit　　将本地工作副本修改后的内容发布到版本中，简写为 ci

copy SRC DST　　将工作副本中的一个文件或目录复制至版本库

delete PATH　　从本地工作副本中删除一个项目

delete URL　　从版本库中删除一个项目
diff　　对比两个版本之间的差别
import　　提交一个路径的副本至版本库
info　　显示本地或远程版本信息

svn 实例：

```
[root@centos6 ~]# svn checkout file:///var/svn/project1 mine
[root@centos6 ~]# svn co svn://192.168.0.254/project1 mine
[root@centos6 ~]# svn co svn://172.16.0.118/web_project /mine
[root@centos6 ~]# svn commit -m "modified foo.html"
[root@centos6 ~]# svn commit -m "modified foo.html" /mine
[root@centos6 ~]# svn delete testfile
[root@centos6 ~]# svn diff
[root@centos6 ~]# svn import -m "New project" /etc file:///var/svn/project
[root@centos6 ~]# svn info
[root@centos6 ~]# svn info svn://172.16.0.118/var/svn/web_project
```

除了核心的 svn 命令，SVN 软件包还提供了一个用于对版本库数据信息进行简单查询的工具，svnlook 命令可以帮助用户完成这些查询工作。

（2）svnlook 命令

描述：subversion 检查工具，通过 svnlook help 可以查看命令帮助。

用法：svnlook 命令 /版本库路径 [选项]

选项：--revision(-r) 指定要检查的特定版本

命令：author　　显示作者
cat　　显示版本库中的文件内容
date　　显示时间标记
log　　显示日志消息
tree　　显示版本库资料树

svnlook 实例：

```
[root@centos6 ~]# svnlook author /var/svn/project1
[root@centos6 ~]# svnlook author -r 2 /var/svn/project1 #查看版本 2 的作者信息
[root@centos6 ~]# svnlook author -r 2 /var/svn/project1 test
                                        #查看版本库中 test 文件的内容
```

我们可以使用 svn 命令或图形工具连接版本库服务器，下载版本数据至本地。上面我们介绍了以命令行的方式访问服务器版本数据库，下面来看客户端主机如何通过 SVN 图形程序访问服务器版本库。通过图形方式在本地计算机生成本地副本版本。图形工具选择的是 Windows 平台的 TortoiseSVN 软件，图 4-8 至图 4-10 显示了使用 TortoiseSVN 工具通过图形访问下载版本

库资源，在 Windows 中安装完成该软件后，在桌面单击鼠标右键即可以找到该工具。TortoiseSVN 是免费的基于 GPL 开发的开源自由软件，它是 subversion 版本控制系统的一个非常优秀的客户端程序，可以帮助我们高效地管理文件与目录。你可以在 http://tortoisesvn.net/网站上找到下载链接，该软件还提供了多语言软件包（也包括中文）。

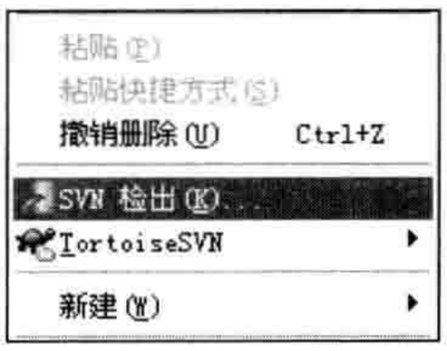


图 4-8

图 4-9

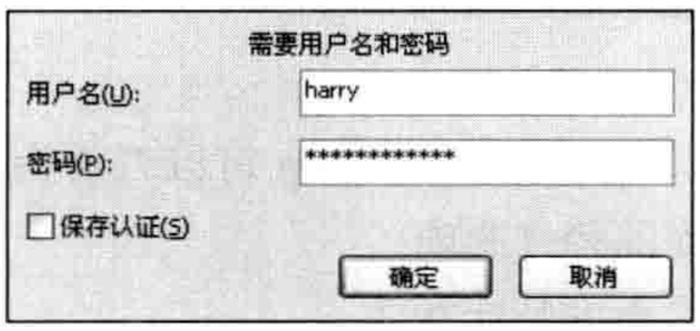


图 4-10

4.5.5 svnserve+SSH 服务器搭建

由于 svnserve 并不支持加密，所以对公司的数据安全危害比较大，基于 SSH 的 svnserve 使得客户端可以通过 SSH 服务调用 SVN 服务程序。客户端需要使用 ssh 程序连接远程服务器的 SSHD 服务，通过 SSH 认证机制验证账户身份后，再自动启动 svnserve 服务。所以服务器端不需要提前启动 SVN 服务。在这种模式下，svnserve.conf 配置文件依然可以进行权限控制，服务器部署流程是启动 SSH 服务，在服务器上创建版本库，向版本库导入数据，最后客户端使用命令或图形方式访问服务器。具体操作步骤如下。

1. 服务器端启动 SSHD 服务器

```
[root@centos6 ~]# service sshd start
[root@centos6 ~]# chkconfig sshd on
```

2. 服务器端创建版本库

```
[root@centos6 ~]# svnadmin create /var/svn/web_project2
[root@centos6 ~]# svn import /var/code2 file:///var/svn/web_project2 -m "Web code"
```

3. 客户端访问

```
[root@centos6 ~]# svn co svn+ssh://172.16.0.118/var/svn/web_project2 /web_code2
```

图 4-11、图 4-12 是通过图形工具访问下载版本库的过程演示，在图 4-11 中填写 svn+ssh 的 URL 路径，在图 4-12 中填写可以远程 SSH 的账户名称。

图 4-11

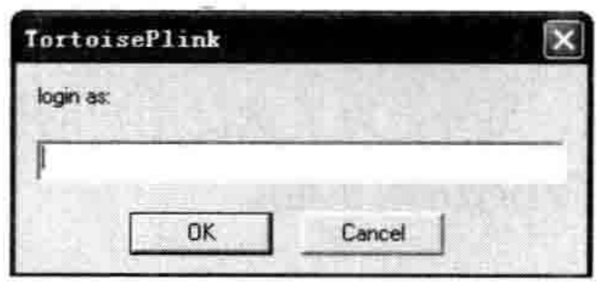


图 4-12

4.5.6　Apache+SVN 服务器搭建

虽然前面两种服务器类型已经可以满足多数人的使用需求，但当客户端没有任何工具可以使用时，我们还可以创建基于 Apache 的 SVN 服务器，这样客户端只要有浏览器，就可以访问版本库服务器。Apache 是目前非常流行的 Web 服务器软件，它允许在几乎所有的计算机平台上运行，目前绝大多数系统平台都可以部署 Apache Web Server，使用 Apache Web Server 访问版本库需要加载 mod_dav 与 mod_dav_svn 模块，Apache 需要通过这两个模块才可以管理 subversion 版本库。关于 Apache 软件的详细介绍，请参考本书 4.9 节的内容。以下列举了使用 Apache 发布 subversion 版本库的若干理由。

- subversion 可以使用 Apache 的多种认证方式。
- 不需要创建系统账户。
- Apache 提供了完善的日志功能。
- 可以通过 TLS 进行数据加密。

- HTTP 以及 HTTPS 可以穿越企业防火墙。
- 客户端简单地通过浏览器访问版本库。

1. 创建服务器版本库

```
[root@centos6 ~]# svnadmin create /var/svn/web_project3
[root@centos6 ~]# svn import /var/web_code3 file:///var/svn/web_project3\
>-m "Term 3"
```

2. 安装 Apache 及相关模块软件

```
[root@centos6 ~]# yum -y install httpd
[root@centos6 ~]# yum -y install mod_dav_svn
```

3. 修改 Apache 配置

为了让 Apache Web Server 可以读取 SVN 版本库中的数据，我们需要修改 httpd 针对 SVN 的配置文件 subversion.conf，在该文件中至少需要确保 mod_dav 和 mod_authz_svn 两个模板会被加载。另外，为了满足用户对数据安全的需求，我们还可以利用 httpd 的认证模块实现基于账户和密码的访问机制。

```
[root@centos6 ~]# vim /etc/httpd/conf.d/subversion.conf
```

```
LoadModule dav_svn_module     modules/mod_dav_svn.so    #加载 dav_svn 模块
LoadModule authz_svn_module   modules/mod_authz_svn.so #加载权限设置模块
#除了这两个模块，还要确保/etc/httpd/conf/httpd.conf 文件中 dav_module 模块也会被加载
#   a) readable and writable by the 'apache' user, and
#默认 Apache 的启动用户为 apache，所以需要 apache 用户对版本库有读写权限
#   b) labelled with the 'httpd_sys_content_t' context if using SELinux
#如果开启了 SELinux，则版本库目录需要有 httpd_sys_content_t 安全上下文标签
# To create a new repository "http://localhost/repos/stuff" using
# this configuration, run as root:

#以下内容是配置文件提供的操作步骤模板
#   # cd /var/www/svn
#   # svnadmin create stuff
#   # chown -R apache.apache stuff
#   # chcon -R -t httpd_sys_content_t stuff
#
<Location /svn>
#当用户的 URL 是以/svn 开始(http://hostname/svn/...),Apache 会将控制权交于 DAV 处理
```

```
    DAV svn
    SVNParentPath /var/svn
    #通过 SVNParentPath 指定所有版本库的主目录(发布所有的版本库)
    #也可以通过 SVNPath 指定某个特定版本库的路径(发布特定的版本库)
        AuthType Basic                              #Apache 认证方式
        AuthName "Please input password"            #提示字符
        AuthUserFile /var/svn/.pass                 #账户文件路径，为了安全，应隐藏该文件
        Require valid-user                          #设置仅为有效用户可以访问
    </Location>
```

```
[root@centos6 ~]# chown -R apache.apache /var/svn          #修改权限
[root@centos6 ~]# chcon -R -t httpd_sys_content_t /var/svn
                                                           #仅当 SELinux 开启时使用
[root@centos6 ~]# htpasswd -c /var/svn/.pass jerry
                                                           #创建账户文件，添加 jerry 用户
[root@centos6 ~]# service httpd start
[root@centos6 ~]# chkconfig httpd on
```

4. 客户端访问

我们可以使用任何一款浏览器通过 URL 访问版本库数据，图 4-13 给出了客户端访问版本库的演示效果。

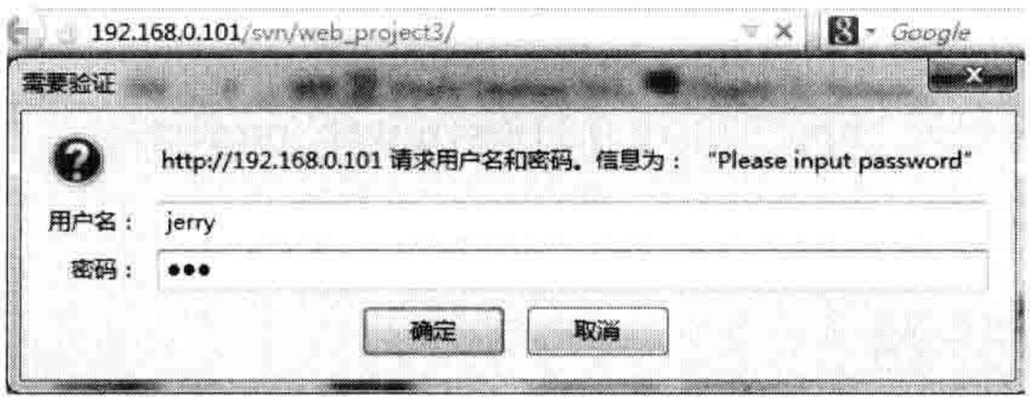


图 4-13

4.5.7　常见问题

1. 访问版本库路径错误

如果通过 svnserve 命令启动服务时指定了发布的具体版本库，则客户端访问时就可以直接使用相对路径访问，如果服务器端使用 svnserve -d -r /var/svn/启动服务后，客户端依然使用 svn co svn://172.16.0.118/var/svn/web_project /mine，系统将提示 svn: No repository found，说明没有找到该版本库，正确的写法是 svn co svn://172.16.0.118/web_project /mine。

2. 每次想访问版本库时，进程就会挂起

首先确保版本库没有被损坏，数据也没有丢失。当进程直接访问版本库时，进程将通过 Berkeley DB 来实现。Berkekey DB 包含日志系统，也就是说，所有的操作在执行前都被记录在日志中。当进程崩溃时，遗留下文件锁，并记录了所有未完成操作的信息，从而导致所有试图访问数据库的进程将因为要访问文件锁而被挂起。若想解除文件锁，可以回滚到前一个正常状态。

3. subversion 提示工作副本已过时

这可能是提交失败导致了你的工作副本被破坏，可以使用 svn revert 回滚、svn update 更新来解决该问题。

4. Windows XP 下，subversion 服务器有时会发布一些错误数据

如果你是在 Windows 平台下搭建 subversion 服务器，可以尝试通过安装 SP1 补丁包解决该问题。

5. Apache 拒绝访问

通过浏览器访问版本库时提示：You don't have permission to access /svn on this server。出现这种提示说明没有权限读取相应的目录，如用户在这里输入 http://172. 16.0.118/svn，就会得到这样的提示，正确的输入应为 http://172.16.0.118/svn/web_project3。

4.6 网络存储服务器

目前计算机领域的存储解决方案主要有直连存储（DAS）、存储区域网络（SAN）以及网络附加存储（NAS）三种。其中，DAS（Direct-Attached Storage）指的是主机总线直接通过 SCSI 接口与存储设备相连，这种连接方式主要应用在家庭个人电脑环境中。SAN（Storage Area Network）指的是一整套存储网络的解决方案，SAN 采用的是光纤通道技术，SAN 就是通过光纤交换机将服务器与存储设备连接在一起的网络整体环境，SAN 的优势在于其先进的光纤通信技术，为当今爆炸式的数据增长环境提供了快速高效的存储方案。NAS（network-Attached Storage）可以使用普通的网络环境，通过以太网交换机等设备连接服务器与存储设备，NAS 的优势在于无须对网络环境进行改造，NAS 可以使用现有的网络环境，而且不同厂家的设备只要采用 TCP/IP 协议，就可以满足设备之间对兼容性的要求，NFS 与 CIFS 就属于网络附加存储的

解决方案，它们都提供了对文件系统的共享。不管是 NAS 还是 SAN，它们都可以实现企业环境中服务器与存储设备相分离的要求。这样服务器就可以专注于服务器业务，而存储设备则专注于数据存储的速度与安全，并且由于服务器与存储设备是分离的，这样多台服务器就可以加载同一存储设备中的数据，实现数据的集中、统一管理。由于传统的 SAN 需要价格昂贵的光纤设备作为基础，所以现在又有了基于 IP 技术的 SAN，可以通过 IP 网络进行数据存储，服务器在不进行任何改造的情况下使用现有的以太网卡就可以访问 IP SAN（ISCSI 就是这样的技术）。NSA 与 SAN 的主要区别在于，NAS 共享的是文件系统，而 SAN 共享的是块设备。本节重点关注 iSCSI 网络存储与数据同步 rsync 系统。图 4-14 给出了存储区域网络的结构图，图中给出了基于光纤通道的 SAN 以及基于 IP 的 SAN 结构。

存储区域网络结构图

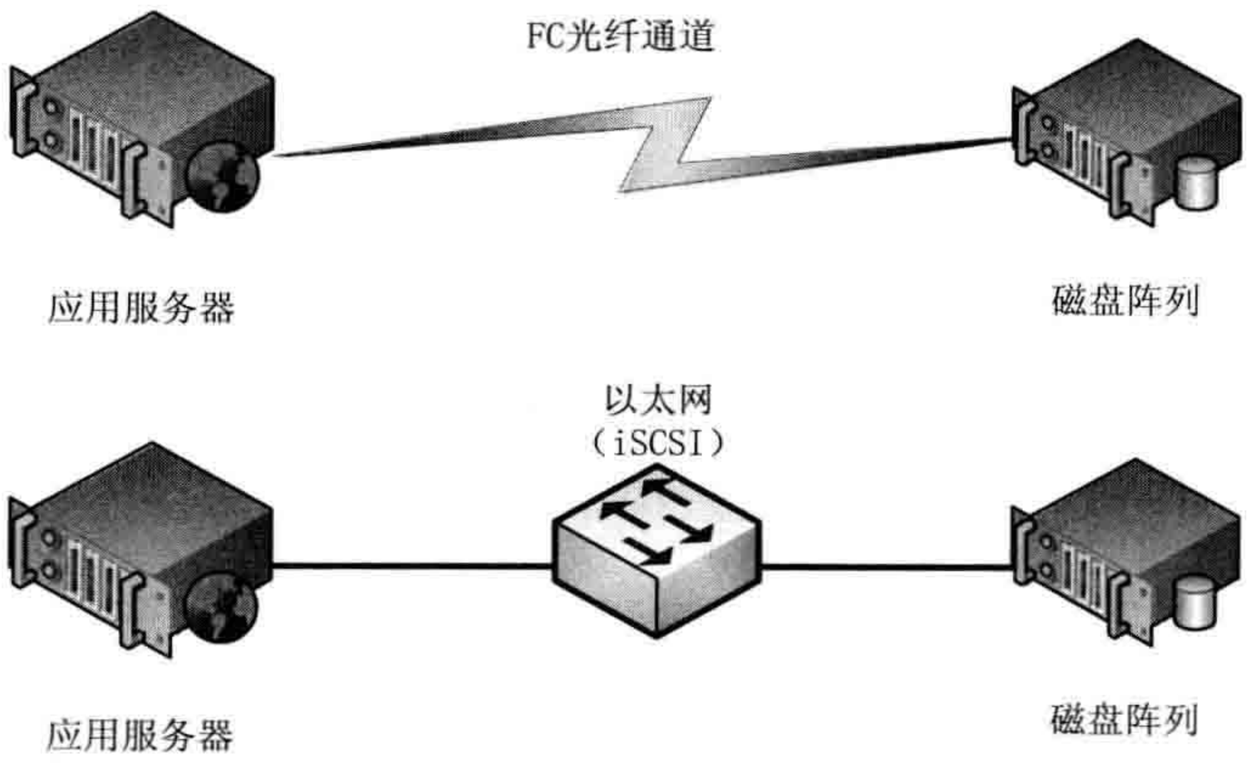


图 4-14

4.6.1　iSCSI 网络存储

iSCSI（Internet Small Computer System Interface）是典型的 IP-SAN 技术，是基于因特网的 SAN 存储技术，该技术使得我们可以在基于 IP 协议的网络上传输 SCSI 命令，在 iSCSI 环境中，客户端（initiators）发送 SCSI 命令给远程的 SCSI 存储设备（targets），实现数据的存储与备份功能。iSCSI 使用 TCP 的 860 与 3260 端口进行通信。iSCSI 与 SCSI 最大的区别在于 iSCSI 摆脱了存储设备的距离限制，使得任何主机都可以通过局域网或广域网访问我们的存储设备，而对于数据中心而言，这是至关重要的。下面我们将在 CentOS 6.3 中部署一个 iSCSI 服务器以及客户端访问的环境，服务器 IP 地址为 192.168.0.254，客户端 IP 地址为 192.168.0.11。

1. 部署 iSCSI 服务

在 CentOS 6.3 平台上部署 iSCSI 服务需要安装 target 软件，安装完成后，我们可以通过命令或修改配置文件这两种方式部署 iSCSI 服务。如果需要使用命令配置服务，可以参考 tgtadm 命令的 man 手册，其中有比较详细的示例。以下操作将使用修改配置文件的方式来进行，但修改配置文件前，我们还需要搞清楚两个概念：LUN、IQN。LUN（Logical Unit Number）是设备的逻辑单元号，一般为一个数字，我们使用 LUN 来标识存储设备。IQN（iSCSI Qualified Name）为 iSCSI 合格名称，一般格式为 iqn.yyyy-mm.<reversed domain name>:identifier，其中，yyyy 表示年，mm 表示月，reversed domain name 是域名的反写，identifier 为标识名称。如：iSCSI 服务器域名为 www.example.com，则 IQN 全称可以写成：iqn.2012-10.com.example.www:disk1。这种 IQN 的书写格式不是创建 iSCSI 服务必须遵守的，但推荐使用这样的标准格式。

部署 ISCSI 服务器时，首先需要安装 scsi-target-utils 软件。另外，为了给其他服务器提供存储，ISCSI 服务器上应该有足够的存储设备，可以使用 fdisk 查看计算机中的磁盘设备列表。其次，最重要的就是修改服务器配置文件，默认的服务器配置文件为/etc/tgt/targets.conf，我们需要在配置文件中添加共享设备，为共享设备创建 LUN 和 IQN。最后启动服务进程即可。

```
[root@centos6 ~]# yum -y install scsi-target-utils
[root@centos6 ~]# fdisk -cul |grep /dev/sd
Disk /dev/sda: 128.8 GB, 128849018880 bytes
/dev/sda1   *        2048     1026047      512000   83  Linux
/dev/sda2         1026048   251658239   125316096   8e  Linux LVM
Disk /dev/sdb: 107.4 GB, 107374182400 bytes
Disk /dev/sdc: 107.4 GB, 107374182400 bytes
/dev/sdc1            2048    41945087    20971520   83  Linux
/dev/sdc2        41945088    83888127    20971520   83  Linux
/dev/sdc3        83888128   125831167    20971520   83  Linux
Disk /dev/sdd: 107.4 GB, 107374182400 bytes
[root@centos6 ~]# vim /etc/tgt/targets.conf
```

```
default-driver iscsi
<target iqn.2012-10.com.example:disk1>
    backing-store /dev/sdb
    backing-store /dev/sdc2
</target>
<target iqn.2012-10.com.example:disk2>
    backing-store /dev/sdd
    lun 10
    vendor_id Jacob Inc.
    incominguser tom tompass12
</target>
<target iqn.2012-10.com.example:disk3>
```

```
        backing-store /dev/sdc3
        lun 20
        initiator-address 192.168.0.102
        incominguser tom tompass12
    </target>
```

/etc/tgt/targets.conf 配置文件说明如下。

第一个 target 定义了两个 LUN，LUN 编号会自动按顺序分配给 LUN1、LUN2，LUN0 一般会分配给 SCSI 控制器。默认允许所有的人访问该 target。/dev/sdb 与/dev/sdc1 是我们准备共享的设备，通过 backing-store 指定设备路径，backing-store 支持文件或块设备。此外，也可以使用 direct-store 指定设备路径，但 direct-store 仅支持本地 SCSI 设备。

第二个 target 定义了一个 LUN，LUN 编号为手动配置的 10，共享设备是/dev/sdd，手动修改设备的厂家信息为 Jacob Inc.。此外，通过 incominguser 选项将该 target 设置为仅允许 tom 账户访问，密码为 tompass12。

第三个 target 定义了一个 LUN，LUN 编号为 20，共享设备为/dev/sdc2，仅允许 tom 账户通过 192.168.0.102 主机访问该 target。

当服务器端启动 tgtd 服务后，我们随时可以通过 tgtadm 命令来查看 iSCSI 的详细信息。以下 tgtadm 的输出内容为精简后的部分输出结果。

```
[root@centos6 ~]# tgtadm --lld iscsi --op show --mode target
Target 1: iqn.2012-10.com.example:disk1
    System information:
        Driver: iscsi
        State: ready
    I_T nexus information:
    LUN information:
    LUN: 0
        Type: controller
        SCSI ID: IET     00010000
        SCSI SN: beaf10
        Size: 0 MB, Block size: 1
        Online: Yes
        Removable media: No
        Prevent removal: No
        Readonly: No
        Backing store type: null
        Backing store path: None
```

```
            Backing store flags:
        Account information:
        ACL information:
            ALL
    Target 2: iqn.2012-10.com.example:disk2
    …此处省略…
            LUN: 10
                Type: disk
                SCSI ID: IET     0002000a
                SCSI SN: beaf210
                Size: 107374 MB, Block size: 512
                Online: Yes
                Removable media: No
                Prevent removal: No
                Readonly: No
                Backing store type: rdwr
                Backing store path: /dev/sdd
                Backing store flags:
        Account information:
            tom
        ACL information:
            ALL
    Target 3: iqn.2012-10.com.example:disk3
    …此处省略…
        LUN: 20
                Type: disk
                SCSI ID: IET     00030014
                SCSI SN: beaf320
                Size: 21475 MB, Block size: 512
                Online: Yes
                Removable media: No
                Prevent removal: No
                Readonly: No
                Backing store type: rdwr
                Backing store path: /dev/sdc3
                Backing store flags:
        Account information:
            tom
        ACL information:
```

```
        192.168.0.102
[root@centos6 ~]# /etc/init.d/tgtd start
[root@centos6 ~]# chkconfig tgtd on
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 3260 -j ACCEPT
[root@centos6 ~]# service iptables save
```

2. 客户端访问

Linux 客户端访问 iSCSI 服务器需要安装 iscsi-initiator-utils 软件包。第一步，通过 iscsiadm 命令的 discoverydb 选项可以发现服务器端 iSCSI 的 IQN。第二步，通过 login 选项加载服务器的 IQN，在本书案例中，客户端主机完成以上两步操作后，通过 fdisk -cul 命令可以发现计算机中多出了三块磁盘。第三步，参考本书 2.5 节对磁盘进行分区格式化后即可使用该 ISCSI 磁盘存储设备了。这里我们不再做分区格式化操作，读者可以自己验证。

注意，有时在客户端使用 discoverydb 无法发现服务器 IQN 信息，此时可以检查防火墙是否设置正确，并检查有没有设置账户及 IP 地址的 ACL 限制，最后在 CentOS 6.3 系统环境中重启服务器才可以被客户端识别新的 IQN。下面的三条命令中，第一条命令用来查看 192.168.0.254 服务器的 ISCSI IQN 信息，第二条命令用来挂载 192.168.0.254 服务器上 IQN 名称为 iqn.2012-10.com.example:disk2 的 ISCSI 设备。最后一条命令用于查看磁盘设备信息。

```
[root@localhost ~]# iscsiadm --mode discoverydb --type sendtargets \
> --portal 192.168.0.254 --discover
192.168.0.254:3260,1 iqn.2012-10.com.example:disk1
192.168.0.254:3260,1 iqn.2012-10.com.example:disk2
192.168.0.254:3260,1 iqn.2012-10.com.example:disk3
[root@localhost ~]# iscsiadm --mode node --targetname \
> iqn.2012-10.com. example:disk2 --portal 192.168.0.254:3260 -login
[root@localhost ~]# fdisk -cul |grep /dev/sd
…部分内容省略…
Disk /dev/sdb: 107.4 GB, 107374182400 bytes
Disk /dev/sdc: 21.5 GB, 21474836480 bytes
Disk /dev/sdd: 107.4 GB, 107374182400 bytes
Disk /dev/sde: 21.5 GB, 21474836480 bytes
```

4.6.2　Rsync 文件同步

Rsync（remote sync）是 UNIX 及类 UNIX 平台下一款神奇的数据镜像备份软件，它不像 FTP 或其他文件传输服务那样需要进行全备份，Rsync 可以根据数据的变化进行差异备份，从而减少数据流量，提高工作效率。你可以使用它进行本地数据或远程数据的复制，Rsync 可以

使用 SSH 安全隧道进行加密数据传输。Rsync 服务器端定义源数据，Rsync 客户端仅在源数据发生改变后才会从服务器上实际复制数据至本地，如果源数据在服务器端被删除，则客户端数据也会被删除，以确保主机之间的数据是同步的。Rsync 使用 TCP 873 端口。

1. 搭建 Rsync 服务器

部署 Rsync 服务器需要创建至少一个配置文件，默认在系统中并不存在 Rsync 配置文件，对于服务器而言，配置文件创建完成后，使用守护进程模式启动 rsync 程序即可。下面将以图 4-15 为原型创建配置文件，将/common 目录发布给所有的客户端主机。因为 Rsync 是 Andrew Tridgell （Samba 的作者）与 Paul Mackerras 合作开发的软件，所以 Rsync 配置文件与 Samba 配置文件比较相似。Rsync 主要分为三个配置文件，分别是：rsyncd.conf（主配置文件）、rsyncd.secrets（密码文件）和 rsyncd.motd（服务器信息文件）。

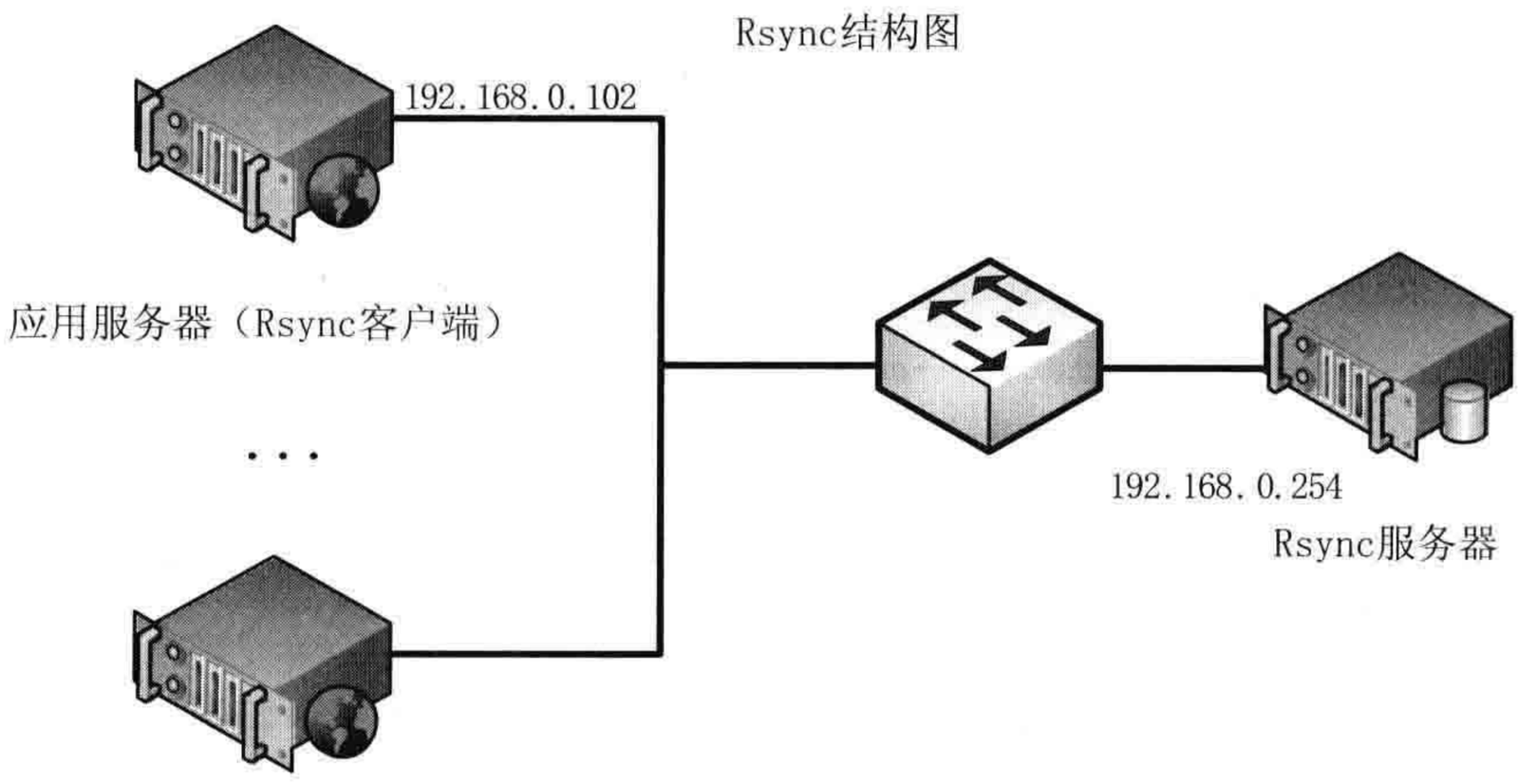


图 4-15

使用 CentOS 6.3 系统安装部署 Rsync 非常方便，安装光盘以及 YUM 源中都提供了 rsync 软件包，使用 YUM 方式安装即可。下面的案例演示了如何共享/common 目录，为此，我们需要创建/common 目录，并复制一些测试文件存放在该目录下。本例需要手动创建配置文件/etc/rsync.conf，该文件具体的语法格式在后面有详细的描述。

```
[root@centos6 ~]# yum -y install rsync
[root@centos6 ~]# mkdir /common; cp /etc/init.d/* /common/
[root@centos6 ~]# vim /etc/rsync.conf
```

```
#/etc/rsyncd.conf
#设置服务器信息提示文件名称，在该文件中编写提示信息
motd file = /etc/rsyncd.motd
#开启 Rsync 数据传输日志功能
transfer logging = yes
#设置日志文件名称，可以通过 log format 参数设置日志格式
log file = /var/log/rsyncd.log
#设置 Rsync 进程号保存文件名称
pid file = /var/run/rsyncd.pid
#设置锁文件名称
lock file = /var/run/rsync.lock
#设置服务器监听的端口号，默认为 873
port = 873
#设置服务器所监听网卡接口的 IP 地址，这里服务器 IP 地址为 192.168.0.254
address = 192.168.0.254
#设置进行数据传输时所使用的账户名称或 ID 号，默认使用 nobody
uid = nobody
#设置进行数据传输时所使用的组名称或 GID 号，默认使用 nobody
gid = nobody
#设置user chroot 为 yes 后，rsync 会首先进行 chroot 设置，将根映射到 path 参数路径下，对客户
#端而言，系统的根就是 path 参数所指定的路径。但这样做需要 root 权限，并且在同步符号
#连接资料时仅会同步名称，而内容将不会同步。
use chroot = no
#是否允许客户端上传数据，这里设置为只读。
read only = yes
#设置并发连接数，0 代表无限制。超出并发数后，如果依然有客户端连接请求，则将会收
#到稍后重试的提示消息
max connections = 10
#模块，Rsync 通过模块定义同步的目录，模块以[name]的形式定义，这与 Samba 定义共
#享目录是一样的效果。在 Rsync 中也可以定义多个模块
[common]
#comment 定义注释说明字串
comment = Web content
#同步目录的真实路径通过 path 指定
path = /common
#忽略一些 IO 错误
ignore errors
#exclude 可以指定例外的目录，即将 common 目录下的某个目录设置为不同步数据
```

```
#exclude = test/
#设置允许连接服务器的账户，账户可以是系统中不存在的用户
auth users = tom,jerry
#设置密码验证文件名称，注意该文件的权限要求为只读，建议权限为 600，仅在设置 auth users
#参数后有效
secrets file = /etc/rsyncd.secrets
#设置允许哪些主机可以同步数据，可以是单个 IP，也可以是网段，多个 IP 与网段之间使用空
#格分隔
hosts allow=192.168.0.0/255.255.255.0
#设置拒绝所有（除 hosts allow 定义的主机外）
hosts deny=*
#客户端请求显示模块列表时，本模块名称是否显示，默认为 true
list = false
```

接下来，通过 echo 的方式创建密码文件/etc/rsyncd.secrets，在该文件中输入两个账户：tom 账户的密码是 pass，jerry 账户的密码是 111。需要注意的是，密码文件不可以对所有的人开放可读权限，为了安全，建议设置权限为 600。创建服务器提示信息文件并向该文件中导入欢迎词。由于 Rsync 默认不是开机启动服务，为了实现开机启动 Rsync 服务，我们可以通过 echo 将 rsync –daemon 追加至开机启动文件/etc/rc.local。最后通过设置 iptables 开启 873 端口的入站数据请求。

```
[root@centos6 ~]# echo "tom:pass" > /etc/rsyncd.secrets
[root@centos6 ~]# echo "jerry:111" >> /etc/rsyncd.secrets
[root@centos6 ~]# chmod 600 /etc/rsyncd.secrets
[root@centos6 ~]# echo “welcome to access” >/etc/rsyncd.motd
[root@centos6 ~]# rsync --daemon
[root@centos6 ~]# echo “/usr/bin/rsync --daemon” >> /etc/rc.local
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 873 -j ACCEPT
[root@centos6 ~]# service iptables save
```

2. 客户端同步数据

现在让我们开始同步数据吧，在客户端主机中同样是使用 rsync 命令进行初始化数据传输，使用同样的程序，但客户端主机不需要--daemon 选项。

```
[root@centos6 ~]# yum -y install rsync
[root@centos6 ~]# rsync -vzrtopg --progress  tom@192.168.0.254::common /test
```

rsync 命令的描述和用法如下。

描述：一个快速、多功能的远程（或本地）数据复制工具。

用法：表 4-6 较全面地介绍了 rsync 的语法格式，SRC 表示源路径，DEST 表示目标路径。

表 4-6

本地复制	
rsync [选项] SRC… [DEST]	
通过远程 shell 复制	
下载数据	rsync [选项] [USER@]HOST:SRC... [DEST]
上传数据	rsync [选项] SRC… [USER@]HOST:DEST
通过 rsync 进程复制	
下载数据	rsync [选项] [USER@]HOST::SRC... [DEST] rsync [选项] rsync://[USER@]HOST[:PORT]/SRC... [DEST]
上传数据	rsync [选项] SRC… [USER@]HOST::DEST rsync [选项] SRC... rsync://[USER@]HOST[:PORT]/DEST

选项：-v,--verbose　显示详细信息。

-q,--quiet　静默模式，无错误信息。

-a,--archive　归档模式，主要保留文件属性，等同于-rlptgoD。

-r,--recursive　递归。

-b,--backup　如果目标路径已经存在同名文件，将旧的文件重命名为~filename，可以使用--suffix 指定不同的备份前缀。

--back-dir　将备份文件保存至指定目录。

--suffix　指定备份文件前缀。

-u,--update　如果目标地址中的文件比将要下载的文件新，则不执行同步。也就是说，不会用旧的文件覆盖新的文件。

-l,--links　保留符号链接。

-p,--perms　保留文件权限属性。

-H,--hard-links　保留硬链接。

-A,--acls　保留 ACL 权限。

-X,--xattrs　保留文件附加属性。

-o,--owner　保留文件所有者属性。

-g,--group　保留文件所属组属性。

--devices　保留设备文件。

--specials　保留特殊文件。

-D　等同于--devices –specials。

-t　保留修改时间属性。

-W,--whole-file	不做增量检查，直接复制全部文件。
-e,--rsh=COMMAND	指定远程 shell。
--existing	仅同步目标路径中已经有的文件，不下载源路径下新的文件。
--delete	删除那些仅在目标路径中存在的文件（源路径中不存在）。
-z,--compress	传输过程中对数据进行压缩。
--include=PATTERN	匹配不排除的文件。
--exclude=PATTERN	匹配需要排除的文件。
--progress	显示数据传输的进度信息。
--partial	保留因故障未传输完成的文件。
-P	等同于--progress –partial。
--password-file=FILE	指定密码文件，将密码写入文件，实现非交互式数据同步。
--list-only	仅列出服务器模块列表，需要 rsync 服务器设置 list = true。

实例：

```
[root@centos6 ~]# rsync -t *.c foo:src/
```

将本机当前目录下的以.c 结尾的文件复制至 foo 主机的 src 目录下。

```
[root@centos6 ~]# rsync -avz foo:src/bar /data/tmp
```

从 foo 主机上以递归方式将 src/bar 目录复制至本机/data/tmp 目录。

```
[root@centos6 ~]# rsync -avz foo:src/bar/ /data/tmp
```

从 foo 主机上以递归方式将 src/bar 目录下的所有内容复制至本机/data/tmp 目录，但在/data/tmp 目录下不会创建 bar 目录。

```
[root@centos6 ~]# rsync -avz /src/foo /dest
```

将本机/src/foo 目录复制至/dest 目录。

```
[root@centos6 ~]# rsync -avz tom@192.168.0.254::common /test3
```

使用 tom 账户连接远程 192.168.0.254 主机的 rsync 进程，将 common 模块定义的 path 路径下载至本地 test3 目录。

```
[root@centos6 ~]# rsync -avz 192.168.0.254::common /dest
```

匿名下载 192.168.0.254 服务器的 common 模块至本地/dest 目录。

```
[root@centos6 ~]# rsync --list-only tom@192.168.0.254::
```

显示 192.168 0.254 服务器所有的模块名称，需要服务器端配置 list = true 才会显示。

```
[root@centos6 ~]# echo "pass" > rsync.pass
```

```
[root@centos6 ~]# rsync -avz --delete --password-file=rsync.pass tom@192.168.
0.254::common /dest
```

客户端主机每次连接服务器都需要输入密码显得非常麻烦，为此，我们可以创建密码文件rsync.pass，在该文件中仅包含密码，最后使用 rsync 命令的--password-file 参数指定密码文件，就可以省去每次输入密码的烦恼。

每次同步数据都需要手动输入命令是非常麻烦的一件事，作为一名运维人员，我们需要更智能化的处理机制，这可以考虑使用 Shell 脚本来解决这样的问题。下面的 rsync_bask.sh 脚本文件就是这样一个 Shell 程序模板：利用脚本创建密码文件，使用密码文件进行非交互式自动化数据同步机制。

如果客户端需要定期对 Rsync 服务器的数据进行备份，可以编写 Shell 脚本，通过计划任务实现数据的定期备份工作。下面的脚本为精简版数据备份脚本范例。

```
[root@centos6 ~]# vim rsync_bak.sh
```

```
#!/bin/bash
#This script does backup through rsync.
#Date:2012-10-1
#Version:1.0 beta
#
export PATH=/bin:/usr/bin:/usr/local/bin
SRC=common
DEST=/data
Server=192.168.0.254
User=tom
#password file must not be other-accessible.
Passfile=/root/rsync.pass
#If the DEST directory not found, then create one.
[ ! -d $DEST ] && mkdir $DEST
[ ! -e $Passfile ] && exit 2
rsync -az --delete --password-file=$Passfile ${User}@${Server}::$SRC $DEST/$(date
+%Y%m%d)
```

4.6.3　Rsync+Inotify 实现文件自动同步

如果仅使用 Rsync 进行数据同步，只能满足企业对数据实时性要求不高的环境，即使使用计划任务也仅可以实现定期的数据同步。而且使用 Rsync 在进行数据同步前需要对所有的文件进行对比，然后进行差异数据同步，然而我们的数据可能只是 1TB 数据中的 1KB 数据发生了改变，在不知道什么时候会发生数据改变的情况下，为了同步 1KB 的数据，我们需要不停地进行 Rsync 连接，对比客户端与服务器之间的数据差异，这样的机制在当前大数据时代背景下是

低效的。往往数据的变化随时都有可能发生，如果多台主机之间要求当数据发生变化后进行实时同步，就需要结合 Inotify（inode notify）工具。目前 Inotify 已经被集成到 Linux 内核[2]中，Inotify 为用户态应用程序提供了文件系统事件通告机制，比如，当发生文件的访问、修改以及删除等事件时，可以立刻通告给用户态应用程序，通过 Inotify 可以实时了解文件系统发生的所有变化。表 4-7 是 Inotify 可以监控的部分常见文件系统事件。

表 4-7

事件名称	描述
IN_ACCESS	文件访问事件
IN_MODIFY	文件修改事件
IN_ATTRIB	文件属性修改事件
IN_OPEN	文件打开事件
IN_CLOSE_WRITE	可写文件被关闭事件
IN_CLOSE_NOWRITE	不可写文件被关闭事件
IN_MOVED_FROM IN_MOVED_TO	文件移动或重命名事件
IN_DELETE	文件或目录删除事件
IN_CREATE	文件或目录创建事件
IN_DELETE_SELF	自删除事件

利用 Inotify 的这种事件通知机制，用户态的应用程序就可以实时监控文件系统的变化，然而 Inotify 仅是内核提供的一种系统功能，用户如果需要使用该功能，还需要安装用户态软件，在 CentOS 6.3 系统中可以使用 inotify-tools 来实现文件系统的实时监控，该软件可以从 https://github.com/rvoicilas/inotify-tools 中下载。

1. 源码安装软件

我们可以去官网下载源码包安装部署该软件，下载源码文件解压后，可以查看对应的 INSTALL 安装说明文档，根据新版本的文档说明，需要使用 autogen.sh 脚本读取 configure.ac 文件来创建 configure 安装脚本。而此步骤需要依赖于 automake 以及 libtool 软件包，如果采用的是 yum 方式安装这两个依赖软件包，系统会将编译 inotify-tool 时所需要的 gcc 软件也一起安装。

```
[root@centos6 ~]# yum -y install automake libtool
[root@centos6 inotify-tools-master]# ./configure
[root@centos6 inotify-tools-master]# make && make install
```

[2] Linux 从 2.6.13 版本的内核以后全部集成了 Inotify 功能。

2. 监控数据

inotify-tool 提供了两个应用程序，分别为 inotifywait 与 inotifywatch。其中，inotifywait 命令的描述和用法如下。

描述：使用 inotify 机制等待文件系统事件，该命令非常适合实时监控文件系统的变化。

用法：inotifywait [-hcmrq] [-e <event>] [-t <seconds>] [--format <fmt>] [--timefmt <fmt>] <file>...

选项：-h,--help　　显示帮助信息。

@<file>　　指定监控路径中的例外文件，应用于不需要监控的文件。

--fromfile <file>　　从文件中读取需要监控与例外的文件名称，每行一个文件名，如果文件名称以@开头，则表示例外文件。

-m,--monitor　　接收到事件后不退出，默认程序在接收一个事件信息后会退出。

-d,--daemon　　与--monitor 类似，但程序会进入后台执行，需要通过--outfile 指定事件信息的输出文件。

-o,--outfile <file>　　将事件信息输出至文件，默认输出至标准输出。

-s,--syslog　　将错误信息输出至 syslog 系统日志，默认输出至标准错误输出。

-r,--recursive　　递归监控。

-q,--quiet　　静默模式，不输出信息。

--exclude <pattern>　　使用正则表达式匹配例外文件，区分大小写。

--excludei <pattern>　　使用正则表达式匹配例外文件，不区分大小写。

-t <seconds>,--timeout <seconds>　　在指定的时间没有发生事件，则退出程序。

-e <event>,--event <event>　　仅监控指定的事件。

-c,--csv　　使用 CSV 格式输出。

--timefmt <fmt>　　设置时间格式，即--format 指定的%T 格式。

--format <fmt>　　指定输出信息格式，具体格式参考 man 手册。

实例 1：创建测试目录/test 和测试用文件/etc/foo，运行 inotifywait 命令监控/test 目录，然后开启一个终端窗口运行命令 cat /test/foo，验证当发生查看文件事件时，是否会有事件通知。

```
[root@centos6 ~]# mkdir -p /test; echo "hello">/test/foo
[root@centos6 ~]# inotifywait /test/
Setting up watches.
Watches established.
/test/ OPEN foo
```

实例 2：编写一个脚本实时监控 NetworkManager 相关日志信息，内容如下。

```
[root@centos6 ~]# cat monitor.sh
#!/bin/bash
while inotifywait -e modify /var/log/messages
do
if tail -n1 /var/log/messages |grep NetworkManager
then
    echo Love
fi
done
```

3. Rsync 与 Inotify 双剑合璧

单一的 Rsync 工具仅可以进行数据同步，单一的 Inotify 仅可以实现实时文件监控，而两者的结合将能满足企业对数据中心实时数据同步的要求。接下来我们用案例说明两者结合部署的流程，图 4-16 为案例结构图。在这个案例中，ABC 公司需要部署一套 Web 服务，然而随着用户访问量的增加，单台服务器已经满足不了大量的并发访问。因此，ABC 公司决定使用集群技术，整合多台服务器处理能力实现负载均衡，从而满足不断增加的并发访问量。

由于 Web 服务器所提供的网站数据需要保持一致，但当服务器越来越多时，公司发现在这些主机之间同步那些随时可能发生改变的网站数据简直就是一场噩梦。我们的解决方案是在后端建立一个数据发布服务器，该服务器作为 Rsync 客户端，通过 Inotify 机制实时监控网站数据，当数据发生变化后调用 Rsync 命令上传数据至多个 Rsync 服务器，这里的 Rsync 服务器就是提供 Web 服务的 Web 服务器。

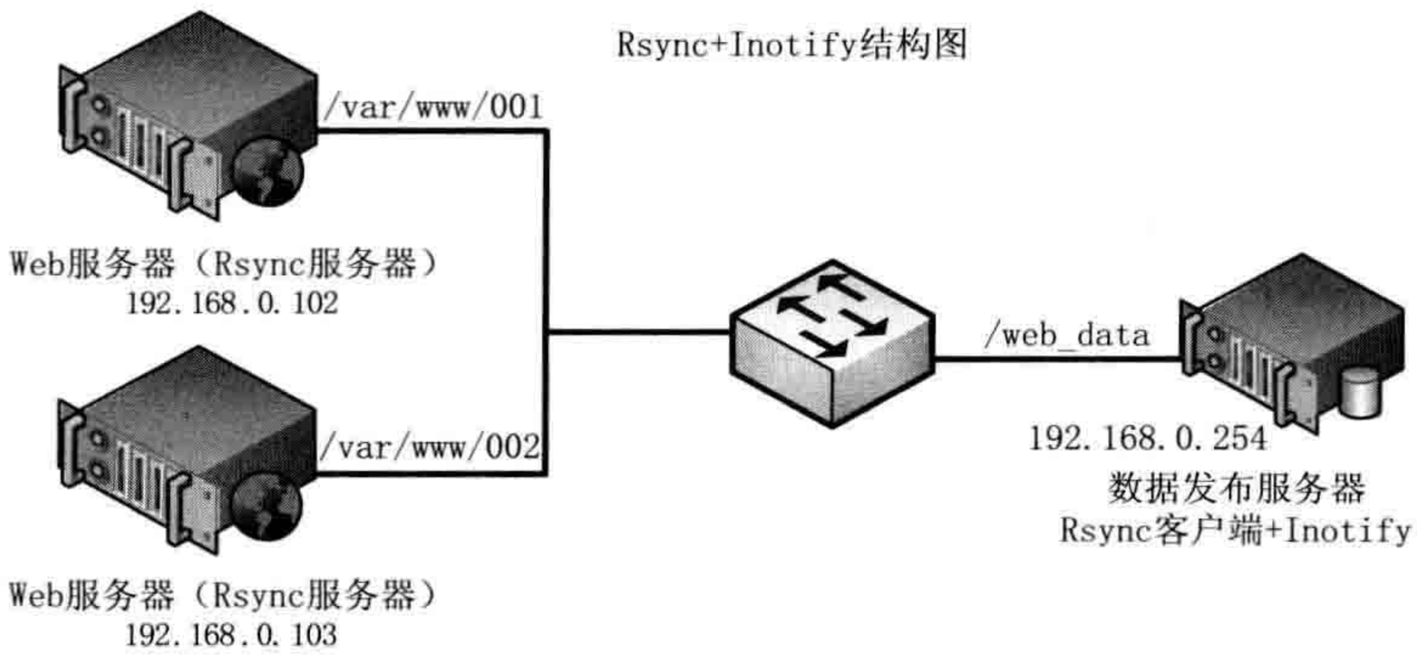


图 4-16

首先需要在多台 Web 服务器上部署 Rsync 服务器，这些 Rsync 服务器要能够提供客户端上传功能，以实现客户端主机将数据推送至 Rsync 服务器，实现数据的实时同步功能。如图 4-16 所示，最终我们只要在 192.168.0.254 主机上修改数据，就可以实时推送数据至两台 Web 服务器，

Web 服务器（主机 192.168.0.102）配置如下（案例中所涉及的账户和密码信息可根据自己的实际需求进行修改）：

```
[root@centos6 ~]# yum -y install rsync
[root@centos6 ~]# mkdir -p /var/www/001
[root@centos6 ~]# chmod 660 /var/www/001
[root@centos6 ~]# chown nobody.nobody /var/www/001
 [root@centos6 ~]# vim /etc/rsync.conf
```

```
#/etc/rsyncd.conf
transfer logging = yes
log file = /var/log/rsyncd.log
pid file = /var/run/rsyncd.pid
lock file = /var/run/rsync.lock
uid = nobody
gid = nobody
use chroot = no
ignore errors
read only = no
[web1]
comment = Web content
path = /var/www/001
auth users = tom
secrets file = /etc/rsyncd.secrets
hosts allow=192.168.0.254
hosts deny=*
list = false
```

```
[root@centos6 ~]# echo "tom:pass" > /etc/rsyncd.secrets
[root@centos6 ~]# chmod 600 /etc/rsyncd.secrets
[root@centos6 ~]# rsync --daemon
[root@centos6 ~]# echo "rsync --daemon" >> /etc/rc.local
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 873 -j ACCEPT
[root@centos6 ~]# service iptables save
```

Web 服务器（主机 192.168.0.103）配置如下：

```
[root@centos6 ~]# yum -y install rsync
[root@centos6 ~]# mkdir -p /var/www/002
[root@centos6 ~]# chmod 660 /var/www/002
[root@centos6 ~]# chown nobody.nobody /var/www/002
[root@centos6 ~]# vim /etc/rsync.conf
```

```
#/etc/rsyncd.conf
transfer logging = yes
log file = /var/log/rsyncd.log
pid file = /var/run/rsyncd.pid
```

```
lock file = /var/run/rsync.lock
uid = nobody
gid = nobody
use chroot = no
ignore errors
read only = no
[web2]
comment = Web content
path = /var/www/002
auth users = tom
secrets file = /etc/rsyncd.secrets
hosts allow=192.168.0.254
hosts deny=*
list = false
```

```
[root@centos6 ~]# echo "tom:pass" > /etc/rsyncd.secrets
[root@centos6 ~]# chmod 600 /etc/rsyncd.secrets
[root@centos6 ~]# rsync --daemon
[root@centos6 ~]# echo "rsync --daemon" >> /etc/rc.local
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 873 -j ACCEPT
[root@centos6 ~]# service iptables save
```

在数据发布服务器（192.168.0.254）上需要下载 inotify-tool 软件包，并编写监控脚本，这里的脚本名称为 notify_rsync.sh（该脚本具体的内容见下文方框中）。当监控到数据发生改变时，自动进行数据同步操作，将数据推送至 Web 服务器。

```
[root@centos6 ~]# yum -y install rsync
[root@centos6 ~]# yum -y install automake libtool
[root@centos6 inotify-tools-master]# ./configure
[root@centos6 inotify-tools-master]# make && make install
[root@centos6 ~]# echo "pass" >/root/rsync.pass
[root@centos6 ~]# chmod 600 /root/rsync.pass
[root@centos6 ~]# vim notify_rsync.sh
```

```
#!/bin/bash
#This Rsync script based on inotify.
#Date:2012-10-1
#Version:1.0 beta
#
export PATH=/bin:/usr/bin:/usr/local/bin
SRC=/web_data/
DEST1=web1
DEST2=web2
Client1=192.168.0.102
Client2=192.168.0.103
```

```
    User=tom
    #password file must not be other-accessible.
    Passfile=/root/rsync.pass
    [ ! -e $Passfile ] && exit 2
    #Wait for change
    inotifywait -mrq --timefmt '%y-%m-%d %H:%M' --format '%T %w%f %e' \
    --event modify,create,move,delete,attrib $SRC|while read line
    do
    echo "$line" > /var/log/inotify_web 2>&1
    /usr/bin/rsync -avz --delete --progress --password-file=$Passfile $SRC \
${User}@$Client1::$DEST1 >>/var/log/sync_web1 2>&1
    /usr/bin/rsync -avz --delete --progress --password-file=$Passfile $SRC \
${User}@$Client2::$DEST2 >>/var/log/sync_web2 2>&1
    done &
```

```
[root@centos6 ~]# chmod a+x notify_rsync.sh
[root@centos6 ~]# /root/notify_rsync.sh
[root@centos6 ~]# echo "/root/notify_rsync.sh" /etc/rc.local
```

4.7　DHCP 服务器

DHCP 是 Dynamic Host Configuration Protocol（动态主机配置协议）的简写形式，使用 DHCP 协议可以为客户端主机自动分配 TCP/IP 参数信息，如 IP 地址、子网掩码、网关、DNS 等信息，服务器可以选择固定分配特定的参数信息给指定的一台主机，也可以设置多台主机分享这些参数信息，所有的客户端竞争[3]获得 TCP/IP 参数信息。客户端主机通过 UDP 广播的形式发送请求数据包至本地网络中的所有设备，仅当 DHCP 服务器在收到请求后根据自身的配置将 TCP/IP 信息租赁给客户端（租期是有限的），当租期到了以后客户端可以再次向服务器发出请求实现续约。在本书第 1 章大规模部署操作系统的实施过程中就需要使用 DHCP 服务为大量主机自动动态分配网络参数信息。在企业办公环境中使用 DHCP 服务可以帮助员工实现移动办公，不管是台式机、笔记本还是平板电脑，只要接入网络就可以自动获得网络参数。

4.7.1　安装软件

dhcp 软件提供了 DHCP 协议的全部实现功能，主配置文件为/etc/dhcp/dhcpd.conf，默认该

[3] 多个客户端发送请求给服务器时，服务器将使用先到先得的机制进行资源分配。

文件几乎为空，但 CentOS 6.3 系统所提供的 RPM 软件包提供了一个配置文件模板，如果需要，可以使用/usr/share/doc/dhcp-4.1.1/dhcpd.conf.sample 作为 dhcp 主配置文件的参考模板，/var/lib/dhcpd/dhcpd.leases 文件中记录了所有服务器已经分配出去的 IP 信息以及相关租期信息。

```
[root@centos6 ~]# yum -y install dhcp
```

4.7.2 配置文件解析

由于默认的配置文件没有参数模块，我们可以使用 dhcpd.conf.sample 文件作为模板修改服务器配置。dhcp 配置文件分为全局设置、子网定义、主机定义，其中子网与主机定义项可以有多个。DHCP 服务器最主要的功能是为本地网络提供网络参数数据，下面我们分析一下 dhcpd.conf.sample 模板文件的参数含义（注意，所有的设置项最后都有分号结束符）。

```
#dhcpd.conf
#定义全局参数：默认搜索域
option domain-name "example.org";
#定义全局参数：域名服务器，多个 DNS 服务器使用逗号隔开
option domain-name-server ns1.example.org, ns2.example.org;
#定义全局参数：默认租期，单位为秒
default-lease-time 600;
#定义全局参数：最大租期，单位为秒
max-lease-time 7200;
#定义 10.152.187.0/255.255.255.0 子网，但没有为该子网设置任何参数，花括号内为空
subnet 10.152.187.0 netmask 255.255.255.0{
}
#定义10.254.239.0/255.255.255.224子网,IP地址池为10.254.239.10至19.254.239.20,
#默认网关为 rtr-239-0-1.example.org, rtr-239-0-2.example.org
subnet 10.254.239.0 netmask 255.255.255.224 {
range 10.254.239.10 10.254.239.20;
option routers rtr-239-0-1.example.org, rtr-239-0-2.example.org;
}
#定义 10.5.5.0/255.255.255.224 子网，IP 地址池为 10.5.5.26 至 10.5.5.30,
#DNS 服务器为 ns1.internal.example.org,默认网关为 10.5.5.1,广播地址为 10.5.5.31,
#默认租期为 600s，最大租期为 7200s，子网定义中的参数与全局参数有冲突时以子网定义参数
#为准
subnet 10.5.5.0 netmask 255.255.255.224 {
  range 10.5.5.26 10.5.5.30;
  option domain-name-servers ns1.internal.example.org;
  option domain-name "internal.example.org";
```

```
  option routers 10.5.5.1;
  option broadcast-address 10.5.5.31;
  default-lease-time 600;
  max-lease-time 7200;
}
#主机定义项：定义主机 fantasia，通过该主机的 MAC 地址，绑定固定 IP 给该客户端，
#以后该主机每次向服务器请求 IP 后，获得的将是 fixed-address 指定的固定 IP 地址。
#当定义多个主机时，host 后的主机名称要求是唯一的
host fantasia {
  hardware 233ethernet 08:00:07:26:c0:a5;
  fixed-address fantasia.fugue.com;
}
```

4.7.3　DHCP 应用案例

近期，ABC 网络公司计划重新规划网络环境，由于之前使用的环境是手动配置网络参数，然而公司内部有些部门员工对计算机专业知识完全没有概念，经常出现的一些简单的网络故障消耗了网络维护人员的大量精力与时间，如 IP 地址冲突、网关设置不正确等故障。鉴于这些问题频频出现，公司决定在内部部署 DHCP 服务，让所有的员工实现零配置即可接入网络，但公司内部的若干台文件服务器以及打印服务器需要使用固定 IP。公司拓扑结构如图 4-17 所示。

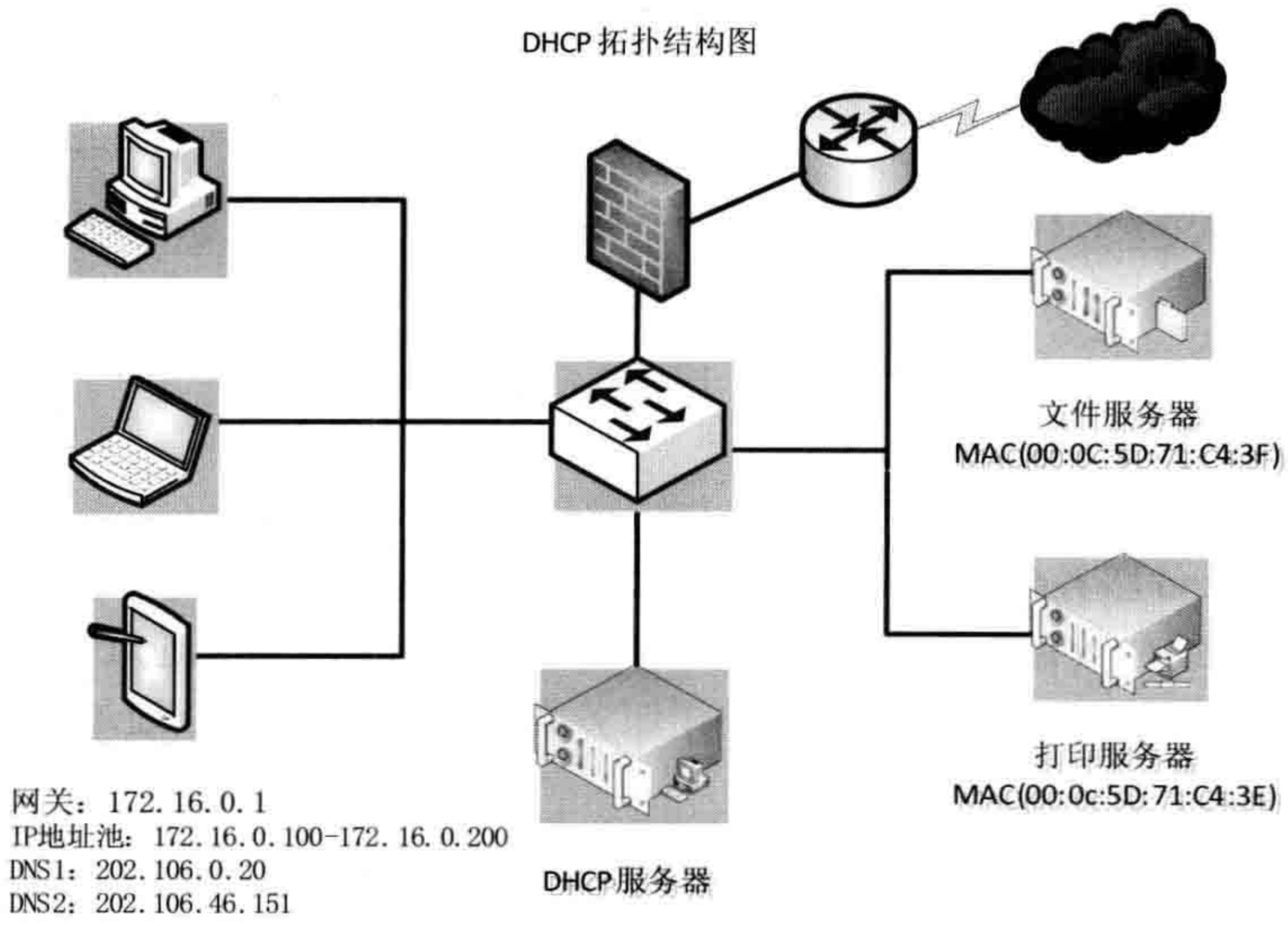


图 4-17

完成案例中的业务部署效果非常简单，首先在 DHCP 服务器主机上安装 dhcp 软件包。其次，根据配置文件模板创建一份符合实际工作环境的配置文件，本例中的配置文件效果如下。

```
[root@centos6 ~]# yum -y install dhcp
[root@centos6 ~]# vim /etc/dhcp/dhcpd.conf
```

```
#dhcpd.conf
default-lease-time 600;
max-lease-time 7200;
subnet 172.16.0.0 netmask 255.255.0.0 {
        range 172.16.0.100 172.16.0.200;
        option domain-name-servers 202.106.0.20, 202.106.46.151;
        option domain-name "abc.com";
        option routers 172.16.0.1;
        option broadcast-address 172.16.255.255;
}
host fileserver {
        hardware ethernet 00:0C:5D:71:C4:3F;
        fixed-address 172.16.0.10;
}
host printserver {
        hardware ethernet 00:0C:5D:71:C4:3E;
        fixed-address 172.16.0.20;
}
```

```
[root@centos6 ~]# service dhcpd restart
[root@centos6 ~]# chkconfig dhcpd on
```

至此，DHCP 服务已经配置完成，公司的所有客户主机只要设置动态获取网络参数，就可以实现零配置接入网络。Linux 客户端主机如何设置通过 DHCP 自动获取 IP 参数，可以参考本书 2.9 节的内容。

4.7.4 常见问题分析

默认状态下，dhcpd 服务会将日志保存在/var/log/messages 文件中，如果遇到服务器故障问题，可以检查该文件。网络参数租期文件为/var/lib/dhcpd/dhcpd.leases，可以通过检查该文件查看服务器已经分配的资源及相关租期信息。下面列出了服务器常见的问题，可以通过日志查看故障信息。

1. 报错：/etc/dhcp/dhcpd.conf line 4: semicolon expected.

该提示信息说明主配置文件第 4 行左右的位置缺少分号，DHCP 主配置文件语法格式要求

所有的选项最后都要有分号结束符。

2. **报错**：Not configured to listen on any interfaces!

该提示信息说明没有检查到任何有效的网络接口配置，一般是 DHCP 服务器本地的网络参数没有配置导致的错误。

3. **报错**：If this is not what you want, please write a subnet declaration in your dhcpd.conf file for the network segment to which interface eth0 is attached.

该提示信息说明主配置文件中的子网定义错误，一般是在配置文件中的子网定义没有与 DHCP 服务器处于相同的网络，比如服务器本地 IP 为 192.168.0.1，而配置文件中仅定义了一个子网是 172.16.0.0/16 网络，此时会出现该报错信息。主配置文件可以定义为多个子网分配网络参数，但至少要有一个与服务器本地是同网络的子网定义。

4. **报错**：DHCPDISCOVER from 00:0c:29:00:5f:17 via eth0: network 172.16.0.0/16: no free leases.

该提示信息说明 MAC 地址为 00:0c:29:00:5f:17 的主机向 DHCP 服务器申请网络参数资源，但服务器地址池中的资源已经全部被分配出去，没有剩余的资源可以分配。

5. **报错**：/etc/dhcp/dhcpd.conf line 18: host fileserver: already exists.

该提示信息说明主配置文件第 18 行的位置定义的 host fileserver 已经存在。在 DHCP 配置文件中，host 定义的主机名称要求是唯一的、不能有重复的主机名称。

4.8　DNS 域名服务器

4.8.1　DNS 简介

DNS 是 Domain Name System（域名系统）的简称，DNS 可以为计算机、服务以及接入互联网或局域网的任何资源提供分层的名称解析功能。DNS 提供了很多功能，其中最主要的功能就是进行域名与 IP 地址之间的解析。在互联网中标记唯一一台计算机使用的是 IP 地址，通过合法的 IP 地址，我们可以与全世界任何一台主机进行通信。然而在当今计算机如此普及的情况

下，以人类现有的智慧与记忆力很难将大量的 IP 地址背诵下来，这时使用域名系统就可以将难以记忆的数字 IP 地址与容易记忆的域名建立映射关系，用户通过输入域名，计算机会寻找指定的 DNS 服务器，请求服务器帮助解析该域名对应的 IP 地址，成功解析后，将获得该域名对应的真实 IP 地址，然后使用该 IP 地址与对方通信。

域名是分级的，一般分为：主机名.三级域名.二级域名.顶级域名.。注意，最后有一个点代表的是根域，是所有域名的起点。域名有点像美国人的姓名一样，姓在后，名在前，而计算机域名中最后的点则是根，其次是根下的顶级域名，然后是二级域名等。图 4-18 为典型的域名树状结构图。例如，斯坦福大学的域名为 www.mit.edu.，代表的是根域下有 edu 子域，edu 子域下面有 mit 子域，mit 子域下有主机 www。注意，一般情况下，我们通过浏览器输入网址域名时，最后一个根域（.）是不需要输入的。一般顶级域代表国家或组织形式，如 cn 代表中国；edu 代表教育机构；com 代表商业公司等。二级域名代表组织或公司名称，三级域名代表组织或公司内部的主机名称。最后通过完全合格的域名（FQDN）可以定位全球唯一的一台主机。这种分层管理机制的优势在于根域服务器不需要管理全世界所有的域名信息，它只需要管理顶级域信息即可，而顶级域服务器只需要管理二级域信息即可。依此类推，实现分层管理，这类似于国家的行政管理机制。

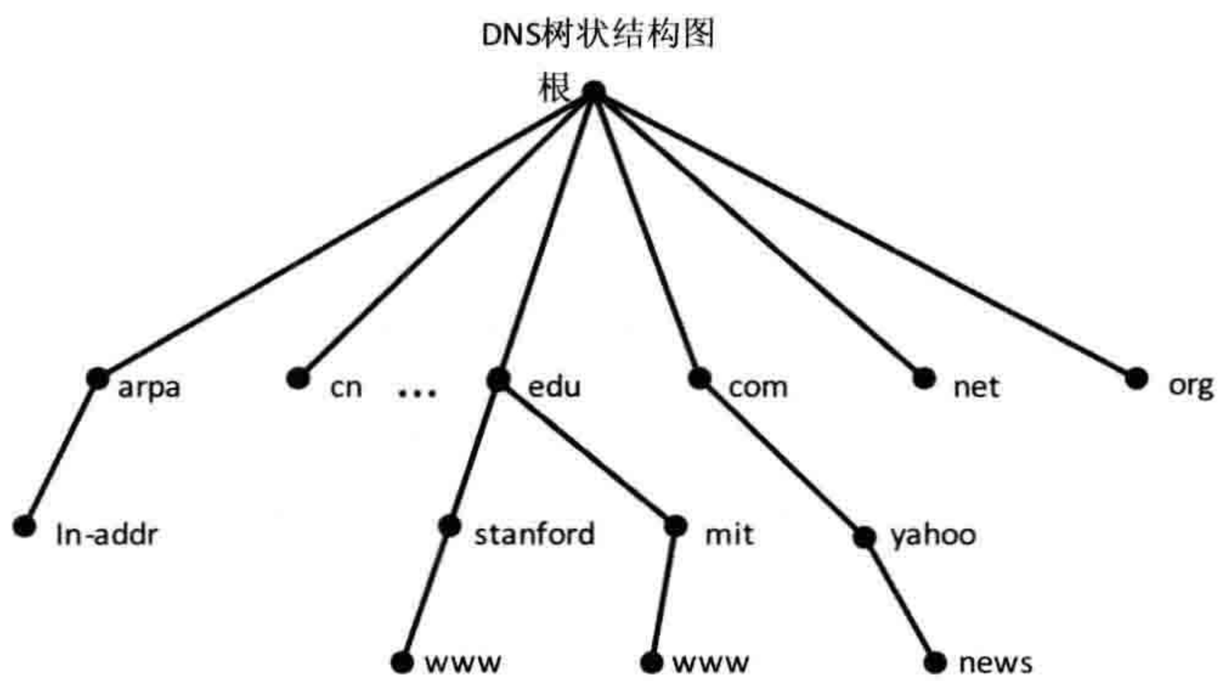


图 4-18

域名查询分为递归查询与迭代查询，如图 4-19 所示。举例说明，如果客户端准备访问斯坦福大学的网站，客户端首先会检查本地缓存中是否有之前的查询记录，如果有，直接读取结果即可，如果没有相关缓存记录，则向本地 DNS 服务器发送查询请求，也就是所谓的递归查询，本地 DNS 服务器如果有答案，就会将答案直接返回给客户端，但本地 DNS 服务器没有正确的答案时，它就需要向根服务器查询，但不是询问 www.stanford.com 对应的 IP 是多少，根服务器仅管理顶级域名，而且所有的顶级域名都属于根的管理范畴，所以此时本地 DNS 服务器向根查询的结果是：根服务器会将它所管理的 com 域对应的 IP 提供给本地 DNS 服务器，本地 DNS

服务器得到 com 域服务器的 IP 后，会再向 com 查询，然而 com 也没有最终的答案，com 域服务器会将它所管理的 stanford 域服务器对应的 IP 提供给本地 DNS 服务器，最后本地 DNS 服务器再向 stanford 域服务器查询，询问该域下主机名称为 www 的计算机对应的 IP 地址，由于 www 主机确实直接属于 stanford 域的管理范畴，所以 stanford 会将最终的正确答案返回给本地 DNS 服务器，最后本地 DNS 服务器将得到的结果返回给客户端，同时本地 DNS 服务器还会将结果缓存起来，当下次再有相同的查询请求时，本地 DNS 服务器就可以直接从缓存中找到结果返回给客户端。

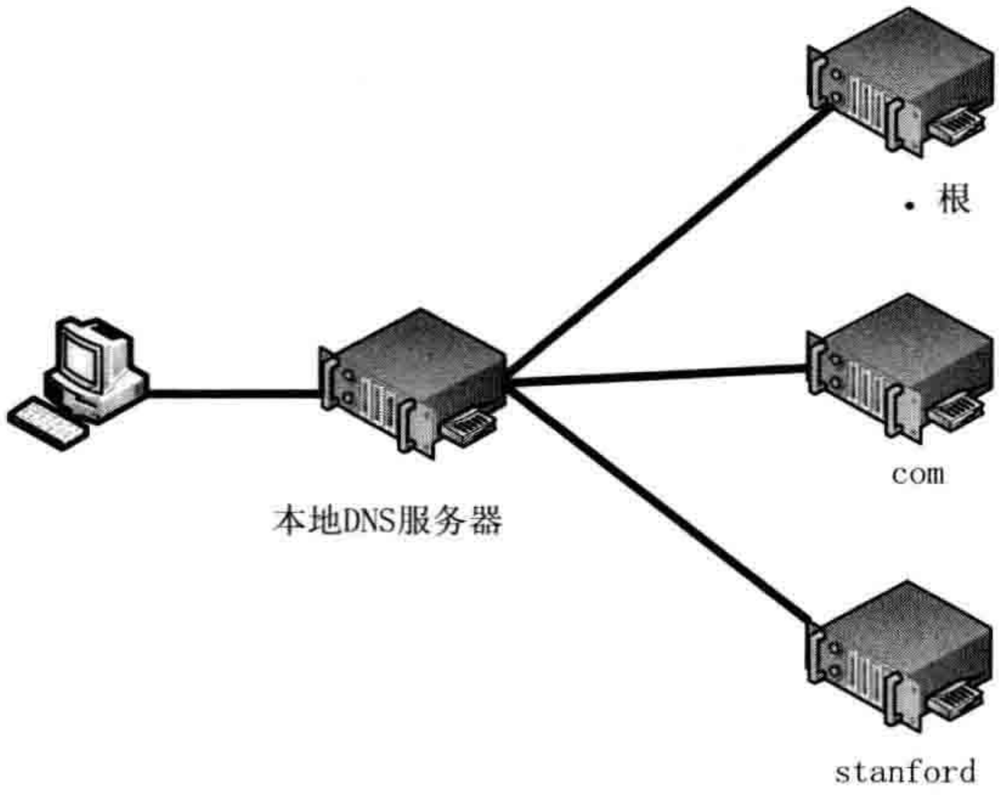


图 4-19

4.8.2　安装 DNS 软件

提供 DNS 服务的软件有很多，而伯克利加州大学研发的 BIND（Berkeley Internet Name Domain）是目前应用最广泛的 DNS 服务系统。该项目软件除了 BIND 主程序外，在 Linux 平台下还提供了 chroot 与 utils 软件包，bind-chroot 软件包的主要功能是使 BIND 软件可以运行在 chroot 模式下，这样 BIND 运行在相对路径的根路径，而不是 Linux 系统真正的根路径，即使有人对 BIND 软件进行攻击与破坏，影响的也仅仅是操作系统中的一个子目录，不会影响整个操作系统平台，以此来提升系统安全性。通过 yum 安装 bind-chroot 后，对 bind 而言，/var/named/chroot/目录就是根路径，所有 BIND 软件的配置文件都在根下的某某路径下。bind-utils 软件包提供了一些 DNS 查询工具，如：dig、host、nslookup 等。

```
[root@centos6 ~]# yum -y install bind
[root@centos6 ~]# yum -y install bind-chroot
[root@centos6 ~]# yum -y install bind-utils
```

4.8.3 配置文件解析

BIND 配置文件主要分为主配置文件与域数据记录文件，主配置文件包括很多使用花括号引起来的定义语句，在定义语句中可以设置多个选项，主配置文件的核心功能就是定义域，以及告知计算机到哪里可以找到相应域的数据记录文件。而域数据记录文件则存储具体的域名与IP 之间的解析记录，DNS 通过读取域数据记录文件来解答客户端的查询请求。

主配置文件一般为/etc/named.conf 文件，但当我们安装了 bind-chroot 软件包后，在后面的内容中提到的一些路径默认指的都是一个虚拟路径，都是相对于虚拟根路径而言的，虚拟根路径默认为/var/named/chroot/目录。如果主配置文件是/etc/named.conf，那么最终主配置文件在本机的绝对路径应该为/var/named/chroot/etc/named.conf，修改主配置文件时一定要注意，如果语法格式有问题，则 DNS 服务是无法正常启动的，一个典型的主配置文件语法格式如下。

```
statement_name {
option1;
option2;
… …;
};
```

下面介绍/etc/named.conf 文件中常用的定义语句。

acl（Access Control List）语句允许我们预定义一组主机，从而控制是允许还是拒绝他人访问域名服务器，BIND 预定义了一些主机访问控制列表，其中，any 可以匹配任意 IP 地址，localhost 可以匹配本地系统上所有的 IP 地址，localnets 匹配本地系统所连接的任意网络，none 将不匹配任何 IP 地址。下面通过实例自定义两个访问控制列表，一个为黑名单，一个为白名单。

```
acl black {
172.16.0.0/16;
192.168.0.12;
192.168.0.18;
};
acl white {
10.0.0.0/8;
192.168.0.0/24;
}
options {
allow-query { white; };
blackhole { black; };
};
```

options 语句用来定义全局配置选项，在全局配置中至少需要定义一个工作路径，默认的工作路径为/var/named/，具体的常见参数见表 4-8。

表 4-8

选项	描述
directory	设置域名服务的工作目录，默认为/var/named
dump-file	运行 rndc dumpdb 备份缓存资料后保存的文件路径与名称
statistics-file	运行 rndc stats 后，统计信息的保存路径与名称
lisen-on port	指定监听的 IPv4 网络接口
allow-query	指定哪些主机可以查询服务器的权威解析记录
allow-query-cache	指定哪些主机可以通过服务器查询非权威解析数据，如递归查询数据
blackhole	设置拒绝哪些主机的查询请求
recursion	是否允许递归查询
forwards	指定一个 IP 地址，所有对本服务器的查询将转发到该 IP 进行解析
max-cache-size	设置缓存文件的最大容量

zone 语句用来定义域及相关选项，定义域也就意味着你希望维护自己公司的域名解析。该语句的重点选项有 type 与 file，表 4-9 为 zone 语句内常用的选项及描述。

表 4-9

选项	描述
type	设置域类型，类型可以是 hint：当本地找不到相关解析后，可以查询根域名服务器 master：定义权威域名服务器 slave：定义辅助域名服务器 forward：定义转发域名服务器
file	定义域数据文件，文件保存在 directory 所定义的目录下
notify	当域数据资料更新后是否主动通知其他域名服务器
masters	定义主域名服务器 IP 地址，当 type 设置为 slave 后此选项才有效
allow-update	允许哪些主机动态更新域数据信息
allow-transfer	哪些从服务器可以从主服务器下载数据文件

下面是一个简单的 zone 语句实例，example.com 是定义的域，type 定义本机为 example.com 域的主域名服务器，该域的解析文件保存在 example.com.zone 文件中，该文件被保存在/var/named/目录下。当定义反向解析域时，需要将网络地址段反过来输入，并以固定的.in-addr.arpa 格式结尾。通过定义类型为 hint 的根域，这样当客户端查询不属于自己维护的域名时，可以寻找根服务器进行迭代查询，最终返回正确的结果，全球的根域名服务器信息保存在 named.ca 文件中，该文件是在安装完 BIND 软件后自动生成的模板文件，我们可以在/usr/share/doc/bind-9.8.2/sample/var/named/目录下找到它。

```
Zone "." IN {
    type hint;
    file "named.ca";
};
zone "example.com" IN {
    type master;
    file "example.com.zone";
};
zone "0.168.192.in-addr.arpa" IN {
       type master;
       file "192.168.0.zone";
};
```

在 BIND 软件的主配置文件中，如果定义了 zone 语句，还需要额外创建域数据文件，默认域数据文件被存储在/var/named 目录下，文件名称由 zone 语句中的 file 选项设定。数据文件分为正向解析数据文件与反向解析数据文件，正向解析文件保存了域名到 IP 地址的映射记录，反向解析文件保存了 IP 地址到域名的映射记录，表 4-10 列出了常用的记录类型及描述。

表 4-10

记录类型	描述
SOA 记录	域权威记录，说明本机服务器为该域的管理服务器
NS 记录	域名服务器记录
A 记录	正向解析记录，域名到 IP 地址的映射
PTR 记录	反向解析记录，IP 地址到域名的映射
CNAME 记录	别名记录，为主机添加别名
MX 记录	邮件记录，指定域内的邮件服务器，需要指定优先级

下面通过一个简短的正向解析的域数据文件，说明各种记录的语法格式，配置文件中 TTL 的值为 DNS 记录的缓存时间，这个值是其他域名服务器将数据存放在缓存中的时间，1D 代表一天。SOA 记录后面的 root.example.ccom.代表域的权威服务器，jacob.google.com 是管理员的邮箱，由于@在数据文件中代表特殊含义，这里使用.来代表@符号，所以实际的邮箱应该是 jacob@google.com，SOA 记录可以跨行输入，跨越多行时使用括号引用。NS 记录代表域名服务器记录，如果公司有多个域名服务器，可以添加多条 NS 记录，但每个 NS 记录在下面都需要有对应的 A 记录。A 记录为正向解析记录，格式为在域名后面输入相应的 IP 地址，IN 代表 Internet，A 代表正向解析记录。在使用 MX 记录指定邮件服务器时，我们给 mail.example.com.服务器设置的优先级为 10，最后 CNAME 记录为别名，可以使用 web.example.com 来代表 www.example.com。具体的配置文件如下。

```
$TTL 1D
@    IN  SOA     root.example.com. jacob.google.com. (
                     0
                     1D
                     1H
                     1W
                     3H)
                IN  NS   root.example.com.
                IN  MX   10  mail.example.com.
root.example.com.   IN  A  172.16.0.254
www.example.com. IN  A  172.16.0.200
                 IN  A  172.16.0.201
ftp.example.com.    IN  A  172.16.0.100
mail             IN  A  172.16.0.25
web              IN  CNAME   www
```

关于配置文件的简写，在 BIND 主配置文件中，zone 语句后定义的域对于数据文件的简写来说很重要，如果区数据文件中不以“.”结尾的名称后会被自动加上该域名称，如 zone 语句定义了 exaple.com，而数据文件中有一个 mail 没有以“.”结尾，那么系统会自动追加 example.com 到 mail 后，有时候管理员将 A 记录错误地写成 www.example.com 这种形式，由于没有以“.”结尾，实际系统识别的应该是 www.example.com.example.com.。此外，在数据文件中使用@符号也同样代表 zone 所定义的域名称。最后，如果在数据文件中某条记录名称为空格或制表符，那么 BIND 系统会直接使用上一条记录的名称，当我们为同一个域名设置多个 A 记录，则客户端请求该域名解析时，服务器会以轮询的方式将结果返回给客户端，可以从一定程度上实现载均衡的功能。

4.8.4　部署主域名服务器

随着 ABC 公司业务的扩张，计算机越来越多的情况下，让员工记忆公司内部服务器所有的 IP 地址简直就是一场噩梦，因此，公司决定采用 DNS 解决方案，这样所有的员工仅需要记忆域名就可以访问各种服务。案例中使用的域为 abc.com 域，这个是正向解析的域，网络采用的是私有网络地址为：172.16.0.0/16。本例仅对公司主要的服务器编写解析记录，如果读者需要更多的记录，可以按照这些模板添加其他主机的记录信息，在公司内部服务器列表见表 4-11。

表 4-11

服务器 IP	服务器名称	功能描述
172.16.0.254	dns1.abc.com	主域名服务器
172.16.0.253	dns2.abc.com	从域名服务器
172.16.0.100	fileserver.abc.com	文件服务器
172.16.0.101	printserver.abc.com	打印机服务器
172.16.0.200	www.abc.com	网站服务器
172.16.0.201	www.abc.com	网站服务器
172.16.0.25	mail.abc.com	邮件服务器
172.16.0.22	ntp.abc.com	时间服务器

1. 安装软件

如前所述，在部署 DNS 服务器时需要安装 bind、bind-chroot 以及 bind-utils 软件包。

```
[root@centos6 ~]# yum -y install bind
[root@centos6 ~]# yum -y install bind-chroot
[root@centos6 ~]# yum -y install bind-utils
```

2. 修改主配置文件

默认 chroot 之后的虚拟根目录/var/named/chroot/etc 下面并没有现成的配置文件，需要在/usr/share/doc/bind-9.3.2/sample/etc 目录下找到配置文件模板，复制该文件至/var/named/chroot/etc 目录下，并根据自己的实际情况修改该配置文件。本书案例均以表 4-11 为模型进行配置。

```
[root@centos6 etc]# cd /usr/share/doc/bind-9.8.2/sample/etc/
[root@centos6 etc]# cp named.conf /var/named/chroot/etc/
[root@centos6 etc]# chown root.named /var/named/chroot/etc/named.conf
[root@centos6 etc]# vim /var/named/chroot/etc/named.conf
```

```
options
{
directory       "/var/named";       // "Working" directory
dump-file       "data/cache_dump.db";
        statistics-file  "data/named_stats.txt";
        memstatistics-file   "data/named_mem_stats.txt";
listen-on port 53   { any; };
allow-query     { any; };
allow-query-cache   { any; };
recursion yes;
};
```

```
acl secondserver {
  172.16.0.253;
};
zone "." IN {
    type hint;
    file "named.ca";
};
zone "abc.com" IN {
    type master;
    allow-transfer { secondserver; };
    file "abc.com.zone";
};
zone "16.172.in-addr.arpa" IN {
    type master;
    allow-transfer { secondserver; };
  file "172.16.zone";
};
```

3. 创建区数据文件

主配置文件仅是对 zone 域的定义，关于域内主机的具体记录的解析，还需要依赖于数据文件的内容，常见的域名解析有正向解析记录、反向解析记录、CNAME 记录以及 MX 记录等。在上一步完成了主配置文件中关于 zone 域的配置后，我们就根据模板创建具体的区数据解析文件，这些文件的创建可以根据软件包中提供的 named.localhost 文件作为模板。下面我们将创建一个用于正向解析的 abc.com.zone 域数据文件，再创建一个用于进行反向解析的 172.16.zone 域数据文件。

```
[root@centos6 etc]# cd /usr/share/doc/bind-9.8.2/sample/var/named/
[root@centos6 named]# cp named.ca /var/named/chroot/var/named/
[root@centos6 etc]# chown root.named /var/named/chroot/var/named/named.ca
[root@centos6 named]# cp named.localhost /var/named/chroot/var/named/abc.com.zone
[root@centos6 named]# chown root.named /var/named/chroot/var/named/abc.com.zone
[root@centos6 named]# vim /var/named/chroot/var/named/abc.com.zone
```

```
$TTL 1D
@   IN SOA  dns1.abc.com. jacob.abc.com. (
                10  ; serial
                1D  ; refresh
                1H  ; retry
                1W  ; expire
                3H )    ; minimum
NS  dns1.abc.com.
NS  dns2.abc.com.
MX 10   mail.abc.com.
```

```
dns1        IN A  172.16.0.254
dns2        IN A  172.16.0.253
ntp.abc.com. IN A  172.16.0.22
mail.abc.com. IN A  172.16.0.25
fileserver      IN A  172.16.0.100
printserver  IN A  172.16.0.101
www    IN A  172.16.0.200
     IN A  172.16.0.201
```

```
[root@centos6 named]# vim /var/named/chroot/var/named/ 172.16.zone
```

```
$TTL 1D
@   IN SOA  dns1.abc.com. jacob.abc.com. (
                10  ; serial
                1D  ; refresh
                1H  ; retry
                1W  ; expire
                3H )    ; minimum
    NS   dns1.abc.com.
    NS   dns2.abc.com.
254.0   IN PTR  dns1.abc.com.
253.0   IN PTR  dns2.abc.com.
22.0    IN PTR  ntp.abc.com.
25.0        IN PTR  mail.abc.com.
100.0   IN PTR  fileserver.abc.com.
101.0   IN PTR  printserver.abc.com.
200.0   IN PTR  www.abc.com.
201.0   IN PTR  www.abc.com.
```

4. 服务管理

在主服务部署完成后，如果不禁用防火墙或通过防火墙开启特定的端口，客户端主机是无法进行查询工作的。下面我们就通过 Linux 自带的防火墙 iptables 来演示如何开启 DNS 服务所需要使用的 53 端口，其中，TCP 的 53 端口用于主从复制，UDP 的 53 端口用于数据查询。

```
[root@centos6 ~]# iptables -I INPUT -p udp --dport 53 -j ACCEPT
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 53 -j ACCEPT
[root@centos6 ~]# service iptables save
[root@centos6 ~]# service named start
[root@centos6 ~]# chkconfig named on
```

5. 客户端验证

客户端正确配置 DNS 服务器后，查询工具比较多，常用的有 nslookup、dig、host，下面分

别演示这些命令的基本用法。

```
[root@centos6 ~]# nslookup www.abc.com
[root@centos6 ~]# nslookup 172.16.0.100
[root@centos6 ~]# dig www.abc.com
[root@centos6 ~]# dig abc.com MX
[root@centos6 ~]# host www.abc.com
```

4.8.5　部署从域名服务器

部署从域名服务器的作用是防止出现单点故障或实现负载均衡。如果只有一台服务器，当该服务器宕机时，将导致所有客户端的地址解析出现问题。另外，为了满足大规模的查询请求，我们可以创建多台 DNS 服务器实现负载均衡。然而如果所有的 DNS 都作为主服务器，会有大量的配置需要我们重复进行，再有就是当解析记录发生改变后，各个服务器之间的区数据文件的版本比较混乱（不方便统一），所以我们需要部署从域名服务器，从域名服务器会从主域名服务器上下载数据文件，只要主服务器修改了数据文件中的记录，从服务器就可以自动同步数据。

1. 安装软件

```
[root@centos6 ~]# yum -y install bind
[root@centos6 ~]# yum -y install bind-chroot
[root@centos6 ~]# yum -y install bind-utils
```

2. 修改配置文件

与主服务器一样，我们需要复制模板配置文件，并修改 named.conf 配置文件与主服务器配置文件。有所不同的是，所有从服务器配置文件中除根域以外的所有 zone 域类型均为 slave，依次声明自己是从服务器，并使用 masters 语句指定与哪台主服务器进行数据同步。

```
[root@centos6 etc]# cd /usr/share/doc/bind-9.8.2/sample/etc/
[root@centos6 etc]# cp named.conf /var/named/chroot/etc/
[root@centos6 etc]# chown root.named /var/named/chroot/etc/named.conf
[root@centos6 etc]# vim /var/named/chroot/etc/named.conf
```

```
options
{
directory       "/var/named";       // "Working" directory
dump-file       "data/cache_dump.db";
        statistics-file  "data/named_stats.txt";
        memstatistics-file   "data/named_mem_stats.txt";
listen-on port 53   { any; };
allow-query     { any; };
```

```
    allow-query-cache   { any; };
    recursion yes;
    };
    zone "." IN {
        type hint;
        file "named.ca";
    };
    zone "abc.com" IN {
        type slave;
     masters { 172.16.0.254; };
        file "abc.com.zone";
    };
    zone "16.172.in-addr.arpa" IN {
        type slave;
     masters { 172.16.0.254; };
     file "172.16.zone";
    };
```

3. 同步数据文件

从服务器进行同步操作前，需要创建一个 BIND 软件读取操作的目录，以便将主服务器的数据文件保存至该目录。

在从服务器上，我们仅需要简单设置以下 BIND 主配置即可，当从服务器的主配置文件修改完成后，通过启动服务，BIND 会自动根据配置文件中的 masters 语句寻找主域名服务器，并将主服务器上的数据文件下载至从服务器本地。完成第一次的数据同步后，从服务器会根据同步过来的数据文件中的 SOA 记录选项，决定下次同步数据的时间，本例中主域名服务器的 SOA 记录括号中有五个选项。

第一个选项 10 是序列号，从服务器会根据这个序列号来决定是否进行同步操作，只有当主服务器中数据文件的序列号大于从服务器中数据文件的序列号，从服务器才会真正与主服务器进行数据同步，该序列号建议使用时间格式，如 20121212001，表示 2012 年 12 月 12 日的第一次修改（序列号只要是数字即可，BIND 不强制要求具体数字格式）。

第二个选项 1D，代表从服务器多久与主服务器进行序列号的对比（仅在主服务器序列号大于从服务器的序列号时，才进行数据同步），D 代表 Day，这里为 1 天。

第三个选项 1H，代表如果从服务器请求连接主服务器时，由于网络延迟、主服务器故障等原因，暂时无法连接到主服务器，那么从服务器会每隔一个小时再试一次。

第四个选项 1W，代表如果从服务器尝试一周后还是未能连接到主服务器，则不再进行连接。

第五个选项 3H，代表的是缓存的时间为 3 小时。

注意　在 CentOS 6.3 系统中，从服务器同步数据文件时需要修改 SELinux 设置，否则将无权限进行同步操作。

```
[root@centos6 ~]# setsebool -P named_write_master_zones=1
[root@centos6 ~]# mkdir -p /var/named/chroot/var/named/slaves/
[root@centos6 ~]# chown root.named /var/named/chroot/var/named/slaves/
[root@centos6 ~]# chmod 775 /var/named/chroot/var/named/slaves/
[root@centos6 ~]# service named start
[root@centos6 ~]# chkconfig named on
```

4.8.6　DNS 视图应用案例

view 语句用来创建视图，视图可以让不同的网络或主机在查询同一个 DNS 记录时得到不同的解析结果，也可以为不同的网络或主机创建不同的域数据文件。大型企业可以利用视图实现负载均衡，例如，当北京地区的网民访问新浪网时，域名服务器可以将结果解析为北京本地的新浪网服务器，上海地区的网民访问新浪网时，域名服务器会将上海的新浪网服务器 IP 地址作为结果返还给用户。这样所有的人访问新浪网页面查看新闻时，总是可以连接距离自己最近的服务器。

在 BIND 9 版本中，如果没有在主配置文件中使用 view 语句，则 BIND 会自动将所有的域定义为一个大的视图。通过在配置文件中使用 view 语句，并结合 match-clients 语句，实现不同用户查询相同记录时所得的结果不同。这样做的最大好处是可以让北京地区的网民访问新浪网时连接的是北京当地的服务器，而上海地区的网民需要连接新浪网时，DNS 解析的结果为上海本地的新浪服务器。

一般我们会为同一个域创建多个视图，不同的视图对应不同的解析文件，此时需要注意的是，有多少个视图就需要创建多少个对应的数据解析文件。当客户端发送查询请求后，服务器根据视图内的 match-clients 语句来匹配客户端主机，匹配成功后，服务器读取特定的视图内 file 指定的数据文件，并将结果返回给客户端，如果没有匹配成功，则继续查询下一个视图。如果所有的试图都未能匹配成功，服务器将返回无相关数据记录的信息给客户端。下面的案例通过修改主域名服务器的主配置文件演示视图的应用，当 172.16.0.88 请求 abc.com 域的相关解析记录时，服务器会读取 abc.com.zone.develop 这个数据解析文件，172.16.0.89 请求 abc.com 域的相关解析记录时，服务器会读取 abc.com.zone.tech 这个数据解析文件。注意，在此需要创建 4 个数据解析文件，相同域的正向域名解析文件有两个，反向域名解析文件有两个，只要在不同的数据解析文件中对相同的记录给予不同的解析结果，就可实现不同的用户得到的返回结果也不

一样。

只要根据自己的实际需要创建相应的数据文件，并在各个数据文件中对同一个数据记录给出不同的解析数据，即可实现智能 DNS 的分离解析功能。这里我们不可能将全北京或全上海的所有 IP 地址写入 match-clients，根据 DNS 查询原理我们知道，所有的终端用户访问网络时都会连接当地 ISP 服务商所提供的 DNS 服务器，这些服务商的 DNS 服务器会根据根服务器的提示迭代查询到我们的权威服务器。也就是说，我们在 match-clients 后面仅需要填写全国主要的 ISP 所提供的 DNS 服务器 IP 地址即可。不同地区服务商的 DNS，我们给予不同的解析结果，最终所有的终端用户会间接获得正确的解析结果。这里我们不再给出具体的数据文件记录，读者可以根据实际情况自行填写数据记录。

以下为主配置文件 named.conf 实现视图应用的案例模板，读者根据自己的实际需要稍作修改即可。

```
[root@centos6 etc]# vim /var/named/chroot/etc/named.conf
```

```
options
{
directory       "/var/named";       // "Working" directory
dump-file       "data/cache_dump.db";
        statistics-file  "data/named_stats.txt";
        memstatistics-file   "data/named_mem_stats.txt";
listen-on port 53   { any; };
allow-query     { any; };
allow-query-cache   { any; };
recursion yes;
};
acl secondserver {
    192.168.0.102;
};
view "developet" {
match-clients           { 172.16.0.88; };
zone "abc.com" IN {
    type master;
    allow-transfer { 192.168.0.102; };
    file "abc.com.zone.develop";
};
zone "16.172.in-addr.arpa" IN {
    type master;
    allow-transfer { 192.168.0.102; };
  file "172.16.zone.develop";
};
};
view "technet" {
```

```
    match-clients           { 172.16.0.89; };
    zone "abc.com" IN {
        type master;
        allow-transfer { 192.168.0.102; };
        file "abc.com.zone.tech";
    };
    zone "16.172.in-addr.arpa" IN {
        type master;
        allow-transfer { 192.168.0.102; };
      file "172.16.zone.tech";
    };
    };
    view "root" {
    match-clients           { any; };
    zone "." IN {
        type hint;
        file "named.ca";
    };
    };
```

4.8.7　常见问题分析

（1）由于 BIND 主配置文件中，默认 allow-query 被设置为仅 localhost 可以进行 DNS 查询，如果要开放 DNS 服务，则需要将 allow-query 修改为特定的主机或任意主机可以查询的 DNS 服务。

（2）在主配置文件中，默认 listen-on 被设置为仅监听本地回环地址，这样客户端也是无法连接服务器进行查询的。

（3）客户端连接服务器发送的查询请求使用的是 UDP 的 53 端口，而从服务器与主服务器同步数据时使用的是 TCP 的 53 端口，开放 DNS 服务后，要注意修改防火墙设置。

（4）在 CentOS 6.3 系统环境下，从域名服务器与主域名服务器同步数据时，默认 SELinux 会拒绝从远程下载的数据文件写入本地磁盘，此时需要修改 SELinux 的布尔值，方法如下：

```
[root@centos6 ~]# setsebool -P named_write_master_zones=1
```

（5）配置文件问题。如果管理员在配置文件中的语法格式有错误，则 named 服务无法正常启动。例如，如果在配置文件语句后面少了分号，启动服务时会提示类似的提示信息：/etc/named.conf:15: missing ';' before 'view'，提示 named.conf 文件第 15 行左右少“;”符号。

（6）文件与目录权限。由于 BIND 相关进程都是以 named 用户身份启动的，当配置文件或数据文件的权限无法被正确读取时，则系统将无法启动 BIND 服务。通过查看/var/log/messages 日志文件，可以看到 none:0: open: /etc/named.conf: permission denied 这样的提示，说明 BIND 软

件启动时无权读取 named.conf 文件。类似的问题还可能出现在从服务器同步的时候，要求从服务器一定要把主服务器的数据文件同步到本地有读写权限的目录下。

（7）缩写问题。由于 BIND 数据文件有缩写功能，这样就有可能产生低级错误，就是在数据文件中输入的完整域名没有以“.”结尾。例如，www.abc.com IN A 172.16.0.200，这样客户将无法查询到 www.abc.com 的解析记录，只能查询 www.abc.com.abc.com 的解析记录，但这应该不是我们所需要的。

4.9 Apache 网站服务器

4.9.1 Apache 简介

Apache HTTP Server 项目是在 Windows 与 UNIX 等平台上都可以运行的跨平台开源 HTTP 服务器软件，该项目的目标是提供安全、高效、可扩展的 HTTP 服务。Apache httpd 自从 1996 年发布以来，如今已经成为世界排名第一的 Web 服务器软件。Apache httpd 最新的稳定版本为 2.4.4。安装 Apache httpd 软件可以选择源码安装或二进制数据包安装，但由于源码安装是可以定制的一种安装方式，这种安装方式的灵活性比较大，可以满足企业对各种环境的不同需求。二进制软件包格式种类繁多，在 CentOS 6.3 系统中安装二进制软件包可以选择 RPM 包安装，这种安装的最大好处就是简单快捷。本书采用的是源码安装软件包，由于源码安装需要大量的依赖包，因此需要先安装这些依赖软件包。

Apache HTTP Sever 非常重要的特性是它采用了模块化设计模型，Apache 模块分为静态模块与动态模块，静态模块是 Apache 最基本的模块，是无法随时添加与卸载的模块，静态模块在编译软件时设定。动态模块是可以随时添加和删除的模块，这样的设计使得企业部署 Apache 时可以获得最大的灵活性，每个企业都可以根据自己的平台以及实际需求，安装使用不同的模块功能。Apache 模块将被编译为动态共享对象（DSO），这些动态共享对象独立于 httpd 程序，DSO 模块可以在编译 Apache 是添加，也可以在后期随时通过 Apache Extension Tool（apxs）工具编译添加模块。Apache HTTP Server 软件安装完成后可以使用 httpd –M 命令查看模块加载清单。

4.9.2 安装 Apache 软件

1. 下载软件包

```
[root@centos6 ~]# wget http://mirror.bit.edu.cn/apache/httpd/httpd-2.4.4.tar.gz
```

```
[root@centos6 ~]# wget http://mirror.bjtu.edu.cn/apache/apr/apr-1.4.6.tar.gz
[root@centos6 ~]# wget http://mirror.bjtu.edu.cn/apache/apr/apr-util-1.5.2.tar.gz
```

2. 安装软件

由于我们采用的是源码安装 Apache，所以这里在安装 Apache HTTP Server 之前需要安装很多依赖软件包，其中有些可以使用 YUM 安装，有些需要去 Apache 官方网站下载源码软件，RPM 依赖包直接使用 YUM 安装即可，源码软件包在前面的操作中已经下载完成，这里手动编译安装即可。

```
[root@centos6 ~]# yum -y install gcc autoconf automake make \
> pcre pcre-devel openssl openssl-devel
[root@centos6 ~]# tar -xzf httpd-2.4.4.tar.gz -C /usr/src/
[root@centos6 ~]# tar -xzf apr-1.4.6.tar.gz -C /usr/src/
[root@centos6 ~]# tar -xzf apr-util-1.5.2.tar.gz -C /usr/src/
[root@centos6 ~]# cd /usr/src/apr-1.4.6/
[root@centos6 apr-1.4.6]# ./configure
[root@centos6 apr-1.4.6]# make && make install
[root@centos6 apr-1.4.6]# cd /usr/src/apr-util-1.5.2/
[root@centos6 apr-util-1.5.2]# ./configure  --with-apr=/usr/local/apr/
[root@centos6 apr-util-1.5.2]# make && make install
[root@centos6 apr-util-1.5.2]# cd /usr/src/httpd-2.4.4/
[root@centos6 httpd-2.4.4]# ./configure --prefix=/usr/local/apache2 --enable-so \
> --enable-ssl  --enable-rewrite --with-mpm=worker --with-suexec-bin \
> --with-apr=/usr/local/apr/
[root@centos6 httpd-2.4.4]# make && make install
```

源码软件包中的 configure 脚本用来检查系统环境、查找依赖文件、设置安装路径等操作，configure 拥有很多选项，表 4-12 为 configure 常用选项说明，读者可以通过./configure --help 查看该脚本支持的所有选项。

表 4-12

选项	描述
--prefix	指定 Apache httpd 程序的安装主目录
--enable-so	开启模块化功能，支持 DSO（动态共享对象）
--enable-ssl	支持 SSL 加密
--enable-rewrite	支持地址重写
--with-mpm	设置 Apache httpd 工作模式
--with-suexec-bin	支持 SUID、SGID
--with-apr	指定 apr 程序绝对路径

3. 启动服务

```
[root@centos6 ~]# /usr/local/apache2/bin/apachectl start
[root@centos6 ~]# netstat -ntulp |grep http
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 80 -j ACCEPT
[root@centos6 ~]# service iptables save
```

安装完成后，Apache 会提供名为 apachectl 启动脚本，脚本在/usr/local/apache2/bin 目录下，该脚本用来进行 Apache httpd 的启动、关闭以及测试功能，具体的参数描述见表 4-13。没有修改配置文件的情况下使用 start 启动 httpd 程序，可能会返回错误提示：“Could not reliably determine the server’s fully qualified domain name”，提示说明 httpd 无法确定服务器域名称，可以修改主配置文件的 ServerName 项来解决。该提示也可以忽略，通过 netstat 命令查看 httpd 已经启动成功。

在客户端使用浏览器访问该 Web 服务器，看到“IT works!”说明服务器可以被正常访问了。

表 4-13

参数	描述
start	启动 httpd 程序，如果已经启动过该程序，则报错
stop	关闭 httpd 程序
restart	重启 httpd 程序
graceful	启动 httpd，不中断现有的 http 连接请求
graceful-stop	关闭 httpd，不中断现有的 http 连接请求
status	查看 httpd 程序当前状态
configtest	检查 httpd 主配置文件语法

4.9.3 配置文件解析

Apache 配置文件默认位于/usr/local/apache2/conf 目录下，在该目录下最主要的配置文件是 httpd.conf 文件，还有一些位于 extra 目录下的附加配置文件。此外，original 目录中存有这些配置文件的所有副本。Apache HTTP Sever 主配置文件主要由指令以及容器组成，容器使用<容器名称>开始，以</容器名称>结尾，容器的指令一般仅在容器内有效。下面是对主配置文件重点指令的具体描述。

1. SeverRoot 指令

ServerRoot 指令设置 Apache 软件的安装主目录，如果采用源码安装，则默认路径为/usr/local/apache2。

2. Listen 指令

Listen 指令设置服务器监听的 IP 以及端口号，默认监听服务器本机所有的 IP 地址的 80 端口。语法格式为：Listen [IP 地址:]端口 [协议]，其中，IP 地址与协议为可选项，默认监听所有的 IP，使用 TCP 协议。一个配置文件中可以多次使用 Listen 指令来开启多个端口。

3. LoadModule 指令

Apache HTTP Server 的特色之一就是其功能多数是以模块方式加载的，如果希望 Apache 动态加载模块，需要在编译 Apache 时通过--enable-so 将 mod_so 以静态方式编译到 Apache 核心模块中，LoadModule 指令的作用就是加载模块。语法格式为：LoadModule 模块 模块文件名称。其中的模块文件一般位于 ServerRoot 目录下的 modules 目录中。

4. LoadFile 指令

LoadFile 指令的功能类似于 LoadModule，区别在于 LoadFile 可以通过绝对路径加载 modules 目录下的模块文件。

5. ServerAdmin 指令

当网站出现故障时，需要为客户提供一个可以帮助解决问题的邮件地址，ServerAdmin 指令的作用是提供这样的邮件地址。

6. ServerName 指令

ServerName 指令设置服务器本机的主机名称以及端口，对 URL 地址的重定向很重要。

7. DocumentRoot 指令

该指令设置 Web 服务对客户端开放可见的文档根目录，也就是客户端访问网站的根路径，默认为/usr/local/apache2/htdocs。

8. ErrorLog 指令

ErrorLog 定位服务器错误日志的位置，默认使用相对路径，为 ServerRoot 目录下的“logs/error_log”文件。

9. ErrorLogFormat 指令

设置错误日志的格式，Apache HTTP Server 预先定义了很多格式字串[4]可以直接引用。

10. CustomLog 指令

该指令设置客户端的访问日志文件名以及日志格式，默认为“logs/access_log”，语法格式为：CustomLog 文件名 格式。

11. LogFormat 指令

该指令描述用户日志文件格式，可以直接使用 Apache 预先设置的格式字串，一般我们会为 LogFormat 指令设置的日志格式创建别名，然后通过 CustomLog 指令调用该日志格式别名。

12. Include 指令

Include 指令允许 Apache 在主配置文件中加载其他的配置文件，该指令语法比较简单，在 Inclde 指令后直接跟上其他附加配置文件路径即可。

13. Options 指令

该指令为特定目录设置选项，语法格式为：Options [+|-]选项 [[+|-]选项]。选项可以设置为 None，代表不启用任何额外的功能，也可以使用如下常用选项。

- All：开启除 MultiViews 之外的所有选项。
- ExecCGI：允许执行 Options 指定目录下的所有 CGI 脚本。
- FollowSymlinks：允许 Options 指定目录下的文件链接到目录外的文件或目录。
- Indexes：如果与 URL 对应的 Options 目录下找不到 DirectoryIndex 指定的首页文档，则 Apache 将会把当前目录的所有文件索引出来。

14. Order 指令

控制默认访问状态以及 Allow 与 Deny 的次序，如果使用 Order deny,allow，则先检查拒绝，再检查允许，当拒绝与允许有冲突时，允许优先，默认规则为允许。如果使用 Order allow, deny，

---

[4] Apache HTTP Server 预先定义了大量的日志格式字串，具体可以参考：http://httpd.apache.org/docs/trunk/zh-cn/mod/mod_log_config.html#logformat。

则先检查允许，再检查拒绝，当允许与拒绝有冲突时，拒绝优先，默认规则为拒绝。

以下为具体案例：

```
Order deny,allow
Deny from all
```

先检查拒绝规则，再检查允许规则，默认为允许。这里的 Deny from all 代表拒绝所有，所以最终结果为拒绝所有。

```
Order Allow,Deny
allow from all
```

先检查允许规则，再检查拒绝规则，allow from all 代表允许所有，所以最终结果为允许所有。

```
Order Allow,Deny
allow from 192.168.0.1
```

先检查允许规则时允许 192.168.0.1，其余为默认值，默认为拒绝所有，最终除 192.168.0.1 外拒绝所有。

```
Order Allow,Deny
allow from 192.168.0.1
Deny from All
```

先检查允许规则时允许 192.168.0.1 访问，但检查拒绝规则时为拒绝所有，而 192.168.0.1 也包含在 all 中。当 Allow 与 Deny 有冲突时，以 Order 最后的规则覆盖其他的规则，本例将使用 Deny 规则覆盖 Allow 规则，最终效果为拒绝所有的人访问。

```
Order Deny,Allow
Deny from all
allow from 192.168.0.1
```

先检查拒绝规则，再检查允许规则，拒绝规则为拒绝所有，允许规则为允许 192.168.0.1，拒绝与允许规则有冲突的部分，本例将采用 Allow 的规则，最终可以实现仅允许 192.168.0.1 访问，其他任何主机均无法访问 Apache 服务。

15. IfDefine 容器

IfDefine 容器封装的指令仅在启动 Apache 时测试条件为真才会被处理，测试条件需要在启动 Apache 时通过 httpd -D 定义。语法格式为：<IfDefine>指令</IfDefine>。

具体案例如下：

```
<IfDefine MemCache>
```

```
LoadModule mem_cache_module modules/mod_mem_cache.so
</IfDefine>
<IfDefine UseCache>
LoadModule cache_module modules/mod_ cache.so
</IfDefine>
```

这样的配置可以让管理员采用多种配置方式启动 Apache，启动 Apache 时，如果使用了 httpd -DuseCache -DMemCache，则 Apache 将加载 mod_mem_cache 与 mod_cache 模块，如果没有使用-D 指定任何参数，则 Apache 将不加载这些模块。

16. IfModule 容器

使用 IfModule 容器可以封装仅在条件满足时才会处理的指令，根据指定的模块是否加载决定条件是否满足，语法格式：<IfModule [!] 模块>指令</IfModule>。

具体案例如下：

```
<IfModule unixd_module>
User daemon
Group daemon
</IfModule>
```

以上配置说明，仅在 Apache 加载了 unixd_module 模块后，User daemon 与 Group daemon 才会被 Apache 处理。

17. Directory 容器

该容器内的指令仅应用于特定的文件系统目录、子目录以及目录下的内容，语法格式为：<Directory directory-path>指令</Directory>。路径可以使用~匹配正则表达式。

具体案例如下：

```
<Directory "/usr/local/apche2/htdocs">
Options Indexs FollowSymLinks
</Directory>
<Directory ~ "^/www/[0-9]{3}">
AllowOverride None
</Directory>
```

以上案例中，Opions Index FollowSymLins 仅对/usr/local/apache2/htdocs 目录有效，AllowOverride None 仅对所有/www 目录下包含三个数字的子目录有效。

18. <DirectoryMatch>

DirectoryMatch 类似于 Directory，但可以直接使用正则表达式匹配，无须像 Directory 一样使用~符号才可以匹配。

19. Files 容器

该容器类似于 Directory 容器，但 Files 容器内的指令仅应用于特定的文件，语法格式为：<Files 文件名>指令</File>。与 Directory 类似，它可以使用~匹配正则表达式。

20. FilesMatch 容器

仅使用正则表达式匹配需要的文件，容器内的指令仅应用于匹配成功的特定文件。FilesMatch 等同于使用了~符号的 Files。

21. Location 容器

Location 容器内定义的指令仅对特定的 URL 有效，语法格式为：<Location URL-path| URL>指令</Location>。如果需要使用正则表达式匹配 URL，可以使用~符号。

22. LocationMathch 容器和 VirtualHost 容器

LocationMatch 仅使用正则表达式匹配 URL，等同于使用了~符号匹配的 Location。

4.9.4　虚拟主机应用案例

虚拟主机是在一台服务器上同时运行多个 Web 业务，Apache HTTP Server 支持基于域名以及基于 IP 虚拟主机类型。在 Apache 配置文件中，虚拟主机指令需要使用 VirtualHost 容器封装。基于 IP 的虚拟主机可以根据不同的 IP 地址以及端口号定位不同的网站请求，但基于 IP 的虚拟主机需要独立的 IP 地址定位连接的网站，而目前 IP 地址是互联网的稀缺资源，所以很多时候我们更喜欢基于域名的虚拟主机，服务器可以根据客户端访问 HTTP 的头部信息来实现网站的分离解析，客户端可以使用不同的域名访问位于同一 IP 地址的服务器资源。

当客户端请求到达后，服务器会根据<VirtualHost IP 地址:[端口号]>参数匹配 IP 地址与端口号，IP 地址可以使用*匹配服务器本地所有的 IP 地址。下面通过一个案例说明基于域名虚拟主

机的实现方式，首先需要将主配置文件中的 Include conf/extra/httpd-vhosts.conf 语句开启，默认该行为注释行，去除#符号即可。Include 指令将读取 httpd-vhosts.conf 文件的内容作为主配置文件的一部分。下面仅列出该配置文件中需要修改的部分。

```
[root@centos6 ~]# vim /usr/local/apache2/conf/extra/httpd-vhosts.conf
```

```
<VirtualHost *:80>
    ServerAdmin Jacob_test@gmail.com
    DocumentRoot "/usr/local/apache2/htdocs/example"
    ServerName www.example.com
    ServerAlias web.example.com
ErrorLog "logs/www.example.com-error_log"
CustomLog "logs/www.example.com-access_log" common
</VirtualHost>

<VirtualHost *:80>
    ServerAdmin Jacob_test@gmail.com
    DocumentRoot "/usr/local/apache2/htdocs/test"
    ServerName www.test.com
    ErrorLog "logs/test. com-error_log"
    CustomLog "logs/test.com-access_log" common
</VirtualHost>
```

因为搭建了两个虚拟主机服务，接下来需要为两个不同的虚拟主机创建各自的页面根目录 example 和 test，这两个目录对应两个不同的虚拟主机。默认 httpd 监听的是 TCP 的 80 端口，因为需要通过防火墙开放 TCP 协议的 80 端口。

```
[root@centos6 ~]# mkdir -p /usr/local/apache2/htdocs/{example,test}
[root@centos6 ~]# echo "example.com" > /usr/local/apache2/htdocs/example/index.html
[root@centos6 ~]# echo "test.com" > /usr/local/apache2/htdocs/test/index.html
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 80 -j ACCEPT
[root@centos6 ~]# service iptables save
[root@centos6 ~]# /usr/local/apache2/bin/apachectl restart
```

Apache HTTP Sever 完成以上配置后，即可实现基于相同 IP 地址的 www.example.com 与 www.test.com 网站。可以使用客户端访问这两个域名测试效果，如果没有合法的 DNS 服务器，可以通过修改 hosts 文件的方式进行域名解析。如果读者需要实现基于 IP 地址的虚拟主机，只需要将 VirtualHost 后面的*修改为固定的 IP 地址即可，多个虚拟主机需要使用多个 VirtualHost 封装。

4.9.5　网站安全应用案例

一般情况下，网站使用的是明文传输模式。但日常生活中我们经常会遇到在进行网银交易时，银行网站会被自动跳转到 SSL（Secure Sockets Layer）加密传输模式，SSL 的功能是提供加密数据。这样，我们的 TCP/IP 协议就可以专心做好自己的事情，网络传输过程中的数据加密则全权委托给 SSL 协议来完成。TLS（Transport Layer Security）是对 SSL 的扩展与优化，它可以提供数据安全，同时确保数据的完整性。Apache HTTP Server 通过 mod_ssl 模块来实现对 SSL/TLS 的支持。

不管是 SSL 还是 TLS，都是基于非对称加密算法实现的网络数据安全，非对称加密算法使用公钥与私钥两把不同的钥匙，公钥与私钥是不可逆的。也就是说，使用公钥无法推算出私钥，使用私钥也永远无法推算出公钥。非对称加密使得拥有公钥的用户加密自己的数据后，通过网络将加密数据发送给拥有私钥的人，这个过程中，如果有人拦截了该加密后的数据，但你是不可能解密该数据的，甚至加密者自己都无法将加密后的数据解开，因为别人没有密钥，而加密者本人也仅拥有公钥，而非对称加密使用的是公钥加密和私钥解密机制。但个人生成的公钥与私钥是不被信任的，只有经过 CA（Certificate Authority）认证后才被认为是可信任的密钥。目前大多数浏览器都内置了国际知名 CA 的根证书中心，如果我们的密钥被 CA 中心认证签名过，那就是合法的数字证书。而 CA 验证密钥需要经过一个非常复杂的流程，所以很多人通过自签名生成数字证书，也就是所谓的自签名证书。在 CentOS 6.3 环境中，如果需要生成密钥与证书，可以选择 openssl 工具创建，具体用法如下。

```
[root@centos6 ~] openssl genrsa -out server.key 2048   #生成私钥
[root@centos6 ~] openssl req -new -x509 -key server.key \
> -out server.crt                                      #根据私钥生成根证书
Country Name (2 letter code) [XX]:CN                   #国家名称
State or Province Name (full name) []:Beijing          #省份
Locality Name (eg, city) [Default City]:Beijing        #城市
Organization Name (eg, company) [Default Company Ltd]:ABC  #组织名称
Organizational Unit Name (eg, section) []:tech         #部门名称
Common Name (eg, your name or your server's hostname) []:web1  #主机名称
Email Address []:abc@gmail.com                         #电子邮件
[root@centos6 ~] cp  {server.key,server.cert}  /usr/local/apache2/conf/
#复制证书
```

部署 TLS/SSL 网站除了需要生成私钥与证书外，还需要修改 Apache 配置文件，主配置文件中需要加载 mod_ssl 以及 mod_socache_shmcb 两个模块，同时使用 Include 指令读取 conf/extra 目录下的 httpd-ssl.conf 配置文件，以下是我们修改以后的 http-ssl.conf 内容。

```
Listen 443                                                    #监听端口
SSLCipherSuite HIGH:MEDIUM:!aNULL:!MD5
SSLPassPhraseDialog  builtin
SSLSessionCache        "shmcb:/usr/local/apache2/logs/ssl_scache(512000)"
#缓存
SSLSessionCacheTimeout  300                                   #超时时间
<VirtualHost _default_:443>                                   #设置虚拟主机
DocumentRoot "/usr/local/apache2/htdocs/secret"               #设置网站根目录
ServerName www.abc.com:443                                    #设置域名及端口
ServerAdmin you@example.com                                   #设置管理员邮箱
ErrorLog "/usr/local/apache2/logs/error_log"                  #设置错误日志
TransferLog "/usr/local/apache2/logs/access_log"              #设置访问日志
SSLEngine on                                                  #开启 SSL 引擎
SSLCertificateFile "/usr/local/apache2/conf/server.crt"    #设置证书文件
SSLCertificateKeyFile "/usr/local/apache2/conf/server.key"#设置私钥文件
<FilesMatch "\.(cgi|shtml|phtml|php)$">
    SSLOptions +StdEnvVars
</FilesMatch>
<Directory "/usr/local/apache2/cgi-bin">
    SSLOptions +StdEnvVars
</Directory>
BrowserMatch "MSIE [2-5]" \
        nokeepalive ssl-unclean-shutdown \
        downgrade-1.0 force-response-1.0
CustomLog "/usr/local/apache2/logs/ssl_request_log" \
         "%t %h %{SSL_PROTOCOL}x %{SSL_CIPHER}x \"%r\" %b"
</VirtualHost>
```

```
[root@centos6 ~]# mkdir -p /usr/local/apache2/htdocs/secret
[root@centos6 ~]# echo "secret" > /usr/local/apache2/htdocs/secret/index.html
[root@centos6 ~] /usr/local/apache2/bin/apachectl restart
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 443 -j ACCEPT
[root@centos6 ~]# service iptables save
```

Apache HTTP Sever 完成以上配置后，客户端主机就可使用 https://www.abc.com 访问加密网站。如果没有合法的 DNS 服务器，可以通过修改 hosts 文件的方式进行域名解析。

4.9.6 常见问题分析

（1）启动 Apache HTTP Server 时提示错误信息：Invalid command 'LanguagePriority', perhaps misspelled or defined by a module not included in the server configuration。提示信息说明在配置文件中使用了 LanguagePriority 指令，但该指令需要加载 mod_negotiation 模块才可以实现相应的功能，解决方法是在主配置文件中通过 LoadModule 加载该模块。

（2）启动 Apache HTTP Server 时提示错误信息：Invalid command 'SSLCipherSuite', perhaps misspelled or defined by a module not included in the server configuration。该提示信息说明在配置文件中使用了 SSLCipherSuite 指令，但该指令需要 mod_ssl 模块才可以实现相应的功能。

（3）启动 Apache HTTP Server 时提示错误信息：SSLSessionCache: 'shmcb' session cache not supported。该提示信息说明 shmcb 会话缓存不支持，需要加载 mod_socache_shmcb 模块才可以支持该功能。

（4）启动 Apache HTTP Server 时提示错误信息：Address already in use，could not bind to address。说明服务器上已经开启了另一个程序正在监听使用该端口，使用 netstat 工具可以查看网络连接状况。

（5）客户端访问时显示的不是首页内容，而是首页目录下的所有文件列表，表明通过 DocumentRoot 指令设置的网站根目录，无法找到由 DirectoryIndex 指令设置的首页文件。

（6）客户端访问加密网站时，如果数字证书是自签名证书，浏览器会提示：此网站的安全证书有问题。因为我们的证书没有经过权威证书中心签名，所以浏览器会提示不安全，如果确定该证书没有问题，可以继续浏览此网站。

4.10　Nginx 网站服务器

4.10.1　Nginx 简介

Nginx 是一款开放源代码的高性能 HTTP 服务器和反向代理服务器，同时支持 IMAP/POP3 代理服务，这是一款自由的软件。俄罗斯设计师 Igor Ysyoev 在 2002 年开始开发该软件，于 2004 年发布第一个公开版本。Nginx 以高性能、高可用、丰富的功能模块、简单明了的配置文档以及低资源占用而著称。Nginx 采用最新的网络 I/O 模型，支持高达 50 000 个并发连接。近年来，Nginx 在国内取得了突飞猛进的发展，很多门户网站开始提供 Nginx 解决方案。

4.10.2　安装 Nginx 软件

与 Apache HTTP Server 一样，这里使用源码编译安装该软件，软件包可以在官方网站 http://nginx.org 下载。编译安装之前，我们需要使用 YUM 提前将所需的软件依赖包安装完成，configure 脚本的具体参数使用及含义可以参考表 4-14。本书案例将 nginx 安装在/usr/local/nginx 目录下。

```
[root@centos6 ~]# wget http://nginx.org/download/nginx-1.4.0.tar.gz
[root@centos6 ~]# tar -xzf nginx-1.4.0.tar.gz -C /usr/src/
[root@centos6 ~]# yum -y install gcc pcre pcre-devel gcc openssl \
>openssl-devel gd gd-devel perl perl-ExtUtils-Embed
[root@centos6 ~]# cd /usr/src/nginx-1.4.0/
[root@centos6 nginx-1.4.0]# ./configure -prefix=/usr/local/nginx \
>--with-ipv6 \
>--with-http_ssl_module \
>--with-http_realip_module \
>--with-http_addition_module \
>--with-http_dav_module \
>--with-http_flv_module \
>--with-http_mp4_module \
>--with-http_gzip_static_module \
>--with-http_perl_module \
>--with-mail \
>--with-mail_ssl_module
[root@centos6 nginx-1.4.0]# make && make install
```

Nginx 与 Apache 一样都采用的是模块化设计，Nginx 模块分为内置模块和第三方模块，其中，内置模块中包括主模块与事件模块。表 4-14 给出的模块为默认自动编译的模块，可以使用--without 参数禁用。表 4-15 为内置模块中的附加模块，需要在编译时通过--with 参数手动开启。编译 Nginx 时还可以通过--add-module=/path/module1 的方式编译第三方模块。

表 4-14

模块名称	描述	禁用选项
Core	Nginx 核心功能	--without-http
Access	基于 IP 的访问控制	--without-http_access_module
Auth Basic	HTTP 用户认证模块	--without-http_auth_basic_module
Auto Index	自动目录索引	--without-http_autoindex_module
Browser	描述用户代理	--without-http_charset_module
Charset	重新编码网页	--without-http_charset_module
Empty GIF	内存中存放一个图片	--without-http_empty_gif_module
FastCGI	FastCGI 支持	--without-http_fastcgi_module
Geo	支持 IP 变量设置	--without-http_geo_module
Gzip	Gzip 压缩	--without-http_gzip_module
Limit Requests	限制客户端连接频率	--without-http_limit_req_module

续表

模块名称	描述	禁用选项
Limit Conn	会话的并发连接	--without-http_limit_conn_module
Map	设置变量	--without-http_map_module
Memcached	Memcache 支持	--without-http_memcached_module
Referer	基于 Referer 头部信息过滤	--without-http_referer_module
Rewrite	使用正则表达式重写请求	--without-http_rewrite_module
SCGI	支持 SCGI 协议	--without-http_scgi_module
Upstream	负载均衡	--without-http_upstream_ip_hash_module
Headers	设置 http 响应的头部信息	
Index	首页	
Log	自定义日志	

表 4-15

模块名称	描述	开启选项
Embedded Perl	支持 Perl	--with-http_perl_module
FLV	支持 Flash 视频	--with-http_flv_module
GeoIP	通过 IP 变量实现负载均衡	--with-http_geoip_module
Google Perftools	支持谷歌的性能优化工具	--with-google_perftools_module
Gzip Precompression	压缩静态文件	--with-http_gzip_static_module
Image Filter	转换图形的过滤器	--with-http_image_filter_module
MP4	支持 MP4	--with-http_mp4_module
Real IP	使用 Nginx 作为后端服务器	--with-http_realip_module
Secure Link	使用密钥保护页面	--with-http_secure_link_module
SSL	支持 HTTPS/SSL	--with-http_ssl_module
Stub Status	查看服务器状态	--with-http_stub_status_module
WebDAV	支持 WebDAV	--with-http_dav_module
Core	邮件代理功能	--with-mail --without-mail_pop3_module --without-mail_imap_module --without-mail_smtp_module
SSL	支持 SSL/TLS 加密邮件协议	--with-mail_ssl_module

Nginx Web 服务器软件安装完成后，程序主目录位于/usr/local/nginx/，该目录下的内容分别

为 conf（主配置文件目录）、html（网站根目录）、logs（日志文件目录）、sbin（主程序目录）。Nginx 默认没有提供启动脚本，需要手动输入命令来管理进程，如果需要更加方便地操作服务器进程，建议将常用的进程管理任务写成脚本。下面是 Nginx 常用的进程管理指令。

```
[root@centos6 ~]# /usr/local/nginx/sbin/nginx                    #启动主程序
[root@centos6 ~]# /usr/local/nginx/sbin/nginx -c \   #指定配置文件启动主程序
>/usr/local/nginx/conf/nginx.conf
[root@centos6 ~]# /usr/local/nginx/sbin/nginx -s stop        #关闭主程序
[root@centos6 ~]# /usr/local/nginx/sbin/nginx -s reload      #重新加载设置
```

Nginx 会将进程号保存在/usr/local/nginx/logs/nginx.pid 文件中，我们可以使用 kill 指令发送信号给该进程号，常用信号参见表 4-16。具体案例如下：

```
[root@centos6 ~]# kill -QUIT `cat /usr/local/nginx/logs/nginx.pid`
[root@centos6 ~]# kill -HUP `cat /usr/local/nginx/logs/nginx.pid`
```

表 4-16

信号名称	描述	信号名称	描述
TERM,INT	快速关闭	QUIT	优雅地关闭，保持现有的客户端连接
HUP	重启应用新的配置文件	USR1	重新打开日志文件
USR2	升级程序	WINCH	优雅地关闭工作进程

4.10.3 配置文件解析

Nginx 默认的配置文件为/usr/local/nginx/conf/nginx.conf，配置文件主要包括全局、event、http、server 设置，event 主要用来定义 Nginx 工作模式，http 提供 Web 功能，server 用来设置虚拟主机，server 必须位于 http 内部，一个配置文件中可以有多个 server。

```
#设置用户与组
#user  nobody;
#启动子进程数，可以通过 ps aux |grep nginx 查看
worker_processes  1;
#错误日志文件，以及日志级别
error_log logs/error.log info;
#进程号保存文件
pid  logs/nginx.pid;
events {
#每个进程可以处理的连接数，受系统文件句柄的限制
    worker_connections  1024;
}
http {
#mime.types 为文件类型定义文件
```

```
    include       mime.types;
    #默认文件类型
    default_type  application/octet-stream;
    #使用 log_format 可以自定义日志格式，名称为 main
        #log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
        #                  '$status $body_bytes_sent "$http_referer" '
    #                  '"$http_user_agent" "$http_x_forwarded_for"';
    #创建访问日志，格式采用 main 定义的格式
        #access_log  logs/access.log  main;
    #是否调用 sendfile()进行数据复制，sendfile()复制数据是在内核级别完成的，所以会比
    #一般的 read、write 更高效
    sendfile on;
    #开启后服务器的响应头部信息产生独立的数据包发送，即一个响应头信息一个包
    tcp_nopush on;
    #保持连接的超时时间
    keepalive_timeout  65;
    #是否采用压缩功能，将页面压缩后传输更节省流量
    gzip  on;
    #使用 server 定义虚拟主机
    server {
    #服务器监听的端口
        listen       80;
    #访问域名
        server_name  www.jacob.com;
    #编码格式，如果网页编码与此设置不同，则将被自动转码
        #charset koi8-r;
    #设置虚拟主机的访问日志
        #access_log  logs/host.access.log  main;
    #对 URL 进行匹配
        location / {
    #设置网页根路径，使用的是相对路径，html 指的是处于 Nginx 安装路径下
            root   html;
    #首页文件，先找 index.html，若没有，再找 index.htm
            index  index.html index.htm;
        }
    #设置错误代码对应的错误页面
        #error_page  404              /404.html;
        # redirect server error pages to the static page /50x.html
        error_page   500 502 503 504  /50x.html;
        location = /50x.html {
            root   html;
        }
        # proxy the PHP scripts to Apache listening on 127.0.0.1:80
        #下面三行注释行表明，若用户访问 URL 以.php 结尾，则自动将该请求转交给
```

```
        #127.0.0.1 服务器，通过 proxy_pass 可以实现代理功能
        #location ~ \.php$ {
        #    proxy_pass   http://127.0.0.1;
        #}
        # pass the PHP scripts to FastCGI server listening on 127.0.0.1:9000
        #location ~ \.php$ {
        #    root           html;
        #    fastcgi_pass   127.0.0.1:9000;
        #    fastcgi_index  index.php;
        #    fastcgi_param  SCRIPT_FILENAME  /scripts$fastcgi_script_name;
        #    include        fastcgi_params;
        #}
        # deny access to .htaccess files, if Apache's document root
        # concurs with nginx's one
        #拒绝所有的人访问 .ht 页面
    #location ~ /\.ht {
        #    deny  all;
        #}
    }
    # another virtual host using mix of IP-, name-, and port-based configuration
    #定义虚拟主机
    #server {
    #    listen       8000;
    #    listen       somename:8080;
    #    server_name  somename  alias  another.alias;
    #    location / {
    #        root   html;
    #        index  index.html index.htm;
    #    }
    #}
    # HTTPS server
    #server {
    #监听 TLS 使用的 443 端口
    #    listen       443;
    #    server_name  localhost;
    #开启 SSL 功能
    #    ssl                  on;
    #指定证书文件，使用相对路径证书需要存放在与 nginx.conf 同目录下
    #    ssl_certificate      cert.pem;
    #指定私钥文件，使用相对路径私钥需要存放在与 nginx.conf 同目录下
    #    ssl_certificate_key  cert.key;
    #    ssl_session_timeout  5m;
    #    ssl_protocols  SSLv2 SSLv3 TLSv1;
    #    ssl_ciphers  HIGH:!aNULL:!MD5;
    #    ssl_prefer_server_ciphers   on;
```

```
        #    location / {
        #        root   html;
        #        index  index.html index.htm;
        #    }
        #}
    }
```

4.10.4　虚拟主机应用案例

下面的例子中，我们创建 4 个基于域名的虚拟主机，第一个虚拟主机使用 www.domain.com 以及除 web.domain.com 外的所有以 domain.com 结尾的域名都可以访问，第二个虚拟主机使用 web.domain.com 访问网站，第三个虚拟主机使用 www.example.com 访问。虚拟主机中最重要的指令是 server_name 和 root，分别用来设置域名与网页根路径。第四个虚拟主机为默认虚拟主机，也就是域名无法匹配以上所有的域名时，使用该虚拟主机响应客户端请求。具体操作如下。

首先需要修改 nginx 主配置文件/usr/local/nginx/conf/nginx.conf，在该文件中添加虚拟主机代码块。其次，为 4 个虚拟主机创建网页根路径，并为每个网站创建测试用的首页文件 index.html。

```
[root@centos6 conf]# vim /usr/local/nginx/conf/nginx.conf
```

```
user  nobody;
worker_processes  1;
error_log  logs/error.log  info;
pid        logs/nginx.pid;
events {
    worker_connections  4;
}
http {
    include       mime.types;
    default_type  application/octet-stream;
    log_format  main '$remote_addr - $remote_user [$time_local] "$request" '
                     '$status $body_bytes_sent "$http_referer" '
                     '"$http_user_agent" "$http_x_forwarded_for"';
    sendfile        on;
    keepalive_timeout  65;
    gzip  on;

    server {
        listen       80;
        server_name  www.domain.com *.domain.com;
        access_log  logs/www.domain.com.access.log  main;
```

```
        location / {
            root   html/domain;
            index  index.html index.htm;
        }
        error_page   500 502 503 504  /50x.html;
        location = /50x.html {
            root   html;
        }
        location ~ /\.ht {
            deny  all;
        }
    }
    server {
        listen       80;
        server_name  web.domain.com;
        location / {
            root   html/web;
            index  index.html index.htm;
        }
    }
    server {
        listen       80;
        server_name  www.example.com;
        location / {
            root   html/example;
            index  index.html index.htm;
        }
    }
    server {
        listen       80 default_server;
        location / {
            root   html/default;
            index  index.html index.htm;
        }
    }
}
```

```
[root@centos6 conf]# mkdir /usr/local/nginx/html/{domain,web,example,default}
[root@centos6 conf]# echo "domain.com" > /usr/local/nginx/html/domain/index.html
[root@centos6 conf]# echo "web.domain.com" > /usr/local/nginx/html/web/index.html
[root@centos6 conf]# echo "example.com" > /usr/local/nginx/html/example/index.html
[root@centos6 conf]# echo "default" > /usr/local/nginx/html/default/index.html
```

为客户端提供 DNS 域名解析后，即可实现基于域名的虚拟主机功能，Nginx 会根据不同的

域名请求响应不同的网站页面。如果没有 DNS 域名解析，可以通过修改 hosts 文件的方式实现。

4.10.5　SSL 网站应用案例

默认情况下，SSL 模块是不被编译的，如果需要部署 SSL 网站，需要在编译 Nginx 时使用 --with-http_ssl_module 参数，而编译该模块需要 OpenSSL 库文件，一般需要安装 openssl 与 openssl-devel 软件。下面通过一个简单的案例说明 Nginx 部署 SSL 网站的流程，修改主配置文件 usr/local/nginx/conf/nginx.conf 的效果如下。

```
[root@centos6 conf]# vim /usr/local/nginx/conf/nginx.conf
worker_processes  1;
error_log  logs/error.log  info;
pid        logs/nginx.pid;
events {
    worker_connections  1024;
}
http {
    include       mime.types;
    default_type  application/octet-stream;
    log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
                      '$status $body_bytes_sent "$http_referer" '
                      '"$http_user_agent" "$http_x_forwarded_for"';
    access_log  logs/access.log  main;
    sendfile        on;
    keepalive_timeout  65;
    gzip  on;
        error_page  404              /404.html;
        error_page   500 502 503 504  /50x.html;
    server {
        listen       443;
        server_name  www.abc.com;
        ssl                  on;
        ssl_certificate      cert.pem;
        ssl_certificate_key  cert.key;
        ssl_session_timeout  5m;
        ssl_protocols  SSLv2 SSLv3 TLSv1;
        ssl_ciphers  HIGH:!aNULL:!MD5;
        ssl_prefer_server_ciphers   on;
        location / {
            root   html;
            index  index.html index.htm;
        }
```

```
        }
    }
```

```
[root@centos6 ~]# cd /usr/local/nginx/conf/
```

为了实现网站数据的加密传输，需要为网站创建证书，本例将使用 openssl 工具创建自签名证书。使用 openssl 生成自签名证书非常简单，第一步，使用 openssl genrsa 生成证书私钥文件 cert.key，密钥长度为 2048B。第二步，使用 openssl req 指令生成自签名证书文件 cert.pem，生成证书的过程中提示输入证书基本信息，读者可以根据自己的实际需要填写这些内容。采用加密方式传输数据的 HTTPS 协议默认使用的端口号是 443，我们需要使用防火墙开启该 TCP 协议的 443 端口。

```
[root@centos6 conf]# openssl genrsa -out cert.key 2048
[root@centos6 conf]# openssl req -new -x509 -key cert.key -out cert.pem
Country Name (2 letter code) [XX]:CN
State or Province Name (full name) []:Beijing
Locality Name (eg, city) [Default City]:Beijing
Organization Name (eg, company) [Default Company Ltd]:ABC
Organizational Unit Name (eg, section) []:tech
Common Name (eg, your name or your server's hostname) []:jacob
Email Address []:jacobxx@gmail.com
[root@centos6 conf]# echo "Error, file not found" >\
>/usr/local/nginx/html/ 404.html
[root@centos6 conf]# /usr/local/nginx/sbin/nginx
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 443 -j ACCEPT
[root@centos6 ~]# service iptables save
```

服务器端启动服务后，可以通过 netstat -nutlp|grep nginx 命令查看端口信息。客户端配置好 DNS 解析后，即可通过浏览器访问 https://www.abc.com 加密网站，其没有 DNS 解析，可以通过修改 hosts 的方式实现。由于是自签名证书，使用浏览器访问时会提示证书不被信任，可以将证书导入浏览器，使得以后的访问不再提示警告信息。

4.10.6 HTTP 响应状态码

日常生活中，用户一般会通过浏览器访问站点页面，即发送页面请求给服务器，然后服务器会根据请求内容做出回应，如果没有问题，服务器会给客户端返回成功状态码，并将相应的页面传给客户端浏览器，但当服务器出现故障时，服务器往往会给客户端发送错误状态码，并根据错误状态码返回错误页面给客户端浏览器。表 4-17 给出了常见的状态码，但并不是全部代码。其中，1XX 代表提示信息，2XX 代表成功信息，3XX 代表重定向信息，4XX 代表客户端错误信息，5XX 代表服务器错误信息。

表 4-17

代码	含义
100	请求已接收，客户端可以继续发送请求
101	Switching Protocals 服务器根据客户端的请求切换协议
200	一切正常
201	服务器已经创建了文档
202	已经接受了请求，但处理尚未完成
203	文档正常返回，但一些头部信息可能不正确
300	客户端请求的资源可以在多个位置找到
301	客户端请求的资源可以在其他位置找到
305	使用代理服务
400	请求语法错误
401	访问被拒绝
401.1	登录失败
403	资源不可用
403.6	IP 地址被拒绝
403.9	用户数过多
404	无法找到指定资源
406	指定资源已找到，但 MIME 类型与客户端要求不兼容
407	要求进行代理身份验证
500	服务器内部错误
500.13	服务器太忙
501	服务器不支持客户端请求的功能
502	网关错误
503	服务不可用
504	网关超时，服务器处于维护或者负载过高无法响应
505	服务器不支持客户端请求的 HTTP 版本

4.11　数据库基础

4.11.1　MySQL 数据库简介

数据库是一个比较模糊的概念，简单的一个数据表格、一份歌曲列表等都可以称为数据库。

如果仅仅是一两个类似的数据表，我们完全可以手动管理这些数据，但在如今这个大数据的年代，数据量都以 TB 为单位时，数据库一般是多个数据表的集合，具体的数据被存放在数据表中，而且大多数情况下，表与表之间都有内在联系。例如，员工信息表与工资表之间就有内在联系，一般都有对应的员工姓名以及员工编号，存在这种表与表相互引用的数据库被称为关系型数据库。而 MySQL 是一个专门的关系型数据库管理系统，它由瑞典 MySQL AB 公司开发，该公司现被 Oracle 公司收购。利用 MySQL 可以创建数据库和数据表、添加数据、修改数据、查询数据等，MySQL 数据库系统的特色是功能强大、速度快、性能优越、稳定性强、使用简单、管理方便。

4.11.2 安装 MySQL 软件

在 CentOS 6.3 平台下安装 MySQL 数据库管理系统的方式有很多，我们可以选择简单的二进制数据包安装，也可以选择源码包安装。由于源码安装方式编译的灵活性，这里我们将采用源码安装的方式作为演示案例。源码安装 MySQL 主要分为安装依赖包、下载 MySQL 源码包、配置安装环境、编译及安装、初始化数据库五大步骤。需要注意的是，MySQL 5.6 使用 cmake 构建安装架构，如果用户坚持使用之前的 configure 安装方式，可以通过源码包中的 BUILD/autorun.sh 脚本生成 configure 文件，从而实现传统的配置安装。不管使用哪种方式，安装部署 MySQL 数据库系统之前都需要安装相应的软件依赖包，这里使用 YUM 提前安装好这些依赖软件包。随后通过 MySQL 官网下载源码软件包后安装该软件。

```
[root@centos6 ~]# yum -y install gcc make cmake ncurses-devel libxml2-devel \
>libtool-ltdl-devel gcc-c++ autoconf automake bison zlib-devel
[root@centos6 ~]# wget http://cdn.mysql.com/Downloads/MySQL-5.6\
>/mysql-5.6.11.tar.gz
```

安装过程中需要为数据库进程创建用户及组账户，因为是服务进程账户，所以，该类账户无须登录系统，设置 Shell 为/sbin/nologin 更加安全。当 MySQL 编译安装完成后，还需要为软件主目录设置正确的用户及组。

```
[root@centos6 ~]# groupadd mysql
[root@centos6 ~]# useradd -r -s /sbin/nologin -g mysql mysql
[root@centos6 ~]# tar -xzf mysql-5.6.11.tar.gz -C /usr/src/
[root@centos6 ~]# cd /usr/src/mysql-5.6.11/
[root@centos6 ~]# cmake . -DENABLE_DOWNLOADS=1
[root@centos6 ~]# make && make install
[root@centos6 ~]# chown -R mysql.mysql /usr/local/mysql
```

数据库软件安装完成后，我们需要使用 mysql_install_db 脚本初始化数据库，使用 user 定义

数据库账户名称，使用 basedir 定义软件主目录，使用 datadir 定义数据库存放目录。初始化完成后，需要将 mysql 主配置文件 my.cnf 复制一份到/etc/目录下。

```
[root@centos6 ~]# /usr/local/mysql/scripts/mysql_install_db --user=mysql \
> --basedir=/usr/local/mysql/ --datadir=/usr/local/mysql/data
[root@centos6 ~]# cp /usr/local/mysql/my.cnf /etc/my.cnf
```

通过以上步骤，我们已经完成了 MySQL 数据库软件的安装，如果想尝试使用强大的 MySQL 数据库管理系统，就需要使用 mysqld_safe 进程手动启动数据库服务进程或通过软件包所提供的启动脚本 mysqld.server 来管理服务进程。

```
[root@centos6 ~]# /usr/local/mysql/bin/mysqld_safe --user=mysql &
[root@centos6 ~]# cp /usr/local/mysql/support-files/mysql.server /etc/init.d/mysqld
[root@centos6 ~]# chkconfig --add mysqld
[root@centos6 ~]# chkconfig mysqld on
[root@centos6 ~]# PATH=$PATH:/usr/local/mysql/bin/
[root@centos6 ~]# echo "export PATH=$PATH:/usr/local/mysql/bin/" >> /etc/profile
```

至此，MySQL 数据库系统已经安装并初始化完成，我们可以通过 mysqld 启动脚本实现开机启动功能。但在 UNIX 环境下，使用 mysql_install_db 程序进行初始化数据库时，默认会创建一个用于测试用的空白数据库 test，同时初始化程序还会在 mysql 数据库中创建 user 数据表，该数据表为 MySQL 数据库系统的账户及权限表，初始情况下，该表有如下一些问题。

- mysql.user 表中的 root 为超级管理员账户，然而该账号默认没有密码，因此可能导致非法用户在没有密码的情况下直接访问数据库。
- mysql.user 中默认创建了匿名账号，这些账号同样没有设置密码。

通过如下命令可以查看 MySQL 默认创建的 user 账号及密码数据。

```
[root@centos6 ~]# mysql -u root -e "SELECT User, Host, Password FROM mysql.user"
```

为此，MySQL 特别提供了一个 Perl 脚本程序专门解决这些问题，运行该脚本的前提是在操作系统中已经安装了 perl 程序。运行脚本后会提示输入 root 密码，首次运行直接按回车键即可。之后该程序会通过一系列的提示询问是否为 root 设置密码、是否移除匿名账户、是否禁止 root 账户从远程访问、是否删除 test 数据库、是否立刻重新加载新的数据。建议所有的提问都选择 Y，特别是在企业生产环境中这一步骤是必需的。

```
[root@bogon ~]# /usr/local/mysql/bin/mysql_secure_installation
```

4.11.3　MySQL 管理工具

MySQL 是基于客户端/服务器体系架构的数据库系统，MySQL 服务器端以守护进程的方式

运行，mysqld 为服务器主进程。当我们需要对数据库进行任何操作时，都需要使用客户端软件来连接服务器进行操作。MySQL 客户端程序有很多，可以使用自带软件 mysql[5]、mysqladmin、mysqldum 等命令对数据库进行数据操作，也可以自己设计动态网站通过 API 连接 MySQL 数据库进行相同的数据操作。另外，在 MySQL 官方网站可以下载到 MySQL WorkBench 工具，该工具是一个图形化 MySQL 数据库客户端管理程序。本节将重点介绍 MySQL 软件包集成的客户端软件。

1. Mysql 工具

mysql 是一个简单的命令行 SQL 工具，该工具支持交互式和非交互式运行，使用 mysql 命令非常容易，只需要在系统命令终端输入 mysql，即可进入管理界面：

```
[root@centos6 ~]# mysql
```

如果使用 mysqladmin 或 mysql_secure_installation 程序为账户设置了密码，则需要在启动 mysql 程序时指定账户名称与密码，进入交互式界面后，即可通过输入 SQL 语句对数据库进行操作，SQL 语句之后要求以“;”、“\g”或者“\G”结尾。退出程序时可以通过 exit 指令或者按 Control+C 快捷键完成。

```
[root@centos6 ~]# mysql -user=user_name -password=your_password 数据库名称
```

通过提前创建 SQL 语句脚本文件，我们可以使用 mysql 工具自动执行数据操作，方法如下。

```
[root@centos6 ~]# cat script.sql
show tables;
select * from mysql.user\G
[root@centos6 ~]# mysql db_name < script.sql > out.tab
```

其中，script.sql 是 SQL 脚本文件，out.tab 为重定向输出文件。

Mysql 命令支持大量的选项，表 4-18 给出了常用的 mysql 选项及描述说明。

表 4-18

选项	描述
--help,-?	显示帮助消息
--auto-rehash	Tab 自动补齐，默认为开启状态
--auto-vertical-output	自动垂直显示，如果显示的结果太宽，将以列格式显示
--batch,-B	不使用历史文件

[5] 注意，这里说的 mysql 指的是数据库客户端软件，而不是 MySQL 数据库系统。

续表

选项	描述
--bind-address=ip_address	使用特定的网络接口连接 MySQL 服务器
--compress	压缩客户端与服务器传输的所有数据
--database=dbname,-D dbname	指定使用的数据库名称
--default-character-set=charset_name	设置默认字符集
--delimiter=str	设置语句分隔符
--host=host_name,-h host_name	通过 host 连接到指定服务器
--password,-p	使用密码连接服务器
--pager=[command]	使用分页程序分页显示，Linux 中可以使用 more 或 less
--port=port_num	使用指定端口号连接服务器
--quick	不缓存查询结果
--unfuffered	每次查询后刷新缓存
--user=user_name,-u user_name	使用指定的账户连接服务器

演示案例如下。

使用 root 账号连接服务器，无密码登录：

```
[root@centos6 ~]#mysql -u root
```

使用 root 账号连接服务器，默认进入 MYSQL 数据库：

```
[root@centos6 ~]#mysql -u root mysql
```

使用 root 账号连接服务器，使用密码 pass 登录：

```
[root@centos6 ~]#mysql -u root -p'pass'
```

使用 root 账号连接 192.168.0.254 服务器，提示输入密码：

```
[root@centos6 ~]#mysql -u root -p -h 192.168.0.254
```

2. mysqladmin 工具

mysqladmin 是一个执行管理操作的工具，使用它可以检查服务器配置、当前运行状态，以及创建、删除数据库等。

语法格式如下：

mysqladmin [选项] 命令 [命令参数] [命令 [命令参数]]

mysqladmin 支持以下这些命令（有些命令需要参数）。

create db_name	创建名为 db_name 的数据库。
debug	将 debug 信息写入错误日志。
drop db_name	删除名为 db_name 的数据库以及数据库中所有的数据表。
extended-status	显示服务器状态变量及变量值。
flush-hosts	刷新所有主机的缓存信息。
flush-logs	刷新所有的日志。
flush-privileges	重新加载权限数据表。
flush-status	清空状态变量。
flush-tables	刷新所有数据表。
kill id,id,…	杀死服务器线程。
password new-pass	设置新的密码。
ping	检查服务器是否可用。
reload	重新加载权限数据表。
refresh	刷新所有的数据表并重启日志文件。
shutdown	关闭服务器。
start-slave	在从服务器上启动复制。
stop-slave	在从服务器上停止复制。

表 4-19 列出了 mysqladmin 常用的选项及描述说明。

表 4-19

选项	描述
--bind-address=ip_address	使用指定网络接口连接服务器
--compress	压缩服务器与客户端直接传输的数据
--default-character-set=charaset_name	设置默认字符集
--host=host_name	连接到 host 指定的服务器主机
--password=[password],-p	使用密码连接服务器
--port=port_num	使用特定端口号连接服务器
--silent	静默模式
--user=user_name	使用指定账户连接服务器

3. mysqldump 工具

mysqldump 是一个数据库逻辑备份程序，我们可以使用它对一个或多个 MySQL 数据库进行备份或将数据库传输至其他 MySQL 服务器。执行 mysqldump 时需要账户拥有 SELECT 权限

才可以备份数据表，SHOW VIEW 权限用于备份视图，TRIGGER 权限用于备份触发器。某些命令选项可能还需要更多的权限才可以完成操作。mysqldump 不是大数据备份的解决方案，因为 mysqldump 需要通过重建 SQL 语句来实现备份功能，对于数据量比较大的数据库备份与还原操作，速度都比较慢。打开 mysqldump 备份文件会发现，其实里面就是数据库的 SQL 语言重现。

对大规模数据备份与还原，使用物理备份会更加适合，通过直接复制数据文件，即可实现快速的数据还原工作。在企业版本的 MySQL 中提供了一个 mysqlbackup 的工具也是可以考虑的解决方案，它提供了各种不同的 MySQL 数据库引擎高性能备份还原功能。

使用 mysqldump 可以备份数据库中的某些数据表，也可以备份整个数据库（不要在数据库后使用数据表名称），还可以备份 MySQL 系统中的所有数据库。对于使用 mysqldump 工具备份的数据库文件，可以使用 mysql 命令工具还原数据。

语法格式如下：

```
mysqldump [选项] db_name [table_name]
mysqldump [选项] --databases db_name …
mysqldump [选项] --all-databases
```

mysqldump 支持表 4-20 列出的所有选项，这些选项也可以通过[mysqldump]和[client]写入配置文件。

表 4-20

选项	描述
--add-drop-database	在备份文件中添加、删除相同数据库的 SQL 语句
--add-drop-table	在备份文件中添加、删除相同数据表的 SQL 语句
--add-drop-trigger	在备份文件中添加、删除相同触发器的 SQL 语句
--add-locks	在备份数据表前后添加表锁定与解锁 SQL 语句
--all-databases	备份所有数据库中的所有数据表
--apply-slave-statements	在 CHANGE MASTER 前添加 STOP SLAVE 语句
--bind-address=ip_address	使用指定的网络接口连接 MySQL 服务器
--comments	为备份文件添加注释
--create-options	在 CREATE TABLE 语句中包含所有的 MySQL 特性
--databases	备份若干个指定的数据库
--debug	创建 debugging 日志
--default-character-set=charsename	设置默认字符集
--host,-h	设置需要连接的主机

续表

选项	描述
--ignore-table	设置不需要备份的数据表，该选项可以使用多次
--lock-all-tables	设置全局锁，锁定所有的数据表以保证备份数据的完整性
--no-create-db,-n	只导出数据而不创建数据库
--no-create-info	只导出数据而不创建数据表
--no-date	不备份数据内容，用于备份表结构
--password,-p	使用密码连接服务器
--port=port_num	使用指定端口号连接服务器
--replace	使用 REPLACE 语句替代 INSERT 语句

命令范例如下。

备份所有数据库：

```
[root@centos6 ~]# mysqldump -u root -p --all-database > all_database_sql
```

备份 mysql 数据库：

```
[root@centos6 ~]# mysqldump -u root -p --database mysql > mysql_database_sql
```

备份 mysql 数据库下的 user 数据包：

```
[root@centos6 ~]# mysqldump -u root -p mysql user > user_table
```

使用 all_database_sql 数据库备份文件还原数据库：

```
[root@centos6 ~]# mysql -u root -p < all_database_sql
```

使用 mysql_database_sql 数据库备份文件还原数据库：

```
[root@centos6 ~]# mysql -u root -p mysql < mysql_database_sql
```

使用 user table 数据库备份文件还原数据表：

```
[root@centos6 ~]# mysql -u root -p mysql < user_table
```

4.11.4 数据库定义语言

MySQL 使用 SQL（结构化查询语言）作为自己的数据库操作语言。该语言是对数据库的定义与操作的语法结构，目前 SQL 是大多数关系型数据库系统的工业标准。也就是说，不管我们使用哪种关系型数据库系统，它们大多数都支持 SQL 语言，这是一个通用的关系型数据库语言。结构化查询语言主要分为六部分：数据查询语言、数据操作语言、数据定义语言、数据控制语言、事务处理语言、指针控制语言。下面我们会分别对常用的 SQL 语句进行详细介绍，所

有的 SQL 语句需要通过客户端软件输入，最后连接服务器运行指令代码。本书所有的 SQL 语句均通过 mysql 客户端软件实现，SQL 语句不区分大小写。

1. CREATE DATABASE

命令描述：该语句用来创建数据库，使用该语句需要执行者拥有 CREATE 权限。

语法格式：CREATE {DATABASE|SCHEMA} [IF NOT EXISTS] db_name [create_specification] …

演示案例：创建名为 hr 的数据库，使用 show databases 显示 MySQL 所有的数据库列表。

```
[root@centos6 ~]# mysql -u root -p
mysql> create database hr;
Query OK, 1 row affected (0.00 sec)
mysql> show databases;
+--------------------------------+
| Database                       |
+--------------------------------+
| information_schema             |
| hr                             |
| mysql                          |
| performance_schema             |
+--------------------------------+
4 rows in set (0.00 sec)
```

2. CREATE TABLE

命令描述：该语句用来在数据库中创建数据库，需要先使用 use db_name 进入数据库。

语法格式如下。

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name (create_definition,…)
[table_options] [opartition_options]
```

或者

```
    CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name (create_definition,…)
[table_options] [opartition_options] select_statement
```

常用数据类型如下。

TINYINT(n)	8 位整数类型
SMALLINT(n)	16 位整数类型
MEDIUMINT(n)	32 位整数类型
INT(n)	32 位整数类型

BIGINT(n)	64 位整数类型
FLOAT(n,d)	单精度浮点数
DOUBLE(n,d)	双精度浮点数
DATE	日期格式
TIME	时间格式
char(n)	固定长度字串
varchar(n)	非定长字串
BIT	二进制数据
BLOB	非定长二进制数据

常用属性如下。

NOT NULL	要求数据为非空值
AUTO_INCREMENT	用户插入新的数据后对应整数数据列自动加 1
PRIMARY KEY	创建主索引列
KEY	普通索引列
DEFAULT CARSET	设置默认字符集
ENGINE	设置默认数据库存储引擎

演示案例：使用 use 语句打开 hr 数据库，使用 create table 创建名为 employees 的数据表。

```
mysql> use hr;
Database changed
mysql> create table employees (
    -> employee_id INT NOT NULL AUTO_INCREMENT,
    -> first_name char(20) NOT NULL,
    -> last_name char(20) NOT NULL,
    -> e_mail varchar(50),
    -> telephone INT(15),
    -> department varchar(20),
    -> hire_date date,
    -> PRIMARY KEY (employee_id),
    -> KEY (department))
    -> ENGINE=innodb DEFAULT CHARSET=UTF8;
mysql> CREATE TABLE test (c CHAR(20) CHARACTER SET utf8 COLLATE utf8_bin);
mysql> CREATE TABLE new_user SELECT * FROM mysql.user;
mysql> show tables;
+----------------- +
| Tables_in_hr      |
+----------------- +
| employees         |
| new_user          |
```

```
| test                |
+-------------------- +
rows in set (0.00 sec)
```

DESCRIBE 语句用于查看数据表的数据结构，使用该语句可以快速了解一个数据表的基本结构。

```
mysql> DESCRIBE employees;
+---------------------+--------------------+----------+----------+-------------------+-----------------------+
| Field               | Type               | Null     | Key      | Default           | Extra                 |
+---------------------+--------------------+----------+----------+-------------------+-----------------------+
| employee_id         | int(11)            | NO       | PRI      | NULL              | auto_increment|
| first_name          | char(20)           | NO       |          | NULL              |                       |
| last_name           | char(20)           | NO       |          | NULL              |                       |
| e_mail              | varchar(50)        | YES      |          | NULL              |                       |
| telephone           | int(15)            | YES      |          | NULL              |                       |
| department          | varchar(20)        | YES      | MUL      | NULL              |                       |
| hire_date           | date               | YES      |          | NULL              |                       |
+---------------------+--------------------+----------+----------+-------------------+-----------------------+
7 rows in set (0.00 sec)
```

3. ALTER DATABASE

命令描述：该语句用来修改数据库属性，属性被保存在数据库目录的 db.opt 文件中。

语法格式：ALTER {DATABASE |SCHEMA } [db_name] alter_specification …

演示案例：使用 alter 语句修改数据库默认的字符集及排序规则。

```
mysql> ALTER DATABASE hr DEFAULT CHARACTER SET=UTF8;
mysql> ALTER DATABASE hr DEFAULT COLLATE=utf8_general_ci;
```

4. ALTER TABLE

命令描述：该语句用来修改数据表结构，如：添加删除的列、创建删除的索引、修改数据类型等。

语法格式：ALTER [IGNORE] TABLE tbl_name [alter_specification [, alter_specification] …]

演示案例：创建数据包 test1 后，通过 alter 语句修改数据表的相关信息。

```
mysql> CREATE TABLE test1 (id INT, name CHAR(20));
mysql> ALTER TABLE test1 RENAME test2;
mysql> ALTER TABLE test2 ADD date TIMESTAMP;
mysql> ALTER TABLE test2 ADD note CHAR(50);
mysql> ALTER TABLE test2 ADD INDEX (date);
mysql> ALTER TABLE test2 MODIFY id TINYINT NOT NULL, CHANGE name first_name
CHAR(20);
```

```
mysql> AlTER TABLE test2 ADD PRIMARY KEY (id);
mysql> ALTER TABLE test2 DROP COLUMN note;
```

5. DROP TABLE

命令描述：该语句可以用来删除一个或多个数据表，所有的表数据以及表定义都将被删除。

语法格式：DROP [TEMPORARY] TABLE [IF EXISTS] tbl_name [, tbl_name] …

演示案例：删除名为 hr 的数据表。

```
mysql> DROP table hr;
```

6. DROP INDEX

命令描述：该语句用来删除特定数据表中的索引。

语法格式：DROP INDEX index_name ON tbl_name [algorithm_option | Lock_option]…

演示案例：分别从 user 及 tbl_name 表中删除索引。

```
mysql> DROP INDEX 'PRIMARY' ON user;
mysql> DROP INDEX 'index' ON tbl_name;
```

7. RENAME TABLE

命令描述：该语句用来对一个或多个数据表进行重命名。

语法格式：RENAME TABLE tbl_name TO new_tbl_name [, tbl_name2 TO new_tbl_name2] …

演示案例：使用临时表名 temp，将数据表 test1 及数据表 test3 的名称对换。

```
mysql> RENAME TABLE test1 TO temp,
-> test3 TO test1,
->temp TO test3;
```

8. DROP DATABASE

命令描述：该语句将删除数据库以及数据库中的所有数据表，请慎用该语句。

重要提示：使用 DROP DATABASE 删除数据库后，用户权限并不会被自动删除。

语法格式：DROP {DATABASE | SCHEMA } [IF EXISTS] db_name

演示案例：删除名为 hr 的数据库。

```
mysql> DROP DATABASE hr;
```

4.11.5　数据库操作语言

1. INSERT

命令描述：该语句的作用是向数据表中插入一行新的数据。

语法格式：

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
[(col_name,…)]
{VALUES | VALUE} ({expre | DEFAULT},…),(…),…
[ON DUPLICATE KEY UPDATE
    col_name=expr
        [,col_name=expr] …]
```

演示案例：往 employees 数据表中插入数据，数据的具体值跟在 VALUES 后面的括号内，可以使用 INSERT 语句一次插入一条数据记录，也可以同时插入多条数据记录。

```
mysql> INSERT INTO employees (employee_id,first_name,last_name,
-> e_mail,telephone,department,hire_date)
-> VALUES
-> (001,"eric","william","test@gmail.com",01065103488,"tech",20110112);
mysql> INSERT INTO employees (employee_id,first_name,last_name)
-> VALUES
-> (001,"eric","william");
mysql> INSERT INTO employees (employee_id,first_name,last_name)
-> VALUES
-> (001,"eric","william"),(002, "jack","smith"),(003,"lucy","black");
mysql> INSERT INTO employees ()
-> VALUES
-> (001,"eric","william","test@gmail.com",01065103488,"tech",20120112);
mysql> SELECT * FROM hr.employees;
```

2. UPDATE

命令描述：该语句用来更新数据表中现有的数据值（仅修改满足 where 条件的数据记录）。

语法格式：

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference
SET col_name={expre1|DEFAULT} [, col_name2={expr2|DEFAULT}] …
[WHERE where_condition]
```

演示案例：更新 employees 数据包，将 hire_date 的值增加 1；当 employee_id 的值为 2 时，

修改 last_name 的值为 hope。

```
mysql> UPDATE employees SET hire_date=hire_date + 1;
mysql> UPDATE hr.employees SET last_name="hope" WHERE employee_id=2;
```

3. LOAD DATA INFILE

命令描述：该语句用来快速从文本文件中读取数据到数据表中。

语法格式：

```
LOAD DATA [LOW_PRIORITY | CONCURRENT] [LOCAL] INFILE 'file_name'
    [REPLACE | IGNORE]
INTO TABLE tbl_name
[CHARACTER SET charset_name]
[{FIELDS | COLUMNS}
    [TERMINATED BY 'string']
    [[OPTIONALLY] ENCLOSED BY 'char']
    [ESCAPED BY 'char']
]
[LINES
    [STARTING BY 'string']
    [TERMINATED BY 'string']
]
[IGNORE number {LINES | ROWS}]
[(col_name_or_user_var,...)]
[SET col_name = expr,...]
```

分隔符：文件默认使用 Tab 键为列分隔符，行分隔符为换行符。可以使用 FIELDS TERMINATED BY 设置列分隔符，通过 LINES STARTING BY 设置行分隔符。

演示案例：提前创建数据文件/tmp/txt 与 txt2，通过 LOAD 语句加载数据至 employees 数据表。

```
[root@cenos6 ~]# cat /tmp/txt
003 ellis   jim example@gmail.com   01065103488 sale    20130102
mysql> LOAD DATA INFILE '/tmp/txt' INTO TABLE hr.employees;
[root@cenos6 ~]# cat /tmp/txt2
004,berry,john,example@gmail.com,01065103488,admin,20110302
mysql> LOAD DATA INFILE '/tmp/txt2' INTO TABLE hr.employees
 ->FIELDS TERMINATED BY ',';
```

4. DELETE

命令描述：该语句用来把满足条件的数据记录删除并返回删除的记录数量。

语法格式：

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name [WHERE where_condition]
```

演示案例：

```
mysql> USE hr;
mysql> DELETE FROM employees WHERE employee_id=5;
mysql> DELETE FROM employees WHERE first_name= "eric ";
mysql> DELETE a1, a2 FROM t1 AS a1 INNER JOIN t2 AS a2
->WHERE a1.id=a2.id;
```

4.11.6　数据库查询语言

数据库查询用 SELECT 语句实现，该语句的描述和用法如下。

命令描述：该语句用来查询数据表中的数据记录。

语法格式：

```
SELECT
    [ALL | DISTINCT | DISTINCTROW ]
    select_expr [, select_expr ...]
    [FROM table_references
    [WHERE where_condition]
    [ORDER BY {col_name | expr | position} [ASC | DESC]]
    [LIMIT]
```

演示案例如下。

查询 employees 数据表中所有的数据记录：

```
mysql> SELECT * FROM employees;
```

查询 employees 数据表中的 employee_id、first_name、department 记录：

```
mysql> SELECT employee_id,first_name,department FROM employees;
```

查询 test 数据表中 score 列数据的所有平均值：

```
mysql> SELECT AVG(score) FROM test;
```

统计有多少条 employee_id 记录，显示最终统计个数：

```
mysql> SELECT COUNT(employee_id) FROM employees;
```

如果有多条相同的 e_mail 记录，DISTINCT 可以去除重复的记录：

```
mysql> SELECT DISTINCT(e_mail) FROM employees;
```

查询 last_name 为 willian 的所有记录，并显示出相应的 first_name 记录：

```
mysql> SELECT first_name FROM employees WHERE last_name="willian";
```

查询数据表中的所有记录，并按照 hire_date 列排序，DESC 为降序，AES 为升序：

```
mysql> SELECT * FROM employees ORDER BY hire_date DESC;
```

限制仅显示数据记录中的前两行记录：

```
mysql> SELECT * FROM employees LIMIT 2;
```

4.11.7 MySQL 与安全

说到 MySQL 数据库的安全性，你可能会联想到大量的相关话题，下面将对几个关键问题进行概括性描述。

- 安全的一般性因素。包括使用强密码，禁止给用户分配不必要的权限，防止 SQL 注入攻击。
- 安装步骤的安全性。确保安装 MySQL 时指定的数据文件、日志文件、程序文件均被存储在安全的地方，未经授权的人均无法读取或写入数据。
- 访问控制安全。包括在数据库中定义账户及相关权限设置。
- MySQL 网络安全。限制仅允许有效的主机可以连接服务器，并且需要账户权限。
- 数据安全。确保已经对 MySQL 数据库文件、配置文件、日志文件进行了充分且可靠的备份。完善的备份机制是数据安全的前提条件。

MySQL 数据库系统基于访问控制列表（ACLs）进行连接、查询以及其他操作，MySQL 权限列表见表 4-21。所有的账户及密码被保存在 mysql 数据库下的 user 数据表中，我们可以通过 mysqladmin 或使用 SQL 语句添加、删除、修改账户与密码信息。需要注意的是，MySQL 账号访问信息需要包含主机信息，如默认 root 是不允许通过远程主机登录的。下面通过若干实例演示账户的创建与删除。

表 4-21

权限	描述
CREATE	创建数据库、数据表、索引的权限
DROP	删除数据库、数据表、视图的权限
GRANT OPTION	允许为其他账户添加和删除权限
LOCK TABLES	允许用户使用 LOCK TABLES 语句锁定数据表
EVENT	执行 EVENT 的权限

续表

权限	描述
ALTER	修改数据的权限
DELETE	删除数据记录的权限
INDEX	创建删除索引的权限
INSERT	向数据表中插入数据的权限
SELECT	对数据库进行数据查询的权限
UPDATE	更新数据记录的权限
CREATE TEMPORARY TABLES	创建临时表的权限
TRIGGER	执行触发器的权限
CREATE VIEW	创建视图的权限
SHOW VIEW	执行 SHOW CREATE VIEW 的权限
ALTER ROUTINE	修改或删除存储过程的权限
CREATE ROUTINE	创建存储过程的权限
EXECUTE	执行存储过程或函数的权限
FILE	赋予读写服务器主机文件的权限
CREATE TABLESPACE	创建表空间的权限
CREATE USER	创建修改 MySQL 账户的权限
PROCESS	显示服务器运行进程信息的权限
RELOAD	允许用户使用 FLUSH 语句
REPLICATION CLIENT	允许使用 SHOW MASTER STATUS 以及 SHOW SLAVE STATUS
REPLICATION SLAVE	允许从服务器连接当前服务器
SHOW DATABASES	允许使用 SHOW DATABASES 查看数据库信息
SHUTDOWN	允许用户关闭 MySQL 服务
SUPER	允许执行关闭服务器进程之类的管理操作
ALL	代表所有可用的权限

1. 通过 GRANT 命令创建账户并赋予权限

创建 tomcat 账户，账户仅可以通过本机连接 MySQL 服务器，账号密码为 pass，该账户对 hr 数据库中的所有数据表拥有所有权限：

```
mysql> GRANT ALL ON hr.* to 'tomcat'@'localhost' IDENTIFIED BY 'pass';
```

创建 tomcat 账户，账户仅可以从主机 192.168.0.88 连接到 MySQL 服务器，账号密码为 pass，

该账户对 hr 数据库中所有的数据表仅拥有查询权限：

```
mysql> GRANT SELECT ON hr.* to 'tomcat'@'192.168.0.88' IDENTIFIED BY 'pass';
```

创建 admin 账户，赋予该账户对所有数据库的管理权限，该账号不需要密码，即可从本机连接 MySQL 服务器：

```
mysql> GRANT RELOAD,PROCESS ON *.* to 'admin'@'localhost';
```

创建 jacob 账户，赋予 jacob 对 hr 数据库中所有数据表的所有权限，账户密码为 pass，该账户可以从任何主机连接服务器：

```
mysql> GRANT ALL ON hr.* to 'jacob'@'%' IDENTIFIED BY 'pass';
```

通过 SHOW GRANTS 语句可以查看账户权限信息：

```
mysql>SHOW GRANTS FOR jerry@172.16.0.253
```

2. 直接修改数据库权限数据表 mysql.user

更新 mysql 数据库下的 user 数据表，修改 root 密码为 newpwd，PASSWORD()的作用用来对密码进行加密存储，最后使用 FLUSH PRIVILEGES 更新该权限数据表。

```
mysql> UPDATE mysql.user SET Password = PASSWORD('newpwd')
-> WHERE User = 'root';
mysql> FLUSH PRIVILEGES;
```

3. 通过 mysqladmin 设置账户密码

修改 root 登录服务器的密码为 centos，-p 的作用是提示输入旧的密码，若没有旧密码，可以忽略：

```
[root@centos6 ~]# mysqladmin -u root -p password 'centos'
```

4. 通过 SET PASSWORD 命令设置密码

```
[root@centos6 ~]# mysql -u root -p
```

设置 root 从本地连接服务器的密码：

```
mysql> SET PASSWORD FOR 'root'@'localhost' = PASSWORD('newpwd');
mysql> SET PASSWORD FOR 'root'@'127.0.0.1' = PASSWORD('newpwd');
```

设置 root 从 172.16.0.22 主机连接 MySQL 服务器的账户密码：

```
mysql> SET PASSWORD FOR 'root'@'172.16.0.22' = PASSWORD('newpwd');
```

5. 使用 REVOKE 语句撤销账户权限

撤销 tomcat 对 hr 数据库的所有权限。

```
mysql> REVOKE ALL ON hr.* FROM tomcat@'%';
```

撤销 jacob 对所有数据库的查询权限。

```
mysql> REVOKE SELECT ON *.* FROM jacob@'%';
```

6. DROP 语句删除账户

```
mysql> DROP USER jerry@'172.16.0.253';
```

4.11.8　MySQL 数据库备份与还原

备份对于数据库而言是至关重要的。当数据文件发生损坏、MySQL 服务出现错误、系统内核崩溃、计算机硬件损坏或者数据被误删等事件时，使用一种有效的数据备份方案，就可以快速解决以上所有的问题。MySQL 提供了多种备份方案，包括：逻辑备份、物理备份、全备份以及增量备份，你可以选择最适合自己使用的方式备份数据。

物理备份通过直接复制包含有数据库内容的目录与文件实现，这种备份方式适用于对重要的大规模数据进行备份，并且要求实现快速还原的生产环境。典型的物理备份就是复制 MySQL 数据库的部分或全部目录，物理备份还可以备份相关的配置文件。但采用物理备份需要 MySQL 处于关闭状态或者对数据库进行锁操作，防止在备份的过程中改变发送数据。物理备份可以使用 mysqlbackup 对 InnoDB 数据进行备份，使用 mysqlhotcopy 对 MyISAM 数据进行备份。另外，也可以使用文件系统级别的 cp、scp、tar、rsync 等命令。

逻辑备份通过保存代表数据库结构及数据内容的描述信息实现，如，保存创建数据结构以及添加数据内容的 SQL 语句，这种备份方式适用于少量数据的备份与还原。逻辑备份需要查询 MySQL 服务器获得数据结构及内容信息，因为需要查询数据库信息并将这些信息转换为逻辑格式，所以相对于物理备份而言比较慢。逻辑备份不会备份日志、配置文件等不属于数据库内容的资料。逻辑备份的优势在于不管是服务层面、数据库层面还是数据表层面的备份都可以实现，由于是以逻辑格式存储的，所以这种备份与系统、硬件无关。

全备份将备份某一时刻所有的数据，增量备份仅备份某一段时间内发生过改变的数据。通过物理或逻辑备份工具就可以完成完全备份，而增量备份需要开启 MySQL 二进制日志，通过日志记录数据的改变，从而实现增量差异备份。

由于物理备份是系统层面的操作，具体工具可以参考本书前面相关章节的数据复制及数据

同步工具的使用，下面将通过一些案例介绍如何使用 MySQL 提供的工具命令进行逻辑备份。

使用 mysqldump 备份所有的数据库，默认该工具会将 SQL 语句信息导出至标准输出，可以通过重定向将输出保存至文件：

```
[root@centos6 ~]# mysqldump --all-databases > bak.sql
```

备份指定的数据库 db1、db2 以及 db3：

```
[root@centos6 ~]# mysqldump --databases db1 db2 db3 > bak.sql
```

备份 db 数据库，当仅备份一个数据库时，--databases 可以省略：

```
[root@centos6 ~]# mysqldump db4 > bak.sql
```

两者之间的差别在于不使用--databases 选项，则备份输出信息中不会包含 CREATE DATABASE 或 USE 语句。不使用--databases 选项备份的数据文件，在后期进行数据还原操作时，如果该数据库不存在，必须先创建该数据库。

使用 mysql 命令读取备份文件，实现数据还原功能：

```
[root@centos6 ~]# mysql < bak.sql
[root@centos6 ~]# mysql db4 < bak.sql
```

4.12 动态网站架构案例

LAMP（Linux+Apache+MySQL+PHP/Python/Perl）架构是一套强大的网站解决方案，LAMP 是多个开源项目的首字母缩写，LAMP 网站架构主要应用于动态网站的 Web 架构，这种 Web 框架具有通用、跨平台、高性能、高负载、稳定等特性，是目前企业部署网站的首选平台。同时，我们也需要看到 Nginx 在国内的应用越来越成熟，相对于 Apache 而言，Nginx 对于静态文件的响应能力要远远高于 Apache 服务器，所以近几年也兴起了 LNMP 的架构。但对于负载 PHP 的网站而言，Apache 仍然是目前的首选产品。本节将通过实际部署两个开源网站项目讲解 LAMP 的实际应用，两个开源项目均采用 PHP 代码编写，分写是 Discuz!论坛系统以及 WordPress 博客系统。图 4-20 为网站架构案例结构图，通过基于域名的虚拟主机实现在一台 Apache 服务器上运行两个不同的网站平台，论坛的域名为 bbs.example.com，博客的域名为 blog.example.com。

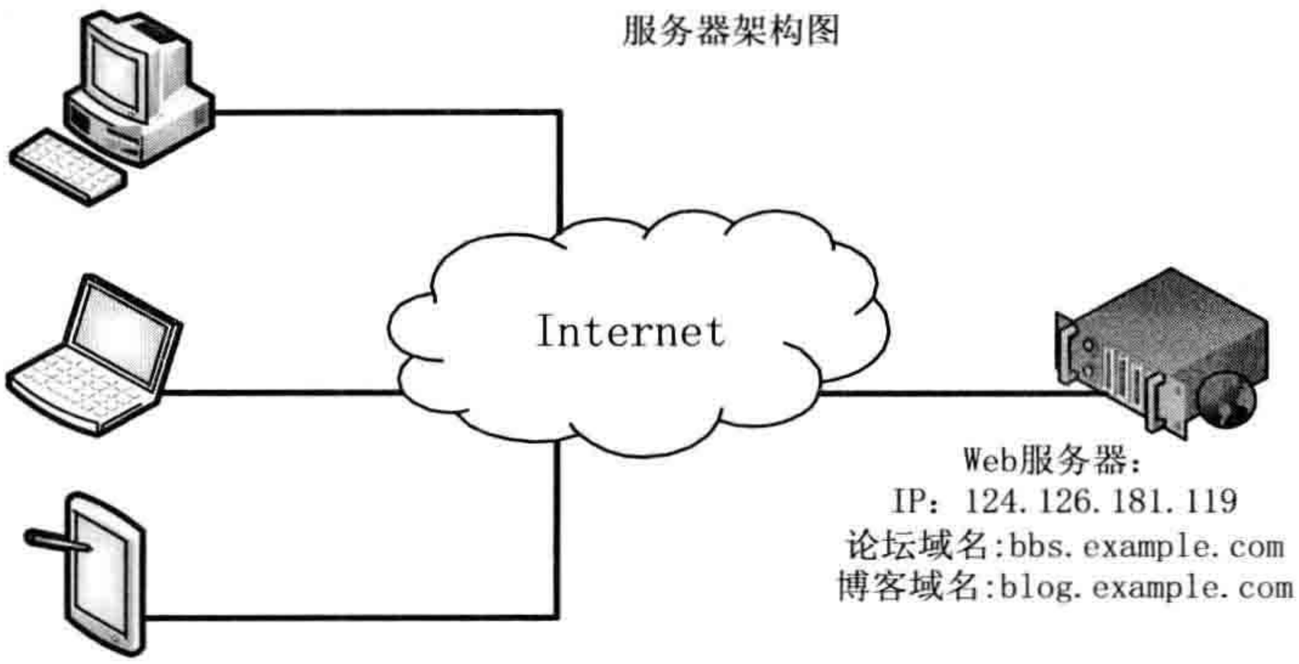


图 4-20

4.12.1　论坛系统应用案例

Discuz!是目前国内应用最广泛的社区论坛建站平台，使用 Discuz!可以实现一站式建站服务，Discuz!从 2001 年发布至今已经历了 12 年的发展历史，大量的应用案例证明了该系统的成熟度、稳定性及负载能力都是值得信赖的。另外，由于 Discuz!采用的是 PHP 语言开发，并且开放源代码，所以 Discuz!也是 PHP 编程人员学习 PHP 代码的模板。

部署 Discuz!论坛所需软件环境包括：WWW 服务器（如，Apache、Nginx 等）、PHP 软件包（版本要求在 4.3.0 以上）、MySQL 数据库（版本要求在 3.23 及以上）。这里将采用 LAMP 平台实现论坛系统的部署，具体步骤如下（所有的软件包均可在对应的官方网站找到，下载 wget 即可）。

1. 下载软件包

```
[root@centos6 ~]# wget http://mirror.bit.edu.cn/apache/httpd/httpd-2.4.4.tar.gz
[root@centos6 ~]# wget http://mirror.bjtu.edu.cn/apache/apr/apr-1.4.6.tar.gz
[root@centos6 ~]# wget http://mirror.bjtu.edu.cn/apache/apr/apr-util-1.5.2.tar.gz
[root@centos6 ~]# wget http://cdn.mysql.com/Downloads/MySQL-5.6/mysql-5.6.11.tar.gz
[root@centos6 ~]# wget http://cn2.php.net/get/php-5.4.14.tar.gz/from/this/mirror
```

2. 部署 AMP 环境

将所有的源码包软件解压到特定的目录下：

```
[root@centos6 ~]# tar -xzf httpd-2.4.4.tar.gz -C /usr/src/
[root@centos6 ~]# tar -xzf apr-1.4.6.tar.gz -C /usr/src/
[root@centos6 ~]# tar -xzf apr-util-1.5.2.tar.gz -C /usr/src/
```

```
[root@centos6 ~]# tar -xzf mysql-5.6.11.tar.gz -C /usr/src/
[root@centos6 ~]# tar -xzf php-5.4.14.tar.gz -C /usr/src/
```

安装依赖包：

```
[root@centos6 ~]# yum -y install gcc gcc-c++ autoconf automake make cmake \
> pcre pcre-devel openssl openssl-devel ncurses-devel libxml2-devel bison \
> zlib-devel libtool-ltdl-devel libtool flex
```

安装 Apache HTTP Server 软件（具体的安装说明见 4.9.2 节）：

```
[root@centos6 ~]# cd /usr/src/apr-1.4.6/
[root@centos6 apr-1.4.6]# ./configure
[root@centos6 apr-1.4.6]# make && make install
[root@centos6 apr-1.4.6]# cd /usr/src/apr-util-1.5.2/
[root@centos6 apr-util-1.5.2]# ./configure  --with-apr=/usr/local/apr/
[root@centos6 apr-util-1.5.2]# make && make install
[root@centos6 apr-util-1.5.2]# cd /usr/src/httpd-2.4.4/
[root@centos6 httpd-2.4.4]# ./configure --prefix=/usr/local/apache2 --enable-so \
> --enable-ssl  --enable-rewrite --with-mpm=worker --with-suexec-bin \
> --with-apr=/usr/local/apr/
[root@centos6 httpd-2.4.4]# make && make install
```

安装 MySQL 数据库系统（具体的安装说明见 4.11.12 节）：

```
[root@centos6 mysql-5.6.11]# cd /usr/src/mysql-5.6.11/
[root@centos6 mysql-5.6.11]# groupadd mysql
[root@centos6 mysql-5.6.11]# useradd -r -s /sbin/nologin -g mysql mysql
[root@centos6 mysql-5.6.11]# cmake . -DENABLE_DOWNLOADS=1
[root@centos6 mysql-5.6.11]# make && make install
[root@centos6 mysql-5.6.11]# chown -R mysql.mysql /usr/local/mysql
[root@centos6 mysql-5.6.11]# /usr/local/mysql/scripts/mysql_install_db --user=mysql \
> --basedir=/usr/local/mysql/ --datadir=/usr/local/mysql/data
[root@centos6 mysql-5.6.11]# cp /usr/local/mysql/my.cnf /etc/my.cnf
[root@centos6 mysql-5.6.11]# cp /usr/local/mysql/support-files/mysql.server \
> /etc/init.d/mysqld
[root@centos6 mysql-5.6.11]# service mysqld start
[root@centos6 mysql-5.6.11]# chkconfig --add mysqld
[root@centos6 mysql-5.6.11]# chkconfig mysqld on
[root@centos6 mysql-5.6.11]# echo "/usr/local/mysql/lib" >> /etc/ld.so.conf
[root@centos6 mysql-5.6.11]# ldconfig
```

Discuz 是基于 PHP 的动态网站，很多数据都需要被保存在数据库中。因此，我们需要在

MySQL 数据库中为论坛创建论坛数据库，并创建数据库管理账户及密码：

```
[root@centos6 ~]# /usr/local/mysql/bin/mysql -u root
mysql> create database bbs;
mysql> GRANT ALL ON bbs.* TO 'bbs_admin'@'localhost' IDENTIFIED BY 'admin123';
mysql> exit
```

安装 PHP 程序：

```
[root@centos6 mysql-5.6.11]# cd /usr/src/php-5.4.14/
[root@centos6 php-5.4.14]# ./configure --prefix=/usr/local/php5 \
>--with-mysql=/usr/local/mysql/ \
>--with-apxs2=/usr/local/apache2/bin/apxs \
>--enable-mbstring \
>--enable-sockets
[root@centos6 php-5.4.14]# make
[root@centos6 php-5.4.14]# make install
[root@centos6 php-5.4.14]# cp php.ini-development /usr/local/php5/lib/php.ini
```

修改 Apache 主配置文件 httpd.conf 以确保该文件中有如下两条指令，这两条指令的功能是加载模块并设置文件类型：

```
LoadModule php5_module  modules/libphp5.so
AddType application/x-httpd-php .php
```

测试（服务器生成 PHP 测试页面，客户端通过 IP 地址[6]访问测试页面）：

```
[root@centos6 ~]# echo "<?php phpinfo(); ?>" > /usr/local/apache2/htdocs/index.php
[root@client ~]# firefox http://124.126.181.119/index.php
```

3. 初始化论坛系统

首先，修改 Apache 主配置文件，通过 DirectoryIndex index.php index.html 指令设置默认首页为 index.php，去除主配置文件 Include conf/extra/httpd-vhosts.conf 指令前的“#”符号。Include 指令将读取 httpd-vhosts.conf 文件的内容作为主配置文件的一部分。

其次，修改虚拟主机配置文件 httpd-vhosts.conf 的内容如下：

```
[root@centos6 ~]# vim /usr/local/apache2/conf/extra/httpd-vhosts.conf
```

```
<VirtualHost *:80>
    ServerAdmin Jacob_test@gmail.com
    DocumentRoot "/usr/local/apache2/htdocs/bbs"
```

[6] 读者需要根据自己的实际情况使用自己的 IP 地址。

```
    ServerName bbs.example.com
ErrorLog "logs/bbs.example.com-error_log"
CustomLog "logs/bbs.example.com-access_log" common
</VirtualHost>
```

```
[root@centos6 ~]# mkdir -p /usr/local/apache2/htdocs/bbs/logs
```

接下来，需要从 http://www.comsenz.com/downloads/install/discuzx 中下载 Discuz!软件包，本书选择的版本为 Discuz! X2.5，下载后解压将 upload 中的文件上传至 bbs 目录。

```
[root@centos6 ~]# mkdir /usr/src/discus
[root@centos6 ~]# unzip /root/Discuz_X2.5_SC_UTF8.zip -d /usr/src/discuz/
[root@centos6 ~]# cp -r /usr/src/discuz/upload/* /usr/local/apache2/htdocs/bbs/
[root@centos6 ~]# chmod -R 777 /usr/local/apache2/htdocs/bbs/{config/,data/}
[root@centos6 ~]# chmod -R 777 /usr/local/apache2/htdocs/bbs/uc_client/data/cache/
[root@centos6 ~]# chmod -R 777 /usr/local/apache2/htdocs/bbs/uc_server/data/
[root@centos6 ~]# iptables -I INPUT -p tcp --dport 80 -j ACCEPT
[root@centos6 ~]# service iptables save
[root@centos6 ~]# /usr/local/apache2/bin/apachectl restart
```

最后，客户端通过浏览器访问 http://bbs.example.com/install/，完成初始化操作，图 4-21 至图 4-24 展示了整个初始化的全部过程。其中，图 4-22 为安装环境检查，包括 PHP 版本、磁盘空间、文件及目录权限，检查没有问题会显示对钩，有问题的项目则被标记为×，只有修复了问题后才可以继续完成下面的操作。图 4-24 需要填写数据库信息，包括数据库名称、账户及密码信息，因此，需要提前在 MySQL 数据库系统中创建相应的数据库及账户密码信息，管理员密码为论坛管理员密码。

图 4-21

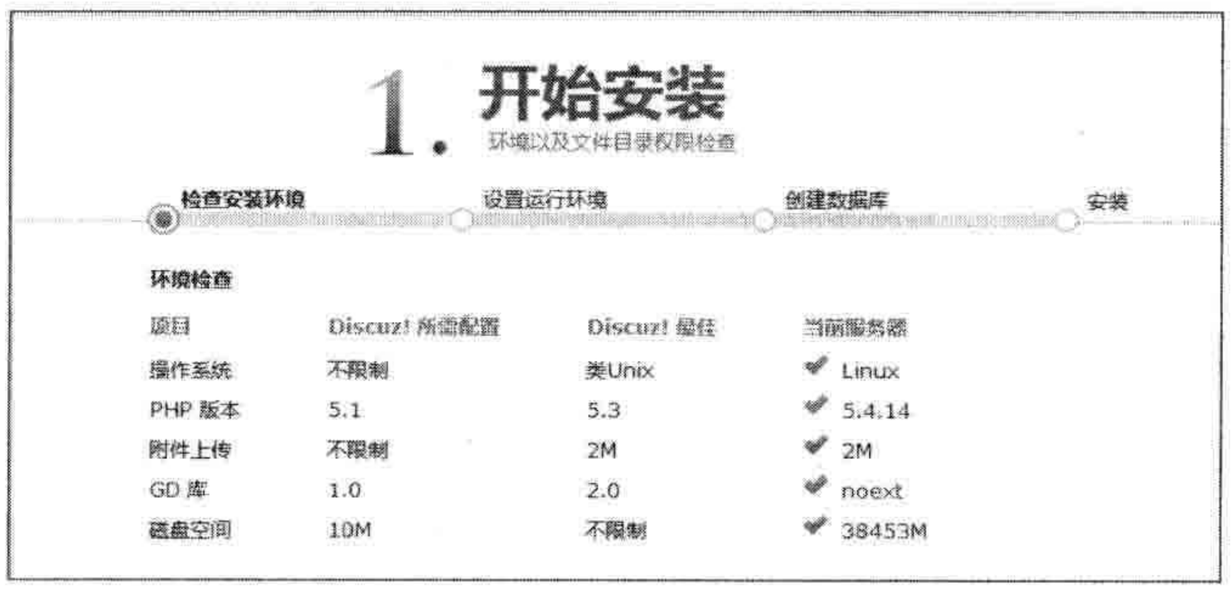


图 4-22

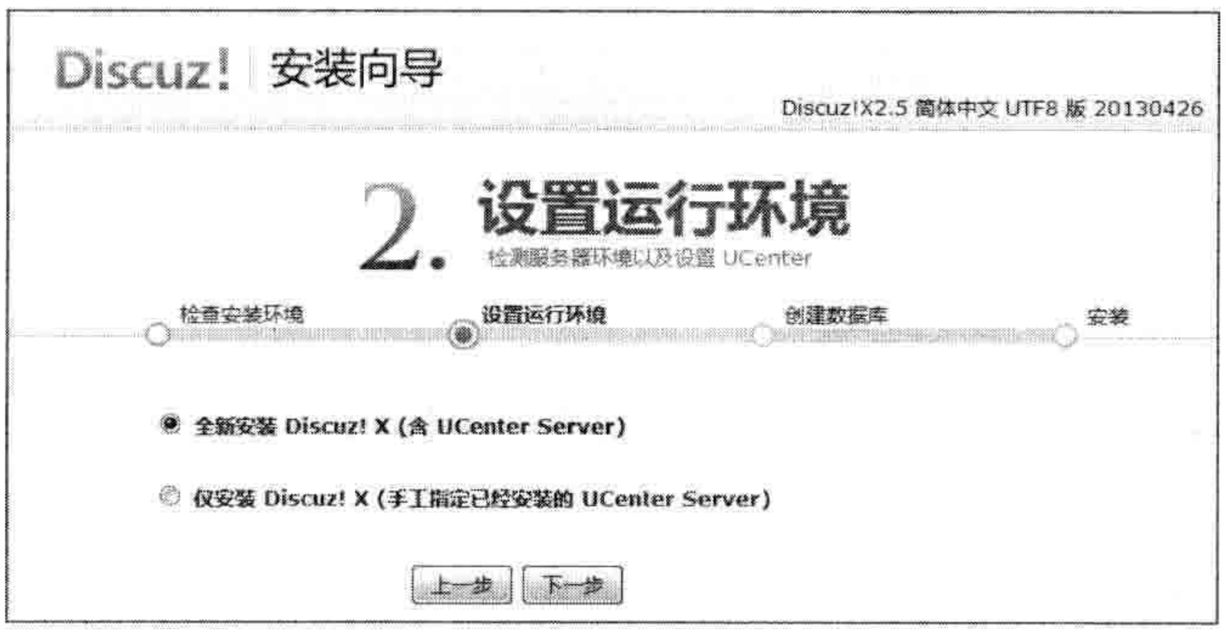


图 4-23

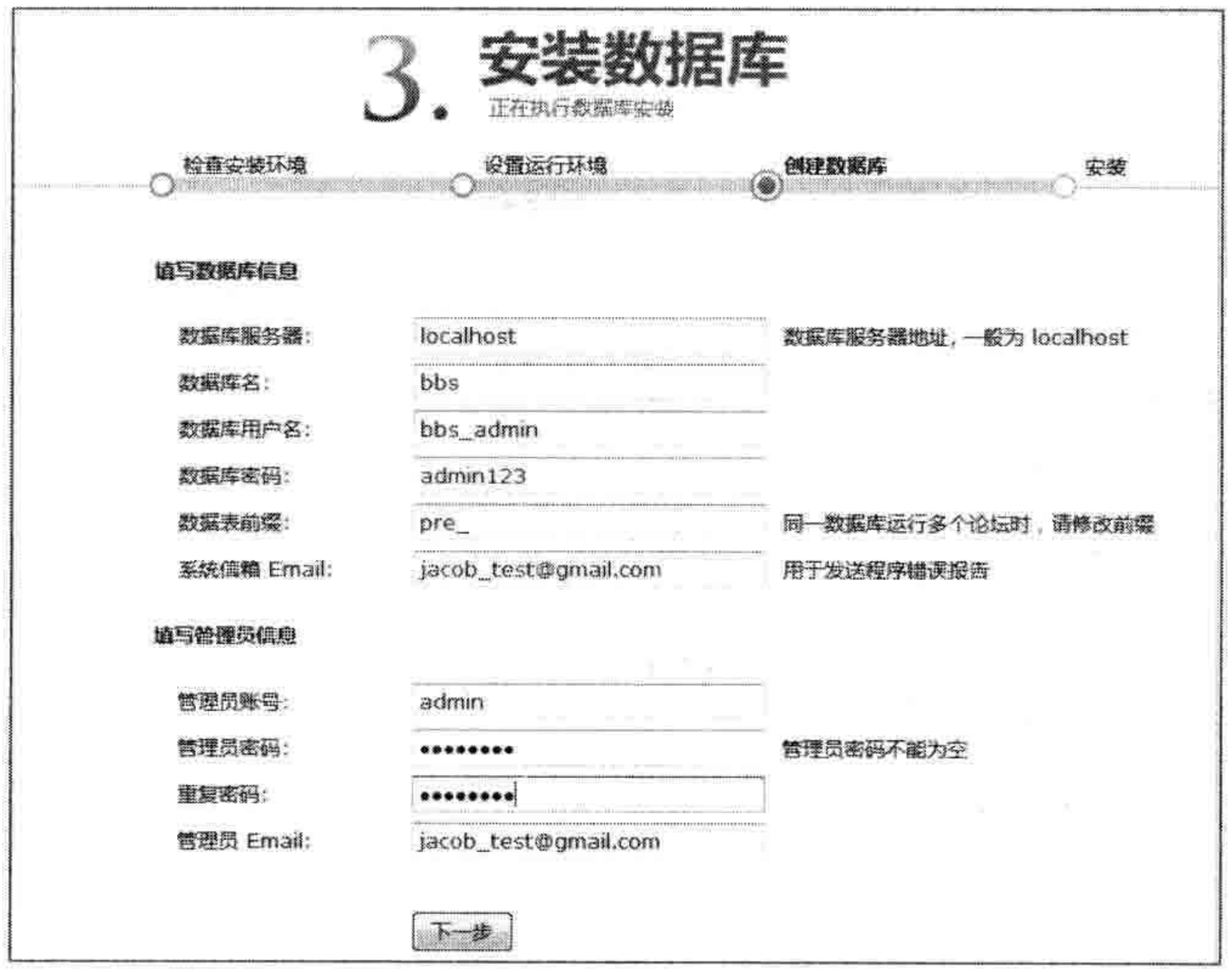


图 4-24

完成以上初始化操作后，一定要将/usr/local/apache2/htdocs/bbs/install/index.php 删除，防止进行多次初始化操作。此时客户端通过浏览器访问论坛域名：http://bbs.example.com/，即可登录论坛首页，首次登录仅包含一个默认板块。

4. 论坛系统设置

论坛初始化完成后，并没有具体板块的内容，管理员可以登录后台进行一些全局、板块、内容、用户等设置。登录论坛首页后单击右上角的管理中心，输入管理员账户及密码后即可进入后台管理界面，后台整体菜单如图 4-25 所示。

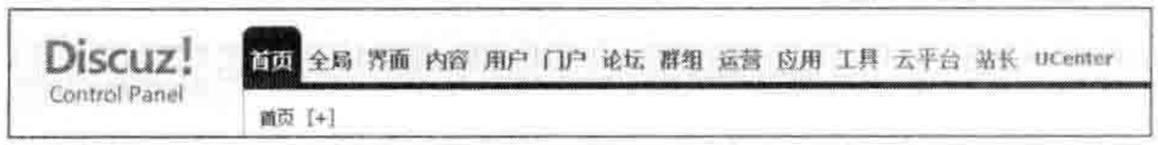


图 4-25

全局菜单：包括站点信息、论坛注册与访问控制、站点功能、性能及 SEO 优化、用户权限、论坛积分、防灌水、上传及水印等设置。

界面菜单：包括论坛首页、导航栏、站点风格、论坛表情管理、提示消息及弹出消息等设置。

内容菜单：包括发帖审核、论坛主题管理、批量删帖、批量群组管理、语言过滤、帖子优化、日志管理、相册与图片管理等设置。

用户菜单：包括用户组、管理组、禁止用户、禁止 IP、积分奖惩、审核用户、推荐关注及好友等设置。

论坛菜单：包括板块管理、板块合并及分类信息设置，板块结构为分区加板块的设计，一个分区下可以创建多个板块内容。

通过后台管理界面的设置，我们可以将论坛设置为与图 4-26 类似的效果。

图 4-26

4.12.2　博客系统应用案例

下面是引用 WordPress 官网的一段简介，这段简介精练地描述了什么是 WordPress 及其主要特色。

WordPress 是一个注重美学、易用性和网络标准的个人信息发布平台。WordPress 虽为免费的开源软件，但其价值无法用金钱来衡量。使用 WordPress 可以搭建功能强大的网络信息发布平台，但更多的是应用于个性化的博客。针对博客的应用，WordPress 能让您省略后台复杂的代码，集中精力做好网站的内容。

部署 WordPress 博客系统非常简单，在论坛案例的基础上额外添加一个 Apache 虚拟站点，初始化博客系统即可，具体步骤如下。

1. 添加虚拟主机

延续上面论坛的案例，继续修改虚拟主机配置文件 httpd-vhosts.conf 内容，添加博客虚拟主机站点如下。

```
[root@centos6 ~]# vim /usr/local/apache2/conf/extra/httpd-vhosts.conf
```

```
<VirtualHost *:80>
    ServerAdmin Jacob_test@gmail.com
    DocumentRoot "/usr/local/apache2/htdocs/bbs"
    ServerName bbs.example.com
ErrorLog "logs/bbs.example.com-error_log"
CustomLog "logs/bbs.example.com-access_log" common
</VirtualHost>

<VirtualHost *:80>
    ServerAdmin Jacob_test@gmail.com
    DocumentRoot "/usr/local/apache2/htdocs/blog"
    ServerName blog.example.com
ErrorLog "logs/blog.example.com-error_log"
CustomLog "logs/blog.example.com-access_log" common
</VirtualHost>
```

```
[root@centos6 ~]# mkdir -p /usr/local/apache2/htdocs/blog/logs
[root@centos6 ~]# /usr/local/apache2/bin/apachectl restart
```

2. 下载 WordPress 并上传至 Apache 虚拟站点目录

```
[root@centos6 ~]# wget http://cn.wordpress.org/wordpress-3.5.1-zh_CN.zip
```

```
[root@centos6 ~]# unzip wordpress-3.5.1-zh_CN.zip -d /usr/src/
[root@centos6 ~]# cp -r /usr/src/wordpress/* /usr/local/apache2/htdocs/blog/
[root@centos6 ~]# chown -R daemon /usr/local/apache2/htdocs/blog
```

3. 创建博客系统数据库及账户密码

```
[root@centos6 ~]# /usr/local/mysql/bin/mysql -u root -p
mysql> create database blog;
mysql> GRANT ALL ON blog.* TO 'blog_admin'@'localhost' IDENTIFIED BY 'admin123';
mysql> exit
```

4. 初始化博客系统

初始化过程比较简单，图 4-27 提示默认没有博客配置文件，通过创建配置文件按钮，系统可以自动创建该文件，但默认配置文件中的数据库信息往往与实际不符，提示信息如图 4-28 所示，通过图 4-29 进行数据库初始化设置，要求输入正确的数据库名称、账户与密码等信息， 最后为博客系统起名、设置管理员账户密码及电子邮件即可，效果如图 4-30 所示。

图 4-27

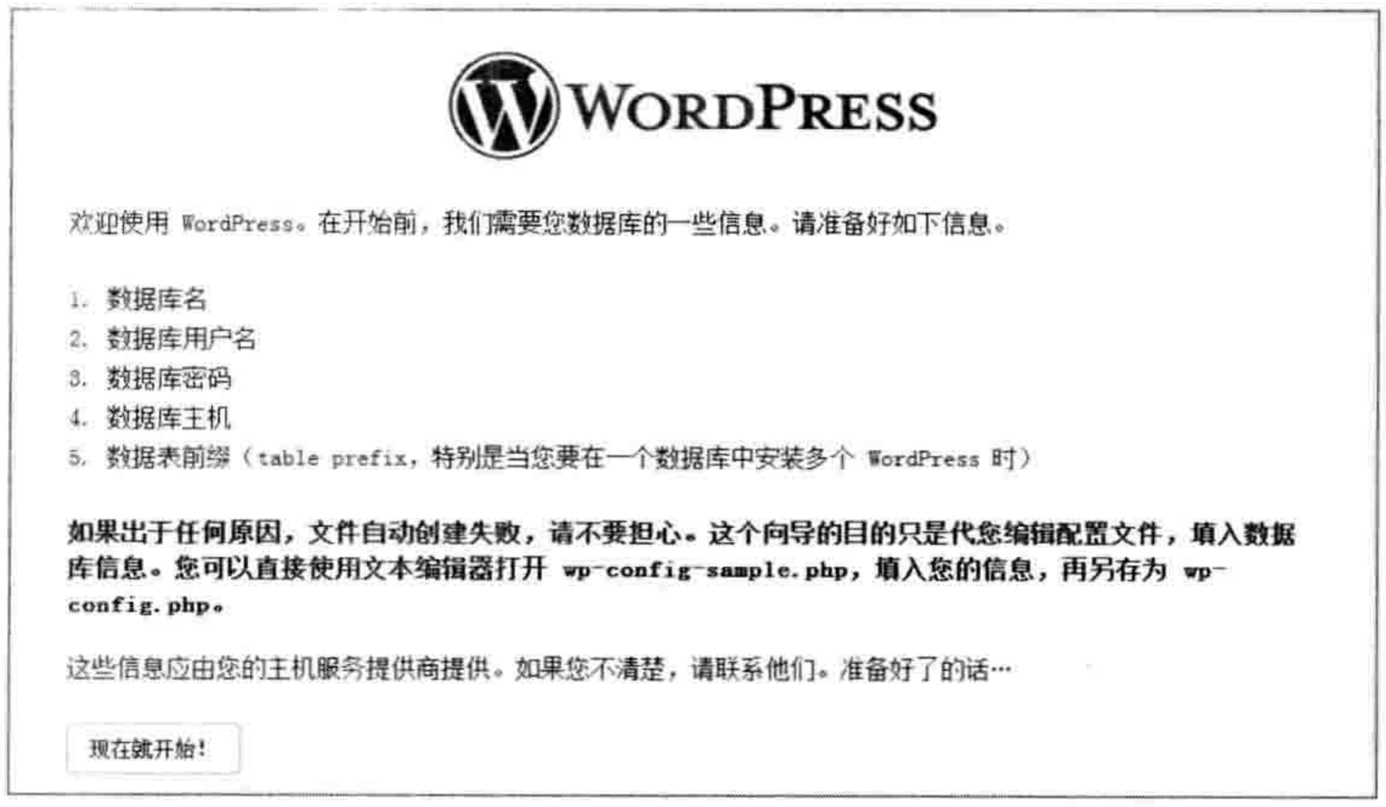


图 4-28

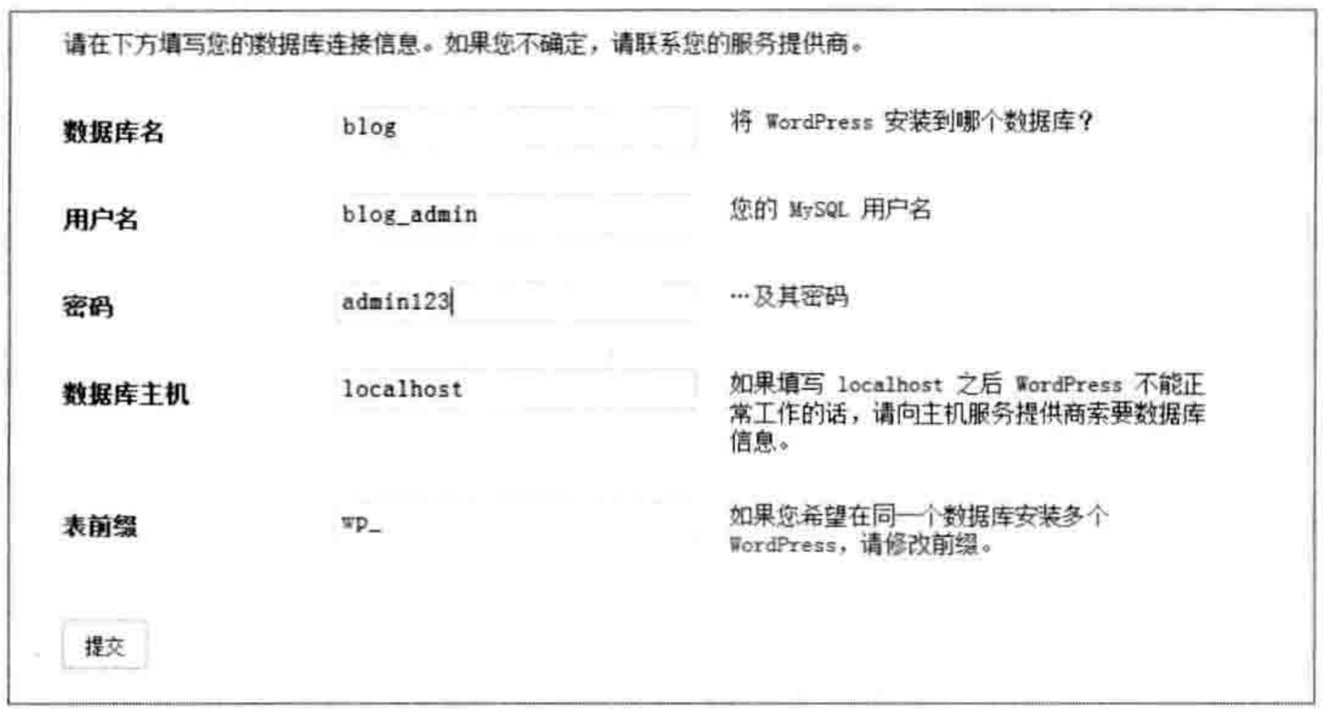


图 4-29

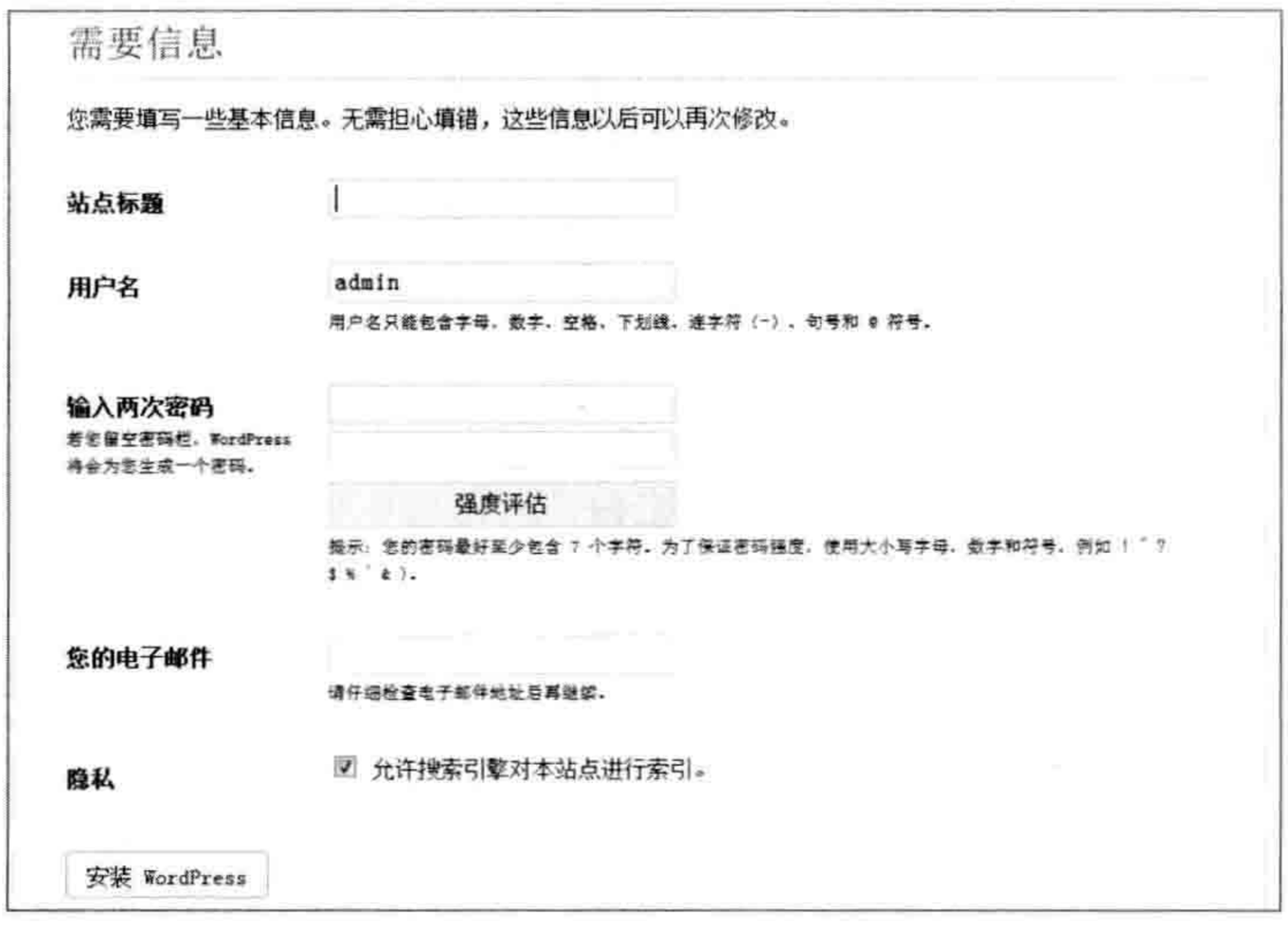


图 4-30

5. 后台管理

浏览博客首页，通过首页的登录链接即可以进入后台管理界面，如图 4-31 所示。管理后台

主要包括博客文章、网站页面、评论、外观、插件、用户等管理功能。WordPress 的优势在于其丰富的插件功能，通过使用插件可以实现博客流量监控、SEO 优化、反垃圾留言等功能。在后台对网站主题、页面进行简单调整后，整体博客平台已经完成，通过设置相应的 DNS 记录后，客户端即可在互联网的任意位置访问该博客系统，客户端访问效果如图 4-32 所示。

图 4-31

图 4-32

第5章

系统监控

5.1 Cacti 监控系统

5.1.1 简介

当公司服务器愈来愈多，规模越来越大时，运维人员了解每台服务器运行状态的工作压力也会越来越大，如果人为地逐个查看每台服务器运行状态肯定是不可行的，此时我们需要一款可以自动收集服务器数据并将所有的数据进行汇总查看的工具软件，最好还可以将搜集到的数据绘制成曲线图。而 Cacti 就是基于 RRDTool 的前端主机性能监控系统，Cacti 前端采用 PHP 代码驱动，通过 SNMP 获得被监控主机的数据信息，使用 RRDTool 存储数据，并根据这些数据绘制图形报表。利用 Cacti，我们不必关心 RRDTool 底层操作就可以通过 Web 页面直接查看被监控主机的性能图表。Cacti 内置的用户管理工具可以创建多个用户并赋予不同的操作权限，每个用户可以独立管理与维护自己的监控设备。图 5-1 为 Cacti 的监控原理图，管理员通过浏览器访问 Cacti，通过 Cacti 管理界面设置定义图表、模板、账户以及被监控节点信息（被监控节点可以是路由、交换、服务器、应用程序），所有的 Cacti 管理数据被存储在 MySQL 数据表中，RRDTool 使用 SNMP 收集被监控节点的数据，这些数据被存储在 rrd 数据库文件中，最终根据

数据生成图表显示在 Cacti 管理界面中。

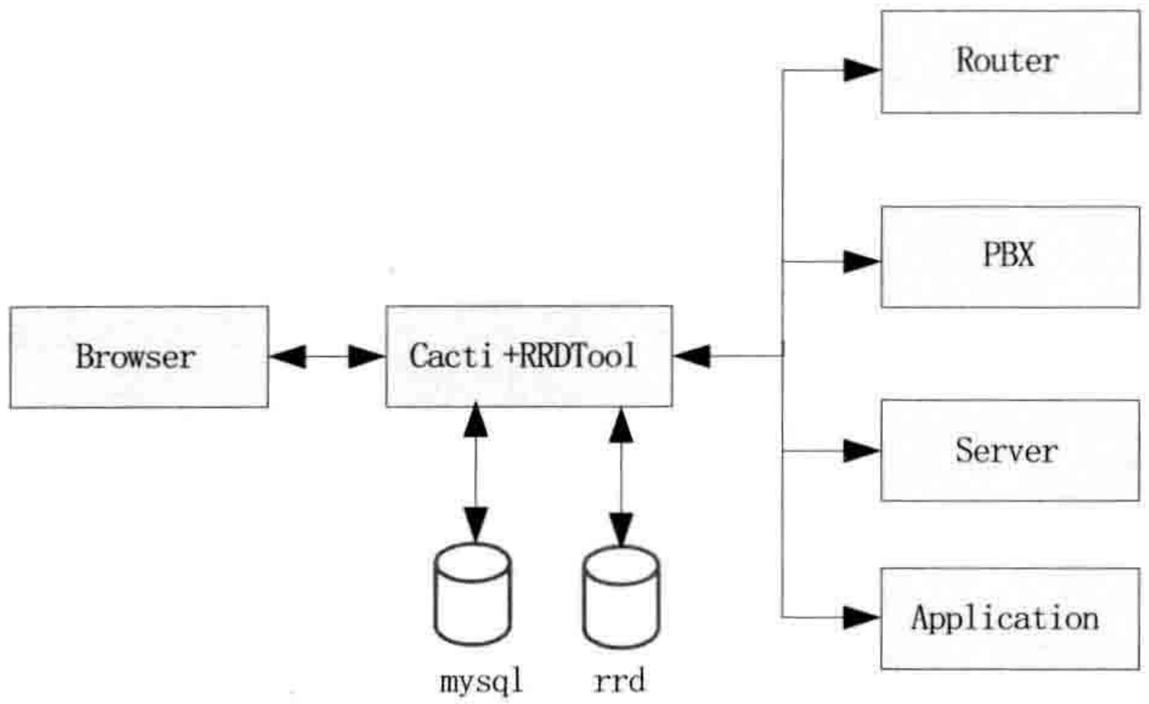


图 5-1

图 5-2 是本节监控案例示意图，整体拓扑结构采用一台 Cacti 监控主机动态监控两台 Web 服务器。案例环境中，管理员从 office_pc 这台计算机上通过浏览器访问 Cacti 服务器提供的 Web 页面，在 Cacti 提供的 Web 管理界面中查看 Web1 以及 Web2 服务器的性能图表。

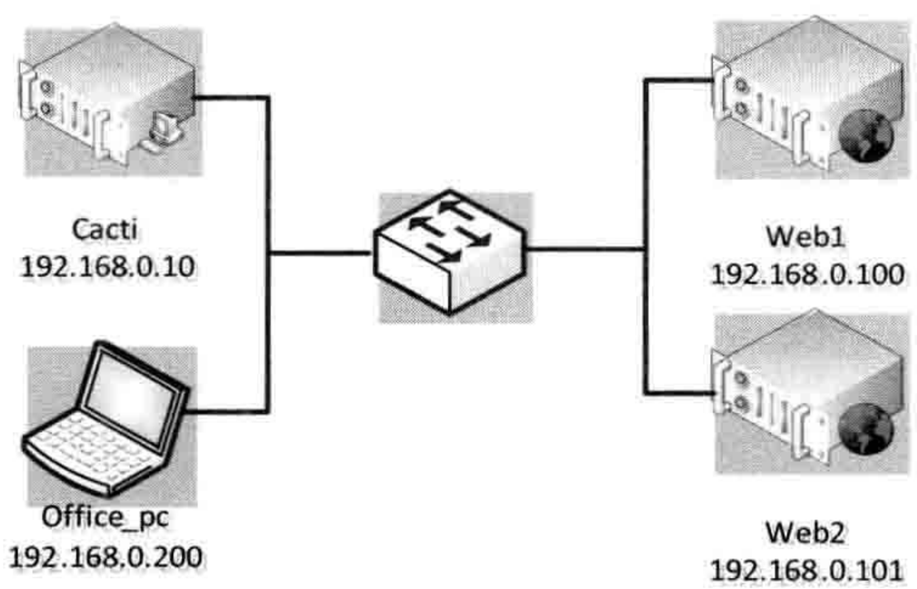


图 5-2

5.1.2 Cacti 监控应用案例

1. 被监控端配置

Cacti 监控其他主机是通过简单的网络管理协议（SNMP）收集被监控端的数据信息的，因此，被监控端 Web1 与 Web2 服务器都需要安装 net-snmp 以及依赖包 lm_sensors，并设置 SNMP 配置文件。lm_sensors 软件可以帮助我们监控硬件信息（如 CPU 的工作电压等数据），具体步骤如下。

安装 snmp 主程序及相关的依赖软件包，被监控主机仅需要修改 snmp 主配置文件即可（根据模板文件，本书对需要修改的地方用黑体字表示）。

```
[root@web1 ~]# yum -y install net-snmp lm_sensors
[root@web1 ~]# vim /etc/snmp/snmpd.conf
```

```
# First, map the community name "public" into a "security name".
com2sec notConfigUser  default       rose0011
# Second, map the security name into a group name.
group   notConfigGroup v1           notConfigUser
group   notConfigGroup v2c           notConfigUser
# Third, create a view for us to let the group have rights to.
view    systemview    included   .1.3.6.1.2.1.1
view    systemview    included   .1.3.6.1.2.1.25.1.1
# Finally, grant the group read-only access to the all view.
access  notConfigGroup ""      any       noauth    exact  all none none
view all    included  .1                                   80
syslocation Unknown (edit /etc/snmp/snmpd.conf)
syscontact Root <root@localhost> (configure /etc/snmp/snmp.local.conf)
dontLogTCPWrappersConnects yes
#备注，以上黑色加粗部分为需要修改的内容
```

SNMP 配置文件主要包括四部分内容需要修改。第一步，映射通信名称到安全用户名称；第二步，将用户加入到组中；第三步，为组创建 systemvies 视图；第四步，创建 all 视图并赋予组只读访问权限。主配置文件设置完成后重启服务，即可通过客户端访问该服务器获得系统信息，SNMP 使用的端口号为 UDP 161 端口以及 TCP 199 端口，我们需要在防火墙规则中开放这些端口数据。

```
[root@web1 ~]# iptables -I INPUT -p udp --dport 161 -j ACCEPT
[root@web1 ~]# iptables -I INPUT -p tcp --dport 199 -j ACCEPT
[root@web1 ~]# service iptables save
[root@web1 ~]# service snmpd start
[root@web1 ~]# chkconfig snmpd start
```

Web1 与 Web2 被监控主机均启动 snmpd 服务后，可以在 Cacti 主机上执行 snmpwalk 命令验证 SNMPD 服务是否运行正常，如果出现如下提示，则说明一切正常（命令输出仅为部分信息）：

```
[root@cacti ~]# snmpwalk -v 1 192.168.0.101 -c rose0011 system
SNMPv2-MIB::sysORID.1 = OID: SNMP-MPD-MIB::snmpMPDMIBObjects.3.1.1
SNMPv2-MIB::sysORID.2 = OID: SNMP-USER-BASED-SM-MIB::usmMIBCompliance
SNMPv2-MIB::sysORID.3 = OID: SNMP-FRAMEWORK-MIB::snmpFrameworkMIBCompliance
SNMPv2-MIB::sysORID.4 = OID: SNMPv2-MIB::snmpMIB
SNMPv2-MIB::sysORID.5 = OID: TCP-MIB::tcpMIB
SNMPv2-MIB::sysORID.6 = OID: IP-MIB::ip
```

2. 安装 Cacti 服务

Cacti 是基于 PHP 的 Web 监控管理系统，所以 Cacti 监控主机需要部署 LAMP 环境，此环境可以参考第 4 章的内容搭建。此外，还需要使用 SNMP 简单网络管理协议从被监控主机提取数据，然后使用 RRDTool 工具保存数据并最终生成图表信息。

安装相关依赖包软件：

```
[root@cacti ~]# yum -y install net-snmp net-snmp-devel net-snmp-utils\
>net snmp-libs lm_sensors php-xml zlib libpng freetype cairo-devel\
>pango-devel gd
```

因为 Cacti 的优势在于其丰富的图表效果，为了将数据绘制成相应的更加直观的图表格式输出，就需要安装 RRDTool 工具，该软件的安装非常简单，使用 configure、make、make install 默认安装即可。

```
[root@cacti ~]# wget http://oss.oetiker.ch/rrdtool/pub/rrdtool-1.4.7.tar.gz
[root@cacti ~]# tar -xzf rrdtool-1.4.7.tar.gz -C /usr/src/
[root@cacti ~]# cd /usr/src/rrdtool-1.4.7/
[root@cacti ~]# ./configure --prefix=/usr/local/
[root@cacti ~]# make && make install
```

设置 Apache 虚拟主机：

因为是基于 Web 的监控平台，而本书 Web 服务器采用的是 Apache HTTP Server，因此，需要修改 Apache 主配置文件 httpd.conf，通过 DirectoryIndex index.php index.html 指令设置默认首页为 index.php，去除 Include conf/extra/httpd-vhosts.conf 指令前的“#”符号。Include 指令将读取 httpd-vhosts.conf 文件的内容作为主配置文件的一部分，并在 httpd-vhosts.conf 文件中加入用来实现监控的虚拟主机。

```
[root@cacti ~]# vim /usr/local/apache2/conf/extra/httpd-vhosts.conf
```

```
<VirtualHost *:80>
    ServerAdmin Jacob_rose@gmail.com
    DocumentRoot "/usr/local/apache2/htdocs/cacti"
    ServerName cacti.example.com
</VirtualHost>
```

修改 PHP 配置文件的时区设置，默认 date.timezone 为注释行，需要手动开启并设置时区，否则进行 Cacti 初始化时会提示 It is not safe to rely on the system's timezone settings。

本书将时区修改为上海，即编辑 PHP 配置文件/usr/local/php5/lib/php.ini，修改其中的时区参数 date.timezone = "Asia/Shanghai"。

部署 Cacti 监控系统，创建 Web 页面根路径 monitor 目录，解压 Cacti 包并移动至 Web 根路

径下，因为 Apache 默认以 daemon 用户及组身份启动，所以需要使用 chown 命令修改目录权限以便 Apache 读取相关页面。

```
[root@cacti ~]# wget http://www.cacti.net/downloads/cacti-0.8.8a.tar.gz
[root@cacti ~]# tar -xzf /root/cacti-0.8.8a.tar.gz -C /usr/src/
[root@cacti ~]# mv /usr/src/cacti-0.8.8a/ /usr/local/apache2/htdocs/cacti
[root@cacti ~]# chown -r daemon.daemon /usr/local/apache2/htdocs/cacti
[root@cacti ~]# iptables -I INPUT -p tcp --dport 80 -j ACCEPT
[root@cacti ~]# service iptables save
[root@cacti ~]# /usr/local/apache2/bin/apachectl restart
```

创建初始化数据库，这里要确保 MySQL 服务器程序是开启的，通过 Cacti 软件包内提供的数据库备份文件，在本机生成名为 cacti 的数据库，创建 cactiuser 账户，并赋予其访问 cacti 数据库的所有权限，该账号密码为 rose0011。

```
[root@cacti ~]# /usr/local/mysql/bin/mysqladmin -u root -p create cacti
[root@cacti ~]# cd /usr/local/apache2/htdocs /cacti
[root@cacti cacti]# /usr/local/mysql/bin/mysql -u root -p cacti < cacti.sql
[root@cacti cacti]# /usr/local/mysql/bin/mysql -u root -p
mysql> GRANT ALL ON cacti.* TO cactiuser@localhost IDENTIFIED BY 'rose0011';
mysql> flush privileges;
mysql> exit
```

设置 Cacti 配置文件，根据上一步数据库初始化操作所创建的数据库信息，修改 Cacti 配置文件中的数据库参数资料，需要修改的内容如下。

```
[root@cacti ~]# /usr/local/apache2/htdocs/cacti/include/config.php
```

```
$database_type = "mysql";              #数据库类型
$database_default = "cacti";           #默认数据库名称
$database_hostname = "localhost";      #数据库服务器主机名称
$database_username = "cactiuser";      #数据库账户
$database_password = "rose0011";       #数据库密码
$url_path = "/";                       #设置监控页面为 Apache 设置的页面根路径
```

3. Cacti 生成图表信息

当被监控主机与监控主机都部署完成后，管理员就可以在任意一台 office_pc 主机上通过浏览器访问 Cacti 管理界面，配置与管理监控服务器。本例中使用域名访问监控主机，在浏览器地址栏中输入 http://cacti.example.com 访问监控服务器，如果读者的实验环境中没有搭建 DNS 服务器，可以通过修改 hosts 文件实现域名解析。第一次登录 Cacti 监控页面时会提示进行基本的初始化操作，图 5-3 提示安装 Cacti 前请仔细阅读相关文档，单击“Next”按钮继续。图 5-4

提示数据库信息并根据实际情况选择是全新安装还是升级。图 5-5 显示 Cacti 所需要的组件信息，如果某些组件未找到，就会以红色显示，图中提示在/usr/bin/php 位置未找到相关文件，而本书编译 PHP 的实际路径为/usr/local/php5/bin/php，将路径修改为正确路径后单击“Next”按钮即可。图 5-6 提示输入账户名与密码，Cacti 初始账户名与密码均为 admin，输入正确后，系统会提示要求强制修改密码，如图 5-7 所示。最后进入 Cacti 管理界面，效果如图 5-8 所示。

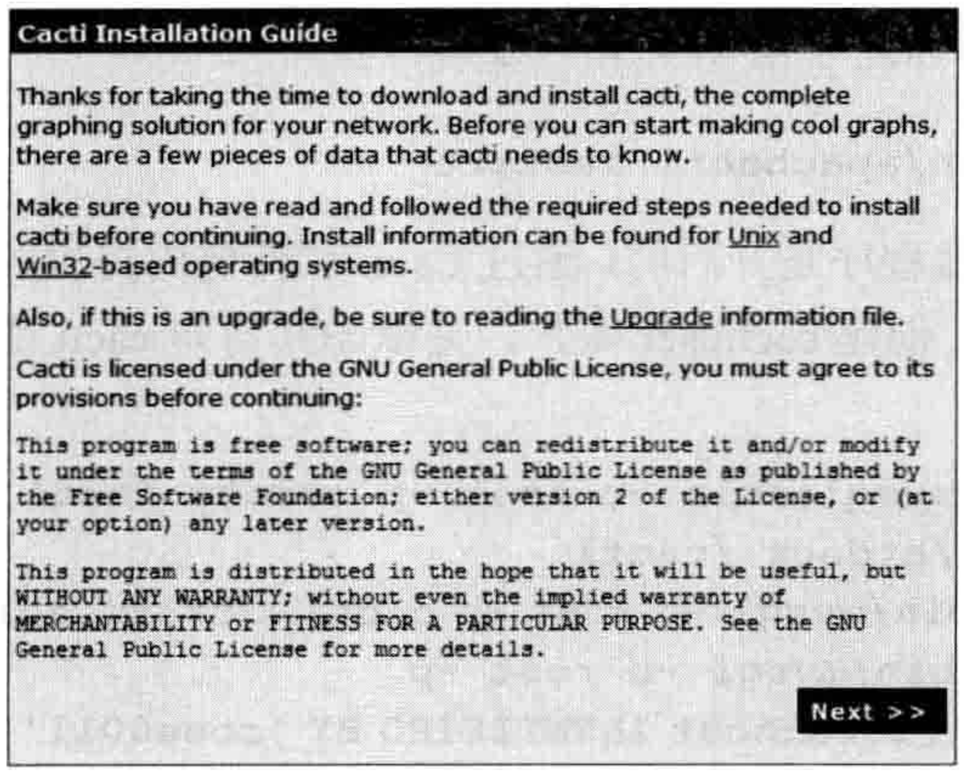


图 5-3

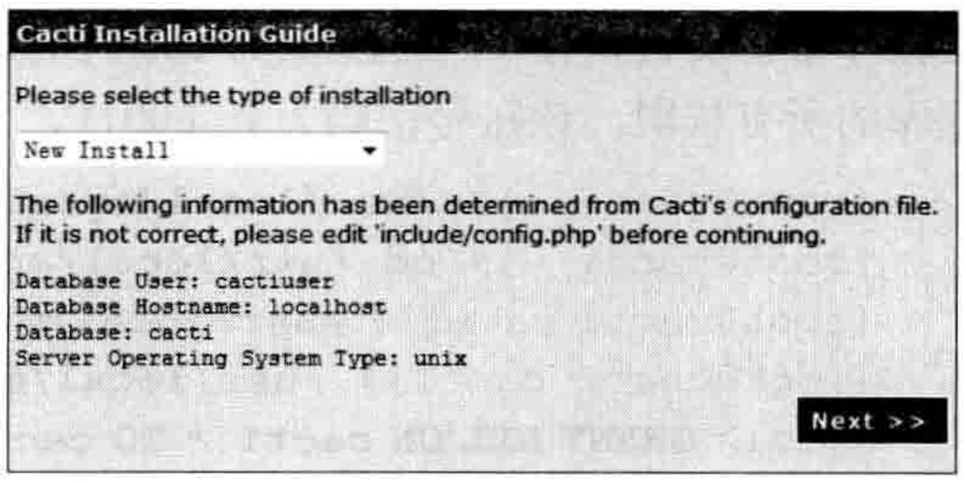


图 5-4

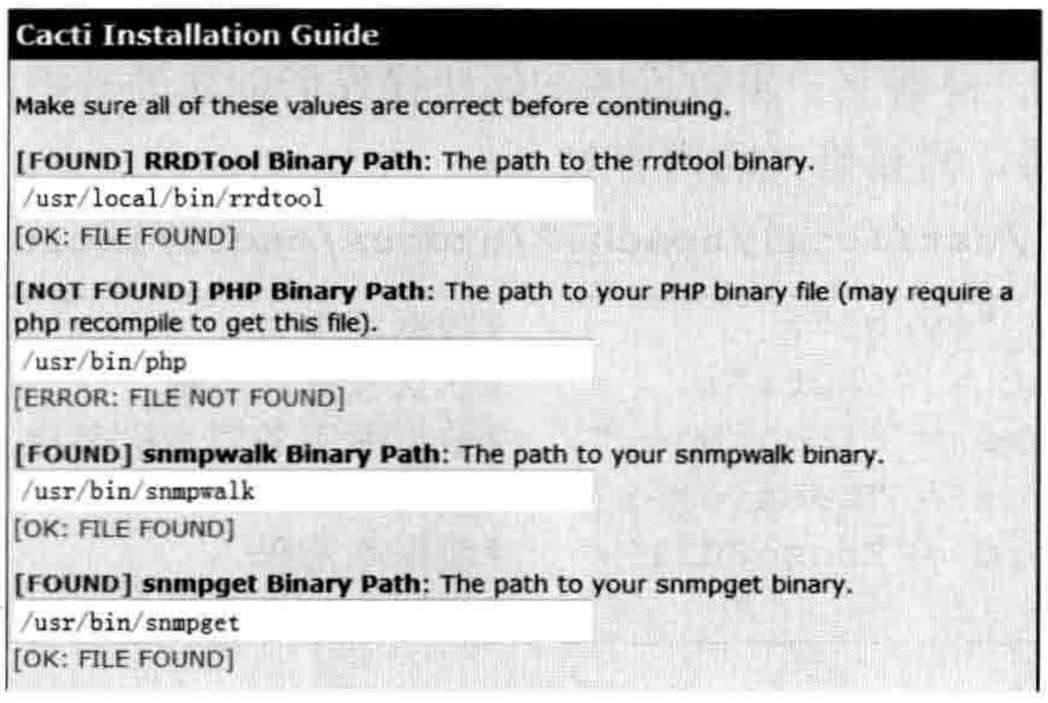


图 5-5

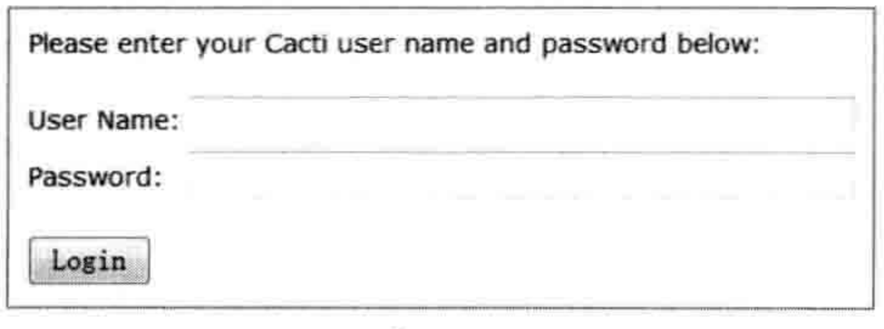


图 5-6

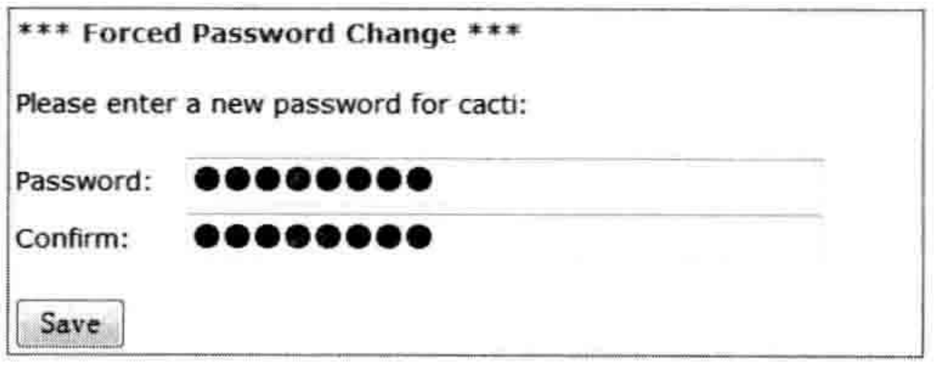


图 5-7

登录 Cacti 管理界面后，如果需要监控其他主机，还需要完成一些简单的设置。首先，需要在管理界面中通过 Devices 菜单将被监控主机添加进来。其次，要为新添加的主机创建监控图形，通过 New Grahps 菜单创建监控图形时可以根据情况有选择地监控对象主机。最后，要创建一棵便于查看图形的图形树，在一棵树下可以加入多个监控对象。

图 5-8

如图 5-9 所示，进入 Devices 菜单后单击“Add”按钮添加被监控的主机，此时系统会要求填写相关的表单信息，需要填写的项目描述见表 5-1，填写完所需项目表单后单击右下角的“Create”按钮，即可创建新的设备主机，同时在下方会出现图形模板以及数据查询模板，管理员可以根据实际需要继续添加需要监控的参数对象，确定后单击“Save”按钮保存所有的设置，效果如图 5-10 所示。本例将 Web1 以及 Web2 两台 Web 服务器主机添加进来。

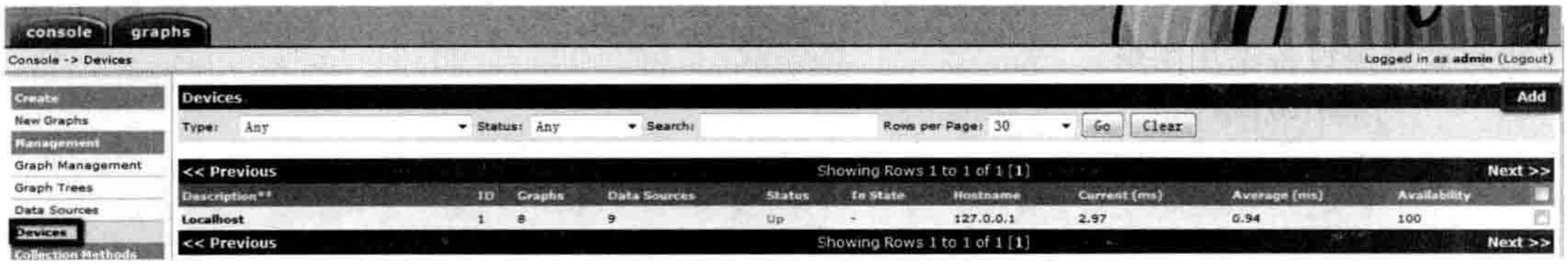


图 5-9

表 5-1

项目名称	描述
Description	设备描述信息，描述被监控主机信息
Hostname	设备名称，可以输入 IP 地址或主机名（需要 DNS 解析）

续表

项目名称	描述
Host Template	定义图形及数据查询模板，案例中选择 ucd/net SNMP Host
Number of Collection Threads	用于搜集数据信息的线程数量，默认即可
Disable Host	停用该主机，仅在不需要监控该主机时选择
Downed Device Detection	宕机检测方式，设置默认值 SNMP Uptime 即可
Ping Timeout Value	测试 Ping 的超时时间，单位为毫秒（ms），默认为 400ms
Ping Retry Count	Ping 重试次数，默认即可
SNMP Version	SNMP 版本，使用默认的 Version 1
SNMP Community	输入被监控端定义的通信名，案例中为 rose0011
SNMP Port	SNMP 端口号，默认 161
SNMP Timeout	SNMP 超时时间，单位为毫秒（ms）
Associated Graph Templates	添加图形监控模板，默认监控 CPU、内存、负载
Associated Data Queries	添加数据查询项目，可以添加

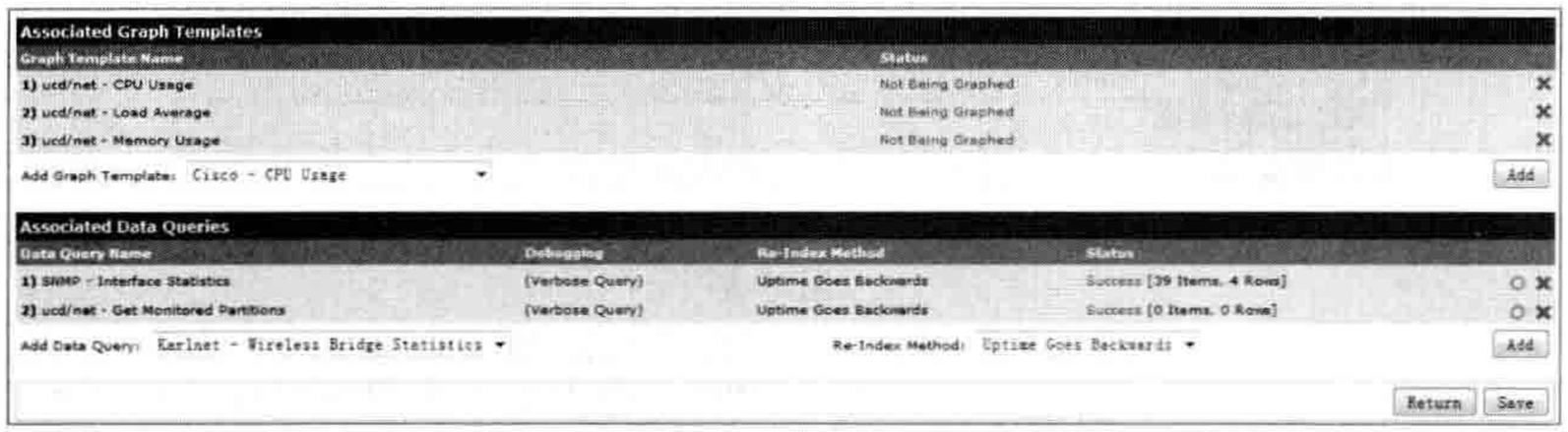


图 5-10

通过以上操作后，设备主机以及需要监控的性能指标已经创建完成。下面需要通过 New Graphs 菜单为 Web1 以及 Web2 创建图形。如图 5-11 所示，进入新建图形界面后在 Host 列表中选择上一步添加的设备主机，并在需要监控的性能模板后点选对钩，设置完成后选择“Create”按钮为被监控设备添加图形。

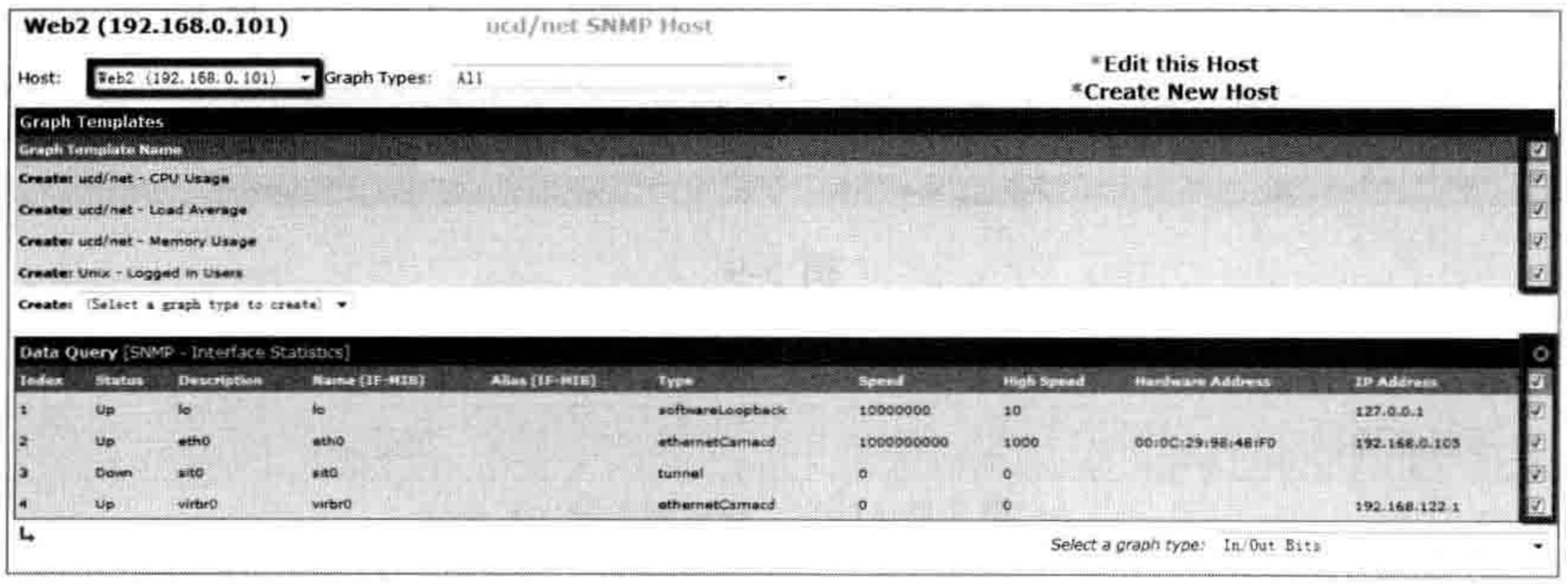


图 5-11

Cacti 监控系统最终通过树状列表来查看被监控主机的性能图形，我们可以创建多棵图形树，在每棵图形树下可以插入多个设备主机的图形信息。Cacti 初始状态有一个名为 Default 的图形树，本例将创建一棵新的图形树，并将 Web1 与 Web2 的图形添加进来。通过单击 Graph Trees 菜单打开图形树的设置界面，如图 5-12 所示，单击“Add”按钮可以添加新的图形树，提示输入 Name（图形树名称）后单击“Create”按钮，本例使用的名称为 Web，此时在 Graph Trees 界面中会出现 Default 与 Web 两棵图形树。单击进入 Web 树中，单击“Add”按钮添加上一步操作中创建的图形，如图 5-13 所示。

图 5-12

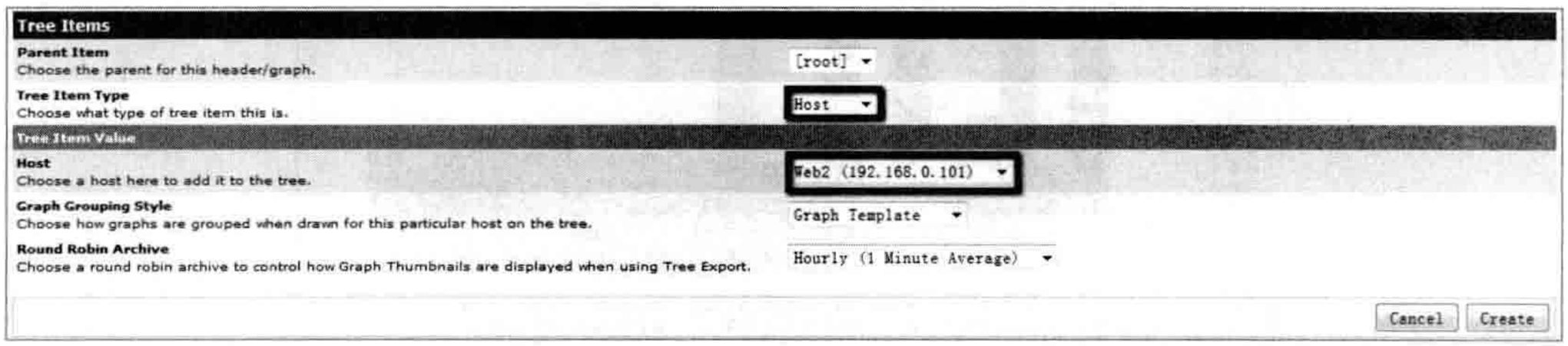


图 5-13

Cacti 监控服务器需要根据以上设置连接所有被监控的主机，通过 SNMP 收集数据信息并最终生成图形窗口，在 Cacti 软件包中已经自带了一个用于收集数据的 PHP 脚本，默认 Cacti 系统需要每五分钟收集一次数据，管理员可以通过计划任务实现每五分钟执行该脚本，也可以手动执行该脚本立即同步数据。

```
[root@cacti ~]# crontab -e
*/5 * * * * /usr/local/php5/bin/php /usr/local/apache2/htdocs/cacti/poller.php
```

最后，我们通过 Cacti 平台的 graphs 选项卡进入图形窗口，该窗口将以树状显示。本例创建了一棵名为 Web 的图形树，并在 Web 树下添加 Web1 与 Web2 两台主机的图形，如图 5-14 所示，在此单击你希望查看的主机即可。由于默认为五分钟下载一次数据，所以最终生成的图形需要等待片刻才可以看出效果，最终查看效果如图 5-15 和图 5-16 所示，分别显示为内存与 CPU 性能图表。

Cacti 除了基本的性能监控功能外，通过安装插件可以提供更多其他有用的特性。比如，如果希望当性能指标达到一个预设值后自动发出报警（通过邮件或手机短信等），Cacti 报警插件

为 thold，管理员通过部署这些插件可以使监控服务器更加强大与灵活[1]。

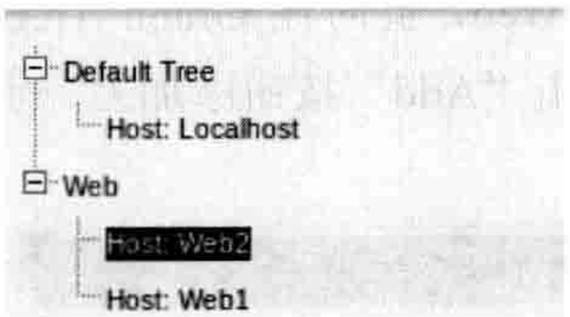


图 5-14

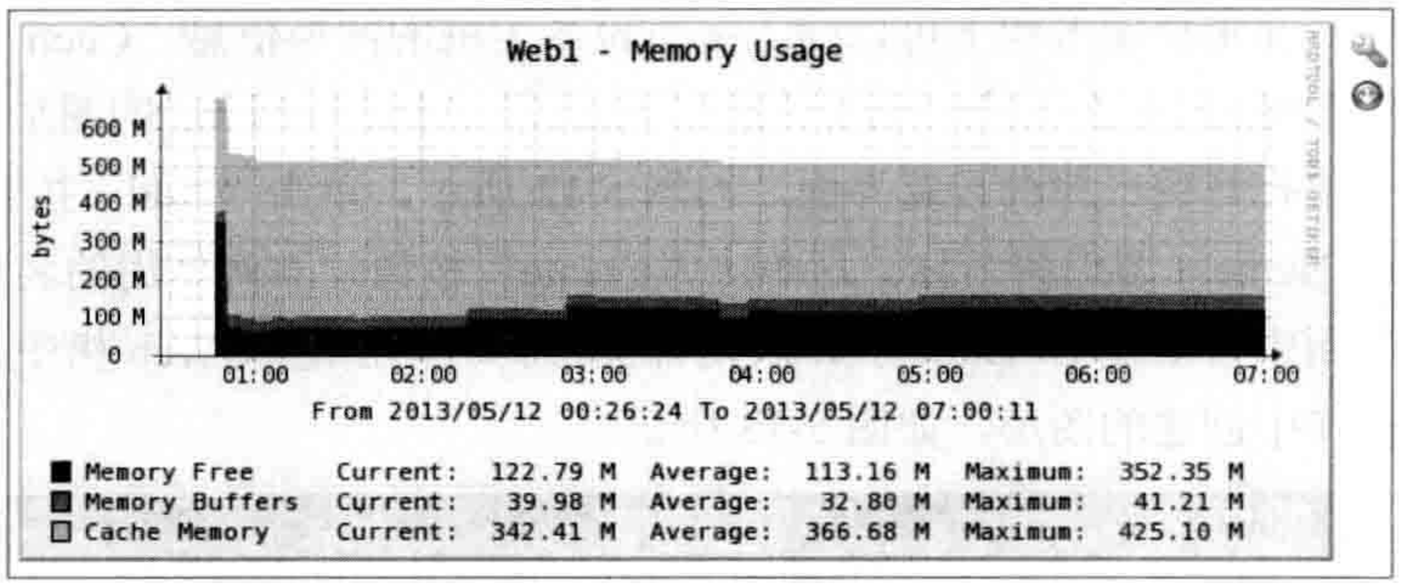


图 5-15

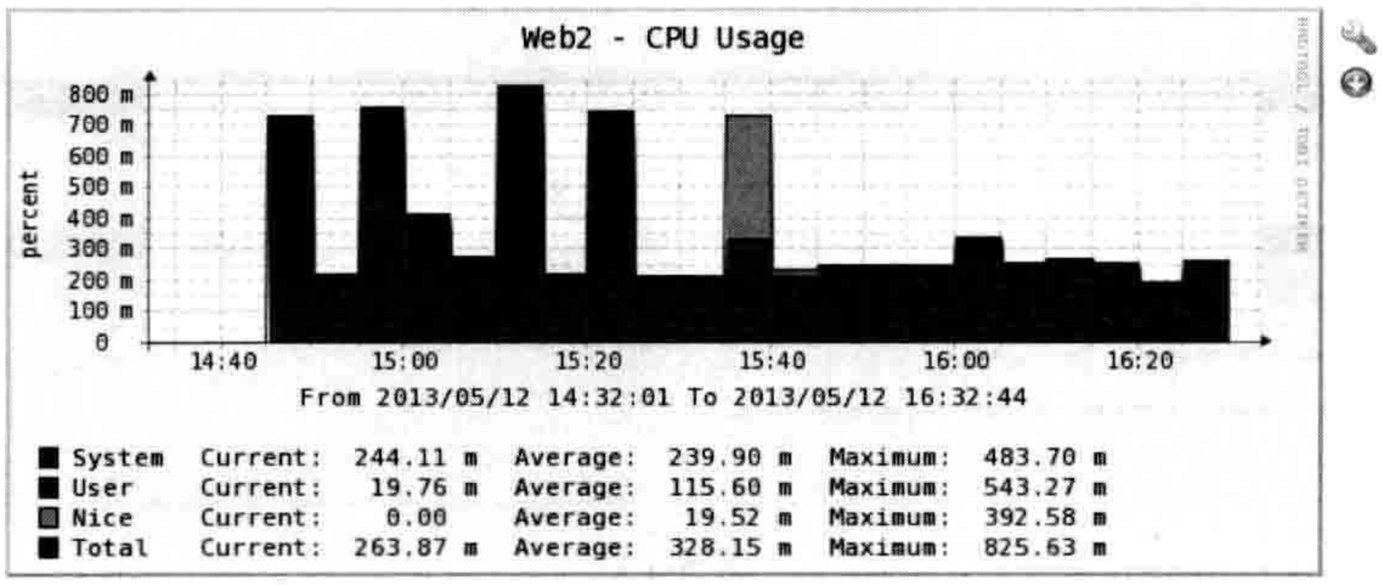


图 5-16

5.2 Nagios 监控系统

5.2.1 简介

Nagios 是一款企业级开源免费的监控工具，该工具可以监控应用服务器、交换机路由器等网络设备，并在服务或设备发生异常时发出报警信息。与 Cacti 不同的是，Nagios 重点专注的是运行在服务器上的服务是否正常，而不会像 Cacti 一样生成图形，Nagios 关注点在于保证服务的正常运行，并且在服务发生问题时提供报警机制。Nagios 强大而灵活的监控预警机制可以帮助企业在出现严重故障前把问题解决，报警信息可以通过邮件或短信发送给管理员，让管理

[1] 更多的 Cacti 插件，可以参考官方网站 http://docs.cacti.net/plugins 的介绍。

员实时掌控服务器运行状态。部署 Nagios 系统除了需要安装主程序，还需要安装相关插件，Nagios 是整个监控平台的主程序，Nagios-plugins 是必选的插件程序。另外，官方网站还提供了一些可选的附加插件。其中，NRPE 插件用来监控远程 Linux 服务器的主机资源，NSClient++ 插件用来监控 Windows 主机，NDOUtils 插件需要结合数据库系统将 Nagios 进程的当前数据和历史数据写入数据库。Nagios 可以监控常见的 HTTP、POP3、SMTP、FTP、SSH、PING 等服务，也可以监控主机的 CPU、磁盘等主机资源。如果这些标准的插件无法满足企业中特殊的监控需求，还可以开发自己的监控插件实现特殊的监控功能。

5.2.2　Nagios 监控应用案例

与前面类似，部署一台 Nagios 服务器用于监控其他服务器上的服务与系统资源是否运行正常，管理员可以通过任意一台办公电脑连接 Nagios 服务器查看 Web 页面，管理监控平台。Nagios 监控拓扑结构如图 5-17 所示。在 Nagios 监控主机上需要安装 HTTPD、Nagios、Nagios-plugins、NRPE 软件包。其中，NRPE 主要用于监控远程主机资源。

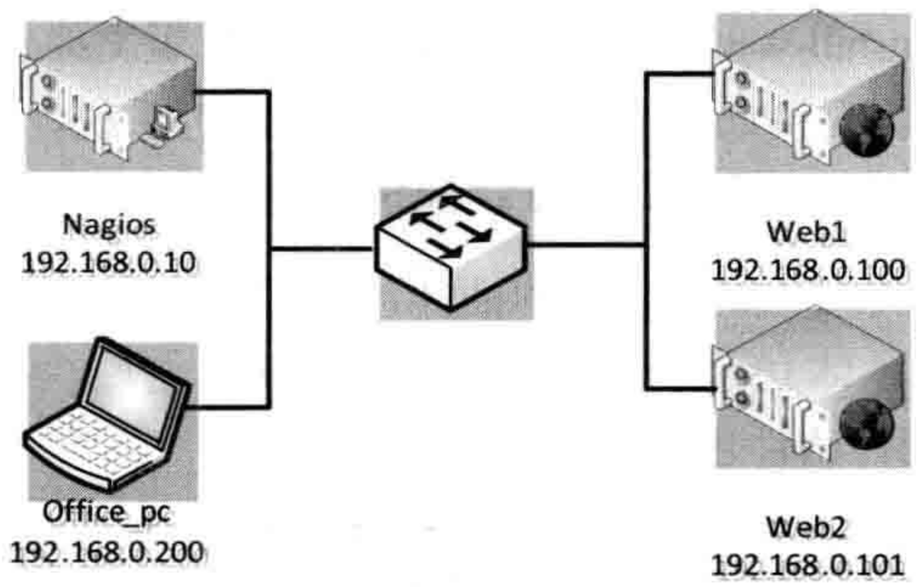


图 5-17

我们需要通过 Web 页面形式来查看并管理整个 Nagios 监控平台，所以首先需要正确地配置 Apache HTTP Server 服务。本节的重点在于监控，所以 Apache 与 PHP 可以使用 YUM 方式快速部署安装。

1. 部署监控服务器软件

监控服务器需要安装 HTTP、PHP、Nagios、Nagios-Plugins 以及 NRPE 软件包。

首先通过 YUM 方式快速安装 Nagios 所需的依赖软件包：

```
[root@nagios ~]# yum -y install gd gd-devel openssl openssl-devel \
> httpd php gcc glibc glibc-common make net-snmp wget
```

安装 Nagios 主程序时，源码包软件可以从 sourceforge.net 网站下载。安装前需要为 nagios 创建账户与组，这样在配置 nagios 软件时就可以使--with-nagios-user 和--with-nagios-group 指定以普通账号的身份运行该软件。最后需要通过多个 make install 安装部署不用的监控配置文件与目录：

```
[root@nagios ~]# wget http://prdownloads.sourceforge.net\
>/sourceforge/nagios/nagios-3.5.0.tar.gz
[root@nagios ~]# groupadd nagios
[root@nagios ~]# useradd -g nagios nagios
[root@nagios ~]# tar -xzf nagios-3.5.0.tar.gz -C /usr/src/
[root@nagios ~]# cd /usr/src/nagios/
[root@nagios nagios]# ./configure --with-nagios-user=nagios\
>--with-nagios- group=nagios
[root@nagios nagios]# make all
[root@nagios nagios]# make install          #安装主程序，CGI 以及 HTML 文件
[root@nagios nagios]# make install-init     #安装启动脚本/etc/init.d/nagios
[root@nagios nagios]# make install-commandmode #安装与配置目录权限
[root@nagios nagios]# make install-config  #安装配置文件模板
```

由于 nagios 最终将以 Web 的形式进行管理与监控，安装过程中使用 make install-webconf 命令将生成一个 Web 配置文件，install-webconf 添加 Apache 附加配置文件/etc/httpd/conf.d/nagios.conf，该文件使得管理员可以通过访问 Web 管理与维护 Nagios 监控系统，安装 Ngios 插件：

```
[root@nagios nagios]# make install-webconf
```

Nagios 的特色之一就是其拥有众多强大的插件，读者可以根据自己的实际需求有选择地安装部署这些插件，插件的具体功能介绍可以参考 nagios 官方网站，本书案例中将安装 nagios-plugins 和 nrpe 两个插件软件包。

```
[root@nagios ~]# wget http://prdownloads.sourceforge.net/sourceforge\
>/nagiosplug/nagios-plugins-1.4.16.tar.gz
[root@nagios ~]# tar -xzf nagios-plugins-1.4.16.tar.gz -C /usr/src/
[root@nagios ~]# cd /usr/src/nagios-plugins-1.4.16/
[root@nagios nagios-plugins-1.4.16]# ./configure --prefix=/usr/local/nagios/
[root@nagios nagios-plugins-1.4.16]# make
[root@nagios nagios-plugins-1.4.16]# make install
[root@nagios ~]# wget http://downloads.sourceforge.net\
>/project/nagios/nrpe-2.x/nrpe-2.14/nrpe-2.14.tar.gz
[root@nagios ~]# tar -xzf nrpe-2.14.tar.gz -C /usr/src/
[root@nagios ~]# cd /usr/src/nrpe-2.14/
[root@nagios ~]# ./configure
[root@nagios ~]# make all
[root@nagios ~]# make install-plugin
[root@nagios ~]# make install-daemon
[root@nagios ~]# make install-daemon-config
```

```
[root@nagios ~]# chown -R nagios.nagios /usr/local/nagios
[root@nagios ~]# setenforce 0
[root@nagios ~]# sed 's/=enforcing/=permissive/' /etc/sysconfig/selinux
```

安装 Nagios 时已经复制了一份 nagios.conf 文件到 Apache 配置文件/etc/httpd/conf.d/目录下，用于设定访问权限。下面根据实际需要创建账户信息文件，系统会提示输入账户密码。

```
[root@nagios ~]# htpasswd -c /usr/local/nagios/etc/htpasswd.users tomcat
[root@nagios ~]# iptables -I INPUT -p tcp --dport 80 -j ACCEPT
[root@nagios ~]# service iptables save
```

启动服务：

```
[root@nagios nagios]# /etc/init.d/httpd start
[root@nagios nagios]# /etc/init.d/nagios start
[root@nagios nagios]# chkconfig httpd on
[root@nagios nagios]# chkconfig --add nagios
[root@nagios nagios]# chkconfig nagios on
```

Nagios 安装完成后，在/usr/local/nagios 目录下会生成若干子目录，表 5-2 列出了每个子目录的名称与描述信息。

表 5-2

目录	描述
/usr/local/nagios/bin	Nagios 主程序目录
/usr/local/nagios/etc	Nagios 配置文件目录
/usr/local/nagios/libexec	Nagios 存放插件程序的目录
/usr/local/nagios/sbin	Nagios 存放 CGI 程序的目录
/usr/local/nagios/share	Nagios 网页文档存放目录
/usr/local/nagios/var	Nagios 日志与进程号文件存放目录

2. 修改 Nagios 配置文件

Nagios 配置文件比较多，表 5-3 为主要配置文件的名称与描述信息，所有的配置文件都位于/usr/local/nagios/etc 目录及 objects 子目录下。

表 5-3

配置文件名称	描述
nagios.conf	Nagios 主配置文件
nrpe.cfg	NRPE 远程监控配置文件
cgi.conf	CGI 程序配置文件

续表

配置文件名称	描述
commands.cfg	命令定义文件，如定义发送报警信息的命令
contacts.cfg	联系人与联系组定义文件，报警信息接收者的邮箱或手机
timeperiods.cfg	时间周期定义文件，定义监控报警的时间周期
templates.cfg	对象定义参考模板
localhost.cfg	监控本机的配置文件模板
printer.cfg	监控打印机的配置文件模板
switch.cfg	监控交换的配置文件模板
windows.cfg	监控 Windows 的配置文件模板

下面依次修改 Nagios 所需的监控配置文件，以满足本例的需求，配置文件中被修改的设置会使用粗体字表示，很多配置文件不需要修改就可以直接使用。

修改主配置文件。首先，需要修改的是 nagios 主配置文件 nagios.cfg，在该文件中使用 cfg_file 配置项加载其他配置文件，因为我们需要监控两台主机设备，为了方便管理，我们将为两台监控主机创建不同的配置文件，所以在该配置文件中需要加载这些用户自己创建的配置文件。本例将创建名为 web1.cfg 和 web2.cfg 的两个监控配置文件，而这两个配置文件需要在 nagios 主配置文件中使用 cfg_file 配置项加载这两个监控配置文件。

```
[root@nagios ~]# vim /usr/local/nagios/etc/nagios.cfg
```

```
#Nagios 主配置文件主要是设置全局配置与调用其他配置文件，这里需要确保以下配置文件会被
#调用
cfg_file=/usr/local/nagios/etc/objects/commands.cfg
cfg_file=/usr/local/nagios/etc/objects/contacts.cfg
cfg_file=/usr/local/nagios/etc/objects/timeperiods.cfg
cfg_file=/usr/local/nagios/etc/objects/templates.cfg
cfg_file=/usr/local/nagios/etc/objects/localhost.cfg
#下面两个配置文件需要手动创建出来，用于监控两台 Web 服务器
cfg_dir=/usr/local/nagios/etc/web1.cfg
cfg_dir=/usr/local/nagios/etc/web2.cfg
... ...其余部分省略... ...
```

修改 CGI 配置文件（cgi.cfg），以下为需要修改的内容，添加 tomcat 账户允许使用 CGI 程序，因为 Nagios 的很多功能是调用的 CGI 程序，所以需要将访问 Web 页面的账户加入进来。

```
[root@nagios ~]# vim /usr/local/nagios/etc/cgi.cfg
```

```
#要求必须经过验证的用户才可以使用 CGI 程序，由于已经使用 htpasswd 创建了 tomcat 账
#户，所以下面所有的验证账户都赋予 tomcat 账户
use_authentication=1
```

```
#设置允许通过 CGI 查看 Nagios 进程信息的账户名称
authorized_for_system_information=nagiosadmin,tomcat
#设置可以查看所有配置信息的账户名称
authorized_for_configuration_information=nagiosadmin,tomcat
#设置可以关闭或重启 Nagios 的账户名称
authorized_for_system_commands=nagiosadmin,tomcat
#设置可以查看所有服务信息的账户名称
authorized_for_all_services=nagiosadmin,tomcat
#设置可以查看所有主机信息的账户名称
authorized_for_all_hosts=nagiosadmin,tomcat
#设置允许执行服务相关命令的账户名称
authorized_for_all_service_commands=nagiosadmin,tomcat
#设置允许执行主机相关命令的账户名称
authorized_for_all_host_commands=nagiosadmin,tomcat
… …其余部分省略… …
```

修改命令配置文件（commands.cfg），该文件定义具体的命令实现方式。如，发送报警邮件具体使用什么工具、邮件内容格式定义。

```
[root@nagios ~]# vim /usr/local/nagios/etc/objects/commands.cfg
```

```
#定义命令，命令名称为 notify-host-by-email，该命令的作用是当主机发生问题时通过 mail
#命令发送报错信息给联系人配置文件中定义的联系人电子邮箱
define command{
        command_name    notify-host-by-email
        command_line    /usr/bin/printf "%b" "***** Nagios *****\n\nNotification
Type:  $NOTIFICATIONTYPE$\nHost:  $HOSTNAME$\nState:  $HOSTSTATE$\nAddress:
$HOSTADDRESS$\nInfo: $HOSTOUTPUT$\n\nDate/Time: $LONGDATETIME$\n" | /bin/mail
-s  "**  $NOTIFICATIONTYPE$  Host  Alert:  $HOSTNAME$  is  $HOSTSTATE$  **"
$CONTACTEMAIL$
        }
#定义命令，命令名称为 notify-service-by-email，该命令的作用是当服务发送问题时通过
#mail 命令发送报错信息给联系人配置文件中定义的联系人电子邮箱
define command{
        command_name    notify-service-by-email
        command_line    /usr/bin/printf "%b" "***** Nagios *****\n\nNotification
Type: $NOTIFICATIONTYPE$\n\nService: $SERVICEDESC$\nHost: $HOSTALIAS$\nAddress:
$HOSTADDRESS$\nState: $SERVICESTATE$\n\nDate/Time: $LONGDATETIME$\n\nAdditional
Info:\n\n$SERVICEOUTPUT$\n" | /bin/mail -s "** $NOTIFICATIONTYPE$ Service Alert:
$HOSTALIAS$/$SERVICEDESC$ is $SERVICESTATE$ **" $CONTACTEMAIL$
        }
#定义命令 check-host-alive，用于 ping 检查指定主机是否存活
define command{
        command_name    check-host-alive
        command_line    $USER1$/check ping -H $HOSTADDRESS$ -w 3000.0,80% -c
```

```
5000.0,100% -p 5
        }
    … …其余部分省略… …
    #以下内容需要手动添加，用于进行远程主机监控，需要安装 NRPE 软件包
    define command{
        command_name check_nrpe
        command_line $USER1$/check_nrpe -H $HOSTADDRESS$ -c $ARG1$
    }
```

命令配置文件的内容比较多，文件中定义了本机及远程的资源与服务进行检查的命令，以及当系统资源与服务发生错误时及时通过报警命令发送邮件或短信给管理员。该文件默认的报警方式是通过 mail 命令投递错误邮件给指定的管理员，使用 mail 命令发送邮件需要提前配置 Postfix 邮件服务器，这些内容不在本书的讨论范围内，读者可以参考其他关于 Postfix 书籍部署企业邮件服务器。如果希望通过短信来获得实时报警信息，可使用免费的飞信程序发送短信给特定的手机，也可以购买短信网关发送手机短信。使用飞信 fetion --mobile=13788888888 --pwd=111 --to=13888888888 --msg-utf8="消息内容" --msg-type=1}就可以实现短信接收报警信息。由于 Linux 版本的飞信程序并非移动官方开发，所以有被封的风险存在。因为现在很多邮件系统都支持短信通知功能，我们也可以考虑通过给支持该功能的邮箱发送邮件，间接实现短信通知功能。

修改联系人配置文件（contacts.cfg），该文件用于定义当监控发现设备或服务出现故障时需要的联系人及联系方式。

```
[root@nagios ~]# vim /usr/local/nagios/etc/objects/contacts.cfg
```

```
define contact{
#联系人姓名
        contact_name    nagiosadmin
#调用 templates.cfg 文件中定义的 generic-contact
        use             generic-contact
#定义联系人别名
        alias            Nagios Admin
#定义联系人邮箱
email           nagios@test.com
        }
define contactgroup{
#定义联系人组 admins
        contactgroup_name       admins
#定义组别名
        alias                   Nagios Administrators
#定义组成员，多个成员用分号隔开
        members                 nagiosadmin
```

```
        }
... ...其余部分省略... ...
```

修改时间定义配置文件，文件名为 timeperiods.cfg，用于定义报警时间周期。

```
[root@nagios ~]# vim /usr/local/nagios/etc/objects/timeperiods.cfg
```

```
define timeperiod{
#定义报警时间为 7×24 小时
        timeperiod_name 24x7
        alias                24 Hours A Day, 7 Days A Week
        Sunday               00:00-24:00
        Monday               00:00-24:00
        Tuesday              00:00-24:00
        Wednesday            00:00-24:00
        Thursday             00:00-24:00
        Friday               00:00-24:00
        Saturday             00:00-24:00
        }
... ...其余部分省略... ...
```

修改监控本机配置文件（localhost.cfg），该文件用于设置如何监控本机服务器资源。

```
[root@nagios ~]# vim /usr/local/nagios/etc/objects/localhost.cfg
```

```
define host{
#调用 templates.cfg 文件中定义的 linux-server，主要定义检查周期、频率以及通知时间段
        use                     linux-server
        host_name               localhost
        alias                   localhost
        address                 127.0.0.1
        }
define service{
#调用 templates.cfg 文件中定义的 loal-service，主要定义检查周期、频率以及通知时间段
        use                     local-service
        host_name               localhost
        service_description  PING
        check_command        check_ping!100.0,20%!500.0,60%
        }
define service{
        use                     local-service
        host_name               localhost
        service_description  HTTP
        check_command        check_http
        notifications_enabled   1
        }
... ...其余部分省略... ...
```

修改模板配置文件（templates.cfg），该文件主要定义一些其他配置文件需要调用的定义，根据自己的时间需要修改报警通知时间、报警方式（notify-host-by-email 将调用 commands.cfg 配置文件中定义的命令发送邮件报警）、报警选项（具体报警选项的含义见表 5-4）。

```
[root@nagios ~]# vim /usr/local/nagios/etc/objects/templates.cfg
```

```
define contact{
        name                            generic-contact
        service_notification_period     24x7
        host_notification_period        24x7
        service_notification_options    w,u,c,r,f,s
        host_notification_options       d,u,r,f,s
        service_notification_commands   notify-service-by-email
        host_notification_commands      notify-host-by-email
        register                        0
        }
define host{
        name                            generic-host
        notifications_enabled           1
        event_handler_enabled           1
        flap_detection_enabled          1
        failure_prediction_enabled      1
        process_perf_data               1
        retain_status_information       1
        retain_nonstatus_information    1
    notification_period                 24x7
        register                        0
        }
…其余部分省略…
```

表 5-4

选项	功能描述
w	Warning 警告
u	Unknown 未知
c	Critical 严重
r	Recover 恢复
d	Down 宕机

修改 NRPE 配置文件（nrpe.cfg），创建用于监控远程主机所需要的命令。

```
[root@nagios ~]# vim /usr/local/nagios/etc/nrpe.cfg
```

```
#NRPE 远程监控所需的具体监控指令
allowed_hosts=127.0.0.1,192.168.0.10
```

```
command[check_users]=/usr/local/nagios/libexec/check_users -w 5 -c 10
command[check_load]=/usr/local/nagios/libexec/check_load -w 15,10,5 -c 30,25,20
command[check_hda1]=/usr/local/nagios/libexec/check_disk -w 20% -c 10% -p /dev/sda
command[check_zombie_procs]=/usr/local/nagios/libexec/check_procs -w 5 -c 10 -s Z
command[check_total_procs]=/usr/local/nagios/libexec/check_procs -w 150 -c 200
command[check_disk]=/usr/local/nagios/libexec/check_disk -w 20% -c 10%
...其余部分省略...
```

创建远程监控配置文件 web1.cfg 与 web2.cfg，用户监控远程服务器系统资源与服务，该文件需要手动创建，可以用 localhost.cfg 作为参考模板。下面为 web1.cfg 文件全文（web2.cfg 仅需要根据 web1.cfg 的内容修改主机名称、IP 地址以及主机组名称即可，这里就不再单独编写出来）。

```
[root@nagios ~]# vim /usr/local/nagios/etc/ objects/web1.cfg
```

```
define host{
        use                     linux-server
        host_name               web1
        alias                   web1.example.com
        address                 192.168.0.104
        }
define hostgroup{
        hostgroup_name          webs
        alias                   Linux Servers
        members                 web1
        }
define service{
        use                     generic-service
        host_name               web1
        service_description     PING
check_command                   check_ping!100.0,20%!500.0,60%
notifications_enabled           1
        }
define service{
        use                     generic-service
        host_name               web1
        service_description     Sys_Load
check_command                   check_nrpe!check_load
notifications_enabled           1
        }
define service{
        use                     generic-service
        host_name               web1
        service_description     Current Users
check_command                   check_nrpe!check_users
notifications_enabled           1
```

```
        }
define service{
    use                         generic-service
    host_name                   web1
    service_description         Total Processes
check_command                   check_nrpe!check_total_procs
notifications_enabled           1
        }
define service{
    use                         generic-service
    host_name                   web1
    service_description         SSH
check_command                   check_ssh
notifications_enabled           1
        }
define service{
    use                         generic-service
    host_name                   web1
    service_description         HTTP
check_command                   check_http
notifications_enabled           1
        }
```

```
[root@nagios nagios]# /etc/init.d/nagios restart          #重新加载所有的配置
```

提示 因为 Nagios 的配置文件比较多，所以经常会出现当修改完配置文件后，Nagios 服务无法启动的情况，此时利用 Nagios 提供的一个检查排错工具可以快速定位问题点：/usr/local/nagios/bin/nagios -v/usr/local/nagios/etc/nagios.cfg。

3. 安装被监控主机软件（web1、web2 所需软件包）

将下面这些操作步骤分别应用于 web1 和 web2 两台主机，远程监控时需要被监控端安装 nagios-plugin 与 nrpe 软件包。下面仅以 web1 主机为例（web2 做相同的操作即可）。

```
[root@web1 ~]# yum -y install openssl openssl-devel
[root@web1 ~]# groupadd nagios
[root@web1 ~]# useradd -g nagios -s /sbin/nologin nagios
 [root@web1 ~]# wget http://prdownloads.sourceforge.net/sourceforge\
>/nagiosplug/nagios-plugins-1.4.16.tar.gz
[root@web1 ~]# tar -xzf nagios-plugins-1.4.16.tar.gz -C /usr/src/
[root@web1 ~]# cd /usr/src/nagios-plugins-1.4.16/
[root@web1 nagios-plugins-1.4.16]# make
[root@web1 nagios-plugins-1.4.16]# make install
[root@web1 nagios-plugins-1.4.16]# chown -R nagios.nagios /usr/local/nagios
```

```
[root@web1 ~]#wget http://jaist.dl.sourceforge.net/project\
>/nagios/nrpe-2.x/nrpe-2.14/nrpe-2.14.tar.gz
[root@web1 ~]# tar -xzf nrpe-2.14.tar.gz -C /usr/src/
[root@web1 ~]# cd /usr/src/nrpe-2.14/
[root@web1 nrpe-2.14]# ./configure
[root@web1 nrpe-2.14]# make all
[root@web1 nrpe-2.14]# make install-plugin
[root@web1 nrpe-2.14]# make install-daemon
[root@web1 nrpe-2.14]# make install-daemon-config
```

4. 修改客户端 NRPE 配置文件

```
[root@web1 ~]# vim /usr/local/nagios/etc/nrpe.cfg
```

```
#NRPE 远程监控所需的具体监控指令
allowed_hosts=127.0.0.1,192.168.0.10
# allowed_hosts 设置允许哪台主机可监控本机
command[check_users]=/usr/local/nagios/libexec/check_users -w 5 -c 10
command[check_load]=/usr/local/nagios/libexec/check_load -w 15,10,5 -c 30,25,20
command[check_hda1]=/usr/local/nagios/libexec/check_disk -w 20% -c 10% -p /dev/sda
command[check_zombie_procs]=/usr/local/nagios/libexec/check_procs -w 5 -c 10 -s Z
command[check_total_procs]=/usr/local/nagios/libexec/check_procs -w 150 -c 200
command[check_disk]=/usr/local/nagios/libexec/check_disk -w 20% -c 10%
...其余部分省略...
```

```
[root@web1 ~]# /usr/local/nagios/bin/nrpe -c /usr/local/nagios/etc/nrpe.cfg -d
[root@web1 ~]# echo "/usr/local/nagios/bin/nrpe \
<-c /usr/local/nagios/etc/nrpe.cfg -d" >> /etc/rc.local
[root@web1 ~]# netstat -nutlp |grep 5666
[root@web1 ~]# iptables -I INPUT -p tcp --dport 5666 -j ACCEPT
[root@web1 ~]# service iptables save
```

5. 验收

管理员在 Nagios 主机上通过 check_nrpe 检测被监控端相关的性能参数，单独使用 check_nrpe 命令可以检测被监控端的 NRPE 版本号。

```
[root@web1 ~]# /usr/local/nagios/libexec/check_nrpe -H 192.168.0.104
```

如果 NRPE 完全没有问题了，此时管理员可以通过客户端浏览器访问：http://nagios.example.com/nagios（需要 DNS 域名解析），访问监控服务器的 Web 页面，进行监控管理与查看监控状态。客户端访问监控服务器的 Web 服务效果如图 5-18 和图 5-19 所示。

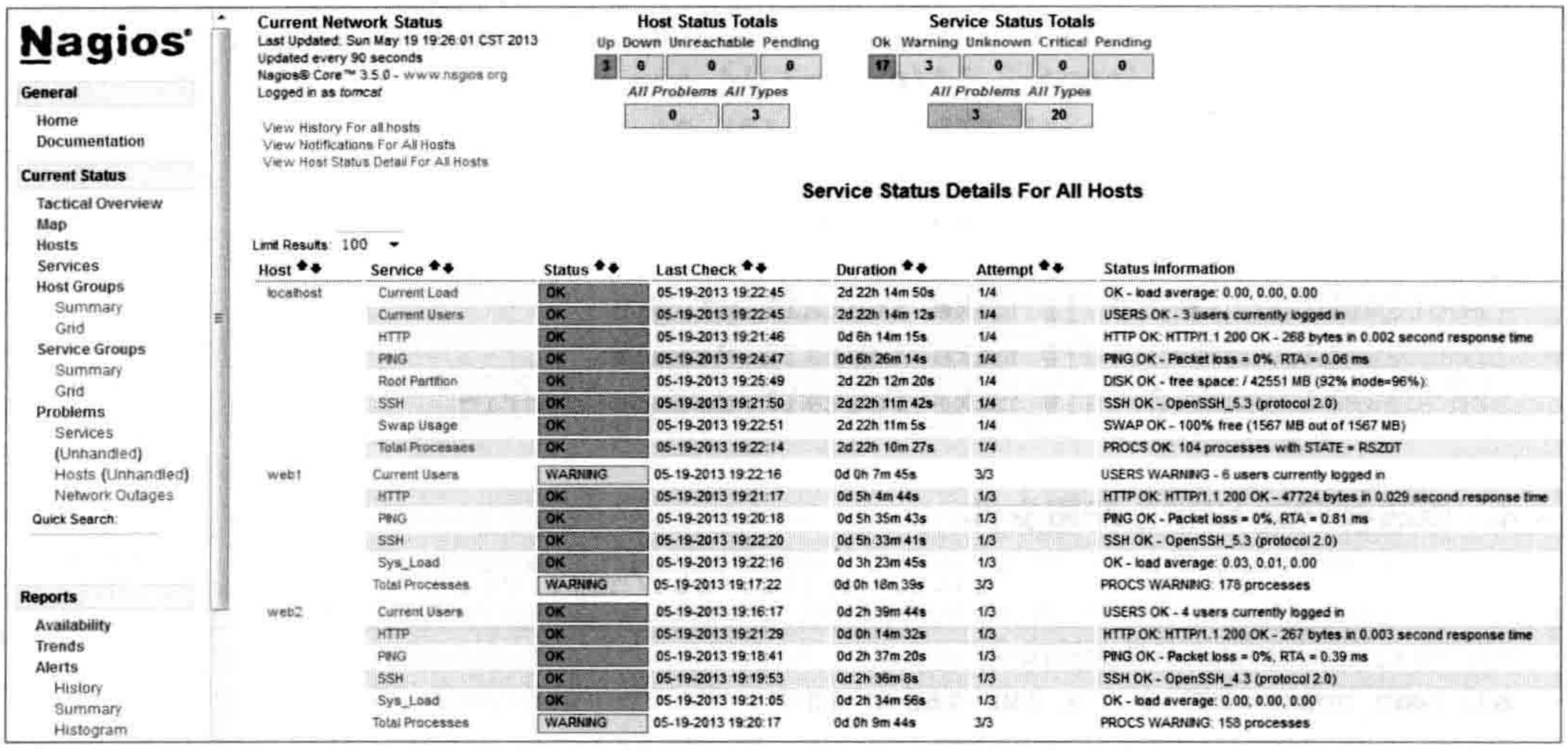

Nagios
General
Home
Documentation
Current Status
Tactical Overview
Map
Hosts
Services
Host Groups
Summary
Grid
Service Groups
Summary
Grid
Problems
Services (Unhandled)
Hosts (Unhandled)
Network Outages
Quick Search:
Reports
Availability
Trends
Alerts
History
Summary
Histogram
Current Network Status
Last Updated: Sun May 19 19:26:01 CST 2013
Updated every 90 seconds
Nagios® Core™ 3.5.0 - www.nagios.org
Logged in as tomcat
View History For all hosts
View Notifications For All Hosts
View Host Status Detail For All Hosts
Host Status Totals
Up Down Unreachable Pending
3 0 0 0
All Problems All Types
0 3
Service Status Totals
Ok Warning Unknown Critical Pending
17 3 0 0 0
All Problems All Types
3 20
Service Status Details For All Hosts
Limit Results: 100
Host Service Status Last Check Duration Attempt Status Information
localhost Current Load OK 05-19-2013 19:22:45 2d 22h 14m 50s 1/4 OK - load average: 0.00, 0.00, 0.00
Current Users OK 05-19-2013 19:22:45 2d 22h 14m 12s 1/4 USERS OK - 3 users currently logged in
HTTP OK 05-19-2013 19:21:46 0d 6h 14m 15s 1/4 HTTP OK: HTTP/1.1 200 OK - 268 bytes in 0.002 second response time
PING OK 05-19-2013 19:24:47 0d 6h 26m 14s 1/4 PING OK - Packet loss = 0%, RTA = 0.06 ms
Root Partition OK 05-19-2013 19:25:49 2d 22h 12m 20s 1/4 DISK OK - free space: / 42551 MB (92% inode=96%):
SSH OK 05-19-2013 19:21:50 2d 22h 11m 42s 1/4 SSH OK - OpenSSH_5.3 (protocol 2.0)
Swap Usage OK 05-19-2013 19:22:51 2d 22h 11m 5s 1/4 SWAP OK - 100% free (1567 MB out of 1567 MB)
Total Processes OK 05-19-2013 19:22:14 2d 22h 10m 27s 1/4 PROCS OK: 104 processes with STATE = RSZDT
web1 Current Users WARNING 05-19-2013 19:22:16 0d 0h 7m 45s 3/3 USERS WARNING - 6 users currently logged in
HTTP OK 05-19-2013 19:21:17 0d 5h 4m 44s 1/3 HTTP OK: HTTP/1.1 200 OK - 47724 bytes in 0.029 second response time
PING OK 05-19-2013 19:20:18 0d 5h 35m 43s 1/3 PING OK - Packet loss = 0%, RTA = 0.81 ms
SSH OK 05-19-2013 19:22:20 0d 5h 33m 41s 1/3 SSH OK - OpenSSH_5.3 (protocol 2.0)
Sys_Load OK 05-19-2013 19:22:16 0d 3h 23m 45s 1/3 OK - load average: 0.03, 0.01, 0.00
Total Processes WARNING 05-19-2013 19:17:22 0d 0h 18m 39s 3/3 PROCS WARNING: 178 processes
web2 Current Users OK 05-19-2013 19:16:17 0d 2h 39m 44s 1/3 USERS OK - 4 users currently logged in
HTTP OK 05-19-2013 19:21:29 0d 0h 14m 32s 1/3 HTTP OK: HTTP/1.1 200 OK - 267 bytes in 0.003 second response time
PING OK 05-19-2013 19:18:41 0d 2h 37m 20s 1/3 PING OK - Packet loss = 0%, RTA = 0.39 ms
SSH OK 05-19-2013 19:19:53 0d 2h 36m 8s 1/3 SSH OK - OpenSSH_4.3 (protocol 2.0)
Sys_Load OK 05-19-2013 19:21:05 0d 2h 34m 56s 1/3 OK - load average: 0.00, 0.00, 0.00
Total Processes WARNING 05-19-2013 19:20:17 0d 0h 9m 44s 3/3 PROCS WARNING: 158 processes

图 5-18

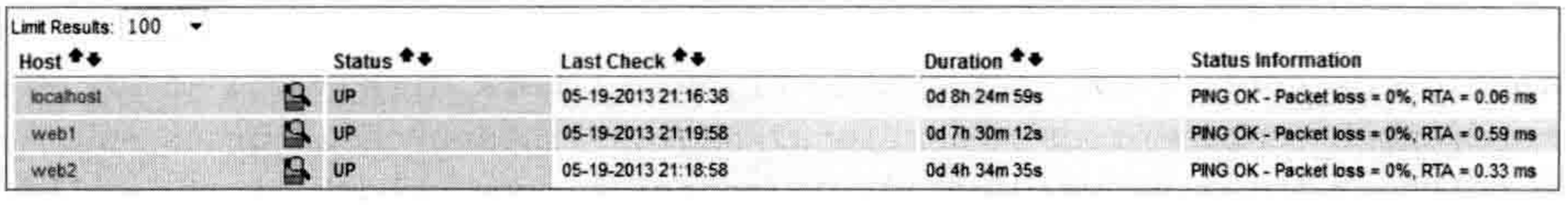

Limit Results: 100
Host Status Last Check Duration Status Information
localhost UP 05-19-2013 21:16:38 0d 8h 24m 59s PING OK - Packet loss = 0%, RTA = 0.06 ms
web1 UP 05-19-2013 21:19:58 0d 7h 30m 12s PING OK - Packet loss = 0%, RTA = 0.59 ms
web2 UP 05-19-2013 21:18:58 0d 4h 34m 35s PING OK - Packet loss = 0%, RTA = 0.33 ms

图 5-19

第6章

网络安全

6.1 iptables 防火墙

现代的计算机环境是一个网络的环境，而且是一个非常开放的网络环境，在这个开放的网络环境中，用户可以获得大量的资讯信息，也可以更加轻松地进行社交、学习等。但这个开放的环境也给我们带来了一些不便，在开放的同时，网络中还伴有大量的攻击、发送垃圾邮件、盗号、欺诈等活动。对于 IT 运维人员而言，如何防止自己的服务器免受攻击？这需要管理员掌握扎实的安全理论基础知识以及专业工具的使用技巧。要防止遭受网络攻击，掌握一种网络访问控制工具的使用是至关重要的，而 netfilter/iptables 是集成在 Linux 2.4.X 版本内核中的包过滤防火墙系统。

netfilter/iptables 框架可以实现数据包过滤、网络地址转换以及数据包管理功能。Linux 中的防火墙系统包括两部分：netfilte 和 iptables。netfilter 位于内核空间，目前是 Linux 内核的组成部分。具体地说，netfilter 是 Linux 内核内部的一系列钩子，这些钩子允许数据表过滤函数挂载到系统内核中。而 iptables 是用户工具，由于 netfilter 在内核空间中，用户通常无法接触内核和修改内核，此时就需要一个像 iptables 这样的命令行工具，使用 iptables 可以添加、删除具体的过滤规则，iptables 默认维护着四个表和五个链，所有的防火墙策略规则都将被分别写入这些表

与链中。Linux 防火墙因为已经集成在内核中，所以相对于应用层防火墙产品而言，基于内核的 Linux 防火墙更加快捷、高效。

iptables 有很好的可扩展性，也就是说，内核架构与 iptables 工具都可以添加扩展功能，CentOS 6.3 系统中防火墙模块存放在/lib/modules/2.6.32-279.el6.x86_64/kernel/net/netfilter 目录下，当需要某个模块功能时，可以通过 modprobe 加载该模块功能。

默认的 iptables 规则表有：fileter 表（过滤规则表）、nat 表（地址转换规则表）、mangle（修改数据标记位规则表）、raw（跟踪数据表规则表）。每个规则表中包含多个数据链：INPUT（入站数据过滤）、OUTPUT（出站数据过滤）、FORWARD（转发数据过滤）、PREROUTING（路由前过滤）和 POSTROUTING（路由后过滤），防火墙规则需要写入到这些具体的数据链中。图 6-1 展示了 Linux 防火墙的过滤框架，从图中可以看出，如果是外部主机发送数据包给防火墙本机，数据将会经过 PREROUTING 链与 INPUT 链；如果是防火墙本机发送数据包到外部主机，数据将会经过 OUTPUT 链与 POSTROUTING 链；如果防火墙作为路由负责转发数据，则数据将经过 PREROUTING 链、FORWARD 链以及 POSTROUTING 链。

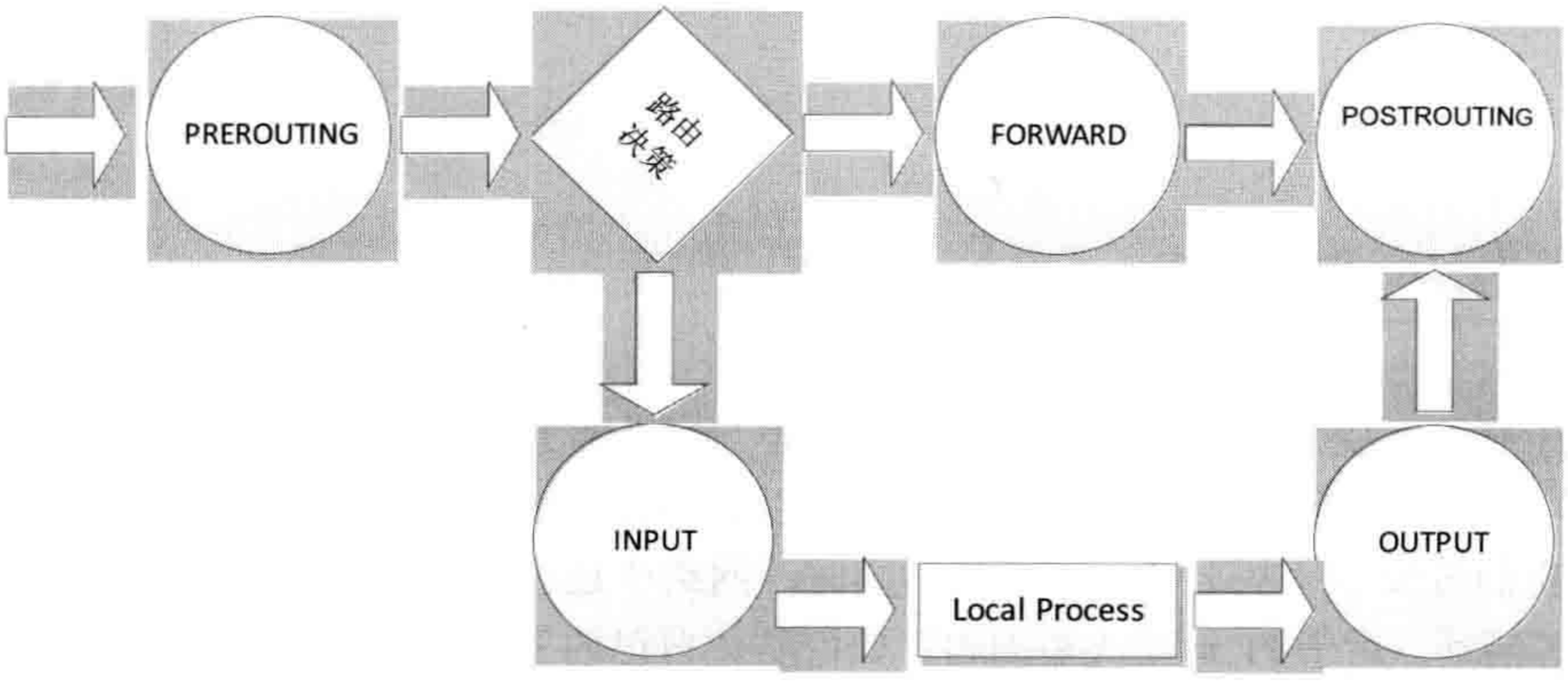


图 6-1

6.1.1 iptables 防火墙语法格式

管理员需要使用 iptables 命令添加、删除防火墙规则，下面我们看看 iptables 命令的具体用法。

命令描述：netfilter 防火墙规则管理工具。

用法：iptables [-t 表名] {-A|-D|-I|-D|-F|-L|-Z|-P} 链名 rule-specification

选项：-t　　指定需要维护的防火墙规则表，不使用-t 时，则默认操作对象为 filter 表。

-A	追加防火墙规则。	
-D	删除防火墙规则。	
-I	插入防火墙规则。	
-F	清空防火墙规则。	
-L	列出防火墙规则。	
-R	替换防火墙规则。	
-Z	清空防火墙数据表统计信息。	
-P	设置链默认规则。	
匹配参数：	[!] -p	匹配协议，!代表取反
	[!] -s	匹配源地址
	[!] -d	匹配目标地址
	[!] -i	匹配入站网卡接口
	[!] -o	匹配出站网卡接口
	[!] --sport	匹配源端口
	[!] --dport	匹配目标端口
	[!] --src-range	匹配源地址范围
	[!] --dst-range	匹配目标地址范围
	[!] --limit	匹配数据表速率
	[!] --mac-source	匹配源 MAC 地址
	[!] --sports	匹配源端口
	[!] --dports	匹配目标端口
	[!] --state	匹配状态（INVALID、ESTABLISHED、NEW、RELATED）
	[!] --string	匹配应用层字串
触发动作：	ACCEPT	允许数据包通过
	DROP	丢弃数据包
	REJECT	拒绝数据包通过
	LOG	将数据包信息记录 syslog 日志
	DNAT	目标地址转换
	SNAT	源地址转换
	MASQUERADE	地址欺骗
	REDIRECT	重定向

iptables 防火墙规则的顺序非常重要，内核按顺序检查这些规则，如果发现有匹配的规则条

目，则立刻执行相关动作，停止继续向下查找规则条目，如果所有的防火墙规则都未能匹配成功，则按照默认策略处理。我们使用-A 选项添加防火墙规则会将该规则追加到整个链的最后，而使用-I 选项添加的规则默认会插入到链中作为第一条规则。下面通过实例简单演示 iptables 命令的使用方法。

查看 filter 表的所有规则：

```
[root@centos6 ~]# iptables -nL
```

查看 nat 表的所有规则：

```
[root@centos6 ~]# iptables -t nat -nL
```

清空 filter 表中的所有规则：

```
[root@centos6 ~]# iptables -F
```

往 filter 表添加一条新的入站规则，丢弃 192.168.0.1 主机发送给防火墙本机的所有数据包。

```
[root@centos6 ~]# iptables -A INPUT -s 192.168.0.1 -j DROP
```

往 filter 表插入一条新的入站规则，拒绝 192.168.0.22 ping 防火墙本机：

```
[root@centos6 ~]# iptables -I INPUT -s 192.168.0.22 -p icmp -j REJECT
```

查看 filter 表中防火墙规则并显示规则编号：

```
[root@centos6 ~]# iptables -nL --line-number
```

删除 filter 表中 INPUT 链的第一条规则：

```
[root@centos6 ~]# iptables -D INPUT 1
```

替换 filter 表中 INPUT 链的第二条规则，拒绝 192.168.0.254 之外的任何主机连接防火墙本机：

```
[root@centos6 ~]# iptables -R INPUT 2 ! -s 192.168.0.254 -j REJECT
```

修改 filter 表中 INPUT 链的默认规则为接收数据包：

```
[root@centos6 ~]# iptables -t filter -P INPUT ACCEPT
```

将 192.168.0.10 主机发送给防火墙本机 22 端口的所有数据包信息记录到 messages 日志：

```
[root@centos6 ~]# iptables -I INPUT -s 192.168.0.10 -p tcp --dport 22 -j LOG
```

允许任何主机从 eth0 网络接口访问防火墙本机的 80 端口：

```
[root@centos6 ~]# iptables -I INPUT -i eth0 --p tcp --dport 80 -j ACCEPT
```

6.1.2　iptables 防火墙应用案例

案例 1：允许任意客户端访问服务器主机提供的日常服务（HTTP、HTTPS、DNS、NTP、SMTP、POP3、SSH），在 Linux 系统中，/etc/services 文件可以帮助我们找到各种服务所对应的标准端口信息。

```
[root@centos6 ~]# iptables -A INPUT -p tcp --dport 22 -j ACCEPT
[root@centos6 ~]# iptables -A INPUT -p udp --dport 25 -j ACCEPT
[root@centos6 ~]# iptables -A INPUT -p tcp --dport 53 -j ACCEPT
[root@centos6 ~]# iptables -A INPUT -p udp --dport 53 -j ACCEPT
[root@centos6 ~]# iptables -A INPUT -p tcp --dport 80 -j ACCEPT
[root@centos6 ~]# iptables -A INPUT -p tcp --dport 110 -j ACCEPT
[root@centos6 ~]# iptables -A INPUT -p tcp --dport 143 -j ACCEPT
[root@centos6 ~]# iptables -A INPUT -p tcp --dport 123 -j ACCEPT
[root@centos6 ~]# iptables -P INPUT DROP
[root@centos6 ~]# iptables -P OUTPUT ACCEPT
```

案例 2：公司拥有一个公有 IP，使用防火墙实现局域网中所有的主机通过 SNAT 共享上网，拓扑结构如图 6-2 所示。我们使用 CentOS 6.3 作为公司软路由，公司内部所有 192.168.0.0/24 网段内的主机连接外网时，防火墙自动将所有数据包的源地址修改为路由器上的公有 IP，最后互联网将信息返回路由后，由路由再转交给真正的后端主机。防火墙源地址转换（SNAT）规则需要被写入到 NAT 表的 POSTROUTING 链。

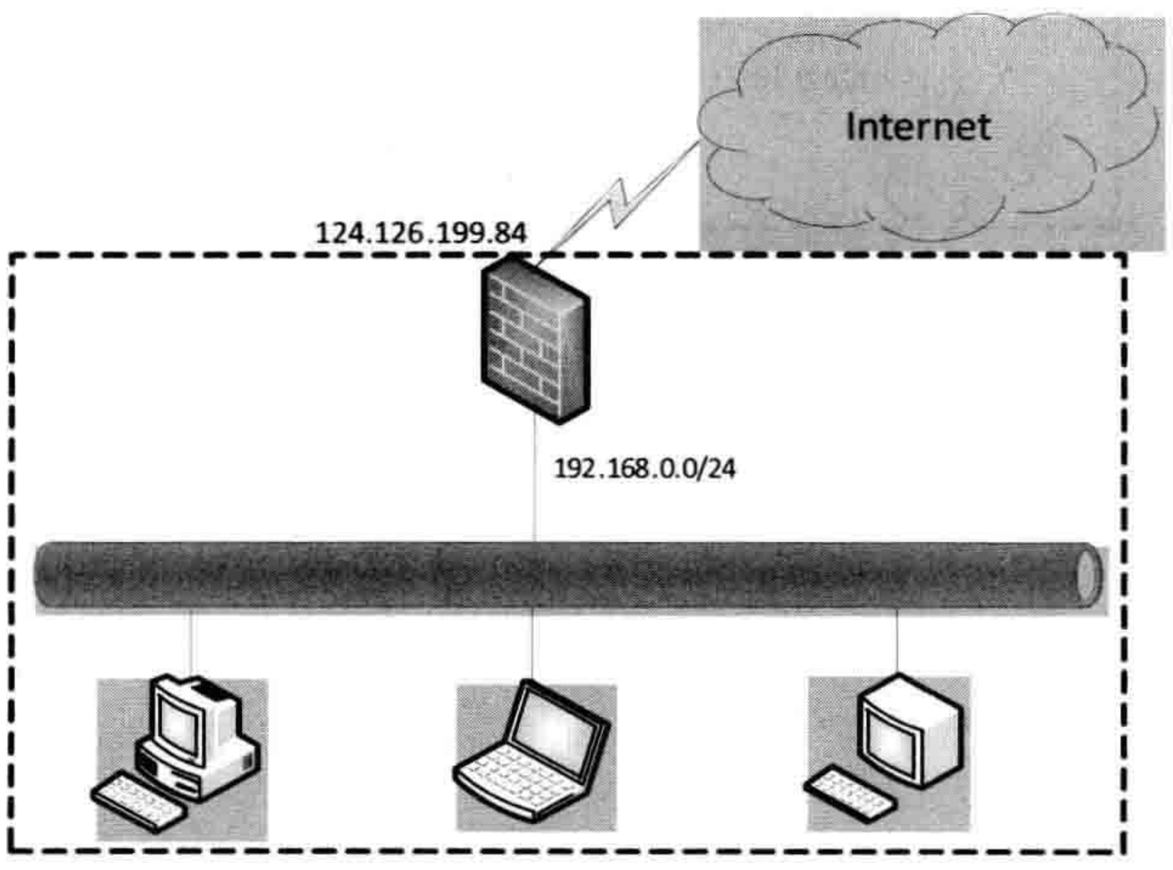


图 6-2

```
[root@router ~]# vim /etc/sysctl.conf
```

```
# Kernel sysctl configuration file for Red Hat Linux
#
```

```
# For binary values, 0 is disabled, 1 is enabled.  See sysctl(8) and
# sysctl.conf(5) for more details.
# 开启路由转发，实现基于 Linux 的软路由功能
net.ipv4.ip_forward = 1
#
… …其余部分省略… …
```

```
[root@router ~]# sysctl -p                    #重新加载内核参数配置文件/etc/sysctl.conf
[root@router ~]# iptables -t nat -I POSTROUTING -s 192.168.0.0/24 \
>-j SNAT --to-source 124.126.199.84
```

案例 3：公司对外有一个公有 IP，内部有 HTTP、MAIL 两台核心服务器，通过防火墙实现客户可以从互联网的任意位置访问位于公司内部的两台服务器资源，拓扑结构类似于图 6-2，只是连接的方向发生了改变，案例 2 是源地址转换（SNAT），本例是目标地址转换（DNAT）。

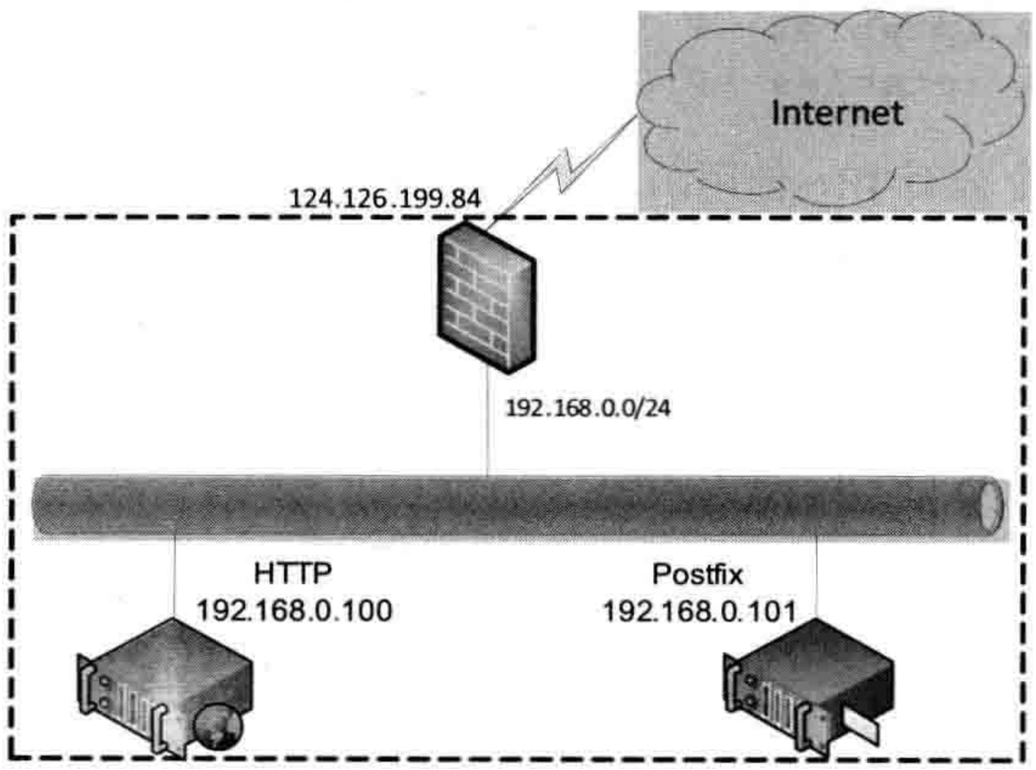


图 6-2

```
[root@router ~]# vim /etc/sysctl.conf
```

```
# Kernel sysctl configuration file for Red Hat Linux
#
# For binary values, 0 is disabled, 1 is enabled.  See sysctl(8) and
# sysctl.conf(5) for more details.
# 开启路由转发，实现基于 Linux 的软路由功能
net.ipv4.ip_forward = 1
#
… …其余部分省略… …
```

```
[root@router ~]# sysctl -p                    #重新加载内核参数配置文件/etc/sysctl.conf
[root@router ~]# iptables -t nat -I PREROUTING -d 124.126.199.84 -p tcp --dport 80 \
>-j DNAT --to-destination 192.168.0.100
[root@router ~]# iptables -t nat -I PREROUTING -d 124.126.199.84 -p tcp --dport 25 \
```

```
>-j DNAT --to-destination 192.168.0.101
[root@router ~]# iptables -t nat -I PREROUTING -d 124.126.199.84 -p tcp --dport 110 \
>-j DNAT --to-destination 192.168.0.101
```

案例 4：数据包因为太大无法一次完成数据的传输，此时数据包将被分割为数据片段再发送出去。接收端接收完数据后，将把这些数据片段重新组合成完整的数据包。但问题在于当数据被分割后，只有前面的初始数据片段包含全部的数据头部信息（IP、TCP、UDP、ICMP 等），后续的数据片段仅包含数据包头部信息的一部分。这时再去检查后续数据片段的头部信息是不可能的。当然，如果你想匹配第二个及后面被分片的数据，可以使用“-f”选项。

丢弃发送至 192.168.1.1 的所有数据以及分片数据：

```
[root@centos6 ~]# iptables -A OUTPUT -f -d 192.168.1.1 -j DROP
```

案例 5：目前网络上的攻击手法层出不穷，很多攻击会采用发送大量无效的数据包给服务器，造成服务器无法响应正常的请求包，iptables 提供了一个 limit 扩展功能，可以限制单位时间内数据包的个数。下面的规则是当每秒钟数据包个数为 500 时接受入站连接，否则拒绝连接。

```
[root@centos6 ~]# iptables -I INPUT -m limit --limit 500/sec -j ACCEPT
[root@centos6 ~]# iptables -P INPUT DROP
```

案例 6：企业环境中，服务器会面临各种各样的攻击，iptables 本身属于三层包过滤防火墙，但也提供了 string 扩展功能，通过--string 也可以根据关键词限制网络连接。将下面两条记录写入基于 Linux 的软路由服务器防火墙规则中，实现拒绝转发包含有关键词/etc/passwd 以及 qq 的数据包，也就是防止将密码文件复制出局域网，并防止内部员工访问 QQ 网站。

```
[root@router ~]# iptables -I FORWARD -m string --algo bm --string "/etc/passwd"
-j REJECT
[root@router ~]# iptables -I FORWARD -m string --algo bm --string "qq" -j REJECT
```

案例 7：根据数据连接状态设置防火墙规则，放行所有的出站数据包。拒绝入站的新连接请求与无效连接，放行入站的回应请求。

```
[root@centos6 ~]# iptables -F
[root@centos6 ~]# iptables -A INPUT -m state --state NEW -j DROP
[root@centos6 ~]# iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
[root@centos6 ~]# iptables -P OUTPUT ACCEPT
```

案例 8：公司采用基于 Linux 的软路由设备，要求在路由设备上设置防火墙规则，记录 192.168.0.1 至 192.168.0.22 地址段内所有的主机发送给路由要求转发的数据包，并允许转发这些数据包。

```
[root@router ~]# iptables -A FORWARD -m iprange --src-range 192.168.0.1-192.168.0.10 \
>-p tcp --dport 80 -j LOG
[root@router ~]# iptables -A FORWARD -m iprange --src-range 192.168.0.1-192.168.0.10 \
>-p tcp --dport 80 -j ACCEPT
```

6.1.3 防火墙备份与还原

默认的 iptables 防火墙规则会立刻生效，但如果不保存，当计算机重启后所有的规则都将丢失，所以对防火墙规则进行及时保存的操作是非常必要的。iptables 软件包提供了两个非常有用的工具，我们可以使用工具处理大量的防火墙规则。这两个工具分别是 iptables-save 和 iptables-restore，使用该工具实现防火墙规则的保存与还原。这两个工具的最大优势是处理庞大的规则集时速度非常快。CentOS 6.3 系统中防火墙规则默认保存在/etc/sysconfig/iptables 文件中，使用 iptables-save 将规则保存至该文件中可以实现保存防火墙规则的作用，计算机重启后会自动加载该文件中的规则。如果使用 iptables-save 将规则保存至其他位置，可以实现备份防火墙规则的作用。当防火墙规则需要做还原操作时，可以使用 iptables-restore 将备份文件直接导入当前防火墙规则。

```
[root@centos6 ~]# iptables-save > /etc/sysconfig/iptables
[root@centos6 ~]# iptables-save > firewall.bak
[root@centos6 ~]# iptables-restore < firewall.bak
```

经过 iptables-save 命令导出的规则文件格式类似于如下内容：

```
# Generated by iptables-save v1.4.7 on Thu May 23 10:46:00 2013
*nat
:PREROUTING ACCEPT [54:4212]
:POSTROUTING ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
-A PREROUTING -d 172.16.0.211/32 -p tcp -m tcp --dport 80 -j DNAT
--to-destination 192.168.0.100
COMMIT
# Completed on Thu May 23 10:46:00 2013
# Generated by iptables-save v1.4.7 on Thu May 23 10:46:00 2013
*filter
:INPUT ACCEPT [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [503:63104]
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -m state --state NEW -j DROP
-A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -p tcp -m iprange --src-range 192.168.0.1-192.168.0.10 -m tcp
--dport 80 -j LOG
-A FORWARD -p tcp -m iprange --src-range 192.168.0.1-192.168.0.10 -m tcp
--dport 80 -j LOG
-A OUTPUT -m string --string "qq" --algo bm --to 65535 -j REJECT --reject-with
icmp-port-unreachable
-A OUTPUT -m string --string "qq.com" --algo bm --to 65535 -j REJECT
--reject-with icmp-port-unreachable
```

```
    -A OUTPUT -m string --string "www.qq.com" --algo bm --to 65535 -j REJECT
--reject-with icmp-port-unreachable
    COMMIT
    # Completed on Thu May 23 10:46:00 2013
```

6.2　SELinux 简介

SELinux（Security-Enhanced Linux）是基于 Linux 内核的强制访问控制机制实现，它是由美国国家安全局开发的项目，旨在增强传统 Linux 操作系统的安全性。SELinux 项目在 2000 年以 GPL 协议形式开源，目前所有的 Linux 内核 2.6 及以上版本中都集成了 SELinux 功能。使用 SELinux 后，系统中的文件、目录、设备甚至端口都作为对象，而用户运行的进程则被当作主题，一个主题能不能访问对象，首先系统会检查传统的账户权限是否允许（DAC 控制权限），如果传统的基于账户的访问权限允许，才会检查 SELinux 的强制访问控制（MAC 访问控制），SELinux 依靠策略来决定是否允许主题访问目标对象。

使用 SELinux 后所有的进程与文件都被标记为一种类型，类型定义了进程的操作域，同时也定义了文件的类型。每个进程都被限制运行在自己独立的域中，由 SELinux 策略规则来定义进程与文件[1]资源以及进程与进程之间的访问权限。所有对资源的访问，仅当有明确的策略规则时才被允许。当有黑客入侵你的某个服务时，也仅仅会影响到相应进程的域，对整个操作系统甚至其他进程域都不会产生影响。传统的 DAC 访问控制依靠用户及组的 ID 号进行权限判断，SELinux 则是基于多个有效的安全标记判断权限，例如，SELinux 用户、角色、类型、级别。

6.2.1　SELinux 配置文件

在 CentOS 6.3 系统中部署 SELinux 非常简单，由于 SELinux 已经作为模块集成到内核中，默认 SELinux 已经处于激活状态。对管理员来说，更多的是需要配置与管理 SELinux，CentOS 6.3 系统中 SELinux 全局配置文件为/etc/sysconfig/selinux，内容如下：

```
[root@centos6 ~]# vim /etc/sysconfig/selinux
```

```
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#     enforcing - SELinux security policy is enforced.
#     permissive - SELinux prints warnings instead of enforcing.
```

[1] 在 Linux 操作系统中一切皆文件（Everything is file）。

```
#     disabled - No SELinux policy is loaded.
SELINUX=enforcing
# SELINUXTYPE= can take one of these two values:
#     targeted - Targeted processes are protected,
#     mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

全局配置文件中除去以#符号开头的注释行，有效配置参数仅两行。SELinux=enforcing 为 SELinux 总开关，有效值可以是 enforcing、permissive 或 disabled。其中，disabled 代表禁用 SELinux 功能，由于 SELinux 是内核模块功能，所以如果设置禁用，需要重启计算机。permissive 代表仅警告模式，处于此状态下时，当主题程序试图访问无权限的资源时，SELinux 会记录日志但不会拦截该访问，也就是最终访问是成功的，只是在 SELinux 日志中记录而已。enforcing 模式代表强制开启，SELinux 会拦截非法的资源访问并记录相关日志。SELNUXTYPE=targeted 设置项，用来设置 SELinux 类型，可以设置为 targeted 类型或 mls 类型，targeted 类型主要对系统中的服务进程进行访问控制，mls 类型将对系统中的所有进程进行控制，启用 mls 后，用户执行简单的命令（如 ls）都会报错。计算机的入侵多数是来自网络的，所以企业中多数采用 targeted 作为 SELinux 的控制类型。使用 getenforce 可以查看系统当前 SELinux 的运行模式。

使用 setenforce 可以临时在 enforcing 模式与 permissive 模式之间切换，切换会被立刻应用于当前系统，计算机重启后无效，永久修改模式需要修改配置文件。

```
[root@centos6 ~]# setenforce 0          #设置 SELinux 为 permissive 模式
[root@centos6 ~]# setenforce 1          #设置 SELinux 为 enforcing 模式
```

6.2.2 SELinux 软件包

CentOS 6.3 系统除了提供基本的 SELinux 功能外，还提供了一些管理工具。下面是对 SELinux 相关软件包的介绍。

- policycoreutils：该软件包提供了用于管理 SELinux 的 restorecon、secon、setfiles、semodule、load_policy 和 setsebool 命令。
- selinux-policy：该软件包负责提供 SELinux 参考策略，还提供了一个开发工具 /usr/share/selinux/devel/policygentool。
- selinux-policy-targeted：提供 targeted 类型的策略。
- libselinux-utils：该软件包提供 avcstat、getenforce、getsebool、setenforce、matchpathcon、selinuxconlist、selinuxdefcon、selinuxenabled、togglesebool 命令管理工具。
- libselinux-python：该软件包提供了 SELinux 应用开发的 Python 资源。

- libselinux：该软件包包含 SELinux 应用程序的 API。
- selinux-policy-mls：该软件包提供了 MLS 安全策略，启用 MLS 模式时需要安装该软件包。
- setroubleshoot：该软件包提供了 SELinux 故障排查工具。
- setroubleshoot-server：该软件包可以将 AVC 拒绝日志消息转换为细节描述，使用 sealert 命令可以查看 SELinux 日志的详细信息。
- policycoreutils-python：该软件包提供了 SELinux 管理工具，包括 semanage、audit2allow、audit2why 以及 chcat。
- policycoreutils-gui：该软件包提供了 system-config-selinux 图形化 SELinux 管理工具。

6.2.3　SELinux 安全上下文

SELinux 会为进程与文件添加安全信息标签，如：SELinux 用户、角色、类型以及可选的级别。当运行 SELinux 后所有这些信息都是访问控制的依据。下面通过一个实例文件查看 SELinux 安全上下文，使用 ls -Z 命令就可以看到文件或目录的这些上下文信息，而 ps aux –Z 则可以查看进程的安全上下文信息：

```
[root@centos6 ~]# ls -Z anaconda-ks.cfg
-rw-------. root root system_u:object_r:admin_home_t:s0 anaconda-ks.cfg
[root@centos6 ~]# ps aux -Z
```

SELinux 的安全上下文包括

用户：角色：类型：级别

每列具体描述如下。

（1）SELinux 用户

用户身份是通过 SELinux 策略授权特定角色集合的账户身份，每个系统账户都通过策略映射到一个 SELinux 用户。使用 root 身份运行 semanage login -l 命令可以查看系统账户与 SELinux 账户之间的映射关系。

```
[root@centos6 ~]# semanage login -l
Login Name           SELinux User                 MLS/MCS Range
__default__          unconfined_u                 s0-s0:c0.c1023
Root                 unconfined_u                 s0-s0:c0.c1023
system_u             system_u                     s0-s0:c0.c1023
```

（2）SELinux 角色

SELinux 部分采用基于角色的访问控制(RBAC)模型，而角色是 RBAC 的重要属性。SELinux 账户被授予特定的角色，而角色被授予操控特定的域。角色是 SELinux 用户与域的媒介。

（3）SELinux 类型

SELinux 类型是 Type Enforcement 的重要属性，类型定义了进程的域以及文件的类型。而 SElinux 策略规则定义了何种类型的主题可以访问其他何种类型的对象资源，仅当 SELinux 策略明确存在允许规则时，访问才可以被接受。

（4）SELinux 级别

级别是 MLS 和 MCS 的属性，一个 MLS 范围是一对级别，书写格式为低级别-高级别，如果两个级别是一致的，也可以仅显示低级别，如，s0-s0 与 s0 是一样的。

当我们使用 passwd 命令修改账户密码时，系统会运行/usr/bin/passwd 程序，该程序运行在 passwd_exec_t 域中，修改密码需要修改/etc/shadow 文件，而该文件的类型为 shadow_t，SELinux 默认的策略规则中允许运行在 passwd_t 域中进程读写被标记为 shadow_t 类型的文件。

6.2.4 SELinux 排错

不管 SELinux 策略是允许还是拒绝资源的访问请求行为，都会记录日志，也就是 AVC（Access Vector Cache）。所有 SELinux 拒绝的消息都会被记录进日志，根据系统中安装运行的服务进程不同，拒绝日志消息会被记录到不同的文件中，表 6-1 列出了进程与日志文件的关系。

表 6-1

日志文件	进程
/var/log/audit/audit.log	auditd 服务开启
/var/log/messages	auditd 服务关闭，rsyslogd 服务开启
/var/log/audit/audit.log，/var/log/messages	安装 setroubleshoot 相关软件包 autitd 与 rsyslogd 同时开启

如果 CentOS 系统中安装并运行了 X Windows、setroubleshoot 以及 setroubleshoot-server 软件包，并且 auditd 服务开启后，SELinux 会提示如图 6-3 所示的报警窗口。单击 Show 可以查看 SELinux 报警的详细信息，并且会显示可能的解决方案。使用 sealert -b 命令同样可以显示该图形警告窗口。

图 6-3

对于大多数生产环境中的服务器而言，更多的是没有部署安装图形界面的 Linux 系统，我们需要手动查看日志文件。在此建议管理员安装 setroubleshoot 相关的软件包，这样可以将原本生涩的 AVC 拒绝日志转换为可读性比较高的 setroubleshoot 日志。查看日志可以使用如下两种方法：

```
[root@centos6 ~]# grep setroubleshoot /var/log/messages
[root@centos6 ~]# grep denied /var/log/audit/audit.log
```

查看 messages 日志会提示，根据黑体字提示运行 sealert 命令才可以看到人性化报错信息。

```
    setroubleshoot: SELinux is preventing /usr/sbin/httpd from read access on
the file index.html. For complete SELinux messages. run sealert -l
7082b8b4-70f4-42fb-92ea-08a51299d080
```

```
[root@centos6 ~]# sealert -l 7082b8b4-70f4-42fb-92ea-08a51299d080
```

```
    SELinux is preventing /usr/sbin/httpd from getattr access on the file
/var/www/html/index.html.

    *****  Plugin restorecon (99.5 confidence) suggests  ***********************

    If you want to fix the label.
    /var/www/html/index.html default label should be httpd_sys_content_t.
    Then you can run restorecon.
    Do
    # /sbin/restorecon -v /var/www/html/index.html

    *****  Plugin catchall (1.49 confidence) suggests  **************************

    If you believe that httpd should be allowed getattr access on the index.html
file by default.
    Then you should report this as a bug.
    You can generate a local policy module to allow this access.
    Do
    allow this access for now by executing:
    # grep httpd /var/log/audit/audit.log | audit2allow -M mypol
    # semodule -i mypol.pp
```

以上报错信息说明：SELinux 策略拒绝了/usr/sbin/httpd 程序访问/var/www/html/ index.html 文件，如果你希望修复该问题，可以重新修改 index.html 文件的安全上下文，默认该文件的安

全上下文应该为 httpd_sys_content_t，我们可以使用/sbin/restorecon -v /var/www/html/index.html 恢复该文件的安全上下文。

6.2.5 修改安全上下文

有多种方式可以修改与管理 SELinux 安全上下文，如：chcon、semanage、fcontext 以及 restorecon 命令。

1. chcon 命令

描述：修改文件 SELinux 安全上下文。

用法：chcon [选项] [-u SELinux 用户] [-r 角色] [-l 范围] [-t 类型] 文件
chcon [选项] --reference=参考文件 文件

选项：-u 修改用户属性
-r 修改角色属性
-l 修改范围属性
-t 修改类型属性

示例：

（1）修改文件安全上下文。

```
[root@centos6 ~]# cp --preserve=all /etc/passwd /root/ #复制文件（保留上下文信息）
[root@centos6 ~]# ls -Z /root/passwd                   #查看文件 SELinux 安全上下文
[root@centos6 ~]# chcon -t admin_home_t /root/passwd #修改文件安全上下文中的类型
[root@centos6 ~]# ls -Z /root/passwd
```

（2）修改目录安全上下文。

```
[root@centos6 ~]# chcon -R -t admin_home_t /root/    #递归修改目录安全上下文
```

（3）根据参考文件修改目标文件安全上下文。

```
[root@centos6 ~]# chcon --reference=/etc/passwd /root/passwd
```

通过 chcon 修改的安全上下文并不是 SELinux 预设的安全上下文，当文件系统重置 SELinux 安全标签或使用 restorecon 命令重置指定目录的安全标签后，所有文件与目录的安全标签会被还原为系统预设值，如果需要修改 SELinux 默认的预设安全上下文，需要使用 semanage 命令添加或修改。

2. semanage 命令

描述：SELinux 策略管理工具。

用法：semanage fcontext [-S store] -{a|d|m|l|n|D} [-frst] file_spec

选项：-a,--add　　添加预设安全上下文
　　-d,--delete　　删除指定的预设安全上下文
　　-D,--deleteall　　删除所有的预设自定义上下文
　　-m,--modify　　修改指定的预设安全上下文
　　-l,--list　　显示预设安全上下文
　　-n,--noheading　　不显示头部信息

示例：

（1）查看 SELinux 策略默认的预设安全上下文信息，系统将列出策略中定义的所有目录与安全上下文信息。

```
[root@centos6 ~]#semanage fcontext -l
```

（2）修改策略，添加一条新的预设安全上下文信息。

```
[root@centos6 ~]# semanage fcontext -a -t samba_share_t /test/test.txt
[root@centos6 ~]# mkdir /test; touch /test/test.txt
[root@centos6 ~]# ls -Z /test/test.txt
```

（3）使用 restorecon 命令，还原 test.txt 文件的安全上下文为预设值。

```
[root@centos6 ~]# restorecon /test/test.txt
[root@centos6 ~]# ls -Z /test/test.txt
```

（4）递归设置目录的预设安全上下文。

```
[root@centos6 ~]# semanage fcontext -a -t httpd_sys_content_t "/site/www(/.*)?"
[root@centos6 ~]# mkdir -p /site/www/{web1,web2}
[root@centos6 ~]# touch /site/www/{web1,web2}/index.html
[root@centos6 ~]# ls -RZ /site/www
[root@centos6 ~]# restorecon -R /site/
```

（5）删除预设安全上下文。

```
[root@centos6 ~]# semanage fcontext -d /test/ test.txt
```

（6）检查预设 SELinux 安全上下文。

```
[root@centos6 ~]# matchpathcon /site/www/
```

6.2.6 查看与修改布尔值

SELinux 布尔值可以实时被修改。如，你可以在不重新加载或编译 SELinux 策略的情况下允许服务访问 NFS 文件系统。getsebool 是用来查看 SELinux 布尔值的命令，用法比较简单，-a 选项用来查看所有的布尔值。一般建议管理员通过管道过滤自己需要的布尔值参数，如 getsebool -a |grep ftp 过滤与 FTP 相关的布尔值信息，显示效果中左侧为关键词，右侧为开关，on 代表开，off 代表关，具体命令如下。

```
[root@centos6 ~]# getsebool -a
abrt_anon_write → off
abrt_handle_event → off
allow_console_login → on
allow_cvs_read_shadow → off
allow_daemons_dump_core → on
allow_daemons_use_tcp_wrapper → off
allow_daemons_use_tty → on
allow_domain_fd_use → on
… …部分内容省略… …
```

除 getsebool 命令外，还可以使用 semanage boolean -l 命令，该命令的输出结果如下（与 getsebool 命令相比，输出信息多了默认状态与当前状态以及相关描述信息）。

```
SEboolean                State  Default                    Description
ftp_home_dir             (off  ,  off)        Allow ftp to read and write files in
the user home directories
smartmon_3ware           (off  ,  off)        Enable additional permissions needed
to support devices on 3ware controllers.
Xdm_sysadm_login         (off  ,  off)        Allow xdm logins as sysadm
xen_use_nfs              (off  ,  off)        Allow xen to manage nfs files
Mozilla_read_content(off  ,  off)        Control Mozilla content access
……部分内容省略……
```

修改 SELinux 布尔值状态也非常简单，使用 setsebool name X 即可实现。其中，name 是布尔值名称，X 代表 on 或 off。默认 setsebool 命令修改的布尔值参数会立即生效，但计算机重启后会被还原，如果希望永久修改，需要使用-p 参数。

```
[root@centos6 ~]# setsebool ftp_home_dir on
[root@centos6 ~]# setsebool -p ftp_home_dir on
```

6.2.7　SELinux 应用案例

实际应用中，建议管理员先将 SELinux 模式调整为 permissive 模式，经过大量的测试后，再实际部署到生产环境。在部署 SELinux 环境的过程中，遇到问题可以参考各种日志文件。另外，CentOS 6.3 系统中还提供了专门的 SELinux 帮助文档，这些帮助文件多数以_selinux 结尾。使用 man -k key 命令可以帮助我们查看与某个关键词相关的帮助手册。但前提是系统已经使用 makewhatis 创建了 whatis 数据库。

```
[root@centos6 ~]# makewhatis                          #生成 whatis 数据库
[root@centos6 ~]# man -k selinux                      #查找 selinux 相关帮助
[root@centos6 ~]# man -k selinux |grep ftp            #查找与 ftp 相关的 SELinux 帮助
```

6.2.8　httpd 相关的 SELinux 安全策略

1. 布尔值

SELinux 策略是可定制的，SELinux 针对 httpd 的策略非常灵活，大量的布尔值可以帮助管理员快速维护与管理相关策略，实现安全快捷的访问策略。

允许 httpd 脚本或模块通过网络连接数据库：

```
[root@centos6 ~]# setsebool -P httpd_can_net-work_connect_db 1
```

允许 httpd 支持 CGI 程序：

```
[root@centos6 ~]# setsebool -P httpd_enable_cgi 1
```

允许 httpd 访问 cifs 文件系统资源：

```
[root@centos6 ~]# setsebool -P httpd_use_cifs 1
```

允许 Apache 使用 mod_auth_pam 模块：

```
[root@centos6 ~]# setsebool  -P allow_httpd_mod_auth_pam 1
```

允许 httpd 访问 NFS 文件系统资源：

```
[root@centos6 ~]# setsebool -P httpd_use_nfs 1
```

允许 http 守护进程发送电子邮件：

```
[root@centos6 ~]# setsebool -P httpd_can_sendmail 1
```

允许 httpd 连接网络 memcache 服务器：

```
[root@centos6 ~]# setsebool -P httpd_can_network_memcache 1
```

2. 安全上下文

如果希望多个进程域（如，Apache、FTP、rsync 等）共享相同的文件，我们可以设置文件安全上下文为 public_content_t 或者 public_content_rw_t，这些安全上下文允许上面提到的所有服务进程域读取文件内容，如果修改为可以读写，则需要 public_content_rw_t 的类型标签。

通过添加 public_content_t 类型标签，允许 httpd 服务读取/var/httpd 目录：

```
[root@centos6 ~]# semanage fcontext -a -t public_content_t "/var/httpd(/.*)?"
[root@centos6 ~]# restorecon -F -R -v /var/httpd
```

通过添加 public_content_rw_t 类型标签，允许 httpd 可读写/var/web 目录及子目录。注意，该设置需要开启布尔值 allow_httpd_anon_write：

```
[root@centos6 ~]# semanage fcontext -a -t public_content_rw_t "/var/web(/.*)?"
[root@centos6 ~]# restorecon -F -R -v /var/httpd/incoming
```

具体的文件与目录资源的安全上下文描述信息见表 6-2。

表 6-2

文件与目录描述	安全上下文类型标签
/var/cache 缓存目录资源	httpd_cache_t
Apache 配置文件	httpd_config_t
作为 CVS 内容的文件资源	httpd_cvs_content_t
Apache 日志文件资源	httpd_log_t
httpd 代理内容资源	httpd_squid_content_t
httpd 系统资源	httpd_sys_content_t
可读写 httpd 系统资源	httpd_sys_rw_content_t

6.2.9 FTP 相关的 SELinux 安全策略

1. 布尔值

允许 ftp 读写用户家目录中的数据：

```
[root@centos6 ~]# setsebool -P ftp_home_dir 1
```

允许本地账号登录 ftp 可以读写文件系统中的所有文件：

```
[root@centos6 ~]# setsebool -P allow_ftpd_full_access 1
```

允许 ftp 连接数据库：

```
[root@centos6 ~]# setsebool -P ftpd_connect_db 1
```

允许 ftp 共享 cifs 文件系统：

```
[root@centos6 ~]# setsebool -P allow_ftpd_use_cifs 1
```

允许 ftp 共享 NFS 文件系统：

```
[root@centos6 ~]# setsebool -P allow_ftpd_use_nfs 1
```

2. 安全上下文

添加预设的 SELinux 安全上下文的类型属性，允许 ftp 读取/var/ftp 目录：

```
[root@centos6 ~]# semanage fcontext -a -t public_content_t "/var/ftp(/.*)?"
[root@centos6 ~]# restorecon -F -R -v /var/ftp
```

添加预设的 SELinux 安全上下文的类型属性，允许 ftp 读写/var/ftp 目录：

```
[root@centos6 ~]# semanage fcontext -a -t public_content_rw_t "/var/ftp(/.*)?"
[root@centos6 ~]# restorecon -F -R -v /var/ftp
```

具体的文件与目录资源的安全上下文描述信息见表 6-3。

表 6-3

文件与目录描述	安全上下文类型标签
/etc/目录下的 ftp 文档	ftpd_etc_t
控制 ftp 程序仅在 ftpd_t 域下运行	ftpd_exec_t
控制 ftp 程序仅在 ftpd_initrc_t 域中运行	ftpd_initrc_exec_t
ftp 锁数据文件	ftpd_lock_t
ftp 在/tmp 目录下生成的临时文件	ftpd_tmp_t

6.2.10　MySQL 相关的 SELinux 安全策略

1. 布尔值

允许用户连接 mysql 服务器：

```
[root@centos6 ~]# setsebool -P allow_user_mysql_connect 1
```

允许 mysqld 服务连接所有的端口号：

```
[root@centos6 ~]# setsebool -P mysql_connect_any 1
```

2. 安全上下文

具体的文件与目录资源的安全上下文描述信息见表 6-4。

表 6-4

文件与目录描述	安全上下文类型标签
mysqld 数据库文件	mysqld_db_t
存储在/etc/目录下的 mysql 文件	mysqld_etc_t
控制 mysql 程序仅在 mysqld_t 域中运行	mysqld_exec_t
控制 mysql 程序仅在 mysqld_initrc_t 域中运行	mysqld_initrc_exec_t
ftp 在/tmp 目录下生成的临时文件	ftpd_tmp_t
控制 mysql 程序仅在 mysqld_safe_t 域中运行	mysqld_safe_exec_t
mysql 存储在/tmp 目录下的临时文件	mysqld_tmp_t
mysql 存储在/var/run 目录下的文件	mysqld_var_run_t

6.2.11 NFS 相关的 SELinux 安全策略

1. 布尔值

允许 ftp 服务器使用 NFS 文件系统共享：

```
[root@centos6 ~]# setsebool -P allow_ftpd_use_nfs 1
```

允许 Git 进程访问 NFS 文件系统：

```
[root@centos6 ~]# setsebool -P git_system_use_nfs 1
```

允许 qemu 使用 NFS 文件系统

```
[root@centos6 ~]# setsebool -P qemu_use_nfs 1
```

允许 rsync 服务器共享 NFS 文件系统：

```
[root@centos6 ~]# setsebool -P rsync_use_nfs 1
```

允许 Cobbler 访问 NFS 文件系统：

```
[root@centos6 ~]# setsebool -P cobbler_use_nfs 1
```

2. 安全上下文

具体的文件与目录资源的安全上下文描述信息见表 6-5。

表 6-5

文件与目录描述	安全上下文类型标签
nfs 数据文件	nfs_t
控制 NFS 程序仅在 nfsd_t 域中运行	nfsd_exec_t
控制 NFS 程序仅在 nfsd_initrc_exec_t 域中运行	nfsd_initrc_exec_t
设置文件为只读 NFS 文件	nfsd_ro_t
设置文件为可读写 NFS 文件	nfsd_rw_t

6.2.12　Samba 相关的 SELinux 安全策略

1. 布尔值

允许 samba 作为域控制添加账户和修改密码：

```
[root@centos6 ~]# setsebool -P samba_domain_controller 1
```

允许 samba 以只读方式共享任意文件或目录：

```
[root@centos6 ~]# setsebool -P samba_export_all_ro 1
```

允许 samba 以可读写方式共享任意文件或目录：

```
[root@centos6 ~]# setsebool -P samba_export_all_rw 1
```

开启 samba 家目录：

```
[root@centos6 ~]# setsebool -P use_samba_home_dirs 1
```

允许 samba 共享账户家目录：

```
[root@centos6 ~]# setsebool -P samba_enable_home_dirs 1
```

2. 安全上下文

具体的文件与目录资源的安全上下文描述信息见表 6-6。

表 6-6

文件与目录描述	安全上下文类型标签
samba 存储在/etc/目录下的文件	samba_etc_t
samba 存储在/tmp/目录下的文件	samba_net_tmp_t
samba 存储在/var/目录下的文件	samba_var_t
samba 密码文件	samba_secrets_t

续表

文件与目录描述	安全上下文类型标签
samba 共享文件	samba_share_t
samba 日志文件	samba_log_t
控制 samba 程序仅在 samba_initrc_t 域中运行	samba_initrc_exec_t
控制 samba 程序仅在 samba_net_t 域中运行	samba_net_exec_t

6.3 OpenVPN

6.3.1 OpenVPN 简介

虚拟专用网络（Virtual Private Network，VPN）是一种利用公共网络设施创建私有专线连接的技术。对于一个大型企业而言，往往存在多个地区，有多个分支结构，这些分支结构与总部之间的连接共享数据需要使用专线，但租用电信相关部门的专线业务需要大量的资金。而 VPN 的作用正是利用现有的公共线路，创建属于自己的专有连接。另外，当 IT 管理员需要在家连接公司网络时，使用 VPN 专线连接也是绝佳的选择，VPN 使用的是加密连接，可以确保数据的安全性。

OpenVPN 是基于 GPL 协议开源的 VPN 实现，OpenVPN 可以使用密钥、证书或账户进行身份验证，本节案例中采用的是证书验证身份。OpenVPN 使用 OpenSSL 库进行加密与证书的管理。OpenVPN 可以使用 TCP 或 UDP 建立数据连接，可以创建二层（TAP）或三层（TUN）的 VPN 连接，该软件可以运行在 Mac OS、Windows、Linux 等多个操作系统平台上，对企业环境中复杂的系统平台而言是一个完美的解决方案。图 6-4 展示了 VPN 的拓扑结构，本例将围绕该拓扑图部署 VPN 环境。客户端（vpnclient.example.com）可以在任意位置通过互联网接入到公司的 VPN 服务器，VPN

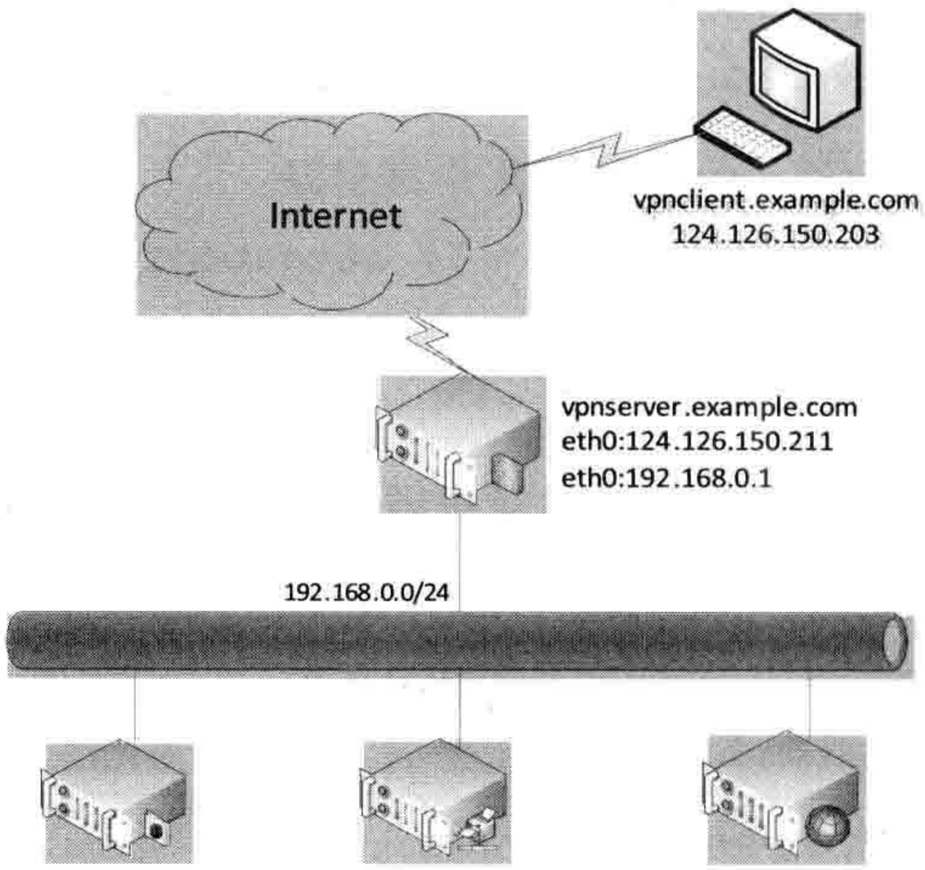


图 6-4

服务器接入的客户端分配 10.8.0.0/24 网段的 IP，开启 VPN 服务器的路由转发功能后，即可实现外部客户端计算机与公司内部服务器（192.168.0.0/24 网络）之间的互联互通。

6.3.2　安装 OpenVPN 服务

OpenVPN 软件需要调用其他库文件，在安装该软件前需要安装相关的依赖包，CentOS 官方提供的 YUM 源中都包含这些软件，可以直接使用 YUM 方式安装。对于 OpenVPN 主软件包，我们可使用从官方网站下载的源码包方式安装。需要注意的是，在国内网络环境下，对官方网站（http://www.openvpn.net）的访问是有限制的，需要通过其他途径下载相关的软件包，比如代理软件等。本例中使用的是证书方式进行验证，easy-rsa 软件包的作用是提供一系列的脚本，用于自动生成相关密钥与证书文件，安装该软件后需要将相关脚本与目录复制到 openvpn 软件根路径。

```
[root@vpnserver ~]# yum -y install lzo lzo-devel openssl openssl-devel\
>gcc pam-devel
[root@vpnserver ~]# tar -xzf openvpn-2.3.1.tar.gz -C /usr/src/
[root@vpnserver ~]# ./configure --prefix=/usr/local/openvpn
[root@vpnserver ~]# make && make install
[root@vpnserver ~]# rpm -vih easy-rsa-2.2.0_master-1.noarch.rpm
[root@vpnserver ~]# cp -r /usr/share/doc/easy-rsa/ /usr/local/openvpn/
```

修改 vars 文件，该文件中保存的是生成证书所需的默认信息参数，如公司名称、电子邮件等信息。

```
[root@vpnserver ~]# vim /usr/local/openvpn/easy-rsa/vars
```

```
… …部分内容省略… …
export KEY_COUNTRY="CN"                          #国家名称
export KEY_PROVINCE="Beijing"                    #省份名称
export KEY_CITY="Beijing"                        #城市名称
export KEY_ORG="Jacob"                           #组织名称
export KEY_EMAIL="test@gmail.com"                #电子邮箱
export KEY_CN="vpnserver.example.com"            #备注名称
export KEY_NAME="server"                         #备注字串
export KEY_OU="tech"                             #部门名称
```

```
[root@vpnserver ~]# source ./vars                #执行该脚本，读取新的信息参数
[root@vpnserver ~]# ./clean-all                  #清空现有的所有证书与密钥
[root@vpnserver ~]# ./build-ca                   #创建新的 CA 根证书
Generating a 1024 bit RSA private key
.....................++++++
.....................................++++++
```

```
writing new private key to 'ca.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [CN]:
State or Province Name (full name) [Beijing]:
Locality Name (eg, city) [Beijing]:
Organization Name (eg, company) [Jacob]:
Organizational Unit Name (eg, section) [tech]:
Common Name (eg, your name or your server's hostname) [vpnserver.example.com]
Name [server]:
Email Address [test@gmail.com]:
```

创建 CA 根证书时，对于证书的国家、城市等信息，该脚本会读取 vars 文件中保存的默认值，如果不需要修改，则直接按回车键确认即可。

build-key-server 脚本用来创建 VPN 服务器私钥，在该过程中，脚本会提示是否使用 CA 对该密钥进行签名生成相应的证书。使用脚本生成 vpnserver.key（私钥文件）、vpnserver.csr（证书请求文件）以及 vpnserver.crt（证书文件），除了创建服务器所需密钥外，还需要使用同样的命令为客户端计算机生成密钥与证书，并将相关密钥与证书复制到客户端计算机 openvpn 软件的相关位置，具体位置根据客户端软件安装与部署的实际情况而定。

```
[root@vpnserver ~]# ./build-key-server vpnserver
……部分输出省略……
Country Name (2 letter code) [CN]:
State or Province Name (full name) [Beijing]:
Locality Name (eg, city) [Beijing]:
Organization Name (eg, company) [Jacob]:
Organizational Unit Name (eg, section) [tech]:
Common Name (eg, your name or your server's hostname) [vpnserver]:vpnserver.
example.com
Name [server]:
Email Address [test@gmail.com]:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
Using configuration from /usr/local/openvpn/easy-rsa/openssl-1.0.0.cnf
Check that the request matches the signature
```

```
Signature ok
The Subject's Distinguished Name is as follows
countryName                   :PRINTABLE:'CN'
stateOrProvinceName           :PRINTABLE:'Beijing'
localityName                  :PRINTABLE:'Beijing'
organizationName              :PRINTABLE:'Jacob'
organizationalUnitName        :PRINTABLE:'tech'
commonName                    :PRINTABLE:'vpnserver.example.com'
name                          :PRINTABLE:'server'
emailAddress                  :IA5STRING:'test@gmail.com'
Certificate is to be certified until May 27 16:27:45 2023 GMT (3650 days)
Sign the certificate? [y/n]:y
1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
[root@vpnserver ~]# ./build-dh          #创建 Diffie Hellman 参数
```

使用 easy-rsa 提供的脚本已经创建了加密数据所需要的密钥与证书，下一步需要将这些密钥与数据复制到 OpenVPN 软件根路径下，并将 openvpn 软件所提供的主配置文件模板复制至 /usr/local/openvpn 目录，修改文件名称为 server.conf。

```
[root@vpnserver ~]# cd /usr/local/openvpn/easy-rsa/keys
[root@vpnserver ~]# cp ./{ca.crt,vpnserver.crt,vpnserver.key,dh1024.pem}\
>/usr/local/openvpn/
[root@vpnserver ~]# cp /usr/src/openvpn-2.3.1/sample/sample-config-files/\
>server.conf/usr/local/openvpn/
[root@vpnserver ~]# vim /usr/local/openvpn/server.conf
```

```
port 1194                                          #设置端口号，默认为 1194
proto udp                                          #设置连接协议
dev tun                                            #创建模拟三层 VPN
ca /usr/local/openvpn/ca.crt                       #调用 CA 证书文件
cert /usr/local/openvpn/vpnserver.crt              #调用服务器证书文件
key /usr/local/openvpn/vpnserver.key               #调用服务器私钥文件
dh /usr/local/openvpn/dh1024.pem                   #调用 Diffie Hellman 文件
server 10.8.0.0 255.255.255.0                      #设置客户端获取的地址池范围
ifconfig-pool-persist ipp.txt                      #客户端虚拟 IP 记录文件
push "route 124.126.150.211"                       #设置客户端获取的网关信息
push "dhcp-option DNS 202.106.0.20"                #设置客户端获取的 DNS 信息
… …其他内容省略… …
```

修改完成 OpenVPN 配置文件后，使用 openvpn 启动服务器进程，--config 选项使用指定的配置文件启动服务，--daemon 选项代表作为守护进程启动服务。网络连接都是双向的，当客户

端连接位于 VPN 服务器后端的计算机时，位于 VPN 后端的计算机发送的数据包如果需要通过 VPN 服务器转发数据，则需要 VPN 服务器主机开启路由转发功能，并通过 iptables 实现 SNAT 功能。

```
[root@vpnserver ~]# /usr/local/openvpn/sbin/openvpn --config \
>/usr/local/openvpn/server.conf --daemon
[root@vpnserver ~]# vim /etc/sysctl.conf
```

```
net.ipv4.ip_forward = 1          #开启路由转发功能
```

```
[root@vpnserver ~]# sysctl -p
[root@vpnserver ~]# iptables -I INPUT -p udp --dport 1194 -j ACCEPT
[root@vpnserver ~]# iptables -t nat -I POSTROUTING -s 192.168.0.0/24 \
>-j SNAT --to-source 124.126.150.211
[root@vpnserver ~]# service iptables save
```

6.3.3 OpenVPN 客户端

OpenVPN 软件既可以作为服务端器程序，也可以实现客户端功能。由于企业环境的客户端系统多数情况下使用的都是 Windows 平台，因此，本例介绍的客户端也是基于 Windows 平台的。Windows 平台 OpenVPN 软件采用官网所提供的 openvpn-install-2.3.1-I001-i686.exe 安装程序，安装该软件时最好将所有的组件全部安装，默认该软件会安装在 C:\Program Files\OpenVPN 目录下，目录结构如表 6-7 所示。

表 6-7

目录位置	描述
C:\Program Files\OpenVPN\bin	OpenVPN 主程序目录
C:\Program Files\OpenVPN\config	OpenVPN 主配置文件及密钥目录
C:\Program Files\OpenVPN\doc	OpenVPN 文档目录
C:\Program Files\OpenVPN\easy-rsa	OpenVPN 自动生成密钥与证书的脚本
C:\Program Files\OpenVPN\log	OpenVPN 日志目录
C:\Program Files\OpenVPN\sample-config	OpenVPN 配置文件模板目录

因为密钥与证书文件可以在 VPN 服务器端生成，如果在部署 OpenVPN 服务器时已经为客户端生成了密钥与经过服务器 CA 签名的证书文件，就可以直接将密钥与证书复制至 C:\Program Files\OpenVPN\config 目录下，也需要将服务器的证书文件（vpnserver.crt）以及服务器 CA 证书（ca.crt）文件复制至 C:\Program Files\OpenVPN\config 目录下。另外，客户端的密钥文件也可以在客户端主机使用 easy-rsa 目录下的批处理程序生成，方法是在 easy-rsa 目录下创建 keys 子目

录，使用 clean-all.bat 批处理程序清空现有的密钥文件，build-key 生成私钥与证书请求文件。对于客户端主机自行生成的证书请求文件，还需要提交给服务器签名生成最终有效的证书文件。

接下来，需要创建并修改 OpenVPN 客户端配置文件，在 sample-config 目录下有客户端配置文件模板 client.ovpn，直接复制该文件到 config 目录，使用记事本修改该文件，内容如下：

```
client                                  #使 OpenVPN 作为客户端软件使用
dev tun
proto udp
dev-node vpnclient                      #设备节点名称，Windows 需要修改
remote 124.126.150.211 1194             #指定 VPN 服务器 IP 地址与端口号
ca ca.crt                               #指定服务器 CA 证书文件
cert vpnclient.example.com.crt          #指定客户端证书文件
key vpnclient.example.com.key           #指定客户端私钥文件
ns-cert-type server
```

需要注意的是，配置文件中的设备节点名称在 Windows 平台下是需要修改的，并且在安装完成 OpenVPN 软件后，客户端计算机中会多出一块虚拟网卡，我们还需要将该网卡的名称修改为 dev-node 定义的设备名称。设置方法是在 Windows 控制面板中找到网络连接，在网络连接中用鼠标右键单击该虚拟网卡，重命名为 vpnclient 即可。

最后，通过“开始”菜单启动 OpenVPN GUI 主程序，在任务栏将出现该程序的图标，如图 6-5 所示，用鼠标右键单击该图标，在弹出的快捷菜单中选择 Connect，即可实现客户端与 VPN 服务器之间的专线连接。使用 ping 命令测试客户端与服务器联通情况，效果如图 6-6 所示。

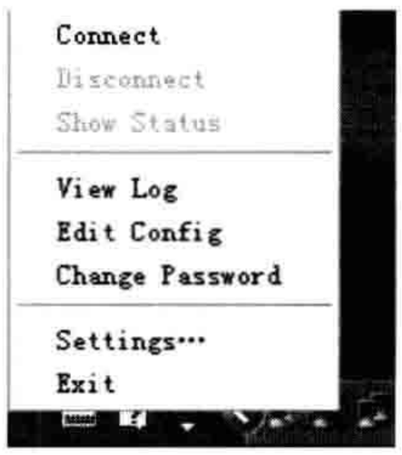


图 6-5

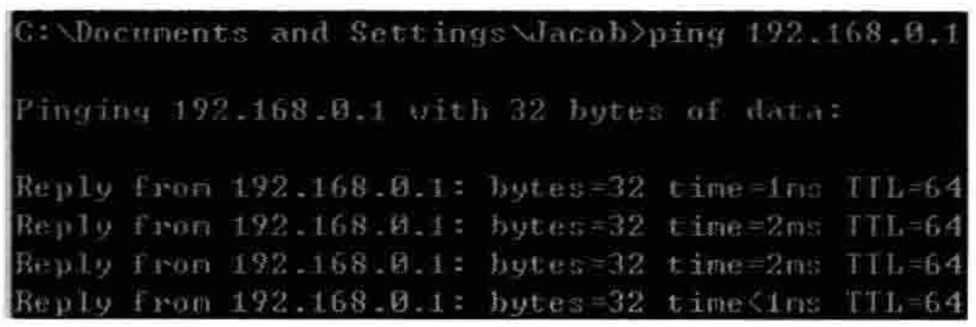
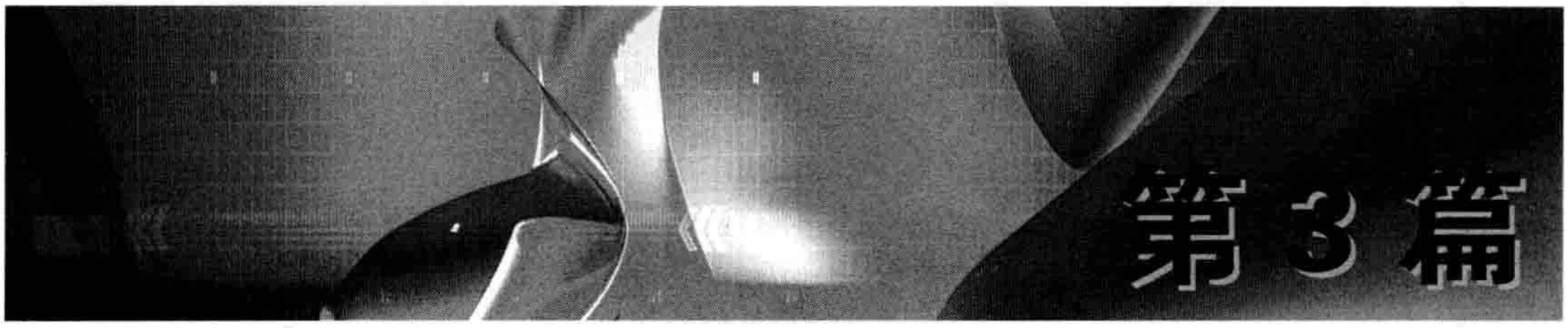


图 6-6



高级应用

第7章

虚拟化技术

7.1 虚拟化产品对比

计算机虚拟化技术是多种技术的综合实现，它包括硬件平台、操作系统、存储以及网络等。简单地说，虚拟化技术就是在单台主机上可以虚拟多个虚假主机，并可以在这些虚拟主机上运行不同的操作系统平台。虚拟化技术的出现可以节约大量的硬件资源与能源消耗，降低资金成本，虚拟化现在已经是每个企业必有的项目。目前所提供的比较成熟的虚拟化解决方案主要有VMware、Xen、KVM 以及 Kyper-V，下面针对其中的三个解决方案做简单功能的对比分析。虚拟化技术通过 Hypervisor 动态模拟与分配计算机硬件资源给虚拟机操作系统（Guest OS），由于 Hypervisor 可以模拟多个硬件资源给多个 Guest OS，因而对 Guest OS 而言，就像运行在独立、真实的硬件资源上一样，架构如图 7-1 所示。

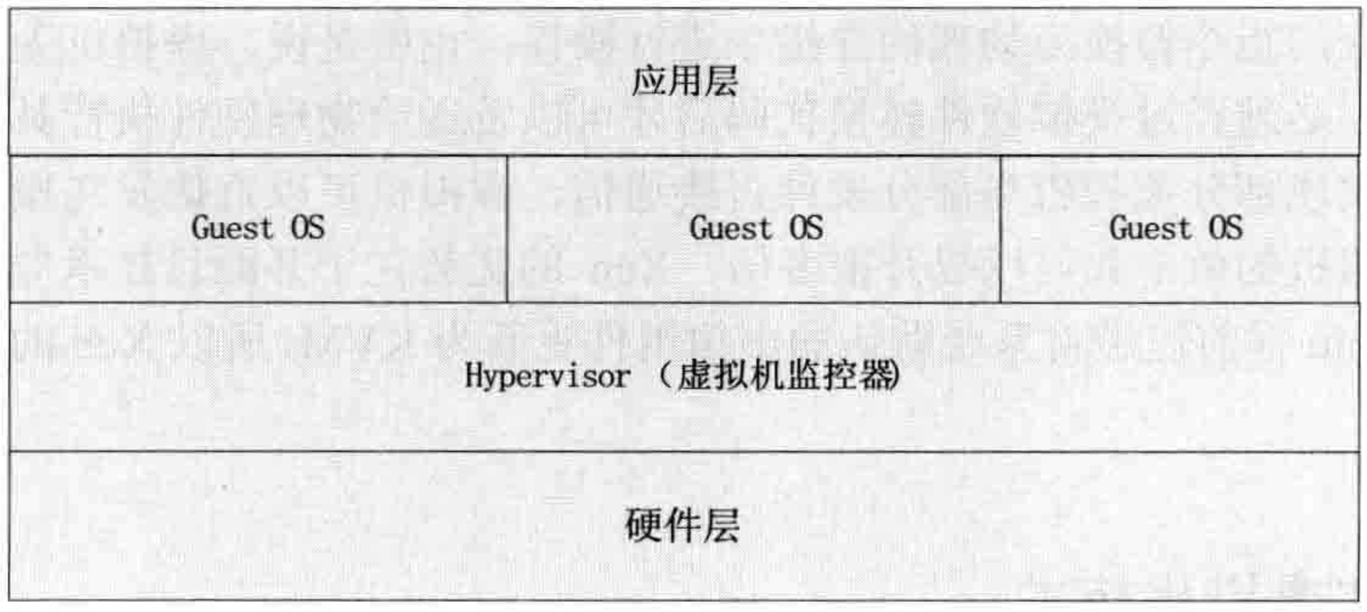


图 7-1

7.1.1　VMware 虚拟化技术

VMware 是全球数据中心解决方案的领导品牌，它为我们提供了高性能、高可用、管理方便的虚拟机管理程序，是目前大多数企业虚拟化环境部署的首选方案。VMware 虚拟化的工作原理是，直接在计算机硬件或主机操作系统上插入一个精简的软件层。该软件层包含一个以动态和透明方式分配硬件资源的虚拟化管理程序。多个操作系统可以同时运行在单台物理机上，彼此之间共享硬件资源。由于是将整台计算机（包括 CPU、内存、操作系统和网络设备）封装起来，因此，虚拟机可与所有标准的 x86 操作系统、应用程序和设备驱动程序完全兼容。可以同时在单台计算机上安全运行多个操作系统和应用程序，每个操作系统和应用程序都可以在需要时访问其所需的资源。

VMware 虚拟化产品最大的优势在于其完善的虚拟化管理平台以及可靠的基础架构，为企业计算环境提供完善、稳定、可靠的虚拟技术。到目前为止，VMware 依然是虚拟化技术行业的领头羊。VMware 虚拟化产品的不足主要包括：首先，VMware 虚拟化技术主要针对的是 x86 架构，对于复杂的企业运算环境，VMware 无法满足所有计算机主机的需要。其次，VMware 在云计算方面的发展略显迟缓，在性能与容灾方面表现不足。最后，VMware 是商业产品，用户需要为此支付大量的许可费用。

7.1.2　Xen 虚拟化技术

Xen 是由剑桥大学开发的开源虚拟机监控器，Xen 的目标是在单台主机上运行上百台全功能的虚拟机。Xen 支持半虚拟化与全虚拟化技术，半虚拟化技术的性能要比全虚拟化技术高很多，全虚拟化技术是通过中间层软件模拟真实的硬件设备，最终虚拟机使用模拟设备时就需要

中间层软件将相应的指令转换为物理硬件指令进行操作。也就是说，虚拟机无法直接发送操作指令给物理硬件，必须经过模拟软件转换代码后才可以发送给物理硬件执行具体操作。而半虚拟化技术则可以实现部分虚拟机与部分硬件直接通信，虚拟机可以直接发送操作指令给部分硬件设备，这样虚拟机的效率就可以提升很多倍。Xen 的优势在于开源且技术非常成熟，但随着 RedHat 以及 Ubuntu 目前已经将系统默认的虚拟组件更新为 KVM，所以 Xen 的市场已经开始受到影响。

7.1.3 KVM 虚拟化技术

KVM（Kernel-based Virtual Machine）是基于 x86 架构上 Linux 操作系统的全虚拟化解决方案。在 CentOS 6.3 系统中，KVM 已经被集成到内核中，相当于使用内核来做虚拟机管理程序。由于 KVM 本身就工作于内核环境中，所以执行效率要比传统意义上的虚拟化技术高很多。KVM 虚拟化需要 Intel VT 或 AMD-V 技术的支持，可以使用下面的命令确定本机 CPU 是否支持以上两种虚拟化技术：

```
[root@centos6 ~]# egrep '(vmx|svm)' /proc/cpuinfo
```

相对于 Xen 而言，KVM 从 Linux 2.6.20 后已经集成到内核模块中，使用 Linux 内核作为虚拟调度器管理虚拟机，更直接的硬件调度代表更高的效率，KVM 核心代码也非常短小精湛，未来 KVM 将逐渐成为主流虚拟化技术。

7.2 KVM 虚拟化应用案例

7.2.1 安装 KVM 组件

安装KVM组件需要先确保系统已经满足了最低安装要求，部署KVM虚拟化需要通过BIOS开启 CPU 的虚拟化功能，确保至少 6GB 的磁盘剩余空间以及 2GB 内存空间。KVM 虚拟机支持的存储方式有：本地磁盘文件、物理磁盘分区、LVM 分区、iSCSI 磁盘、GFS2 文件系统、光纤 LUNs 设备等。

1. 在部署操作系统时安装 KVM 组件

KVM 虚拟化组件可以在安装操作系统的过程中选择 Virtualization、Virtualiztion Client、Virtualization Platform、Virtualization Tools 组件包。

通过 Kickstart 方式安装系统时需要在 Kickstart 配置文件中加入如下内容：

```
@virtualization
@virtualization-client
@virtualization-platform
@virtualization-tools
```

2. 在现有的操作系统平台上安装虚拟化组件

配置 YUM 源后，可以通过 CentOS 官方的源中找到虚拟化组件，可以直接通过组包来安装虚拟化组件，表 7-1 详细介绍了组包及软件包描述说明。

表 7-1

组包名称	描述	所包含软件包
Virtualization	提供主机虚拟化环境	qemu-kvm,qemu-img,qemu-kvm-tools
Virtualization Client	安装与管理虚拟机实例的客户端软件	python-virtinst,virt-manager,virt-viewer, virt-top
Virtualization Platform	提供访问与控制虚拟机和容器的接口	libvirt, libvirt-client, fence-virtd-libvirt, fence-virtd-multicast, fence-virtd-serial, libvirt-cim, libvirt-java, libvirt-qpid, perl-sys-virt
Virtualization Tools	脱机虚拟机镜像管理工具	Libguestfs, libguestfs-java, libguestfs-mount, libguestfs-tools, virt-v2v

使用命令“yum groupinstall"组包名称"”安装这些组包即可，例如，需要安装 Virtualization Tools 组包，运行 yum groupinstall "Virtualization Tools"即可。

7.2.2　创建虚拟机操作系统

系统中的虚拟化组件安装完成后，就可以创建虚拟主机了，如果安装了图形环境，可以使用 virt-manager 命令开启 KVM 管理器界面，单击“New”按钮创建虚拟主机，也可以通过命令行工具 virt-install 命令直接安装部署虚拟主机。

1. 使用 virt-install 命令创建虚拟机

virt-install 命令既可以交互式运行，也可以以自动方式创建与部署虚拟机系统，配合 Kickstart

技术可以实现无人值守安装虚拟机操作系统。该命令提供了很多选项，使用 virt-install --help 可以查看选项帮助。

下面是 virt-install 命令的描述和用法。

描述：安装部署虚拟主机。

格式：virt-install [选项]…

选项：

-h	#查看帮助。
--connect=CONNECT	#连接非默认 hypervisor，默认为 qemu:///system。
-n name	#新的虚拟主机实例名称。
-r MEMORY	#虚拟主机内存设置。
--arch=ARCH	#CPU 架构。
--machine=MACHINE	#虚拟机模拟器类型，Xen 或 KVM 等。
--vcpus=VCPUS	#虚拟主机 CPU 个数。
-c CDROM	#设置光盘镜像或光盘设备路径。
-l LOCATION	#指定安装源路径。
--pxe	#使用 PXE 协议启动。
--import	#导入现有的虚拟机。
-x EXTRA	#附加内核参数，如-x "ks=http://server/server.ks"。
--os-type=Type	#操作系统类型，如 linux、Windows 等。
--disk=DISKOPTS	#设置虚拟机磁盘，如--disk /dev/storage/path。
-w NETWORK	#设置虚拟机网络，如-w bridge=br0。
--nonetworks	#虚拟机不设置任何网络接口。
-m MAC	#设置虚拟机 MAC 地址。
--vnc	#设置通过 VNC 查看虚拟机。
--hvm	#使用全虚拟化技术。
--paravirt	#使用半虚拟化技术。
--soundhw MODEL	#设置声卡设备，MODEL 可以为 ich6、ac97 等。
--autostart	#设置虚拟机为开机启动。

官方帮助文档案例如下。

安装 Fedora 13，使用 virtio 驱动的磁盘与网卡设备，创建 8GB 的存储文件，使用 CDROM 安装操作系统，使用 VNC 查看虚拟机界面：

```
#virt-install \
--connect qemu:///system \
```

```
--virt-type kvm \
--name demo \
--ram 500 \
--disk path=/var/lib/libvirt/images/demo.img,size=8 \
--graphics vnc \
--cdrom /dev/cdrom \
--os-variant fedora13
```

安装 Fedora9，使用 LVM 分区，虚拟网络设置为从 PXE 启动，使用 VNC 连接虚拟机界面：

```
# virt-install \
--connect qemu:///system \
--name demo \
--ram 500 \
--disk path=/dev/HostVG/DemoVM \
--network network=default \
--virt-type qemu
--graphics vnc \
--os-variant fedora9
```

在真实的磁盘中安装虚拟机，使用默认的 QEMU hypervisor，使用 SDL 连接虚拟机界面，通过远程初始化及安装：

```
# virt-install \
--connect qemu:///system \
--name demo \
--ram 500 \
--disk path=/dev/hdc \
--network bridge=eth1 \
--arch ppc64 \
--graphics sdl \
--location
http://download.fedora.redhat.com/pub/fedora/linux/core/6/x86_64/os/
```

2. 使用图形管理工具创建虚拟机

virt-manager 是 CentOS 6.3 系统中所提供的虚拟机管理程序，用来通过图形创建以及管理虚拟机。下面介绍 virt-manager 创建虚拟机的具体方法。

第一步，在命令终端输入 virt-manager 开启图形管理程序，效果如图 7-2 所示。

第二步，选择 hypervisor 并单击“Connect”按钮，连接 hypervisor，默认为 qemu，如图 7-3 与图 7-4 所示，默认连接的是本机 hypervisor，选择 Connect to remote host，在 host 框中输入计算机主机名或 IP 地址后，也可以连接其他主机的 hypervisor。

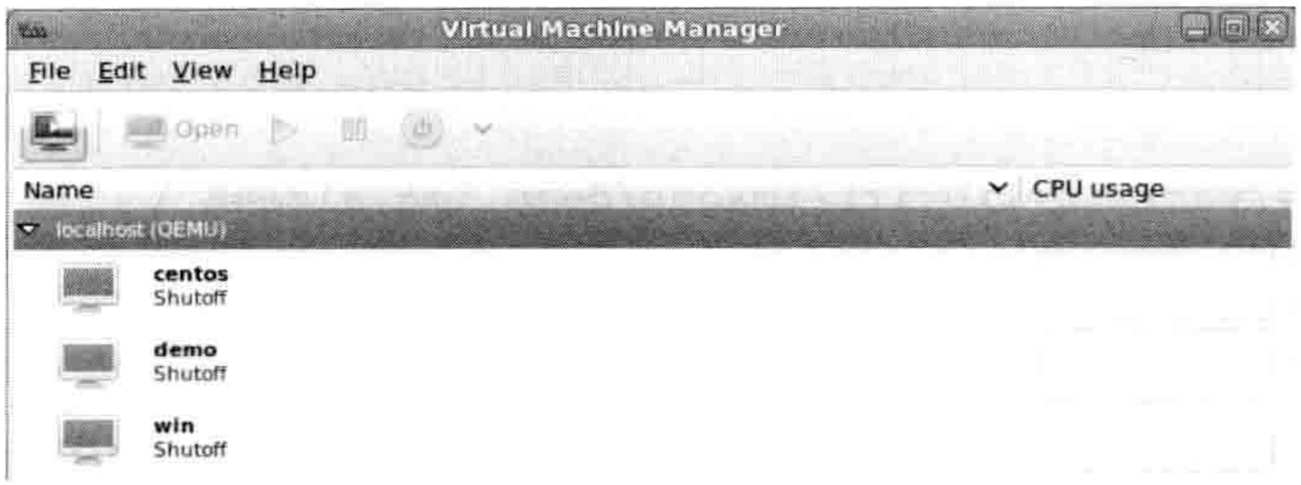


图 7-2

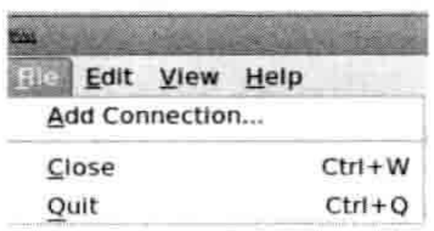


图 7-3

图 7-4

第三步，单击“Create a new virtual machine”按钮，如图 7-5 所示，开启新建虚拟机向导，新建虚拟机向导中将提示进行以下内容的设置。

- 虚拟机名称以及安装方式，安装方式可以选择 ISO、光盘、PXE 等方式，如图 7-6 所示。
- 安装介质路径，选择具体的光驱或 ISO 文件路径，以及操作系统类型，如图 7-7 所示。

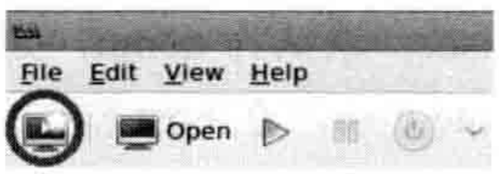


图 7-5

图 7-6

- 配置内存与 CPU 参数，如图 7-8 所示。
- 配置存储参数，可以选择创建新的虚拟存储文件，也可以选择使用已存在的存储设备，默认虚拟机镜像文件路径为/var/lib/libvirt/images，如图 7-9 所示。
- 配置网络、架构以及其他硬件参数，网络选择 Bridge 桥接可以实现虚拟机直接访问外部网络，如图 7-10 所示。

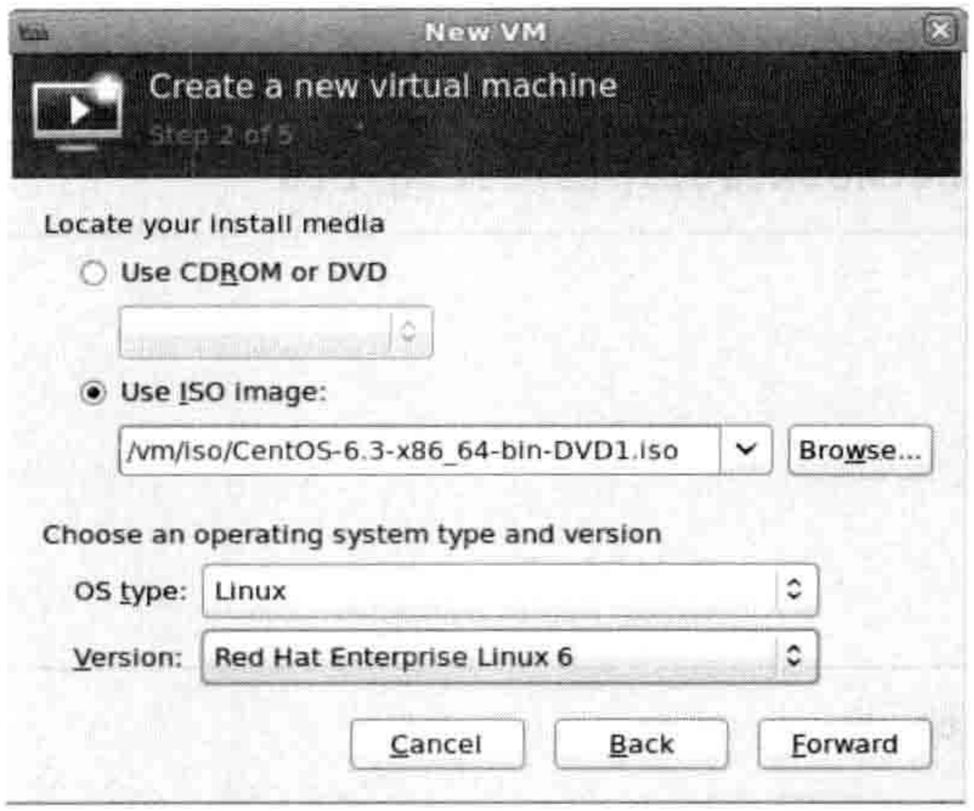


图 7-7

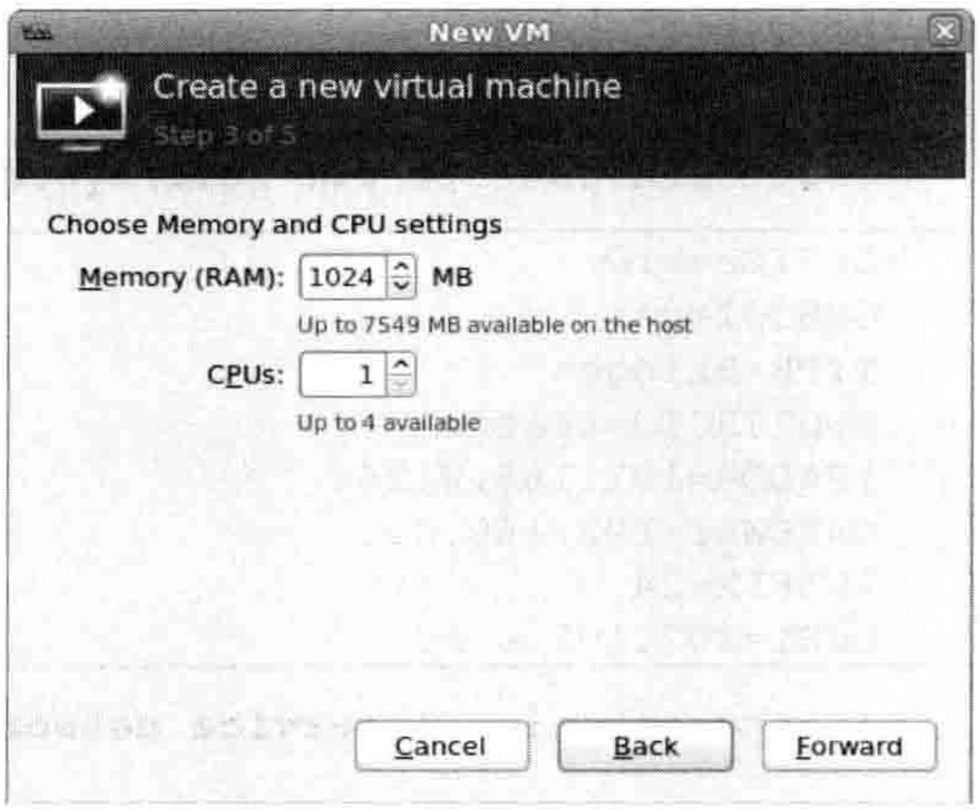


图 7-8

图 7-9

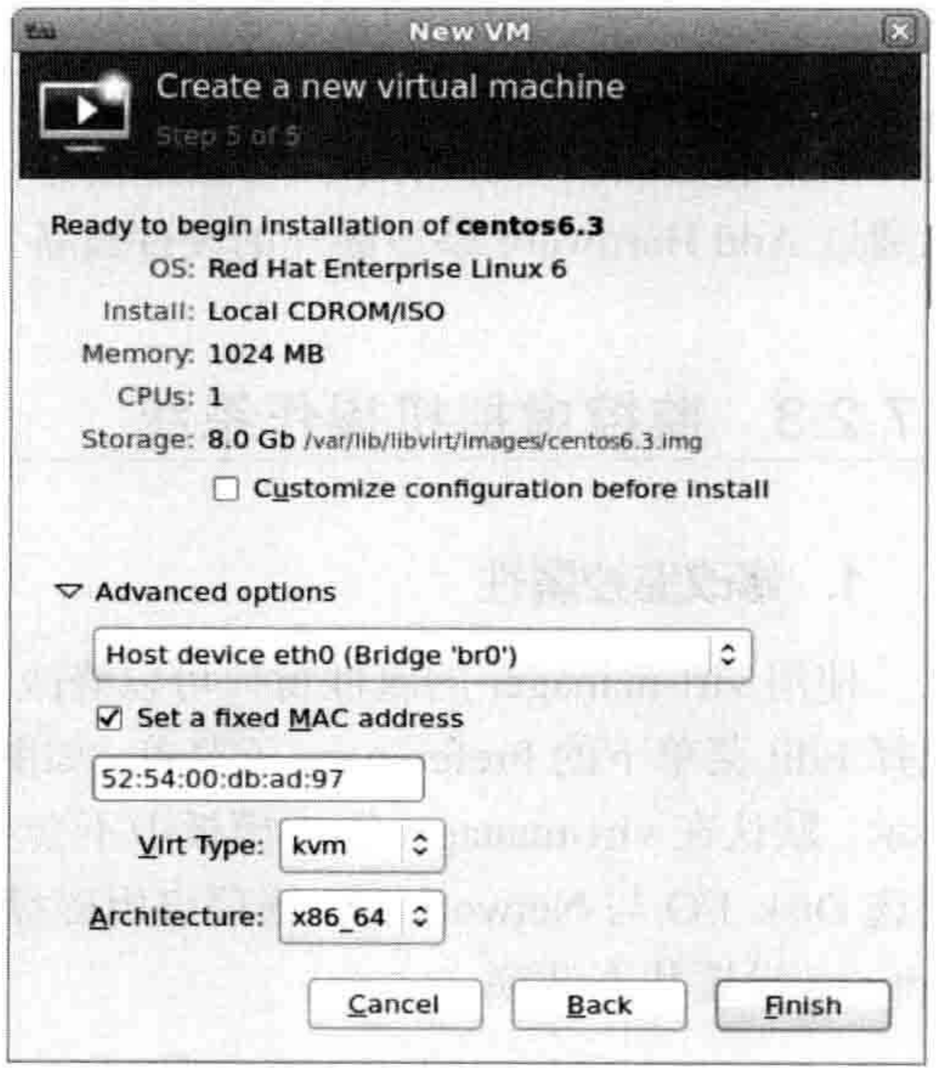


图 7-10

虚拟机选择桥接网络时，需要宿主主机（真实机）创建一个共享的桥接网络设备，也就是所谓的网桥。如果宿主主机的物理网卡名称为 eth0，则创建虚拟桥接设备（ifcfg-br0 文件需要手动创建）的具体操作步骤如下。

```
[root@centos6 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE=eth0
ONBOOT=yes
BOOTPROTO=none
BRIDGE=br0
[root@centos6 ~]# vim /etc/sysconfig/network-scripts/ifcfg-br0
DEVICE=br0
ONBOOT=yes
TYPE=Bridge
BOOTPROTO=static
IPADDR=192.168.0.254
GATEWAY=192.168.0.1
PREFIX=24
DNS1=202.106.0.20
[root@centos6 ~]# service network restart
```

第四步，安装部署虚拟机操作系统，具体操作步骤可以参考本书第 1 章系统安装部分的内容。

虚拟主机部署完成后，可以通过硬件信息按钮查看该虚拟机硬件的信息，如图 7-11 所示。在硬件信息列表中，我们随时都可以通过 Add Hardware 添加额外的硬件设备。

图 7-11

7.2.3 监控虚拟机操作系统

1. 修改监控属性

使用 virt-manager 的属性窗口可以修改虚拟主机的性能监控属性，开启 virt-manager 程序后选择 Edit 菜单下的 Preferences 子菜单，如图 7-12 所示，即可开启虚拟机选项对话框，如图 7-13 所示。默认在 virt-manager 监控模板中不会显示磁盘以及网络接口 I/O 性能图表，我们可以通过勾选 Disk I/O 与 Network I/O 选项启用磁盘与网络接口性能查询功能。另外，可通过修改 Stats Options 设置状态选项。

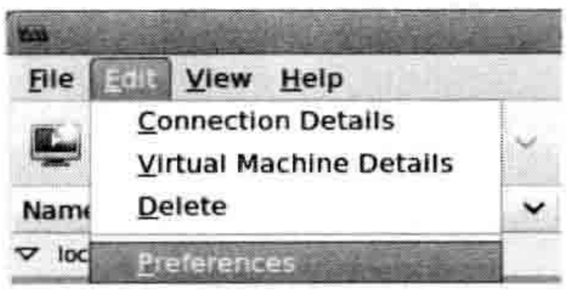


图 7-12

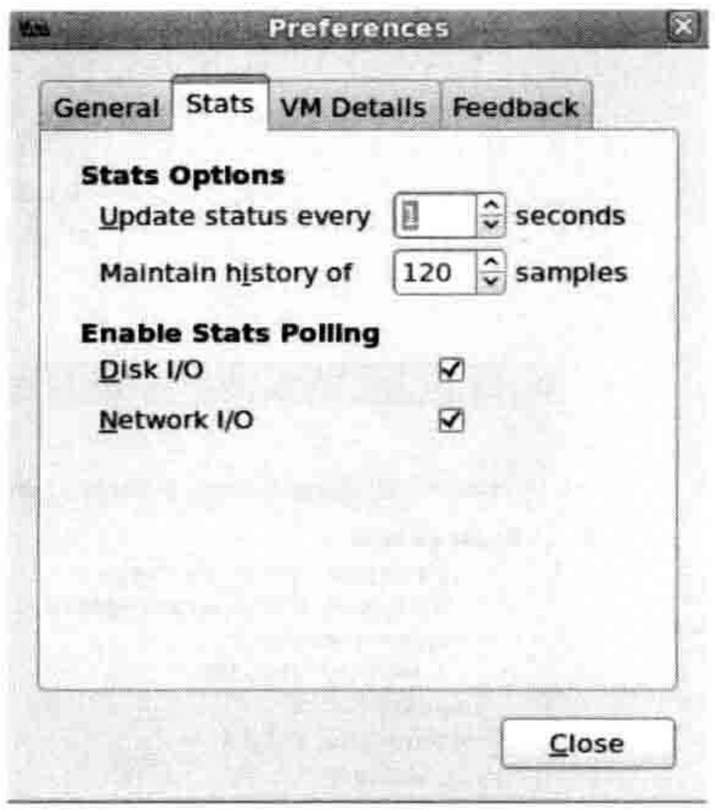


图 7-13

2. 监控图形

打开 virt-manager 程序，单击 View 菜单下的 Graph 子菜单，根据需要决定是否点选 CPU Usage、Disk I/O、Network I/O，分别可以显示 CPU 使用率、磁盘读写以及网络读写性能图表，全部选择后的效果如图 7-14 所示。

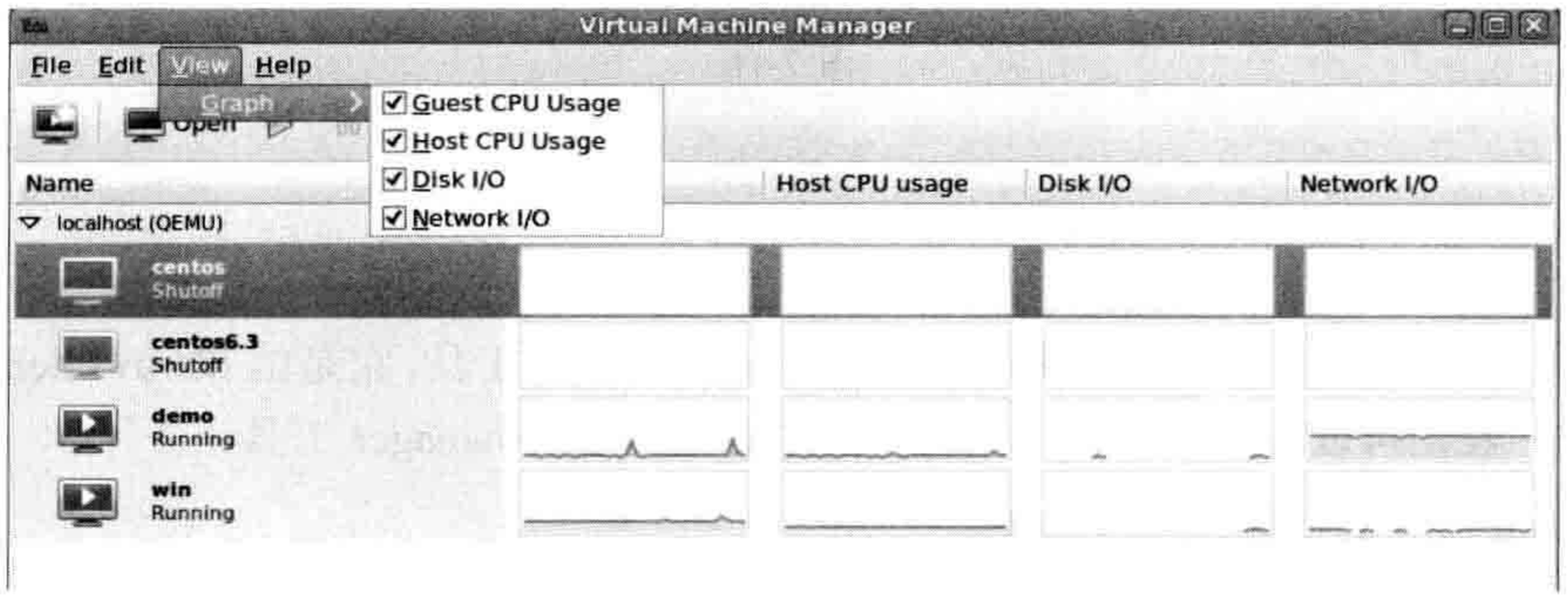


图 7-14

3. 查看虚拟机详细信息

virt-manager 还可以查看虚拟机连接的详细信息，通过 Edit 菜单下的 Condition Details，我们可以进一步查看 hypervisor、真实主机 CPU 以及内存使用情况、虚拟网络设置、存储池设置以及网络接口信息，如图 7-15 和图 7-16 所示。

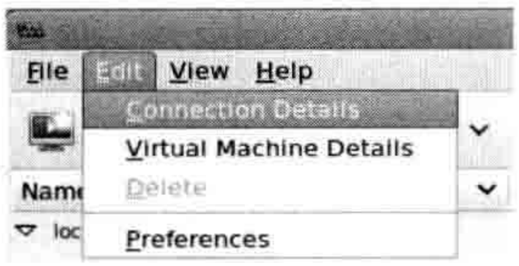


图 7-15

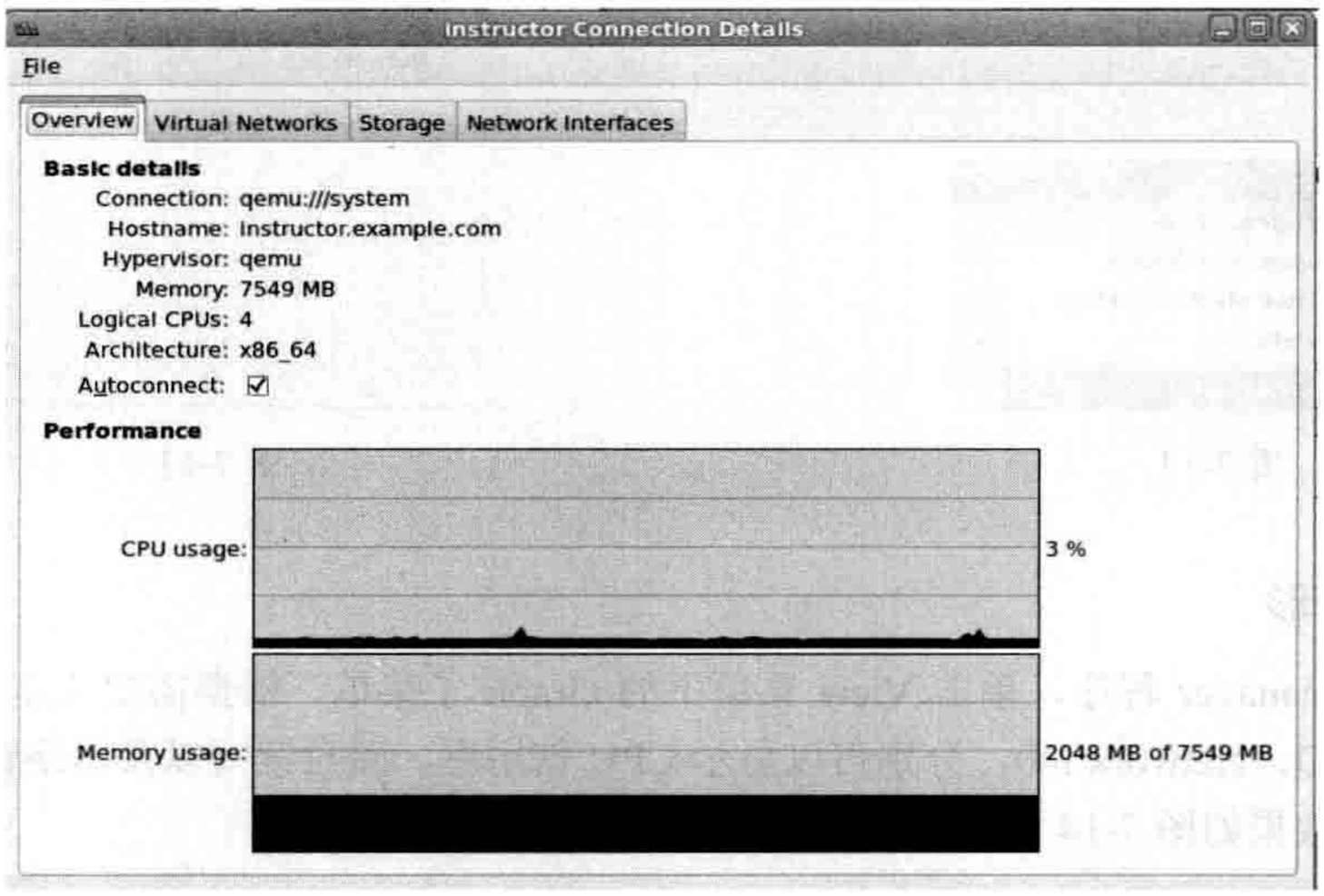


图 7-16

7.2.4 命令工具使用技巧

virsh 是非常优秀的 hypervisor 和虚拟主机的命令行管理工具，它是由 libvirt-client 软件包所提供的命令，我们可以使用该命令工具替代图形界面的 virt-manager 工具。

有关 virsh 命令的描述和用法如下。

描述：虚拟机管理工具。

用法：virsh [选项] 命令 [参数]

选项：-h　　显示帮助信息。

　　-c,--connect URL　　指定连接 URL。

　　-l,--log 文件名　　输入日志至指定文件。

命令：help　　#显示 virsh 支持的所有命令

　　list　　#查看正在运行的虚拟主机列表，--all 显示所有的列表

```
autostart DOMAIN_ID          #配置虚拟主机为开机自启动，--disable 表示禁用自启动
connect URI                  #连接指定 hypervisor
dumpxml    DOMIAN_ID         #输出特定虚拟主机的 XML 格式配置文件
create FILE                  #通过 XML 文件创建一个新的虚拟主机
define FILE                  #使用 XML 文件定义一个新的虚拟主机
destroy DOMAIN_ID            #通过 id 中止特定的虚拟主机
dominfo    DOMAIN_ID         #返回特定虚拟主机的基本信息
domid DOMAIN_NAME            #返回特定虚拟主机 id
domstate DOMAIN_ID           #返回特定虚拟主机的状态信息
edit DOMAIN_ID               #编辑虚拟机的 XML 配置文件
reboot DOMAIN_ID             #重启特定的虚拟主机
shutdown DOMAIN_ID           #关闭特定的虚拟主机
destroy DOMAIN_ID            #强制关闭虚拟主机
start DOMAIN_ID              #通过 id 强制中止特定的虚拟主机
save DOMAIN_ID FILE          #保存运行中的虚拟主机内存信息到文件
restore FILE                 #使用 save 命令保存的文件还原虚拟主机状态
screenshot DOMAIN_ID         #对特定的虚拟主机抓屏，并保存图片
suspend DOMAIN_ID            #暂停特定的虚拟主机
resume DOMAIN_ID             #恢复特定的虚拟主机
setmem DOMAIN_ID             #设置特定的虚拟主机内存大小，默认单位为 Kbyte
setmaxmem DOMAIN_ID          #设置特定的虚拟主机最大内存大小
setvcpus DOMAIN_ID           #设置特定的虚拟主机虚拟 CPU 的个数
vcpuinfo DOMAIN_ID           #显示基本的 CPU 信息
attach-device DOMAIN_ID FILE      #根据 XML 配置文件为虚拟主机添加设备
net-create FILE              #根据 XML 文件创建虚拟网络
net-dumpxml network          #输出 XML 格式的虚拟网络信息
net-list                     #返回激活的网络接口信息
net-start network            #激活特定的网络
iface-define FILE            #使用 XML 文件定义主机网络接口
iface-list                   #显示激活的主机网络接口信息
snapshot-create              #创建快照
snapshot-delete              #删除快照
snapshot-dumxml              #备份快照
```

snapshot-list　　#显示快照列表

案例如下。

案例 1：查看当前正在运行的虚拟主机列表，查看所有的虚拟主机列表。

```
[root@centos6 ~]# virsh list
 Id Name         State
----------------------------------
  1 demo         running
  2 win          running
[root@centos6 ~]# virsh list --all
Id Name         State
----------------------------------
  1 demo         running
  2 win          running
- centos         shut off
  - centos6.3    shut off
```

案例 2：设置 demo 虚拟主机为开机自动启动。

```
[root@centos6 ~]# virsh autostart demo
Domain demo marked as autostarted
```

案例 3：备份 demo 虚拟主机的配置文件，将配置文件保存为 demo.xml。

```
[root@centos6 ~]# virsh dumpxml demo > demo.xml
```

案例 4：使用 demo.xml 配置文件生成新的虚机主机，由于 demo.xml 文件中有一些 demo 主机的特定配置信息，所以在使用 demo.xml 创建新的虚拟主机之前，需要将其中的信息稍加修改，<name></name>定义的是虚拟主机名称，<uuid></uuid>定义的是虚拟主机唯一的序列编号，<mac address/>定义的是虚拟主机的 MAC 地址。以上这些信息都是必须要修改的内容，其中可以使用 uuidgen 随机生成 UUID。

```
[root@centos6 ~]# uuidgen
b2a187e3-e17d-4551-9c19-bc7c38a8926d
[root@centos6 ~]# vim demo.xml
```

```
<domain type='kvm' id='1'>
  <name>demo-dump</name>
  <uuid>b2a187e3-e17d-4551-9c19-bc7c38a8926d</uuid>
  <memory>2000049</memory>
  <currentMemory>2000896</currentMemory>
  <vcpu>1</vcpu>
  <os>
    <type arch='x86_64' machine='rhel6.2.0'>hvm</type>
    <boot dev='hd'/>
```

```
    </os>
    <features>
      <acpi/>
      <apic/>
      <pae/>
    </features>
    <clock offset='utc'/>
    <on_poweroff>destroy</on_poweroff>
    <on_reboot>restart</on_reboot>
    <on_crash>restart</on_crash>
    <devices>
      <emulator>/usr/libexec/qemu-kvm</emulator>
      <disk type='block' device='disk'>
        <driver name='qemu' type='raw' cache='none' io='native'/>
        <source dev='/dev/GLSguest/demo'/>
        <target dev='vda' bus='virtio'/>
        <alias name='virtio-disk0'/>
        <address type='pci' domain='0x0000' bus='0x00' slot='0x04' function=
'0x0'/>
      </disk>
      <interface type='bridge'>
        <mac address='52:54:00:00:00:fb'/>
        <source bridge='br0'/>
        <target dev='vnet0'/>
        <model type='virtio'/>
        <alias name='net0'/>
        <address type='pci' domain='0x0000' bus='0x00' slot='0x03' function=
'0x0'/>
      </interface>
      <serial type='pty'>
        <source path='/dev/pts/1'/>
        <target port='0'/>
        <alias name='serial0'/>
      </serial>
      <console type='pty' tty='/dev/pts/1'>
        <source path='/dev/pts/1'/>
        <target type='serial' port='0'/>
        <alias name='serial0'/>
      </console>
      <input type='tablet' bus='usb'>
        <alias name='input0'/>
      </input>
      <input type='mouse' bus='ps2'/>
      <graphics type='vnc' port='5900' autoport='yes'/>
      <video>
        <model type='cirrus' vram='9216' heads='1'/>
```

```
        <alias name='video0'/>
        <address type='pci' domain='0x0000' bus='0x00' slot='0x02' function=
'0x0'/>
      </video>
      <memballoon model='virtio'>
        <alias name='balloon0'/>
        <address type='pci' domain='0x0000' bus='0x00' slot='0x05' function=
'0x0'/>
      </memballoon>
    </devices>
    <seclabel type='dynamic' model='selinux' relabel='yes'>
      <label>system_u:system_r:svirt_t:s0:c368,c589</label>
      <imagelabel>system_u:object_r:svirt_image_t:s0:c368,c589</imagelabel>
    </seclabel>
  </domain>
```

```
[root@centos6 ~]# virsh create demo.xml
Domain demo-dump created from demo.xml
```

案例 5：查看虚拟主机 demo 的基本信息。

```
[root@centos6 ~]# virsh dominfo 1
Id:             1
Name:           demo
UUID:           135fa6ec-8f6e-0b6e-7cd1-e26b7ba29ab4
OS Type:        hvm
State:          running
CPU(s):         1
CPU time:       122.8s
Max memory:     2000049 kB
Used memory:    2000896 kB
Persistent:     yes
Autostart:      enable
Managed save:   no
Security model: selinux
Security DOI:   0
Security label: system_u:system_r:svirt_t:s0:c368,c589 (permissive)
```

案例 6：开启 centos 这台虚机主机。

```
[root@centos6 ~]# virsh start centos
Domain centos started
```

案例 7：设置配置指定（win）虚拟主机的内存大小值、最大内存值以及 CPU 个数，使虚拟机下次重启后生效。

```
[root@centos6 ~]# virsh setmaxmem win 2097152 --config
[root@centos6 ~]# virsh setmem win 2097152 --config
```

```
[root@centos6 ~]# virsh setvcpus win 2 --config
```

案例 8：查看并备份 default 网络配置文件，将配置文件保存为 default_net.xml。

```
[root@centos6 ~]# virsh net-list
[root@centos6 ~]# virsh net-dumpxml default > default_net.xml
```

7.2.5　虚拟存储与虚拟网络

1. 存储池

KVM 存储池是被 libvirt 所管理的文件、目录或存储设备，存储池可以位于本地，也可以通过网络共享，存储池最终可以被虚拟主机所使用。默认 libvirt 使用基于目录的存储池设计，/var/lib/libvirt/images 目录就是默认的存储池。本地存储池可以是本地的一个目录、磁盘设备、物理分区或 LVM 卷，但本地存储池不适合于大规模产品部署，也不支持虚拟机迁移功能。网络共享存储池使用标准的网络协议进行存储设备的共享，它支持 SAN、IP-SAN、NFS、GFS2 等协议。在 KVM 虚拟化技术中，存储池可以包含多个存储卷，对虚拟主机而言，这些存储卷将被识别为物理硬件存储设备。

下面将以目录存储以及共享 NFS 存储为例，演示 KVM 存储池及存储卷的创建流程。

（1）创建基于目录的虚拟存储池与存储卷。

1）创建目录：

```
[root@centos6 ~]# mkdir /var/lib/libvirt/test
```

2）修改文件安全相关属性：

```
[root@centos6 ~]# chown root.root /var/lib/libvirt/test
[root@centos6 ~]# chmod 700 /var/lib/libvirt/test
[root@centos6 ~]# semanage fcontext -a -t virt_image_t /var/lib/libvirt/test
[root@centos6 ~]# restorecon -v /var/lib/libvirt/test
```

3）创建存储池：

```
[root@centos6 ~]# virt-manager                    #开启虚拟机管理器
```

依次选择“Edit”→“Connection Details”菜单，如图 7-17 所示，开启虚拟机连接具体设置窗口，在 Storage 选项卡中，选择左下角位置的添加图标，创建存储池，如图 7-18 所示。在创建存储池的第一步中输入存储池名称与类型。本例中存储池名称为 test，类型为基于目录的存储池，如图 7-19 所示；第二步需要输入存储池目录的具体路径，如图 7-20 所示。至此，基于目录的存储池已创建完成，如图 7-21 所示。下面需要在池中创建存储卷。

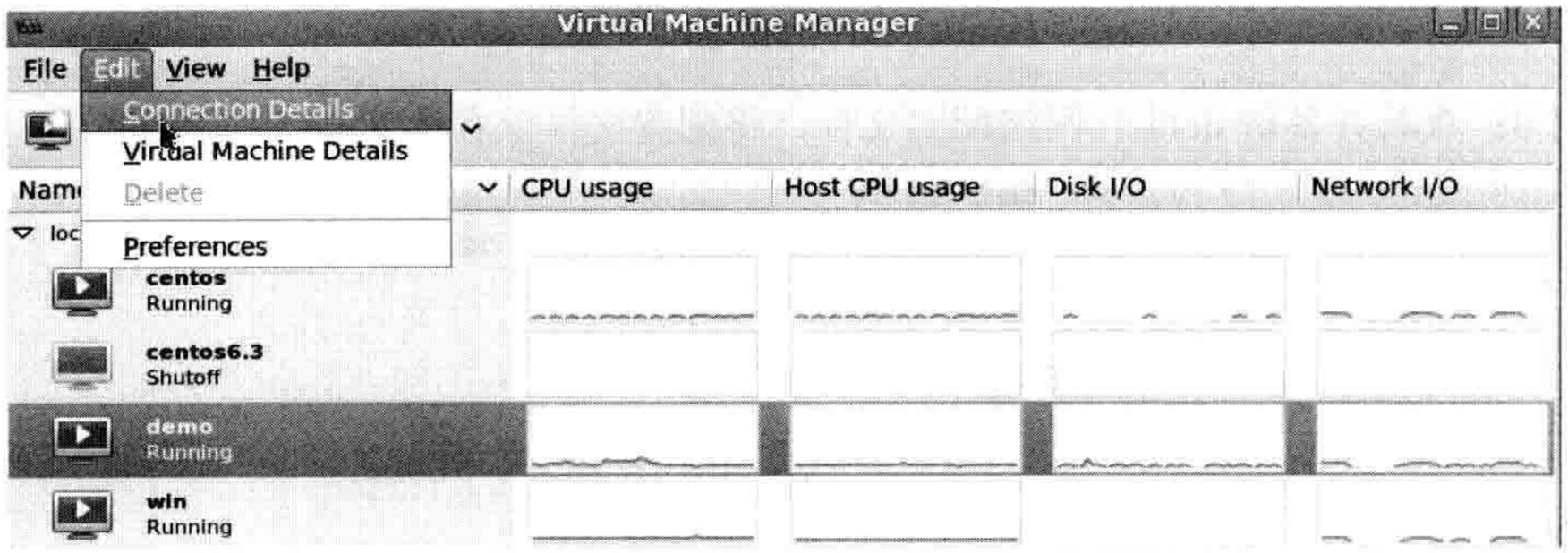


图 7-17

图 7-18

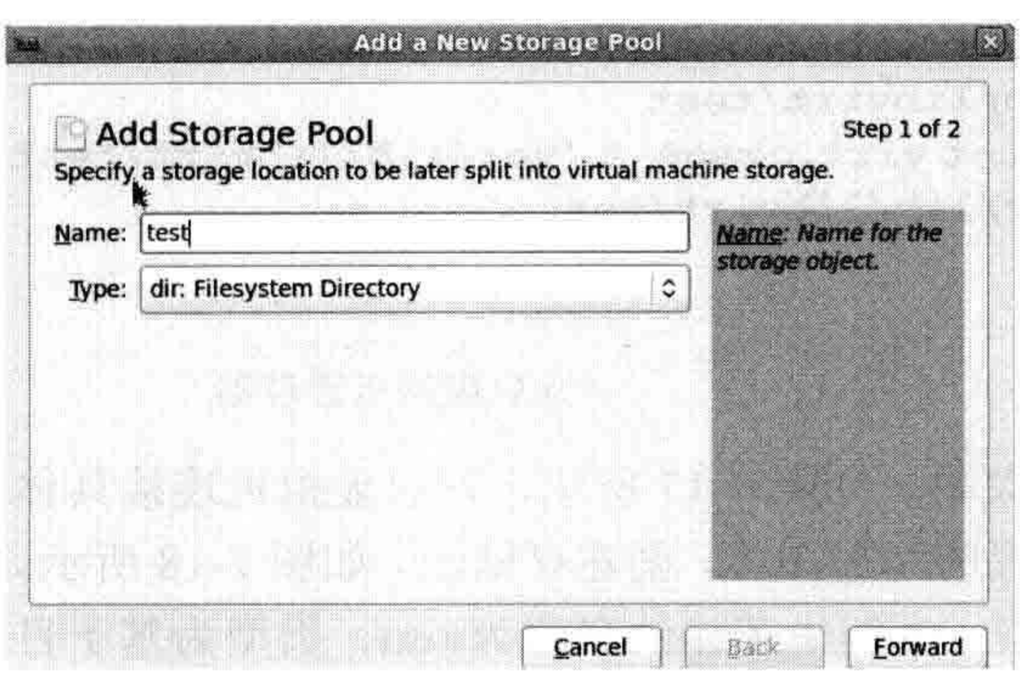


图 7-19

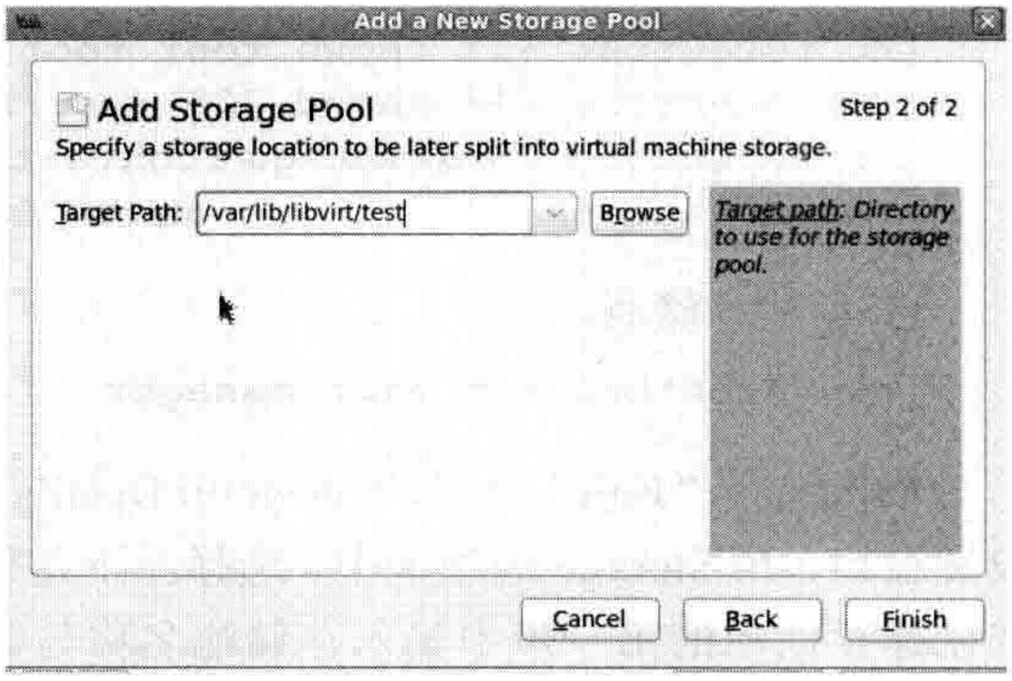


图 7-20

图 7-21

4）创建存储卷。

在完成存储池的创建后，我们就可以在池中创建存储卷了，如图 7-21 所示，单击选择需要创建卷的存储池，本例为 test 存储池，然后选择“New Volume”按钮，根据提示输入卷名称、格式类型、最大存储空间、实际分配空间，如图 7-22 所示。单击“Finish”按钮完成存储卷的创建，如图 7-23 所示。

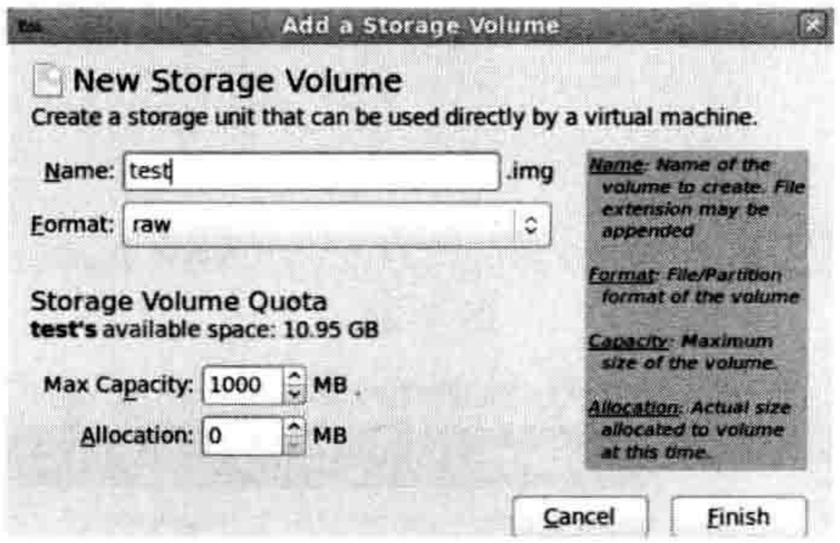


图 7-22

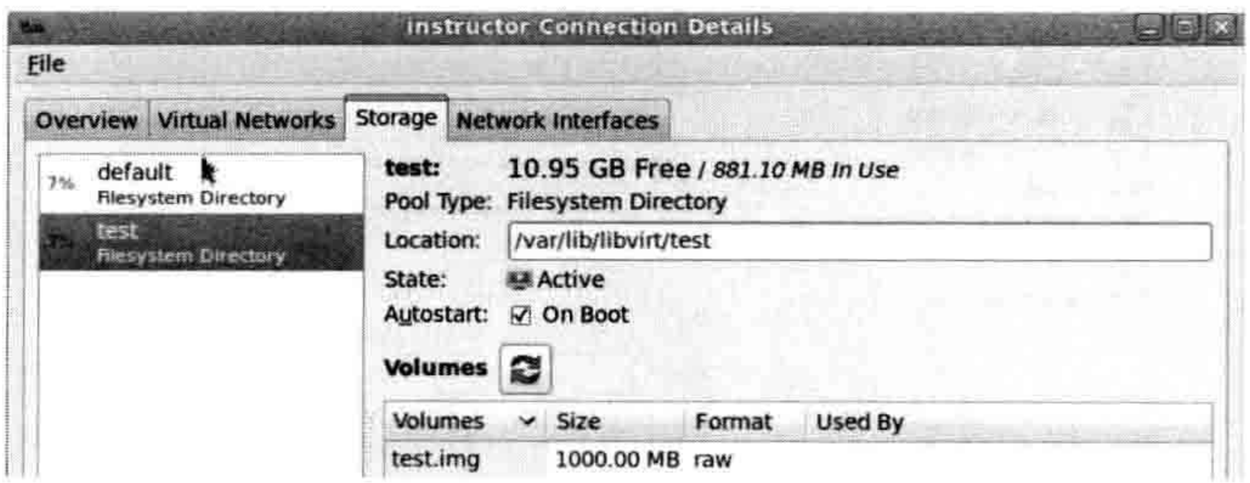


图 7-23

（2）为虚拟机添加使用存储设备。

存储卷创建完成后，我们的虚拟主机就可以使用这些卷了，通过 virt-manager 选择一个虚拟机双击打开控制窗口。点选设备按钮，显示设备列表，如图 7-24 所示，选择“Add Hardware”按钮添加新的设备，在添加虚拟硬件窗口中选择 Storage 存储设备，因为我们已经为虚拟主机创建好了存储卷设备。因此，我们选择已经存在的存储空间，并输入存储卷的绝对路径，如图 7-25 所示。进入虚拟主机操作系统，通过 fdisk 查看磁盘信息将包含刚刚添加的存储卷设备，标记为 /dev/vdb，如图 7-26 所示。

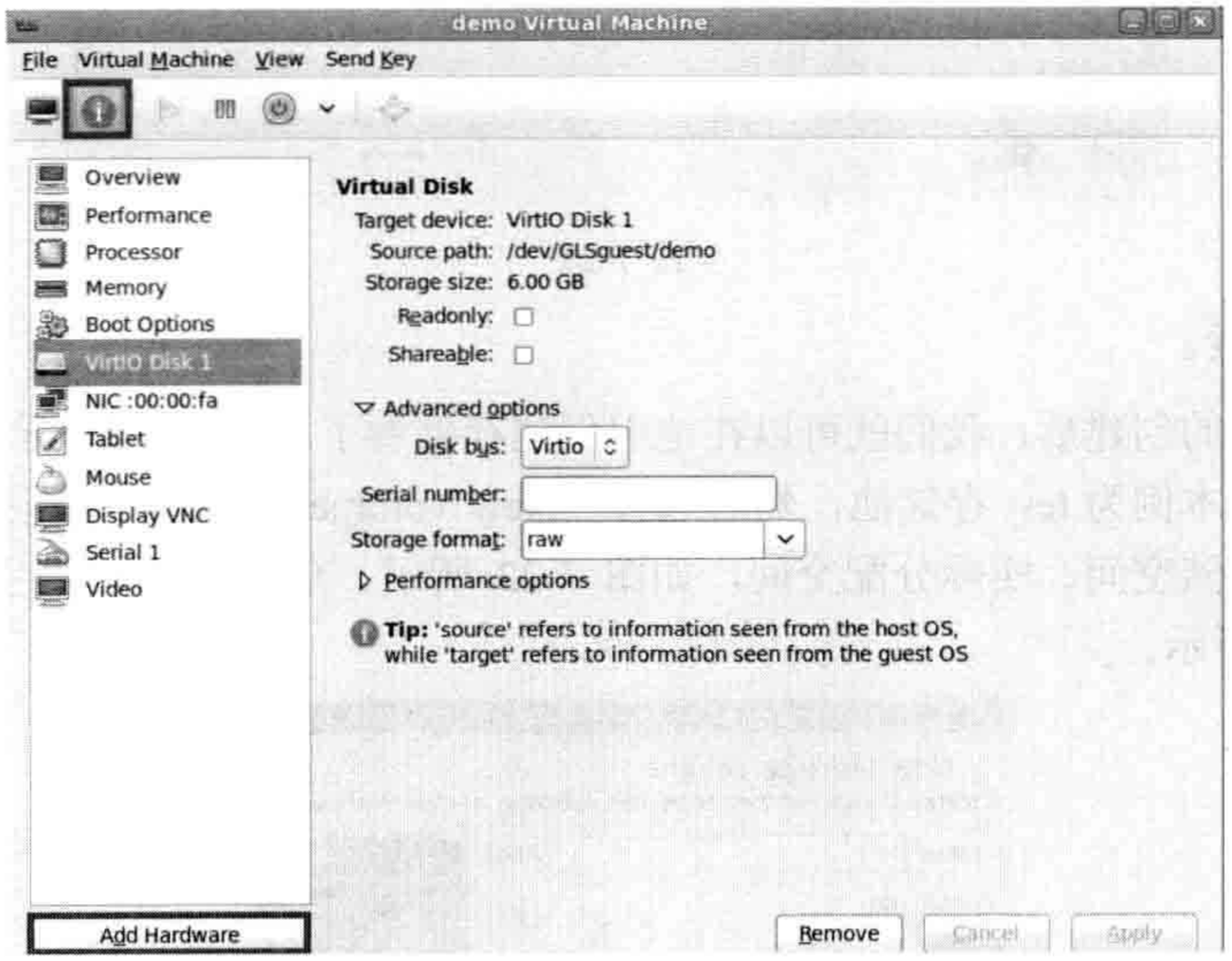


图 7-24

图 7-25

```
Disk /dev/vdb: 1048 MB, 1048576000 bytes
16 heads, 63 sectors/track, 2031 cylinders, total 2048000 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000

Disk /dev/vdb doesn't contain a valid partition table
```

图 7-26

下面将演示如何在 KVM 虚拟化环境中创建基于 NFS 的存储池以及存储卷设备，这里假设已经准备好了一台 NFS 共享服务器（关于如何部署 NFS 服务器，可以参考 4.1 节的相关内容）。

1）创建存储池。

与基于目录的存储池类似，我们通过如图 7-18 所示的添加按钮，创建一个新的存储池。第一步依然是填写存储池名称以及类型，如图 7-27 所示；第二步根据实际情况填写 NFS 服务器主机名或 IP 地址（192.168.0.254）、服务器共享目录（/nfs）、本地挂载点（/var/lib/libvirt/nfs），如图 7-28 所示，单击“Finish”按钮完成。

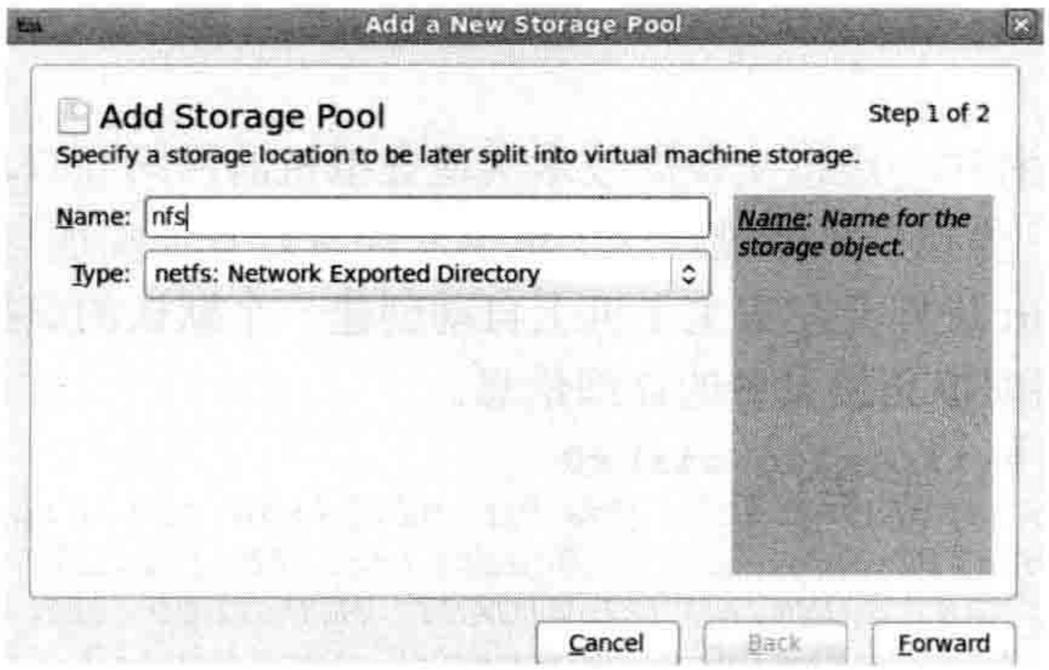


图 7-27

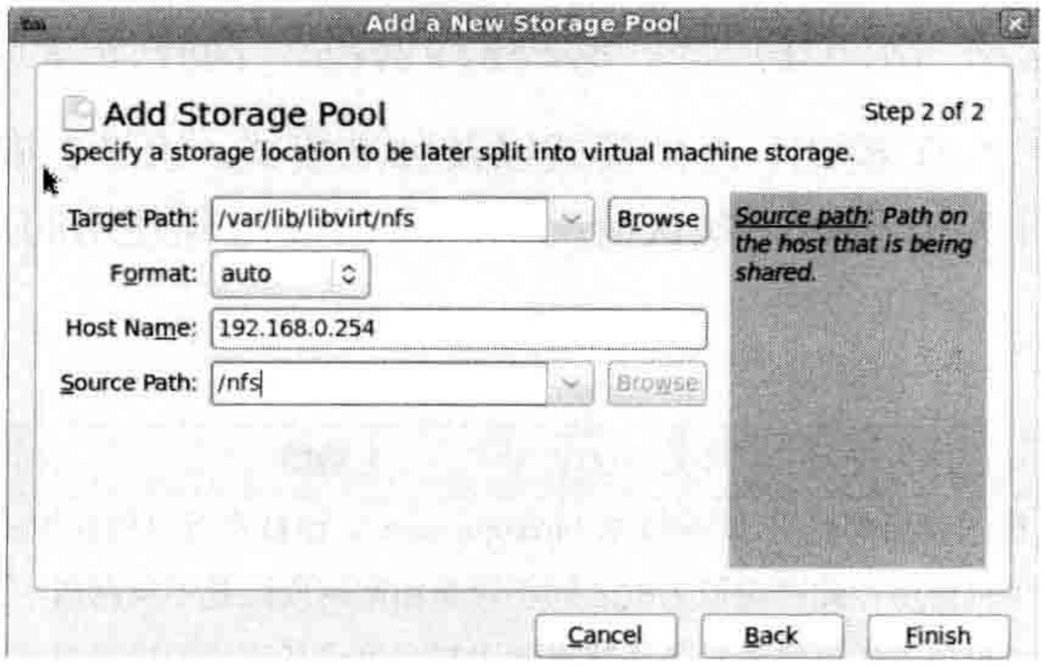


图 7-28

2）创建存储卷。

与基于目录的存储卷类似，我们通过“New Volume”按钮创建存储卷，根据提示输入卷名称、格式类型、最大存储容量、分配存储容量，如图 7-29 所示。

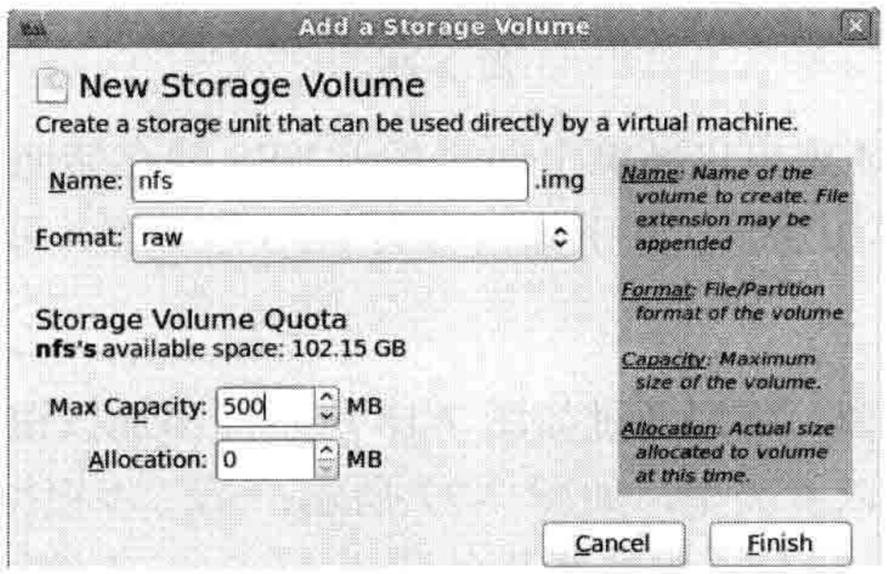


图 7-29

2. 虚拟网络设置

KVM 虚拟化使用一种称为虚拟交换的技术实现虚拟机的网络互联，虚拟交换是运行在宿主主机上的一个软件结构，虚拟主机通过与这个虚拟交换进行直连实现与外界通信。当我们部署完成虚拟化环境后，libvirt 进程会在宿主主机上自动创建一个默认的虚拟交换 virbr0，我们可以通过 ifconfig 命令查看该虚拟交换设备的详细信息。

```
[root@centos6 ~]# ifconfig virbr0
virbr0    Link encap:Ethernet  HWaddr 52:54:00:55:7F:D9
          inet addr:192.168.122.1  Bcast:192.168.122.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:4926 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 b)  TX bytes:974652 (951.8 KiB)
```

虚拟交换可以运行在 NAT 模式、路由模式以及独立模式，表 7-2 描述了这些模式的特性与功能，虚拟交换结构如图 7-30 所示。默认情况下，libvirtd 进程使用的是 NAT 作为虚拟交换的工作模式。

表 7-2

模式	功能
NAT 模式	NAT 模式使用的是 IP 欺骗技术（masquerade），该技术可以使虚拟主机使用宿主主机的 IP 地址与外部主机通信，此时虚拟主机之间不管是相同网段还是不同网段，都可以相互通信。但默认位于宿主主机外部的计算机设备不可以同虚拟交换设备连接的虚拟机通信。也就是说，虚拟主机不可以连接真实机外面的其他主机

续表

模式	功能
路由模式	路由模式虚拟交换将与真实设备的物理网络相连接，实现数据包的出站与入站。宿主主机将承担路由的角色
独立模式	独立模式使每个虚拟交换设备都是一个独立的网络，并且与宿主主机也是相互独立的。连接在相同虚拟交换设备的虚拟主机是可以进行通信的

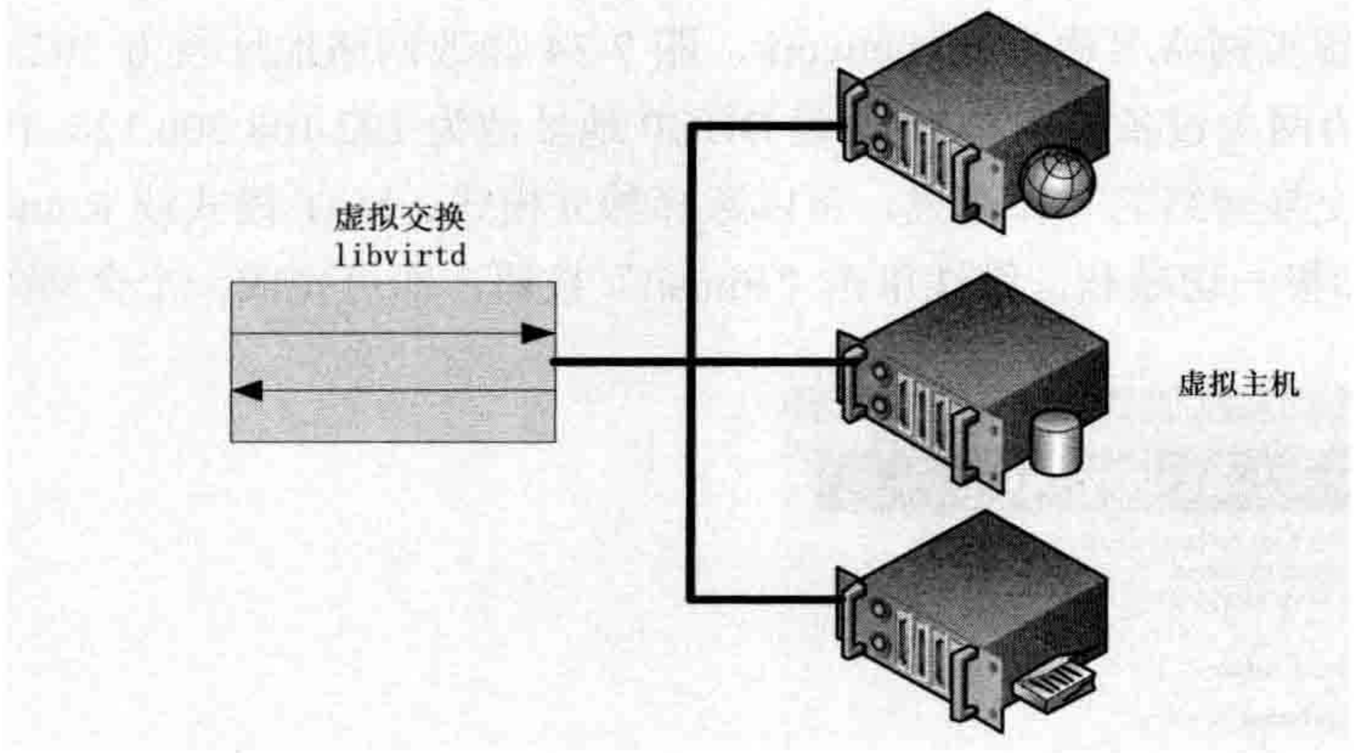


图 7-30

设置虚拟交换网络最简单的方式是通过 virt-manager 工具，通过 virt-manager 命令开启虚拟机管理器，通过 Edit 菜单下面的 Connection Details 子菜单开启 Host Details 窗口，选择 Virtual Networks 选项卡，如图 7-31 所示。

图 7-31

默认 libvirtd 已经在 KVM 虚拟化环境中创建了一个基于 NAT 模式的 default 虚机交换，设备名称为 virbr0，状态处于激活状态，虚拟交换网络为 192.168.0.122.0/24，DHCP 地址池为 192.168.122.2-192.168.122.254，virbr0 设备的 IP 地址默认为 192.168.122.1。我们可以使用图 7-31 中的添加按钮创建新的虚拟交换网络。

使用图 7-31 中展示的添加按钮（+），屏幕将出现如图 7-32 所示的创建虚拟网络对话框，提示创建一个虚拟交换网络需要填写名称、IP 地址与子网掩码、DHCP 地址池以及工作模式。图 7-33 提示输入虚拟网络名称为 mynetwork，图 7-34 修改网络地址段为 192.168.200.0/24，默认 192.168.200.1 为网关设备，图 7-35 设置 DHCP 地址池为 192.168.200.128~192.168.200.254，图 7-36 选择虚拟交换网络的工作模式，可以选择独立模式、NAT 模式或 Routed 模式，图 7-37 为确认对话框，如果一切顺利，继续单击“Finish”按钮，即可完成一个全新的虚拟网络设置。

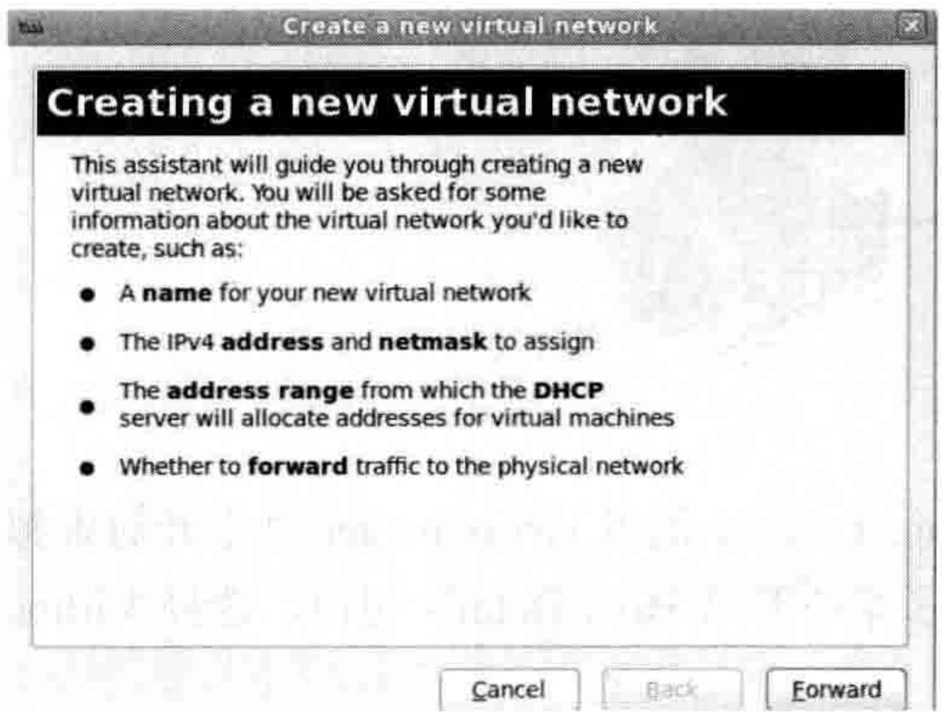


图 7-32

图 7-33

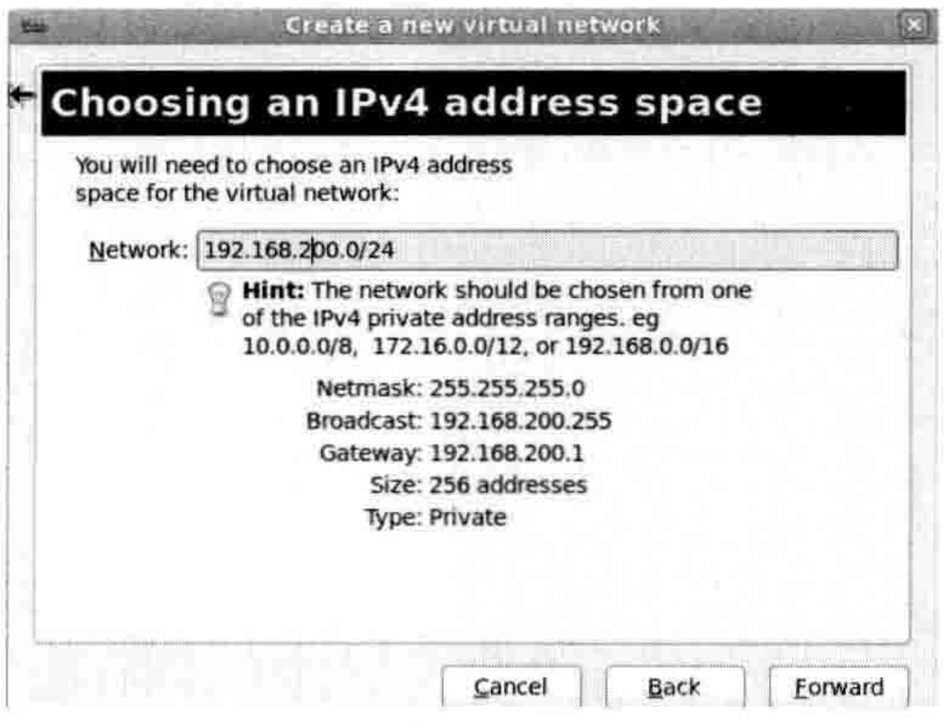


图 7-34

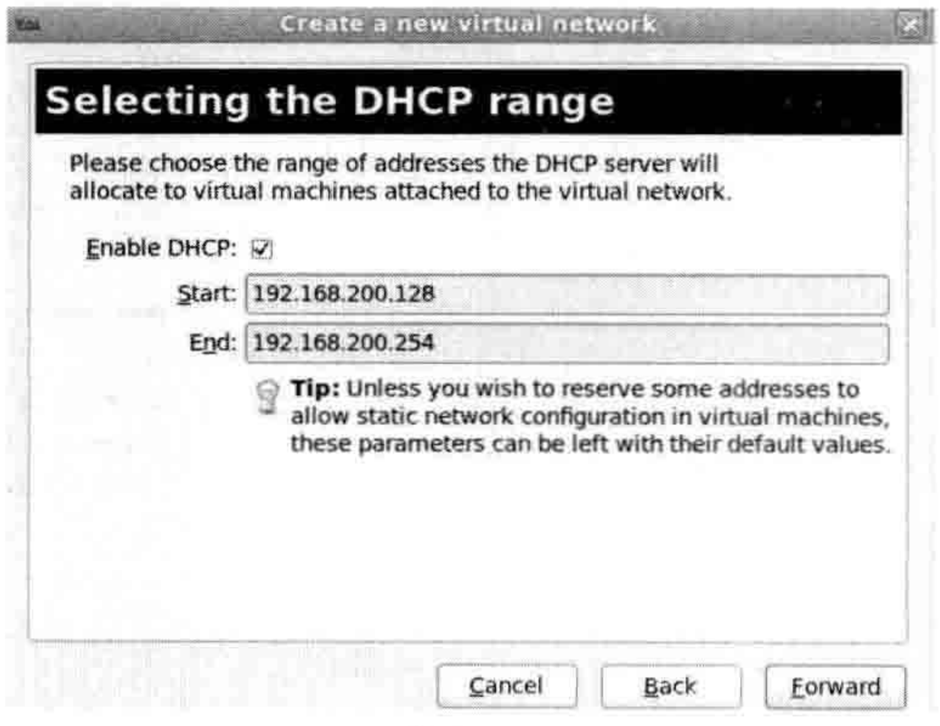


图 7-35

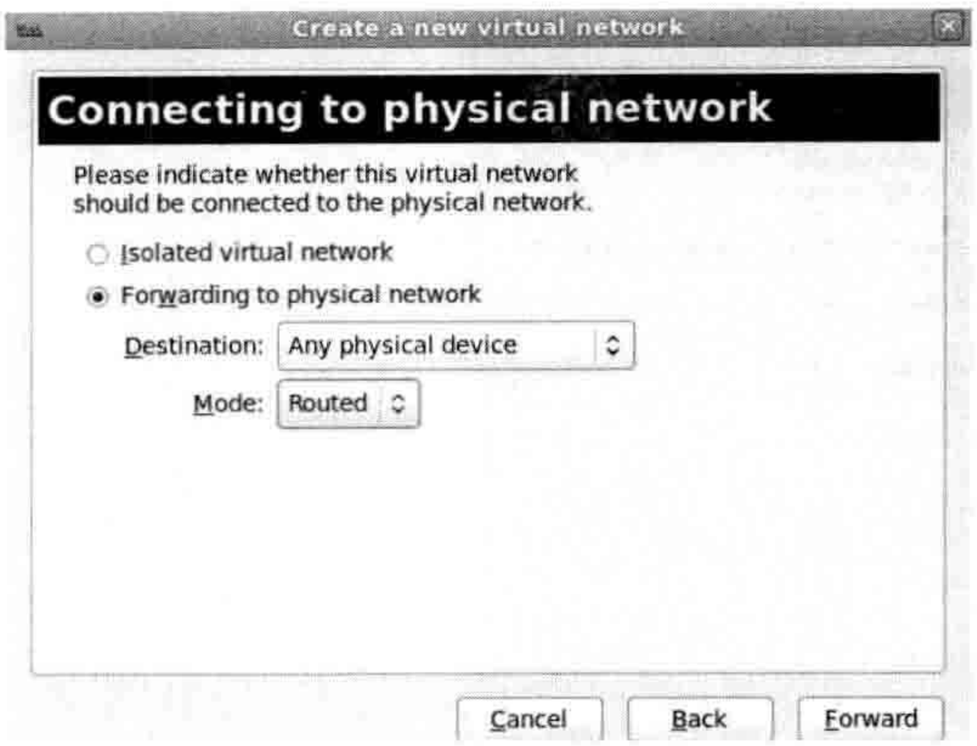


图 7-36

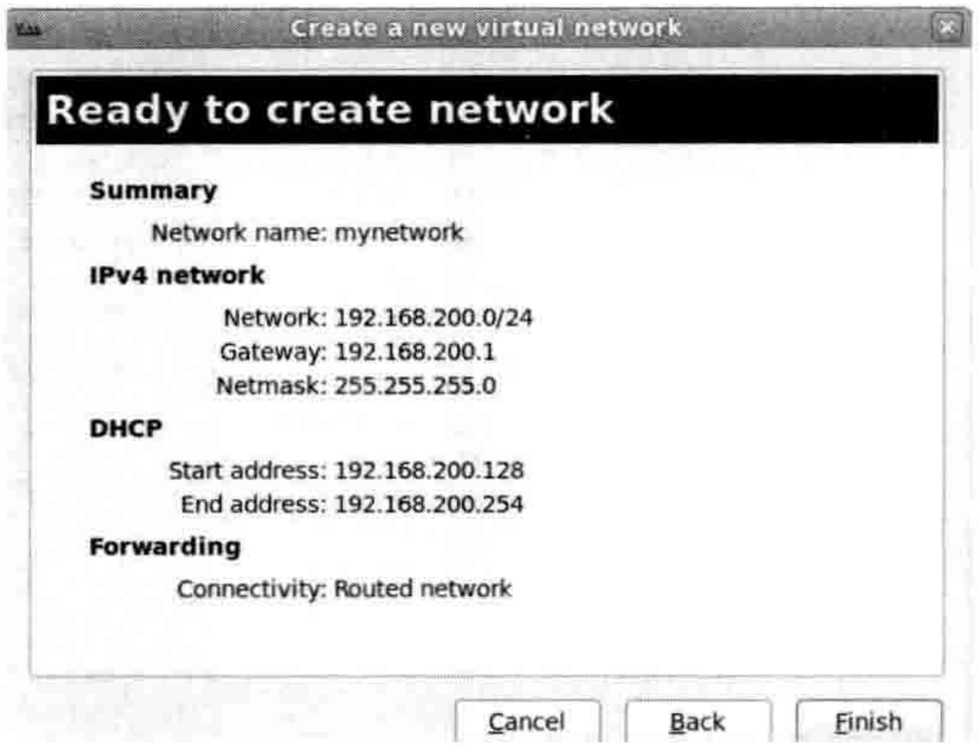


图 7-37

创建完成虚拟交换后，所有的虚拟主机都可以添加虚拟网卡接口连接该虚拟交换设备，打开虚拟主机后，通过如图 7-38 所示的硬件管理图标显示硬件列表，在硬件列表下方使用 Add Hardware 按钮添加虚拟网络接口，屏幕将显示如图 7-39 所示的添加新硬件对话框，选择 Network 网络设备，在右侧填写该网络接口需要连接的宿主主机的虚拟交换设备，这里我们选择的是刚刚创建的名为 mynetwork 的虚拟交换设备。除此之外，还需要填写该虚拟主机 MAC 地址以及设备驱动方式。

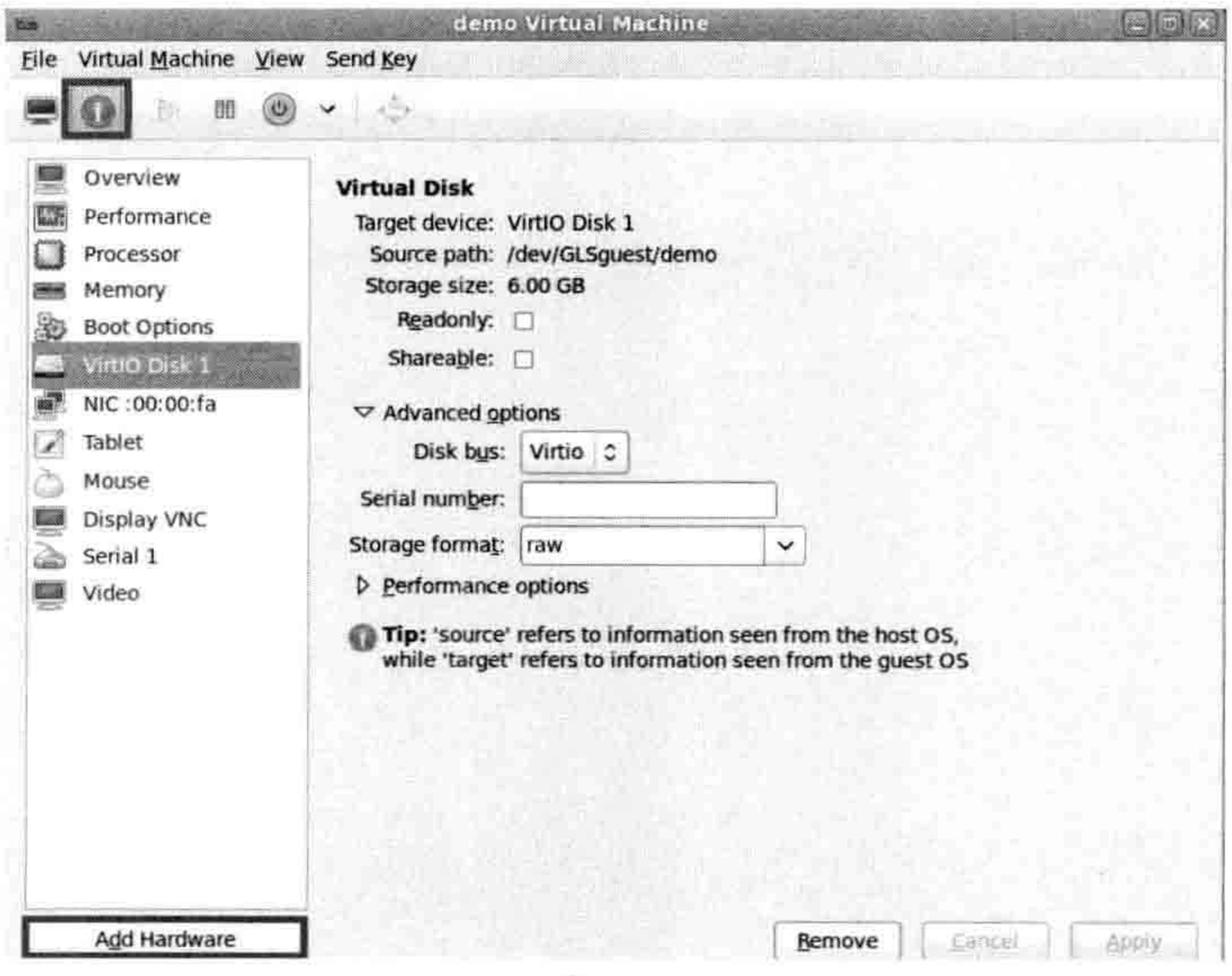


图 7-38

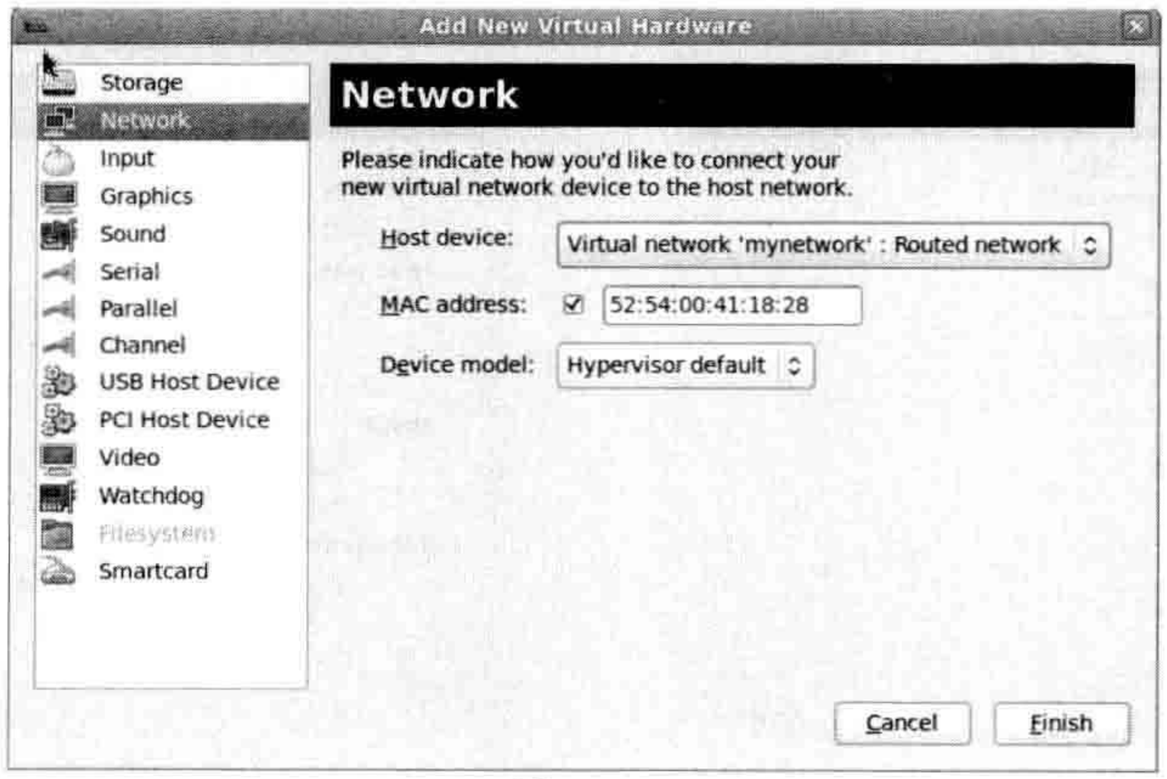

Add New Virtual Hardware
Storage
Network
Input
Graphics
Sound
Serial
Parallel
Channel
USB Host Device
PCI Host Device
Video
Watchdog
Filesystem
Smartcard
Network
Please indicate how you'd like to connect your new virtual network device to the host network.
Host device:
Virtual network 'mynetwork' : Routed network
MAC address:
52:54:00:41:18:28
Device model:
Hypervisor default
Cancel
Finish

图 7-39

第8章

集群及高可用

8.1 集群

随着互联网的发展，大量的客户端请求蜂拥而至，同时服务器的负载也越来越大，然而单台服务器的负载又是有限的，这样就会导致服务器响应客户端请求的时间越长，甚至产生拒绝服务的情况。另外，目前的网站多数是 7×24 小时提供不间断网络服务，如果仅采用单点服务器对外提供网络服务，那么在出现单点故障时，将导致整个网络服务中断。这时我们需要部署集群架构，最终将成百上千台主机有机地结合在一起，以满足当前大数据时代的海量访问负载。在部署集群环境时可以选择的产品有很多，有些是基于硬件实现的，有些是基于软件实现的。其中负载均衡的硬件设备有 F5 的 GIG-IP、Radware 的 AppDirector，以及梭子鱼的负载均衡设备等，软件有基于 Linux 的 LVS、Nginx 和 HAProxy 等产品。在集群环境中的核心是负载均衡以及高可用，本章将重点围绕这两点核心功能进行软件实现。

8.1.1 LVS 负载均衡简介

LVS（Linux Virtual Server）即 Linux 虚拟服务器，是由章文嵩博士主导开发的开源负载均

衡项目，目前 LVS 已经被集成到 Linux 内核模块中。该项目在 Linux 内核中实现了基于 IP 的数据请求负载均衡调度方案，其体系结构如图 8-1 所示，终端互联网用户从外部访问公司的外部负载均衡服务器，终端用户的 Web 请求会发送给 LVS 调度器，调度器根据自己预设的算法决定将该请求发送给后端的某台 Web 服务器，比如，轮询算法可以将外部的请求平均分发给后端的所有服务器，终端用户访问 LVS 调度器虽然会被转发到后端真实的服务器，但如果真实服务器连接的是相同的存储，提供的服务也都是相同的服务，最终用户不管是访问哪台真实的服务器，得到的服务内容都是一样的，整个集群对用户而言都是透明的。最后根据 LVS 工作模式的不同，真实服务器会选择不同的方式将用户需要的数据发送给终端用户，LVS 工作模式分为 NAT 模式、TUN 模式以及 DR 模式。

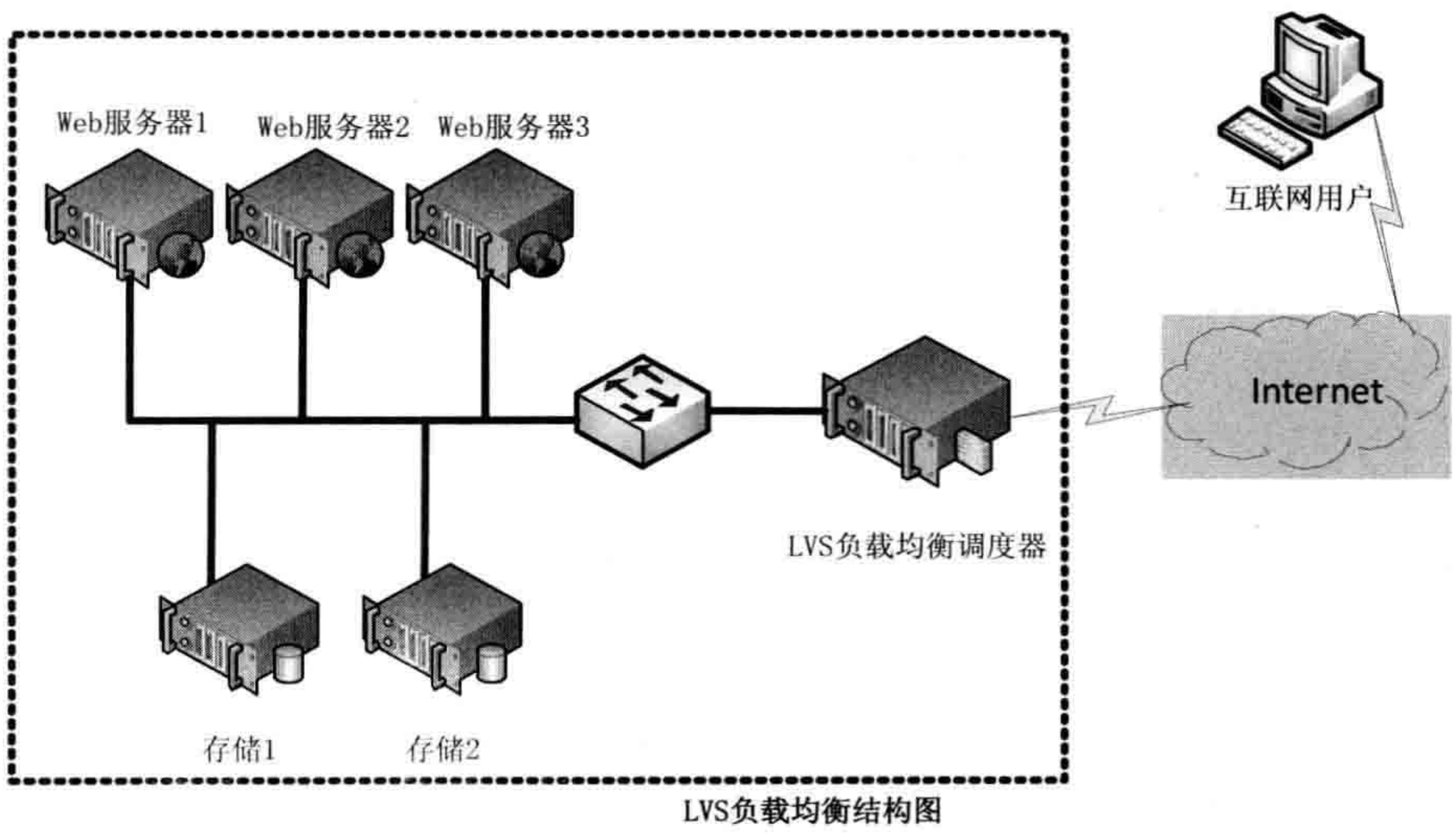


图 8-1

8.1.2 基于 NAT 的 LVS 负载均衡

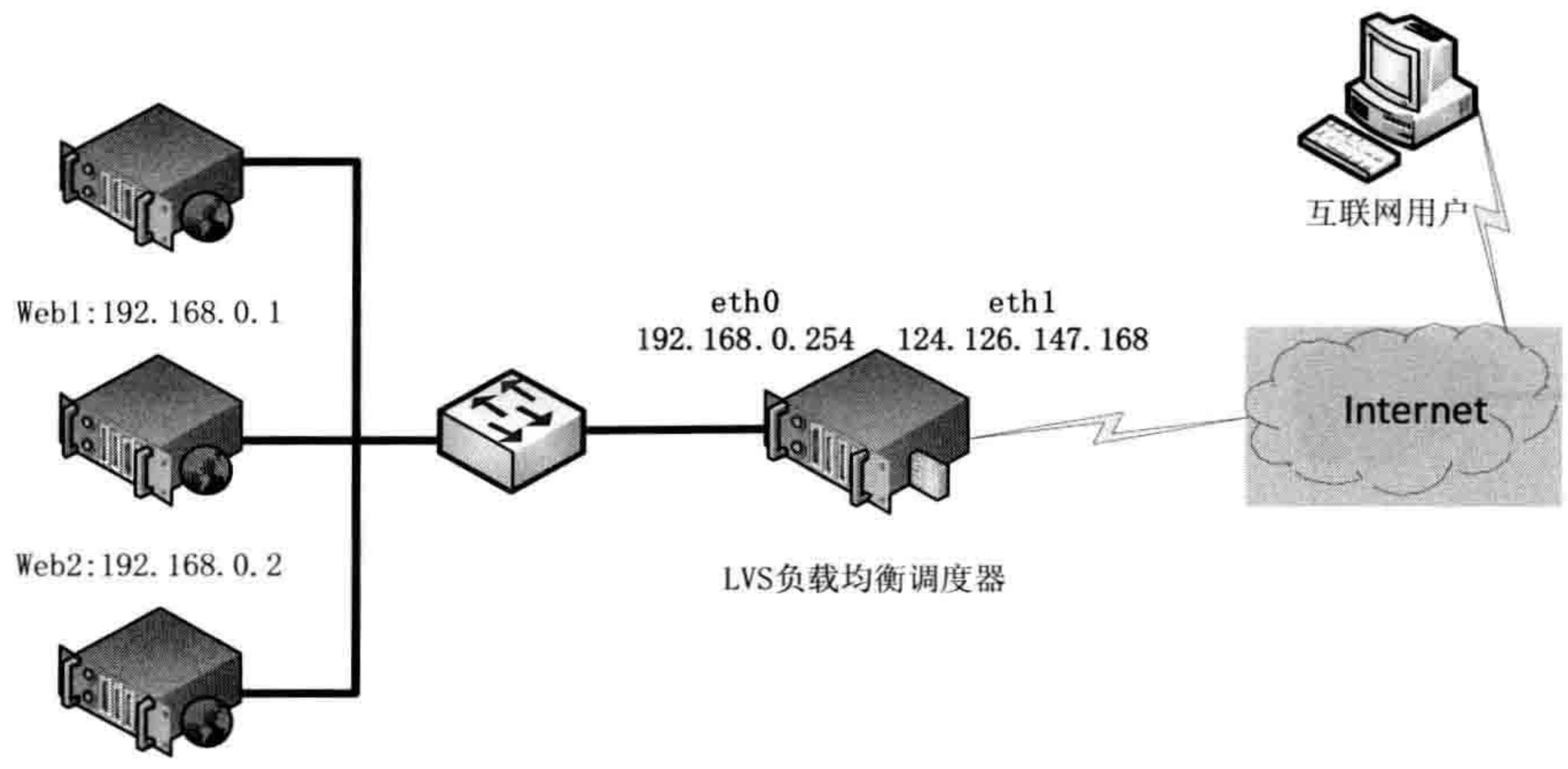
NAT（Network Address Translation）即网络地址转换，其作用是通过数据报头的修改，使位于企业内部的私有 IP 可以访问外网，以及外部用户可以访问位于公司内部的私有 IP 主机。VS/NAT 工作模式拓扑结构如图 8-2 所示，LVS 负载调度器使用两块网卡配置不同的 IP 地址，eth0 设置为私钥 IP 与内部网络通过交换设备相互连接，eth1 设置为外网 IP 与外部网络联通。

第一步，用户通过互联网 DNS 服务器解析到公司负载均衡设备上面的外网 IP 地址，相对于真实服务器而言，LVS 的外网 IP 又称为 VIP（Virtual IP Address），用户通过访问 VIP，即可

连接后端的真实服务器（Real Server），而这一切对用户而言都是透明的，用户以为自己访问的就是真实的服务器，但他并不知道自己访问的 VIP 仅仅是一个调度器，也不清楚后端的真实服务器到底在哪里、有多少台真实服务器。

第二步，用户将数据请求发送至 124.126.147.168，此时 lVS 将根据预设的算法选择后端的一台真实服务器（192.168.0.1~192.168.0.3），将数据请求包转发给真实服务器，并且在转发之前 LVS 会修改数据包中的目标地址以及目标端口，目标地址与目标端口将被修改为选出的真实服务器 IP 地址以及相应的端口。

第三步，真实的服务器将响应数据包返回给 LVS 调度器，调度器在得到响应数据包后会将源地址与源端口修改为 VIP 及调度器相应的端口，修改完成后，由调度器将响应数据包发送回终端用户。另外，由于 LVS 调度器有一个连接 Hash 表，该表中会记录连接请求及转发信息，当同一个连接的下一个数据包发送给调度器时，从该 Hash 表中可以直接找到之前的连接记录，并根据该记录信息选出相同的真实服务器及端口信息。



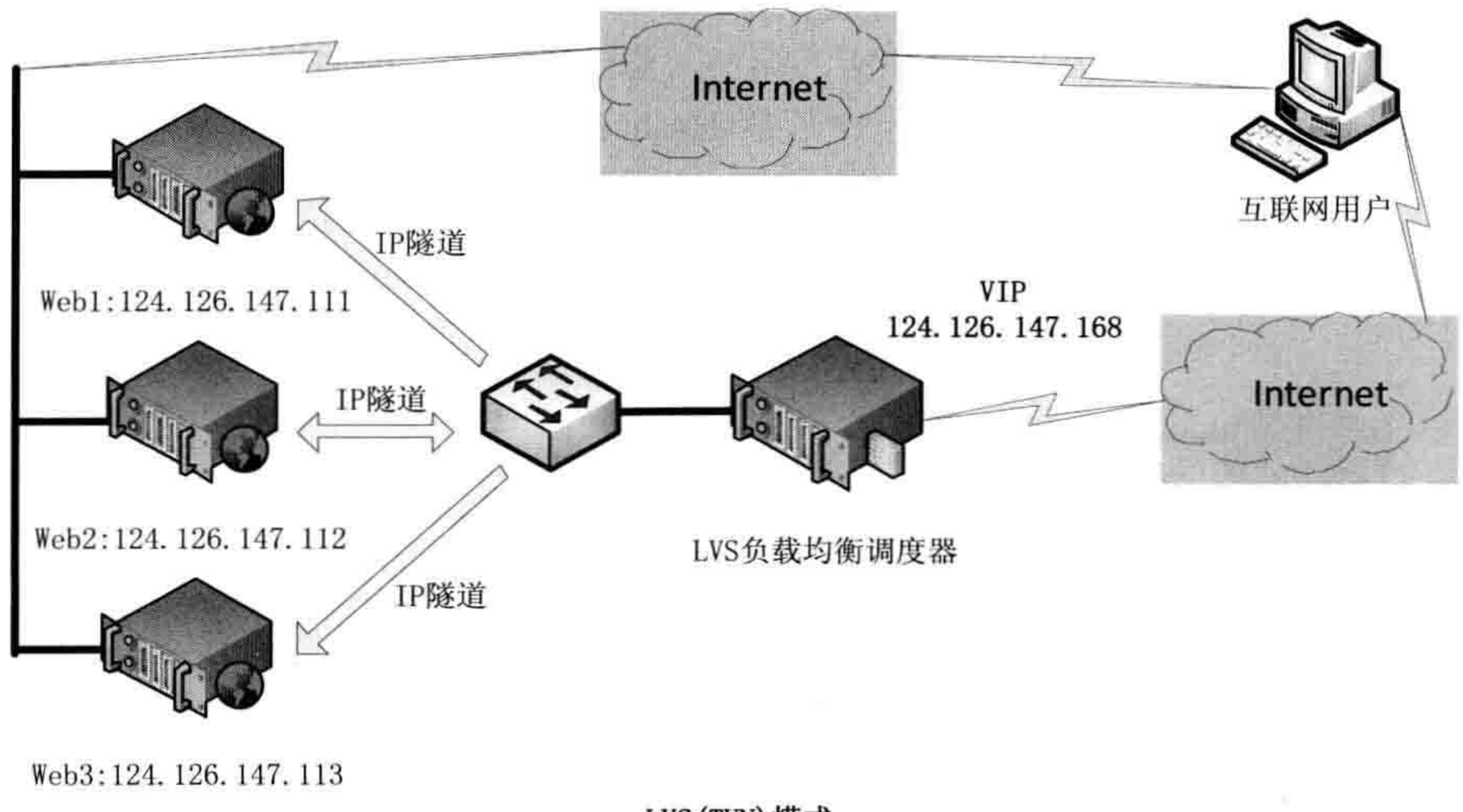
LVS(NAT)模式

图 8-2

8.1.3　基于 TUN 的 LVS 负载均衡

在 LVS（NAT）模式的集群环境中，由于所有的数据请求及响应的数据包都需要经过 LVS 调度器转发，如果后端服务器的数量大于 10 台，则调度器就会成为整个集群环境的瓶颈。我们知道，数据请求包往往远远小于响应数据包的大小。因为响应数据包中包含有客户需要的具体数据，所以 LVS（TUN）的思路就是将请求与响应数据分离，让调度器仅处理数据请求，而让

真实服务器将响应数据包直接返回给客户端。VS/TUN 工作模式拓扑结构如图 8-3 所示。其中，IP 隧道（IP tunning）是一种数据包封装技术，它可以将原始数据包封装并添加新的包头（内容包括新的源地址及端口、目标地址及端口），从而实现将一个目标为调度器 VIP 地址的数据包封装，通过隧道转发给后端的真实服务器（Real Server），通过将客户端发往调度器的原始数据包封装，并在其基础上添加新的数据包头（修改目标地址为调度器选择出来的真实服务器的 IP 地址及对应端口），LVS（TUN）模式要求真实服务器可以直接与外部网络连接，真实服务器在收到请求数据包后直接给客户端主机响应数据。



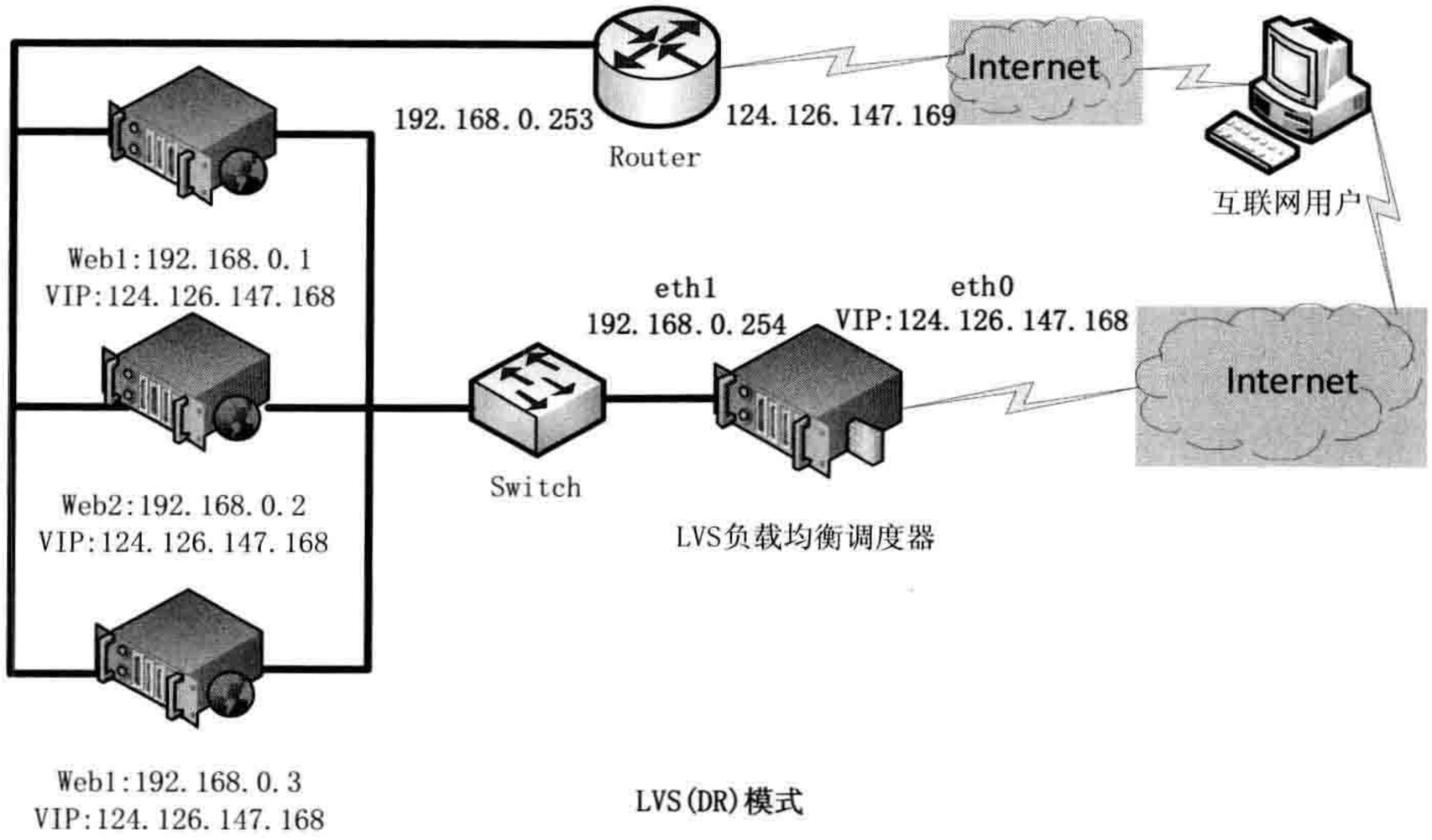
LVS(TUN)模式

图 8-3

8.1.4 基于 DR 的 LVS 负载均衡

在 LVS（TUN）模式下，由于需要在 LVS 调度器与真实服务器之间创建隧道连接，这同样会增加服务器的负担。与 LVS（TUN）类似，DR 模式也叫直接路由模式，其体系结构如图 8-4 所示，该模式中 LVS 依然仅承担数据的入站请求以及根据算法选出合理的真实服务器，最终由后端真实服务器负责将响应数据包发送返回给客户端。与隧道模式不同的是，直接路由模式（DR 模式）要求调度器与后端服务器必须在一个局域网内，VIP 地址需要在调度器与后端所有的服务器间共享，因为最终的真实服务器给客户端回应数据包时需要设置源 IP 为 VIP 地址，目标 IP 为客户端 IP，这样客户端访问的是调度器的 VIP 地址，回应的源地址也依然是该 VIP 地址（真实服务器上的 VIP），客户端是感觉不到后端服务器存在的。由于多台计算机都设置了同样一个

VIP 地址，所以在直接路由模式中要求调度器的 VIP 地址是对外可见的，客户端需要将请求数据包发送到调度器主机，而所有的真实服务器的 VIP 地址必须配置在 Non-ARP 的网络设备上，也就是该网络设备并不会向外广播自己的 MAC 及对应的 IP 地址，真实服务器的 VIP 对外界是不可见的，但真实服务器却可以接收目标地址为 VIP 的网络请求，并在回应数据包时将源地址设置为该 VIP 地址。调度器根据算法在选出真实服务器后，在不修改数据报文的情况下，将数据帧的MAC地址修改为选出的服务器的MAC地址，通过交换机将该数据帧转发给真实服务器。整个过程中，真实服务器的 VIP 不需要对外界可见。



LVS(DR)模式

图 8-4

8.1.5　LVS 负载均衡调度算法

根据前面的介绍，我们了解了 LVS 的三种模式，但不管实际环境中采用的是哪种模式，调度器进行调度的策略与算法都是LVS的核心技术，LVS在内核中主要实现了以下八种调度算法。

- 轮询调度
- 加权轮询调度
- 最小连接调度
- 加权最小连接调度
- 基于局部性最少的连接
- 带复制的基于局部性最少连接

- 目标地址散列调度
- 源地址散列调度

轮询算法（RR）就是按依次循环的方式将请求调度到不同的服务器上，该算法最大的特点就是实现简单。轮询算法假设所有的服务器处理请求的能力都是一样的，调度器会将所有的请求平均分配给每个真实服务器。

加权轮询算法（WRR）主要是对轮询算法的一种优化与补充，LVS 会考虑每台服务器的性能，并给每台服务器添加一个权值，如果服务器 A 的权值为 1，服务器 B 的权值为 2，则调度器调度到服务器 B 的请求会是服务器 A 的两倍。权值越高的服务器，处理的请求越多。

最小连接调度算法（LC）将把请求调度到连接数量最小的服务器上，而加权最小连接算法（WLC）则是给每个服务器一个权值，调度器会尽可能保持服务器连接数量与权值之间的平衡。

基于局部性的最少连接调度算法（lblc）是请求数据包的目标 IP 地址的一种调度算法，该算法先根据请求的目标 IP 地址寻找最近该目标 IP 地址所使用的服务器，如果这台服务器依然可用，并且有能力处理该请求，调度器会尽量选择相同的服务器，否则会继续选择其他可行的服务器。带复制的基于局部性最少连接算法（lblcr）记录的不是一个目标 IP 与一台服务器之间连接记录，它会维护一个目标 IP 到一组服务器之间的映射关系，防止单点服务器负载过高。

目标地址散列调度算法（DH）也是根据目标 IP 地址通过散列函数将目标 IP 与服务器建立映射关系，出现服务器不可用或负载过高的情况下，发往该目标 IP 的请求会固定发给该服务器。

源地址散列调度算法（SH）与目标地址散列调度算法类似，但它是根据源地址散列算法进行静态分配固定的服务器资源。

8.1.6 部署 LVS 服务

LVS 现在已经集成在 Linux 内核模块中，但整个 LVS 环境又分为内核层与用户层，内核层负责核心算法的实现，用户层需要安装 ipvsadm 工具，通过命令将管理员需要的工作模式与实现算法传递给内核来实现。LVS 的内核模块名称为 ip_vs，命令行工具在 CentOS 6.3 光盘中已经自带，我们可以使用 YUM 方式安装 ipvsadm，也可以自行去官方网站下载使用源码安装。

1. YUM 安装

YUM 安装需要确保本机可以连接 YUM 源，可以从 YUM 源中下载 RPM 格式软件包。

```
[root@lvs ~]# yum -y install ipvsadm
```

2. 源码安装

源码安装软件前需要使用 YUM 安装相关的依赖软件包。Ipvsadm 源码软件可以从官方网站下载，下载后使用标准的 make、make install，编译、安装即可。

```
[root@lvs ~]# yum -y install gcc popt-devel popt-devel popt-static \
>libnl libnl-devel
[root@lvs ~]# wget http://www.linuxvirtualserver.org/software/kernel-2.6/
ipvsadm-1.26.tar.gz
[root@lvs ~]# tar -xzf ipvsadm-1.26.tar.gz -C /usr/src/
[root@lvs ~]# cd /usr/src/ipvsadm-1.26/
[root@lvs ~]# make
[root@lvs ~]# make install
```

无论使用哪种方式安装 ipvsadm 软件，安装完成后都会生成一个同名的命令工具，我们需要使用该命令来管理配置 LVS 虚拟服务器组和相应的调度算法。

ipvsadm 命令的描述和用法如下。

描述：Linux 虚拟服务器管理工具。

用法：ipvsadm 选项　　服务器地址 -s 算法

　　ipvsadm 选项　　服务器地址 -r 真实服务器地址 [工作模式] [权重] …

选项：-A　　添加一个虚拟服务，使用 IP 地址、端口号、协议来唯一定义一个虚拟服务

　　-E　　编辑一个虚拟服务

　　-D　　删除一个虚拟服务

　　-C　　清空虚拟服务表

　　-R　　从标准输入中还原虚拟服务规则

　　-S　　保存虚拟服务规则至标准输出，输出的规则可以使用-R 导入还原

　　-a　　在虚拟服务中添加一台真实服务器

　　-e　　在虚拟服务中编辑一台真实服务器

　　-d　　在虚拟服务中减少一台真实服务器

　　-L　　显示虚拟服务列表

　　-t　　使用 TCP 服务，该参数后需要跟主机与端口信息

　　-u　　使用 UDP 服务，该参数后需要跟主机与端口信息

　　-s　　指定 LVS 所采用的调度算法

　　-r　　设置真实服务器 IP 地址与端口信息

　　-g　　设置 LVS 工作模式为 DR 直连路由模式

　　-i　　设置 LVS 工作模式为 TUN 隧道模式

-m　　设置 LVS 工作模式为 NAT 地址转换模式
-w　　设置指定服务器的权重
-c　　连接状态，需要配合-L 使用
-n　　数字格式输出

该命令示例如下。

添加一个虚拟服务，设置调度算法为轮询，所有使用的 TCP 协议访问 124.126.147.168 的 80 端口的请求，最终被调度器通过 NAT 模式转发给了 192.168.0.1、192.168.0.2、192.168.0.3 这三台主机的 80 端口：

```
[root@lvs ~]# ipvsadm -A -t 124.126.147.168:80 -s rr
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.1:80 -m
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.2:80 -m
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.3:80 -m
```

查看 LVS 规则表：

```
[root@lvs Desktop]# ipvsadm -Ln
IP Virtual Server version 1.2.1 (size=4096)
Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
TCP  124.126.126.147:80 rr
  -> 192.168.0.1:80               Masq    1      0          0
  -> 192.168.0.2:80               Masq    1      0          0
  -> 192.168.0.3:80               Masq    1      0          0
```

查看当前 IPVS 调度状态：

```
[root@lvs Desktop]# ipvsadm -Lnc
IPVS connection entries
pro expire state       source             virtual            destination
TCP 00:57  SYN_RECV    172.16.0.253:52830 192.168.0.111:80   10.0.0.3:80
TCP 00:58  SYN_RECV    172.16.0.253:52827 192.168.0.111:80   10.0.0.2:80
TCP 00:57  SYN_RECV    172.16.0.253:52829 192.168.0.111:80   10.0.0.2:80
TCP 00:57  SYN_RECV    172.16.0.253:52826 192.168.0.111:80   10.0.0.3:80
TCP 00:58  SYN_RECV    172.16.0.253:52828 192.168.0.111:80   10.0.0.3:80
```

删除为虚拟服务提供 Web 功能的真实服务器 192.168.0.3：

```
[root@lvs Desktop]# ipvsadm -d -t 124.126.126.147:80 -r 192.168.0.3
```

虚拟服务规则表备份与还原：

```
[root@lvs Desktop]# ipvsadm -S >/tmp/ip_vs.bak          #备份至文件
[root@lvs Desktop]# ipvsadm -C                          #清空规则表
[root@lvs Desktop]# ipvsadm -R </tmp/ip_vs.bak          #从文件还原
```

修改虚拟服务的调度算法：

```
[root@lvs Desktop]# ipvsadm -E -t 124.126.126.147:80 -s wrr
```

创建一个使用 WRR 算法的虚拟服务，工作模式为直接路由（DR）模式，在该虚拟服务上添加两台真实服务器，并为每台真实服务器设置权重：

```
[root@lvs ~]# ipvsadm -A -t 124.126.147.169:80 -s wrr
[root@lvs ~]# ipvsadm -a -t 124.126.147.169:80 -r 192.168.0.1:80 -i -w 1
[root@lvs ~]# ipvsadm -a -t 124.126.147.169:80 -r 192.168.0.2:80 -i -w 2
```

8.1.7　LVS 负载均衡应用案例

案例 1：基于 NAT 工作模式的负载均衡

基于 NAT 工作模式的负载均衡架构如图 8-2 所示，服务器 IP 地址设置见表 8-1。所有访问 lvs.example.com 这台主机的 VIP 地址的 80 端口的请求数据包都将被均衡地调度到三台真实的服务器上。

表 8-1

服务器名称	网络接口	IP 地址
lvs.example.com	eth1	124.126.147.168
	eth0	192.168.0.254
web1.example.com	eth0	192.168.0.1
web2.example.com	eth0	192.168.0.2
web3.example.com	eth0	192.168.0.3

LVS 负载均衡调度器设置。首先，需要为整个拓扑环境设置网络参数，案例中使用静态 IP 地址的方式配置网络，由于 LVS 调度器使用了两块网卡，这里需要修改 ifcfg-eth0 以及 ifcfg-eth1 两个网卡配置文件。

```
[root@lvs ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.254
NETMASK=255.255.255.0
DNS1=202.106.0.20
```

```
[root@lvs ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth1
```

```
DEVICE=eth1
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=124.126.147.168
NETMASK=255.255.255.240
GATEWAY=124.126.147.169
DNS1=202.106.0.20
```

```
[root@lvs ~]# service network restart
[root@lvs ~]# yum -y install ipvsadm
```

使用 ipvsadm 命令工具添加一个虚拟服务，并为该虚拟服务设置一组具体的后端服务器主机和虚拟服务的调度算法。

```
[root@lvs ~]# ipvsadm -A -t 124.126.147.168:80 -s rr
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.1:80 -m
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.2:80 -m
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.3:80 -m
[root@lvs ~]# service ipvsadm save                          #保存调度规则
[root@lvs ~]# sed -l /ip_forward/s/0/1/ /etc/sysctl.conf #开启路由转发功能
[root@lvs ~]# sysctl -p                                     #重新加载 sysctl.conf
[root@lvs ~]# iptables -F
[root@lvs ~]# iptables -X
[root@lvs ~]# service iptables save
```

后端真实 Web 服务器设置（这里为了可以验证调度器每次选取的是不同的服务器，我们将每个真实 Web 服务器的页面内容设置为不同的内容，而真实生产环境中所有的 Web 服务器都应该提供相同的页面内容）。与 LVS 调度器主机一样，首先需要根据实际的网络环境为真实服务器设置网络参数，通过修改 ifcfg-eth0 即可实现。

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.1
NETMASK=255.255.255.0
GATEWAY=192.168.0.254
```

由于 LVS 调度器会将请求转发给后端真实服务器，所以所有的后端服务器都需要配置 Web 服务。为了使内容结构简单、突出重点，本例仅生成一个测试用网页文件 index.html。

```
[root@web1 ~]# yum -y install httpd
```

```
[root@web1 ~]# service httpd start
[root@web1 ~]# service network restart
[root@web1 ~]# echo "192.168.0.1" > /var/www/html/index.html
[root@web1 ~]# iptables -F
[root@web1 ~]# iptables -X
[root@web1 ~]# service iptables save
```

下面是 web2 和 web3 服务器主机的具体操作步骤。因为都承担着后端真实 Web 服务器的角色，所以操作步骤与 web1 主机基本一致，IP 地址等细致问题需要读者根据自己的实际需要稍作修改。

```
[root@web2 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.2
NETMASK=255.255.255.0
GATEWAY=192.168.0.254
[root@web2 ~]# yum -y install httpd
[root@web2 ~]# service httpd start
[root@web2 ~]# service network restart
[root@web2 ~]# echo "192.168.0.2" > /var/www/html/index.html
[root@web2 ~]# iptables -F
[root@web2 ~]# iptables -X
[root@web2 ~]# service iptables save
[root@web3 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.3
NETMASK=255.255.255.0
GATEWAY=192.168.0.254
[root@web3 ~]# yum -y install httpd
[root@web3 ~]# service httpd start
[root@web3 ~]# service network restart
[root@web3 ~]# echo "192.168.0.3" > /var/www/html/index.html
[root@web3 ~]# iptables -F
[root@web3 ~]# iptables -X
[root@web3 ~]# service iptables save
```

验证：

客户端使用浏览器访问 http:// 124.126.147.168，最终可以访问到真实服务器所提供的页面内容，由于 LVS 采用 RR（轮询）算法，所以不同的连接请求将被平均分配到不同的后端服务器上。本例中由于所有的页面内容都不相同，所以不同的客户端访问 http:// 124.126.147.168 后将得到不同的页面内容。

案例 2：基于 DR 工作模式的负载均衡

基于 DR 工作模式的结构如图 8-4 所示，服务器 IP 地址设置见表 8-2。所有访问 lvs.example.com 这台主机的 VIP 地址（124.126.147.168）的 80 端口都将被均衡地调度到三台真实的服务器上。真实的服务器在收到调度器转发过来的请求包后直接通过 Router 路由器发送响应数据给客户端，这里要求所有真实服务器的网关地址指向 Router 的内网 IP 地址，由于调度器与真实服务器都设置了 VIP 地址，所以要求真实服务器在 Non-ARP 设置上配置 VIP 地址。本例所使用的所有服务器均为 Linux 操作系统，我们使用 Linux 来承担软路由的功能。

表 8-2

服务器名称	网络接口	IP 地址
Router	外网接口（eth1）	124.126.147.169
	内网接口（eth0）	192.168.0.253
lvs.example.com	eth0（VIP）	124.126.147.168
	eth1	192.168.0.254
web1.example.com	eth0	192.168.0.1
	lo:0（VIP）	124.126.147.168
web2.example.com	eth0	192.168.0.2
	lo:0（VIP）	124.126.147.168
web3.example.com	eth0	192.168.0.3
	lo:0（VIP）	124.126.147.168

LVS 负载均衡调度器设置，首先部署网络拓扑环境，修改网卡配置文件。

```
[root@lvs ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=124.126.147.168
NETMASK=255.0.0.0
DNS1=202.106.0.20
```

```
[root@lvs ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth1
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.254
NETMASK=255.255.255.0
DNS1=202.106.0.20
```

```
[root@lvs ~]# service network restart
```

接下来安装 ipvsadm 管理工具，创建虚拟服务，添加真实服务器组，并为虚拟服务设置适当的调度算法。

```
[root@lvs ~]# yum -y install ipvsadm
[root@lvs ~]# ipvsadm -A -t 124.126.147.168:80 -s wrr
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.1:80 -g -w 1
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.2:80 -g -w 2
[root@lvs ~]# ipvsadm -a -t 124.126.147.168:80 -r 192.168.0.3:80 -g -w 3
[root@lvs ~]# service ipvsadm save
[root@lvs ~]# iptables -F
[root@lvs ~]# iptables -X
[root@lvs ~]# service iptables save
```

真实 Web 服务器设置如下（这里为了可以验证调度器每次选取的是不同的服务器，我们将每个真实 Web 服务器的页面内容设置为不同的内容，而真实生产环境中所有的 Web 服务器都应该提供相同的页面内容）。注意，因为调度器与真实服务器都设置了 VIP 地址，所以这里要求所有的真实服务器要禁止对 VIP 地址的 ARP 响应，方法是通过 arp_ignore 与 arp_announce 来实现的：

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.1
NETMASK=255.255.255.0
GATEWAY=192.168.0.253
```

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-lo:0
```

```
DEVICE=lo:0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=124.126.147.168
NETMASK=255.255.255.255
```

```
GATEWAY=192.168.0.253
```

```
[root@web1 ~]# vim /etc/sysctl.conf
```

```
net.ipv4.conf.eth0.arp_ignore = 1
net.ipv4.conf.eth0.arp_announce = 2
net.ipv4.conf.all.arp_ignore = 1
net.ipv4.conf.all.arp_announce = 2
… …
```

```
[root@web1 ~]# sysctl -p
[root@web1 ~]# yum -y install httpd
[root@web1 ~]# service httpd start
[root@web1 ~]# service network restart
[root@web1 ~]# echo "192.168.0.1" > /var/www/html/index.html
[root@web1 ~]# iptables -F
[root@web1 ~]# iptables -X
[root@web1 ~]# service iptables save
[root@web2 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.2
NETMASK=255.255.255.0
GATEWAY=192.168.0.253
```

```
[root@web2 ~]# vim /etc/sysconfig/network-scripts/ifcfg-lo:0
```

```
DEVICE=lo:0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=124.126.147.168
NETMASK=255.255.255.255
GATEWAY=192.168.0.253
```

```
[root@web2 ~]# vim /etc/sysctl.conf
```

```
net.ipv4.conf.eth0.arp_ignore = 1
net.ipv4.conf.eth0.arp_announce = 2
net.ipv4.conf.all.arp_ignore = 1
net.ipv4.conf.all.arp_announce = 2
… …
```

```
[root@web2 ~]# sysctl -p
[root@web2 ~]# yum -y install httpd
[root@web2 ~]# service httpd start
```

```
[root@web2 ~]# service network restart
[root@web2 ~]# echo "192.168.0.2" > /var/www/html/index.html
[root@web2 ~]# iptables -F
[root@web2 ~]# iptables -X
[root@web2 ~]# service iptables save
[root@web3 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=192.168.0.3
NETMASK=255.255.255.0
GATEWAY=192.168.0.253
```

```
[root@web3 ~]# vim /etc/sysconfig/network-scripts/ifcfg-lo:0
```

```
DEVICE=lo:0
BOOTPROTO=static
ONBOOT=yes
TYPE=Ethernet
IPADDR=124.126.147.168
NETMASK=255.255.255.255
GATEWAY=192.168.0.253
```

```
[root@web3 ~]# vim /etc/sysctl.conf
```

```
net.ipv4.conf.eth0.arp_ignore = 1
net.ipv4.conf.eth0.arp_announce = 2
net.ipv4.conf.all.arp_ignore = 1
net.ipv4.conf.all.arp_announce = 2
… …
```

```
[root@web3 ~]# sysctl -p
[root@web3 ~]# yum -y install httpd
[root@web3 ~]# service httpd start
[root@web3 ~]# service network restart
[root@web3 ~]# echo "192.168.0.3" > /var/www/html/index.html
[root@web3 ~]# iptables -F
[root@web3 ~]# iptables -X
[root@web3 ~]# service iptables save
```

路由器（Router）采用 Linux 作为软件路由来实现本例，具体操作如下。

```
[root@router ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
```

```
    TYPE=Ethernet
    IPADDR=192.168.0.253
    NETMASK=255.255.255.0
```

```
[root@router ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth1
```

```
    DEVICE=eth1
    BOOTPROTO=static
    ONBOOT=yes
    TYPE=Ethernet
    IPADDR=
    IPADDR=124.126.147.168
    NETMASK=255.0.0.0
```

```
[root@router ~]# vim /etc/sysctl.conf
```

```
    net.ipv4.ip_forward = 1
    … …
```

```
[root@router ~]# sysctl -p
[root@router ~]# service network restart
[root@router ~]# iptables -F
[root@router ~]# iptables -X
[root@router ~]# service iptables save
```

验证：

客户端使用浏览器访问 http:// 124.126.147.168，最终可以访问到真实服务器所提供的页面内容，由于 LVS 采用 WRR（加权轮询）算法，不同的连接请求将被分配到不同的后端服务器上，但服务器之间的优先级不同，本例中由于所有的页面内容都不相同，所以客户端多次访问后将得到不同的页面内容。

8.1.8 常见问题分析

（1）路由转发功能。

在 LVS（NAT）工作模式中，LVS 调度器必须能够提供数据转发功能，而 CentOS 6.3 系统默认的规则没有开启路由转发功能，需要手动修改/etc/sysctl.conf 文件开启该功能。

（2）在 LVS（NAT）工作模式中，调度器除了担当调度的角色外，还需要担当路由的角色，但 CentOS 6.3 系统的防火墙转发规则默认为禁止转发，因此需要将转发规则清空，具体操作如下：

```
[root@lvs ~]# iptables -F
[root@lvs ~]# iptables -X
[root@lvs ~]# service iptables save
```

（3）在 LVS（DR）工作模式下，由于所有的真实服务器都配置了 VIP 地址，因此需要设置服务器不进行针对 VIP 地址的 ARP 广播，Linux 中可以直接通过 arp_ignore 与 arp_announce 来实现。

arp_ignore 用来定义网卡在响应外部 ARP 请求时的响应级别。

- 0：默认值，任何网络接口收到 ARP 请求后，如果本机的任意接口有该 MAC，则予以响应。
- 1：某个网络接口收到 ARP 请求后，判断请求的 MAC 地址是否是本接口，是则回应，否则不回应。LVS 调度器会将客户请求转发给真实服务器的 eth0 接口，而真实服务器的 VIP 地址配置在本地回环设备上。

arp_announce 用来定义网卡广播 ARP 包时的级别。

- 0：默认值，任何网络接口接收到 ARP 请求后，如果本机的任意接口有该 MAC，则予以响应。
- 1：尽量避免响应 MAC 地址非本网络接口 MAC 地址的 ARP 请求。
- 2：不响应 MAC 地址非本网络接口 MAC 地址的 ARP 请求。

8.2　Keepalived 双机热备

8.2.1　Keepalived 简介

Keepalived 是使用 C 语言编写的路由热备软件，该项目软件的主要目标是为 Linux 系统提供简单高效的负载均衡及高可用解决方案。负载均衡架构依赖于知名的 IPVS（Linux Virtual Server，即基于 IP 的虚拟服务器）内核模块，Keepalived 由一组检查器根据服务器的健康状况动态地维护和管理服务器池。另外，Keepalived 通过 VRRP（Virtual Router Redundancy Protocol）协议实现高可用架构，VRRP 是路由灾备的实现基础。在 8.1 节中我们看到了 LVS 调用一组服务器提供虚拟服务的强大负载均衡能力。但 LVS 的核心是调度器，所有的数据请求都需要经过调度器进行调度转发。因此，万一调度器发生故障，则整个集群系统将全部崩溃，所以我们需要 Keepalived 来实现集群系统的高可用。部署两台或多台 LVS 调度器，仅有一台调度器作为主服务器，其他调度器作为备用，当主调度器发生故障时，Keepalived 可以自动将备用调度器升级为主调度器，最终实现整个集群系统的高负载、高可用。

8.2.2 VRRP 协议简介

VRRP（Virtual Router Redundancy Protocol）协议是为了在静态路由环境下防止单点故障而设计的主从灾备协议，VRRP 实现在主设备发生故障时将业务自动切换至从设备，而这一切对用户而言是透明的。VRRP 将两台或多台路由设备虚拟成一个设备，对外仅提供一个虚拟的路由 IP 地址，而多台路由设备同一时刻仅可以有一台设备拥有该虚拟 IP 地址，该设备就是主路由设备，其他设备为备份设备。主设备会不断地发送自己的状态信息给备用路由设备，当备用设备接收不到主设备的状态信息时，备用设备将根据自身的优先级立刻选举出新的主设备，并提供所有的业务功能。VRRP 协议需要为每个路由设备定义虚拟路由 ID（VRID）以及设备优先级别，所有主备路由设备的 VRID 必须一样，所有 VRID 相同的路由设备组成一个虚拟路由设备组，组内优先级高的路由设备将被选举为主路由设备。虚拟路由设备 ID 与优先级均为 0 至 255 之间的整数，如果优先级相等，则继续对比路由设备的实际 IP 地址，IP 地址越大，优先级就越高。

有一种特殊情况是，如果将虚拟路由 IP 地址设置为多台路由设备中某台设备的真实 IP 地址，则该路由设备将永远处于主设备状态。

8.2.3 安装 Keepalived 服务

安装源码软件包前需要通过 YUM 方式安装相关的依赖软件包。Keepalived 源码软件可以通过官方网站下载，下载后使用 configure、make、make install 标准方式安装该软件。软件安装完成后，为了后期操作的方便，我们还为该软件的部分文件与目录创建了快捷方式。

```
[root@lvs_1 ~]# yum -y install gcc openssl-devel popt-devel \
>libnl libnl-devel kernel-devel ipvsadm
[root@lvs_1 ~]# http://www.keepalived.org/software/keepalived-1.2.7.tar.gz
[root@lvs_1 ~]# tar -xzf keepalived-1.2.7.tar.gz -C /usr/src/
[root@lvs_1 ~]# cd /usr/src/ keepalived-1.2.7/
[root@lvs_1 ~]#./configure
[root@lvs_1 ~]# make && make install
[root@lvs_1 ~]# ln -s /usr/local/etc/keepalived/ /etc
[root@lvs_1 ~]# ln -s /usr/local/etc/rc.d/init.d/keepalived /etc/init.d/
[root@lvs_1 ~]# ln -s /usr/local/etc/sysconfig/keepalived /etc/sysconfig/
[root@lvs_1 ~]# ln -s /usr/local/sbin/keepalived /usr/sbin/
```

8.2.4　配置文件解析

Keepalived 主配置文件的名称为/etc/keepalived/keepalived.conf，配置文件主要分为全局配置块、VRRP 配置块、LVS 配置块，每个配置块都以{}包裹，以#或!开头的行在配置文件中代表注释行。具体的 Keepalived 核心配置参数说明见表 8-3。

```
global_defs {
  …
}
vrrp_instance VI_1 {
  …
}
virtual_server 192.168.200.100 443 {
  …
}
  …
```

表 8-3

设置项	功能描述
notification_email	定义邮件列表，当主服务器出现故障进行主从切换时会发送邮件给邮件列表中的所有人
notification_email_from	定义邮件发送者
smtp_server	设置邮件服务器 IP 地址
router_id	设备标识，一般可以设置为主机名
enable_traps	启用 SNMP 跟踪
static_ipaddress	设置系统真实的静态 IP 地址，也可以通过系统层面设置
static_	设置系统真实的静态路由地址，也可以通过系统层面设置
vrrp_instance	定义 VRRP 实例，描述主从初始状态、虚拟路由 ID、优先级等
state	设置初始设备状态，MASTER 为主设备，BACKUP 为从设备
interface	可以绑定 VRRP 的网络接口，如 eth0、eth1 等
use_vmac	使用 VRRP 虚拟 MAC 地址
dont_track_primary	忽略接口故障，默认为设置
track_interface	设置要跟踪监控的网络接口
mcast_src_ip	发送状态通告信息的源 IP 地址
virtual_router_id	设置虚拟路由 ID，相同的 VRID 为一个组，要求主从 VRID 要一致，该值为 0~255
priority	设置虚拟路由组中设备的优先级，优先级高者将被推举为主设备
advert_int	检查间隔，默认为 1 秒
authentication	定义主从验证设置，相同的 VRRP 实例中主从验证方式与密码要一致

续表

设置项	功能描述
auth_type	设置验证方式（PASS 或 AH），官方文档建议使用密码
auth_pass	验证密码
virtual_ipaddress	定义虚拟 IP 地址，在 VRRP 实例中仅主设备实时拥有该 IP 地址 格式： <IP>/<MASK> dev <STRING> scop <SCOPE> label <LABLE> 如：192.168.100.100/24 dev eth2 label eth2:1
virtual_routes	定义虚拟路由，如：192.168.200.0/24 via 192.168.100.254
virtual_server_group	定义 LVS 虚拟服务设置块，定义虚拟 IP 及端口号
delay_loop	服务器轮询间隔时间
lb_algo	LVS 负载均衡算法，可以设置为：rr|wrr|lc|wlc|lblc|sh|dh
lb_kind	LVS 负载均衡工作模式，可以设置为：NAT|DR|TUN
persistence_timeout	会话保持时间，当用户多次请求时，调度器可以转发给相同的真实服务器
protocol	数据转发协议
real_server	定义后端真实服务器，添加所有的真实服务器的 IP 地址与端口信息
Weight	设置服务器权重
HTTP_GET	对后端真实服务器进行 HTTP 健康检查
TCP_CHECK	对后端真实服务器进行 TCP 健康检查
connect_port	健康检查的端口号
bindto	健康检查的 IP 地址
connect_timeout	连接超时时间，单位为秒
nb_get_retry	连接重试次数（number of get retry）
delay_before_retry	连接重试的间隔时间

8.2.5 Keepalived+LVS 应用案例

下面将演示如何使用 Keepalived 实现基于 LVS（DR）模式的集群方案，结构如图 8-5 所示，终端用户通过路由访问当前拥有 VIP 的调度器（需要在路由设备上设置 NAT 规则），我们使用 Keepalived 实现 VIP 在两台 LVS 之间切换，默认的 LVS1 为主调度设备，LVS1 调度器出现故障后，VIP 可以自动切换至 LVS2。当前工作的调度器负责把客户请求转发给选出来的后端真实服务器，由于使用的是 DR 工作模式，所有的真实服务器需要在自己的 Non-ARP 设备上设置 VIP，

最终由真实服务器将响应数据包通过路由回传给终端客户。图中的路由设备同样使用 Linux 服务器来承担软路由的功能，注意添加 NAT 路由转发规则，具体的配置列表见表 8-4。

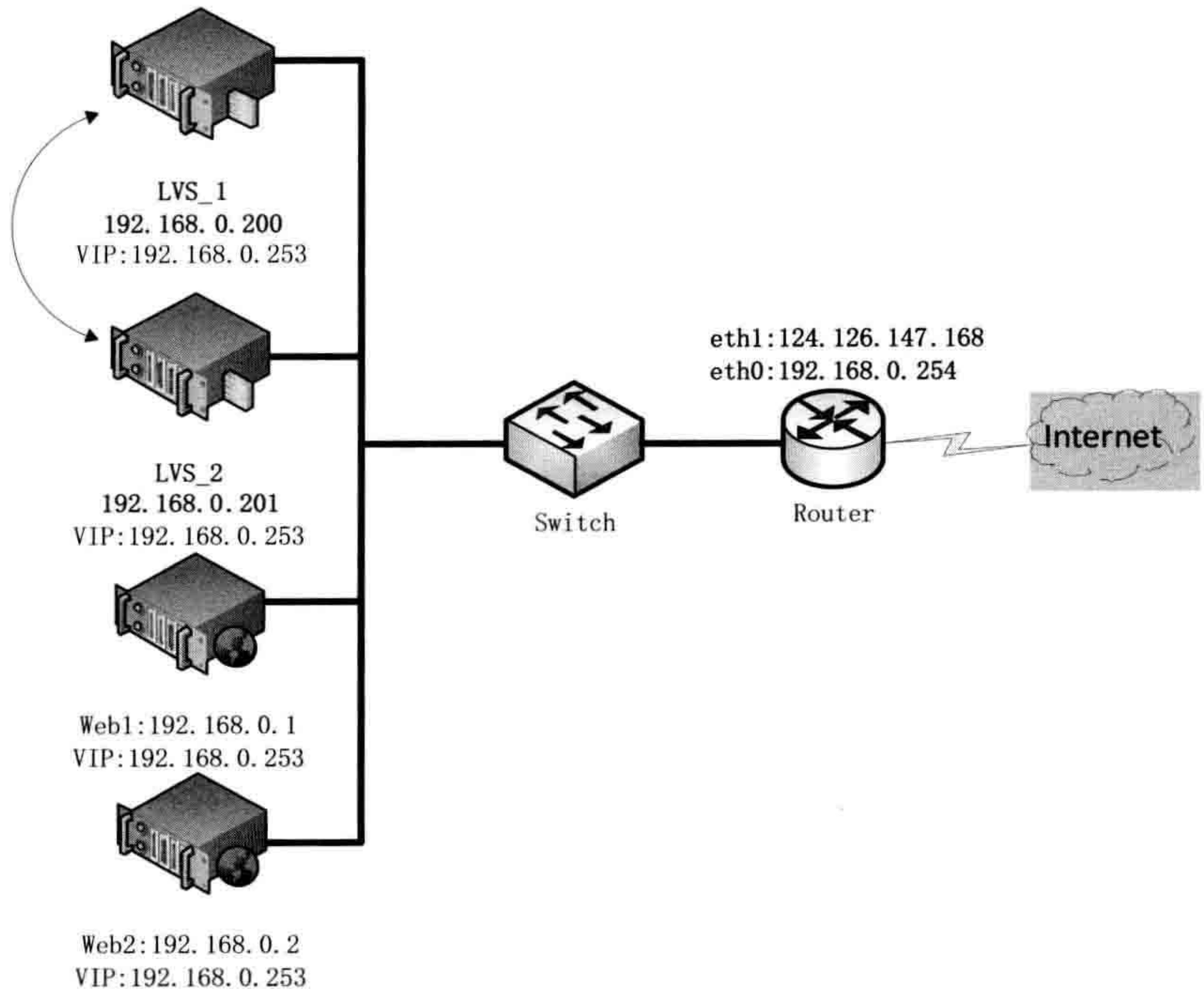


图 8-5

表 8-4

服务器名称	网络接口	IP 地址
router	外网接口（eth1）	124.126.147.168
	内网接口（eth0）	192.168.0.254
lvs_1.example.com	eth0	192.168.0.200
	eth0（VIP）	192.168.0.253
lvs_1.example.com	eth0	192.168.0.201
	eth0（VIP）	192.168.0.253
web1.example.com	eth0	192.168.0.1
	lo:0（VIP）	192.168.0.253
web2.example.com	eth0	192.168.0.2
	lo:0（VIP）	192.168.0.253

web1 操作步骤如下：首先需要通过网卡配置文件为服务器主机设置网络参数。需要注意的是，这里需要为回环设备创建网卡别名 lo:0，并为该虚拟别名网卡设置 VIP 地址参数。

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
IPADDR=192.168.0.1
NETMASK=255.255.255.0
GATEWAY=192.168.0.254
ONBOOT=yes
```

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-lo:0
```

```
DEVICE=lo:0
BOOTPROTO=static
IPADDR=192.168.0.253
NETMASK=255.255.255.255
GATEWAY=192.168.0.254
ONBOOT=yes
```

由于网络中多个设备均被设置了 VIP 地址，为了防止出现地址冲突问题，需要通过 sysctl.conf 文件修改内核 ARP 相关参数，我们需要在该文件中加入如下四行内容。

```
[root@web1 ~]# vim /etc/sysctl.conf
```

```
net.ipv4.conf.eth0.arp_ignore = 1
net.ipv4.conf.eth0.arp_announce = 2
net.ipv4.conf.all.arp_ignore = 1
net.ipv4.conf.all.arp_announce = 2
… …
```

当我们修改 sysctl.conf 文件后，该文件中的配置并不会立刻生效，默认需要重启计算机内核才会识别到这些参数。而如果使一个在线服务器设备重启计算机肯定是无法接受的，这时可以使用 sysctl -p 命令，其作用是使 sysctl.conf 文件中的设置立刻可以生效。

```
[root@web1 ~]# sysctl -p
[root@web1 ~]# service network restart
```

由于是 Web 服务器，所以该主机需要安装部署 HTTP 服务，并设置防火墙规则以允许其他主机访问本机 HTTP 服务。

```
[root@web1 ~]# iptables -F
[root@web1 ~]# iptables -X
[root@web1 ~]# service iptables save
[root@web1 ~]# yum -y install httpd
[root@web1 ~]# echo "192.168.0.1" > /var/www/html/index.html
[root@web1 ~]# service httpd restart
```

```
[root@web1 ~]# chkconfig httpd on
```

web2 操作步骤如下：因为都是后端真实 Web 服务器，所以 web2 的具体操作与 web1 服务器的基本相同，这里不再赘述，具体说明可以参考 web1 相关的操作说明。

```
[root@web2 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
BOOTPROTO=static
IPADDR=192.168.0.2
NETMASK=255.255.255.0
GATEWAY=192.168.0.254
ONBOOT=yes
```

```
[root@web2 ~]# vim /etc/sysconfig/network-scripts/ifcfg-lo:0
```

```
DEVICE=lo:0
BOOTPROTO=static
IPADDR=192.168.0.253
NETMASK=255.255.255.255
GATEWAY=192.168.0.254
ONBOOT=yes
```

```
[root@web2 ~]# vim /etc/sysctl.conf
```

```
net.ipv4.conf.eth0.arp_ignore = 1
net.ipv4.conf.eth0.arp_announce = 2
net.ipv4.conf.all.arp_ignore = 1
net.ipv4.conf.all.arp_announce = 2
… …
```

```
[root@web2 ~]# sysctl -p
[root@web2 ~]# service network restart
[root@web2 ~]# iptables -F
[root@web2 ~]# iptables -X
[root@web2 ~]# service iptables save
[root@web2 ~]# yum -y install httpd
[root@web2 ~]# echo "192.168.0.2" > /var/www/html/index.html
[root@web2 ~]# service httpd restart
[root@web2 ~]# chkconfig httpd on
```

LVS_1 调度器操作步骤如下（首先通过网卡配置文件设置网络参数）：

```
[root@lvs_1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
```

```
IPADDR=192.168.0.200
PREFIX=24
GATEWAY=192.168.0.254
TYPE="Ethernet"
```

```
[root@lvs_1 ~]# service network restart
```

在调度器主机上还需要安装部署 keepalived，具体的安装说明可以参考 8.2.3 节的内容。

```
[root@lvs_1 ~]# yum -y install gcc openssl-devel popt-devel \
>libnl libnl-devel kernel-devel ipvsadm
[root@lvs_1 ~]# http://www.keepalived.org/software/keepalived-1.2.7.tar.gz
[root@lvs_1 ~]# tar -xzf keepalived-1.2.7.tar.gz -C /usr/src/
[root@lvs_1 ~]# cd /usr/src/ keepalived-1.2.7/
[root@lvs_1 ~]#./configure
[root@lvs_1 ~]# make && make install
[root@lvs_1 ~]# ln -s /usr/local/etc/keepalived/ /etc
[root@lvs_1 ~]# ln -s /usr/local/etc/rc.d/init.d/keepalived /etc/init.d/
[root@lvs_1 ~]# ln -s /usr/local/etc/sysconfig/keepalived /etc/sysconfig/
[root@lvs_1 ~]# ln -s /usr/local/sbin/keepalived /usr/sbin/
```

为了通过 keepalived 实现服务的高可用，我们需要修改 keepalived 主配置文件，本例将 lvs_1 初始化设置为主设备，lvs_2 设置为从设备，并确保 lvs_1 与 lvs_2 这两台主机可以相互检查心跳，保证当其中一台调度器设备出现故障后，另一台设备可以及时、自动激活替换损坏的故障设备。此外，还需要在该配置文件中添加虚拟服务，并为该虚拟服务添加后端真实 Web 服务器组和相应的调度算法。该文件的具体配置选项的含义及说明见表 8-3。

```
[root@lvs_1 ~]# vim /etc/keepalived/keepalived.conf
```

```
! Configuration File for keepalived

global_defs {
   notification_email {
test@gmail.com
   }
   notification_email_from root@localhost
   smtp_server 127.0.0.1
   smtp_connect_timeout 30
   router_id lvs_1
}

vrrp_instance LVS_HA {
    state MASTER          #lvs_2 为 BACKUP
    interface eth0
    virtual_router_id 60        #lvs_2 的 VRID 必须与 lvs_1 相同
    priority 100                #在相同的 VRID 组中优先级高者为主设备
```

```
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1111
    }
    virtual_ipaddress {
192.168.0.253/24
    }
}

virtual_server 192.168.0.253 80 {
    delay_loop 6
    lb_algo rr
    lb_kind DR
    nat_mask 255.255.255.0
    persistence_timeout 50
    protocol TCP

    real_server 192.168.0.1 80 {
        weight 1
TCP_CHECK {
    connect_timeout 20
    connect_port 80
    nb_get_retry 3
}
    }
    real_server 192.168.0.2 80 {
        weight 1
        TCP_CHECK {
                connect_timeout 20
                connect_port 80
                nb_get_retry 3
        }
    }
}
```

```
[root@lvs_1 ~]# service keepalived start
[root@lvs_1 ~]# chkconfig keepalived on
[root@lvs_1 ~]# ip addr show                    #查看虚拟 IP
[root@lvs_1 ~]# ipvsadm -Ln                     #查看 LVS 规则
[root@lvs_1 ~]# iptables -F
[root@lvs_1 ~]# iptables -X
[root@lvs_1 ~]# service iptables save
```

部署 lvs_2 调度服务器。因为都是调度器设备，所以整体操作流程与 lvs_1 调度器一致，不同于 lvs_1 的是初始状态 lvs_2 设备主机为从备用设备，具体操作如下。

```
[root@lvs_2 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.201
PREFIX=24
GATEWAY=192.168.0.254
TYPE="Ethernet"
```

```
[root@lvs_1 ~]# service network restart
[root@lvs_2 ~]# yum -y install gcc openssl-devel popt-devel \
>libnl libnl-devel kernel-devel ipvsadm
[root@lvs_2 ~]# http://www.keepalived.org/software/keepalived-1.2.7.tar.gz
[root@lvs_2 ~]# tar -xzf keepalived-1.2.7.tar.gz -C /usr/src/
[root@lvs_2 ~]# cd /usr/src/ keepalived-1.2.7/
[root@lvs_2 ~]#./configure
[root@lvs_2 ~]# make && make install
[root@lvs_2 ~]# ln -s /usr/local/etc/keepalived/ /etc
[root@lvs_2 ~]# ln -s /usr/local/etc/rc.d/init.d/keepalived /etc/init.d/
[root@lvs_2 ~]# ln -s /usr/local/etc/sysconfig/keepalived /etc/sysconfig/
[root@lvs_2 ~]# ln -s /usr/local/sbin/keepalived /usr/sbin/
[root@lvs_2 ~]# vim /etc/keepalived/keepalived.conf
```

```
! Configuration File for keepalived

global_defs {
   notification_email {
test@gmail.com
   }
   notification_email_from root@localhost
   smtp_server 127.0.0.1
   smtp_connect_timeout 30
   router_id lvs_2
}

vrrp_instance LVS_HA {
    state BACKUP
    interface eth0
    virtual_router_id 60
    priority 50
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1111
    }
    virtual_ipaddress {
```

```
192.168.0.253/24
    }
}

virtual_server 192.168.0.253 80 {
    delay_loop 6
    lb_algo rr
    lb_kind DR
    nat_mask 255.255.255.0
    persistence_timeout 50
    protocol TCP

    real_server 192.168.0.1 80 {
        weight 1
TCP_CHECK {
    connect_timeout 20
    connect_port 80
    nb_get_retry 3
}
    }
    real_server 192.168.0.2 80 {
        weight 1
        TCP_CHECK {
                connect_timeout 20
                connect_port 80
                nb_get_retry 3
        }
    }
}
```

如图 8-5 所示，我们的案例环境中负载内外网通信的中间设备是一台路由设备，本书案例采用的是 Linux 软路由作为演示主机，实际的生产环境会是真实的硬件路由设备，Router 路由操作步骤如下：

```
[root@router ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE=eth0
ONBOOT=yes
BOOTPROTO=static
IPADDR=192.168.0.254
PREFIX=24
DNS1=202.106.0.20
```

```
[root@router ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth1
```

```
DEVICE=eth1
ONBOOT=yes
```

```
BOOTPROTO=static
IPADDR=124.126.147.168
PREFIX=8
DNS1=202.106.0.20
```

```
[root@router ~]# service network restart
[root@router ~]# iptables -F
[root@router ~]# iptables -X
[root@router ~]# iptables -t nat -F
[root@router ~]# iptables -t nat -X
```

因为使用的是 Linux 软路由，所以下面需要使用 iptables 实现 NAT 地址转发功能，实现案例拓扑结构中内部与外部网络的数据通信。

```
[root@router ~]# iptables -t nat -I PREROUTING -d 124.126.147.168 -p tcp -dport 80 \
>-j DNAT --to-destination 192.168.0.253:80
[root@router ~]# iptables -t nat -I POSTROUTING -s 192.168.0.0/24 -p tcp \
>-j SNAT --to-source 124.126.147.168
[root@router ~]# service iptables save
[root@router ~]# vim /etc/sysctl.conf
```

```
net.ipv4.ip_forward = 1
```

```
[root@router ~]# sysctl -p
```

客户端验证如下：

不同的客户端访问 http://124.126.147.168，将得到来自不同 Real Server 的响应数据包。

8.2.6 常见问题分析

（1）如何激活并检查 IPVS 内核模块。

```
[root@lvs ~]# modprobe ip_vs                    #加载 ip_vs 模块
[root@lvs ~]# lsmod | grep ip_vs                #显示当前加载的模块并过滤 ip_vs
[root@lvs ~]# cat /proc/net/ip_vs               #查看 IPVS 版本
```

（2）如何查看 LVS 统计信息。

```
[root@lvs ~]# ipvsadm -Lnc
```

（3）Keepalived 支持哪些监控检查。

Keepalived 支持对真实服务器的健康检查功能，其中包括 HTTP_GET、SSL_GET、TCP_CHECK、SMTP_CHECK、MISC_CHECK。

（4）使用 HTTP_GET 或 SSL_GET 进行健康检查时，如何设置 digest 值。

```
[root@lvs ~]# genhash -s 192.168.0.1 -p 80 -u index.html
```

（5）如何设置当客户端访问服务器 VIP 的任意端口都可以被调度至后端 Real Server：

```
[root@lvs ~]# ipvsadm -A -t 192.168.0.254:0 -s rr
```

（6）对客户端进行大量压力测试的软件如下。

Apache ab、httperf、jmeter 等软件都可以进行压力测试。

8.3　Squid 代理服务器

8.3.1　Squid 简介

Squid 是一个支持 HTTP、HTTPS、FTP 等服务的 Web 缓存代理软件，它可以通过缓存页面实现降低带宽占用并优化页面响应时间。此外，Squid 还具有强大的访问控制功能。Squid 可以运行在各种系统平台上，包括 Windows。Squid 会将页面缓存在内存及硬盘中，所以 Squid 对内存及硬盘要求比较高，更大的内存及硬盘意味着更多的缓存及更高的缓存命中率。Squid 确保返回给用户的数据不是过时的数据，所以我们需要根据实际情况不定期地清空缓存数据。

Squid 代码请求流程为：客户端访问 Squid 代理服务器，由代理服务器代表客户访问后端 Real Server（真实服务器），Real Server 将响应数据返回给代理服务器，代理服务器将响应数据返回给客户端，同时将页面缓存在本地内存及硬盘中，当下次客户端再有相同的数据请求时，代理服务器将直接从本地缓存中提取数据返回给客户端。

8.3.2　安装 Squid 服务

Squid 在大多数操作系统平台下都有特定的二进制软件包，采用二进制软件包安装是部署及运行 Squid 最快速的方式，每种操作系统都有自己特定的软件包管理器，我们可以使用系统自带的软件包管理器来维护二进制软件包。另外，Squid 还为我们提供了源码包，这种方式允许以自定义的方式安装 Squid。Squid 的官方网站（http://www.squid-cache.org/Versions/）提供了所有二进制数据及源码包的下载链接。CentOS 6.3 系统的安装光盘中也自带了 Squid 的 RPM 格式软件包，这里使用 yum 方式安装该软件。

```
[root@squid ~]# yum -y install squid
[root@squid ~]# service squid start
```

8.3.3 常见代理服务器类型

代理服务器有多种类型，一般意义上常分为标准正向代理服务器、透明代理服务器、反向代理服务器。

正向代理服务器主要应用于内部网络希望访问外部网络时缓存页面数据，正向代理结构如图 8-6 所示，由于目前 IP 地址稀缺，企业内部成百上千台计算机不可能同时都直接连接到 Internet 上，目前的解决方案是通过一个统一的网络接口连接 Internet。而 Squid 就可以提供这么一个连接 Internet 的接口。所有的客户端通过设置代理服务器连接至 Squid，通过 Squid 代理上网。在此模型下，Squid 主要负责提供缓存加速及访问控制的功能。正向代理是对用户的一种代理操作。

透明代理与传统的正向代理类似，区别在于传统的正向代理需要每个客户端进行代理服务器的设置，而透明代理对用户是透明的，不需要终端用户进行特殊设置。透明代理需要结合网关进行部署，所有的操作均由管理员在网关服务器及代理服务器进行设置，这些对用户是透明的。

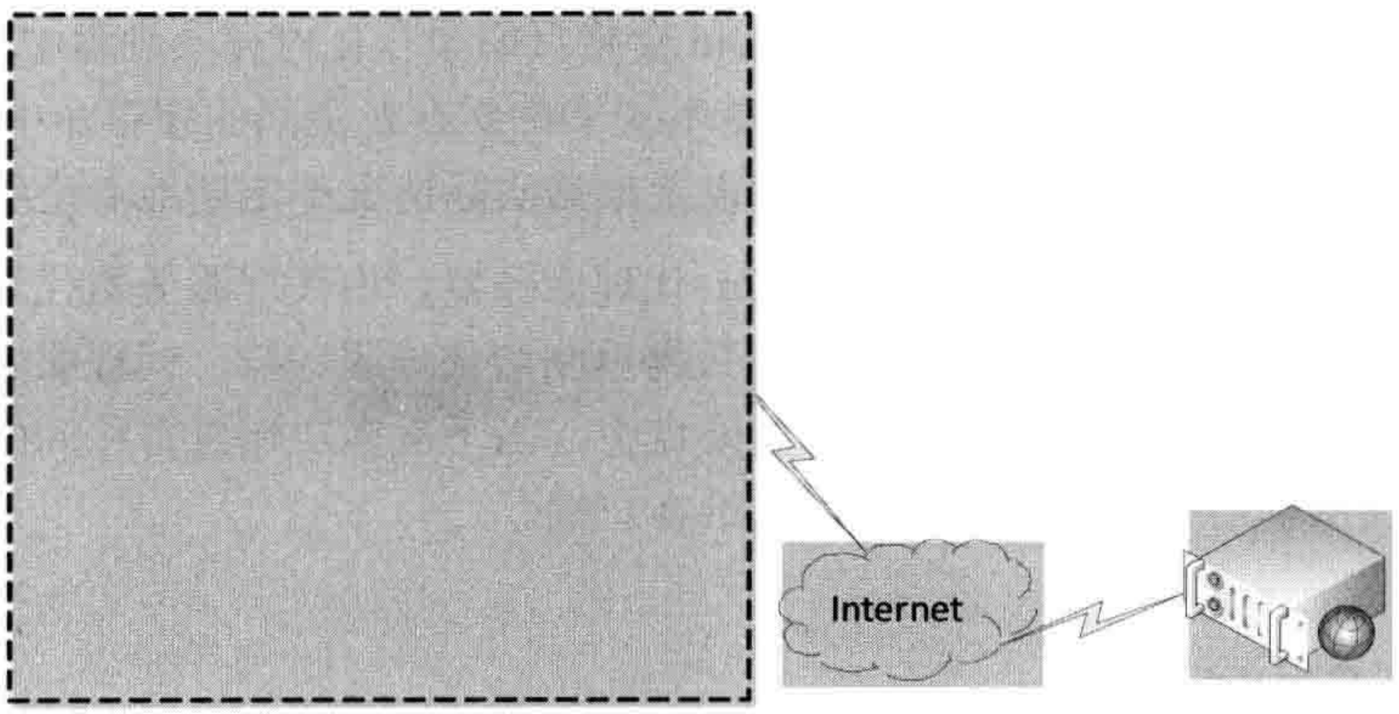


图 8-6

反向代理结构如图 8-7 所示，反向代理结合智能 DNS 即可以实现基本的 CDN（内容分发网络）框架。通过 DNS 的视图功能，我们可以为来自不同地区的 DNS 请求解析不同的结果返回给客户端，例如，当北京地区的客户请求 DNS 解析时，DNS 服务器将解析的结果指向北京地区的 Squid 代理服务器，上海地区的用户请求 DNS 解析的结果则是上海地区的 Squid 代理服务器。而此时的代理服务器则代表后端真实服务器，用户感觉不到自己是在访问代理服务器，而且由于使用了缓存技术，当大的并发访问量到达时，Squid 也可以更快地为用户返回响应数据。

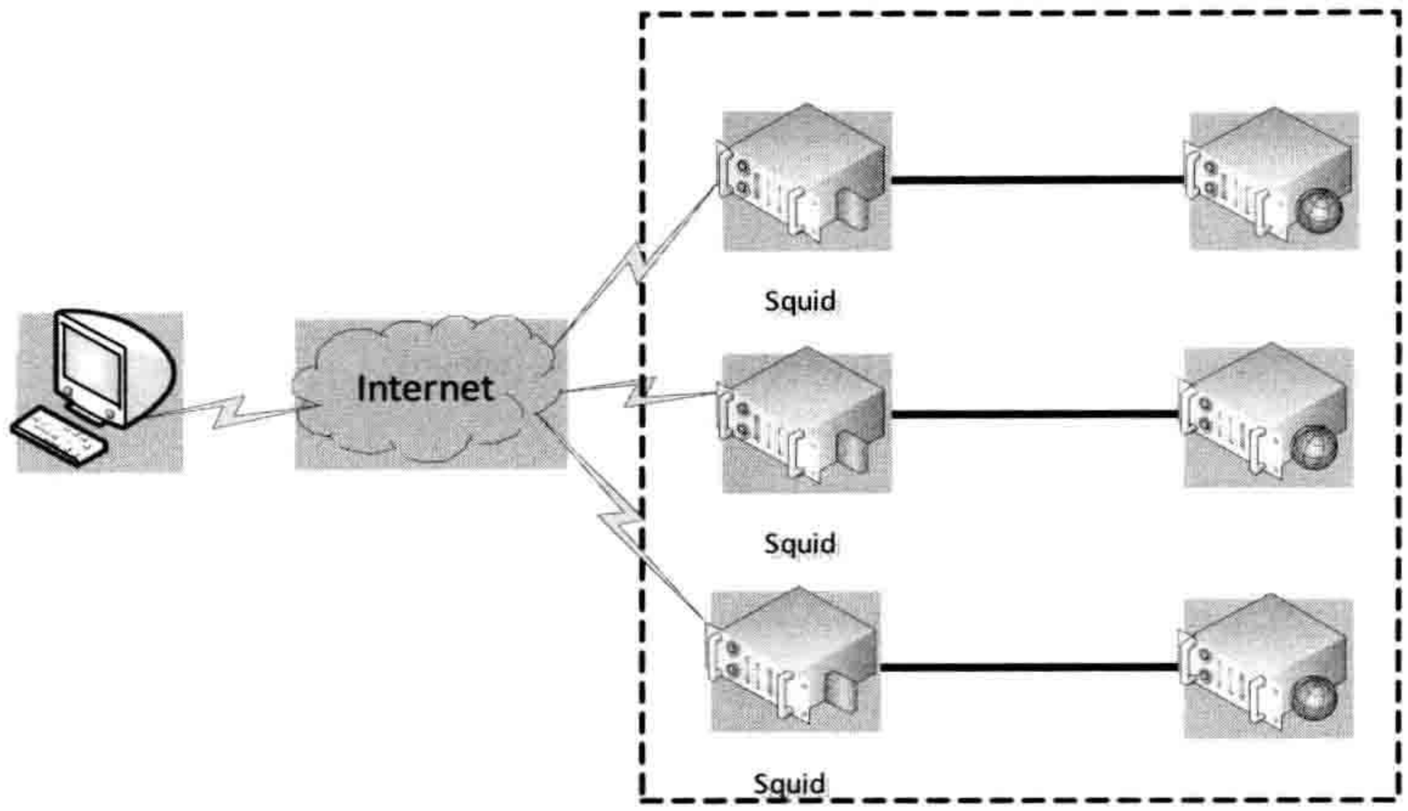


图 8-7

8.3.4　配置文件解析

使用 YUM 方式安装 Squid 时，默认的配置文件为/etc/squid/squid.conf。该配置文件由配置指令及配置选项构成，配置文件使用#符号作为注释。Squid 支持的功能丰富，配置语句也非常多，表 8-5 给出了 Squid 主要的配置语句及相关说明。

表 8-5

配置项	描述
http_port	设置 Squid 监听的网络端口，默认为 3128 端口。使用低于 1024 端口时，需要使用 root 启动 Squid，ISP 一般使用 8080 端口。 vhost 参数实现反向代理功能
cache_dir	设置缓存存储位置及大小，配置文件中可以使用多个 cache_dir 设置多个存储位置。 cache_dir ufs /usr/local/squid/var/cache 100 16 256 缓存路径为/usr/local/squid/var/cache，100 代表存储容量，单位为 MB，16 及 256 代表子目录个数，用于分类分层管理缓存数据
access_log	设置日志路径及日志格式，如 access_log /var/log/squid/access.log combined
acl	定义访问控制列表（ACL）对象，可以控制源地址、端口、时间等，如 acl localnet src 192.168.0.0/255.255.255.0，设置名称为 loclnet 的 ACL 列表，控制对象为源地址网段 192.168.0.0/24
http_access	访问控制，允许使用 ACL 指令定义的 localnet 效果如下： http_access allow localnet

续表

配置项	描述
cache_peer	Squid 支持在一组代理之间进行缓存请求转发，组内的成员代理之间称为邻居，邻居关系分为：父子和兄弟。 cache_peer 用来设置可以连接的其他代理服务器，其格式如下： cache_peer hostname type http-port icp-port [option] hostname 为邻居的主机名或 IP； type 类型为父亲（parent）、兄弟（sibling）、广播（multicast）； http-port 指定邻居 http 端口号； icp-port 指定 ICP 端口号； option（选项）有以下可选值： proxy-only 实现不缓存来自邻居的任何响应 weight=n 设置服务器权重 ttl=n 该选项仅对广播邻居有效，设置广播 TTL 值 round-robin 仅在设置两个以上父 Cache 才有效，可以实现轮询的负载均衡 max-conn=n 设置连接邻居 Cache 的并发量 originserver 设置邻居为源真实服务器，做反向代理使用
visible_hostname	设置主机名，默认 Squid 将自动检测系统主机名

8.3.5 Squid 应用案例

配置 Squid 实现反向代理，结构如图 8-8 所示。终端用户请求 Web 服务时，DNS 将请求的域名解析为反向代理服务器的 IP 地址，代理服务器首先检查本地是否有缓存，如果有则直接将响应数据包返回给客户端，否则代理服务器转发客户请求给后端真实源服务器。服务器网络配置见表 8-6。本例中，我们利用 Squid 强大的代理缓存技术来提高并发访问的快速响应，通过缓存可以更快地为客户端提供服务。如果图 8-8 的拓扑被部署在全国范围内，使用多个反向代理，再结合 DNS 服务器的视图功能（智能 DNS），就可以为企业提供强大而稳定的 CDN 系统。

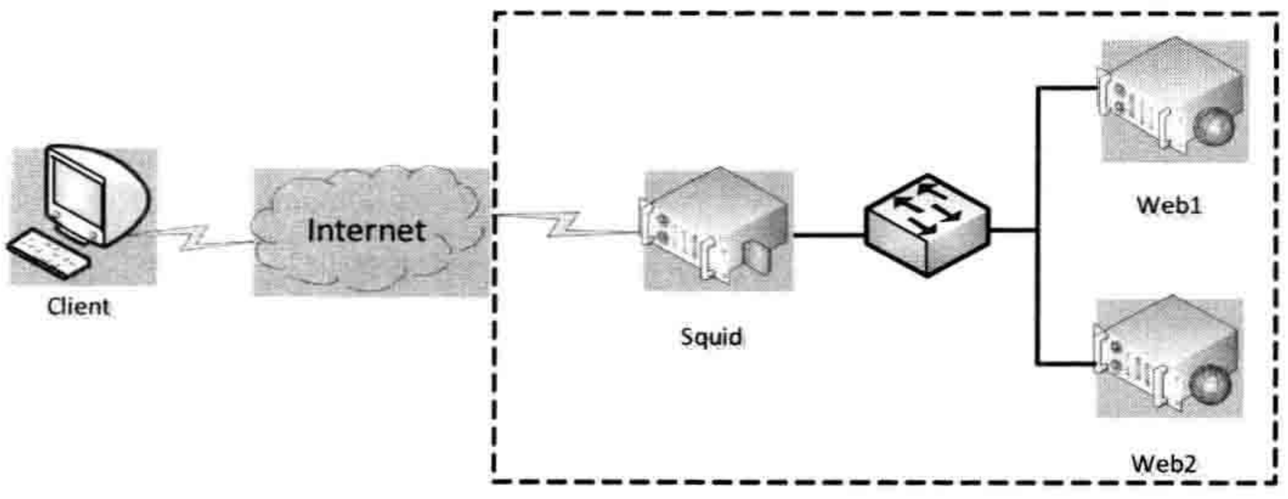


图 8-8

Squid 一般仅缓存静态页面，比如 HTML 网页、图片、歌曲等，默认不缓存 CGI 脚本程序或者 PHP、JSP。Squid 可以根据 HTTP 头信息来决定如何缓存数据，以及缓存周期，如 Expires（缓存有效期）、Cache-Control（是否进行缓冲）。

表 8-6

服务器名称	网络配置
squid.example.com	eth0:10.10.10.10
	eth1:192.168.0.254
web1.example.com	eth0:192.168.0.1
web2.example.com	eth0:192.168.0.2
client.example.com	eth0:10.10.10.100

1. 代理服务器的设置

首先，需要配置网络参数环境。

```
[root@squid ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=10.10.10.10
PREFIX=8
TYPE="Ethernet"
```

```
[root@squid ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth1
```

```
DEVICE="eth1"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.254
PREFIX=24
TYPE="Ethernet"
```

```
[root@squid ~]# service network restart
```

其次，还需要安装部署 Squid 代理服务。Squid 可以使用光盘中自动的 RPM 格式软件包，也可以使用源码软件包安装。下面我们使用的是简单方便的 RPM 软件包，安装完成后需要修改 Squid 配置文件，具体配置选项的含义如下。

```
[root@squid ~]# yum -y install squid
[root@squid ~]# vim /etc/squid/squid.conf
```

```
acl manager proto cache_object                    #定义缓存管理 ACL
```

```
    acl localhost src 127.0.0.1/32 ::1          #定义源地址为本地回环地址的 ACL
    acl to_localhost dst 127.0.0.0/8 0.0.0.0/32 ::1    #定义目标地址的 ACL
    acl localnet src 10.0.0.0/8                 #定义源地址为 10.0.0.0/8 的 ACL
    acl localnet src 172.16.0.0/12              #定义源地址 ACL
    acl localnet src 192.168.0.0/16
    acl localnet src fc00::/7
    acl localnet src fe80::/10
    acl SSL_ports port 443                      #定义安全端口为 443 的 ACL
    acl Safe_ports port 80                      #定义安全端口为 80 的 ACL
    acl Safe_ports port 21                      #定义安全端口为 21 的 ACL
    acl Safe_ports port 443     # https
    acl Safe_ports port 70      # gopher
    acl Safe_ports port 210     # wais
    acl Safe_ports port 1025-65535 # unregistered ports
    acl Safe_ports port 280     # http-mgmt
    acl Safe_ports port 488     # gss-http
    acl Safe_ports port 591     # filemaker
    acl Safe_ports port 777     # multiling http
    acl CONNECT method CONNECT                  #定义连接方式为 CONNECT 的 ACL
    http_access allow manager localhost         #仅允许本机主机进行缓存管理
    http_access deny manager                    #拒绝其他主机的所有缓存管理
    #拒绝所有非 Safe_ports 的连接，Safe_ports 为配置文件中所定义的若干端口
    http_access deny !Safe_ports
    http_access deny CONNECT !SSL_ports
    http_access allow localnet
    http_access allow localhost
    http_access allow all                       #允许所有
    visible_hostname squid.example.com          #设置主机名称
    cache_mem 2048 MB                           #内存缓存总容量
    #内存可以缓存的单个文件最大容量为 4MB
    maximum_object_size_in_memory 4096 KB
    #磁盘可以缓存的单个文件最大容量为 4MB
    maximum_object_size 4096 KB
    #cache_dir 定义硬盘缓冲目录为/var/spool/squid，缓存最大容量为 800MB，Squid 将在缓存
    #目录下创建 16 个一级子目录和 256 个二级子目录，这些目录用来分类管理缓存数据
    cache_dir ufs /var/spool/squid 800 16 256
    error_directory /usr/share/squid/errors/zh-cn  #定义报错文件的存放目录
    cache_log /var/log/squid/cache.log             #缓存日志文件
    cache_mgr admin@test.com                       #管理员邮箱
    #代理服务器监听的端口，accel 设置 squid 为加速模式，vhost 实现反向代理
    http_port 80    accel vhost
    #同时监听代理的 3128 端口，方便使用 squidclient 工具对缓存进行管理
    http_port 3128
    #配置后台源服务器，originserver 模拟本机服务器为源服务器；80 为 HTTP 端口；
```

```
    #设置 ICP 端口为 0（源服务器不支持 ICP 查询）；no-query 表示禁止使用 ICP 对源服务器进
    #行查询；round-robin 可以让代理轮询多台源服务器， 可以使用 weigh=N 为源服务器指定
    #权重；name 用来设置源服务器的唯一名称，代理转发数据至相同主机的不同端口非常有用
    cache_peer 192.168.0.1 parent 80 0 no-query originserver round-robin
name=server1
    cache_peer 192.168.0.2 parent 80 0 no-query originserver round-robin
name=server1
    hierarchy_stoplist cgi-bin ?                                    #禁止缓存 CGI 脚本
    #使用 refresh_pattern 应用在没有过期时间的数据，squid 顺序检查 refresh_pattern 并
    #确定数据是否过期，格式如下：
    #refresh_pattern [-i] regexp min percent max [option]
    #refresh_pattern 使用正则表达式匹配数据对象（-i 选项表示不区分大小写）
    #min 为过期的最低时间（单位为分钟），即对象在缓存中至少要存放的时间
    #max 为最大时间
    #percent 为百分比，如果 squid 最后修改系数算法低于该百分比则数据不过期
    refresh_pattern ^ftp:       1440    20% 10080
    refresh_pattern ^gopher:    1440    0%  1440
    refresh_pattern -i (/cgi-bin/|\?) 0     0%  0
    refresh_pattern .   0   20% 4320
```

```
[root@squid ~]# service iptables stop
[root@squid ~]# service squid start
[root@squid ~]# chkconfig squid on
```

2. 两台源服务器的设置

设置正确的网络参数，安装并启动 httpd 服务。

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.1
PREFIX=24
TYPE="Ethernet"
```

```
[root@web1 ~]# service network restart
[root@web1 ~]# service iptables stop
[root@web1 ~]# chkconfig iptables off
[root@web1 ~]# yum -y install httpd
[root@web1 ~]# echo "192.168.0.1" > /var/www/html/index.html
[root@web1 ~]# service httpd start
[root@web1 ~]# chkconfig httpd on
[root@web2 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.2
PREFIX=24
TYPE="Ethernet"
```

```
[root@web2 ~]# service network restart
[root@web2 ~]# service iptables stop
[root@web2 ~]# chkconfig iptables off
[root@web2 ~]# yum -y install httpd
[root@web2 ~]# echo "192.168.0.2" > /var/www/html/index.html
[root@web2 ~]# service httpd start
[root@web2 ~]# chkconfig httpd on
```

3. 客户端的验证

客户端配置正确的网络参数后，通过浏览器访问代理服务器：http://10.10.10.10。由于我们在代理服务器中设置了两台源服务器，并且使用的是轮询查询，所以客户端刷新后可以轮询查看到两个不同的页面。

8.4 HAProxy 负载均衡

8.4.1 HAProxy 简介

HAProxy 是免费、高效、可靠的高可用及负载均衡解决方案。该软件非常适合于处理高负载站点的七层数据请求。HAProxy 的工作模式使其可以非常容易且安全地集成到我们现有的站点架构中。使用类似的代理软件还可以对外屏蔽内部的真实 Web 服务器，防止内部服务器遭受外部攻击。HAProxy 拓扑结构如图 8-9 所示，终端用户通过访问 HAProxy 代理

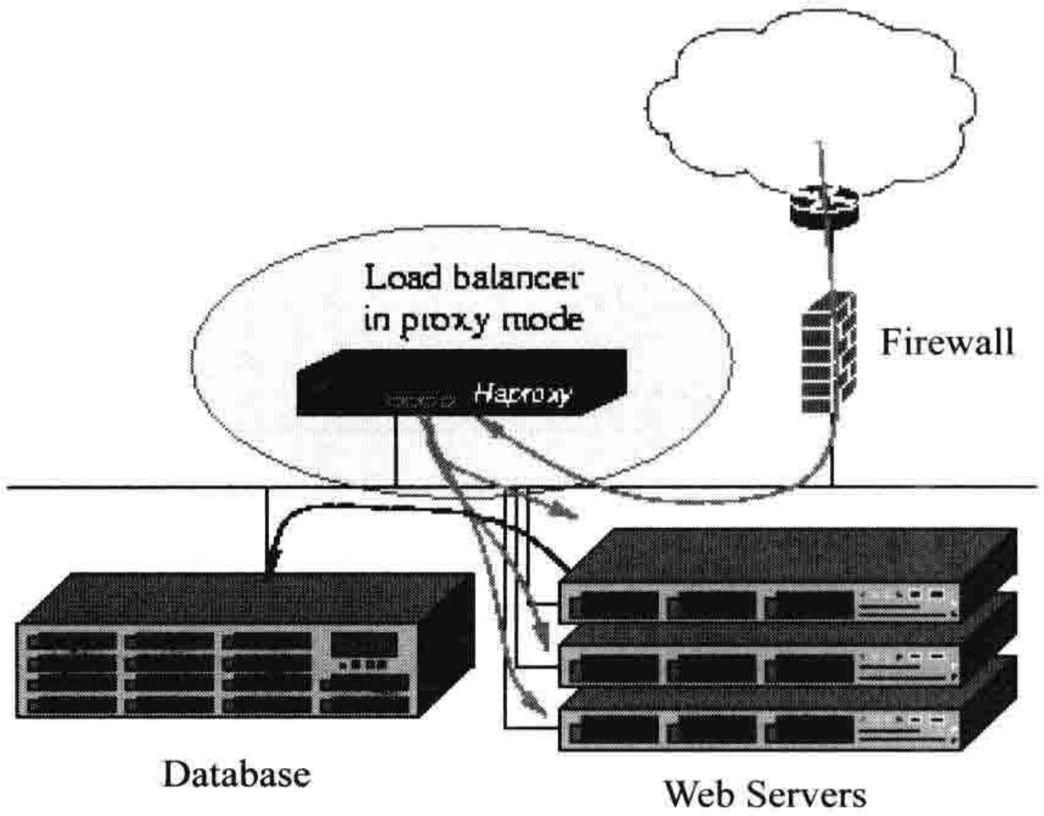


图 8-9

服务器获得站点页面，而代理服务器在收到客户端请求后会根据自身规则将数据请求转发给后端真实服务器。为了让同一客户端访问服务器时可以保持会话（同一客户端第二次访问网站时可以被转发至相同的后端真实服务器），HAProxy 有三种解决方案：客户端 IP、Cookie 以及 Session。第一种方式中，HAProxy 将客户端 IP 进行 Hash 计算并保存，以此确保当相同的 IP 访问代理服务器时可以转发到固定的真实服务器上。第二种方式中，HAProxy 依靠真实服务器发送给客户端的 Cookie 信息进行会话保持。第三种方式中，HAProxy 将保存真实服务器的 Session 及服务器标识，实现会话保持功能。

HAProxy 软件包可以在其官方网站上下载。下面通过源码方式安装该软件，在使用 make 命令生成 makefile 文件时，TARGET=linux2628 代表 Linux 操作系统使用的内核版本为 2.6.28 及以上版本。

```
[root@haproxy ~]# yum -y install gcc
[root@haproxy ~]# wget http://haproxy.1wt.eu/download/1.4/src/haproxy-1.4.
24.tar.gz
[root@haproxy ~]# tar -xzf haproxy-1.4.24.tar.gz -C /usr/src/
[root@haproxy ~]# cd /usr/src/haproxy-1.4.24/
[root@haproxy ~]# make TARGET=linux2628
[root@haproxy ~]# make install
```

8.4.2　配置文件解析

HAProxy 安装后没有默认的配置文件，需要手动创建。本例将创建/etc/haproxy.cfg 配置文件，当启动 HAProxy 服务时需要使用-f 选项指定配置文件路径。HAProxy 配置文件主要包含全局设置段与代理段，global 代表全局段，defaults、listen、frontend、backend 为代理段。frontend 用来匹配客户端请求的域名或 URI 等，backend 定义后端服务器集群，listen 是 frontend 与 backend 的集合，有时仅使用 listen 即可替代 frontend 与 backend。下面给出了一个监听 80 端口的 HTTP 代理案例，代理服务器将转发数据请求到单一后台服务器 127.0.0.1:8000。

```
global
        daemon
        maxconn 256
defaults
        mode http
        timeout connect 5000ms
        timeout client 50000ms
        timeout server 50000ms
frontend http-in
        bind *:80
        default_backend servers
```

```
backend servers
        server server1 127.0.0.1:8000 maxconn 32
```

表 8-7 列出了 HAProxy 主配置文件的具体语法格式及描述。

表 8-7

配置项		描述
global	chroot <jail dir>	将工作目录切换至<jail dir>并执行 chroot。该配置可增强 HAProxy 的安全性，但需要使用超级账户启动 HAProxy 程序
	daemon	配置 HAProxy 以后台进程模式工作
	uid <number>	配置进程的账户 ID，建议设置为 HAProxy 专用账户
	gid <unmber>	配置进程的组 ID，建议设置为 HAProxy 专用组
	log <address> <facility>	配置全局 syslog 服务器，可以设置两台日志服务器
	nbproc <number>	指定后台进程的数量
	pidfile <pidfile>	将进程 ID 号写入<pidfile>文件
	ulimit-n <number>	设置每个进程的最大文件描述符数量
	maxconn <number>	设置每个进程支持的最大并发数
	tune.bufsize <number>	设置 buffer 大小，默认值为 16384，单位为字节（B）
代理设置	mode	HAProxy 工作模式，可选项为：tcp、http、health
	timeout check <timeout>	设置检查超时时间
	contimeout <timeout>	设置连接超时时间
	balance roundrobin	默认负载均衡工作模式，轮询
	bind <address>:<port>	定义一个或多个监听地址或端口
	stats auth admin:admin	设置监控界面的用户名称与密码
	stats refresh <number>	统计页面刷新间隔时间
	option httplog	使用 http 日志
	cookie <name>	启用基于 cookie 的保持连接功能
	option forwardfor	允许插入 X-Forwarded-For 数据包头给后端真实服务器，可以让后端服务器获得客户端的真实 IP 地址
	option abortonclose	服务器负载高时，自动关闭队列里处理时间比较长的连接请求
	option allbackups	当后端服务器全部宕机时，是否激活所有备用的服务器，默认仅启动第一个备用服务器
	option dontlognull	不记录空连接日志，主要用于不记录健康检查日志

续表

配置项	描述
option redispatch	在 HTTP 模式，如果使用 cookie 的服务器宕机，客户端还会坚持连接它，该选项在后端服务器宕机时强制将请求转发给其他健康主机
monitor-uri <uri>	检查<uri>文件是否存在，依次判断主机健康状态
monitor-fail if site_dead	当服务器宕机时，返回 503 错误代码，需要定义 ACL
option httpchk <uri>	使用 HTTP 协议检查服务器健康状态
retries <value>	服务器连接失败后的重试次数
timeout client <n>	设置客户端最大超时时间为 n，默认单位为毫秒（ms）
timeout server <n>	设置服务器端最大超时时间为 n，默认单位为毫秒（ms）
timeout connect <n>	设置连接最大超时时间为 n，默认单位为毫秒（ms）
default_backend	配置文件中没有 use_backend 规则时，设置默认后端服务器组，服务器组由 backend 定义
use_backend	当条件满足时，指定后端服务器组，需要设置 ACL
acl <name> <criterion>	定义访问控制列表，配置文件中通过 name 调用该 ACL，常用限制包括：dst（目标地址）、dst_port（目标端口）、src（源地址）、hdr（连接头部信息）、path_reg（访问路径匹配正则）、url（统一资源定位符）

ACL 访问控制列表案例如下：

```
global
        daemon
        maxconn 256
defaults
        mode http
        timeout connect 5000ms
        timeout client 50000ms
        timeout server 50000ms
frontend http-in
        bind *:80
#定义 ACL 控制源地址为 192.168.0.0/24 网段
acl badboys src 192.168.0.0/24
#定义 ACL 控制请求包头信息通过正则匹配，-i 代表不区分大小写
acl example_acl hdr_reg(host) -i ^(www.example.com|web.example.com)$
#当用户访问请求触发名为 badboys 的 ACL 规则时，禁止该客户的请求数据
block if badboys
#当用户访问请求触发名为 example_acl 的 ACL 规则时，代理会将请求转发为
#servers_2 后端服务器组
use_backend servers_2 if example_acl
```

```
    #定义没有 ACL 时默认使用的服务器组
          default_backend servers_1
    backend servers_1                          #定义后端服务器组 servers_1
          server server1 127.0.0.1:8000 maxconn 32    #定义服务器组中的具体服务器
    backend servers_2                          #定义后端服务器组 servers_2
          server server1 127.0.0.1:8080 maxconn 3
```

8.4.3 HAProxy 应用案例

本节将以生产环境为原型，简化网络拓扑后使用 HAProxy 实现图 8-10 所示的高性能代理服务架构，整个网络参数见表 8-8。本例将使用 listen 定义一个监控端口；使用 frontend 定义一个前端 80 端口；通过 backend 分别定义名为 inside_servers 和 external_servers 的服务器组；使用 default_backend 定义默认服务器组为 external_servers；定义 ACL 规则时，如果内网（192.168.0.0/24）访问 Web 服务，则由 inside_servers 服务器组提供 Web 页面。

external_servers 服务器组中包含 web1.example.com 和 web2.example.com 两台服务器，inside_servers 服务器组中仅包含 web3.example.com 一台服务器。

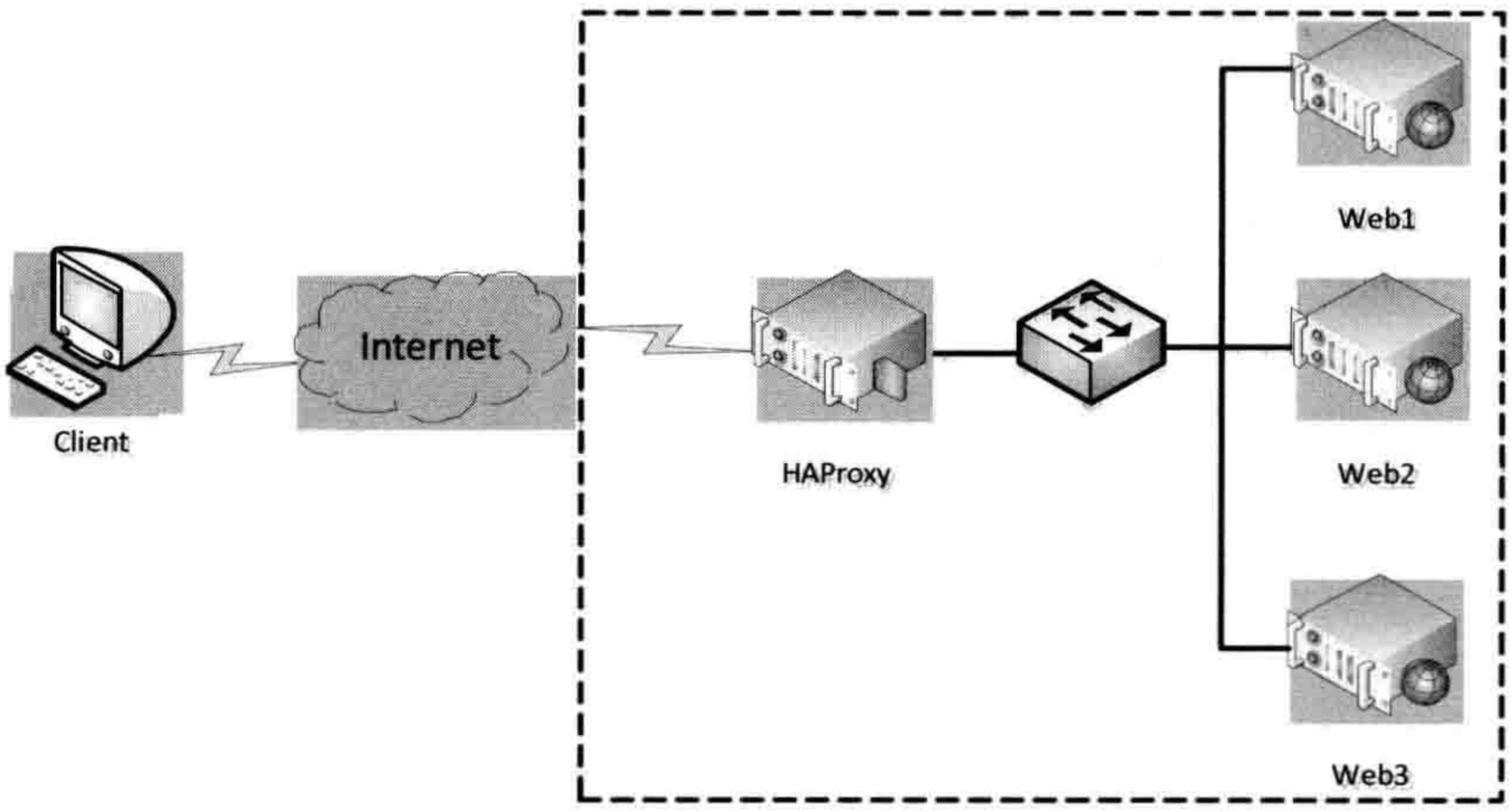


图 8-10

表 8-8

服务器名称	网络配置
haproxy.example.com	eth0:10.10.10.10
	eth1:192.168.0.254
web1.example.com	eth0:192.168.0.1

续表

服务器名称	网络配置
web2.example.com	eth0:192.168.0.2
web3.example.com	eth0:192.168.0.3
client.example.com	eth0:10.10.10.100

三台 Web 服务器基本采用相同的设置，下面仅以 web1 为例，为了演示 HAProxy 可以轮询访问后端服务器，我们将三台服务器的页面设置为不同的内容以示区别。

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.1
PREFIX=24
GATEWAY=192.168.0.254
TYPE="Ethernet"
```

```
[root@web1 ~]# service network restart
[root@web1 ~]# yum -y install httpd
[root@web1 ~]# echo "192.168.0.1" > /var/www/html/index.html
[root@web1 ~]# service iptables stop
```

HAProxy 代理服务器设置如下。

```
[root@haproxy ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=10.10.10.10
PREFIX=8
TYPE="Ethernet"
```

```
[root@haproxy ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth1
```

```
DEVICE="eth1"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.254
PREFIX=24
TYPE="Ethernet"
```

```
[root@haproxy ~]# service network restart
[root@haproxy ~]# service iptables stop
[root@haproxy ~]# vim /etc/security/limits.conf  #内核调优，文件中插入如下两行
```

```
*       soft    nofile      65535
*       hard    nofile      65535
```

```
[root@haproxy ~]# yum -y install gcc
[root@haproxy ~]# wget http://haproxy.1wt.eu/download/1.4/src/haproxy-1.
4.24.tar.gz
[root@haproxy ~]# tar -xzf haproxy-1.4.24.tar.gz -C /usr/src/
[root@haproxy ~]# cd /usr/src/haproxy-1.4.24/
[root@haproxy ~]# make TARGET=linux2628
[root@haproxy ~]# make install
[root@haproxy ~]# mkdir /var/haproxy                    #chroot 所需要目录
[root@haproxy ~]# vim /etc/haproxy.cfg                  #创建配置文件
```

```
global
maxconn 4096
log 127.0.0.1 local3 info
chroot /var/haproxy
uid 99
gid 99
daemon
nbproc 1
pidfile /var/run/haproxy.pid
ulimit-n 65535
stats socket /var/tmp/stats
defaults
log global
mode http
maxconn 20480
option httplog
option httpclose
option dontlognull
option forwardfor
option redispatch
option abortonclose
stats refresh 30
retries 3
balance roundrobin
cookie SRV
timeout check 2000ms
timeout connect 5000ms
timeout server 50000ms
timeout client 50000ms
listen admin_status                          #定义 HAProxy 的监控界面
bind 0.0.0.0:6553
mode http
log 127.0.0.1 local3 info
```

```
stats enable
stats refresh 5s                          #监控统计页面自动刷新时间为 5s
stats realm   Haproxy\ Statistics         #登录监控统计页面提示符
stats uri /admin?stats                    #监控页面 URL 路径
stats auth admin1:AdMiN123                #查看 HAProxy 监控页面的账户与密码
stats hide-version                        #隐藏 HAProxy 版本信息

frontend web_serivce                      #定义终端用户访问的前端服务器
bind 0.0.0.0:80
mode http
log global
option httplog
option httpclose
option forwardfor
acl inside_src src 192.168.0.0/24   #定义 ACL
#use_backend 调用 ACL 定义，如果源地址为 192.168.0.0/24，则代理服务器将会把请求
#转发给 inside_servers 服务器组
use_backend inside_servers if inside_src
default_backend external_servers
backend external_servers
mode http
balance roundrobin                                #轮询真实服务器
#检查真实服务器的 index.html 文件，以此判断服务器的健康状态
option httpchk GET /index.html
#定义后端真实服务器，向 cookie 信息中插入 web1 信息，check 代表允许对服务器进行
#健康检查，健康检查的时间间隔为 2000ms，连续两次健康检查成功则认为服务器是
#有效开启的，连续三次健康检查失败后，认为服务器已经宕机，服务器权重为 1
server web1 192.168.0.1:80 cookie web1 check inter 2000 rise 2 fall 3 weight 1
server web2 192.168.0.2:80 cookie web2 check inter 2000 rise 2 fall 3 weight 1
backend inside_servers
mode http
balance roundrobin
option httpchk GET /index.html
server web1 192.168.0.3:80 cookie web3 check inter 1500 rise 3 fall 3 weight 1
```

```
[root@haproxy ~]# vim /etc/rsyslog.conf          #修改日志配置文件，插入以下三行
```

```
$ModLoad imudp
$UDPServerRun 514
local3.*                                  /var/log/haproxy.log
```

```
[root@haproxy ~]# service rsyslog restart        #重启系统日志服务
[root@haproxy ~]# haproxy -f /etc/haproxy.cfg  #指定配置文件启动 HAProxy 服务
[root@haproxy ~]# echo "/usr/local/sbin/haproxy -f /etc/haproxy.cfg" >> /etc/rc.local
```

客户端验证如下：

首先为客户端主机配置正确的网络环境，确保客户端与 HAProxy 代理服务器可以直接连通。使用浏览器访问 http://10.10.10.10:6553/admin?stats 查看代理服务器状态统计信息，如图 8-11 所示。

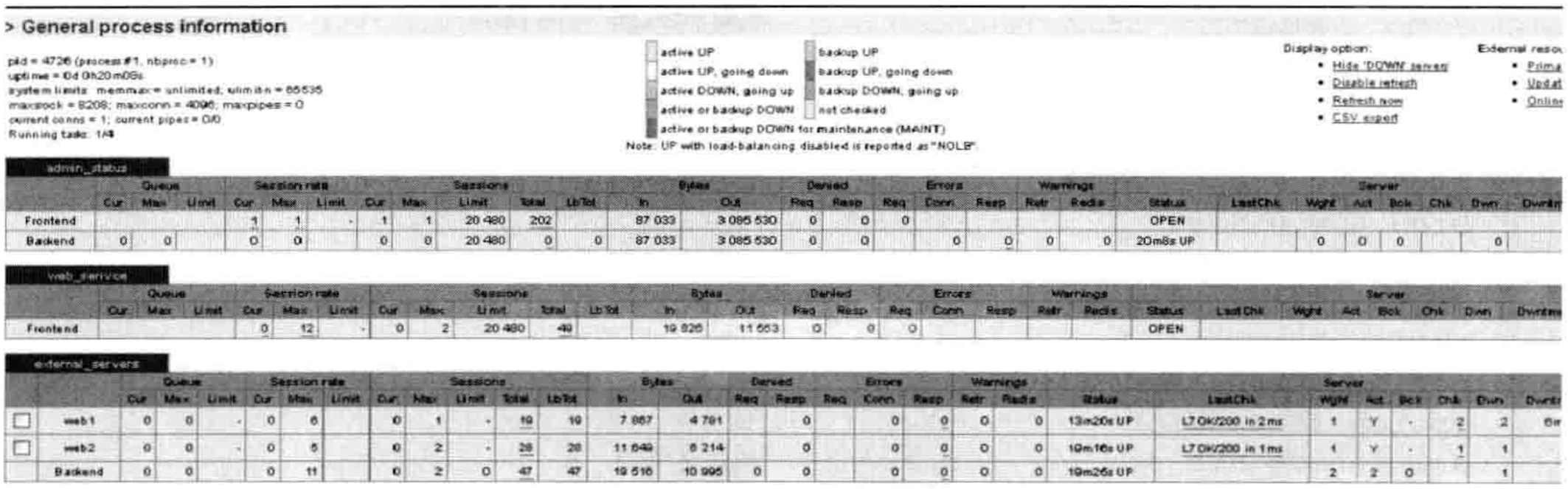


图 8-11

配置客户端主机 IP 地址为 10.10.10.100，通过浏览器访问 http://10.10.10.10，刷新将分别得到 web1.example.com 以及 web2.example.com 两台主机返回的页面信息。如果客户端主机 IP 地址配置为 192.168.0.0/24 网络内的 IP，则访问 http://192.168.0.254，服务器返回的页面将永远都是 web3.example.com 主机的页面信息。

8.5 Nginx 高级应用

Nginx 除了可以作为后端 HTTP 服务器外，它还是一个高性能的反向代理服务器。在负载均衡的架构中，Nginx 可以为我们提供非常稳定且高效的基于七层的负载解决方案。此外，Nginx 可以根据规则以轮询、IP 哈希、URL 哈希等方式调度后端真实服务器，如同 HAProxy 一样，Nginx 也支持对后端服务器的健康检查功能。目前国内的新浪、网易等公司均已部署了 Nginx 实现基于七层的负载均衡功能。

8.5.1 Nginx 负载均衡

为了实现 Nginx 的反向代理以及负载均衡功能，我们需要用到 HttpProxyModule 和 HttpUpstreamModule 模块。其中，HttpProxyModule 模块的作用是将用户的数据请求转发至其他服务器，HttpUpstreamModule 模块的作用是提供简单的负载均衡技术（轮询、最少连接、客户端 IP）。这两个模块为 Nginx 默认的自动编译模块，如果不需要编译这些模块，可以用

--without-http_proxy_module 与--without-http_upstream_ip_hash_module 两个编译参数禁用这些功能。

下面通过一个简单的例子说明 Nginx 作为反向代理服务器的配置方法。其中，location 指令可以根据 URI 定义不同的配置，比如，当用户访问 http://www.example.com/a 时，我们可以定义将数据请求转发给后端 A 服务器；当用户访问 http://www.example.com/b 时，我们可以定义将数据请求转发给后端 B 服务器。location 指令可以直接匹配字串，也可以匹配正则表达式，使用正则表达式时必须指定前缀~或~*，~表示区分大小写，~*代表不区分大小写，如果在匹配时希望达到精确匹配，则可以使用=前缀符号。使用 proxy_pass 指令可以根据 location 匹配的情况建立被代理的服务器或 URI 与代理服务器直接的映射关系。由于用户直接访问的是代理服务器，然后代理服务器负责转发数据请求给真实服务器。因此，后端真实服务器将无法获得客户端的 IP 地址，我们可以使用 X-Forwarded-For 实现重新定义数据包包头，记录用户真实的 IP 地址。

```
location ~* \.(mp3|mp4)$ {                    #匹配 URL 以 mp3 或 mp4 结尾的请求
  proxy_pass http://localhost:8080  #location 条件满足时将请求转发给本机 8080 端口
  }
location / {                                   #匹配任意 URL
proxy_pass  http://localhost:8000;  #location 条件满足时将请求转发给本机 8000 端口
    proxy_set_header  X-Forwarded-For  $remote_addr;  #保留用户真实的 IP 地址
}
```

在负载均衡模块的指令中，upstream 负责定义后端真实服务器集合，这样就可以通过 proxy_pass 和 fastcgi_pass 将请求转发给一组服务器，在 upstream 中嵌套使用 server 指令可以设置具体的服务器及相关参数。ip_hast 指令可以根据用户 IP 地址的 Hash 值分配固定的后端服务器。weight 的值代表服务器的权重，down 指令可以设置某台服务器暂时处于宕机不可用状态。简单的案例是：定义一个名为 backend 的服务器组，使用 server 定义组内所有的真实服务器成员，并将 www2.example.com 设置为宕机状态，当用户使用域名 http://www.example.com 访问代理服务器时，Nginx 代理服务器会将请求转发给 backend 服务器组。

```
http {
upstream backend {
ip_hash;
server www1.example.com weight=2;
server www2.example.com own;
server www3.example.com ;
}

server {
listen 80;
server_name www.example.com;
location / {
```

```
    proxy_pass http://backend;
    }
    }
    }
```

8.5.2 Nginx 负载均衡案例

Nginx 反向代理负载均衡拓扑结构如图 8-12 所示，服务器网络参数见表 8-9。案例中的 Nginx 使用 upstream 定义一组服务器，使用 proxy_pass 进行代理转发。

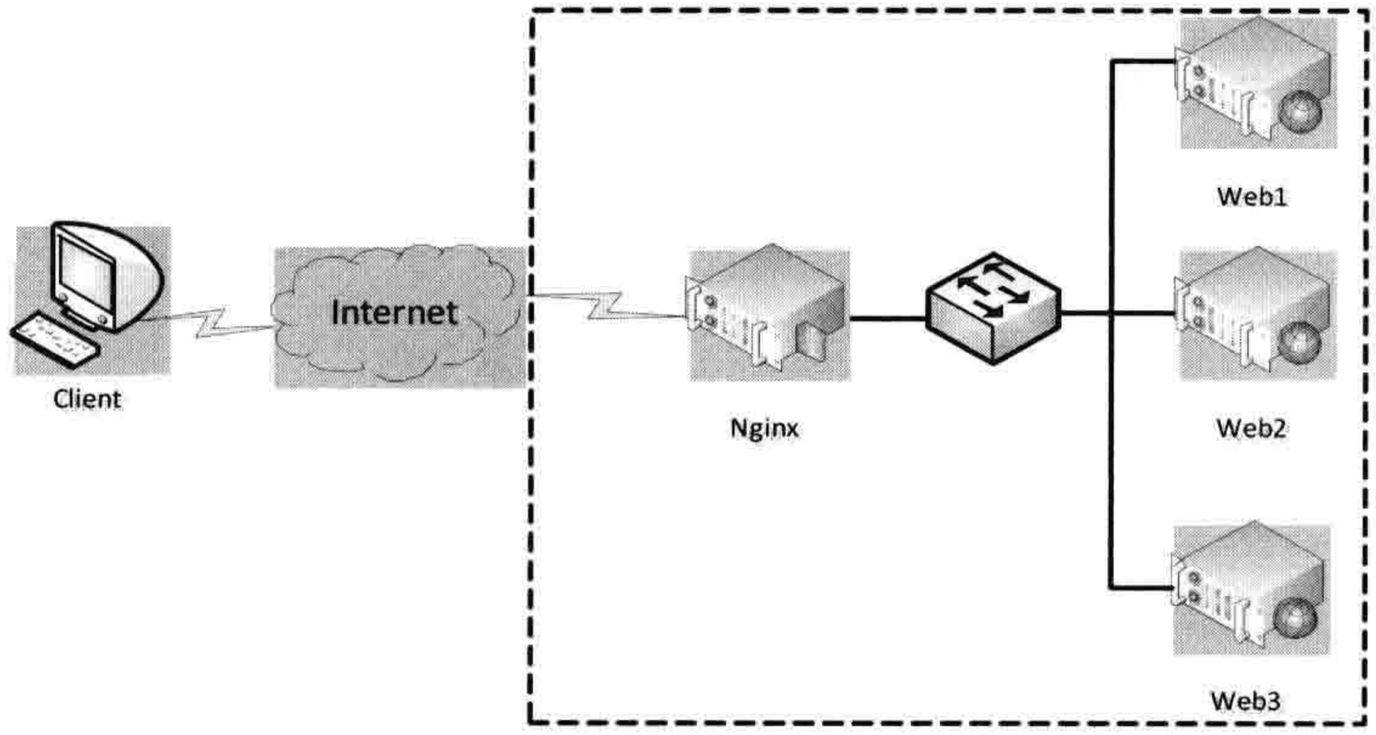


图 8-12

表 8-9

服务器名称	网络配置
nginx.example.com	eth0:10.10.10.10
	eth1:192.168.0.254
web1.example.com	eth0:192.168.0.1
web2.example.com	eth0:192.168.0.2
web3.example.com	eth0:192.168.0.3
client.example.com	eth0:10.10.10.100

三台 Web 服务器基本采用相同的设置，注意，IP 地址需要改变，下面仅以 web1 为例（为了演示效果，Web 服务器页面设置为不同的内容以示区别）。

```
[root@web1 ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
```

```
ONBOOT="yes"
IPADDR=192.168.0.1
PREFIX=24
GATEWAY=192.168.0.254
TYPE="Ethernet"
```

```
[root@web1 ~]# service network restart
[root@web1 ~]# yum --y install httpd
[root@web1 ~]# echo "192.168.0.1" > /var/www/html/index.html
[root@web1 ~]# service iptables stop
```

Nginx 代理服务器设置：

```
[root@nginx ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=10.10.10.10
PREFIX=8
TYPE="Ethernet"
```

```
[root@nginx ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth1
```

```
DEVICE="eth1"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.254
PREFIX=24
TYPE="Ethernet"
```

```
[root@nginx ~]# service network restart
[root@nginx ~]# service iptables stop
[root@nginx ~]# wget http://nginx.org/download/nginx-1.4.0.tar.gz
[root@nginx ~]# tar -xzf nginx-1.4.0.tar.gz -C /usr/src/
[root@nginx ~]# yum --y install gcc pcre pcre-devel gcc openssl \
>openssl-devel gd gd-devel perl perl-ExtUtils-Embed
[root@nginx ~]# cd /usr/src/nginx-1.4.0/
[root@nginx nginx-1.4.0]# ./configure -prefix=/usr/local/nginx \
>--with-ipv6 \
>--with-http_ssl_module \
>--with-http_realip_module \
>--with-http_addition_module \
>--with-http_dav_module \
>--with-http_gzip_static_module \
>--with-http_perl_module \
>--with-mail_ssl_module
[root@nginx nginx-1.4.0]# make && make install
```

```
[root@nginx ~]# vim /usr/local/nginx/conf/nginx.conf
```

```
user  nobody;
worker_processes  1;
error_log  logs/error.log  notice;
pid        logs/nginx.pid;

events {
worker_connections  5024;
}

http {
include       mime.types;
default_type  application/octet-stream;
log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
                  '$status $body_bytes_sent "$http_referer" '
                  '"$http_user_agent" "$http_x_forwarded_for" ';
sendfile        on;
tcp_nopush     on;
server_tokens off;
keepalive_timeout  65;
keepalive_requests 100;
#启动网页压缩功能
gzip on;
#容量小于1000B的内容不压缩
gzip_min_length 1000;
#压缩缓存的个数与容量
gzip_buffers 16 32k;
#指定压缩文件类型
gzip_types text/plain application/xml;
#压缩级别，可以是1至9之间的数字，数字越大，压缩效果越好，速度也越慢
gzip_comp_level 2;
#设置客户端请求的缓存大小
client_body_buffer_size 128K;
#设置客户端请求所允许的最大文件容量
client_max_body_size 100m;
#客户端请求的Header缓存大小
large_client_header_buffers 4 8K;
#启用代理缓冲功能
proxy_buffering on;
#从被代理服务器（真实服务器）所获取的第一部分响应数据的缓存大小
proxy_buffer_size 8k;
#从被代理服务器读取响应数据的缓存个数与容量
proxy_buffers 8 128K;
#设置缓存目录及其他缓存参数，缓存路径为/usr/local/nginx/cache，levels用来设置子
```

```
    #目录个数，keys_zone 定义缓冲名称及容量（名称为 one，容量为 100MB），inactive 定义
    #缓存的存活时间为 1 天，max_size 设置硬盘缓存容量为 2GB。这里定义的缓存为 one，将
    #在下面使用 proxy_cache 指令调用该缓存
    proxy_cache_path /usr/local/nginx/cache levels=1:2 keys_zone=one:100m inactive=
1d max_size=2G;
    #与后端服务器建立 TCP 握手的超时时间
    proxy_connect_timeout 60s;
    #设置后端服务器组，名称为 servers
    upstream servers {
    #ip_hash 使用 IP 哈希值确保相同 IP 的客户端使用相同的后端服务器，不使用该参数调
    #度器将采用轮询的方式进行数据转发
    #ip_hash;
    #max_fails 设置连接后端服务器 3 次失败则认为服务器处于无效状态，fail_timeout 设置
    #连接后端服务器超时时间为 30s，weight 设置权重为 2
    server 192.168.0.1:80 max_fails=3 fail_timeout=30s weight=2;
    server 192.168.0.2:80 max_fails=3 fail_timeout=30s weight=2;
    }
    server {
          listen       80;
          server_name  www.example.com;
          access_log  logs/host.access.log  main;
          location / {
    proxy_pass http://servers;
    proxy_cache one;
    proxy_set_header X-Forwarded-For $remote_addr;
          }
    }
    }
```

```
[root@nginx ~]# echo "/usr/local/nginx/sbin/nginx" >> /etc/rc.local
```

配置客户端主机 IP 地址为 10.10.10.100，通过浏览器访问 http://10.10.10.10，刷新将分别得到 web1.example.com 以及 web2.example.com 两台主机返回的页面信息。

8.5.3　Nginx rewrite 规则

由于互联网资源分布在世界各地，因此，如果用户希望查看某个资源，就需要使用 URL 来唯一定位该资源（如：http://google.com）。而 URL 重写则可以在 Web 服务器上面订制规则将用户访问资源的 URL 进行重新改写，改写后再交给 Web 服务器进程来处理。实际的生产环境经

常使用该功能将用户拼写错误的 URL 或不存在的 URL 重定向到特定的固定页面，如网站首页或 404 资源找不到的错误报错页面；还可以利用地址重写优化用户的输入及使用体验，如将 http://www.google.com/test.html 重写为 http://www.google.com/test. jsp?id=123456。这样用户仅需要输入简单的 URL 就可以定位到服务器后台包含复杂路径及参数的资源。Nginx 的 HttpRewriteModule 模块支持基于 PCRE（Perl Compatible Regular Expressions）的地址重写功能。

Nginx 的 rewrite 语法格式与 Apache 非常类似：rewrite regex replacement [flag]，其中，flag 可以被设置为 last、break、redirect、permanent。last 标记将结束当前 rewrite 指令并重新搜索 location 匹配；break 标记也将结束当前 rewrite 指令，并且不再处理 location 块的其他匹配；redirect 标记将返回临时重定向代码 302；permanent 将返回永久重定向代码 301。

在进行地址重定向的时候，我们一般会结合 location 或 if 语句进行，if 语句的语法格式为：

```
if （条件）{ ...}
```

其中的 if 语句可以放置在配置文件中的 server 块或 location 块，可以使用表 8-10 中的测试操作符。

表 8-10

操作符	描述
~	区分大小写匹配
~*	不区分大小写匹配
=	精确匹配
-f	测试文件是否存在，存在且为文件则结果为真
-d	测试目录是否存在，存在则结果为真
-e	测试文件或目录是否存在，存在则结果为真
-x	测试文件是否可执行
!	区分，可以与以上操作符相结合

rewrite 地址重写及 return 应用的语法说明如下：

```
#测试客户端浏览器是否使用的是 Microsoft IE 浏览器，如果是，则用户访问的任何资源都将被重
#定向到/msie/目录下的相应资源，$http_user_agent 与$1 是 Nginx 内置的变量，
#http_user_agent 会返回客户端所使用的浏览器标识，$1 为位置变量，即将用户原本需要
#访问的资源重定向到/msie/目录下的相应资源
if ($http_user_agent ~ MSIE) {
rewrite ^(.*)$ /msie/$1 break;
}
#如果用户使用的是 iPhone 或 iPod 访问服务器，则将请求重定向至其他服务器
#这里使用了一个 URI 概念，我们知道，URL 是统一资源定位符，而 URL 是通用资源标识符，
```

```
#如果一个URL为http://www.example.com/test/index.html，相对路径URI为/test/index.html
if ($http_user_agent ~* '(iPhone|iPod)' {
rewrite ^.+ http://mobile.site.com$uri;
}
#用户使用 POST 方式请求数据时，则返回 405 错误
if ($request_method = POST) {
return 405;
}
rewrite ^(/download/.*)/media/(.*)\..*$ $1/mp3/$2.mp3 last;
rewrite ^(/download/.*)/audio/(.*)\..*$ $1/mp3/$2.ra last;
```

下面是一个完整的地址重写应用案例，根据用户使用的浏览器不同进行不同的地址重定向。此外，设置当用户访问 wp_administrator 目录下的资源时需要进行重定向 admin 目录。

```
[root@nginx ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

```
DEVICE="eth0"
BOOTPROTO="static"
ONBOOT="yes"
IPADDR=192.168.0.254
PREFIX=24
TYPE="Ethernet"
```

```
[root@nginx ~]# service network restart
[root@nginx ~]# service iptables stop
[root@nginx ~]# vim /usr/local/nginx/conf/nginx.conf
```

```
user  nobody;
worker_processes  1;
error_log  logs/error.log  notice;
pid        logs/nginx.pid;

events {
    worker_connections  5024;
}

http {
    include       mime.types;
    default_type  application/octet-stream;
    log_format  main '$remote_addr - $remote_user [$time_local] "$request" '
                     '$status $body_bytes_sent "$http_referer" '
                     '"$http_user_agent" "$http_x_forwarded_for"';
    sendfile        on;
    tcp_nopush     on;
```

```
        server_tokens off;
        keepalive_timeout  65;
        keepalive_requests 100;
        gzip  on;
        gzip_min_length 1000;
        gzip_buffers 16 32k;
        gzip_types text/plain application/xml;
        gzip_comp_level 2;
        error_page  404              /404.html;
        error_page   500 502 503 504  /50x.html;

        client_body_buffer_size 128K;
        client_max_body_size 100m;
        large_client_header_buffers 4 8K;

        server {
            listen 80;
         root html;
         index index.html;
            server_name  www.example.com;
            access_log  logs/host.access.log  main;
            if ($http_user_agent ~ MSIE) {
         rewrite ^(.*)$ /msie/$1 break;
         }
    if ($http_user_agent ~ Firefox) {
         rewrite ^(.*)$ /firefox/$1 break;
         }
         location /php_admin {
         rewrite ^/php_admin/.*$ /admin permanent;
         }
        }
    }
```

```
[root@nginx ~]# mkdir /usr/local/nginx/html/{msie,firefox,admin}
[root@nginx ~]# echo "firefox browser" > /usr/local/nginx/html/firefox/index.html
[root@nginx ~]# echo "IE browser" > /usr/local/nginx/html/msie/index.html
[root@nginx ~]# echo "Admin Page" > /usr/local/nginx/html/admin/index.html
[root@nginx ~]# /usr/local/nginx/sbin/nginx
```

客户端使用火狐浏览器访问 http://192.168.0.254/index.html 将获得页面内容 firefox browser，使用 IE 浏览器访问 http://192.168.0.254/index.html 将获得页面内容 IE browser，访问 http://192.168.0.254/php_admin 将获得页面内容 Admin Page。

8.6　MySQL 高可用

8.6.1　MySQL 复制

使用 MySQL 复制功能可以将主服务器上的数据复制到多台从服务器上。默认情况下，复制是异步传输方式，从服务器不需要总是连接主服务器去更新数据。也就是说，数据更新可以在远距离连接的情况下进行，甚至在使用拨号网络的临时连接环境下也可以进行。根据自定义设置，我们可以对所有的数据库或部分数据库甚至是部分数据表进行复制。通过主从复制在企业级应用环境中就不必再担心数据库的单点故障，当一台服务器宕机时，其他服务器一样可以提供非常稳定、可靠的数据服务。

MySQL 复制的优势如下。

- 高性能：通过将请求分配给多台不同的从服务器提高性能与速度。在这种环境中，所有对数据库的写操作必须提供给主服务器，但读操作可以被平均分配给多台从服务器。
- 数据安全：数据是从主服务器复制到从服务器的，而且从服务器可以随时暂停复制，这样我们就可实现数据备份与还原。
- 远程数据分享：如果企业拥有多处位于其他地理位置的分公司，而这些分公司希望共享总部的数据库资源，就可以使用复制实现数据的共享，而且分公司不必时时连接主服务器，可以仅在需要时进行复制。

MySQL 复制有两种核心的格式，一种是基于 SQL 语句的复制 SBR（Statement Based Replication），另一种是基于行的复制 RBR（Row Based Replication），另外，有时也可以使用混合模式复制。MySQL 5.6 默认采用的是基于语句的复制。

MySQL 服务器之间的复制是基于二进制日志机制的。在主服务器上，当 MySQL 实例进行写操作时会同时生成一条操作事件日志并写入二进制日志文件中，而从服务器则负责读取主服务器上的二进制日志，并在从服务器本机上重新执行该事件，从而实现复制数据至本地服务器。当主服务器开启二进制日志功能后，所有的 SQL 语句都将被记录至日志。从服务器复制这些二进制日志条目，并根据自己的需要决定哪些语句需要重新在从服务器上执行，而哪些语句不需要再执行，我们无法控制主服务器仅记录特定语句到日志文件中。如果进行其他设置，所有主服务器中的日志事件都将在从服务器上重新执行，当然，你可以配置从服务器仅执行主服务器中的部分日志事件。因为从服务器会记录二进制日志的进度与位置（比如执行到了第几条语句），所以从服务器可以断开与主服务器之间的连接，并在重新建立连接后继续进行复制工作。

在具体的操作过程中，主服务器与从服务器都需要配置一个唯一的 ID 编号。另外，所有的从服务器必须设置主服务器的主机名、日志文件名、文件位置等参数。

8.6.2 一步一步操作 MySQL 复制

本节将演示如何一步一步地实现 MySQL 服务器之间的数据复制（演示步骤中的命令提示符为 master，则代表指令在 MySQL 主服务器上运行，提示符为 slave1 则代表指令在 MySQL 从服务器上运行）。整个过程一般分为以下几个步骤进行。

- 在主服务器上开启二进制日志功能，设置唯一的服务器 ID 编号，这些设置需要重启 MySQL 服务。
- 在所有的从服务器上设置唯一的服务器 ID 编号，这些设置需要重启 MySQL 服务。
- 在主服务器上为不用的从服务器创建可以读取主服务器日志文件的账户，或使用相同的统一账户。
- 在进行数据复制之前，你还需要记录主服务器上二进制日志的位置标记。

1. 数据复制环境中主服务器的设置

在实际的生产环境中，可能在我们还没有部署数据复制前，数据库中就已经存在大量的数据。所以，这里事先创建一个测试用数据库及数据表，用来演示如何对已经存在的数据进行数据同步备份。

```
[root@master ~]# mysql -u root -p
mysql> create database hr;
mysql> use hr;
mysql> create table employees(
-> employee_id INT NOT NULL AUTO_INCREMENT,
-> name char(20) NOT NULL,
-> e_mail varchar(50),
-> PRIMARY KEY(employee_id));
mysql> INSERT INTO employees values
-> (1,'TOM','tom@example.com'),
-> (2,'Jerry','jerry@example.com');
mysql> exit
```

我们需要在主服务器上开启二进制日志并设置服务器编号，服务器唯一编号必须是 1 至 2^{32}-1 之间的整数，根据自己的实际情况进行设置。进行这些设置需要关闭 MySQL 数据库并编

辑 my.cnf 或 my.ini 文件，然后在[mysqld]设置段添加相应的配置选项。关于 MySQL 软件的安装，这里不再赘述，请参考 4.11.2 节的内容。

```
[root@master ~]# vim /etc/my.cnf
```

```
[mysqld]
log-bin=Jacob-bin                    #启用二进制日志，并设置二进制日志文件前缀
server-id=254                        #设置服务器编号
```

```
[root@master ~]# service mysqld restart
[root@master ~]# service iptables stop
```

提示 在配置文件中不可以使用 skip-networking 参数选项，否则从服务器将无法与主服务器进行连接并复制数据。

2. 数据复制环境中从服务器的设置

如果从服务器 ID 编号没有设置，或服务器 ID 编号与主服务器有冲突，就必须关闭 MySQL 服务，并重新编辑配置文件，设置唯一的服务器编号，最后重启 MySQL 服务。如果有多台从服务器，则所有的服务器 ID 编号都必须是唯一的。可以考虑将服务器 ID 编号与服务器 IP 地址关联，这样 ID 编号可以同时唯一标识一台服务器计算机，如采用 IP 地址的最后一位作为 MySQL 服务器 ID 编号。

```
[root@slave1 ~]# vim /etc/my.cnf
```

```
[mysqld]
server-id=2
```

```
[root@slave1 ~]# service mysqld restart
[root@slave1 ~]# service iptables stop
```

对复制而言，MySQL 从服务器上二进制日志功能是不需要开启的。但是，你也可以通过启用从服务器的二进制日志功能，实现数据备份与恢复。此外，在一些更复杂的拓扑环境中，MySQL 从服务器也可以扮演其他从服务器的主服务器。

3. 创建复制账号

执行数据复制时，所有的从服务器都需要使用账户与密码连接 MySQL 主服务器，所以在主服务器上必须存在至少一个用户账户及相应的密码供从服务器连接。这个账户必须拥有 REPLICATION SLAVE 权限，你可以为不同的从服务器创建不同的账户与密码，也可以使用统一的账户与密码。MySQL 可以使用 CREATE USER 语句创建用户，使用 GRANT 语句为账户赋

权。如果该用户仅为数据库复制所使用，则该账户仅需要 REPLICATION SLAVE 权限即可。下面的例子将在 MySQL 主服务器上创建一个拥有复制权限的 slave_cp 账户，该账户可以从 example.com 域内的任何主机连接主服务器，密码为 SlaveAdmin。

```
[root@master ~]# mysql -u root -p
mysql> CREATE USER 'slave_cp'@'%.example.com' IDENTIFIED BY 'SlaveAdmin';
mysql> GRANT REPLICATION SLAVE ON *.* TO 'slave_cp'@'%.example.com';
mysql> exit
```

4. 获取主服务器二进制日志信息

在进行主从数据复制之前，我们来了解一些主服务器的二进制日志文件的基本信息，这些信息在对从服务器的设置中需要用到，它包括主服务器二进制文件名称及当前日志记录位置，这样从服务器就可以知道从哪里开始进行复制操作。我们可以使用如下操作查看主服务器二进制日志数据信息。

```
[root@master ~]# mysql -u root -p
mysql> FLUSH TABLES WITH READ LOCK;
mysql> SHOW MASTER STATUS;
+-------+---------+-------------+-----------------+-------------------+
| File  | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_Set |
+-------+---------+-------------+-----------------+-------------------+
| jacob-log.000001 |1276    |     |                 |                   |
+-------+---------+-------------+-----------------+-------------------+
mysql> UNLOCK TABLES;
```

其中，File 列显示的是二进制日志文件名，Position 为当前日志记录位置。

FLUSH TABLES WITH READ LOCK 命令的作用是对所有数据库的表执行只读锁定，只读锁定后所有数据库的写操作将被拒绝，但读操作可以继续。执行锁定可以防止在查看二进制日志信息的同时有人对数据进行修改操作，最后使用 UNLOCK TABLES 语句对全局锁执行结束操作。

5. 对现有数据库进行快照备份

如果在使用二进制日志进行数据复制以前，MySQL 数据库系统中已经存在大量的数据资源，对这些资料进行数据备份的一种方法是使用 mysqldump 工具，在主服务器上使用该工具对数据备份后，即可在从服务器上进行数据还原操作。当希望的数据达到主从一致后，就可以使用数据复制功能进行自动同步操作。具体操作如下（作者的环境中主服务器 IP 地址为 172.16.0.254，从服务器 IP 地址为 172.16.0.1，实际的生成环境中需要根据自己的需要有选择地对数据库进行备份与还原）：

```
[root@master ~]# mysqldump --all-databases --lock-all-tables > /tmp/dbdump.sql
[root@master ~]# scp /tmp/dbdump.sql 172.16.0.1:/tmp/
[root@slave1 ~]# mysql  -u root -p < dbdump.sql
```

6. 配置从服务器连接主服务器进行数据复制

数据复制的关键操作是配置从服务器去连接主服务器进行数据复制，我们需要告知从服务器建立网络连接所有必要的信息。使用 CHANGE MASTER TO 语句即可完成该项工作，MASTER_HOST 指定主服务器主机名或 IP 地址，MASTER_USER 为主服务器上创建的拥有复制权限的账户名称，MASTER_PASSWORD 为该账户的密码，MASTER_LOG_FILE 指定主服务器二进制日志文件名称，MASTER_LOG_POS 为主服务器二进制日志当前记录的位置。START SLAVE 开启从服务器功能进行主从连接，**SHOW SLAVE STATUS** 查看从服务器状态。

```
[root@slave1 ~]# mysql -u root -p
mysql> CHANGE MASTER TO
    -> MASTER_HOST='172.16.0.155',
    -> MASTER_USER='data_cp',
    -> MASTER_PASSWORD='SlaveAdmin',
    -> MASTER_LOG_FILE='jacob-log.000001',
-> MASTER_LOG_POS=1351;
mysql> START SLAVE;
mysql> SHOW SLAVE STATUS\G;
```

7. 数据同步验证

所有的主从服务器均设置完毕后，我们可以通过在主服务器上创建新的数据资料，然后在从服务器上查看，所有的数据将自动同步。

```
[root@master ~]# mysql -u root -p
mysql> create database test2;
mysql> use test2;
mysql> create table t_table(
->name char(20),
-> age int,
-> note varchar(50));
mysql> INSERT INTO t_table values
-> ('linda',23, 'Beijing'),
-> ('jerry',33, 'shanghai'),;
mysql> exit
[root@slave1 ~]# mysql -u root -p
mysql> select * from test2.t_table;
mysql> exit
```

電子工業出版社 PUBLISHING HOUSE OF ELECTRONICS INDUSTRY http://www.phei.com.cn Broadview® WWW.BROADVIEW.COM.CN

博文视点·IT出版旗舰品牌

十载耕耘奠定专业地位

以书为证彰显卓越品质

博文视点诚邀精锐作者加盟

《C++Primer（中文版）（第5版）》、《淘宝技术这十年》、《代码大全》、《Windows内核情景分析》、《加密与解密》、《编程之美》、《VC++深入详解》、《SEO实战密码》、《PPT演义》……

“圣经”级图书光耀夺目，被无数读者朋友奉为案头手册传世经典。

潘爱民、毛德操、张亚勤、张宏江、昝辉Zac、李刚、曹江华……

“明星”级作者济济一堂，他们的名字熠熠生辉，与IT业的蓬勃发展紧密相连。

十年的开拓、探索和励精图治，成就**博**古通今、**文**圆质方、**视**角独特、**点**石成金之计算机图书的风向标杆：博文视点。

“凤翱翔于千仞兮，非梧不栖”，博文视点欢迎更多才华横溢、锐意创新的作者朋友加盟，与大师并列于IT专业出版之巅。

英雄帖

江湖风云起，代有才人出。
IT界群雄并起，逐鹿中原。
博文视点诚邀天下技术英豪加入，
指点江山，激扬文字
传播信息技术，分享IT心得

• 专业的作者服务 •

博文视点自成立以来一直专注于IT专业技术图书的出版，拥有丰富的与技术图书作者合作的经验，并参照IT技术图书的特点，打造了一支高效运转、富有服务意识的编辑出版团队。我们始终坚持：

善待作者——我们会把出版流程整理得清晰简明，为作者提供优厚的稿酬服务，解除作者的顾虑，安心写作，展现出最好的作品。

尊重作者——我们尊重每一位作者的技术实力和生活习惯，并会参照作者实际的工作、生活节奏，量身制定写作计划，确保合作顺利进行。

提升作者——我们打造精品图书，更要打造知名作者。博文视点致力于通过图书提升作者的个人品牌和技术影响力，为作者的事业开拓带来更多的机会。

联系我们

博文视点官网：http://www.broadview.com.cn　　CSDN官方博客：http://blog.csdn.net/broadview2006/

投稿电话：010-51260888　88254368　　投稿邮箱：jsj@phei.com.cn

新浪微博 weibo.com　@博文视点Broadview　　微信公众账号　博文视点Broadview